

VoyagerOS64

Generated by Doxygen 1.9.4

1 VoyagerOS64 Kernel Documentation	1
1.1 Current 0.0.5 architecture	1
1.2 Scheduler model	1
1.3 Deliberate 0.0.5 boundaries	1
1.4 Historical implementations	1
1.5 Runtime qualification	2
2 Module Index	3
2.1 Modules	3
3 Data Structure Index	5
3.1 Data Structures	5
4 File Index	9
4.1 File List	9
5 Module Documentation	13
5.1 Boot and Kernel Bring-up	13
5.1.1 Detailed Description	13
5.2 Memory Management	14
5.2.1 Detailed Description	14
5.3 Early Physical Allocator	14
5.3.1 Detailed Description	15
5.4 Physical Memory Manager	15
5.4.1 Detailed Description	15
5.5 Kernel Heap	15
5.5.1 Detailed Description	16
5.6 Paging and Address Translation	16
5.6.1 Detailed Description	16
5.7 Interrupts and Descriptor Tables	16
5.7.1 Detailed Description	17
5.8 Scheduler and Processes	17
5.8.1 Detailed Description	17
5.9 Device Drivers	17
5.9.1 Detailed Description	18
5.10 PS/2 Keyboard	18
5.10.1 Detailed Description	18
5.11 Terminal and Console	18
5.11.1 Detailed Description	18
5.12 Voyager Shell (VSH)	19
5.12.1 Detailed Description	19
5.13 Diagnostics and Qualification	19
5.13.1 Detailed Description	19
5.14 Timekeeping	19

5.14.1 Detailed Description	20
5.15 Retired / Historical Code	20
6 Data Structure Documentation	21
6.1 __attribute__ Struct Reference	21
6.1.1 Detailed Description	21
6.1.2 Field Documentation	21
6.1.2.1 attributes	22
6.1.2.2 base	22
6.1.2.3 base_high	22
6.1.2.4 base_low	22
6.1.2.5 base_mid	22
6.1.2.6 cs	22
6.1.2.7 ist	23
6.1.2.8 limit	23
6.1.2.9 rsv0	23
6.2 block_footer Struct Reference	23
6.2.1 Detailed Description	23
6.2.2 Field Documentation	23
6.2.2.1 size_and_flags	24
6.3 block_header Struct Reference	24
6.3.1 Detailed Description	24
6.3.2 Field Documentation	24
6.3.2.1 next_free	24
6.3.2.2 prev_free	25
6.3.2.3 size_and_flags	25
6.4 cpu_ctx Struct Reference	25
6.4.1 Detailed Description	26
6.4.2 Field Documentation	26
6.4.2.1 cs	26
6.4.2.2 ds	26
6.4.2.3 err	26
6.4.2.4 es	26
6.4.2.5 r10	26
6.4.2.6 r11	27
6.4.2.7 r12	27
6.4.2.8 r13	27
6.4.2.9 r14	27
6.4.2.10 r15	27
6.4.2.11 r8	27
6.4.2.12 r9	28
6.4.2.13 rax	28

6.7.2.12 font_bits_size	37
6.7.2.13 font_bool	37
6.7.2.14 font_bool_size	37
6.7.2.15 font_height	37
6.7.2.16 font_scale_x	37
6.7.2.17 font_scale_y	37
6.7.2.18 font_width	38
6.7.2.19 framebuffer	38
6.7.2.20 glyph_height	38
6.7.2.21 glyph_width	38
6.7.2.22 grid	38
6.7.2.23 grid_size	38
6.7.2.24 height	39
6.7.2.25 map	39
6.7.2.26 map_size	39
6.7.2.27 offset_x	39
6.7.2.28 offset_y	39
6.7.2.29 old_cursor_x	39
6.7.2.30 old_cursor_y	40
6.7.2.31 pitch	40
6.7.2.32 queue	40
6.7.2.33 queue_i	40
6.7.2.34 queue_size	40
6.7.2.35 saved_state_cursor_x	40
6.7.2.36 saved_state_cursor_y	41
6.7.2.37 saved_state_text_bg	41
6.7.2.38 saved_state_text_fg	41
6.7.2.39 term	41
6.7.2.40 text_bg	41
6.7.2.41 text_fg	41
6.7.2.42 width	42
6.8 fbterm_queue_item Struct Reference	42
6.8.1 Detailed Description	42
6.8.2 Field Documentation	42
6.8.2.1 c	43
6.8.2.2 x	43
6.8.2.3 y	43
6.9 GDT Struct Reference	43
6.9.1 Detailed Description	44
6.9.2 Field Documentation	44
6.9.2.1 kernelCS	44
6.9.2.2 kernelData	44

6.14.1 Detailed Description	53
6.14.2 Field Documentation	53
6.14.2.1	53
6.14.2.2	53
6.14.2.3 cr0	53
6.14.2.4 cr2	53
6.14.2.5 cr3	54
6.14.2.6 cr4	54
6.14.2.7 cs	54
6.14.2.8 dss	54
6.14.2.9 error_code	54
6.14.2.10	54
6.14.2.11 rax	55
6.14.2.12 rbp	55
6.14.2.13 rbx	55
6.14.2.14 rcx	55
6.14.2.15 rdi	55
6.14.2.16 rdx	55
6.14.2.17 rflags	56
6.14.2.18 rip	56
6.14.2.19 rsi	56
6.14.2.20 rsp	56
6.14.2.21 vector	56
6.15 kswitches Struct Reference	56
6.15.1 Detailed Description	57
6.15.2 Field Documentation	57
6.15.2.1 acpi_support	57
6.15.2.2 addr_debug	57
6.15.2.3 fpu_allowed	57
6.15.2.4 hw_rng_support	58
6.15.2.5 liballoc_debug	58
6.15.2.6 mem_readout_unit	58
6.15.2.7 sched_debug	58
6.15.2.8 stack_trace_on_fault	58
6.15.2.9 stack_trace_size	58
6.15.2.10 timestamp	59
6.16 limine_5_level_paging_request Struct Reference	59
6.16.1 Detailed Description	59
6.16.2 Member Function Documentation	59
6.16.2.1 LIMINE_PTR()	59
6.16.3 Field Documentation	59
6.16.3.1 id	60

6.23.3 Field Documentation	66
6.23.3.1 revision	66
6.24 limine_efi_system_table_request Struct Reference	66
6.24.1 Detailed Description	67
6.24.2 Member Function Documentation	67
6.24.2.1 LIMINE_PTR()	67
6.24.3 Field Documentation	67
6.24.3.1 id	67
6.24.3.2 revision	67
6.25 limine_efi_system_table_response Struct Reference	68
6.25.1 Detailed Description	68
6.25.2 Member Function Documentation	68
6.25.2.1 LIMINE_PTR()	68
6.25.3 Field Documentation	68
6.25.3.1 revision	68
6.26 limine_entry_point_request Struct Reference	68
6.26.1 Detailed Description	69
6.26.2 Member Function Documentation	69
6.26.2.1 LIMINE_PTR() [1/2]	69
6.26.2.2 LIMINE_PTR() [2/2]	69
6.26.3 Field Documentation	69
6.26.3.1 id	69
6.26.3.2 revision	70
6.27 limine_entry_point_response Struct Reference	70
6.27.1 Detailed Description	70
6.27.2 Field Documentation	70
6.27.2.1 revision	70
6.28 limine_file Struct Reference	71
6.28.1 Detailed Description	71
6.28.2 Member Function Documentation	71
6.28.2.1 LIMINE_PTR() [1/3]	72
6.28.2.2 LIMINE_PTR() [2/3]	72
6.28.2.3 LIMINE_PTR() [3/3]	72
6.28.3 Field Documentation	72
6.28.3.1 gpt_disk_uuid	72
6.28.3.2 gpt_part_uuid	72
6.28.3.3 mbr_disk_id	72
6.28.3.4 media_type	73
6.28.3.5 part_uuid	73
6.28.3.6 partition_index	73
6.28.3.7 revision	73
6.28.3.8 size	73

6.32.3.2 revision	80
6.33 limine_hhdm_response Struct Reference	81
6.33.1 Detailed Description	81
6.33.2 Field Documentation	81
6.33.2.1 offset	81
6.33.2.2 revision	81
6.34 limine_kernel_address_request Struct Reference	81
6.34.1 Detailed Description	82
6.34.2 Member Function Documentation	82
6.34.2.1 LIMINE_PTR()	82
6.34.3 Field Documentation	82
6.34.3.1 id	82
6.34.3.2 revision	82
6.35 limine_kernel_address_response Struct Reference	82
6.35.1 Detailed Description	83
6.35.2 Field Documentation	83
6.35.2.1 physical_base	83
6.35.2.2 revision	83
6.35.2.3 virtual_base	83
6.36 limine_kernel_file_request Struct Reference	83
6.36.1 Detailed Description	84
6.36.2 Member Function Documentation	84
6.36.2.1 LIMINE_PTR()	84
6.36.3 Field Documentation	84
6.36.3.1 id	84
6.36.3.2 revision	84
6.37 limine_kernel_file_response Struct Reference	85
6.37.1 Detailed Description	85
6.37.2 Member Function Documentation	85
6.37.2.1 LIMINE_PTR()	85
6.37.3 Field Documentation	85
6.37.3.1 revision	85
6.38 limine_memmap_entry Struct Reference	85
6.38.1 Detailed Description	86
6.38.2 Field Documentation	86
6.38.2.1 base	86
6.38.2.2 length	86
6.38.2.3 type	86
6.39 limine_memmap_request Struct Reference	86
6.39.1 Detailed Description	87
6.39.2 Member Function Documentation	87
6.39.2.1 LIMINE_PTR()	87

6.45.3.1 id	93
6.45.3.2 revision	93
6.46 limine_smbios_response Struct Reference	94
6.46.1 Detailed Description	94
6.46.2 Member Function Documentation	94
6.46.2.1 LIMINE_PTR() [1/2]	94
6.46.2.2 LIMINE_PTR() [2/2]	94
6.46.3 Field Documentation	94
6.46.3.1 revision	94
6.47 limine_smp_request Struct Reference	95
6.47.1 Detailed Description	95
6.47.2 Member Function Documentation	95
6.47.2.1 LIMINE_PTR()	95
6.47.3 Field Documentation	95
6.47.3.1 flags	95
6.47.3.2 id	95
6.47.3.3 revision	96
6.48 limine_stack_size_request Struct Reference	96
6.48.1 Detailed Description	96
6.48.2 Member Function Documentation	96
6.48.2.1 LIMINE_PTR()	96
6.48.3 Field Documentation	96
6.48.3.1 id	97
6.48.3.2 revision	97
6.48.3.3 stack_size	97
6.49 limine_stack_size_response Struct Reference	97
6.49.1 Detailed Description	97
6.49.2 Field Documentation	97
6.49.2.1 revision	98
6.50 limine_terminal Struct Reference	98
6.50.1 Detailed Description	98
6.50.2 Member Function Documentation	98
6.50.2.1 LIMINE_PTR()	98
6.50.3 Field Documentation	98
6.50.3.1 columns	99
6.50.3.2 rows	99
6.51 limine_terminal_request Struct Reference	99
6.51.1 Detailed Description	99
6.51.2 Member Function Documentation	99
6.51.2.1 LIMINE_PTR() [1/2]	99
6.51.2.2 LIMINE_PTR() [2/2]	100
6.51.3 Field Documentation	100

6.58 madt_nmi Struct Reference	107
6.58.1 Detailed Description	107
6.58.2 Field Documentation	107
6.58.2.1 flags	107
6.58.2.2 lint	107
6.58.2.3 processor	108
6.59 memtest_slot Struct Reference	108
6.59.1 Detailed Description	108
6.59.2 Field Documentation	108
6.59.2.1 ptr	108
6.59.2.2 size	108
6.59.2.3 tag	109
6.60 memtest_stats Struct Reference	109
6.60.1 Detailed Description	109
6.60.2 Field Documentation	109
6.60.2.1 allocations	109
6.60.2.2 frees	109
6.60.2.3 live_bytes	110
6.60.2.4 operations	110
6.60.2.5 peak_live_bytes	110
6.60.2.6 reallocations	110
6.61 out_fct_wrap_type Struct Reference	110
6.61.1 Detailed Description	110
6.61.2 Field Documentation	111
6.61.2.1 arg	111
6.61.2.2 fct	111
6.62 PageTableOffset Struct Reference	111
6.62.1 Detailed Description	111
6.62.2 Field Documentation	111
6.62.2.1 p4Offset	112
6.62.2.2 pdOffset	112
6.62.2.3 pdpOffset	112
6.62.2.4 ptOffset	112
6.63 process Struct Reference	112
6.63.1 Detailed Description	113
6.63.2 Field Documentation	113
6.63.2.1 next	113
6.63.2.2 pid	113
6.63.2.3 rsp	114
6.63.2.4 state	114
6.64 rsdp Struct Reference	114
6.64.1 Detailed Description	114

6.70.1 Detailed Description	123
6.70.2 Field Documentation	123
6.70.2.1 autoflush	123
6.70.2.2 bold	123
6.70.2.3 callback	123
6.70.2.4 charsets	124
6.70.2.5 clear	124
6.70.2.6 cols	124
6.70.2.7 control_sequence	124
6.70.2.8 csi	124
6.70.2.9 current_charset	124
6.70.2.10 current_primary	125
6.70.2.11 dec_private	125
6.70.2.12 deinit	125
6.70.2.13 disable_cursor	125
6.70.2.14 discard_next	125
6.70.2.15 double_buffer_flush	125
6.70.2.16 enable_cursor	126
6.70.2.17 esc_values	126
6.70.2.18 esc_values_i	126
6.70.2.19 escape	126
6.70.2.20 escape_offset	126
6.70.2.21 full_refresh	126
6.70.2.22 g_select	127
6.70.2.23 get_cursor_pos	127
6.70.2.24 in_bootloader	127
6.70.2.25 insert_mode	127
6.70.2.26 move_character	127
6.70.2.27 raw_putchar	127
6.70.2.28 restore_state	128
6.70.2.29 reverse_video	128
6.70.2.30 revscroll	128
6.70.2.31 rows	128
6.70.2.32 rrr	128
6.70.2.33 save_state	128
6.70.2.34 saved_cursor_x	129
6.70.2.35 saved_cursor_y	129
6.70.2.36 saved_state_bold	129
6.70.2.37 saved_state_current_charset	129
6.70.2.38 saved_state_current_primary	129
6.70.2.39 saved_state_reverse_video	129
6.70.2.40 scroll	130

6.73.2.15 rsp2	138
6.74 TSS_Entry Struct Reference	138
6.74.1 Detailed Description	138
6.74.2 Field Documentation	138
6.74.2.1 base_high	138
6.74.2.2 base_low	139
6.74.2.3 base_mid	139
6.74.2.4 base_up	139
6.74.2.5 flags	139
6.74.2.6 flags2	139
6.74.2.7 length	139
6.74.2.8 reserved0	139
7 File Documentation	141
7.1 src/acpi/acpi.c File Reference	141
7.1.1 Function Documentation	142
7.1.1.1 acpi_find_sdt()	142
7.1.1.2 acpi_init()	143
7.1.1.3 halt()	144
7.1.1.4 use_xsdt()	144
7.1.2 Variable Documentation	144
7.1.2.1 rsdp	144
7.1.2.2 rsdp_req	144
7.1.2.3 rsdt	145
7.1.2.4 sdt	145
7.2 acpi.c	145
7.3 src/acpi/madt.c File Reference	147
7.3.1 Function Documentation	147
7.3.1.1 madt_init()	147
7.3.1.2 typeof()	148
7.4 madt.c	148
7.5 src/apic/ioapic.c File Reference	149
7.5.1 Function Documentation	150
7.5.1.1 halt()	150
7.5.1.2 io_apic_from_gsi()	150
7.5.1.3 io_apic_gsi_count()	151
7.5.1.4 io_apic_read()	151
7.5.1.5 io_apic_set_gsi_redirect()	152
7.5.1.6 io_apic_set_irq_redirect()	153
7.5.1.7 io_apic_write()	153
7.6 ioapic.c	154
7.7 src/apic/lapic.c File Reference	155

7.11.1.7 check_fpu()	175
7.11.1.8 check_fxr()	175
7.11.1.9 check_htt()	176
7.11.1.10 check_mca()	177
7.11.1.11 check_mce()	177
7.11.1.12 check_oxsave()	178
7.11.1.13 check_pae()	179
7.11.1.14 check_pcid()	180
7.11.1.15 check_rdrand()	180
7.11.1.16 check_rdseed()	181
7.11.1.17 check_sep()	182
7.11.1.18 check_sse()	182
7.11.1.19 check_tm()	183
7.11.1.20 check_tsc()	184
7.11.1.21 check_xsave()	185
7.11.1.22 cpuid_check_acpi()	185
7.11.1.23 cpuid_check_apic()	186
7.11.1.24 cpuid_check_avx()	186
7.11.1.25 cpuid_check_ds()	186
7.11.1.26 cpuid_check_fpu()	187
7.11.1.27 cpuid_check_fxr()	187
7.11.1.28 cpuid_check_htt()	187
7.11.1.29 cpuid_check_mca()	188
7.11.1.30 cpuid_check_mce()	188
7.11.1.31 cpuid_check_oxsave()	188
7.11.1.32 cpuid_check_pae()	189
7.11.1.33 cpuid_check_pcid()	189
7.11.1.34 cpuid_check_rdrand()	189
7.11.1.35 cpuid_check_rdseed()	190
7.11.1.36 cpuid_check_sep()	190
7.11.1.37 cpuid_check_sse()	190
7.11.1.38 cpuid_check_tm()	191
7.11.1.39 cpuid_check_tsc()	191
7.11.1.40 cpuid_check_xsave()	191
7.11.1.41 cpuid_readout()	192
7.11.1.42 fpu_init()	193
7.11.1.43 get_apic_base_address()	193
7.11.1.44 get_model()	194
7.11.1.45 get_xsave_size()	194
7.11.1.46 halt()	194
7.11.1.47 no_avx()	195
7.11.1.48 no_oxsave()	195

7.11.1.49 no_sse()	196
7.11.1.50 no_xsave()	196
7.11.1.51 rand_asm()	197
7.11.1.52 rdrand_asm()	197
7.11.1.53 rdseed_asm()	198
7.11.1.54 read_XCR0()	198
7.11.1.55 test_em()	198
7.11.2 Variable Documentation	198
7.11.2.1 ALIGN_16BIT	198
7.11.2.2 has_ACPI	199
7.12 cpuUtils.c	199
7.13 src/drivers/keyboard/keyboard.c File Reference	202
7.13.1 Macro Definition Documentation	203
7.13.1.1 KBD_STACK_SIZE	203
7.13.2 Function Documentation	204
7.13.2.1 k_getchar()	204
7.13.2.2 kbd_pop()	204
7.13.2.3 kbd_push()	204
7.13.2.4 keyboard_handler()	205
7.13.2.5 keyboard_init()	206
7.13.3 Variable Documentation	207
7.13.3.1 k_char	207
7.13.3.2 kbd_head	207
7.13.3.3 kbd_stack	207
7.13.3.4 kbd_tail	207
7.14 keyboard.c	208
7.15 src/early_alloc.c File Reference	209
7.15.1 Macro Definition Documentation	210
7.15.1.1 ALIGN_DOWN	210
7.15.1.2 ALIGN_UP	210
7.15.1.3 PAGE_SIZE	210
7.15.2 Function Documentation	210
7.15.2.1 early_alloc_page()	211
7.15.2.2 early_init()	211
7.15.2.3 early_pages_used()	212
7.15.2.4 early_reserved_region()	212
7.15.2.5 early_set_region()	213
7.15.2.6 memmap_ready()	213
7.15.2.7 memzero()	214
7.15.2.8 pick_region_from_memmap()	214
7.15.3 Variable Documentation	214
7.15.3.1 g_base	215

7.15.3.2 g_cursor	215
7.15.3.3 g_end	215
7.15.3.4 g_region_forced	215
7.15.3.5 memmap_req	215
7.16 early_alloc.c	216
7.17 src/frame_supplier.c File Reference	218
7.17.1 Variable Documentation	218
7.17.1.1 g_boot_phys_page	218
7.18 frame_supplier.c	218
7.19 src/gdt.c File Reference	219
7.19.1 Function Documentation	220
7.19.1.1 gdt_load_tss()	220
7.19.1.2 gdt_reload()	221
7.19.1.3 LoadGDT_Stage1()	221
7.19.2 Variable Documentation	222
7.19.2.1 desc	222
7.19.2.2 gdt	223
7.19.2.3 ist1Stack	223
7.19.2.4 ist2Stack	223
7.19.2.5 rsp0	223
7.19.2.6 tss	224
7.19.2.7 TssStack	224
7.20 gdt.c	224
7.21 src/idt.c File Reference	226
7.21.1 Macro Definition Documentation	227
7.21.1.1 YIELD_VECTOR	227
7.21.2 Function Documentation	227
7.21.2.1 halt()	227
7.21.2.2 idt_allocate_vector()	228
7.21.2.3 idt_free_vector()	228
7.21.2.4 idt_init()	229
7.21.2.5 idt_reg_test()	230
7.21.2.6 idt_reload()	230
7.21.2.7 idt_set_descriptor()	231
7.21.2.8 test_handler()	231
7.21.2.9 yield_isr_test()	232
7.21.2.10 yield_register()	233
7.21.3 Variable Documentation	233
7.21.3.1 idt	234
7.21.3.2 idtr	234
7.21.3.3 isr_delta	234
7.21.3.4 isr_stub_table	234

7.21.3.5 vectors	234
7.22 idt.c	235
7.23 src/include/acpi/acpi.h File Reference	236
7.23.1 Detailed Description	237
7.23.2 Function Documentation	238
7.23.2.1 acpi_find_sdt()	238
7.23.2.2 acpi_init()	239
7.24 acpi.h	239
7.25 src/include/acpi/madt.h File Reference	240
7.25.1 Detailed Description	241
7.25.2 Function Documentation	241
7.25.2.1 madt_init()	241
7.25.2.2 VECTOR_TYPE() [1/4]	242
7.25.2.3 VECTOR_TYPE() [2/4]	242
7.25.2.4 VECTOR_TYPE() [3/4]	242
7.25.2.5 VECTOR_TYPE() [4/4]	242
7.25.3 Variable Documentation	242
7.25.3.1 PACKED	243
7.26 madt.h	243
7.27 src/include/apic/ioapic.h File Reference	243
7.27.1 Detailed Description	244
7.27.2 Function Documentation	245
7.27.2.1 io_apic_set_gsi_redirect()	245
7.27.2.2 io_apic_set_irq_redirect()	246
7.28 ioapic.h	246
7.29 src/include/apic/lapic.h File Reference	247
7.29.1 Detailed Description	248
7.29.2 Function Documentation	248
7.29.2.1 lapic_eoi()	248
7.29.2.2 lapic_get_id()	249
7.29.2.3 lapic_init()	249
7.29.2.4 lapic_send_ipi()	249
7.29.2.5 lapic_timer_calibrate()	250
7.29.2.6 lapic_timer_oneshot()	250
7.30 lapic.h	251
7.31 src/include/consts.h File Reference	251
7.31.1 Macro Definition Documentation	252
7.31.1.1 KERNEL_VERSION	252
7.32 consts.h	252
7.33 src/include/cpu.h File Reference	252
7.33.1 Detailed Description	255
7.33.2 Macro Definition Documentation	255

7.33.2.1	_CPU_H	255
7.33.2.2	CPUID_AVX	255
7.33.2.3	CPUID_AVX512	255
7.33.2.4	CPUID_SEP	255
7.33.2.5	CPUID_XSAVE	255
7.33.3	Function Documentation	256
7.33.3.1	cpu_init()	256
7.33.3.2	cpuid()	256
7.33.3.3	disable_interrupts()	256
7.33.3.4	enable_interrupts()	257
7.33.3.5	fxrstor()	257
7.33.3.6	fxsave()	258
7.33.3.7	get_fs_base()	258
7.33.3.8	get_gs_base()	258
7.33.3.9	get_kernel_gs_base()	259
7.33.3.10	interrupt_state()	260
7.33.3.11	interrupt_toggle()	260
7.33.3.12	rdrand()	261
7.33.3.13	rdseed()	262
7.33.3.14	rdtsc()	262
7.33.3.15	set_fs_base()	262
7.33.3.16	set_gs_base()	263
7.33.3.17	set_kernel_gs_base()	263
7.33.3.18	this_cpu()	264
7.33.3.19	wrmsr()	265
7.33.3.20	xrstor()	266
7.33.3.21	xsave()	266
7.33.4	Variable Documentation	266
7.33.4.1	fpu_bank_size	266
7.33.4.2	fpu_rest	267
7.33.4.3	fpu_save	267
7.33.4.4	sysenter	267
7.34	cpu.h	267
7.35	src/include/cpuUtils.h File Reference	270
7.35.1	Detailed Description	272
7.35.2	Macro Definition Documentation	272
7.35.2.1	_CPU_UTILS_H	272
7.35.2.2	CPUID_VENDOR_AMD	272
7.35.2.3	CPUID_VENDOR_AO486	272
7.35.2.4	CPUID_VENDOR_BHYVE	273
7.35.2.5	CPUID_VENDOR_CENTAUR	273
7.35.2.6	CPUID_VENDOR_CYRIX	273

7.35.2.7 CPUID_VENDOR_ELBRUS	273
7.35.2.8 CPUID_VENDOR_HYGON	273
7.35.2.9 CPUID_VENDOR_HYPERV	273
7.35.2.10 CPUID_VENDOR_INTEL	274
7.35.2.11 CPUID_VENDOR_KVM	274
7.35.2.12 CPUID_VENDOR_NEXGEN	274
7.35.2.13 CPUID_VENDOR_NSC	274
7.35.2.14 CPUID_VENDOR_OLDAMD	274
7.35.2.15 CPUID_VENDOR_OLDAO486	274
7.35.2.16 CPUID_VENDOR_OLDTRANSMETA	275
7.35.2.17 CPUID_VENDOR_PARALLELS	275
7.35.2.18 CPUID_VENDOR_PARALLELS_ALT	275
7.35.2.19 CPUID_VENDOR_QEMU	275
7.35.2.20 CPUID_VENDOR_QNX	275
7.35.2.21 CPUID_VENDOR_RISE	275
7.35.2.22 CPUID_VENDOR_SIS	276
7.35.2.23 CPUID_VENDOR_TRANSMETA	276
7.35.2.24 CPUID_VENDOR_UMC	276
7.35.2.25 CPUID_VENDOR_VIA	276
7.35.2.26 CPUID_VENDOR_VIRTUALBOX	276
7.35.2.27 CPUID_VENDOR_VMWARE	276
7.35.2.28 CPUID_VENDOR_VORTEX	277
7.35.2.29 CPUID_VENDOR_XEN	277
7.35.2.30 CPUID_VENDOR_ZHAOXIN	277
7.35.3 Function Documentation	277
7.35.3.1 alloc_xsave()	277
7.35.3.2 check_acpi()	278
7.35.3.3 check_apic()	278
7.35.3.4 check_avx()	279
7.35.3.5 check_ds()	280
7.35.3.6 check_fpu()	281
7.35.3.7 check_fxsr()	281
7.35.3.8 check_htt()	282
7.35.3.9 check_mca()	283
7.35.3.10 check_mce()	283
7.35.3.11 check_oxsave()	284
7.35.3.12 check_pae()	285
7.35.3.13 check_pcid()	286
7.35.3.14 check_rdrand()	286
7.35.3.15 check_rdseed()	287
7.35.3.16 check_sep()	288
7.35.3.17 check_sse()	288

7.35.3.18	check_tm()	289
7.35.3.19	check_tsc()	290
7.35.3.20	check_xsave()	291
7.35.3.21	cpuid_readout()	291
7.35.3.22	fpu_init()	292
7.35.3.23	get_model()	293
7.35.3.24	rand_asm()	293
7.35.4	Variable Documentation	294
7.35.4.1	has_ACPI	294
7.35.4.2	xsave_bank	294
7.36	cpuUtils.h	295
7.37	src/include/drivers/keyboard/keyboard.h File Reference	296
7.37.1	Detailed Description	296
7.37.2	Macro Definition Documentation	296
7.37.2.1	KEYBOARD_DATA_PORT	297
7.37.2.2	KEYBOARD_STATUS_PORT	297
7.37.3	Function Documentation	297
7.37.3.1	k_getchar()	297
7.37.3.2	kbd_pop()	298
7.37.3.3	keyboard_handler()	298
7.37.3.4	keyboard_init()	299
7.38	keyboard.h	300
7.39	src/include/drivers/keyboard/keyboard_map.h File Reference	300
7.39.1	Variable Documentation	301
7.39.1.1	keyboard_ucharmap	301
7.39.1.2	keyboard_map	301
7.40	keyboard_map.h	302
7.41	src/include/early_alloc.h File Reference	305
7.41.1	Detailed Description	306
7.41.2	Typedef Documentation	306
7.41.2.1	paddr_t	307
7.41.3	Function Documentation	307
7.41.3.1	early_alloc_page()	307
7.41.3.2	early_init()	307
7.41.3.3	early_pages_used()	308
7.41.3.4	early_reserved_region()	308
7.41.3.5	early_set_region()	308
7.42	early_alloc.h	309
7.43	src/include/errno.h File Reference	309
7.43.1	Macro Definition Documentation	311
7.43.1.1	E2BIG	312
7.43.1.2	EACCES	312

7.43.1.3 EADDRINUSE	312
7.43.1.4 EADDRNOTAVAIL	312
7.43.1.5 EADV	312
7.43.1.6 EAFNOSUPPORT	312
7.43.1.7 EAGAIN	313
7.43.1.8 EALREADY	313
7.43.1.9 EBADE	313
7.43.1.10 EBADF	313
7.43.1.11 EBADFD	313
7.43.1.12 EBADMSG	313
7.43.1.13 EBADR	314
7.43.1.14 EBADRQC	314
7.43.1.15 EBADSLT	314
7.43.1.16 EBFONT	314
7.43.1.17 EBUSY	314
7.43.1.18 ECANCELED	314
7.43.1.19 ECHILD	315
7.43.1.20 ECHRNG	315
7.43.1.21 ECOMM	315
7.43.1.22 ECONNABORTED	315
7.43.1.23 ECONNREFUSED	315
7.43.1.24 ECONNRESET	315
7.43.1.25 EDEADLK	316
7.43.1.26 EDEADLOCK	316
7.43.1.27 EDESTADDRREQ	316
7.43.1.28 EDOM	316
7.43.1.29 EDOTDOT	316
7.43.1.30 EDQUOT	316
7.43.1.31 EEXIST	317
7.43.1.32 EFAULT	317
7.43.1.33 EFBIG	317
7.43.1.34 EHOSTDOWN	317
7.43.1.35 EHOSTUNREACH	317
7.43.1.36 EHWPOISON	317
7.43.1.37 EIDRM	318
7.43.1.38 EILSEQ	318
7.43.1.39 EINPROGRESS	318
7.43.1.40 EINTR	318
7.43.1.41 EINVAL	318
7.43.1.42 EIO	318
7.43.1.43 EISCONN	319
7.43.1.44 EISDIR	319

7.43.1.45 EISNAM	319
7.43.1.46 EKEYEXPIRED	319
7.43.1.47 EKEYREJECTED	319
7.43.1.48 EKEYREVOKED	319
7.43.1.49 EL2HLT	320
7.43.1.50 EL2NSYNC	320
7.43.1.51 EL3HLT	320
7.43.1.52 EL3RST	320
7.43.1.53 ELIBACC	320
7.43.1.54 ELIBBAD	320
7.43.1.55 ELIBEXEC	321
7.43.1.56 ELIBMAX	321
7.43.1.57 ELIBSCN	321
7.43.1.58 ELNRNG	321
7.43.1.59 ELOOP	321
7.43.1.60 EMEDIUMTYPE	321
7.43.1.61 EMFILE	322
7.43.1.62 EMLINK	322
7.43.1.63 EMSGSIZE	322
7.43.1.64 EMULTIHOP	322
7.43.1.65 ENAMETOOLONG	322
7.43.1.66 ENAVAIL	322
7.43.1.67 ENETDOWN	323
7.43.1.68 ENETRESET	323
7.43.1.69 ENETUNREACH	323
7.43.1.70 ENFILE	323
7.43.1.71 ENOANO	323
7.43.1.72 ENOBUFS	323
7.43.1.73 ENOCSI	324
7.43.1.74 ENODATA	324
7.43.1.75 ENODEV	324
7.43.1.76 ENOENT	324
7.43.1.77 ENOEXEC	324
7.43.1.78 ENOKEY	324
7.43.1.79 ENOLCK	325
7.43.1.80 ENOLINK	325
7.43.1.81 ENOMEDIUM	325
7.43.1.82 ENOMEM	325
7.43.1.83 ENOMSG	325
7.43.1.84 ENONET	325
7.43.1.85 ENOPKG	326
7.43.1.86 ENOPROTOPT	326

7.43.1.87 ENOSPC	326
7.43.1.88 ENOSR	326
7.43.1.89 ENOSTR	326
7.43.1.90 ENOSYS	326
7.43.1.91 ENOTBLK	327
7.43.1.92 ENOTCONN	327
7.43.1.93 ENOTDIR	327
7.43.1.94 ENOTEMPTY	327
7.43.1.95 ENOTNAM	327
7.43.1.96 ENOTRECOVERABLE	327
7.43.1.97 ENOTSOCK	328
7.43.1.98 ENOTSUP	328
7.43.1.99 ENOTTY	328
7.43.1.100 ENOTUNIQ	328
7.43.1.101 ENXIO	328
7.43.1.102 EOPNOTSUPP	328
7.43.1.103 EOVERFLOW	329
7.43.1.104 EOWNERDEAD	329
7.43.1.105 EPERM	329
7.43.1.106 EPNOSUPPORT	329
7.43.1.107 EPIPE	329
7.43.1.108 EPROTO	329
7.43.1.109 EPROTONOSUPPORT	330
7.43.1.110 EPROTOTYPE	330
7.43.1.111 ERANGE	330
7.43.1.112 EREMCHG	330
7.43.1.113 EREMOTE	330
7.43.1.114 EREMOTEIO	330
7.43.1.115 ERESTART	331
7.43.1.116 ERFKILL	331
7.43.1.117 EROFS	331
7.43.1.118 ESHUTDOWN	331
7.43.1.119 ESOCKTNOSUPPORT	331
7.43.1.120 ESPIPE	331
7.43.1.121 ESRCH	332
7.43.1.122 ESRMNT	332
7.43.1.123 ESTALE	332
7.43.1.124 ESTRPIPE	332
7.43.1.125 ETIME	332
7.43.1.126 ETIMEDOUT	332
7.43.1.127 ETOOMANYREFS	333
7.43.1.128 ETXTBSY	333

7.43.1.129 EUCLEAN	333
7.43.1.130 EUNATCH	333
7.43.1.131 EUSERS	333
7.43.1.132 EWOULDBLOCK	333
7.43.1.133 EXDEV	334
7.43.1.134 EXFULL	334
7.44 errno.h	334
7.45 src/include/gdt.h File Reference	336
7.45.1 Detailed Description	338
7.45.2 Macro Definition Documentation	338
7.45.2.1 GDTAccess16Code	338
7.45.2.2 GDTAccess16Data	338
7.45.2.3 GDTAccess32Code	338
7.45.2.4 GDTAccess32Data	338
7.45.2.5 GDTAccessDPL	339
7.45.2.6 GDTAccessKernelCode	339
7.45.2.7 GDTAccessKernelData	339
7.45.2.8 GDTAccessUserCode	339
7.45.2.9 GDTAccessUserData	339
7.45.2.10 GDTKernelBaseSelector	339
7.45.2.11 GDTTSSSegment	340
7.45.2.12 GDTUserBaseSelector	340
7.45.3 Enumeration Type Documentation	340
7.45.3.1 GDTAccessFlag	340
7.45.4 Function Documentation	340
7.45.4.1 gdt_load_tss()	340
7.45.4.2 gdt_reload()	341
7.45.4.3 LoadGDT_Stage1()	342
7.45.5 Variable Documentation	342
7.45.5.1 rsp0	342
7.46 gdt.h	343
7.47 src/include/global_defs.h File Reference	344
7.47.1 Macro Definition Documentation	345
7.47.1.1 ALIGN_16BIT	345
7.47.1.2 ALIGN_4K	345
7.47.1.3 ASSERT	345
7.47.1.4 CAS	345
7.47.1.5 MAX	346
7.47.1.6 MIN	346
7.47.1.7 PACKED	346
7.47.2 Typedef Documentation	346
7.47.2.1 HANDLE	346

7.47.2.2	lsize_t	347
7.47.2.3	mode_t	347
7.47.2.4	ssize_t	347
7.48	global_defs.h	347
7.49	src/include/idt.h File Reference	348
7.49.1	Detailed Description	349
7.49.2	Macro Definition Documentation	349
7.49.2.1	IDT_CPU_EXCEPTION_COUNT	350
7.49.2.2	IDT_DESCRIPTOR_CALL	350
7.49.2.3	IDT_DESCRIPTOR_EXCEPTION	350
7.49.2.4	IDT_DESCRIPTOR_EXTERNAL	350
7.49.2.5	IDT_DESCRIPTOR_PRESENT	350
7.49.2.6	IDT_DESCRIPTOR_RING1	351
7.49.2.7	IDT_DESCRIPTOR_RING2	351
7.49.2.8	IDT_DESCRIPTOR_RING3	351
7.49.2.9	IDT_DESCRIPTOR_X16_INTERRUPT	351
7.49.2.10	IDT_DESCRIPTOR_X16_TRAP	351
7.49.2.11	IDT_DESCRIPTOR_X32_INTERRUPT	351
7.49.2.12	IDT_DESCRIPTOR_X32_TASK	352
7.49.2.13	IDT_DESCRIPTOR_X32_TRAP	352
7.49.2.14	IDT_HDW_INTERRUPT_COUNT	352
7.49.2.15	IDT_MAX_DESCRIPTOR	352
7.49.3	Function Documentation	352
7.49.3.1	idt_allocate_vector()	353
7.49.3.2	idt_free_vector()	353
7.49.3.3	idt_init()	354
7.49.3.4	idt_reg_test()	355
7.49.3.5	idt_reload()	355
7.49.3.6	idt_set_descriptor()	356
7.49.3.7	yield_register()	356
7.49.4	Variable Documentation	357
7.49.4.1	isr_delta	357
7.49.4.2	isr_stub_table	357
7.50	idt.h	358
7.51	src/include/interrupts.h File Reference	358
7.51.1	Detailed Description	359
7.52	interrupts.h	360
7.53	src/include/io.h File Reference	360
7.53.1	Detailed Description	361
7.53.2	Macro Definition Documentation	361
7.53.2.1	_IO_H	361
7.53.3	Function Documentation	361

7.53.3.1 inb()	361
7.53.3.2 io_wait()	362
7.53.3.3 outb()	363
7.54 io.h	363
7.55 src/include/kernel.h File Reference	364
7.55.1 Detailed Description	365
7.55.2 Macro Definition Documentation	365
7.55.2.1 _KERNEL_H	365
7.55.3 Variable Documentation	365
7.55.3.1 bootspace	365
7.55.3.2 early_term	365
7.55.3.3 init_done	366
7.55.3.4 kerror_mode	366
7.56 kernel.h	366
7.57 src/include/KernelUtils.h File Reference	366
7.57.1 Detailed Description	368
7.57.2 Macro Definition Documentation	368
7.57.2.1 _KERNEL_UTILS_H	368
7.57.3 Function Documentation	368
7.57.3.1 bytes_to_gib()	368
7.57.3.2 bytes_to_mib()	369
7.57.3.3 get_memory_size()	369
7.57.3.4 get_memory_size_gib()	370
7.57.3.5 print_memmap()	370
7.57.3.6 restore_floats()	371
7.57.3.7 save_floats()	371
7.57.4 Variable Documentation	371
7.57.4.1 k_mode	371
7.57.4.2 term_context	371
7.58 KernelUtils.h	372
7.59 src/include/lib/hashmap.h File Reference	372
7.59.1 Macro Definition Documentation	373
7.59.1.1 _HASHMAP_H	373
7.59.1.2 HASHMAP_DELETE	374
7.59.1.3 HASHMAP_GET	374
7.59.1.4 HASHMAP_INIT	374
7.59.1.5 HASHMAP_INSERT	375
7.59.1.6 HASHMAP_KEY_DATA_MAX	375
7.59.1.7 HASHMAP_REMOVE	375
7.59.1.8 HASHMAP_SGET	375
7.59.1.9 HASHMAP_SINSERT	376
7.59.1.10 HASHMAP_SREMOVE	376

7.59.1.11 HASHMAP_TYPE	376
7.59.2 Function Documentation	377
7.59.2.1 hash()	377
7.60 hashmap.h	377
7.61 src/include/lib/vector.h File Reference	380
7.61.1 Macro Definition Documentation	382
7.61.1.1 VECTOR_ENSURE_LENGTH	382
7.61.1.2 VECTOR_FIND	382
7.61.1.3 VECTOR_FOR_EACH	383
7.61.1.4 VECTOR_INIT	383
7.61.1.5 VECTOR_INSERT	383
7.61.1.6 VECTOR_INVALID_INDEX	384
7.61.1.7 VECTOR_ITEM	384
7.61.1.8 VECTOR_PUSH_BACK	384
7.61.1.9 VECTOR_PUSH_FRONT	384
7.61.1.10 VECTOR_REMOVE	385
7.61.1.11 VECTOR_REMOVE_BY_VALUE	385
7.61.1.12 VECTOR_TYPE	385
7.62 vector.h	386
7.63 src/include/limine.h File Reference	388
7.63.1 Macro Definition Documentation	390
7.63.1.1 LIMINE_5_LEVEL_PAGING_REQUEST	390
7.63.1.2 LIMINE_BOOT_TIME_REQUEST	390
7.63.1.3 LIMINE_BOOTLOADER_INFO_REQUEST	391
7.63.1.4 LIMINE_COMMON_MAGIC	391
7.63.1.5 LIMINE_DTB_REQUEST	391
7.63.1.6 LIMINE_EFI_SYSTEM_TABLE_REQUEST	391
7.63.1.7 LIMINE_ENTRY_POINT_REQUEST	391
7.63.1.8 LIMINE_FRAMEBUFFER_REQUEST	392
7.63.1.9 LIMINE_FRAMEBUFFER_RGB	392
7.63.1.10 LIMINE_HHDM_REQUEST	392
7.63.1.11 LIMINE_KERNEL_ADDRESS_REQUEST	392
7.63.1.12 LIMINE_KERNEL_FILE_REQUEST	392
7.63.1.13 LIMINE_MEDIA_TYPE_GENERIC	393
7.63.1.14 LIMINE_MEDIA_TYPE_OPTICAL	393
7.63.1.15 LIMINE_MEDIA_TYPE_TFTP	393
7.63.1.16 LIMINE_MEMMAP_ACPI_NVS	393
7.63.1.17 LIMINE_MEMMAP_ACPI_RECLAIMABLE	393
7.63.1.18 LIMINE_MEMMAP_BAD_MEMORY	393
7.63.1.19 LIMINE_MEMMAP_BOOTLOADER_RECLAIMABLE	394
7.63.1.20 LIMINE_MEMMAP_FRAMEBUFFER	394
7.63.1.21 LIMINE_MEMMAP_KERNEL_AND_MODULES	394

7.63.1.22	LIMINE_MEMMAP_REQUEST	394
7.63.1.23	LIMINE_MEMMAP_RESERVED	394
7.63.1.24	LIMINE_MEMMAP_USABLE	394
7.63.1.25	LIMINE_MODULE_REQUEST	395
7.63.1.26	LIMINE_PTR	395
7.63.1.27	LIMINE_RSDP_REQUEST	395
7.63.1.28	LIMINE_SMBIOS_REQUEST	395
7.63.1.29	LIMINE_SMP_REQUEST	395
7.63.1.30	LIMINE_STACK_SIZE_REQUEST	396
7.63.1.31	LIMINE_TERMINAL_CB_BELL	396
7.63.1.32	LIMINE_TERMINAL_CB_DEC	396
7.63.1.33	LIMINE_TERMINAL_CB_KBD_LEDS	396
7.63.1.34	LIMINE_TERMINAL_CB_LINUX	396
7.63.1.35	LIMINE_TERMINAL_CB_MODE	396
7.63.1.36	LIMINE_TERMINAL_CB_POS_REPORT	397
7.63.1.37	LIMINE_TERMINAL_CB_PRIVATE_ID	397
7.63.1.38	LIMINE_TERMINAL_CB_STATUS_REPORT	397
7.63.1.39	LIMINE_TERMINAL_CTX_RESTORE	397
7.63.1.40	LIMINE_TERMINAL_CTX_SAVE	397
7.63.1.41	LIMINE_TERMINAL_CTX_SIZE	397
7.63.1.42	LIMINE_TERMINAL_FULL_REFRESH	398
7.63.1.43	LIMINE_TERMINAL_REQUEST	398
7.63.2	Typedef Documentation	398
7.63.2.1	limine_entry_point	398
7.63.2.2	limine_goto_address	398
7.63.2.3	limine_terminal_callback	398
7.63.2.4	limine_terminal_write	399
7.64	limine.h	399
7.65	src/include/lock.h File Reference	405
7.65.1	Macro Definition Documentation	406
7.65.1.1	_LOCK_C_H	406
7.65.1.2	SMARTLOCK_INIT	406
7.65.1.3	SPINLOCK_INIT	406
7.65.2	Typedef Documentation	407
7.65.2.1	spinlock_t	407
7.65.3	Function Documentation	407
7.65.3.1	smartlock_acquire()	407
7.65.3.2	smartlock_release()	407
7.65.3.3	spinlock_acquire()	407
7.65.3.4	spinlock_release()	408
7.65.3.5	spinlock_test_and_acq()	409
7.66	lock.h	409

7.67 src/include/mm/kheap.h File Reference	410
7.67.1 Macro Definition Documentation	410
7.67.1.1 KHEAP_H	410
7.67.2 Function Documentation	411
7.67.2.1 kheap_init()	411
7.67.2.2 kheap_reserve_pages()	411
7.68 kheap.h	411
7.69 src/include/mm/kmalloc.h File Reference	411
7.69.1 Detailed Description	413
7.69.2 Macro Definition Documentation	413
7.69.2.1 KMALLOC_H	413
7.69.3 Function Documentation	413
7.69.3.1 kfree()	413
7.69.3.2 kmalloc()	414
7.69.3.3 kmalloc_dump()	415
7.69.3.4 kmalloc_init()	416
7.69.3.5 kmalloc_usable_size()	416
7.69.3.6 krealloc()	417
7.70 kmalloc.h	418
7.71 src/include/mm/memtest.h File Reference	418
7.71.1 Detailed Description	419
7.71.2 Macro Definition Documentation	419
7.71.2.1 VOYAGER_MEMTEST_H	419
7.71.3 Function Documentation	420
7.71.3.1 memtest_run()	420
7.72 memtest.h	421
7.73 src/include/paging/frame_supplier.h File Reference	421
7.73.1 Detailed Description	422
7.73.2 Macro Definition Documentation	423
7.73.2.1 FRAME_SUPPLIER_H	423
7.73.3 Typedef Documentation	423
7.73.3.1 phys_page_supplier_t	423
7.73.4 Function Documentation	423
7.73.4.1 PagingGetFreeFrame()	423
7.73.5 Variable Documentation	424
7.73.5.1 g_boot_phys_page	424
7.74 frame_supplier.h	424
7.75 src/include/paging/neo_framealloc.h File Reference	424
7.75.1 Detailed Description	426
7.75.2 Function Documentation	426
7.75.2.1 frame_alloc()	426
7.75.2.2 frame_free_count()	427

7.75.2.3	frame_init()	428
7.75.2.4	frame_is_allocated()	428
7.75.2.5	frame_total_count()	429
7.75.2.6	n_frame_free()	430
7.76	neo_framealloc.h	431
7.77	src/include/paging/paging.h File Reference	432
7.77.1	Detailed Description	433
7.77.2	Macro Definition Documentation	433
7.77.2.1	_PAGING_H	434
7.77.2.2	HIGHER_HALF_KERNEL_MEMORY_OFFSET	434
7.77.2.3	HIGHER_HALF_MEMORY_OFFSET	434
7.77.2.4	PAGE_ADDRESS_MASK	434
7.77.2.5	PAGE_FLAG_MASK	434
7.77.2.6	PAGING_EXPORT	434
7.77.3	Enumeration Type Documentation	434
7.77.3.1	PagingFlag	434
7.77.4	Function Documentation	435
7.77.4.1	__attribute__()	435
7.77.4.2	IsHigherHalf()	435
7.77.4.3	IsKernelHigherHalf()	436
7.77.4.4	PagingDuplicate()	436
7.77.4.5	PagingIdentityMap()	437
7.77.4.6	PagingMapMemory()	437
7.77.4.7	PagingPhysicalMemory()	438
7.77.4.8	PagingSetActivePageTable()	439
7.77.4.9	PagingUnmapMemory()	439
7.77.4.10	TranslateToHighHalfMemoryAddress()	440
7.77.4.11	TranslateToKernelMemoryAddress()	441
7.77.4.12	TranslateToKernelPhysicalMemoryAddress()	441
7.77.4.13	TranslateToPhysicalMemoryAddress()	441
7.78	paging.h	442
7.79	src/include/paging/paging_bootstrap.h File Reference	443
7.79.1	Detailed Description	444
7.79.2	Function Documentation	445
7.79.2.1	paging_bootstrap()	445
7.79.2.2	paging_pml4_phys()	446
7.79.2.3	paging_pml4_virt()	447
7.80	paging_bootstrap.h	447
7.81	src/include/panic.h File Reference	447
7.81.1	Detailed Description	448
7.81.2	Macro Definition Documentation	448
7.81.2.1	_PANIC_H	448

7.81.3 Function Documentation	448
7.81.3.1 <code>__attribute__()</code>	448
7.82 <code>panic.h</code>	448
7.83 <code>src/include/pic.h</code> File Reference	449
7.83.1 Detailed Description	450
7.83.2 Macro Definition Documentation	450
7.83.2.1 <code>PIC_EOI</code>	450
7.83.2.2 <code>PIC_ICW1_ICW4</code>	450
7.83.2.3 <code>PIC_ICW1_INIT</code>	450
7.83.2.4 <code>PIC_ICW1_INTERVAL4</code>	451
7.83.2.5 <code>PIC_ICW1_LEVEL</code>	451
7.83.2.6 <code>PIC_ICW1_SINGLE</code>	451
7.83.2.7 <code>PIC_ICW4_8086</code>	451
7.83.2.8 <code>PIC_ICW4_AUTO</code>	451
7.83.2.9 <code>PIC_ICW4_BUF_MASTER</code>	451
7.83.2.10 <code>PIC_ICW4_BUF_SLAVE</code>	452
7.83.2.11 <code>PIC_MASTER</code>	452
7.83.2.12 <code>PIC_MASTER_COMMAND</code>	452
7.83.2.13 <code>PIC_MASTER_DATA</code>	452
7.83.2.14 <code>PIC_SLAVE</code>	452
7.83.2.15 <code>PIC_SLAVE_COMMAND</code>	452
7.83.2.16 <code>PIC_SLAVE_DATA</code>	453
7.83.3 Function Documentation	453
7.83.3.1 <code>pic_enable()</code>	453
7.83.3.2 <code>pic_mask_irq()</code>	453
7.83.3.3 <code>pic_remap_offsets()</code>	454
7.83.3.4 <code>pic_send_eoi()</code>	455
7.83.3.5 <code>pic_unmask_irq()</code>	456
7.84 <code>pic.h</code>	457
7.85 <code>src/include/printf.h</code> File Reference	457
7.85.1 Macro Definition Documentation	458
7.85.1.1 <code>_PRINTF_H_</code>	458
7.85.1.2 <code>printf</code>	459
7.85.1.3 <code>snprintf</code>	459
7.85.1.4 <code>sprintf</code>	459
7.85.1.5 <code>vprintf</code>	460
7.85.1.6 <code>vsnprintf</code>	460
7.85.2 Function Documentation	460
7.85.2.1 <code>_putchar()</code>	460
7.85.2.2 <code>fctprintf()</code>	462
7.85.2.3 <code>printf_()</code>	463
7.85.2.4 <code>snprintf_()</code>	465

7.85.2.5 <code>sprintf_()</code>	466
7.85.2.6 <code>vprintf_()</code>	467
7.85.2.7 <code>vsprintf_()</code>	467
7.86 <code>printf.h</code>	468
7.87 <code>src/include/proc.h</code> File Reference	468
7.87.1 Function Documentation	469
7.87.1.1 <code>proc_yield()</code>	469
7.88 <code>proc.h</code>	470
7.89 <code>src/include/proc_utils.h</code> File Reference	470
7.89.1 Detailed Description	470
7.89.2 Function Documentation	471
7.89.2.1 <code>soak()</code>	471
7.90 <code>proc_utils.h</code>	471
7.91 <code>src/include/registers.h</code> File Reference	472
7.91.1 Detailed Description	473
7.91.2 Function Documentation	473
7.91.2.1 <code>rdmsr()</code>	473
7.91.2.2 <code>readCR2()</code>	474
7.91.2.3 <code>readCR3()</code>	474
7.91.2.4 <code>readCR4()</code>	475
7.91.2.5 <code>readCRO()</code>	476
7.91.2.6 <code>readRSP()</code>	476
7.91.2.7 <code>writeCR0()</code>	477
7.91.2.8 <code>writeCR3()</code>	477
7.91.2.9 <code>writeCR4()</code>	477
7.91.2.10 <code>writeMSR()</code>	478
7.92 <code>registers.h</code>	478
7.93 <code>src/include/sched.h</code> File Reference	478
7.93.1 Detailed Description	480
7.93.2 Macro Definition Documentation	480
7.93.2.1 <code>_SCHED_H</code>	480
7.93.2.2 <code>SYSCALL_EXIT</code>	481
7.93.2.3 <code>SYSCALL_FORK</code>	481
7.93.2.4 <code>SYSCALL_YIELD</code>	481
7.93.3 Typedef Documentation	481
7.93.3.1 <code>process_t</code>	481
7.93.4 Enumeration Type Documentation	481
7.93.4.1 <code>proc_state_t</code>	481
7.93.5 Function Documentation	482
7.93.5.1 <code>create_process()</code>	482
7.93.5.2 <code>init_scheduler()</code>	483
7.93.5.3 <code>schedule()</code>	483

7.93.5.4 scheduler_start()	484
7.93.5.5 switch_to()	485
7.93.6 Variable Documentation	486
7.93.6.1 allow_sched	486
7.93.6.2 current	486
7.94 sched.h	487
7.95 src/include/serial.h File Reference	487
7.95.1 Detailed Description	488
7.95.2 Macro Definition Documentation	488
7.95.2.1 SERIAL_H	488
7.95.3 Function Documentation	489
7.95.3.1 get_ts()	489
7.95.3.2 serial_debug()	490
7.95.3.3 serial_print()	491
7.95.3.4 serial_print_line()	491
7.95.3.5 serial_print_with_ts()	492
7.96 serial.h	493
7.97 src/include/shell.h File Reference	493
7.97.1 Detailed Description	494
7.97.2 Macro Definition Documentation	494
7.97.2.1 _SHELL_H	494
7.97.2.2 VSH_CMD_BUFFER_SIZE	494
7.97.2.3 VSH_VERSION	494
7.97.3 Function Documentation	494
7.97.3.1 cmd_parser()	494
7.97.3.2 vsh_loop()	495
7.97.3.3 vsh_readline()	496
7.98 shell.h	497
7.99 src/include/stack_trace.h File Reference	498
7.99.1 Detailed Description	499
7.99.2 Macro Definition Documentation	499
7.99.2.1 _STACK_TRACE_H	499
7.99.3 Function Documentation	499
7.99.3.1 dump_hex()	499
7.99.3.2 print_stack_size()	500
7.99.3.3 stack_dump()	501
7.99.3.4 stack_dump_asm()	501
7.99.3.5 stack_dump_recursive()	502
7.99.3.6 stack_trace()	503
7.99.3.7 stack_trace_asm()	504
7.99.4 Variable Documentation	505
7.99.4.1 stack_req	505

7.100	stack_trace.h	505
7.101	src/include/string.h File Reference	506
7.101.1	Function Documentation	506
7.101.1.1	int_to_str()	507
7.101.1.2	memcmp()	507
7.101.1.3	memcpy()	508
7.101.1.4	memmove()	508
7.101.1.5	memset()	508
7.101.1.6	str2int()	509
7.101.1.7	str2int2()	509
7.101.1.8	strlen()	509
7.101.1.9	strncpy()	510
7.101.1.10	strtok()	510
7.101.1.11	u64_to_str()	511
7.102	string.h	511
7.103	src/include/syscall.h File Reference	511
7.104	syscall.h	511
7.105	src/include/terminal/framebuffer.h File Reference	512
7.105.1	Detailed Description	513
7.105.2	Macro Definition Documentation	513
7.105.2.1	FBTERM_FONT_GLYPHS	513
7.105.3	Function Documentation	513
7.105.3.1	fbterm_init()	513
7.106	framebuffer.h	516
7.107	src/include/terminal/term.h File Reference	517
7.107.1	Detailed Description	518
7.107.2	Macro Definition Documentation	518
7.107.2.1	TERM_CB_BELL	519
7.107.2.2	TERM_CB_DEC	519
7.107.2.3	TERM_CB_KBD_LEDS	519
7.107.2.4	TERM_CB_LINUX	519
7.107.2.5	TERM_CB_MODE	519
7.107.2.6	TERM_CB_POS_REPORT	519
7.107.2.7	TERM_CB_PRIVATE_ID	520
7.107.2.8	TERM_CB_STATUS_REPORT	520
7.107.2.9	TERM_MAX_ESC_VALUES	520
7.107.3	Function Documentation	520
7.107.3.1	term_context_reinit()	520
7.107.3.2	term_write()	521
7.108	term.h	523
7.109	src/include/time.h File Reference	525
7.109.1	Detailed Description	527

7.109.2 Macro Definition Documentation	527
7.109.2.1 _TIME_H	527
7.109.2.2 PIT_DIVIDEND	527
7.109.2.3 PIT_FREQ	527
7.109.3 Typedef Documentation	528
7.109.3.1 time_t	528
7.109.4 Function Documentation	528
7.109.4.1 dayofweek()	528
7.109.4.2 init_PIT()	529
7.109.4.3 pit_get_current_count()	529
7.109.4.4 pit_set_frequency()	530
7.109.4.5 pit_set_reload_value()	531
7.109.4.6 pit_sleep()	531
7.109.4.7 print_date()	532
7.109.4.8 print_load_time()	532
7.109.4.9 print_sys_time()	533
7.109.4.10 read_rtc()	534
7.109.4.11 sys_clock_handler()	535
7.109.4.12 sys_clock_handler_alt()	535
7.109.4.13 time_init()	536
7.109.4.14 timer_arm()	537
7.109.4.15 timer_disarm()	537
7.109.4.16 timer_new()	538
7.109.4.17 timespec_add()	538
7.109.4.18 timespec_sub()	538
7.109.5 Variable Documentation	539
7.109.5.1 day	539
7.109.5.2 hour	539
7.109.5.3 minute	540
7.109.5.4 month	540
7.109.5.5 second	540
7.109.5.6 system_timer_fractions	540
7.109.5.7 system_timer_ms	540
7.109.5.8 time_mono	541
7.109.5.9 time_real	541
7.109.5.10 year	541
7.110 time.h	541
7.111 src/include/tss.h File Reference	542
7.111.1 Detailed Description	543
7.111.2 Macro Definition Documentation	544
7.111.2.1 _TSS_H	544
7.112 tss.h	544

7.113 src/include/vgafont.h File Reference	544
7.113.1 Variable Documentation	545
7.113.1.1 vgafont	545
7.114 vgafont.h	545
7.115 src/include/voyager_docs.h File Reference	549
7.115.1 Detailed Description	549
7.116 voyager_docs.h	549
7.117 src/int2srt.c File Reference	550
7.117.1 Function Documentation	550
7.117.1.1 int_to_str()	550
7.117.1.2 u64_to_str()	551
7.118 int2srt.c	551
7.119 src/interrupts.c File Reference	552
7.119.1 Function Documentation	552
7.119.1.1 dyn_isr_handler()	553
7.119.1.2 irq_handler()	553
7.119.1.3 isr_exception_handler()	553
7.119.2 Variable Documentation	554
7.119.2.1 exception_messages	554
7.120 interrupts.c	554
7.121 src/io.c File Reference	556
7.121.1 Function Documentation	556
7.121.1.1 inb()	557
7.121.1.2 io_wait()	558
7.121.1.3 outb()	559
7.122 io.c	559
7.123 src/kernel.c File Reference	560
7.123.1 Macro Definition Documentation	561
7.123.1.1 Black	561
7.123.1.2 Blue	562
7.123.1.3 Cyan	562
7.123.1.4 Green	562
7.123.1.5 KERNEL_VERSION	562
7.123.1.6 Purple	562
7.123.1.7 Red	562
7.123.1.8 White	563
7.123.1.9 Yellow	563
7.123.2 Function Documentation	563
7.123.2.1 _start()	563
7.123.2.2 breakpoint()	564
7.123.2.3 halt()	565
7.123.2.4 hello_general_floatius()	565

7.123.2.5	limine_debug()	565
7.123.2.6	memory_bringup()	566
7.123.2.7	proc_a()	567
7.123.2.8	proc_b()	567
7.123.2.9	scheduler_heartbeat()	568
7.123.2.10	start_interrupts()	568
7.123.2.11	stop_interrupts()	568
7.123.2.12	supply_from_early()	569
7.123.2.13	supply_from_frame()	570
7.123.2.14	task_switch_int()	570
7.123.2.15	walk_stack()	570
7.123.3	Variable Documentation	571
7.123.3.1	bootspace	571
7.123.3.2	early_term	571
7.123.3.3	fbr_req	571
7.123.3.4	hhdm_request	571
7.123.3.5	init_done	571
7.123.3.6	Kaddress_req	572
7.123.3.7	kerror_mode	572
7.123.3.8	memmap_req	572
7.123.3.9	term_bg	572
7.123.3.10	term_context	572
7.123.3.11	term_fg	572
7.124	kernel.c	573
7.125	src/KernelUtils.c File Reference	576
7.125.1	Function Documentation	577
7.125.1.1	bytes_to_gib()	578
7.125.1.2	bytes_to_mib()	578
7.125.1.3	get_memory_size()	579
7.125.1.4	get_memory_size_gib()	579
7.125.1.5	print_memmap()	580
7.125.2	Variable Documentation	580
7.125.2.1	k_mode	581
7.125.2.2	memmap_req	581
7.126	KernelUtils.c	581
7.127	src/limine_requests.c File Reference	583
7.127.1	Function Documentation	583
7.127.1.1	__attribute__()	583
7.128	limine_requests.c	584
7.129	src/lock_c.c File Reference	585
7.130	lock_c.c	585
7.131	src/memcmp.c File Reference	585

7.131.1 Function Documentation	586
7.131.1.1 memcmp()	586
7.132 memcmp.c	586
7.133 src/memcpy.c File Reference	587
7.133.1 Function Documentation	587
7.133.1.1 memcpy()	587
7.134 memcpy.c	588
7.135 src/memmove.c File Reference	588
7.135.1 Function Documentation	589
7.135.1.1 memmove()	589
7.136 memmove.c	589
7.137 src/memset.c File Reference	590
7.137.1 Function Documentation	590
7.137.1.1 memset()	590
7.138 memset.c	591
7.139 src/mm/kmalloc.c File Reference	591
7.139.1 Macro Definition Documentation	593
7.139.1.1 ALIGN_DOWN	593
7.139.1.2 ALIGN_UP	593
7.139.1.3 ALIGNMENT	593
7.139.1.4 ALLOC_FLAG	594
7.139.1.5 BLK_SIZE	594
7.139.1.6 CLR_ALLOC	594
7.139.1.7 FTR_SIZE	594
7.139.1.8 HDR_SIZE	594
7.139.1.9 HEAP_BASE	594
7.139.1.10 HEAP_LIMIT	595
7.139.1.11 IS_ALLOC	595
7.139.1.12 MIN_SPLIT_SIZE	595
7.139.1.13 OVERHEAD	595
7.139.1.14 PAGE_SIZE	595
7.139.1.15 PAGING_FLAG_NO_EXECUTE	595
7.139.1.16 PAGING_FLAG_PRESENT	596
7.139.1.17 PAGING_FLAG_WRITABLE	596
7.139.1.18 SET_ALLOC	596
7.139.1.19 SET_SIZE	596
7.139.1.20 USABLE	596
7.139.2 Typedef Documentation	596
7.139.2.1 block_t	597
7.139.2.2 footer_t	597
7.139.3 Function Documentation	597
7.139.3.1 allocate_from_block()	597

7.139.3.2 coalesce()	598
7.139.3.3 current_pml4_phys()	599
7.139.3.4 fl_find_fit()	599
7.139.3.5 fl_push()	600
7.139.3.6 fl_remove()	600
7.139.3.7 FOOTER_OF()	601
7.139.3.8 heap_map_more()	602
7.139.3.9 kfree()	602
7.139.3.10 kmalloc()	603
7.139.3.11 kmalloc_dump()	604
7.139.3.12 kmalloc_init()	605
7.139.3.13 kmalloc_usable_size()	606
7.139.3.14 krealloc()	606
7.139.3.15 NEXT_BLOCK()	607
7.139.3.16 panic()	608
7.139.3.17 PREV_BLOCK()	609
7.139.3.18 ReadCR3()	609
7.139.3.19 total_size_for_payload()	610
7.139.4 Variable Documentation	610
7.139.4.1 g_free_list	611
7.139.4.2 g_heap_curr	611
7.139.4.3 g_heap_lock	611
7.140 kmalloc.c	611
7.141 src/mm/memtest.c File Reference	615
7.141.1 Macro Definition Documentation	617
7.141.1.1 MEMTEST_GROWTH_SIZE	617
7.141.1.2 MEMTEST_GROWTH_SLOTS	617
7.141.1.3 MEMTEST_PAGE_SIZE	617
7.141.1.4 MEMTEST_PMM_MAX_FRAMES	618
7.141.1.5 MEMTEST_RANDOM_MAX_ALLOC	618
7.141.1.6 MEMTEST_RANDOM_OPS	618
7.141.1.7 MEMTEST_SLOT_COUNT	618
7.141.2 Typedef Documentation	618
7.141.2.1 memtest_slot_t	618
7.141.2.2 memtest_stats_t	618
7.141.3 Function Documentation	619
7.141.3.1 basic_heap_test()	619
7.141.3.2 clear_slots()	619
7.141.3.3 fill_slot()	620
7.141.3.4 fragmentation_test()	621
7.141.3.5 heap_growth_test()	621
7.141.3.6 memtest_fail()	622

7.141.3.7 memtest_run()	623
7.141.3.8 page_pattern()	624
7.141.3.9 payload_pattern()	625
7.141.3.10 pmm_roundtrip_test()	625
7.141.3.11 random_cycle()	626
7.141.3.12 randomized_heap_test()	626
7.141.3.13 realloc_test()	627
7.141.3.14 release_slot()	628
7.141.3.15 set_slot()	629
7.141.3.16 stats_add_live()	629
7.141.3.17 verify_alignment()	630
7.141.3.18 verify_all()	630
7.141.3.19 verify_no_overlap()	631
7.141.3.20 verify_prefix()	632
7.141.3.21 verify_slot()	632
7.141.3.22 xorshift64()	633
7.141.4 Variable Documentation	633
7.141.4.1 g_basic_sizes	633
7.141.4.2 g_pmm_frames	633
7.141.4.3 g_slots	634
7.141.4.4 hhdmm_request	634
7.142 memtest.c	634
7.143 src/neo_framealloc.c File Reference	642
7.143.1 Macro Definition Documentation	643
7.143.1.1 ALIGN_DOWN	643
7.143.1.2 ALIGN_UP	643
7.143.1.3 PAGE_SIZE	644
7.143.2 Function Documentation	644
7.143.2.1 bm_clear()	644
7.143.2.2 bm_set()	644
7.143.2.3 bm_test()	645
7.143.2.4 compute_max_phys_and_count()	645
7.143.2.5 early_alloc_page()	646
7.143.2.6 early_init()	646
7.143.2.7 early_reserved_region()	647
7.143.2.8 frame_addr_of()	648
7.143.2.9 frame_alloc()	648
7.143.2.10 frame_free_count()	650
7.143.2.11 frame_index_of()	650
7.143.2.12 frame_init()	651
7.143.2.13 frame_is_allocated()	651
7.143.2.14 frame_total_count()	652

7.143.2.15 mark_range_allocated()	653
7.143.2.16 mark_range_usable_and_free()	653
7.143.2.17 memset8()	654
7.143.2.18 memzero()	655
7.143.2.19 n_frame_free()	655
7.143.2.20 panic()	656
7.143.3 Variable Documentation	657
7.143.3.1 __kernel_end	657
7.143.3.2 __kernel_start	657
7.143.3.3 g_alloc_bm	657
7.143.3.4 g_frame_count	657
7.143.3.5 g_free_count	658
7.143.3.6 g_hint	658
7.143.3.7 g_max_phys	658
7.143.3.8 g_memlock	658
7.143.3.9 g_usable_bm	658
7.143.3.10 hhdmm_request	658
7.143.3.11 Kaddress_req	659
7.143.3.12 memmap_req	659
7.143.3.13 module_request	659
7.144 neo_framealloc.c	659
7.145 src/paging.c File Reference	662
7.145.1 Function Documentation	663
7.145.1.1 DuplicateRecursive()	664
7.145.1.2 GetOrAllocEntry()	665
7.145.1.3 GetOrNullifyEntry()	666
7.145.1.4 OffsetToVirtualAddress()	666
7.145.1.5 PagingDuplicate()	667
7.145.1.6 PagingIdentityMap()	667
7.145.1.7 PagingMapMemory()	669
7.145.1.8 PagingPhysicalMemory()	670
7.145.1.9 PagingUnmapMemory()	671
7.145.1.10 ReadCR3()	671
7.145.1.11 VirtualAddressToOffsets()	672
7.145.1.12 WriteCR3()	673
7.146 paging.c	673
7.147 src/paging/paging_bootstrap.c File Reference	676
7.147.1 Macro Definition Documentation	678
7.147.1.1 ADDR_MASK	678
7.147.1.2 ADDR_MASK_1G	678
7.147.1.3 ADDR_MASK_2M	678
7.147.1.4 ALIGN_DOWN	679

7.147.1.5 ALIGN_UP	679
7.147.1.6 PAGE_1G_SIZE	679
7.147.1.7 PAGE_2M_SIZE	679
7.147.1.8 PAGE_SIZE	679
7.147.1.9 PTE_NX	679
7.147.1.10 PTE_P	680
7.147.1.11 PTE_PS	680
7.147.1.12 PTE_U	680
7.147.1.13 PTE_W	680
7.147.2 Function Documentation	680
7.147.2.1 __attribute__()	680
7.147.2.2 alloc_table()	681
7.147.2.3 ensure_hhdm_cover_phys_page()	681
7.147.2.4 ensure_hhdm_cover_phys_range()	682
7.147.2.5 get_max_phys()	682
7.147.2.6 get_or_make_next()	683
7.147.2.7 get_or_make_pd()	683
7.147.2.8 get_total_usable_mem_bytes()	684
7.147.2.9 install_new_cr3()	684
7.147.2.10 map_1g()	685
7.147.2.11 map_2m()	685
7.147.2.12 map_4k()	686
7.147.2.13 map_hhdm()	686
7.147.2.14 map_identity_minimal()	687
7.147.2.15 map_kernel_higher_half()	687
7.147.2.16 map_range_4k()	688
7.147.2.17 map_range_huge_first()	689
7.147.2.18 memzero()	689
7.147.2.19 paging_bootstrap()	690
7.147.2.20 paging_pml4_phys()	690
7.147.2.21 paging_pml4_virt()	691
7.147.2.22 panic()	692
7.147.2.23 phys_to_virt()	692
7.147.2.24 pre_switch_audit_and_patch()	693
7.147.2.25 probe_hhdm_phys()	693
7.147.2.26 probe_va()	694
7.147.2.27 readCR3()	695
7.147.2.28 readRSP()	695
7.147.2.29 scan_hhdm_range()	695
7.147.2.30 vaddr_indices()	696
7.147.2.31 virt_to_phys_kernel()	696
7.147.2.32 walk_va_to_pa()	697

7.147.2.33 writeCR3()	697
7.147.3 Variable Documentation	698
7.147.3.1 __bss_end	698
7.147.3.2 __data_start	698
7.147.3.3 __kernel_end	698
7.147.3.4 __kernel_start	698
7.147.3.5 __rodata_end	698
7.147.3.6 __rodata_start	698
7.147.3.7 __text_end	699
7.147.3.8 __text_start	699
7.147.3.9 framebuffer_bytes	699
7.147.3.10 framebuffer_phys	699
7.147.3.11 g_alloc_bm_bytes	699
7.147.3.12 g_alloc_bm_phys_base	699
7.147.3.13 g_hhdm	700
7.147.3.14 g_pml4_phys	700
7.147.3.15 g_pml4_virt	700
7.147.3.16 g_usable_bm_bytes	700
7.147.3.17 g_usable_bm_phys_base	700
7.147.3.18 hhdm_request	700
7.147.3.19 Kaddress_req	701
7.147.3.20 mmap_req	701
7.148 paging_bootstrap.c	701
7.149 src/panic.c File Reference	709
7.149.1 Function Documentation	709
7.149.1.1 halt()	710
7.149.1.2 panic()	710
7.150 panic.c	712
7.151 src/pic.c File Reference	712
7.151.1 Function Documentation	713
7.151.1.1 pic_enable()	713
7.151.1.2 pic_mask_irq()	714
7.151.1.3 pic_remap_offsets()	715
7.151.1.4 pic_send_eoi()	716
7.151.1.5 pic_unmask_irq()	717
7.152 pic.c	718
7.153 src/printf.c File Reference	719
7.153.1 Macro Definition Documentation	720
7.153.1.1 FLAGS_ADAPT_EXP	721
7.153.1.2 FLAGS_CHAR	721
7.153.1.3 FLAGS_HASH	721
7.153.1.4 FLAGS_LEFT	721

7.153.1.5	FLAGS_LONG	721
7.153.1.6	FLAGS_LONG_LONG	721
7.153.1.7	FLAGS_PLUS	722
7.153.1.8	FLAGS_PRECISION	722
7.153.1.9	FLAGS_SHORT	722
7.153.1.10	FLAGS_SPACE	722
7.153.1.11	FLAGS_UPPERCASE	722
7.153.1.12	FLAGS_ZEROPAD	722
7.153.1.13	PRINTF_DEFAULT_FLOAT_PRECISION	723
7.153.1.14	PRINTF_FTOA_BUFFER_SIZE	723
7.153.1.15	PRINTF_MAX_FLOAT	723
7.153.1.16	PRINTF_NTOA_BUFFER_SIZE	723
7.153.1.17	PRINTF_SUPPORT_EXPONENTIAL	723
7.153.1.18	PRINTF_SUPPORT_FLOAT	723
7.153.1.19	PRINTF_SUPPORT_LONG_LONG	724
7.153.1.20	PRINTF_SUPPORT_PTRDIFF_T	724
7.153.2	Typedef Documentation	724
7.153.2.1	out_fct_type	724
7.153.3	Function Documentation	724
7.153.3.1	_atoi()	724
7.153.3.2	_etoa()	725
7.153.3.3	_ftoa()	727
7.153.3.4	_is_digit()	729
7.153.3.5	_ntoa_format()	730
7.153.3.6	_ntoa_long()	732
7.153.3.7	_ntoa_long_long()	734
7.153.3.8	_out_buffer()	736
7.153.3.9	_out_char()	737
7.153.3.10	_out_fct()	738
7.153.3.11	_out_null()	739
7.153.3.12	_out_rev()	740
7.153.3.13	_strlen_s()	742
7.153.3.14	_vsprintf()	743
7.153.3.15	fctprintf()	744
7.153.3.16	printf_()	745
7.153.3.17	snprintf_()	747
7.153.3.18	sprintf_()	748
7.153.3.19	vprintf_()	749
7.153.3.20	vsprintf_()	749
7.154	printf.c	750
7.155	src/proc.c File Reference	764
7.156	proc.c	764

7.157 src/Proc_utils.c File Reference	765
7.157.1 Function Documentation	765
7.157.1.1 soak()	766
7.157.2 Variable Documentation	766
7.157.2.1 soak_counter	766
7.158 Proc_utils.c	767
7.159 src/putchar.c File Reference	767
7.159.1 Macro Definition Documentation	768
7.159.1.1 LOGBUF_SIZE	768
7.159.2 Function Documentation	768
7.159.2.1 _putchar()	768
7.159.2.2 buffer_or_flush()	770
7.159.2.3 flush_serial_line()	772
7.159.2.4 ticks_to_ms()	774
7.159.2.5 u64_to_str()	775
7.159.2.6 voyager_page_tables_active()	776
7.159.3 Variable Documentation	778
7.159.3.1 logbuf	778
7.159.3.2 loglen	779
7.159.3.3 tsc_hz	779
7.160 putchar.c	779
7.161 src/registers.c File Reference	781
7.161.1 Function Documentation	782
7.161.1.1 rdmsr()	782
7.161.1.2 writeMSR()	782
7.162 registers.c	783
7.163 src/sched.c File Reference	783
7.163.1 Macro Definition Documentation	784
7.163.1.1 STACK_ALIGN	784
7.163.1.2 STACK_SIZE	784
7.163.2 Function Documentation	784
7.163.2.1 create_process()	784
7.163.2.2 init_scheduler()	785
7.163.2.3 schedule()	786
7.163.2.4 scheduler_start()	787
7.163.2.5 task_returned()	788
7.163.3 Variable Documentation	788
7.163.3.1 allow_sched	789
7.163.3.2 current	789
7.163.3.3 next_pid	789
7.163.3.4 run_queue_head	789
7.163.3.5 run_queue_tail	789

7.164 sched.c	790
7.165 src/serial.c File Reference	791
7.165.1 Function Documentation	792
7.165.1.1 serial_print()	792
7.165.1.2 serial_print_line()	793
7.165.1.3 serial_print_with_ts()	794
7.166 serial.c	794
7.167 src/shell.c File Reference	795
7.167.1 Function Documentation	795
7.167.1.1 cmd_parser()	796
7.167.1.2 halt()	797
7.167.1.3 shell_streq()	797
7.167.1.4 vsh_loop()	797
7.167.1.5 vsh_readline()	798
7.168 shell.c	799
7.169 src/stack_trace.c File Reference	801
7.169.1 Function Documentation	802
7.169.1.1 dump_hex()	802
7.169.1.2 halt()	803
7.169.1.3 print_stack_size()	804
7.169.1.4 stack_dump()	804
7.169.1.5 stack_dump_recursive()	805
7.169.1.6 stack_trace()	805
7.169.1.7 stack_trace_asm()	807
7.169.1.8 walk_stack()	807
7.169.2 Variable Documentation	808
7.169.2.1 stack_req	808
7.170 stack_trace.c	808
7.171 src/str2int.c File Reference	809
7.171.1 Function Documentation	810
7.171.1.1 str2int()	810
7.171.1.2 str2int2()	810
7.172 str2int.c	811
7.173 src/strlen.c File Reference	811
7.173.1 Function Documentation	812
7.173.1.1 strlen()	812
7.174 strlen.c	812
7.175 src/strncpy.c File Reference	813
7.175.1 Function Documentation	813
7.175.1.1 strncpy()	813
7.176 strncpy.c	813
7.177 src/strtok.c File Reference	814

7.177.1 Function Documentation	814
7.177.1.1 strtok()	814
7.178 strtok.c	815
7.179 src/terminal/framebuffer.c File Reference	815
7.179.1 Macro Definition Documentation	817
7.179.1.1 FONT_BYTES	817
7.179.2 Function Documentation	817
7.179.2.1 compare_char()	817
7.179.2.2 draw_cursor()	818
7.179.2.3 fbterm_clear()	818
7.179.2.4 fbterm_deinit()	819
7.179.2.5 fbterm_disable_cursor()	819
7.179.2.6 fbterm_double_buffer_flush()	820
7.179.2.7 fbterm_enable_cursor()	820
7.179.2.8 fbterm_full_refresh()	821
7.179.2.9 fbterm_get_cursor_pos()	821
7.179.2.10 fbterm_init()	822
7.179.2.11 fbterm_move_character()	824
7.179.2.12 fbterm_raw_putchar()	825
7.179.2.13 fbterm_restore_state()	825
7.179.2.14 fbterm_revscroll()	826
7.179.2.15 fbterm_save_state()	826
7.179.2.16 fbterm_scroll()	827
7.179.2.17 fbterm_set_cursor_pos()	827
7.179.2.18 fbterm_set_text_bg()	828
7.179.2.19 fbterm_set_text_bg_bright()	828
7.179.2.20 fbterm_set_text_bg_default()	829
7.179.2.21 fbterm_set_text_bg_rgb()	829
7.179.2.22 fbterm_set_text_fg()	830
7.179.2.23 fbterm_set_text_fg_bright()	830
7.179.2.24 fbterm_set_text_fg_default()	831
7.179.2.25 fbterm_set_text_fg_rgb()	831
7.179.2.26 fbterm_swap_palette()	832
7.179.2.27 memcpy()	832
7.179.2.28 memset()	832
7.179.2.29 plot_char()	833
7.179.2.30 plot_char_fast()	834
7.179.2.31 push_to_queue()	834
7.179.3 Variable Documentation	835
7.179.3.1 builtin_font	835
7.180 framebuffer.c	835
7.181 src/terminal/term.c File Reference	848

7.181.1 Macro Definition Documentation	849
7.181.1.1 CHARSET_DEC_SPECIAL	849
7.181.1.2 CHARSET_DEFAULT	849
7.181.2 Function Documentation	849
7.181.2.1 control_sequence_parse()	849
7.181.2.2 dec_private_parse()	850
7.181.2.3 dec_special_to_cp437()	851
7.181.2.4 escape_parse()	852
7.181.2.5 linux_private_parse()	854
7.181.2.6 mode_toggle()	855
7.181.2.7 restore_state()	856
7.181.2.8 save_state()	857
7.181.2.9 sgr()	858
7.181.2.10 term_context_reinit()	859
7.181.2.11 term_putchar()	860
7.181.2.12 term_write()	862
7.181.3 Variable Documentation	864
7.181.3.1 col256	864
7.182 term.c	865
7.183 src/time.c File Reference	877
7.183.1 Macro Definition Documentation	879
7.183.1.1 CHANNEL_ZERO	879
7.183.1.2 CMOS_ADDR	879
7.183.1.3 CMOS_DATA	879
7.183.1.4 CURRENT_YEAR	880
7.183.1.5 MODE_CMD	880
7.183.2 Function Documentation	880
7.183.2.1 config_PIT()	880
7.183.2.2 dayofweek()	880
7.183.2.3 delta_int()	881
7.183.2.4 get_RTC_register()	881
7.183.2.5 get_update_flag()	882
7.183.2.6 init_PIT()	882
7.183.2.7 pit_get_current_count()	883
7.183.2.8 pit_set_frequency()	884
7.183.2.9 pit_set_reload_value()	884
7.183.2.10 pit_sleep()	885
7.183.2.11 print_date()	885
7.183.2.12 print_load_time()	886
7.183.2.13 print_sys_time()	887
7.183.2.14 read_rtc()	887
7.183.2.15 sys_clock_handler()	888

7.183.2.16 sys_clock_handler_alt()	889
7.183.2.17 task_switch_handler()	889
7.183.2.18 time_init()	890
7.183.2.19 timer_arm()	890
7.183.2.20 timer_disarm()	891
7.183.2.21 VECTOR_TYPE()	891
7.183.3 Variable Documentation	892
7.183.3.1 century_register	892
7.183.3.2 day	892
7.183.3.3 hour	892
7.183.3.4 minute	892
7.183.3.5 month	892
7.183.3.6 month_str	893
7.183.3.7 pit_ticks	893
7.183.3.8 second	893
7.183.3.9 system_timer_fractions	893
7.183.3.10 system_timer_ms	894
7.183.3.11 task_timer_count	894
7.183.3.12 time_mono	894
7.183.3.13 time_real	894
7.183.3.14 timer_fired	894
7.183.3.15 timers_lock	895
7.183.3.16 weekday	895
7.183.3.17 year	895
7.184 time.c	895

Chapter 1

VoyagerOS64 Kernel Documentation

VoyagerOS64 is a hobby x86-64 kernel. The 0.0.5 development line focuses on replacing several overlapping memory-management experiments with one well-defined bootstrap/PMM/heap chain and on restoring a small, trust-worthy cooperative scheduler.

1.1 Current 0.0.5 architecture

The boot-time ownership chain is:

```
Limine
-> early_alloc
-> paging_bootstrap
-> neo_framealloc
-> kmalloc
-> scheduler / drivers / VSH
```

The most important invariant is that physical and virtual addresses are not interchangeable. Physical-memory management deals in physical frames. Paging creates virtual mappings. Kernel code reaches physical frames through the HHDM or another explicit mapping.

1.2 Scheduler model

Version 0.0.5 uses cooperative round-robin scheduling. Tasks voluntarily yield by calling `schedule()`. The PIT preemption gate remains disabled on purpose; `switch_to()` is an ordinary-context stack switch and is not an IRQ return path.

1.3 Deliberate 0.0.5 boundaries

The following are not complete 0.0.5 facilities: preemptive scheduling, task exit/reaping, wait semantics, Unix-style fork/exec, per-process address spaces, scheduler-integrated FPU/XSAVE state, SMP scheduling, and APIC-based interrupt routing as the primary path.

1.4 Historical implementations

Several older allocators, VMM experiments, the former tube/stream layer, and C++ allocation/locking experiments remain in-tree for archaeology and design history. GNUmakefile explicitly excludes those implementations from the active kernel link. Their presence in the source tree does not make them part of the 0.0.5 runtime architecture.

1.5 Runtime qualification

The Stage-2 memory stack has survived direct PMM round-trips, allocation boundary tests, fragmentation/split/coalesce tests, krealloc preservation, multi-megabyte heap growth, deterministic randomized torture, and repeated same-boot runs. The cooperative scheduler has also survived dynamic run-queue growth into the low twenties of runnable tasks during soak testing.

Chapter 2

Module Index

2.1 Modules

Here is a list of all modules:

Boot and Kernel Bring-up	13
Memory Management	14
Early Physical Allocator	14
Physical Memory Manager	15
Kernel Heap	15
Paging and Address Translation	16
Interrupts and Descriptor Tables	16
Scheduler and Processes	17
Device Drivers	17
PS/2 Keyboard	18
Terminal and Console	18
Voyager Shell (VSH)	19
Diagnostics and Qualification	19
Timekeeping	19
Retired / Historical Code	20

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

<code>__attribute__</code>		
	Packed 16-byte x86-64 IDT gate descriptor	21
<code>block_footer</code>		23
<code>block_header</code>		24
<code>cpu_ctx</code>		
	Saved general-purpose CPU context used by interrupt/APIC groundwork	25
<code>cpu_local</code>		
	Per-CPU state reserved for APIC/SMP development	30
<code>fbterm_char</code>		
	One rendered terminal cell and its foreground/background colours	32
<code>fbterm_context</code>		
	Framebuffer terminal backend state	33
<code>fbterm_queue_item</code>		
	Deferred framebuffer-cell update used by the renderer queue	42
<code>GDT</code>		
	Complete VoyagerOS64 <code>GDT</code> image	43
<code>GDT_Desc</code>		
	Operand layout consumed by LGDT	46
<code>GDT_Entry</code>		
	Legacy-format 8-byte segment descriptor	47
<code>GDT_Entry_16</code>		
	Descriptor storage used for the 16-bit compatibility entries	48
<code>GDT_Entry_32</code>		
	Descriptor storage used for the 32-bit compatibility entries	50
<code>isr_xframe_t</code>		
	Extended interrupt frame matching the current ISR assembly/C boundary	52
<code>kswitches</code>		
	Compile-time/runtime-style kernel behavior switches	56
<code>limine_5_level_paging_request</code>		59
<code>limine_5_level_paging_response</code>		60
<code>limine_boot_time_request</code>		61
<code>limine_boot_time_response</code>		62
<code>limine_bootloader_info_request</code>		62
<code>limine_bootloader_info_response</code>		63
<code>limine_dtb_request</code>		65

limine_dtb_response	66
limine_efi_system_table_request	66
limine_efi_system_table_response	68
limine_entry_point_request	68
limine_entry_point_response	70
limine_file	71
limine_framebuffer	74
limine_framebuffer_request	77
limine_framebuffer_response	78
limine_hhdm_request	79
limine_hhdm_response	81
limine_kernel_address_request	81
limine_kernel_address_response	82
limine_kernel_file_request	83
limine_kernel_file_response	85
limine_memmap_entry	85
limine_memmap_request	86
limine_memmap_response	88
limine_module_request	89
limine_module_response	90
limine_rsdp_request	91
limine_rsdp_response	92
limine_smbios_request	92
limine_smbios_response	94
limine_smp_request	95
limine_stack_size_request	96
limine_stack_size_response	97
limine_terminal	98
limine_terminal_request	99
limine_terminal_response	100
limine_uuid	101
madt_header	
Common two-byte header shared by variable-length MADT records	103
madt_io_apic	
MADT I/O APIC record	103
madt_iso	
MADT Interrupt Source Override record	105
madt_lapic	
MADT Processor Local APIC record	106
madt_nmi	
MADT local-APIC NMI record	107
memtest_slot	108
memtest_stats	109
out_fct_wrap_type	110
PageTableOffset	
Four-level x86-64 page-table indices for a virtual address	111
process	
Minimal 0.0.5 process control block	112
rsdp	114
rsdt	116
sdt	
Common ACPI System Description Table header	117
smartlock	119
stack_frame	
Frame-pointer chain node used by the C stack walker	120
stack_return	
Collected stack-return-address array and its populated length	121

<code>term_context</code>	Terminal parser state plus backend/client callback table	122
<code>timer</code>	Small software-timer record used by the current timer layer	132
<code>timespec</code>	Seconds/nanoseconds time representation used by kernel timers	133
<code>TSS</code>	Packed architectural x86-64 Task State Segment	134
<code>TSS_Entry</code>	64-bit <code>TSS</code> system descriptor split into architectural fields	138

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

src/cpu.c	165
src/cpuUtils.c	169
src/early_alloc.c	209
src/frame_supplier.c	218
src/gdt.c	219
src/idt.c	226
src/int2srt.c	550
src/interrupts.c	552
src/io.c	556
src/kernel.c	560
src/KernelUtils.c	576
src/limine_requests.c	583
src/lock_c.c	585
src/memcmp.c	585
src/memcpy.c	587
src/memmove.c	588
src/memset.c	590
src/neo_framealloc.c	642
src/paging.c	662
src/panic.c	709
src/pic.c	712
src/printf.c	719
src/proc.c	764
src/Proc_utils.c	765
src/putchar.c	767
src/registers.c	781
src/sched.c	783
src/serial.c	791
src/shell.c	795
src/stack_trace.c	801
src/str2int.c	809
src/strlen.c	811
src/strncpy.c	813
src/strtok.c	814
src/time.c	877

src/acpi/acpi.c	141
src/acpi/madt.c	147
src/apic/ioapic.c	149
src/apic/lapic.c	155
src/drivers/keyboard/keyboard.c	202
src/include/consts.h	251
src/include/cpu.h	
Low-level CPU context, feature, and register-state helpers	252
src/include/cpuUtils.h	
CUID feature discovery and early CPU capability helpers	270
src/include/early_alloc.h	
Bootstrap physical-page allocator used before the permanent PMM	305
src/include/errno.h	309
src/include/gdt.h	
X86-64 Global Descriptor Table and TSS descriptor interface	336
src/include/global_defs.h	344
src/include/idt.h	
X86-64 Interrupt Descriptor Table and dynamic vector interface	348
src/include/interrupts.h	
Register snapshot passed through Voyager's interrupt-dispatch path	358
src/include/io.h	
X86 port-I/O primitives	360
src/include/kernel.h	
Global kernel bring-up state shared by low-level subsystems	364
src/include/KernelUtils.h	
Kernel feature switches, memory-reporting helpers, and shared utilities	366
src/include/limine.h	388
src/include/lock.h	405
src/include/panic.h	
Fatal kernel error-reporting interface	447
src/include/pic.h	
Legacy 8259 PIC programming interface	449
src/include/printf.h	457
src/include/proc.h	468
src/include/proc_utils.h	
Small process/scheduler diagnostic tasks	470
src/include/registers.h	
Low-level control-register, stack-pointer, and MSR access helpers	472
src/include/sched.h	
Cooperative process scheduler interface	478
src/include/serial.h	
Serial-console and timestamped diagnostic output helpers	487
src/include/shell.h	
Voyager shell (VSH) public interface	493
src/include/stack_trace.h	
Stack-size reporting, frame walking, and diagnostic stack dumps	498
src/include/string.h	506
src/include/syscall.h	511
src/include/time.h	
RTC, PIT, and kernel timekeeping interfaces	525
src/include/tss.h	
X86-64 Task State Segment layout	542
src/include/vgafont.h	544
src/include/voyager_docs.h	
Doxygen landing pages and subsystem groups for VoyagerOS64	549
src/include/acpi/acpi.h	
ACPI table discovery and SDT access interface	236

src/include/acpi/madt.h	
ACPI MADT record structures and parsed interrupt-topology lists	240
src/include/apic/ioapic.h	
I/O APIC redirection helpers for experimental APIC routing	243
src/include/apic/lapic.h	
Local APIC experimental interface	247
src/include/drivers/keyboard/keyboard.h	
PS/2 keyboard IRQ and shell-facing character queue	296
src/include/drivers/keyboard/keyboard_map.h	300
src/include/lib/hashmap.h	372
src/include/lib/vector.h	380
src/include/mm/kheap.h	410
src/include/mm/kmalloc.h	
General-purpose Stage-2 kernel heap interface	411
src/include/mm/memtest.h	
Stage-2 PMM/heap qualification harness	418
src/include/paging/frame_supplier.h	
Boot-stage physical-page supplier indirection used by paging	421
src/include/paging/neo_framealloc.h	
Permanent Stage-2 physical frame allocator interface	424
src/include/paging/paging.h	
Generic x86-64 page-table helpers and address-space constants	432
src/include/paging/paging_bootstrap.h	
Construction and installation of Voyager-owned bootstrap page tables	443
src/include/terminal/framebuffer.h	
Framebuffer-backed implementation of the Voyager terminal interface	512
src/include/terminal/term.h	
Backend-independent terminal parser/context interface	517
src/mm/kmalloc.c	591
src/mm/memtest.c	615
src/paging/paging_bootstrap.c	676
src/terminal/framebuffer.c	815
src/terminal/term.c	848

Chapter 5

Module Documentation

5.1 Boot and Kernel Bring-up

Entry, bootloader handoff, subsystem ordering, and ownership transfer.

Files

- file `acpi.h`
ACPI table discovery and SDT access interface.
- file `cpu.h`
Low-level CPU context, feature, and register-state helpers.
- file `cpuUtils.h`
CPUID feature discovery and early CPU capability helpers.
- file `kernel.h`
Global kernel bring-up state shared by low-level subsystems.
- file `KernelUtils.h`
Kernel feature switches, memory-reporting helpers, and shared utilities.
- file `registers.h`
Low-level control-register, stack-pointer, and MSR access helpers.

5.1.1 Detailed Description

Entry, bootloader handoff, subsystem ordering, and ownership transfer.

5.2 Memory Management

Bootstrap allocation, physical-frame ownership, and kernel heap.

Collaboration diagram for Memory Management:

```
graph TD; MM[Memory Management];
```

Modules

- **Early Physical Allocator**
One-way bootstrap allocator used before the permanent PMM is ready.
- **Physical Memory Manager**
Ownership and allocation of 4 KiB physical frames.
- **Kernel Heap**
General-purpose kernel virtual-memory allocation after bring-up.

5.2.1 Detailed Description

Bootstrap allocation, physical-frame ownership, and kernel heap.

5.3 Early Physical Allocator

One-way bootstrap allocator used before the permanent PMM is ready.

Collaboration diagram for Early Physical Allocator:

```
graph TD; EPA[Early Physical Allocator];
```

Files

- file `early_alloc.h`
Bootstrap physical-page allocator used before the permanent PMM.

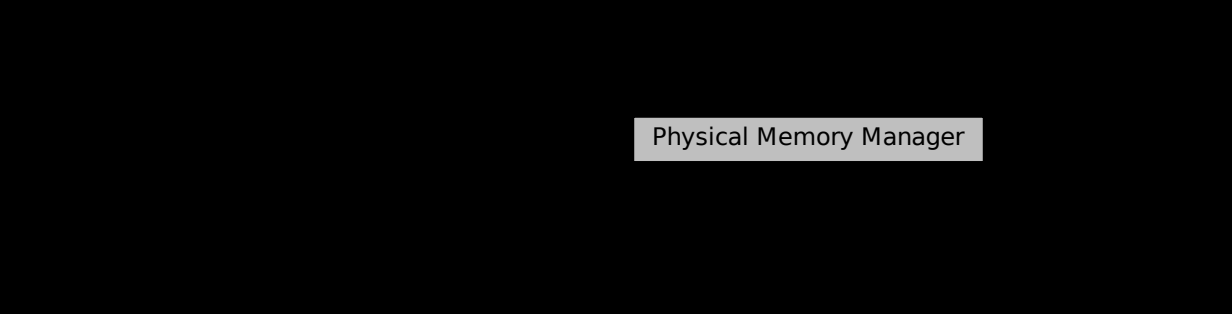
5.3.1 Detailed Description

One-way bootstrap allocator used before the permanent PMM is ready.

5.4 Physical Memory Manager

Ownership and allocation of 4 KiB physical frames.

Collaboration diagram for Physical Memory Manager:



Physical Memory Manager

Files

- file `neo_framealloc.h`
Permanent Stage-2 physical frame allocator interface.

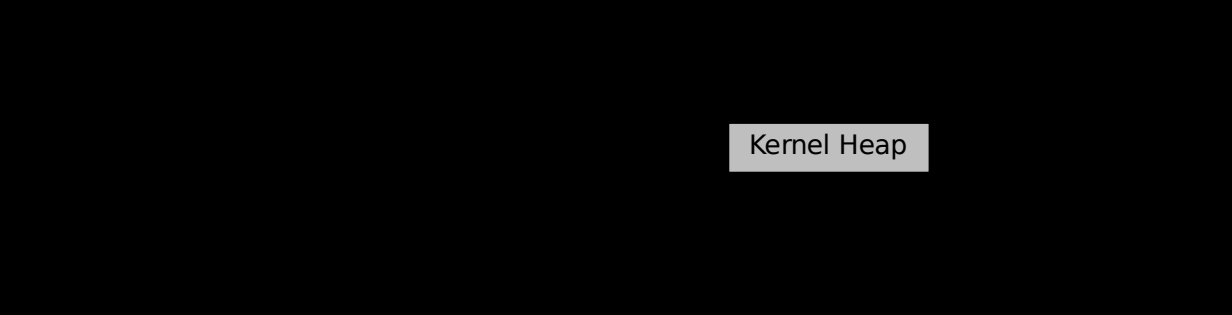
5.4.1 Detailed Description

Ownership and allocation of 4 KiB physical frames.

5.5 Kernel Heap

General-purpose kernel virtual-memory allocation after bring-up.

Collaboration diagram for Kernel Heap:



Kernel Heap

Files

- file `kmalloc.h`
General-purpose Stage-2 kernel heap interface.

5.5.1 Detailed Description

General-purpose kernel virtual-memory allocation after bring-up.

5.6 Paging and Address Translation

x86-64 page-table construction, mappings, CR3 ownership, and HHDM use.

Files

- file `frame_supplier.h`
Boot-stage physical-page supplier indirection used by paging.
- file `paging.h`
Generic x86-64 page-table helpers and address-space constants.
- file `paging_bootstrap.h`
Construction and installation of Voyager-owned bootstrap page tables.

5.6.1 Detailed Description

x86-64 page-table construction, mappings, CR3 ownership, and HHDM use.

5.7 Interrupts and Descriptor Tables

`GDT`, `IDT`, `PIC/PIT`, interrupt dispatch, and related low-level state.

Files

- file `madt.h`
ACPI MADT record structures and parsed interrupt-topology lists.
- file `ioapic.h`
I/O APIC redirection helpers for experimental APIC routing.
- file `lapic.h`
Local APIC experimental interface.
- file `gdt.h`
x86-64 Global Descriptor Table and `TSS` descriptor interface.
- file `idt.h`
x86-64 Interrupt Descriptor Table and dynamic vector interface.
- file `interrupts.h`
Register snapshot passed through Voyager's interrupt-dispatch path.
- file `pic.h`
Legacy 8259 PIC programming interface.
- file `tss.h`
x86-64 Task State Segment layout.

5.7.1 Detailed Description

GDT, IDT, PIC/PIT, interrupt dispatch, and related low-level state.

5.8 Scheduler and Processes

Cooperative 0.0.5 task creation and round-robin context switching.

Files

- file `sched.h`
Cooperative process scheduler interface.

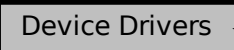
5.8.1 Detailed Description

Cooperative 0.0.5 task creation and round-robin context switching.

5.9 Device Drivers

Hardware drivers exposed to the rest of the kernel.

Collaboration diagram for Device Drivers:



Device Drivers

Modules

- `PS/2 Keyboard`
IRQ-driven keyboard input and the shell-facing character FIFO.

Files

- file `io.h`
x86 port-I/O primitives.

5.9.1 Detailed Description

Hardware drivers exposed to the rest of the kernel.

5.10 PS/2 Keyboard

IRQ-driven keyboard input and the shell-facing character FIFO.

Collaboration diagram for PS/2 Keyboard:



```
graph TD
    subgraph PS2_Keyboard [PS/2 Keyboard]
        direction TB
        subgraph Files
            keyboard_h[keyboard.h]
        end
    end
```

Files

- file `keyboard.h`
PS/2 keyboard IRQ and shell-facing character queue.

5.10.1 Detailed Description

IRQ-driven keyboard input and the shell-facing character FIFO.

5.11 Terminal and Console

Boot console, framebuffer terminal, serial diagnostics, and output ownership.

Files

- file `serial.h`
Serial-console and timestamped diagnostic output helpers.
- file `framebuffer.h`
Framebuffer-backed implementation of the Voyager terminal interface.
- file `term.h`
Backend-independent terminal parser/context interface.

5.11.1 Detailed Description

Boot console, framebuffer terminal, serial diagnostics, and output ownership.

5.12 Voyager Shell (VSH)

Cooperative interactive shell and built-in kernel commands.

Files

- file `shell.h`
Voyager shell (VSH) public interface.

5.12.1 Detailed Description

Cooperative interactive shell and built-in kernel commands.

5.13 Diagnostics and Qualification

Built-in validation, stress tests, and low-level debug instrumentation.

Files

- file `memtest.h`
Stage-2 PMM/heap qualification harness.
- file `panic.h`
Fatal kernel error-reporting interface.
- file `proc_utils.h`
Small process/scheduler diagnostic tasks.
- file `stack_trace.h`
Stack-size reporting, frame walking, and diagnostic stack dumps.

5.13.1 Detailed Description

Built-in validation, stress tests, and low-level debug instrumentation.

5.14 Timekeeping

RTC, PIT-derived timing, date formatting, and system time services.

Files

- file `time.h`
RTC, PIT, and kernel timekeeping interfaces.

5.14.1 Detailed Description

RTC, PIT-derived timing, date formatting, and system time services.

5.15 Retired / Historical Code

Preserved experiments that are not linked into the active 0.0.5 kernel.

Preserved experiments that are not linked into the active 0.0.5 kernel.

The build explicitly retires older allocator/VMM/stream implementations. When reading the source tree, prefer the active Stage-2 interfaces documented under [Memory Management](#) and [Paging and Address Translation](#) rather than assuming every historical file participates in the current runtime.

Chapter 6

Data Structure Documentation

6.1 `__attribute__` Struct Reference

Packed 16-byte x86-64 IDT gate descriptor.

```
#include <idt.h>
```

Data Fields

- `uint16_t base_low`
- `uint16_t cs`
- `uint8_t ist`
- `uint8_t attributes`
- `uint16_t base_mid`
- `uint32_t base_high`
- `uint32_t rsv0`
- `uint16_t limit`
- `uint64_t base`

6.1.1 Detailed Description

Packed 16-byte x86-64 IDT gate descriptor.

Operand format consumed by LIDT.

Definition at line 47 of file `idt.h`.

6.1.2 Field Documentation

6.1.2.1 attributes

```
uint8_t __attribute__::attributes
```

Definition at line 52 of file [idt.h](#).

6.1.2.2 base

```
uint64_t __attribute__::base
```

Definition at line 62 of file [idt.h](#).

6.1.2.3 base_high

```
uint32_t __attribute__::base_high
```

Definition at line 54 of file [idt.h](#).

6.1.2.4 base_low

```
uint16_t __attribute__::base_low
```

Definition at line 49 of file [idt.h](#).

6.1.2.5 base_mid

```
uint16_t __attribute__::base_mid
```

Definition at line 53 of file [idt.h](#).

6.1.2.6 cs

```
uint16_t __attribute__::cs
```

Definition at line 50 of file [idt.h](#).

6.1.2.7 ist

```
uint8_t __attribute__((ist))
```

Definition at line 51 of file [idt.h](#).

6.1.2.8 limit

```
uint16_t __attribute__((limit))
```

Definition at line 61 of file [idt.h](#).

6.1.2.9 rsv0

```
uint32_t __attribute__((rsv0))
```

Definition at line 55 of file [idt.h](#).

The documentation for this struct was generated from the following file:

- [src/include/idt.h](#)

6.2 block_footer Struct Reference

Data Fields

- [size_t size_and_flags](#)

6.2.1 Detailed Description

Definition at line 48 of file [kmallocc.c](#).

6.2.2 Field Documentation

6.2.2.1 size_and_flags

```
size_t block_footer::size_and_flags
```

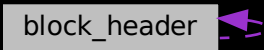
Definition at line 50 of file [kmalloc.c](#).

The documentation for this struct was generated from the following file:

- [src/mm/kmalloc.c](#)

6.3 block_header Struct Reference

Collaboration diagram for block_header:



A diagram showing a box labeled 'block_header' with a purple arrow pointing to it from the right.

Data Fields

- `size_t` [size_and_flags](#)
- `struct block_header *` [prev_free](#)
- `struct block_header *` [next_free](#)

6.3.1 Detailed Description

Definition at line 39 of file [kmalloc.c](#).

6.3.2 Field Documentation

6.3.2.1 next_free

```
struct block_header* block_header::next_free
```

Definition at line 44 of file [kmalloc.c](#).

6.3.2.2 prev_free

```
struct block_header* block_header::prev_free
```

Definition at line 43 of file [kmalloc.c](#).

6.3.2.3 size_and_flags

```
size_t block_header::size_and_flags
```

Definition at line 41 of file [kmalloc.c](#).

The documentation for this struct was generated from the following file:

- [src/mm/kmalloc.c](#)

6.4 cpu_ctx Struct Reference

Saved general-purpose CPU context used by interrupt/APIC groundwork.

```
#include <cpu.h>
```

Data Fields

- uint64_t [ds](#)
- uint64_t [es](#)
- uint64_t [rax](#)
- uint64_t [rbx](#)
- uint64_t [rcx](#)
- uint64_t [rdx](#)
- uint64_t [rsi](#)
- uint64_t [rdi](#)
- uint64_t [rbp](#)
- uint64_t [r8](#)
- uint64_t [r9](#)
- uint64_t [r10](#)
- uint64_t [r11](#)
- uint64_t [r12](#)
- uint64_t [r13](#)
- uint64_t [r14](#)
- uint64_t [r15](#)
- uint64_t [err](#)
- uint64_t [rip](#)
- uint64_t [cs](#)
- uint64_t [rflags](#)
- uint64_t [rsp](#)
- uint64_t [ss](#)

6.4.1 Detailed Description

Saved general-purpose CPU context used by interrupt/APIC groundwork.

Definition at line 22 of file `cpu.h`.

6.4.2 Field Documentation

6.4.2.1 cs

```
uint64_t cpu_ctx::cs
```

Definition at line 43 of file `cpu.h`.

6.4.2.2 ds

```
uint64_t cpu_ctx::ds
```

Definition at line 24 of file `cpu.h`.

6.4.2.3 err

```
uint64_t cpu_ctx::err
```

Definition at line 41 of file `cpu.h`.

6.4.2.4 es

```
uint64_t cpu_ctx::es
```

Definition at line 25 of file `cpu.h`.

6.4.2.5 r10

```
uint64_t cpu_ctx::r10
```

Definition at line 35 of file `cpu.h`.

6.4.2.6 `r11`

```
uint64_t cpu_ctx::r11
```

Definition at line 36 of file `cpu.h`.

6.4.2.7 `r12`

```
uint64_t cpu_ctx::r12
```

Definition at line 37 of file `cpu.h`.

6.4.2.8 `r13`

```
uint64_t cpu_ctx::r13
```

Definition at line 38 of file `cpu.h`.

6.4.2.9 `r14`

```
uint64_t cpu_ctx::r14
```

Definition at line 39 of file `cpu.h`.

6.4.2.10 `r15`

```
uint64_t cpu_ctx::r15
```

Definition at line 40 of file `cpu.h`.

6.4.2.11 `r8`

```
uint64_t cpu_ctx::r8
```

Definition at line 33 of file `cpu.h`.

6.4.2.12 r9

```
uint64_t cpu_ctx::r9
```

Definition at line 34 of file [cpu.h](#).

6.4.2.13 rax

```
uint64_t cpu_ctx::rax
```

Definition at line 26 of file [cpu.h](#).

6.4.2.14 rbp

```
uint64_t cpu_ctx::rbp
```

Definition at line 32 of file [cpu.h](#).

6.4.2.15 rbx

```
uint64_t cpu_ctx::rbx
```

Definition at line 27 of file [cpu.h](#).

6.4.2.16 rcx

```
uint64_t cpu_ctx::rcx
```

Definition at line 28 of file [cpu.h](#).

6.4.2.17 rdi

```
uint64_t cpu_ctx::rdi
```

Definition at line 31 of file [cpu.h](#).

6.4.2.18 `rdx`

```
uint64_t cpu_ctx::rdx
```

Definition at line 29 of file `cpu.h`.

6.4.2.19 `rflags`

```
uint64_t cpu_ctx::rflags
```

Definition at line 44 of file `cpu.h`.

6.4.2.20 `rip`

```
uint64_t cpu_ctx::rip
```

Definition at line 42 of file `cpu.h`.

6.4.2.21 `rsi`

```
uint64_t cpu_ctx::rsi
```

Definition at line 30 of file `cpu.h`.

6.4.2.22 `rsp`

```
uint64_t cpu_ctx::rsp
```

Definition at line 45 of file `cpu.h`.

6.4.2.23 `ss`

```
uint64_t cpu_ctx::ss
```

Definition at line 46 of file `cpu.h`.

The documentation for this struct was generated from the following file:

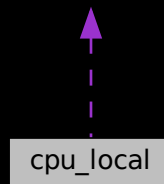
- `src/include/cpu.h`

6.5 cpu_local Struct Reference

Per-CPU state reserved for APIC/SMP development.

```
#include <cpu.h>
```

Collaboration diagram for cpu_local:



Data Fields

- int `cpu_number`
- bool `bsp`
- bool `active`
- int `last_run_queue_index`
- uint32_t `lapic_id`
- uint32_t `lapic_freq`
- struct TSS `tss`
- struct thread * `idle_thread`
- void(* `timer_function`)(int, struct `cpu_ctx` *)

6.5.1 Detailed Description

Per-CPU state reserved for APIC/SMP development.

Definition at line 50 of file `cpu.h`.

6.5.2 Field Documentation

6.5.2.1 active

```
bool cpu_local::active
```

Definition at line 54 of file `cpu.h`.

6.5.2.2 `bsp`

```
bool cpu_local::bsp
```

Definition at line 53 of file `cpu.h`.

6.5.2.3 `cpu_number`

```
int cpu_local::cpu_number
```

Definition at line 52 of file `cpu.h`.

6.5.2.4 `idle_thread`

```
struct thread* cpu_local::idle_thread
```

Definition at line 59 of file `cpu.h`.

6.5.2.5 `lapic_freq`

```
uint32_t cpu_local::lapic_freq
```

Definition at line 57 of file `cpu.h`.

6.5.2.6 `lapic_id`

```
uint32_t cpu_local::lapic_id
```

Definition at line 56 of file `cpu.h`.

6.5.2.7 `last_run_queue_index`

```
int cpu_local::last_run_queue_index
```

Definition at line 55 of file `cpu.h`.

6.5.2.8 timer_function

```
void(* cpu_local::timer_function) (int, struct cpu_ctx *)
```

Definition at line 60 of file `cpu.h`.

6.5.2.9 tss

```
struct TSS cpu_local::tss
```

Definition at line 58 of file `cpu.h`.

The documentation for this struct was generated from the following file:

- `src/include/cpu.h`

6.6 fbterm_char Struct Reference

One rendered terminal cell and its foreground/background colours.

```
#include <framebuffer.h>
```

Data Fields

- `uint32_t c`
- `uint32_t fg`
- `uint32_t bg`

6.6.1 Detailed Description

One rendered terminal cell and its foreground/background colours.

Definition at line 19 of file `framebuffer.h`.

6.6.2 Field Documentation

6.6.2.1 bg

```
uint32_t fbterm_char::bg
```

Definition at line 23 of file `framebuffer.h`.

6.6.2.2 c

```
uint32_t fbterm_char::c
```

Definition at line 21 of file [framebuffer.h](#).

6.6.2.3 fg

```
uint32_t fbterm_char::fg
```

Definition at line 22 of file [framebuffer.h](#).

The documentation for this struct was generated from the following file:

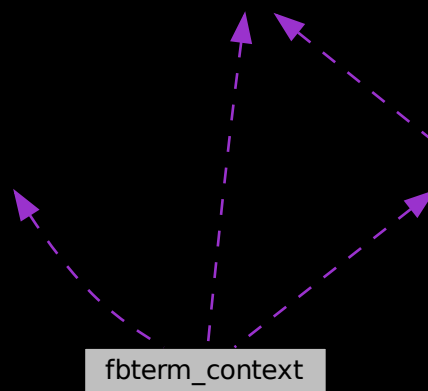
- [src/include/terminal/framebuffer.h](#)

6.7 fbterm_context Struct Reference

Framebuffer terminal backend state.

```
#include <framebuffer.h>
```

Collaboration diagram for fbterm_context:



Data Fields

- struct `term_context` `term`
- size_t `font_width`
- size_t `font_height`
- size_t `glyph_width`
- size_t `glyph_height`
- size_t `font_scale_x`
- size_t `font_scale_y`
- size_t `offset_x`
- size_t `offset_y`
- volatile uint32_t * `framebuffer`
- size_t `pitch`
- size_t `width`
- size_t `height`
- size_t `bpp`
- size_t `font_bits_size`
- uint8_t * `font_bits`
- size_t `font_bool_size`
- bool * `font_bool`
- uint32_t `ansi_colours` [8]
- uint32_t `ansi_bright_colours` [8]
- uint32_t `default_fg`
- uint32_t `default_bg`
- size_t `canvas_size`
- uint32_t * `canvas`
- size_t `grid_size`
- size_t `queue_size`
- size_t `map_size`
- struct `fbterm_char` * `grid`
- struct `fbterm_queue_item` * `queue`
- size_t `queue_i`
- struct `fbterm_queue_item` ** `map`
- uint32_t `text_fg`
- uint32_t `text_bg`
- bool `cursor_status`
- size_t `cursor_x`
- size_t `cursor_y`
- uint32_t `saved_state_text_fg`
- uint32_t `saved_state_text_bg`
- size_t `saved_state_cursor_x`
- size_t `saved_state_cursor_y`
- size_t `old_cursor_x`
- size_t `old_cursor_y`

6.7.1 Detailed Description

Framebuffer terminal backend state.

The generic `term_context` is embedded first so callers can use the returned object through the backend-independent terminal API. The remaining fields hold font geometry, framebuffer/canvas storage, cell state, queued updates, colours, and cursor state.

Definition at line 41 of file `framebuffer.h`.

6.7.2 Field Documentation

6.7.2.1 ansi_bright_colours

```
uint32_t fbterm_context::ansi_bright_colours[8]
```

Definition at line 67 of file [framebuffer.h](#).

6.7.2.2 ansi_colours

```
uint32_t fbterm_context::ansi_colours[8]
```

Definition at line 66 of file [framebuffer.h](#).

6.7.2.3 bpp

```
size_t fbterm_context::bpp
```

Definition at line 59 of file [framebuffer.h](#).

6.7.2.4 canvas

```
uint32_t* fbterm_context::canvas
```

Definition at line 71 of file [framebuffer.h](#).

6.7.2.5 canvas_size

```
size_t fbterm_context::canvas_size
```

Definition at line 70 of file [framebuffer.h](#).

6.7.2.6 cursor_status

`bool fbterm_context::cursor_status`

Definition at line 86 of file [framebuffer.h](#).

6.7.2.7 cursor_x

`size_t fbterm_context::cursor_x`

Definition at line 87 of file [framebuffer.h](#).

6.7.2.8 cursor_y

`size_t fbterm_context::cursor_y`

Definition at line 88 of file [framebuffer.h](#).

6.7.2.9 default_bg

`uint32_t fbterm_context::default_bg`

Definition at line 68 of file [framebuffer.h](#).

6.7.2.10 default_fg

`uint32_t fbterm_context::default_fg`

Definition at line 68 of file [framebuffer.h](#).

6.7.2.11 font_bits

`uint8_t* fbterm_context::font_bits`

Definition at line 62 of file [framebuffer.h](#).

6.7.2.12 font_bits_size

```
size_t fbterm_context::font_bits_size
```

Definition at line 61 of file [framebuffer.h](#).

6.7.2.13 font_bool

```
bool* fbterm_context::font_bool
```

Definition at line 64 of file [framebuffer.h](#).

6.7.2.14 font_bool_size

```
size_t fbterm_context::font_bool_size
```

Definition at line 63 of file [framebuffer.h](#).

6.7.2.15 font_height

```
size_t fbterm_context::font_height
```

Definition at line 46 of file [framebuffer.h](#).

6.7.2.16 font_scale_x

```
size_t fbterm_context::font_scale_x
```

Definition at line 50 of file [framebuffer.h](#).

6.7.2.17 font_scale_y

```
size_t fbterm_context::font_scale_y
```

Definition at line 51 of file [framebuffer.h](#).

6.7.2.18 font_width

```
size_t fbterm_context::font_width
```

Definition at line 45 of file [framebuffer.h](#).

6.7.2.19 framebuffer

```
volatile uint32_t* fbterm_context::framebuffer
```

Definition at line 55 of file [framebuffer.h](#).

6.7.2.20 glyph_height

```
size_t fbterm_context::glyph_height
```

Definition at line 48 of file [framebuffer.h](#).

6.7.2.21 glyph_width

```
size_t fbterm_context::glyph_width
```

Definition at line 47 of file [framebuffer.h](#).

6.7.2.22 grid

```
struct fbterm_char* fbterm_context::grid
```

Definition at line 77 of file [framebuffer.h](#).

6.7.2.23 grid_size

```
size_t fbterm_context::grid_size
```

Definition at line 73 of file [framebuffer.h](#).

6.7.2.24 height

```
size_t fbterm_context::height
```

Definition at line 58 of file [framebuffer.h](#).

6.7.2.25 map

```
struct fbterm_queue_item** fbterm_context::map
```

Definition at line 82 of file [framebuffer.h](#).

6.7.2.26 map_size

```
size_t fbterm_context::map_size
```

Definition at line 75 of file [framebuffer.h](#).

6.7.2.27 offset_x

```
size_t fbterm_context::offset_x
```

Definition at line 53 of file [framebuffer.h](#).

6.7.2.28 offset_y

```
size_t fbterm_context::offset_y
```

Definition at line 53 of file [framebuffer.h](#).

6.7.2.29 old_cursor_x

```
size_t fbterm_context::old_cursor_x
```

Definition at line 95 of file [framebuffer.h](#).

6.7.2.30 old_cursor_y

```
size_t fbterm_context::old_cursor_y
```

Definition at line 96 of file [framebuffer.h](#).

6.7.2.31 pitch

```
size_t fbterm_context::pitch
```

Definition at line 56 of file [framebuffer.h](#).

6.7.2.32 queue

```
struct fbterm_queue_item* fbterm_context::queue
```

Definition at line 79 of file [framebuffer.h](#).

6.7.2.33 queue_i

```
size_t fbterm_context::queue_i
```

Definition at line 80 of file [framebuffer.h](#).

6.7.2.34 queue_size

```
size_t fbterm_context::queue_size
```

Definition at line 74 of file [framebuffer.h](#).

6.7.2.35 saved_state_cursor_x

```
size_t fbterm_context::saved_state_cursor_x
```

Definition at line 92 of file [framebuffer.h](#).

6.7.2.36 saved_state_cursor_y

size_t fbterm_context::saved_state_cursor_y

Definition at line 93 of file [framebuffer.h](#).

6.7.2.37 saved_state_text_bg

uint32_t fbterm_context::saved_state_text_bg

Definition at line 91 of file [framebuffer.h](#).

6.7.2.38 saved_state_text_fg

uint32_t fbterm_context::saved_state_text_fg

Definition at line 90 of file [framebuffer.h](#).

6.7.2.39 term

struct [term_context](#) fbterm_context::term

Definition at line 43 of file [framebuffer.h](#).

6.7.2.40 text_bg

uint32_t fbterm_context::text_bg

Definition at line 85 of file [framebuffer.h](#).

6.7.2.41 text_fg

uint32_t fbterm_context::text_fg

Definition at line 84 of file [framebuffer.h](#).

6.7.2.42 width

```
size_t fbterm_context::width
```

Definition at line 57 of file `framebuffer.h`.

The documentation for this struct was generated from the following file:

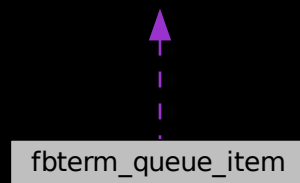
- `src/include/terminal/framebuffer.h`

6.8 fbterm_queue_item Struct Reference

Deferred framebuffer-cell update used by the renderer queue.

```
#include <framebuffer.h>
```

Collaboration diagram for fbterm_queue_item:



Data Fields

- `size_t x`
- `size_t y`
- `struct fbterm_char c`

6.8.1 Detailed Description

Deferred framebuffer-cell update used by the renderer queue.

Definition at line 27 of file `framebuffer.h`.

6.8.2 Field Documentation

6.8.2.1 c

```
struct fbterm_char fbterm_queue_item::c
```

Definition at line 30 of file `framebuffer.h`.

6.8.2.2 x

```
size_t fbterm_queue_item::x
```

Definition at line 29 of file `framebuffer.h`.

6.8.2.3 y

```
size_t fbterm_queue_item::y
```

Definition at line 29 of file `framebuffer.h`.

The documentation for this struct was generated from the following file:

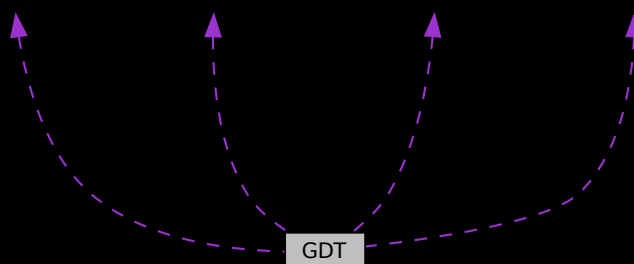
- `src/include/terminal/framebuffer.h`

6.9 GDT Struct Reference

Complete VoyagerOS64 GDT image.

```
#include <gdt.h>
```

Collaboration diagram for GDT:



Data Fields

- struct `GDT_Entry` `null`
- struct `GDT_Entry_16` `seg_16_code`
- struct `GDT_Entry_16` `seg_16_data`
- struct `GDT_Entry_32` `seg_32_code`
- struct `GDT_Entry_32` `seg_32_data`
- struct `GDT_Entry` `kernelCS`
- struct `GDT_Entry` `kernelData`
- struct `GDT_Entry` `userNull`
- struct `GDT_Entry` `userData`
- struct `GDT_Entry` `userCode`
- struct `TSS_Entry` `tss`

6.9.1 Detailed Description

Complete VoyagerOS64 `GDT` image.

Field order is part of the selector ABI above: changing the order changes descriptor offsets and therefore requires updating selector constants and low-level reload code together.

Definition at line 102 of file `gdt.h`.

6.9.2 Field Documentation

6.9.2.1 `kernelCS`

```
struct GDT_Entry GDT::kernelCS
```

Definition at line 109 of file `gdt.h`.

6.9.2.2 `kernelData`

```
struct GDT_Entry GDT::kernelData
```

Definition at line 110 of file `gdt.h`.

6.9.2.3 `null`

```
struct GDT_Entry GDT::null
```

Definition at line 104 of file `gdt.h`.

6.9.2.4 seg_16_code

```
struct GDT_Entry_16 GDT::seg_16_code
```

Definition at line 105 of file `gdt.h`.

6.9.2.5 seg_16_data

```
struct GDT_Entry_16 GDT::seg_16_data
```

Definition at line 106 of file `gdt.h`.

6.9.2.6 seg_32_code

```
struct GDT_Entry_32 GDT::seg_32_code
```

Definition at line 107 of file `gdt.h`.

6.9.2.7 seg_32_data

```
struct GDT_Entry_32 GDT::seg_32_data
```

Definition at line 108 of file `gdt.h`.

6.9.2.8 tss

```
struct TSS_Entry GDT::tss
```

Definition at line 114 of file `gdt.h`.

6.9.2.9 userCode

```
struct GDT_Entry GDT::userCode
```

Definition at line 113 of file `gdt.h`.

6.9.2.10 userData

```
struct GDT_Entry GDT::userData
```

Definition at line 112 of file `gdt.h`.

6.9.2.11 userNull

```
struct GDT_Entry GDT::userNull
```

Definition at line 111 of file `gdt.h`.

The documentation for this struct was generated from the following file:

- `src/include/gdt.h`

6.10 GDT_Desc Struct Reference

Operand layout consumed by LGDT.

```
#include <gdt.h>
```

Data Fields

- `uint16_t size`
- `uint64_t offset`

6.10.1 Detailed Description

Operand layout consumed by LGDT.

Definition at line 43 of file `gdt.h`.

6.10.2 Field Documentation

6.10.2.1 offset

```
uint64_t GDT_Desc::offset
```

Definition at line 46 of file `gdt.h`.

6.10.2.2 size

```
uint16_t GDT_Desc::size
```

Definition at line 45 of file [gdt.h](#).

The documentation for this struct was generated from the following file:

- [src/include/gdt.h](#)

6.11 GDT_Entry Struct Reference

Legacy-format 8-byte segment descriptor.

```
#include <gdt.h>
```

Data Fields

- [uint16_t limit_low](#)
- [uint16_t base_low](#)
- [uint8_t base_middle](#)
- [uint8_t access_flag](#)
- [uint8_t limit_flags](#)
- [uint8_t base_high](#)

6.11.1 Detailed Description

Legacy-format 8-byte segment descriptor.

Definition at line 50 of file [gdt.h](#).

6.11.2 Field Documentation

6.11.2.1 access_flag

```
uint8_t GDT_Entry::access_flag
```

Definition at line 55 of file [gdt.h](#).

6.11.2.2 base_high

```
uint8_t GDT_Entry::base_high
```

Definition at line 57 of file [gdt.h](#).

6.11.2.3 base_low

```
uint16_t GDT_Entry::base_low
```

Definition at line 53 of file [gdt.h](#).

6.11.2.4 base_middle

```
uint8_t GDT_Entry::base_middle
```

Definition at line 54 of file [gdt.h](#).

6.11.2.5 limit_flags

```
uint8_t GDT_Entry::limit_flags
```

Definition at line 56 of file [gdt.h](#).

6.11.2.6 limit_low

```
uint16_t GDT_Entry::limit_low
```

Definition at line 52 of file [gdt.h](#).

The documentation for this struct was generated from the following file:

- [src/include/gdt.h](#)

6.12 GDT_Entry_16 Struct Reference

Descriptor storage used for the 16-bit compatibility entries.

```
#include <gdt.h>
```

Data Fields

- uint16_t `limit_low`
- uint16_t `base_low`
- uint8_t `base_middle`
- uint8_t `access_flag`
- uint8_t `limit_flags`
- uint8_t `base_high`

6.12.1 Detailed Description

Descriptor storage used for the 16-bit compatibility entries.

Definition at line 61 of file `gdt.h`.

6.12.2 Field Documentation

6.12.2.1 `access_flag`

```
uint8_t GDT_Entry_16::access_flag
```

Definition at line 66 of file `gdt.h`.

6.12.2.2 `base_high`

```
uint8_t GDT_Entry_16::base_high
```

Definition at line 68 of file `gdt.h`.

6.12.2.3 `base_low`

```
uint16_t GDT_Entry_16::base_low
```

Definition at line 64 of file `gdt.h`.

6.12.2.4 base_middle

```
uint8_t GDT_Entry_16::base_middle
```

Definition at line 65 of file [gdt.h](#).

6.12.2.5 limit_flags

```
uint8_t GDT_Entry_16::limit_flags
```

Definition at line 67 of file [gdt.h](#).

6.12.2.6 limit_low

```
uint16_t GDT_Entry_16::limit_low
```

Definition at line 63 of file [gdt.h](#).

The documentation for this struct was generated from the following file:

- [src/include/gdt.h](#)

6.13 GDT_Entry_32 Struct Reference

Descriptor storage used for the 32-bit compatibility entries.

```
#include <gdt.h>
```

Data Fields

- [uint16_t limit_low](#)
- [uint16_t base_low](#)
- [uint8_t base_middle](#)
- [uint8_t access_flag](#)
- [uint8_t limit_flags](#)
- [uint8_t base_high](#)

6.13.1 Detailed Description

Descriptor storage used for the 32-bit compatibility entries.

Definition at line 72 of file [gdt.h](#).

6.13.2 Field Documentation

6.13.2.1 access_flag

```
uint8_t GDT_Entry_32::access_flag
```

Definition at line 77 of file [gdt.h](#).

6.13.2.2 base_high

```
uint8_t GDT_Entry_32::base_high
```

Definition at line 79 of file [gdt.h](#).

6.13.2.3 base_low

```
uint16_t GDT_Entry_32::base_low
```

Definition at line 75 of file [gdt.h](#).

6.13.2.4 base_middle

```
uint8_t GDT_Entry_32::base_middle
```

Definition at line 76 of file [gdt.h](#).

6.13.2.5 limit_flags

```
uint8_t GDT_Entry_32::limit_flags
```

Definition at line 78 of file [gdt.h](#).

6.13.2.6 limit_low

```
uint16_t GDT_Entry_32::limit_low
```

Definition at line 74 of file [gdt.h](#).

The documentation for this struct was generated from the following file:

- [src/include/gdt.h](#)

6.14 isr_xframe_t Struct Reference

Extended interrupt frame matching the current ISR assembly/C boundary.

```
#include <interrupts.h>
```

Data Fields

- struct {
 - uint64_t [cr4](#)
 - uint64_t [cr3](#)
 - uint64_t [cr2](#)
 - uint64_t [cr0](#) } [control_registers](#)
- struct {
 - uint64_t [rdi](#)
 - uint64_t [rsi](#)
 - uint64_t [rdx](#)
 - uint64_t [rcx](#)
 - uint64_t [rbx](#)
 - uint64_t [rax](#) } [general_registers](#)
- struct {
 - uint64_t [rbp](#)
 - uint64_t [vector](#)
 - uint64_t [error_code](#)
 - uint64_t [rip](#)
 - uint64_t [cs](#)
 - uint64_t [rflags](#)
 - uint64_t [rsp](#)
 - uint64_t [dss](#) } [base_frame](#)

6.14.1 Detailed Description

Extended interrupt frame matching the current ISR assembly/C boundary.

The layout is an ABI between low-level interrupt stubs and C handlers. It contains captured control registers, selected general-purpose registers, and the architectural interrupt-return frame. Do not reorder fields without updating the assembly that constructs/consumes this object.

Note

This interrupt frame is distinct from the cooperative scheduler's saved context. VoyagerOS64 0.0.5 does not switch tasks by abandoning this frame inside a hardware IRQ.

Definition at line 22 of file `interrupts.h`.

6.14.2 Field Documentation

6.14.2.1

```
struct { ... } isr_xframe_t::base_frame
```

6.14.2.2

```
struct { ... } isr_xframe_t::control_registers
```

6.14.2.3 `cr0`

```
uint64_t isr_xframe_t::cr0
```

Definition at line 29 of file `interrupts.h`.

6.14.2.4 `cr2`

```
uint64_t isr_xframe_t::cr2
```

Definition at line 28 of file `interrupts.h`.

6.14.2.5 cr3

```
uint64_t isr_xframe_t::cr3
```

Definition at line 27 of file [interrupts.h](#).

6.14.2.6 cr4

```
uint64_t isr_xframe_t::cr4
```

Definition at line 26 of file [interrupts.h](#).

6.14.2.7 cs

```
uint64_t isr_xframe_t::cs
```

Definition at line 48 of file [interrupts.h](#).

6.14.2.8 dss

```
uint64_t isr_xframe_t::dss
```

Definition at line 51 of file [interrupts.h](#).

6.14.2.9 error_code

```
uint64_t isr_xframe_t::error_code
```

Definition at line 46 of file [interrupts.h](#).

6.14.2.10

```
struct { ... } isr_xframe_t::general_registers
```

6.14.2.11 `rax`

```
uint64_t isr_xframe_t::rax
```

Definition at line 39 of file `interrupts.h`.

6.14.2.12 `rbp`

```
uint64_t isr_xframe_t::rbp
```

Definition at line 44 of file `interrupts.h`.

6.14.2.13 `rbx`

```
uint64_t isr_xframe_t::rbx
```

Definition at line 38 of file `interrupts.h`.

6.14.2.14 `rcx`

```
uint64_t isr_xframe_t::rcx
```

Definition at line 37 of file `interrupts.h`.

6.14.2.15 `rdi`

```
uint64_t isr_xframe_t::rdi
```

Definition at line 34 of file `interrupts.h`.

6.14.2.16 `rdx`

```
uint64_t isr_xframe_t::rdx
```

Definition at line 36 of file `interrupts.h`.

6.14.2.17 rflags

```
uint64_t isr_xframe_t::rflags
```

Definition at line 49 of file [interrupts.h](#).

6.14.2.18 rip

```
uint64_t isr_xframe_t::rip
```

Definition at line 47 of file [interrupts.h](#).

6.14.2.19 rsi

```
uint64_t isr_xframe_t::rsi
```

Definition at line 35 of file [interrupts.h](#).

6.14.2.20 rsp

```
uint64_t isr_xframe_t::rsp
```

Definition at line 50 of file [interrupts.h](#).

6.14.2.21 vector

```
uint64_t isr_xframe_t::vector
```

Definition at line 45 of file [interrupts.h](#).

The documentation for this struct was generated from the following file:

- [src/include/interrupts.h](#)

6.15 kswitches Struct Reference

Compile-time/runtime-style kernel behavior switches.

```
#include <KernelUtils.h>
```

Data Fields

- `uint8_t stack_trace_size`
- `uint8_t stack_trace_on_fault`
- `uint8_t acpi_support`
- `uint8_t sched_debug`
- `uint8_t addr_debug`
- `uint8_t hw_rng_support`
- `uint8_t mem_readout_unit`
- `uint8_t liballoc_debug`
- `uint8_t fpu_allowed`
- `uint8_t timestamp`

6.15.1 Detailed Description

Compile-time/runtime-style kernel behavior switches.

The current tree exposes one constant `k_mode` instance used by bring-up and diagnostics to enable or suppress optional behavior. These fields are not a stable userspace ABI; they are internal kernel policy/debug controls.

Definition at line 20 of file `KernelUtils.h`.

6.15.2 Field Documentation

6.15.2.1 `acpi_support`

```
uint8_t kswitches::acpi_support
```

Definition at line 24 of file `KernelUtils.h`.

6.15.2.2 `addr_debug`

```
uint8_t kswitches::addr_debug
```

Definition at line 26 of file `KernelUtils.h`.

6.15.2.3 `fpu_allowed`

```
uint8_t kswitches::fpu_allowed
```

Definition at line 30 of file `KernelUtils.h`.

6.15.2.4 hw_rng_support

```
uint8_t kswitches::hw_rng_support
```

Definition at line 27 of file [KernelUtils.h](#).

6.15.2.5 liballoc_debug

```
uint8_t kswitches::liballoc_debug
```

Definition at line 29 of file [KernelUtils.h](#).

6.15.2.6 mem_readout_unit

```
uint8_t kswitches::mem_readout_unit
```

Definition at line 28 of file [KernelUtils.h](#).

6.15.2.7 sched_debug

```
uint8_t kswitches::sched_debug
```

Definition at line 25 of file [KernelUtils.h](#).

6.15.2.8 stack_trace_on_fault

```
uint8_t kswitches::stack_trace_on_fault
```

Definition at line 23 of file [KernelUtils.h](#).

6.15.2.9 stack_trace_size

```
uint8_t kswitches::stack_trace_size
```

Definition at line 22 of file [KernelUtils.h](#).

6.15.2.10 timestamp

```
uint8_t kswitches::timestamp
```

Definition at line 31 of file [KernelUtils.h](#).

The documentation for this struct was generated from the following file:

- [src/include/KernelUtils.h](#)

6.16 limine_5_level_paging_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- [LIMINE_PTR](#) (struct [limine_5_level_paging_response](#) *) response

Data Fields

- uint64_t [id](#) [4]
- uint64_t [revision](#)

6.16.1 Detailed Description

Definition at line 218 of file [limine.h](#).

6.16.2 Member Function Documentation

6.16.2.1 LIMINE_PTR()

```
limine_5_level_paging_request::LIMINE_PTR (  
    struct limine\_5\_level\_paging\_response * )
```

6.16.3 Field Documentation

6.16.3.1 id

```
uint64_t limine_5_level_paging_request::id[4]
```

Definition at line 220 of file [limine.h](#).

6.16.3.2 revision

```
uint64_t limine_5_level_paging_request::revision
```

Definition at line 221 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.17 limine_5_level_paging_response Struct Reference

```
#include <limine.h>
```

Data Fields

- `uint64_t` [revision](#)

6.17.1 Detailed Description

Definition at line 213 of file [limine.h](#).

6.17.2 Field Documentation

6.17.2.1 revision

```
uint64_t limine_5_level_paging_response::revision
```

Definition at line 215 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.18 limine_boot_time_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- **LIMINE_PTR** (struct **limine_boot_time_response** *) response

Data Fields

- uint64_t **id** [4]
- uint64_t **revision**

6.18.1 Detailed Description

Definition at line 465 of file **limine.h**.

6.18.2 Member Function Documentation

6.18.2.1 LIMINE_PTR()

```
limine_boot_time_request::LIMINE_PTR (  
    struct limine_boot_time_response * )
```

6.18.3 Field Documentation

6.18.3.1 id

```
uint64_t limine_boot_time_request::id[4]
```

Definition at line 467 of file **limine.h**.

6.18.3.2 revision

```
uint64_t limine_boot_time_request::revision
```

Definition at line 468 of file **limine.h**.

The documentation for this struct was generated from the following file:

- **src/include/limine.h**

6.19 limine_boot_time_response Struct Reference

```
#include <limine.h>
```

Data Fields

- uint64_t [revision](#)
- int64_t [boot_time](#)

6.19.1 Detailed Description

Definition at line [459](#) of file [limine.h](#).

6.19.2 Field Documentation

6.19.2.1 boot_time

```
int64_t limine_boot_time_response::boot_time
```

Definition at line [462](#) of file [limine.h](#).

6.19.2.2 revision

```
uint64_t limine_boot_time_response::revision
```

Definition at line [461](#) of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.20 limine_bootloader_info_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- [LIMINE_PTR](#) (struct [limine_bootloader_info_response](#) *) response

Data Fields

- uint64_t `id` [4]
- uint64_t `revision`

6.20.1 Detailed Description

Definition at line 65 of file `limine.h`.

6.20.2 Member Function Documentation

6.20.2.1 LIMINE_PTR()

```
limine_bootloader_info_request::LIMINE_PTR (  
    struct limine_bootloader_info_response * )
```

6.20.3 Field Documentation

6.20.3.1 id

```
uint64_t limine_bootloader_info_request::id[4]
```

Definition at line 67 of file `limine.h`.

6.20.3.2 revision

```
uint64_t limine_bootloader_info_request::revision
```

Definition at line 68 of file `limine.h`.

The documentation for this struct was generated from the following file:

- `src/include/limine.h`

6.21 limine_bootloader_info_response Struct Reference

```
#include <limine.h>
```

Public Member Functions

- `LIMINE_PTR` (char *) name
- `LIMINE_PTR` (char *) version

Data Fields

- `uint64_t` `revision`

6.21.1 Detailed Description

Definition at line 58 of file `limine.h`.

6.21.2 Member Function Documentation

6.21.2.1 `LIMINE_PTR()` [1/2]

```
limine_bootloader_info_response::LIMINE_PTR (  
    char * )
```

6.21.2.2 `LIMINE_PTR()` [2/2]

```
limine_bootloader_info_response::LIMINE_PTR (  
    char * )
```

6.21.3 Field Documentation

6.21.3.1 `revision`

```
uint64_t limine_bootloader_info_response::revision
```

Definition at line 60 of file `limine.h`.

The documentation for this struct was generated from the following file:

- `src/include/limine.h`

6.22 limine_dtb_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- **LIMINE_PTR** (struct **limine_dtb_response** *) response

Data Fields

- uint64_t **id** [4]
- uint64_t **revision**

6.22.1 Detailed Description

Definition at line 506 of file **limine.h**.

6.22.2 Member Function Documentation

6.22.2.1 LIMINE_PTR()

```
limine_dtb_request::LIMINE_PTR (  
    struct limine_dtb_response * )
```

6.22.3 Field Documentation

6.22.3.1 id

```
uint64_t limine_dtb_request::id[4]
```

Definition at line 508 of file **limine.h**.

6.22.3.2 revision

```
uint64_t limine_dtb_request::revision
```

Definition at line 509 of file **limine.h**.

The documentation for this struct was generated from the following file:

- **src/include/limine.h**

6.23 limine_dtb_response Struct Reference

```
#include <limine.h>
```

Public Member Functions

- **LIMINE_PTR** (void *) dtb_ptr

Data Fields

- uint64_t **revision**

6.23.1 Detailed Description

Definition at line 500 of file [limine.h](#).

6.23.2 Member Function Documentation

6.23.2.1 LIMINE_PTR()

```
limine_dtb_response::LIMINE_PTR (  
    void * )
```

6.23.3 Field Documentation

6.23.3.1 revision

```
uint64_t limine_dtb_response::revision
```

Definition at line 502 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.24 limine_efi_system_table_request Struct Reference

```
#include <limine.h>
```


Public Member Functions

- `LIMINE_PTR` (struct `limine_efi_system_table_response` *) response

Data Fields

- `uint64_t` `id` [4]
- `uint64_t` `revision`

6.24.1 Detailed Description

Definition at line 445 of file `limine.h`.

6.24.2 Member Function Documentation

6.24.2.1 LIMINE_PTR()

```
limine_efi_system_table_request::LIMINE_PTR (
    struct limine_efi_system_table_response * )
```

6.24.3 Field Documentation

6.24.3.1 id

```
uint64_t limine_efi_system_table_request::id[4]
```

Definition at line 447 of file `limine.h`.

6.24.3.2 revision

```
uint64_t limine_efi_system_table_request::revision
```

Definition at line 448 of file `limine.h`.

The documentation for this struct was generated from the following file:

- `src/include/limine.h`

6.25 limine_efi_system_table_response Struct Reference

```
#include <limine.h>
```

Public Member Functions

- **LIMINE_PTR** (void *) address

Data Fields

- uint64_t **revision**

6.25.1 Detailed Description

Definition at line 439 of file [limine.h](#).

6.25.2 Member Function Documentation

6.25.2.1 LIMINE_PTR()

```
limine_efi_system_table_response::LIMINE_PTR (  
    void * )
```

6.25.3 Field Documentation

6.25.3.1 revision

```
uint64_t limine_efi_system_table_response::revision
```

Definition at line 441 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.26 limine_entry_point_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- `LIMINE_PTR` (struct `limine_entry_point_response *`) `response`
- `LIMINE_PTR` (`limine_entry_point`) `entry`

Data Fields

- `uint64_t` `id` [4]
- `uint64_t` `revision`

6.26.1 Detailed Description

Definition at line 342 of file `limine.h`.

6.26.2 Member Function Documentation

6.26.2.1 `LIMINE_PTR()` [1/2]

```
limine_entry_point_request::LIMINE_PTR (  
    limine_entry_point )
```

6.26.2.2 `LIMINE_PTR()` [2/2]

```
limine_entry_point_request::LIMINE_PTR (  
    struct limine_entry_point_response * )
```

6.26.3 Field Documentation

6.26.3.1 `id`

```
uint64_t limine_entry_point_request::id[4]
```

Definition at line 344 of file `limine.h`.

6.26.3.2 revision

```
uint64_t limine_entry_point_request::revision
```

Definition at line 345 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.27 limine_entry_point_response Struct Reference

```
#include <limine.h>
```

Data Fields

- `uint64_t` [revision](#)

6.27.1 Detailed Description

Definition at line 337 of file [limine.h](#).

6.27.2 Field Documentation

6.27.2.1 revision

```
uint64_t limine_entry_point_response::revision
```

Definition at line 339 of file [limine.h](#).

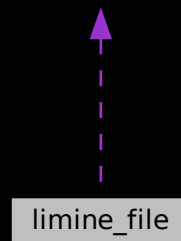
The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.28 limine_file Struct Reference

```
#include <limine.h>
```

Collaboration diagram for limine_file:



Public Member Functions

- `LIMINE_PTR` (void *) address
- `LIMINE_PTR` (char *) path
- `LIMINE_PTR` (char *) cmdline

Data Fields

- `uint64_t` revision
- `uint64_t` size
- `uint32_t` media_type
- `uint32_t` unused
- `uint32_t` tftp_ip
- `uint32_t` tftp_port
- `uint32_t` partition_index
- `uint32_t` mbr_disk_id
- `struct limine_uuid` gpt_disk_uuid
- `struct limine_uuid` gpt_part_uuid
- `struct limine_uuid` part_uuid

6.28.1 Detailed Description

Definition at line 33 of file `limine.h`.

6.28.2 Member Function Documentation

6.28.2.1 LIMINE_PTR() [1/3]

```
limine_file::LIMINE_PTR (  
    char * )
```

6.28.2.2 LIMINE_PTR() [2/3]

```
limine_file::LIMINE_PTR (  
    char * )
```

6.28.2.3 LIMINE_PTR() [3/3]

```
limine_file::LIMINE_PTR (  
    void * )
```

6.28.3 Field Documentation

6.28.3.1 gpt_disk_uuid

```
struct limine\_uuid limine_file::gpt_disk_uuid
```

Definition at line [46](#) of file [limine.h](#).

6.28.3.2 gpt_part_uuid

```
struct limine\_uuid limine_file::gpt_part_uuid
```

Definition at line [47](#) of file [limine.h](#).

6.28.3.3 mbr_disk_id

```
uint32_t limine_file::mbr_disk_id
```

Definition at line [45](#) of file [limine.h](#).

6.28.3.4 media_type

```
uint32_t limine_file::media_type
```

Definition at line 40 of file [limine.h](#).

6.28.3.5 part_uuid

```
struct limine_uuid limine_file::part_uuid
```

Definition at line 48 of file [limine.h](#).

6.28.3.6 partition_index

```
uint32_t limine_file::partition_index
```

Definition at line 44 of file [limine.h](#).

6.28.3.7 revision

```
uint64_t limine_file::revision
```

Definition at line 35 of file [limine.h](#).

6.28.3.8 size

```
uint64_t limine_file::size
```

Definition at line 37 of file [limine.h](#).

6.28.3.9 tftp_ip

```
uint32_t limine_file::tftp_ip
```

Definition at line 42 of file [limine.h](#).

6.28.3.10 tftp_port

```
uint32_t limine_file::tftp_port
```

Definition at line 43 of file [limine.h](#).

6.28.3.11 unused

```
uint32_t limine_file::unused
```

Definition at line 41 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.29 limine_framebuffer Struct Reference

```
#include <limine.h>
```

Public Member Functions

- [LIMINE_PTR](#) (void *) address
- [LIMINE_PTR](#) (void *) edid

Data Fields

- [uint64_t](#) width
- [uint64_t](#) height
- [uint64_t](#) pitch
- [uint16_t](#) bpp
- [uint8_t](#) memory_model
- [uint8_t](#) red_mask_size
- [uint8_t](#) red_mask_shift
- [uint8_t](#) green_mask_size
- [uint8_t](#) green_mask_shift
- [uint8_t](#) blue_mask_size
- [uint8_t](#) blue_mask_shift
- [uint8_t](#) unused [7]
- [uint64_t](#) edid_size

6.29.1 Detailed Description

Definition at line 121 of file [limine.h](#).

6.29.2 Member Function Documentation

6.29.2.1 LIMINE_PTR() [1/2]

```
limine_framebuffer::LIMINE_PTR (
    void * )
```

6.29.2.2 LIMINE_PTR() [2/2]

```
limine_framebuffer::LIMINE_PTR (
    void * )
```

6.29.3 Field Documentation

6.29.3.1 blue_mask_shift

```
uint8_t limine_framebuffer::blue_mask_shift
```

Definition at line 134 of file [limine.h](#).

6.29.3.2 blue_mask_size

```
uint8_t limine_framebuffer::blue_mask_size
```

Definition at line 133 of file [limine.h](#).

6.29.3.3 bpp

```
uint16_t limine_framebuffer::bpp
```

Definition at line 127 of file [limine.h](#).

6.29.3.4 edid_size

```
uint64_t limine_framebuffer::edid_size
```

Definition at line 136 of file [limine.h](#).

6.29.3.5 green_mask_shift

```
uint8_t limine_framebuffer::green_mask_shift
```

Definition at line 132 of file [limine.h](#).

6.29.3.6 green_mask_size

```
uint8_t limine_framebuffer::green_mask_size
```

Definition at line 131 of file [limine.h](#).

6.29.3.7 height

```
uint64_t limine_framebuffer::height
```

Definition at line 125 of file [limine.h](#).

6.29.3.8 memory_model

```
uint8_t limine_framebuffer::memory_model
```

Definition at line 128 of file [limine.h](#).

6.29.3.9 pitch

```
uint64_t limine_framebuffer::pitch
```

Definition at line 126 of file [limine.h](#).

6.29.3.10 red_mask_shift

```
uint8_t limine_framebuffer::red_mask_shift
```

Definition at line 130 of file [limine.h](#).

6.29.3.11 red_mask_size

```
uint8_t limine_framebuffer::red_mask_size
```

Definition at line 129 of file [limine.h](#).

6.29.3.12 unused

```
uint8_t limine_framebuffer::unused[7]
```

Definition at line 135 of file [limine.h](#).

6.29.3.13 width

```
uint64_t limine_framebuffer::width
```

Definition at line 124 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.30 limine_framebuffer_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- [LIMINE_PTR](#) (struct [limine_framebuffer_response](#) *) response

Data Fields

- uint64_t [id](#) [4]
- uint64_t [revision](#)

6.30.1 Detailed Description

Definition at line 147 of file [limine.h](#).

6.30.2 Member Function Documentation

6.30.2.1 LIMINE_PTR()

```
limine_framebuffer_request::LIMINE_PTR (
    struct limine\_framebuffer\_response * )
```

6.30.3 Field Documentation

6.30.3.1 id

```
uint64_t limine_framebuffer_request::id[4]
```

Definition at line 149 of file [limine.h](#).

6.30.3.2 revision

```
uint64_t limine_framebuffer_request::revision
```

Definition at line 150 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.31 limine_framebuffer_response Struct Reference

```
#include <limine.h>
```

Public Member Functions

- [LIMINE_PTR](#) (struct [limine_framebuffer](#) **) framebuffers

Data Fields

- uint64_t `revision`
- uint64_t `framebuffer_count`

6.31.1 Detailed Description

Definition at line 140 of file `limine.h`.

6.31.2 Member Function Documentation

6.31.2.1 LIMINE_PTR()

```
limine_framebuffer_response::LIMINE_PTR (  
    struct limine_framebuffer ** )
```

6.31.3 Field Documentation

6.31.3.1 framebuffer_count

```
uint64_t limine_framebuffer_response::framebuffer_count
```

Definition at line 143 of file `limine.h`.

6.31.3.2 revision

```
uint64_t limine_framebuffer_response::revision
```

Definition at line 142 of file `limine.h`.

The documentation for this struct was generated from the following file:

- `src/include/limine.h`

6.32 limine_hhdm_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- `LIMINE_PTR` (struct `limine_hhdm_response` *) response

Data Fields

- `uint64_t` `id` [4]
- `uint64_t` `revision`

6.32.1 Detailed Description

Definition at line 105 of file `limine.h`.

6.32.2 Member Function Documentation

6.32.2.1 LIMINE_PTR()

```
limine_hhdm_request::LIMINE_PTR (  
    struct limine_hhdm_response * )
```

6.32.3 Field Documentation

6.32.3.1 id

```
uint64_t limine_hhdm_request::id[4]
```

Definition at line 107 of file `limine.h`.

6.32.3.2 revision

```
uint64_t limine_hhdm_request::revision
```

Definition at line 108 of file `limine.h`.

The documentation for this struct was generated from the following file:

- `src/include/limine.h`

6.33 limine_hhdm_response Struct Reference

```
#include <limine.h>
```

Data Fields

- uint64_t [revision](#)
- uint64_t [offset](#)

6.33.1 Detailed Description

Definition at line 99 of file [limine.h](#).

6.33.2 Field Documentation

6.33.2.1 offset

```
uint64_t limine_hhdm_response::offset
```

Definition at line 102 of file [limine.h](#).

6.33.2.2 revision

```
uint64_t limine_hhdm_response::revision
```

Definition at line 101 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.34 limine_kernel_address_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- [LIMINE_PTR](#) (struct [limine_kernel_address_response](#) *) response

Data Fields

- uint64_t `id` [4]
- uint64_t `revision`

6.34.1 Detailed Description

Definition at line 486 of file `limine.h`.

6.34.2 Member Function Documentation

6.34.2.1 LIMINE_PTR()

```
limine_kernel_address_request::LIMINE_PTR (  
    struct limine_kernel_address_response * )
```

6.34.3 Field Documentation

6.34.3.1 id

```
uint64_t limine_kernel_address_request::id[4]
```

Definition at line 488 of file `limine.h`.

6.34.3.2 revision

```
uint64_t limine_kernel_address_request::revision
```

Definition at line 489 of file `limine.h`.

The documentation for this struct was generated from the following file:

- `src/include/limine.h`

6.35 limine_kernel_address_response Struct Reference

```
#include <limine.h>
```


Data Fields

- uint64_t [revision](#)
- uint64_t [physical_base](#)
- uint64_t [virtual_base](#)

6.35.1 Detailed Description

Definition at line [479](#) of file [limine.h](#).

6.35.2 Field Documentation

6.35.2.1 [physical_base](#)

```
uint64_t limine_kernel_address_response::physical_base
```

Definition at line [482](#) of file [limine.h](#).

6.35.2.2 [revision](#)

```
uint64_t limine_kernel_address_response::revision
```

Definition at line [481](#) of file [limine.h](#).

6.35.2.3 [virtual_base](#)

```
uint64_t limine_kernel_address_response::virtual_base
```

Definition at line [483](#) of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.36 limine_kernel_file_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- `LIMINE_PTR` (struct `limine_kernel_file_response` *) response

Data Fields

- `uint64_t` `id` [4]
- `uint64_t` `revision`

6.36.1 Detailed Description

Definition at line 363 of file `limine.h`.

6.36.2 Member Function Documentation

6.36.2.1 LIMINE_PTR()

```
limine_kernel_file_request::LIMINE_PTR (  
    struct limine_kernel_file_response * )
```

6.36.3 Field Documentation

6.36.3.1 id

```
uint64_t limine_kernel_file_request::id[4]
```

Definition at line 365 of file `limine.h`.

6.36.3.2 revision

```
uint64_t limine_kernel_file_request::revision
```

Definition at line 366 of file `limine.h`.

The documentation for this struct was generated from the following file:

- `src/include/limine.h`

6.37 limine_kernel_file_response Struct Reference

```
#include <limine.h>
```

Public Member Functions

- **LIMINE_PTR** (struct **limine_file** *) kernel_file

Data Fields

- uint64_t **revision**

6.37.1 Detailed Description

Definition at line 357 of file **limine.h**.

6.37.2 Member Function Documentation

6.37.2.1 LIMINE_PTR()

```
limine_kernel_file_response::LIMINE_PTR (  
    struct limine_file * )
```

6.37.3 Field Documentation

6.37.3.1 revision

```
uint64_t limine_kernel_file_response::revision
```

Definition at line 359 of file **limine.h**.

The documentation for this struct was generated from the following file:

- src/include/**limine.h**

6.38 limine_memmap_entry Struct Reference

```
#include <limine.h>
```

Data Fields

- `uint64_t` `base`
- `uint64_t` `length`
- `uint64_t` `type`

6.38.1 Detailed Description

Definition at line 307 of file `limine.h`.

6.38.2 Field Documentation

6.38.2.1 `base`

```
uint64_t limine_memmap_entry::base
```

Definition at line 309 of file `limine.h`.

6.38.2.2 `length`

```
uint64_t limine_memmap_entry::length
```

Definition at line 310 of file `limine.h`.

6.38.2.3 `type`

```
uint64_t limine_memmap_entry::type
```

Definition at line 311 of file `limine.h`.

The documentation for this struct was generated from the following file:

- `src/include/limine.h`

6.39 `limine_memmap_request` Struct Reference

```
#include <limine.h>
```

Public Member Functions

- `LIMINE_PTR` (struct `limine_memmap_response` *) response

Data Fields

- `uint64_t` `id` [4]
- `uint64_t` `revision`

6.39.1 Detailed Description

Definition at line 321 of file `limine.h`.

6.39.2 Member Function Documentation

6.39.2.1 LIMINE_PTR()

```
limine_memmap_request::LIMINE_PTR (
    struct limine_memmap_response * )
```

6.39.3 Field Documentation

6.39.3.1 id

```
uint64_t limine_memmap_request::id[4]
```

Definition at line 323 of file `limine.h`.

6.39.3.2 revision

```
uint64_t limine_memmap_request::revision
```

Definition at line 324 of file `limine.h`.

The documentation for this struct was generated from the following file:

- `src/include/limine.h`

6.40 limine_memmap_response Struct Reference

```
#include <limine.h>
```

Public Member Functions

- **LIMINE_PTR** (struct **limine_memmap_entry** **) entries

Data Fields

- uint64_t **revision**
- uint64_t **entry_count**

6.40.1 Detailed Description

Definition at line 314 of file **limine.h**.

6.40.2 Member Function Documentation

6.40.2.1 LIMINE_PTR()

```
limine_memmap_response::LIMINE_PTR (  
    struct limine_memmap_entry ** )
```

6.40.3 Field Documentation

6.40.3.1 entry_count

```
uint64_t limine_memmap_response::entry_count
```

Definition at line 317 of file **limine.h**.

6.40.3.2 revision

```
uint64_t limine_memmap_response::revision
```

Definition at line 316 of file **limine.h**.

The documentation for this struct was generated from the following file:

- **src/include/limine.h**

6.41 limine_module_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- **LIMINE_PTR** (struct **limine_module_response** *) response

Data Fields

- uint64_t **id** [4]
- uint64_t **revision**

6.41.1 Detailed Description

Definition at line 384 of file **limine.h**.

6.41.2 Member Function Documentation

6.41.2.1 LIMINE_PTR()

```
limine_module_request::LIMINE_PTR (  
    struct limine_module_response * )
```

6.41.3 Field Documentation

6.41.3.1 id

```
uint64_t limine_module_request::id[4]
```

Definition at line 386 of file **limine.h**.

6.41.3.2 revision

```
uint64_t limine_module_request::revision
```

Definition at line 387 of file **limine.h**.

The documentation for this struct was generated from the following file:

- **src/include/limine.h**

6.42 limine_module_response Struct Reference

```
#include <limine.h>
```

Public Member Functions

- **LIMINE_PTR** (struct **limine_file** **) modules

Data Fields

- uint64_t **revision**
- uint64_t **module_count**

6.42.1 Detailed Description

Definition at line 377 of file **limine.h**.

6.42.2 Member Function Documentation

6.42.2.1 LIMINE_PTR()

```
limine_module_response::LIMINE_PTR (  
    struct limine_file ** )
```

6.42.3 Field Documentation

6.42.3.1 module_count

```
uint64_t limine_module_response::module_count
```

Definition at line 380 of file **limine.h**.

6.42.3.2 revision

```
uint64_t limine_module_response::revision
```

Definition at line 379 of file **limine.h**.

The documentation for this struct was generated from the following file:

- **src/include/limine.h**

6.43 limine_rsdg_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- **LIMINE_PTR** (struct **limine_rsdg_response** *) response

Data Fields

- uint64_t **id** [4]
- uint64_t **revision**

6.43.1 Detailed Description

Definition at line 404 of file **limine.h**.

6.43.2 Member Function Documentation

6.43.2.1 LIMINE_PTR()

```
limine_rsdg_request::LIMINE_PTR (  
    struct limine_rsdg_response * )
```

6.43.3 Field Documentation

6.43.3.1 id

```
uint64_t limine_rsdg_request::id[4]
```

Definition at line 406 of file **limine.h**.

6.43.3.2 revision

```
uint64_t limine_rsdg_request::revision
```

Definition at line 407 of file **limine.h**.

The documentation for this struct was generated from the following file:

- **src/include/limine.h**

6.44 limine_rsdg_response Struct Reference

```
#include <limine.h>
```

Public Member Functions

- **LIMINE_PTR** (void *) address

Data Fields

- uint64_t **revision**

6.44.1 Detailed Description

Definition at line 398 of file [limine.h](#).

6.44.2 Member Function Documentation

6.44.2.1 LIMINE_PTR()

```
limine_rsdg_response::LIMINE_PTR (  
    void * )
```

6.44.3 Field Documentation

6.44.3.1 revision

```
uint64_t limine_rsdg_response::revision
```

Definition at line 400 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.45 limine_smbios_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- `LIMINE_PTR` (struct `limine_smbios_response` *) response

Data Fields

- `uint64_t` `id` [4]
- `uint64_t` `revision`

6.45.1 Detailed Description

Definition at line 425 of file `limine.h`.

6.45.2 Member Function Documentation

6.45.2.1 LIMINE_PTR()

```
limine_smbios_request::LIMINE_PTR (  
    struct limine_smbios_response * )
```

6.45.3 Field Documentation

6.45.3.1 id

```
uint64_t limine_smbios_request::id[4]
```

Definition at line 427 of file `limine.h`.

6.45.3.2 revision

```
uint64_t limine_smbios_request::revision
```

Definition at line 428 of file `limine.h`.

The documentation for this struct was generated from the following file:

- `src/include/limine.h`

6.46 limine_smbios_response Struct Reference

```
#include <limine.h>
```

Public Member Functions

- [LIMINE_PTR](#) (void *) entry_32
- [LIMINE_PTR](#) (void *) entry_64

Data Fields

- uint64_t [revision](#)

6.46.1 Detailed Description

Definition at line [418](#) of file [limine.h](#).

6.46.2 Member Function Documentation

6.46.2.1 LIMINE_PTR() [1/2]

```
limine_smbios_response::LIMINE_PTR (  
    void * )
```

6.46.2.2 LIMINE_PTR() [2/2]

```
limine_smbios_response::LIMINE_PTR (  
    void * )
```

6.46.3 Field Documentation

6.46.3.1 revision

```
uint64_t limine_smbios_response::revision
```

Definition at line [420](#) of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.47 limine_smp_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- **LIMINE_PTR** (struct limine_smp_response *) response

Data Fields

- uint64_t **id** [4]
- uint64_t **revision**
- uint64_t **flags**

6.47.1 Detailed Description

Definition at line 283 of file [limine.h](#).

6.47.2 Member Function Documentation

6.47.2.1 LIMINE_PTR()

```
limine_smp_request::LIMINE_PTR (  
    struct limine_smp_response * )
```

6.47.3 Field Documentation

6.47.3.1 flags

```
uint64_t limine_smp_request::flags
```

Definition at line 288 of file [limine.h](#).

6.47.3.2 id

```
uint64_t limine_smp_request::id[4]
```

Definition at line 285 of file [limine.h](#).

6.47.3.3 revision

```
uint64_t limine_smp_request::revision
```

Definition at line 286 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.48 limine_stack_size_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- [LIMINE_PTR](#) (struct [limine_stack_size_response](#) *) response

Data Fields

- uint64_t [id](#) [4]
- uint64_t [revision](#)
- uint64_t [stack_size](#)

6.48.1 Detailed Description

Definition at line 84 of file [limine.h](#).

6.48.2 Member Function Documentation

6.48.2.1 LIMINE_PTR()

```
limine_stack_size_request::LIMINE_PTR (  
    struct limine\_stack\_size\_response * )
```

6.48.3 Field Documentation

6.48.3.1 id

```
uint64_t limine_stack_size_request::id[4]
```

Definition at line 86 of file [limine.h](#).

6.48.3.2 revision

```
uint64_t limine_stack_size_request::revision
```

Definition at line 87 of file [limine.h](#).

6.48.3.3 stack_size

```
uint64_t limine_stack_size_request::stack_size
```

Definition at line 89 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.49 limine_stack_size_response Struct Reference

```
#include <limine.h>
```

Data Fields

- `uint64_t` [revision](#)

6.49.1 Detailed Description

Definition at line 79 of file [limine.h](#).

6.49.2 Field Documentation

6.49.2.1 revision

```
uint64_t limine_stack_size_response::revision
```

Definition at line 81 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.50 limine_terminal Struct Reference

```
#include <limine.h>
```

Public Member Functions

- [LIMINE_PTR](#) (struct [limine_framebuffer](#) *) framebuffer

Data Fields

- [uint64_t](#) [columns](#)
- [uint64_t](#) [rows](#)

6.50.1 Detailed Description

Definition at line 183 of file [limine.h](#).

6.50.2 Member Function Documentation

6.50.2.1 LIMINE_PTR()

```
limine_terminal::LIMINE_PTR (  
    struct limine\_framebuffer * )
```

6.50.3 Field Documentation

6.50.3.1 columns

```
uint64_t limine_terminal::columns
```

Definition at line 185 of file [limine.h](#).

6.50.3.2 rows

```
uint64_t limine_terminal::rows
```

Definition at line 186 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.51 limine_terminal_request Struct Reference

```
#include <limine.h>
```

Public Member Functions

- [LIMINE_PTR](#) (struct [limine_terminal_response](#) *) response
- [LIMINE_PTR](#) ([limine_terminal_callback](#)) callback

Data Fields

- [uint64_t](#) [id](#) [4]
- [uint64_t](#) [revision](#)

6.51.1 Detailed Description

Definition at line 198 of file [limine.h](#).

6.51.2 Member Function Documentation

6.51.2.1 LIMINE_PTR() [1/2]

```
limine_terminal_request::LIMINE_PTR (
    limine\_terminal\_callback )
```

6.51.2.2 LIMINE_PTR() [2/2]

```
limine_terminal_request::LIMINE_PTR (
    struct limine_terminal_response * )
```

6.51.3 Field Documentation

6.51.3.1 id

```
uint64_t limine_terminal_request::id[4]
```

Definition at line 200 of file [limine.h](#).

6.51.3.2 revision

```
uint64_t limine_terminal_request::revision
```

Definition at line 201 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.52 limine_terminal_response Struct Reference

```
#include <limine.h>
```

Public Member Functions

- [LIMINE_PTR](#) (struct [limine_terminal](#) **) terminals
- [LIMINE_PTR](#) ([limine_terminal_write](#)) write

Data Fields

- uint64_t [revision](#)
- uint64_t [terminal_count](#)

6.52.1 Detailed Description

Definition at line 190 of file [limine.h](#).

6.52.2 Member Function Documentation

6.52.2.1 LIMINE_PTR() [1/2]

```
limine_terminal_response::LIMINE_PTR (
    limine_terminal_write )
```

6.52.2.2 LIMINE_PTR() [2/2]

```
limine_terminal_response::LIMINE_PTR (
    struct limine_terminal ** )
```

6.52.3 Field Documentation

6.52.3.1 revision

```
uint64_t limine_terminal_response::revision
```

Definition at line 192 of file [limine.h](#).

6.52.3.2 terminal_count

```
uint64_t limine_terminal_response::terminal_count
```

Definition at line 193 of file [limine.h](#).

The documentation for this struct was generated from the following file:

- [src/include/limine.h](#)

6.53 limine_uuid Struct Reference

```
#include <limine.h>
```

Data Fields

- `uint32_t a`
- `uint16_t b`
- `uint16_t c`
- `uint8_t d[8]`

6.53.1 Detailed Description

Definition at line 21 of file `limine.h`.

6.53.2 Field Documentation

6.53.2.1 a

```
uint32_t limine_uuid::a
```

Definition at line 23 of file `limine.h`.

6.53.2.2 b

```
uint16_t limine_uuid::b
```

Definition at line 24 of file `limine.h`.

6.53.2.3 c

```
uint16_t limine_uuid::c
```

Definition at line 25 of file `limine.h`.

6.53.2.4 d

```
uint8_t limine_uuid::d[8]
```

Definition at line 26 of file `limine.h`.

The documentation for this struct was generated from the following file:

- `src/include/limine.h`

6.54 madt_header Struct Reference

Common two-byte header shared by variable-length MADT records.

```
#include <madt.h>
```

Data Fields

- `uint8_t id`
- `uint8_t length`

6.54.1 Detailed Description

Common two-byte header shared by variable-length MADT records.

Definition at line 15 of file `madt.h`.

6.54.2 Field Documentation

6.54.2.1 id

```
uint8_t madt_header::id
```

Definition at line 17 of file `madt.h`.

6.54.2.2 length

```
uint8_t madt_header::length
```

Definition at line 18 of file `madt.h`.

The documentation for this struct was generated from the following file:

- `src/include/acpi/madt.h`

6.55 madt_io_apic Struct Reference

MADT I/O APIC record.

```
#include <madt.h>
```

Data Fields

- uint8_t `apic_id`
- uint8_t `reserved`
- uint32_t `address`
- uint32_t `gsib`

6.55.1 Detailed Description

MADT I/O APIC record.

Definition at line 31 of file `madt.h`.

6.55.2 Field Documentation

6.55.2.1 address

```
uint32_t madt_io_apic::address
```

Definition at line 36 of file `madt.h`.

6.55.2.2 apic_id

```
uint8_t madt_io_apic::apic_id
```

Definition at line 34 of file `madt.h`.

6.55.2.3 gsib

```
uint32_t madt_io_apic::gsib
```

Definition at line 37 of file `madt.h`.

6.55.2.4 reserved

```
uint8_t madt_io_apic::reserved
```

Definition at line 35 of file `madt.h`.

The documentation for this struct was generated from the following file:

- `src/include/acpi/madt.h`

6.56 madt_iso Struct Reference

MADT Interrupt Source Override record.

```
#include <madt.h>
```

Data Fields

- `uint8_t bus_source`
- `uint8_t irq_source`
- `uint32_t gsi`
- `uint16_t flags`

6.56.1 Detailed Description

MADT Interrupt Source Override record.

Definition at line 41 of file `madt.h`.

6.56.2 Field Documentation

6.56.2.1 bus_source

```
uint8_t madt_iso::bus_source
```

Definition at line 44 of file `madt.h`.

6.56.2.2 flags

```
uint16_t madt_iso::flags
```

Definition at line 47 of file `madt.h`.

6.56.2.3 gsi

```
uint32_t madt_iso::gsi
```

Definition at line 46 of file `madt.h`.

6.56.2.4 irq_source

```
uint8_t madt_iso::irq_source
```

Definition at line 45 of file [madt.h](#).

The documentation for this struct was generated from the following file:

- [src/include/acpi/madt.h](#)

6.57 madt_lapic Struct Reference

MADT Processor Local APIC record.

```
#include <madt.h>
```

Data Fields

- [uint8_t processor_id](#)
- [uint8_t apic_id](#)
- [uint32_t flags](#)

6.57.1 Detailed Description

MADT Processor Local APIC record.

Definition at line 22 of file [madt.h](#).

6.57.2 Field Documentation

6.57.2.1 apic_id

```
uint8_t madt_lapic::apic_id
```

Definition at line 26 of file [madt.h](#).

6.57.2.2 flags

```
uint32_t madt_lapic::flags
```

Definition at line 27 of file [madt.h](#).

6.57.2.3 processor_id

```
uint8_t madt_lapic::processor_id
```

Definition at line 25 of file [madt.h](#).

The documentation for this struct was generated from the following file:

- [src/include/acpi/madt.h](#)

6.58 madt_nmi Struct Reference

MADT local-APIC NMI record.

```
#include <madt.h>
```

Data Fields

- [uint8_t processor](#)
- [uint16_t flags](#)
- [uint8_t lint](#)

6.58.1 Detailed Description

MADT local-APIC NMI record.

Definition at line 51 of file [madt.h](#).

6.58.2 Field Documentation

6.58.2.1 flags

```
uint16_t madt_nmi::flags
```

Definition at line 55 of file [madt.h](#).

6.58.2.2 lint

```
uint8_t madt_nmi::lint
```

Definition at line 56 of file [madt.h](#).

6.58.2.3 processor

```
uint8_t madt_nmi::processor
```

Definition at line 54 of file [madt.h](#).

The documentation for this struct was generated from the following file:

- [src/include/acpi/madt.h](#)

6.59 memtest_slot Struct Reference

Data Fields

- [uint8_t * ptr](#)
- [size_t size](#)
- [uint8_t tag](#)

6.59.1 Detailed Description

Definition at line 26 of file [memtest.c](#).

6.59.2 Field Documentation

6.59.2.1 ptr

```
uint8_t* memtest_slot::ptr
```

Definition at line 28 of file [memtest.c](#).

6.59.2.2 size

```
size_t memtest_slot::size
```

Definition at line 29 of file [memtest.c](#).

6.59.2.3 tag

```
uint8_t memtest_slot::tag
```

Definition at line 30 of file `memtest.c`.

The documentation for this struct was generated from the following file:

- `src/mm/memtest.c`

6.60 memtest_stats Struct Reference

Data Fields

- `uint64_t operations`
- `uint64_t allocations`
- `uint64_t frees`
- `uint64_t reallocations`
- `uint64_t live_bytes`
- `uint64_t peak_live_bytes`

6.60.1 Detailed Description

Definition at line 33 of file `memtest.c`.

6.60.2 Field Documentation

6.60.2.1 allocations

```
uint64_t memtest_stats::allocations
```

Definition at line 36 of file `memtest.c`.

6.60.2.2 frees

```
uint64_t memtest_stats::frees
```

Definition at line 37 of file `memtest.c`.

6.60.2.3 live_bytes

```
uint64_t memtest_stats::live_bytes
```

Definition at line 39 of file [memtest.c](#).

6.60.2.4 operations

```
uint64_t memtest_stats::operations
```

Definition at line 35 of file [memtest.c](#).

6.60.2.5 peak_live_bytes

```
uint64_t memtest_stats::peak_live_bytes
```

Definition at line 40 of file [memtest.c](#).

6.60.2.6 reallocations

```
uint64_t memtest_stats::reallocations
```

Definition at line 38 of file [memtest.c](#).

The documentation for this struct was generated from the following file:

- [src/mm/memtest.c](#)

6.61 out_fct_wrap_type Struct Reference

Data Fields

- void(* [fct](#))(char character, void *[arg](#))
- void * [arg](#)

6.61.1 Detailed Description

Definition at line 126 of file [printf.c](#).

6.61.2 Field Documentation

6.61.2.1 arg

```
void* out_fct_wrap_type::arg
```

Definition at line 129 of file [printf.c](#).

6.61.2.2 fct

```
void(* out_fct_wrap_type::fct) (char character, void *arg)
```

Definition at line 128 of file [printf.c](#).

The documentation for this struct was generated from the following file:

- [src/printf.c](#)

6.62 PageTableOffset Struct Reference

Four-level x86-64 page-table indices for a virtual address.

```
#include <paging.h>
```

Data Fields

- `uint64_t` [p4Offset](#)
- `uint64_t` [pdpOffset](#)
- `uint64_t` [pdOffset](#)
- `uint64_t` [ptOffset](#)

6.62.1 Detailed Description

Four-level x86-64 page-table indices for a virtual address.

Definition at line 114 of file [paging.h](#).

6.62.2 Field Documentation

6.62.2.1 p4Offset

```
uint64_t PageTableOffset::p4Offset
```

Definition at line 116 of file [paging.h](#).

6.62.2.2 pdOffset

```
uint64_t PageTableOffset::pdOffset
```

Definition at line 118 of file [paging.h](#).

6.62.2.3 pdpOffset

```
uint64_t PageTableOffset::pdpOffset
```

Definition at line 117 of file [paging.h](#).

6.62.2.4 ptOffset

```
uint64_t PageTableOffset::ptOffset
```

Definition at line 119 of file [paging.h](#).

The documentation for this struct was generated from the following file:

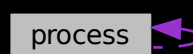
- [src/include/paging/paging.h](#)

6.63 process Struct Reference

Minimal 0.0.5 process control block.

```
#include <sched.h>
```

Collaboration diagram for process:



Data Fields

- int `pid`
- uint64_t * `rsp`
- `proc_state_t` `state`
- struct `process` * `next`

6.63.1 Detailed Description

Minimal 0.0.5 process control block.

Processes currently share the kernel address space. Each process owns a separately allocated kernel stack; `rsp` records the saved cooperative context for `switch_to()`. The `next` pointer forms the circular run queue.

Warning

`sched_0l.asm` depends on the layout of this structure, in particular `rsp` being at byte offset 8. Keep the assembly and C definition in sync if fields are reordered or resized.

Definition at line 48 of file `sched.h`.

6.63.2 Field Documentation

6.63.2.1 next

```
struct process* process::next
```

Next process in the circular run queue.

Definition at line 60 of file `sched.h`.

6.63.2.2 pid

```
int process::pid
```

Monotonically assigned kernel process identifier.

Definition at line 51 of file `sched.h`.

6.63.2.3 rsp

```
uint64_t* process::rsp
```

Saved stack pointer consumed/produced by `switch_to()`.

Definition at line 54 of file `sched.h`.

6.63.2.4 state

```
proc_state_t process::state
```

Current scheduler state.

Definition at line 57 of file `sched.h`.

The documentation for this struct was generated from the following file:

- `src/include/sched.h`

6.64 rsdp Struct Reference

Data Fields

- char `signature` [8]
- uint8_t `checksum`
- char `oem_id` [6]
- uint8_t `revision`
- uint32_t `rsdt_addr`
- uint32_t `length`
- uint64_t `xsdt_addr`
- uint8_t `ext_checksum`
- char `reserved` [3]

6.64.1 Detailed Description

Definition at line 24 of file `acpi.c`.

6.64.2 Field Documentation

6.64.2.1 checksum

```
uint8_t rsdp::checksum
```

Definition at line 27 of file [acpi.c](#).

6.64.2.2 ext_checksum

```
uint8_t rsdp::ext_checksum
```

Definition at line 33 of file [acpi.c](#).

6.64.2.3 length

```
uint32_t rsdp::length
```

Definition at line 31 of file [acpi.c](#).

6.64.2.4 oem_id

```
char rsdp::oem_id[6]
```

Definition at line 28 of file [acpi.c](#).

6.64.2.5 reserved

```
char rsdp::reserved[3]
```

Definition at line 34 of file [acpi.c](#).

6.64.2.6 revision

```
uint8_t rsdp::revision
```

Definition at line 29 of file [acpi.c](#).

6.64.2.7 rsdt_addr

```
uint32_t rsdp::rsdt_addr
```

Definition at line 30 of file [acpi.c](#).

6.64.2.8 signature

```
char rsdp::signature[8]
```

Definition at line 26 of file [acpi.c](#).

6.64.2.9 xsdt_addr

```
uint64_t rsdp::xsdt_addr
```

Definition at line 32 of file [acpi.c](#).

The documentation for this struct was generated from the following file:

- [src/acpi/acpi.c](#)

6.65 rsdt Struct Reference

Data Fields

- [char data \[\]](#)

6.65.1 Detailed Description

Definition at line 37 of file [acpi.c](#).

6.65.2 Field Documentation

6.65.2.1 data

```
char rsdt::data[]
```

Definition at line 40 of file [acpi.c](#).

The documentation for this struct was generated from the following file:

- [src/acpi/acpi.c](#)

6.66 sdt Struct Reference

Common ACPI System Description Table header.

```
#include <acpi.h>
```

Data Fields

- char [signature](#) [4]
- uint32_t [length](#)
- uint8_t [revision](#)
- uint8_t [checksum](#)
- char [oem_id](#) [6]
- char [oem_table_id](#) [6]
- uint32_t [oem_revision](#)
- uint32_t [creator_id](#)
- uint32_t [creator_revision](#)

6.66.1 Detailed Description

Common ACPI System Description Table header.

The fixed header is shared by ACPI tables located through [acpi_find_sdt\(\)](#). Table-specific structures begin after/around this header as defined by ACPI.

Definition at line 18 of file [acpi.h](#).

6.66.2 Field Documentation

6.66.2.1 checksum

```
uint8_t sdt::checksum
```

Definition at line 23 of file [acpi.h](#).

6.66.2.2 creator_id

```
uint32_t sdt::creator_id
```

Definition at line 27 of file [acpi.h](#).

6.66.2.3 creator_revision

```
uint32_t sdt::creator_revision
```

Definition at line 28 of file [acpi.h](#).

6.66.2.4 length

```
uint32_t sdt::length
```

Definition at line 21 of file [acpi.h](#).

6.66.2.5 oem_id

```
char sdt::oem_id[6]
```

Definition at line 24 of file [acpi.h](#).

6.66.2.6 oem_revision

```
uint32_t sdt::oem_revision
```

Definition at line 26 of file [acpi.h](#).

6.66.2.7 oem_table_id

```
char sdt::oem_table_id[6]
```

Definition at line 25 of file [acpi.h](#).

6.66.2.8 revision

```
uint8_t sdt::revision
```

Definition at line 22 of file [acpi.h](#).

6.66.2.9 signature

```
char sdt::signature[4]
```

Definition at line 20 of file [acpi.h](#).

The documentation for this struct was generated from the following file:

- [src/include/acpi/acpi.h](#)

6.67 smartlock Struct Reference

```
#include <lock.h>
```

Data Fields

- [size_t refcount](#)
- [struct thread * thread](#)

6.67.1 Detailed Description

Definition at line 12 of file [lock.h](#).

6.67.2 Field Documentation

6.67.2.1 refcount

```
size_t smartlock::refcount
```

Definition at line 14 of file [lock.h](#).

6.67.2.2 thread

```
struct thread* smartlock::thread
```

Definition at line 15 of file [lock.h](#).

The documentation for this struct was generated from the following file:

- [src/include/lock.h](#)

6.68 stack_frame Struct Reference

Frame-pointer chain node used by the C stack walker.

```
#include <stack_trace.h>
```

Collaboration diagram for `stack_frame`:



stack_frame

Data Fields

- struct [stack_frame](#) * [rbp](#)
- [uint64_t](#) [rip](#)

6.68.1 Detailed Description

Frame-pointer chain node used by the C stack walker.

Definition at line 27 of file [stack_trace.h](#).

6.68.2 Field Documentation

6.68.2.1 rbp

```
struct stack\_frame* stack\_frame::rbp
```

Definition at line 29 of file [stack_trace.h](#).

6.68.2.2 rip

```
uint64_t stack_frame::rip
```

Definition at line 30 of file [stack_trace.h](#).

The documentation for this struct was generated from the following file:

- [src/include/stack_trace.h](#)

6.69 stack_return Struct Reference

Collected stack-return-address array and its populated length.

```
#include <stack_trace.h>
```

Data Fields

- `uint64_t *` [addresses](#)
- `uint64_t` [array_size](#)

6.69.1 Detailed Description

Collected stack-return-address array and its populated length.

Definition at line 20 of file [stack_trace.h](#).

6.69.2 Field Documentation

6.69.2.1 addresses

```
uint64_t* stack_return::addresses
```

Definition at line 22 of file [stack_trace.h](#).

6.69.2.2 array_size

```
uint64_t stack_return::array_size
```

Definition at line 23 of file [stack_trace.h](#).

The documentation for this struct was generated from the following file:

- [src/include/stack_trace.h](#)

6.70 term_context Struct Reference

Terminal parser state plus backend/client callback table.

```
#include <term.h>
```

Data Fields

- size_t `tab_size`
- bool `autoflush`
- bool `scroll_enabled`
- bool `control_sequence`
- bool `csi`
- bool `escape`
- bool `rrr`
- bool `discard_next`
- bool `bold`
- bool `reverse_video`
- bool `dec_private`
- bool `insert_mode`
- uint8_t `g_select`
- uint8_t `charsets` [2]
- size_t `current_charset`
- size_t `escape_offset`
- size_t `esc_values_i`
- size_t `saved_cursor_x`
- size_t `saved_cursor_y`
- size_t `current_primary`
- size_t `scroll_top_margin`
- size_t `scroll_bottom_margin`
- uint32_t `esc_values` [TERM_MAX_ESC_VALUES]
- bool `saved_state_bold`
- bool `saved_state_reverse_video`
- size_t `saved_state_current_charset`
- size_t `saved_state_current_primary`
- size_t `rows`
- size_t `cols`
- bool `in_bootloader`
- void(* `raw_putchar`)(struct `term_context` *, uint8_t c)
- void(* `clear`)(struct `term_context` *, bool move)
- void(* `enable_cursor`)(struct `term_context` *)
- bool(* `disable_cursor`)(struct `term_context` *)
- void(* `set_cursor_pos`)(struct `term_context` *, size_t x, size_t y)
- void(* `get_cursor_pos`)(struct `term_context` *, size_t *x, size_t *y)
- void(* `set_text_fg`)(struct `term_context` *, size_t fg)
- void(* `set_text_bg`)(struct `term_context` *, size_t bg)
- void(* `set_text_fg_bright`)(struct `term_context` *, size_t fg)
- void(* `set_text_bg_bright`)(struct `term_context` *, size_t bg)
- void(* `set_text_fg_rgb`)(struct `term_context` *, uint32_t fg)
- void(* `set_text_bg_rgb`)(struct `term_context` *, uint32_t bg)
- void(* `set_text_fg_default`)(struct `term_context` *)
- void(* `set_text_bg_default`)(struct `term_context` *)
- void(* `move_character`)(struct `term_context` *, size_t new_x, size_t new_y, size_t old_x, size_t old_y)

- void(* scroll)(struct term_context *)
- void(* revscroll)(struct term_context *)
- void(* swap_palette)(struct term_context *)
- void(* save_state)(struct term_context *)
- void(* restore_state)(struct term_context *)
- void(* double_buffer_flush)(struct term_context *)
- void(* full_refresh)(struct term_context *)
- void(* deinit)(struct term_context *, void (*)(void *, size_t))
- void(* callback)(struct term_context *, uint64_t, uint64_t, uint64_t, uint64_t)

6.70.1 Detailed Description

Terminal parser state plus backend/client callback table.

The first portion tracks parser, cursor, charset, scroll-region, and saved terminal state. A backend such as fbterm fills the rendering callbacks; an optional client callback receives higher-level terminal control events.

VoyagerOS64 0.0.5 uses this abstraction after the CR3 handoff so interactive terminal output is owned by kernel code rather than Limine's terminal callback implementation.

Definition at line 35 of file `term.h`.

6.70.2 Field Documentation

6.70.2.1 autoflush

```
bool term_context::autoflush
```

Definition at line 40 of file `term.h`.

6.70.2.2 bold

```
bool term_context::bold
```

Definition at line 47 of file `term.h`.

6.70.2.3 callback

```
void(* term_context::callback) (struct term_context *, uint64_t, uint64_t, uint64_t, uint64_t)
```

Definition at line 102 of file `term.h`.

6.70.2.4 charsets

```
uint8_t term_context::charsets[2]
```

Definition at line 52 of file [term.h](#).

6.70.2.5 clear

```
void(* term_context::clear) (struct term_context *, bool move)
```

Definition at line 73 of file [term.h](#).

6.70.2.6 cols

```
size_t term_context::cols
```

Definition at line 69 of file [term.h](#).

6.70.2.7 control_sequence

```
bool term_context::control_sequence
```

Definition at line 42 of file [term.h](#).

6.70.2.8 csi

```
bool term_context::csi
```

Definition at line 43 of file [term.h](#).

6.70.2.9 current_charset

```
size_t term_context::current_charset
```

Definition at line 53 of file [term.h](#).

6.70.2.10 current_primary

```
size_t term_context::current_primary
```

Definition at line 58 of file [term.h](#).

6.70.2.11 dec_private

```
bool term_context::dec_private
```

Definition at line 49 of file [term.h](#).

6.70.2.12 deinit

```
void(* term_context::deinit) (struct term_context *, void(*) (void *, size_t))
```

Definition at line 98 of file [term.h](#).

6.70.2.13 disable_cursor

```
bool(* term_context::disable_cursor) (struct term_context *)
```

Definition at line 75 of file [term.h](#).

6.70.2.14 discard_next

```
bool term_context::discard_next
```

Definition at line 46 of file [term.h](#).

6.70.2.15 double_buffer_flush

```
void(* term_context::double_buffer_flush) (struct term_context *)
```

Definition at line 96 of file [term.h](#).

6.70.2.16 enable_cursor

```
void(* term_context::enable_cursor) (struct term_context *)
```

Definition at line 74 of file [term.h](#).

6.70.2.17 esc_values

```
uint32_t term_context::esc_values[TERM_MAX_ESC_VALUES]
```

Definition at line 61 of file [term.h](#).

6.70.2.18 esc_values_i

```
size_t term_context::esc_values_i
```

Definition at line 55 of file [term.h](#).

6.70.2.19 escape

```
bool term_context::escape
```

Definition at line 44 of file [term.h](#).

6.70.2.20 escape_offset

```
size_t term_context::escape_offset
```

Definition at line 54 of file [term.h](#).

6.70.2.21 full_refresh

```
void(* term_context::full_refresh) (struct term_context *)
```

Definition at line 97 of file [term.h](#).

6.70.2.22 g_select

```
uint8_t term_context::g_select
```

Definition at line 51 of file [term.h](#).

6.70.2.23 get_cursor_pos

```
void(* term_context::get_cursor_pos) (struct term_context *, size_t *x, size_t *y)
```

Definition at line 77 of file [term.h](#).

6.70.2.24 in_bootloader

```
bool term_context::in_bootloader
```

Definition at line 70 of file [term.h](#).

6.70.2.25 insert_mode

```
bool term_context::insert_mode
```

Definition at line 50 of file [term.h](#).

6.70.2.26 move_character

```
void(* term_context::move_character) (struct term_context *, size_t new_x, size_t new_y, size_t old_x, size_t old_y)
```

Definition at line 86 of file [term.h](#).

6.70.2.27 raw_putchar

```
void(* term_context::raw_putchar) (struct term_context *, uint8_t c)
```

Definition at line 72 of file [term.h](#).

6.70.2.28 restore_state

```
void(* term_context::restore_state) (struct term_context *)
```

Definition at line 95 of file [term.h](#).

6.70.2.29 reverse_video

```
bool term_context::reverse_video
```

Definition at line 48 of file [term.h](#).

6.70.2.30 revscroll

```
void(* term_context::revscroll) (struct term_context *)
```

Definition at line 92 of file [term.h](#).

6.70.2.31 rows

```
size_t term_context::rows
```

Definition at line 69 of file [term.h](#).

6.70.2.32 rrr

```
bool term_context::rrr
```

Definition at line 45 of file [term.h](#).

6.70.2.33 save_state

```
void(* term_context::save_state) (struct term_context *)
```

Definition at line 94 of file [term.h](#).

6.70.2.34 saved_cursor_x

```
size_t term_context::saved_cursor_x
```

Definition at line 56 of file [term.h](#).

6.70.2.35 saved_cursor_y

```
size_t term_context::saved_cursor_y
```

Definition at line 57 of file [term.h](#).

6.70.2.36 saved_state_bold

```
bool term_context::saved_state_bold
```

Definition at line 62 of file [term.h](#).

6.70.2.37 saved_state_current_charset

```
size_t term_context::saved_state_current_charset
```

Definition at line 64 of file [term.h](#).

6.70.2.38 saved_state_current_primary

```
size_t term_context::saved_state_current_primary
```

Definition at line 65 of file [term.h](#).

6.70.2.39 saved_state_reverse_video

```
bool term_context::saved_state_reverse_video
```

Definition at line 63 of file [term.h](#).

6.70.2.40 scroll

```
void(* term_context::scroll) (struct term_context *)
```

Definition at line 91 of file `term.h`.

6.70.2.41 scroll_bottom_margin

```
size_t term_context::scroll_bottom_margin
```

Definition at line 60 of file `term.h`.

6.70.2.42 scroll_enabled

```
bool term_context::scroll_enabled
```

Definition at line 41 of file `term.h`.

6.70.2.43 scroll_top_margin

```
size_t term_context::scroll_top_margin
```

Definition at line 59 of file `term.h`.

6.70.2.44 set_cursor_pos

```
void(* term_context::set_cursor_pos) (struct term_context *, size_t x, size_t y)
```

Definition at line 76 of file `term.h`.

6.70.2.45 set_text_bg

```
void(* term_context::set_text_bg) (struct term_context *, size_t bg)
```

Definition at line 79 of file `term.h`.

6.70.2.46 set_text_bg_bright

```
void(* term_context::set_text_bg_bright) (struct term_context *, size_t bg)
```

Definition at line 81 of file [term.h](#).

6.70.2.47 set_text_bg_default

```
void(* term_context::set_text_bg_default) (struct term_context *)
```

Definition at line 85 of file [term.h](#).

6.70.2.48 set_text_bg_rgb

```
void(* term_context::set_text_bg_rgb) (struct term_context *, uint32_t bg)
```

Definition at line 83 of file [term.h](#).

6.70.2.49 set_text_fg

```
void(* term_context::set_text_fg) (struct term_context *, size_t fg)
```

Definition at line 78 of file [term.h](#).

6.70.2.50 set_text_fg_bright

```
void(* term_context::set_text_fg_bright) (struct term_context *, size_t fg)
```

Definition at line 80 of file [term.h](#).

6.70.2.51 set_text_fg_default

```
void(* term_context::set_text_fg_default) (struct term_context *)
```

Definition at line 84 of file [term.h](#).

6.70.2.52 set_text_fg_rgb

```
void(* term_context::set_text_fg_rgb) (struct term_context *, uint32_t fg)
```

Definition at line 82 of file [term.h](#).

6.70.2.53 swap_palette

```
void(* term_context::swap_palette) (struct term_context *)
```

Definition at line 93 of file [term.h](#).

6.70.2.54 tab_size

```
size_t term_context::tab_size
```

Definition at line 39 of file [term.h](#).

The documentation for this struct was generated from the following file:

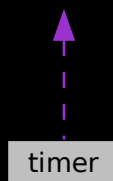
- [src/include/terminal/term.h](#)

6.71 timer Struct Reference

Small software-timer record used by the current timer layer.

```
#include <time.h>
```

Collaboration diagram for timer:



Data Fields

- `ssize_t` `index`
- `bool` `fired`
- `struct timespec` `when`

6.71.1 Detailed Description

Small software-timer record used by the current timer layer.

Definition at line 30 of file `time.h`.

6.71.2 Field Documentation

6.71.2.1 `fired`

```
bool timer::fired
```

Definition at line 33 of file `time.h`.

6.71.2.2 `index`

```
ssize_t timer::index
```

Definition at line 32 of file `time.h`.

6.71.2.3 `when`

```
struct timespec timer::when
```

Definition at line 34 of file `time.h`.

The documentation for this struct was generated from the following file:

- `src/include/time.h`

6.72 timespec Struct Reference

Seconds/nanoseconds time representation used by kernel timers.

```
#include <time.h>
```

Data Fields

- `time_t tv_sec`
- `long tv_nsec`

6.72.1 Detailed Description

Seconds/nanoseconds time representation used by kernel timers.

Definition at line 23 of file `time.h`.

6.72.2 Field Documentation

6.72.2.1 tv_nsec

```
long timespec::tv_nsec
```

Definition at line 26 of file `time.h`.

6.72.2.2 tv_sec

```
time_t timespec::tv_sec
```

Definition at line 25 of file `time.h`.

The documentation for this struct was generated from the following file:

- `src/include/time.h`

6.73 TSS Struct Reference

Packed architectural x86-64 Task State Segment.

```
#include <tss.h>
```

Data Fields

- uint32_t reserved0
- uint64_t rsp0
- uint64_t rsp1
- uint64_t rsp2
- uint64_t reserved1
- uint64_t ist1
- uint64_t ist2
- uint64_t ist3
- uint64_t ist4
- uint64_t ist5
- uint64_t ist6
- uint64_t ist7
- uint64_t reserved2
- uint16_t reserved3
- uint16_t iopbOffset

6.73.1 Detailed Description

Packed architectural x86-64 Task State Segment.

The RSP fields provide privilege-level stack pointers and the IST fields provide dedicated interrupt stacks when configured by the IDT. The I/O map base occupies the final architectural field.

Note

The presence of a hardware TSS does not imply Voyager uses hardware task switching; the 0.0.5 scheduler performs its own cooperative stack switch in software.

Definition at line 24 of file `tss.h`.

6.73.2 Field Documentation

6.73.2.1 iopbOffset

```
uint16_t TSS::iopbOffset
```

Definition at line 40 of file `tss.h`.

6.73.2.2 ist1

```
uint64_t TSS::ist1
```

Definition at line 31 of file `tss.h`.

6.73.2.3 ist2

```
uint64_t TSS::ist2
```

Definition at line 32 of file [tss.h](#).

6.73.2.4 ist3

```
uint64_t TSS::ist3
```

Definition at line 33 of file [tss.h](#).

6.73.2.5 ist4

```
uint64_t TSS::ist4
```

Definition at line 34 of file [tss.h](#).

6.73.2.6 ist5

```
uint64_t TSS::ist5
```

Definition at line 35 of file [tss.h](#).

6.73.2.7 ist6

```
uint64_t TSS::ist6
```

Definition at line 36 of file [tss.h](#).

6.73.2.8 ist7

```
uint64_t TSS::ist7
```

Definition at line 37 of file [tss.h](#).

6.73.2.9 reserved0

```
uint32_t TSS::reserved0
```

Definition at line 26 of file [tss.h](#).

6.73.2.10 reserved1

```
uint64_t TSS::reserved1
```

Definition at line 30 of file [tss.h](#).

6.73.2.11 reserved2

```
uint64_t TSS::reserved2
```

Definition at line 38 of file [tss.h](#).

6.73.2.12 reserved3

```
uint16_t TSS::reserved3
```

Definition at line 39 of file [tss.h](#).

6.73.2.13 rsp0

```
uint64_t TSS::rsp0
```

Definition at line 27 of file [tss.h](#).

6.73.2.14 rsp1

```
uint64_t TSS::rsp1
```

Definition at line 28 of file [tss.h](#).

6.73.2.15 rsp2

```
uint64_t TSS::rsp2
```

Definition at line 29 of file [tss.h](#).

The documentation for this struct was generated from the following file:

- [src/include/tss.h](#)

6.74 TSS_Entry Struct Reference

64-bit [TSS](#) system descriptor split into architectural fields.

```
#include <gdt.h>
```

Data Fields

- [uint16_t](#) [length](#)
- [uint16_t](#) [base_low](#)
- [uint8_t](#) [base_mid](#)
- [uint8_t](#) [flags](#)
- [uint8_t](#) [flags2](#)
- [uint8_t](#) [base_high](#)
- [uint32_t](#) [base_up](#)
- [uint32_t](#) [reserved0](#)

6.74.1 Detailed Description

64-bit [TSS](#) system descriptor split into architectural fields.

Definition at line 83 of file [gdt.h](#).

6.74.2 Field Documentation

6.74.2.1 base_high

```
uint8_t TSS_Entry::base_high
```

Definition at line 90 of file [gdt.h](#).

6.74.2.2 base_low

`uint16_t TSS_Entry::base_low`

Definition at line 86 of file [gdt.h](#).

6.74.2.3 base_mid

`uint8_t TSS_Entry::base_mid`

Definition at line 87 of file [gdt.h](#).

6.74.2.4 base_up

`uint32_t TSS_Entry::base_up`

Definition at line 91 of file [gdt.h](#).

6.74.2.5 flags

`uint8_t TSS_Entry::flags`

Definition at line 88 of file [gdt.h](#).

6.74.2.6 flags2

`uint8_t TSS_Entry::flags2`

Definition at line 89 of file [gdt.h](#).

6.74.2.7 length

`uint16_t TSS_Entry::length`

Definition at line 85 of file [gdt.h](#).

6.74.2.8 reserved0

`uint32_t TSS_Entry::reserved0`

Definition at line 92 of file [gdt.h](#).

The documentation for this struct was generated from the following file:

- [src/include/gdt.h](#)

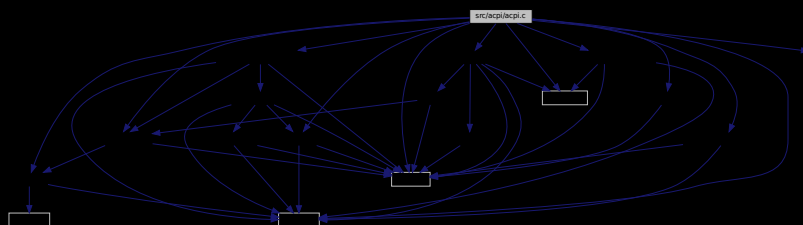
Chapter 7

File Documentation

7.1 src/acpi/acpi.c File Reference

```
#include "../include/acpi/acpi.h"
#include "../include/acpi/madt.h"
#include "../include/cpuUtils.h"
#include "../include/global_defs.h"
#include "../include/limine.h"
#include "../include/paging/paging.h"
#include "../include/panic.h"
#include "../include/printf.h"
#include "../include/string.h"
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
```

Include dependency graph for acpi.c:



Data Structures

- struct `rsdp`
- struct `rsdt`

Functions

- void `halt` ()
- static bool `use_xsdt` (void)
Check whether the ACPI revision/table pointers select the XSDT path.
- void `acpi_init` (void)
Initialize Voyager's ACPI table-discovery state.
- void * `acpi_find_sdt` (const char signature[static 4], size_t index)
Locate an ACPI SDT by four-byte signature and occurrence index.

Here is the caller graph for this function:



7.1.1.2 acpi_init()

```
void acpi_init (
                void )
```

Initialize Voyager's ACPI table-discovery state.

Called during kernel bring-up when CPUID/platform probing reports ACPI support. Mandatory-table failures are routed through the kernel panic/error path.

Definition at line 56 of file acpi.c.

Here is the call graph for this function:

Here is the caller graph for this function:



7.1.2.3 rsdt

```
struct rsdt* rsdt = NULL [static]
```

Definition at line 44 of file `acpi.c`.

7.1.2.4 sdt

```
struct sdt* sdt
```

Definition at line 22 of file `acpi.c`.

7.2 acpi.c

[Go to the documentation of this file.](#)

```
00001 #include "../include/acpi/acpi.h"
00002 #include "../include/acpi/madt.h"
00003 #include "../include/cpuUtils.h"
00004 #include "../include/global_defs.h"
00005 #include "../include/limine.h"
00006 #include "../include/paging/paging.h"
00007 #include "../include/panic.h"
00008 #include "../include/printf.h"
00009 #include "../include/string.h"
00010 #include <stdbool.h>
00011 #include <stddef.h>
00012 #include <stdint.h>
00013
00014 extern void halt();
00015
00016 static volatile struct limine_rsdp_request rsdp_req = {
00017     .id = LIMINE_RSDP_REQUEST, .revision = 0
00018 };
00019
00020 };
00021
00022 struct sdt *sdt;
00023
00024 struct rsdp
00025 {
00026     char    signature[8];
00027     uint8_t checksum;
00028     char    oem_id[6];
00029     uint8_t revision;
00030     uint32_t rsdt_addr;
00031     uint32_t length;
00032     uint64_t xsdt_addr;
00033     uint8_t ext_checksum;
00034     char    reserved[3];
00035 };
00036
00037 struct rsdt
00038 {
00039     struct sdt;
00040     char data[];
00041 };
00042
00043 static struct rsdp *rsdp = NULL;
00044 static struct rsdt *rsdt = NULL;
00045
00050 static inline bool use_xsdt(void)
00051 {
00052     return rsdp->revision >= 2 && rsdp->xsdt_addr != 0;
00053 }
00054
00055 /* Public API documentation lives in include/acpi/acpi.h. */
00056 void acpi_init(void)
00057 {
00058     struct limine_rsdp_response *rsdp_resp = rsdp_req.response;
```

```

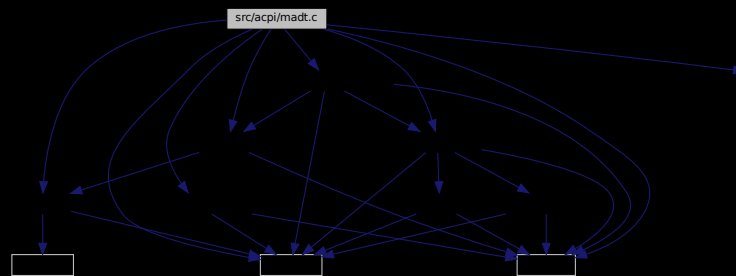
00059
00060     if (rsdp_resp == NULL || rsdp_resp->address == NULL)
00061     {
00062         panic("ACPI Not Supported by This Machine!");
00063     }
00064     else if (has_ACPI == false)
00065     {
00066         panic("ACPI Not Supported by This Machine!");
00067     }
00068
00069     rsdp = rsdp_resp->address;
00070
00071     if (use_xsdt())
00072     {
00073         rsdt = (struct rsdt *) (rsdp->xsdt_addr + HIGHER_HALF_MEMORY_OFFSET);
00074     }
00075     else
00076     {
00077         rsdt = (struct rsdt *) ((uint64_t) rsdp->rsdt_addr +
00078                                HIGHER_HALF_MEMORY_OFFSET);
00079     }
00080
00081     printf("acpi: Revision: %lu\n", rsdp->revision);
00082     printf("acpi: Uses XSDT? %s\n", use_xsdt() ? "true" : "false");
00083     printf("acpi: RSDT at 0x%llx\n", rsdt);
00084
00085     struct sdt *fadt = acpi_find_sdt("FACP", 0);
00086     if (fadt != NULL && fadt->length >= 116)
00087     {
00088         uint32_t fadt_flags = *((uint32_t *) fadt + 28);
00089
00090         if ((fadt_flags & (1 << 20)) != 0)
00091         {
00092             panic("VoyagerOS64 Does Not Support HW Reduced ACPI");
00093         }
00094     }
00095
00096     madt_init();
00097 }
00098
00099 void *acpi_find_sdt(const char signature[static 4], size_t index)
00100 {
00101     size_t entry_count =
00102         (rsdt->length - sizeof(struct rsdt)) / (use_xsdt() ? 8 : 4);
00103
00104     for (size_t i = 0; i < entry_count; i++)
00105     {
00106         struct sdt *sdt = NULL;
00107         if (use_xsdt())
00108         {
00109             sdt = (struct sdt *) (*((uint64_t *) rsdt->data + i) +
00110                                HIGHER_HALF_MEMORY_OFFSET);
00111         }
00112         else
00113         {
00114             sdt = (struct sdt *) (*((uint32_t *) rsdt->data + i) +
00115                                HIGHER_HALF_MEMORY_OFFSET);
00116         }
00117
00118         if (memcmp(sdt->signature, signature, 4) != 0)
00119         {
00120             continue;
00121         }
00122
00123         if (index > 0)
00124         {
00125             index--;
00126             continue;
00127         }
00128
00129         printf("acpi: Found '%S' at 0x%llx, length=%lu\n",
00130                signature,
00131                4,
00132                sdt,
00133                sdt->length);
00134         return sdt;
00135     }
00136
00137     printf("acpi: Could not find '%S'\n", signature, 4);
00138     return NULL;
00139 }

```


7.3 src/acpi/madt.c File Reference

```
#include "../include/acpi/madt.h"
#include "../include/acpi/acpi.h"
#include "../include/global_defs.h"
#include "../include/lib/vector.h"
#include "../include/panic.h"
#include "../include/printf.h"
#include <stddef.h>
#include <stdint.h>
```

Include dependency graph for madt.c:



Functions

- `typeof` (madt_lapics)
- void `madt_init` (void)
Initialize MADT by looking for APIC.

7.3.1 Function Documentation

7.3.1.1 madt_init()

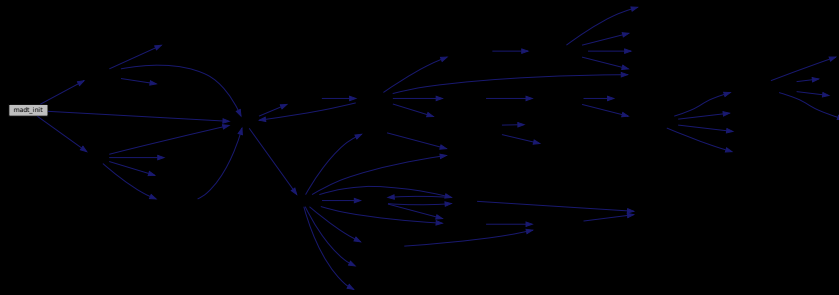
```
void madt_init (
    void )
```

Initialize MADT by looking for APIC.

Locate and parse the ACPI MADT into Voyager's topology vectors.

Definition at line 28 of file `madt.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.3.1.2 typeof()

```
typeof (
    madt_lapics )
```

Definition at line 10 of file `madt.c`.

7.4 madt.c

[Go to the documentation of this file.](#)

```
00001 #include "../include/acpi/madt.h"
00002 #include "../include/acpi/acpi.h"
00003 #include "../include/global_defs.h"
00004 #include "../include/lib/vector.h"
00005 #include "../include/panic.h"
00006 #include "../include/printf.h"
00007 #include <stddef.h>
00008 #include <stdint.h>
00009
00010 typeof(madt_lapics) madt_lapics = (typeof(madt_lapics)) VECTOR_INIT;
00011 typeof(madt_io_apics) madt_io_apics = (typeof(madt_io_apics)) VECTOR_INIT;
00012 typeof(madt_isos) madt_isos = (typeof(madt_isos)) VECTOR_INIT;
00013 typeof(madt_nmis) madt_nmis = (typeof(madt_nmis)) VECTOR_INIT;
00014
00015 extern void halt();
00016
00017 struct madt
00018 {
00019     struct sdt;
```

```

00020     uint32_t local_controller_addr;
00021     uint32_t flags;
00022     char     entries_data[];
00023 };
00024
00028 void madt_init(void)
00029 {
00030     struct madt *madt = acpi_find_sdt("APIC", 0);
00031     if (madt == NULL)
00032     {
00033         panic("System Does Not Have an MADT!");
00034     }
00035
00036     size_t offset = 0;
00037     for (;;)
00038     {
00039         if (madt->length - sizeof(struct madt) - offset < 2)
00040         {
00041             break;
00042         }
00043
00044         struct madt_header *header =
00045             (struct madt_header *) (madt->entries_data + offset);
00046         switch (header->id)
00047         {
00048             case 0:
00049                 printf("madt: Found local APIC #lu\n", madt_lapics.length);
00050                 VECTOR_PUSH_BACK(&madt_lapics, (struct madt_lapic *) header);
00051                 break;
00052             case 1:
00053                 printf("madt: Found IO APIC #lu\n", madt_io_apics.length);
00054                 VECTOR_PUSH_BACK(&madt_io_apics,
00055                     (struct madt_io_apic *) header);
00056                 break;
00057             case 2:
00058                 printf("madt: Found ISO #lu\n", madt_isos.length);
00059                 VECTOR_PUSH_BACK(&madt_isos, (struct madt_iso *) header);
00060                 break;
00061             case 4:
00062                 printf("madt: Found NMI #lu\n", madt_nmis.length);
00063                 VECTOR_PUSH_BACK(&madt_nmis, (struct madt_nmi *) header);
00064                 break;
00065         }
00066
00067         offset += MAX(header->length, 2);
00068     }
00069 }

```

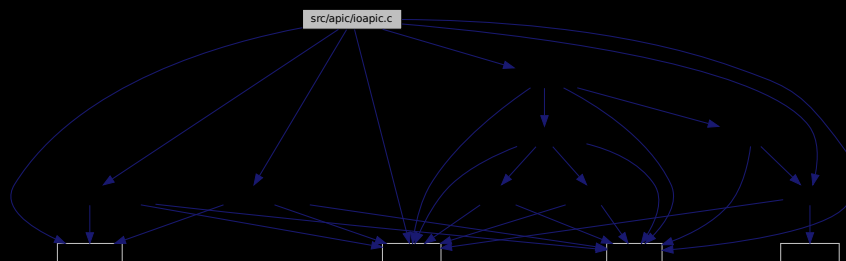
7.5 src/apic/ioapic.c File Reference

```

#include "../include/apic/ioapic.h"
#include "../include/acpi/madt.h"
#include "../include/paging/paging.h"
#include "../include/printf.h"
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

```

Include dependency graph for ioapic.c:



Functions

- void `halt` (void)
- static uint32_t `io_apic_read` (struct `madt_io_apic` *`io_apic`, uint32_t `reg`)
- static void `io_apic_write` (struct `madt_io_apic` *`io_apic`, uint32_t `reg`, uint32_t `value`)
- static size_t `io_apic_gsi_count` (struct `madt_io_apic` *`io_apic`)
- static struct `madt_io_apic` * `io_apic_from_gsi` (uint32_t `gsi`)
- void `io_apic_set_irq_redirect` (uint32_t `lapic_id`, uint8_t `vector`, uint8_t `irq`, bool `status`)
Program an IRQ redirection toward one Local APIC.
- void `io_apic_set_gsi_redirect` (uint32_t `lapic_id`, uint8_t `vector`, uint8_t `gsi`, uint16_t `flags`, bool `status`)
Program a GSI redirection toward one Local APIC.

7.5.1 Function Documentation

7.5.1.1 `halt()`

```
void halt (
    void )
```

Here is the caller graph for this function:

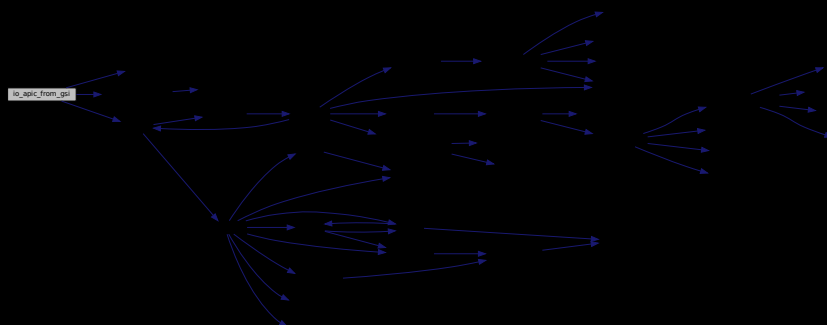


7.5.1.2 `io_apic_from_gsi()`

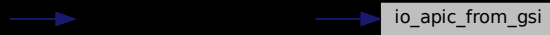
```
static struct madt_io_apic * io_apic_from_gsi (
    uint32_t gsi ) [static]
```

Definition at line 31 of file `ioapic.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

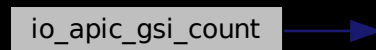


7.5.1.3 io_apic_gsi_count()

```
static size_t io_apic_gsi_count (
    struct madt_io_apic * io_apic ) [static]
```

Definition at line 26 of file `ioapic.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

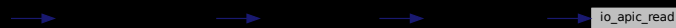


7.5.1.4 io_apic_read()

```
static uint32_t io_apic_read (
    struct madt_io_apic * io_apic,
    uint32_t reg ) [static]
```

Definition at line 11 of file `ioapic.c`.

Here is the caller graph for this function:



7.5.1.5 io_apic_set_gsi_redirect()

```
void io_apic_set_gsi_redirect (
    uint32_t lapic_id,
    uint8_t vector,
    uint8_t gsi,
    uint16_t flags,
    bool status )
```

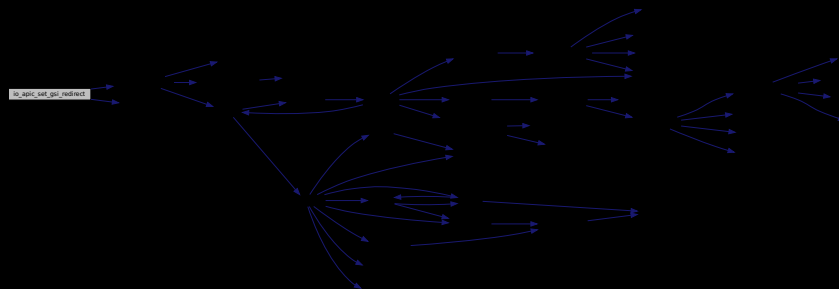
Program a GSI redirection toward one Local APIC.

Parameters

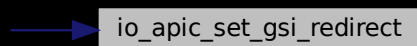
<i>lapic_id</i>	Destination Local APIC ID.
<i>vector</i>	Destination IDT vector.
<i>gsi</i>	Global System Interrupt number.
<i>flags</i>	ACPI/MADT polarity and trigger flags associated with the source.
<i>status</i>	Current implementation's enable/mask selector.

Definition at line 69 of file [ioapic.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.5.1.6 io_apic_set_irq_redirect()

```
void io_apic_set_irq_redirect (
    uint32_t lapic_id,
    uint8_t vector,
    uint8_t irq,
    bool status )
```

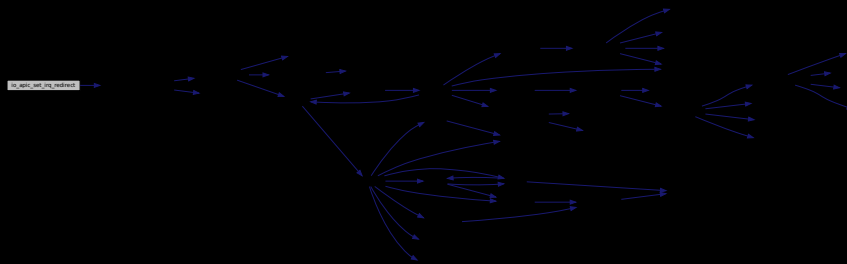
Program an IRQ redirection toward one Local APIC.

Parameters

<i>lapic_id</i>	Destination Local APIC ID.
<i>vector</i>	Destination IDT vector.
<i>irq</i>	Legacy IRQ/source number.
<i>status</i>	Current implementation's enable/mask selector.

Definition at line 50 of file [ioapic.c](#).

Here is the call graph for this function:

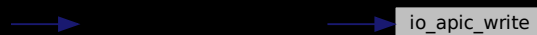


7.5.1.7 io_apic_write()

```
static void io_apic_write (
    struct madt_io_apic * io_apic,
    uint32_t reg,
    uint32_t value ) [static]
```

Definition at line 19 of file [ioapic.c](#).

Here is the caller graph for this function:



7.6 ioapic.c

Go to the documentation of this file.

```

00001 #include "../include/apic/ioapic.h"
00002 #include "../include/acpi/madt.h"
00003 #include "../include/paging/paging.h"
00004 #include "../include/printf.h"
00005 #include <stdbool.h>
00006 #include <stddef.h>
00007 #include <stdint.h>
00008
00009 extern void halt(void);
00010
00011 static uint32_t io_apic_read(struct madt_io_apic *io_apic, uint32_t reg)
00012 {
00013     uint64_t base = (uint64_t) io_apic->address + HIGHER_HALF_MEMORY_OFFSET;
00014     *(volatile uint32_t *) base = reg;
00015     return *(volatile uint32_t *) (base + 16);
00016 }
00017
00018 static void
00019 io_apic_write(struct madt_io_apic *io_apic, uint32_t reg, uint32_t value)
00020 {
00021     uint64_t base = (uint64_t) io_apic->address + HIGHER_HALF_MEMORY_OFFSET;
00022     *(volatile uint32_t *) base = reg;
00023     *(volatile uint32_t *) (base + 16) = value;
00024 }
00025
00026 static size_t io_apic_gsi_count(struct madt_io_apic *io_apic)
00027 {
00028     return (io_apic_read(io_apic, 1) & 0xff0000) >> 16;
00029 }
00030
00031 static struct madt_io_apic *io_apic_from_gsi(uint32_t gsi)
00032 {
00033     for (size_t i = 0; i < madt_io_apics.length; i++)
00034     {
00035         struct madt_io_apic *io_apic = VECTOR_ITEM(&madt_io_apics, i);
00036         if (gsi >= io_apic->gsib &&
00037             gsi < io_apic->gsib + io_apic_gsi_count(io_apic))
00038         {
00039             return io_apic;
00040         }
00041     }
00042
00043     printf("%s\n", "!!!!!!!!!!!!!!!!!!!!!!KERNEL PANIC!!!!!!!!!!!!!!!!!!!!");
00044     printf("Cannot determine IO APIC from GSI %lu\n", gsi);
00045     printf("%s\n", "!!!!!!!!!!!!!!!!!!!!!!KERNEL PANIC!!!!!!!!!!!!!!!!!!!!");
00046     halt();
00047     return NULL;
00048 }
00049
00050 void io_apic_set_irq_redirect(uint32_t lapic_id,
00051                               uint8_t vector,
00052                               uint8_t irq,
00053                               bool status)
00054 {
00055     for (size_t i = 0; i < madt_isos.length; i++)
00056     {
00057         struct madt_iso *iso = VECTOR_ITEM(&madt_isos, i);
00058         if (iso->irq_source != irq)
00059             continue;
00060
00061         io_apic_set_gsi_redirect(
00062             lapic_id, vector, iso->gsi, iso->flags, status);
00063         return;
00064     }
00065
00066     io_apic_set_gsi_redirect(lapic_id, vector, irq, 0, status);
00067 }
00068
00069 void io_apic_set_gsi_redirect(
00070     uint32_t lapic_id, uint8_t vector, uint8_t gsi, uint16_t flags, bool status)
00071 {
00072     struct madt_io_apic *io_apic = io_apic_from_gsi(gsi);
00073
00074     uint64_t redirect = vector;
00075     if ((flags & (1 << 1)) != 0)
00076         redirect |= (1 << 13);
00077
00078     if ((flags & (1 << 3)) != 0)
00079         redirect |= (1 << 15);
00080
00081     if (!status)
00082         redirect |= (1 << 16);

```



```

00083
00084     redirect |= (uint64_t) lapic_id « 56;
00085
00086     uint32_t io_redirect_table = (gsi - io_apic->gsib) * 2 + 16;
00087     io_apic_write(io_apic, io_redirect_table, (uint32_t) redirect);
00088     io_apic_write(io_apic, io_redirect_table + 1, (uint32_t) (redirect » 32));
00089 }

```

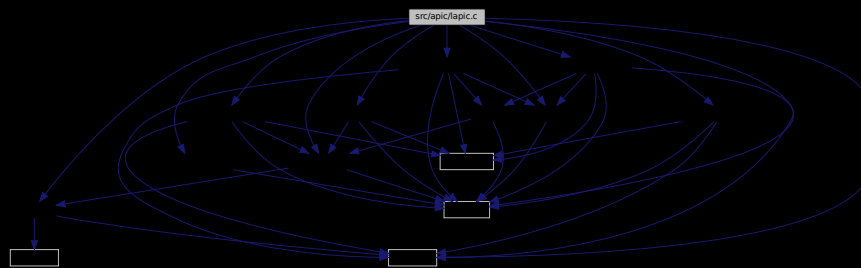
7.7 src/apic/lapic.c File Reference

```

#include "../include/apic/lapic.h"
#include "../include/cpu.h"
#include "../include/cpuUtils.h"
#include "../include/global_defs.h"
#include "../include/idt.h"
#include "../include/paging/paging.h"
#include "../include/printf.h"
#include "../include/registers.h"
#include "../include/time.h"
#include <stddef.h>
#include <stdint.h>

```

Include dependency graph for lapic.c:



Macros

- `#define LAPIC_REG_ID 0x20`
- `#define LAPIC_REG_EOI 0x0b0`
- `#define LAPIC_REG_SPURIOUS 0x0f0`
- `#define LAPIC_REG_CMCI 0x2f0`
- `#define LAPIC_REG_ICR0 0x300`
- `#define LAPIC_REG_ICR1 0x310`
- `#define LAPIC_REG_LVT_TIMER 0x320`
- `#define LAPIC_REG_TIMER_INITCNT 0x380`
- `#define LAPIC_REG_TIMER_CURCNT 0x390`
- `#define LAPIC_REG_TIMER_DIV 0x3e0`
- `#define LAPIC_EOI_ACK 0x00`

Functions

- static uint32_t `lapic_read` (uint32_t reg)
Read a Local APIC MMIO register through the higher-half mapping.
- static void `lapic_write` (uint32_t reg, uint32_t val)
Write a Local APIC MMIO register through the higher-half mapping.
- static void `lapic_timer_stop` (void)
Stop and mask the experimental LAPIC timer.
- static void `lapic_timer_handler` (int vector, struct `cpu_ctx` *ctx)
LAPIC timer interrupt handler.
- void `lapic_init` (void)
Initialize the current CPU's Local APIC support path.
- void `lapic_eoi` (void)
Send end-of-interrupt acknowledgement to the Local APIC.
- void `lapic_timer_oneshot` (uint32_t us, void *function)
Arm the experimental Local APIC timer in one-shot mode.
- void `lapic_timer_calibrate` (void)
Calibrate the Local APIC timer against the current timing source.
- uint32_t `lapic_get_id` (void)
Return the current processor's Local APIC ID.

Variables

- static uint8_t `timer_vec` = 0

7.7.1 Macro Definition Documentation

7.7.1.1 LAPIC_EOI_ACK

```
#define LAPIC_EOI_ACK 0x00
```

Definition at line 23 of file `lapic.c`.

7.7.1.2 LAPIC_REG_CMCI

```
#define LAPIC_REG_CMCI 0x2f0
```

Definition at line 16 of file `lapic.c`.

7.7.1.3 LAPIC_REG_EOI

```
#define LAPIC_REG_EOI 0x0b0
```

Definition at line 14 of file [lapic.c](#).

7.7.1.4 LAPIC_REG_ICR0

```
#define LAPIC_REG_ICR0 0x300
```

Definition at line 17 of file [lapic.c](#).

7.7.1.5 LAPIC_REG_ICR1

```
#define LAPIC_REG_ICR1 0x310
```

Definition at line 18 of file [lapic.c](#).

7.7.1.6 LAPIC_REG_ID

```
#define LAPIC_REG_ID 0x20
```

Definition at line 13 of file [lapic.c](#).

7.7.1.7 LAPIC_REG_LVT_TIMER

```
#define LAPIC_REG_LVT_TIMER 0x320
```

Definition at line 19 of file [lapic.c](#).

7.7.1.8 LAPIC_REG_SPURIOUS

```
#define LAPIC_REG_SPURIOUS 0x0f0
```

Definition at line 15 of file [lapic.c](#).

7.7.1.9 LAPIC_REG_TIMER_CURCNT

```
#define LAPIC_REG_TIMER_CURCNT 0x390
```

Definition at line 21 of file [lapic.c](#).

7.7.1.10 LAPIC_REG_TIMER_DIV

```
#define LAPIC_REG_TIMER_DIV 0x3e0
```

Definition at line 22 of file [lapic.c](#).

7.7.1.11 LAPIC_REG_TIMER_INITCNT

```
#define LAPIC_REG_TIMER_INITCNT 0x380
```

Definition at line 20 of file [lapic.c](#).

7.7.2 Function Documentation

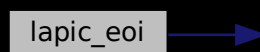
7.7.2.1 `lapic_eoi()`

```
void lapic_eoi (  
    void )
```

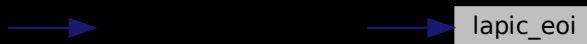
Send end-of-interrupt acknowledgement to the Local APIC.

Definition at line 99 of file [lapic.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.7.2.2 lapic_get_id()

```
uint32_t lapic_get_id (
    void )
```

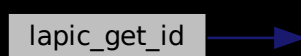
Return the current processor's Local APIC ID.

Returns

Local APIC identifier reported by the current implementation.

Definition at line 138 of file lapic.c.

Here is the call graph for this function:



7.7.2.3 lapic_init()

```
void lapic_init (
                void )
```

Initialize the current CPU's Local APIC support path.

Definition at line 71 of file lapic.c.

Here is the call graph for this function:

7.7.2.4 lapic_read()

```
static uint32_t lapic_read (
    uint32_t reg ) [inline], [static]
```

Read a Local APIC MMIO register through the higher-half mapping.

7.7.2.7 lapic_timer_oneshot()

```
void lapic_timer_oneshot (
    uint32_t us,
    void * function )
```

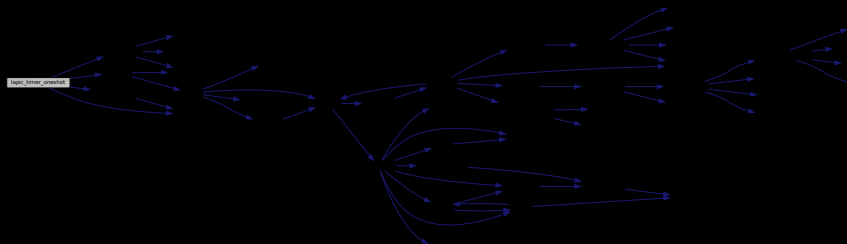
Arm the experimental Local APIC timer in one-shot mode.

Parameters

<i>us</i>	Requested delay in microseconds.
<i>function</i>	Callback pointer stored for the timer interrupt path.

Definition at line 104 of file [lapic.c](#).

Here is the call graph for this function:



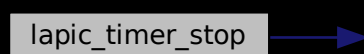
7.7.2.8 lapic_timer_stop()

```
static void lapic_timer_stop (
    void ) [inline], [static]
```

Stop and mask the experimental LAPIC timer.

Definition at line 50 of file [lapic.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.7.2.9 lapic_write()

```
static void lapic_write (
    uint32_t reg,
    uint32_t val ) [inline], [static]
```

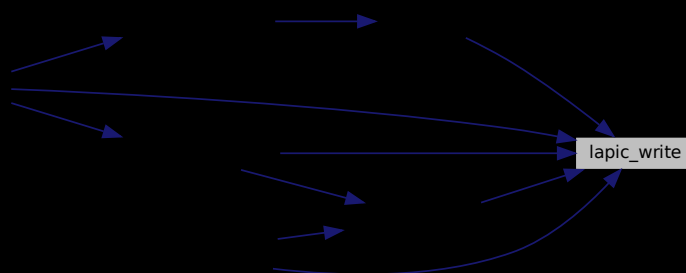
Write a Local APIC MMIO register through the higher-half mapping.

Parameters

<i>reg</i>	Byte offset of the LAPIC register.
<i>val</i>	32-bit value to write.

Definition at line 43 of file `lapic.c`.

Here is the caller graph for this function:



7.7.3 Variable Documentation

7.7.3.1 timer_vec

```
uint8_t timer_vec = 0 [static]
```

Definition at line 25 of file [lapic.c](#).

7.8 lapic.c

Go to the documentation of this file.

```
00001 #include "../include/apic/lapic.h"
00002 #include "../include/cpu.h"
00003 #include "../include/cpuUtils.h"
00004 #include "../include/global_defs.h"
00005 #include "../include/idt.h"
00006 #include "../include/paging/paging.h"
00007 #include "../include/printf.h"
00008 #include "../include/registers.h"
00009 #include "../include/time.h"
00010 #include <stddef.h>
00011 #include <stdint.h>
00012
00013 #define LAPIC_REG_ID 0x20 // LAPIC ID
00014 #define LAPIC_REG_EOI 0xb0 // End of interrupt
00015 #define LAPIC_REG_SPURIOUS 0xf0
00016 #define LAPIC_REG_CMCI 0x2f0 // LVT Corrected machine check interrupt
00017 #define LAPIC_REG_ICR0 0x300 // Interrupt command register
00018 #define LAPIC_REG_ICR1 0x310
00019 #define LAPIC_REG_LVT_TIMER 0x320
00020 #define LAPIC_REG_TIMER_INITCNT 0x380 // Initial count register
00021 #define LAPIC_REG_TIMER_CURCNT 0x390 // Current count register
00022 #define LAPIC_REG_TIMER_DIV 0x3e0
00023 #define LAPIC_EOI_ACK 0x00
00024
00025 static uint8_t timer_vec = 0;
00026
00032 static inline uint32_t lapic_read(uint32_t reg)
00033 {
00034     return *((volatile uint32_t *) ((uintptr_t) 0xf0000000 +
00035                                     HIGHER_HALF_MEMORY_OFFSET + reg));
00036 }
00037
00043 static inline void lapic_write(uint32_t reg, uint32_t val)
00044 {
00045     *((volatile uint32_t *) ((uintptr_t) 0xf0000000 +
00046                             HIGHER_HALF_MEMORY_OFFSET + reg)) = val;
00047 }
00048
00050 static inline void lapic_timer_stop(void)
00051 {
00052     lapic_write(LAPIC_REG_TIMER_INITCNT, 0);
00053     lapic_write(LAPIC_REG_LVT_TIMER, 1 << 16);
00054 }
00055
00061 static void lapic_timer_handler(int vector, struct cpu_ctx *ctx)
00062 {
00063     lapic_eoi();
00064     if (this_cpu()->timer_function != NULL)
00065     {
00066         this_cpu()->timer_function(vector, ctx);
00067     }
00068 }
00069
00070 /* Public API documentation lives in include/apic/lapic.h. */
00071 void lapic_init(void)
00072 {
00073     ASSERT((rdmsr(0x1b) & 0xfffff000) == 0xf0000000);
00074
00075     // Configure spurious IRQ
00076     lapic_write(LAPIC_REG_SPURIOUS,
00077                lapic_read(LAPIC_REG_SPURIOUS) | (1 << 8) | 0xff);
00078
00079     // Timer interrupt
00080     if (timer_vec == 0)
00081     {
00082         timer_vec = idt_allocate_vector();
00083         idt_set_descriptor(
00084             timer_vec, isr_stub_table[timer_vec], IDT_DESCRIPTOR_EXTERNAL, 001);
00085     }
00086 }
```

```

00086     isr_delta[timer_vec] = lapic_timer_handler;
00087
00088     printf("%s", "Allocated Dynamic ISR at Vector: ");
00089     printf("%i\n", timer_vec);
00090     printf("%s", "ISR Delta Data: ");
00091     printf("0x%llx\n", isr_delta[timer_vec]);
00092
00093     lapic_write(LAPIC_REG_LVT_TIMER,
00094                 lapic_read(LAPIC_REG_LVT_TIMER) | (1 << 8) | timer_vec);
00095
00096     lapic_timer_calibrate();
00097 }
00098
00099 void lapic_eoi(void)
00100 {
00101     lapic_write(LAPIC_REG_EOI, LAPIC_EOI_ACK);
00102 }
00103
00104 void lapic_timer_oneShot(uint32_t us, void *function)
00105 {
00106     lapic_timer_stop();
00107
00108     bool old_int_state = interrupt_toggle(false);
00109     this_cpu()->timer_function = function;
00110     interrupt_toggle(old_int_state);
00111
00112     uint32_t ticks = us * (this_cpu()->lapic_freq / 1000000);
00113     lapic_write(LAPIC_REG_LVT_TIMER, timer_vec);
00114     lapic_write(LAPIC_REG_TIMER_DIV, 0);
00115     lapic_write(LAPIC_REG_TIMER_INITCNT, ticks);
00116 }
00117
00118 void lapic_timer_calibrate(void)
00119 {
00120     lapic_timer_stop();
00121
00122     // Initialize PIT
00123     lapic_write(LAPIC_REG_LVT_TIMER, (1 << 16) | 0xff);
00124     lapic_write(LAPIC_REG_TIMER_DIV, 0);
00125     pit_set_reload_value(0xffff); // Reset PIT
00126
00127     int init_tick = pit_get_current_count();
00128     int samples = 0xffffffff;
00129     lapic_write(LAPIC_REG_TIMER_INITCNT, (uint32_t) samples);
00130     while (lapic_read(LAPIC_REG_TIMER_CURCNT) != 0)
00131         ;
00132     int final_tick = pit_get_current_count();
00133     int total_ticks = init_tick - final_tick;
00134     this_cpu()->lapic_freq = (samples / total_ticks) * PIT_DIVIDEND;
00135     lapic_timer_stop();
00136 }
00137
00138 uint32_t lapic_get_id(void)
00139 {
00140     return lapic_read(LAPIC_REG_ID);
00141 }

```

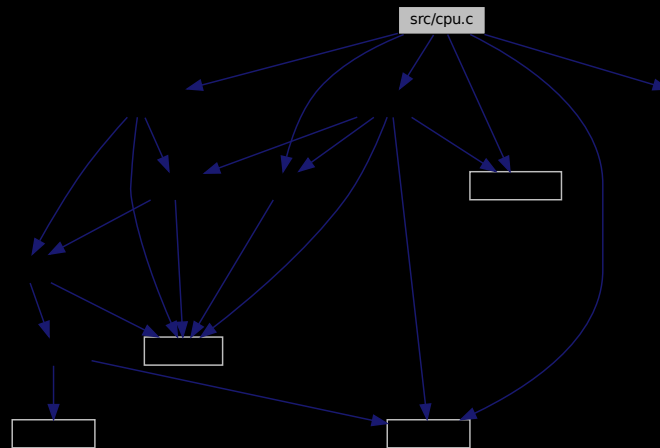
7.9 src/cpu.c File Reference

```

#include "include/cpu.h"
#include "include/gdt.h"
#include "include/panic.h"
#include "include/registers.h"
#include <stdbool.h>
#include <stddef.h>

```

Include dependency graph for `cpu.c`:



Functions

- struct `cpu_local` * `this_cpu` (void)
Return the current CPU-local record when that facility is available.

Variables

- bool `sysenter`
- size_t `fpu_bank_size`
- void(* `fpu_save`)(void *ctx) = NULL
- void(* `fpu_rest`)(void *ctx) = NULL

7.9.1 Function Documentation

7.9.1.1 `this_cpu()`

```
struct cpu_local * this_cpu (
    void )
```

Return the current CPU-local record when that facility is available.

Returns

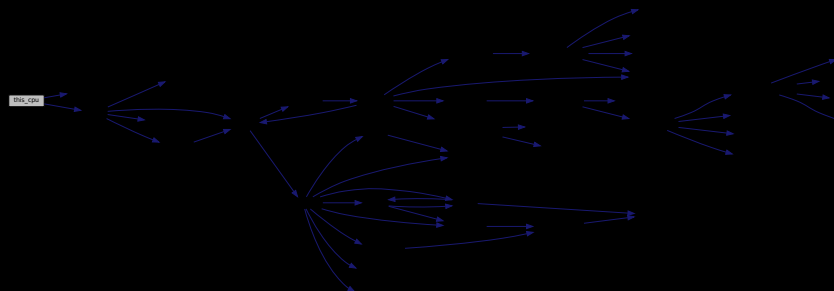
Pointer to CPU-local state, or the implementation's current fallback.

Note

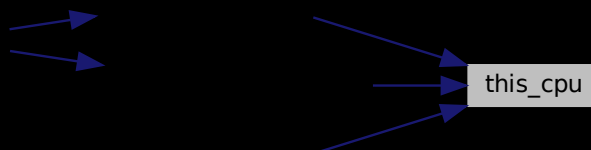
CPU-local/SMP state is not yet a qualified 0.0.5 scheduler facility.

Definition at line 14 of file `cpu.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.9.2 Variable Documentation

7.9.2.1 fpu_bank_size

`size_t fpu_bank_size`

Definition at line 10 of file `cpu.c`.

7.9.2.2 fpu_rest

```
void(* fpu_rest) (void *ctx) (  
    void * ctx ) = NULL
```

Definition at line 12 of file [cpu.c](#).

7.9.2.3 fpu_save

```
void(* fpu_save) (void *ctx) (  
    void * ctx ) = NULL
```

Definition at line 11 of file [cpu.c](#).

7.9.2.4 sysenter

```
bool sysenter
```

Definition at line 8 of file [cpu.c](#).

7.10 cpu.c

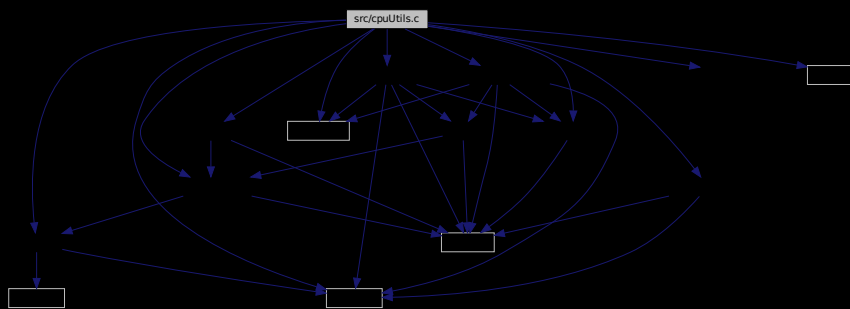
[Go to the documentation of this file.](#)

```
00001 #include "include/cpu.h"  
00002 #include "include/gdt.h"  
00003 #include "include/panic.h"  
00004 #include "include/registers.h"  
00005 #include <stdbool.h>  
00006 #include <stddef.h>  
00007  
00008 bool sysenter;  
00009  
00010 size_t fpu_bank_size;  
00011 void (*fpu_save)(void *ctx) = NULL;  
00012 void (*fpu_rest)(void *ctx) = NULL;  
00013  
00014 struct cpu_local *this_cpu(void)  
00015 {  
00016     if (interrupt_state())  
00017         panic("Calling this_cpu() With Interrupts on is Forbidden!");  
00018  
00019     return NULL;  
00020 }
```

7.11 src/cpuUtils.c File Reference

```
#include "include/KernelUtils.h"
#include "include/cpu.h"
#include "include/cpuUtils.h"
#include "include/global_defs.h"
#include "include/mm/kmalloc.h"
#include "include/panic.h"
#include "include/printf.h"
#include "include/registers.h"
#include <cpuid.h>
#include <stdbool.h>
#include <stddef.h>
```

Include dependency graph for cpuUtils.c:



Functions

- int `cpuid_check_sse` (void)
- int `cpuid_check_xsave` (void)
- int `cpuid_check_pcid` (void)
- int `cpuid_check_pae` (void)
- int `cpuid_check_mce` (void)
- int `cpuid_check_apic` (void)
- int `cpuid_check_mca` (void)
- int `cpuid_check_acpi` (void)
- int `cpuid_check_ds` (void)
- int `cpuid_check_tm` (void)
- int `cpuid_check_sep` (void)
- int `cpuid_check_htt` (void)
- int `cpuid_check_rdseed` (void)
- int `cpuid_check_rdrand` (void)
- int `cpuid_check_fpu` (void)
- int `cpuid_check_oxsave` (void)
- int `cpuid_check_avx` (void)
- int `cpuid_check_fxr` (void)
- int `cpuid_check_tsc` (void)
- uint32_t `get_apic_base_address` (void)
- int `test_em` (void)
- void `cfg_XCR0` (void)
- uint64_t `rdrand_asm` (void)

- uint64_t `rdseed_asm` (void)
- uint64_t `read_XCR0` (void)
- uint64_t `get_xsave_size` (void)
- void `halt` (void)
- uint64_t `rand_asm` (void)
 - Obtain a hardware-generated random value through the assembly helper.*
- void `cpuid_readout` (void)
 - Print the verbose boot-time CPUID/capability report.*
- int `get_model` (void)
 - Return the detected processor model identifier used by current code.*
- void `fpu_init` (void)
 - Initialize the current floating-point/SIMD setup path.*
- void `alloc_xsave` (void)
 - Allocate/configure storage used by the current XSAVE experiment.*
- void `check_fpu` (void)
- void `check_fxsr` (void)
- void `no_sse` (void)
- void `no_xsave` (void)
- void `no_oxsave` (void)
- void `no_avx` (void)
- void `check_tsc` (void)
- void `check_rdseed` (void)
- void `check_rdrand` (void)
- void `check_sep` (void)
- void `check_sse` (void)
- void `check_oxsave` (void)
- void `check_avx` (void)
- void `check_xsave` (void)
- void `check_pcid` (void)
- void `check_pae` (void)
- void `check_htt` (void)
- void `check_mce` (void)
- void `check_apic` (void)
- void `check_mca` (void)
- void `check_acpi` (void)
- void `check_ds` (void)
- void `check_tm` (void)

Variables

- size_t `xsave_bank` `ALIGN_16BIT`
- bool `has_ACPI`

7.11.1 Function Documentation

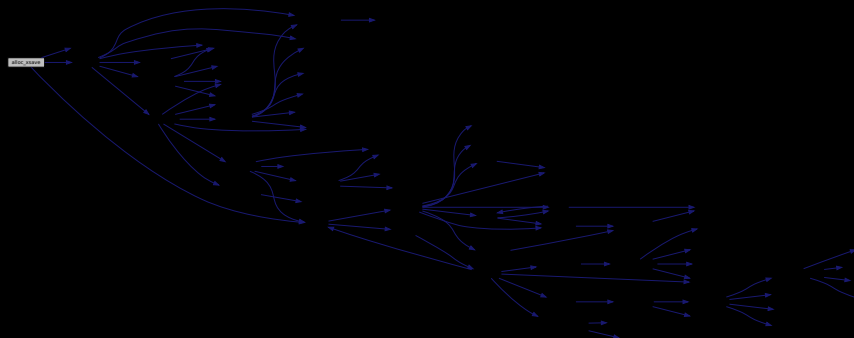
7.11.1.1 alloc_xsave()

```
void alloc_xsave (  
    void )
```

Allocate/configure storage used by the current XSAVE experiment.

Definition at line 119 of file [cpuUtils.c](#).

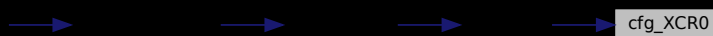
Here is the call graph for this function:



7.11.1.2 cfg_XCR0()

```
void cfg_XCR0 (  
    void )
```

Here is the caller graph for this function:

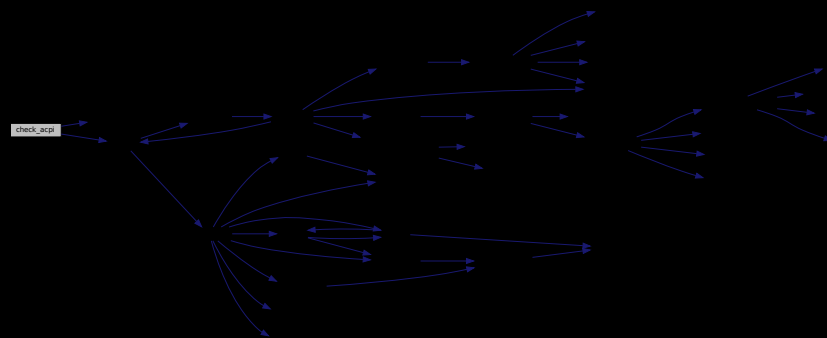


7.11.1.3 check_acpi()

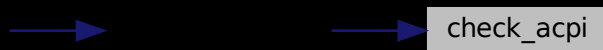
```
void check_acpi (  
    void )
```

Definition at line 284 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

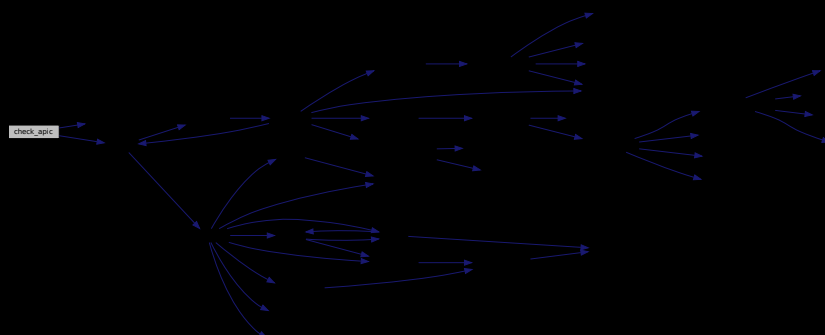


7.11.1.4 check_apic()

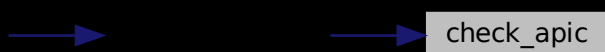
```
void check_apic (  
    void )
```

Definition at line 272 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



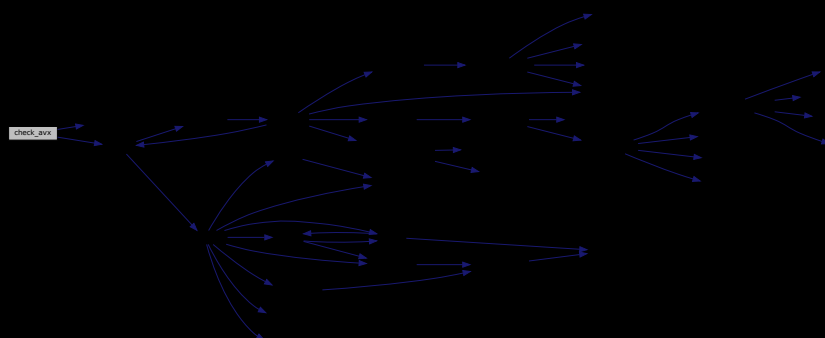
7.11.1.5 `check_avx()`

```
void check_avx (
    void )
```

Individual feature-probe helpers used by `cpuid_readout()`.

Definition at line 229 of file `cpuUtils.c`.

Here is the call graph for this function:

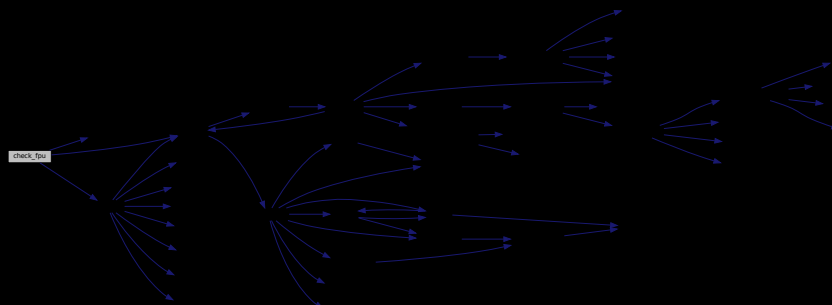


7.11.1.7 check_fpu()

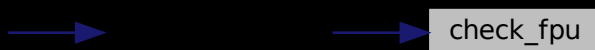
```
void check_fpu (  
    void )
```

Definition at line 130 of file [cpuUtils.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

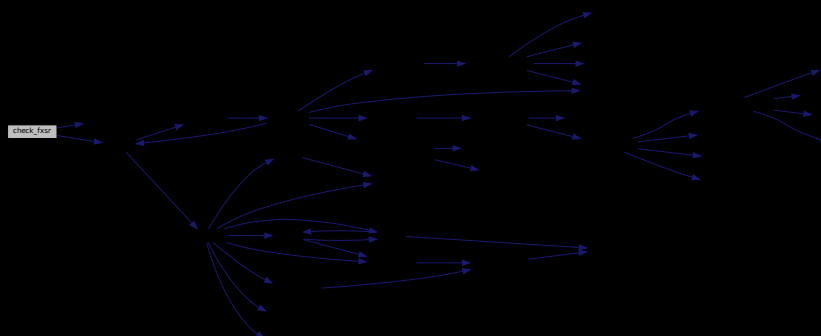


7.11.1.8 check_fxsr()

```
void check_fxsr (  
    void )
```

Definition at line 145 of file [cpuUtils.c](#).

Here is the call graph for this function:

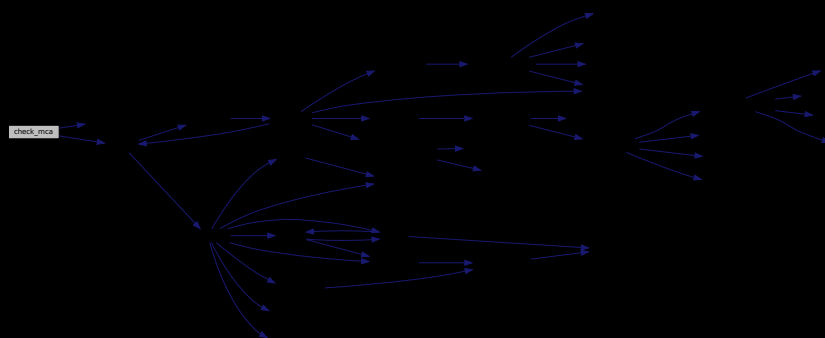


7.11.1.10 check_mca()

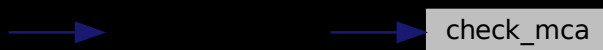
```
void check_mca (  
    void )
```

Definition at line 278 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

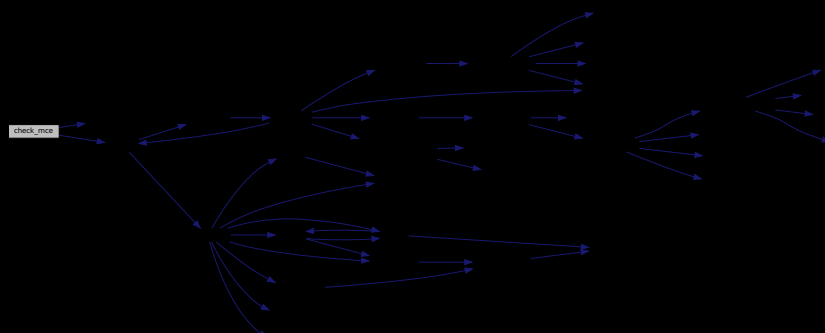


7.11.1.11 check_mce()

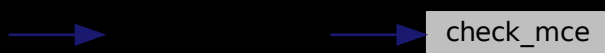
```
void check_mce (  
    void )
```

Definition at line 266 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

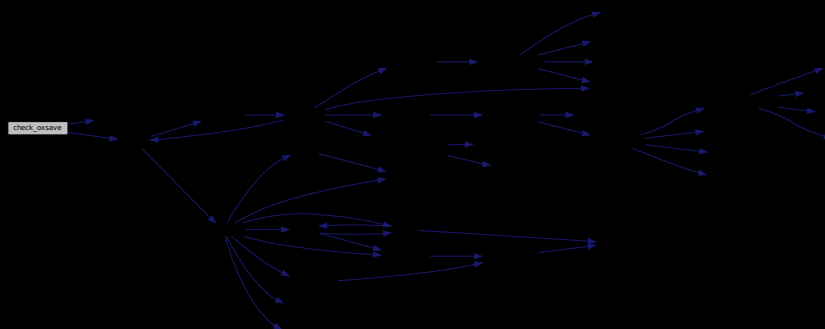


7.11.1.12 `check_oxsave()`

```
void check_oxsave (
    void )
```

Definition at line 222 of file `cpuUtils.c`.

Here is the call graph for this function:



→ → check_oxsave

```
void check_pae (
                void )
```

Here is the call graph for this function:

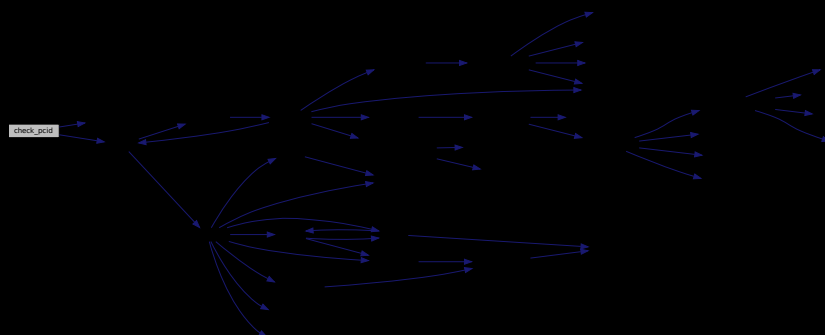
→ → check_pae

7.11.1.14 check_pcid()

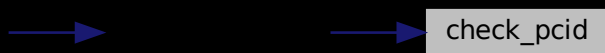
```
void check_pcid (  
    void )
```

Definition at line 248 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

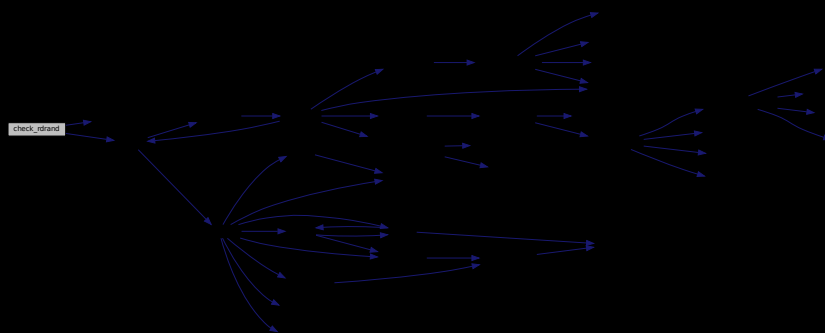


7.11.1.15 check_rdrand()

```
void check_rdrand (  
    void )
```

Definition at line 187 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.11.1.16 check_rdseed()

```
void check_rdseed (
                void )
```

Definition at line 181 of file cpuUtils.c.

Here is the call graph for this function:

Here is the caller graph for this function:

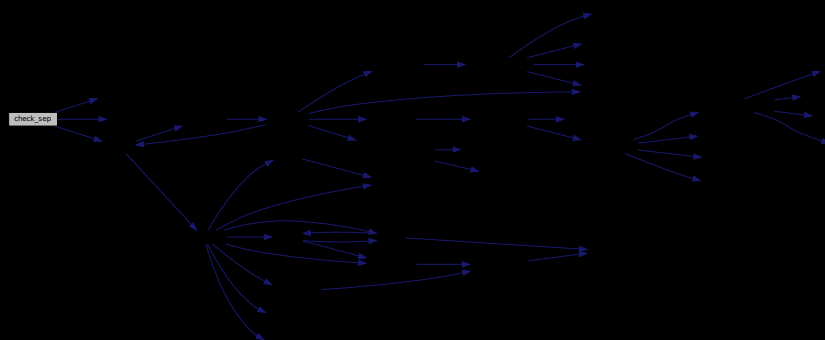


7.11.1.17 check_sep()

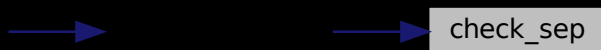
```
void check_sep (
    void )
```

Definition at line 193 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

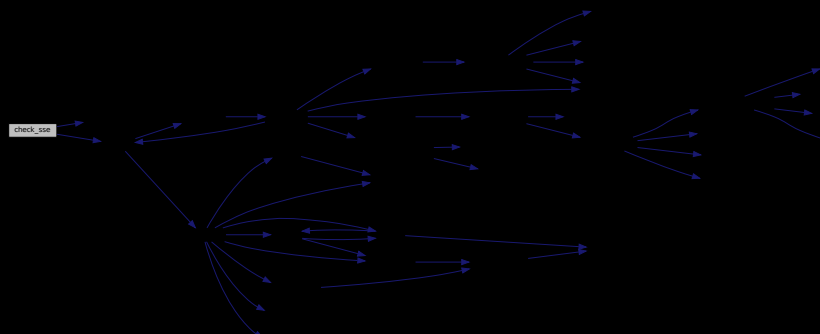


7.11.1.18 check_sse()

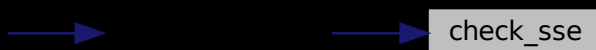
```
void check_sse (
    void )
```

Definition at line 211 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

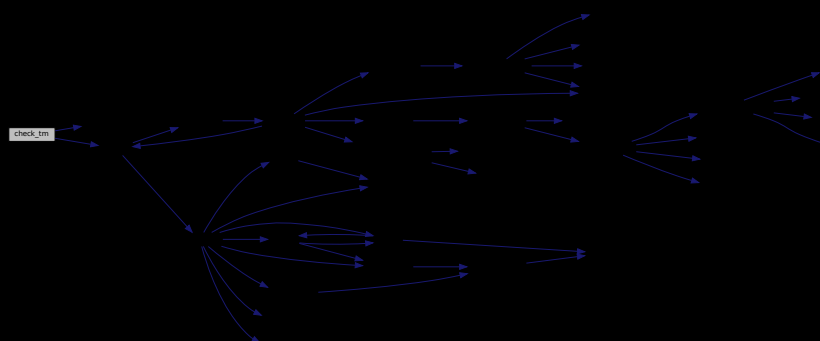


7.11.1.19 check_tm()

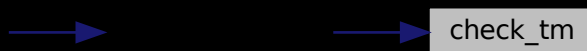
```
void check_tm (
    void )
```

Definition at line 297 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

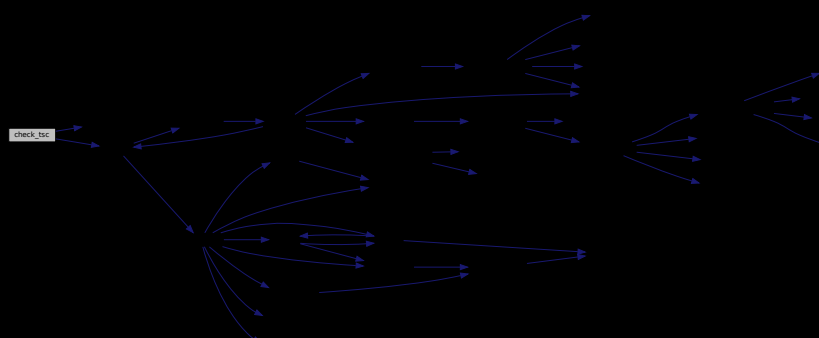


7.11.1.20 `check_tsc()`

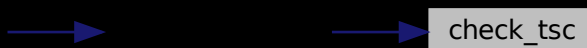
```
void check_tsc (
    void )
```

Definition at line 175 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

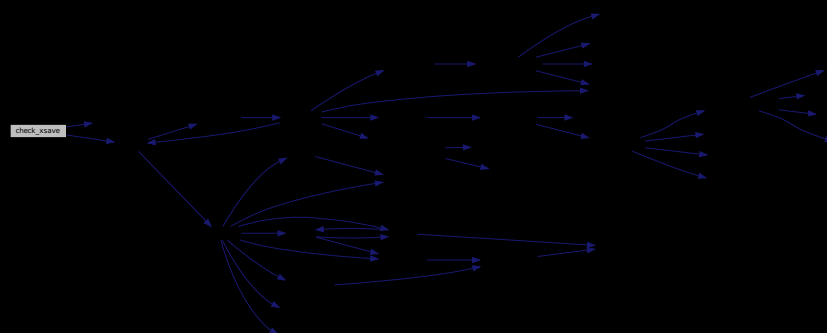


7.11.1.21 check_xsave()

```
void check_xsave (  
    void )
```

Definition at line 236 of file [cpuUtils.c](#).

Here is the call graph for this function:



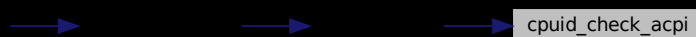
Here is the caller graph for this function:



7.11.1.22 cpuid_check_acpi()

```
int cpuid_check_acpi (  
    void )
```

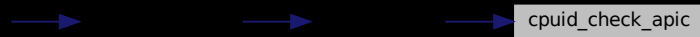
Here is the caller graph for this function:



7.11.1.23 cpuid_check_apic()

```
int cpuid_check_apic (  
    void )
```

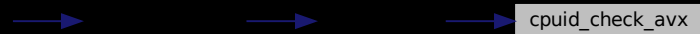
Here is the caller graph for this function:



7.11.1.24 cpuid_check_avx()

```
int cpuid_check_avx (  
    void )
```

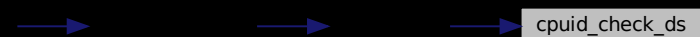
Here is the caller graph for this function:



7.11.1.25 cpuid_check_ds()

```
int cpuid_check_ds (  
    void )
```

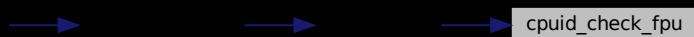
Here is the caller graph for this function:



7.11.1.26 cpuid_check_fpu()

```
int cpuid_check_fpu (  
    void )
```

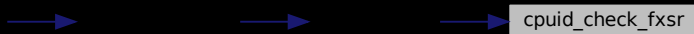
Here is the caller graph for this function:



7.11.1.27 cpuid_check_fxsr()

```
int cpuid_check_fxsr (  
    void )
```

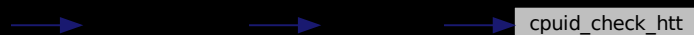
Here is the caller graph for this function:



7.11.1.28 cpuid_check_htt()

```
int cpuid_check_htt (  
    void )
```

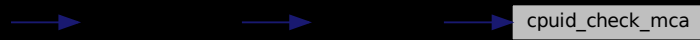
Here is the caller graph for this function:



7.11.1.29 cpuid_check_mca()

```
int cpuid_check_mca (  
    void )
```

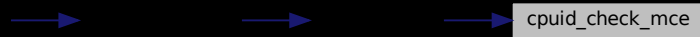
Here is the caller graph for this function:



7.11.1.30 cpuid_check_mce()

```
int cpuid_check_mce (  
    void )
```

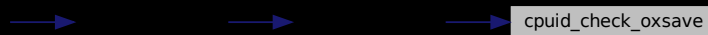
Here is the caller graph for this function:



7.11.1.31 cpuid_check_oxsave()

```
int cpuid_check_oxsave (  
    void )
```

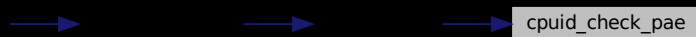
Here is the caller graph for this function:



7.11.1.32 cpuid_check_pae()

```
int cpuid_check_pae (  
    void )
```

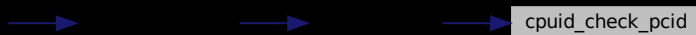
Here is the caller graph for this function:



7.11.1.33 cpuid_check_pcid()

```
int cpuid_check_pcid (  
    void )
```

Here is the caller graph for this function:



7.11.1.34 cpuid_check_rdrand()

```
int cpuid_check_rdrand (  
    void )
```

Here is the caller graph for this function:



7.11.1.35 cpuid_check_rdseed()

```
int cpuid_check_rdseed (  
    void )
```

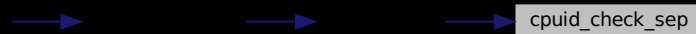
Here is the caller graph for this function:



7.11.1.36 cpuid_check_sep()

```
int cpuid_check_sep (  
    void )
```

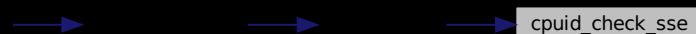
Here is the caller graph for this function:



7.11.1.37 cpuid_check_sse()

```
int cpuid_check_sse (  
    void )
```

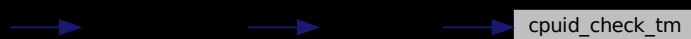
Here is the caller graph for this function:



7.11.1.38 cpuid_check_tm()

```
int cpuid_check_tm (  
    void )
```

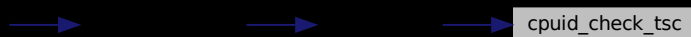
Here is the caller graph for this function:



7.11.1.39 cpuid_check_tsc()

```
int cpuid_check_tsc (  
    void )
```

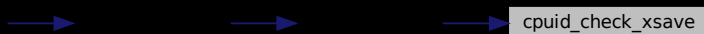
Here is the caller graph for this function:



7.11.1.40 cpuid_check_xsave()

```
int cpuid_check_xsave (  
    void )
```

Here is the caller graph for this function:



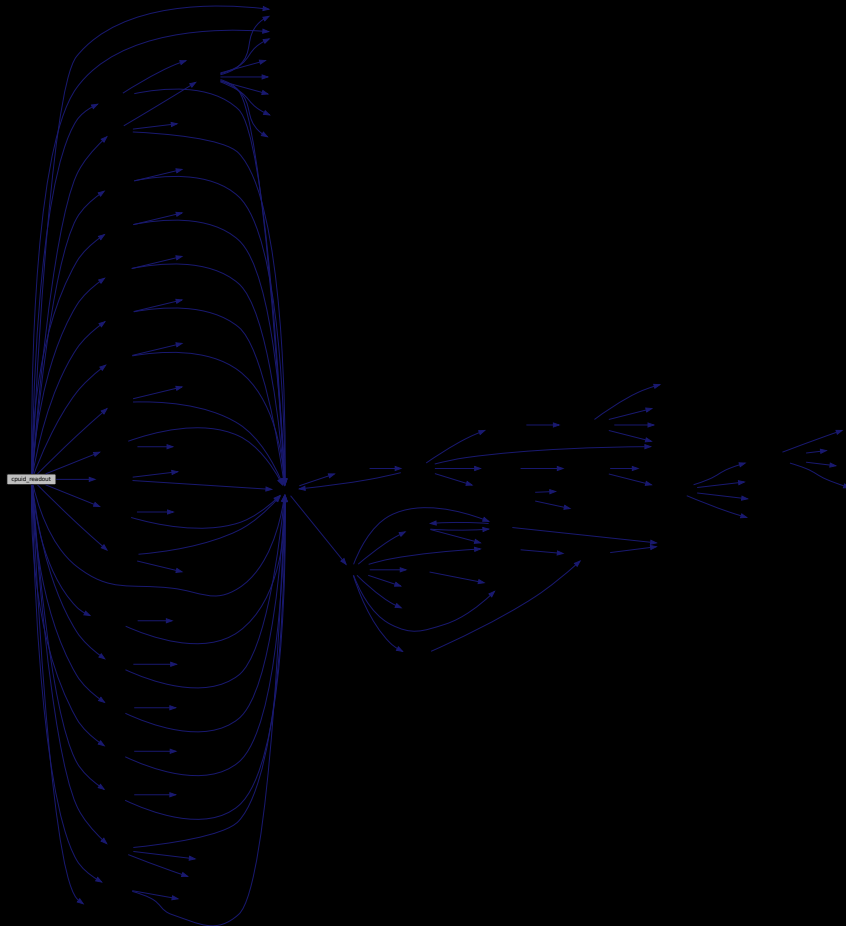
7.11.1.41 cpuid_readout()

```
void cpuid_readout (  
    void )
```

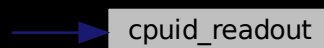
Print the verbose boot-time CPUID/capability report.

Definition at line 60 of file [cpuUtils.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



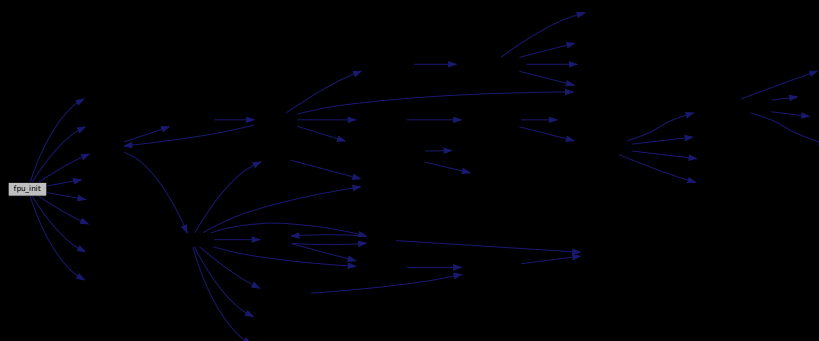
7.11.1.42 fpu_init()

```
void fpu_init (
    void )
```

Initialize the current floating-point/SIMD setup path.

Definition at line 102 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.11.1.43 get_apic_base_address()

```
uint32_t get_apic_base_address (
    void )
```

Here is the caller graph for this function:



7.11.1.44 `get_model()`

```
int get_model (  
    void )
```

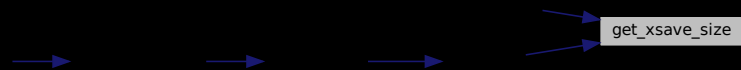
Return the detected processor model identifier used by current code.

Definition at line 95 of file `cpuUtils.c`.

7.11.1.45 `get_xsave_size()`

```
uint64_t get_xsave_size (  
    void )
```

Here is the caller graph for this function:



7.11.1.46 `halt()`

```
void halt (  
    void )
```

Here is the caller graph for this function:

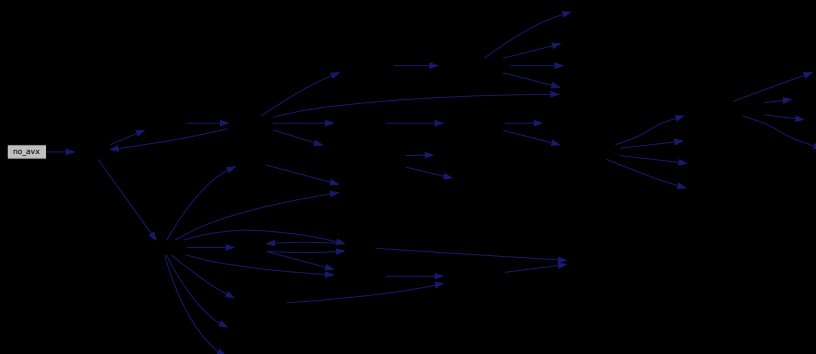


7.11.1.47 no_avx()

```
void no_avx (  
    void )
```

Definition at line 169 of file [cpuUtils.c](#).

Here is the call graph for this function:

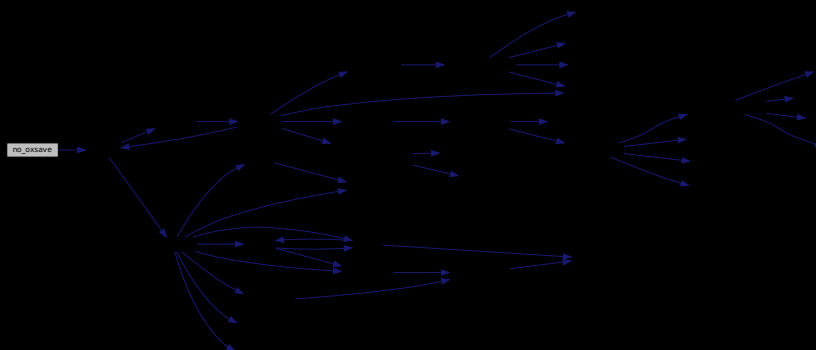


7.11.1.48 no_oxsave()

```
void no_oxsave (  
    void )
```

Definition at line 163 of file [cpuUtils.c](#).

Here is the call graph for this function:

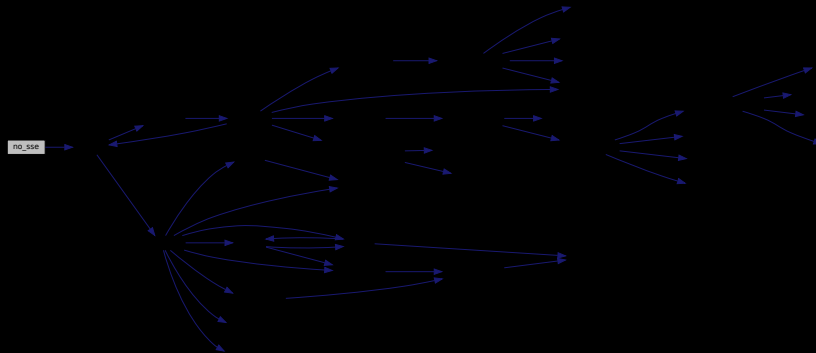


7.11.1.49 no_sse()

```
void no_sse (  
    void )
```

Definition at line 151 of file `cpuUtils.c`.

Here is the call graph for this function:

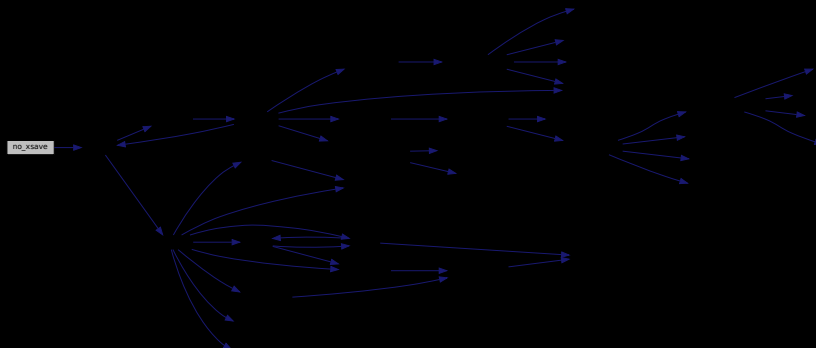


7.11.1.50 no_xsave()

```
void no_xsave (  
    void )
```

Definition at line 157 of file `cpuUtils.c`.

Here is the call graph for this function:



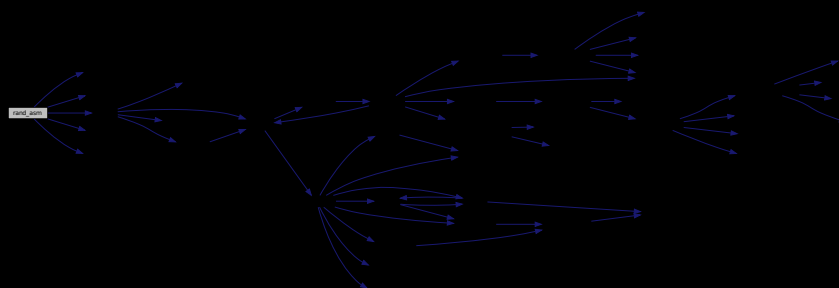
7.11.1.51 rand_asm()

```
uint64_t rand_asm (  
    void )
```

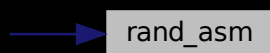
Obtain a hardware-generated random value through the assembly helper.

Definition at line 44 of file [cpuUtils.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.11.1.52 rdrand_asm()

```
uint64_t rdrand_asm (  
    void )
```

Here is the caller graph for this function:



7.11.1.53 rdseed_asm()

```
uint64_t rdseed_asm (  
    void )
```

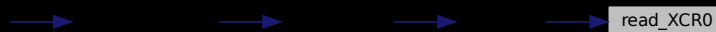
Here is the caller graph for this function:



7.11.1.54 read_XCR0()

```
uint64_t read_XCR0 (  
    void )
```

Here is the caller graph for this function:



7.11.1.55 test_em()

```
int test_em (  
    void )
```

7.11.2 Variable Documentation

7.11.2.1 ALIGN_16BIT

```
size_t xsave_bank ALIGN_16BIT
```

Definition at line 41 of file `cpuUtils.c`.

7.11.2.2 has_ACPI

bool has_ACPI

CPUID-derived ACPI capability state.

Definition at line 42 of file `cpuUtils.c`.

7.12 cpuUtils.c

Go to the documentation of this file.

```

00001 #include "include/KernelUtils.h"
00002 #include "include/cpu.h"
00003 #include "include/cpuUtils.h"
00004 #include "include/global_defs.h"
00005 #include "include/mm/kmalloc.h"
00006 #include "include/panic.h"
00007 #include "include/printf.h"
00008 #include "include/registers.h"
00009 #include <cpuid.h>
00010 #include <stdbool.h>
00011 #include <stddef.h>
00012
00013 extern int cpuid_check_sse(void);
00014 extern int cpuid_check_xsave(void);
00015 extern int cpuid_check_pcid(void);
00016 extern int cpuid_check_pae(void);
00017 extern int cpuid_check_mce(void);
00018 extern int cpuid_check_apic(void);
00019 extern int cpuid_check_mca(void);
00020 extern int cpuid_check_acpi(void);
00021 extern int cpuid_check_ds(void);
00022 extern int cpuid_check_tm(void);
00023 extern int cpuid_check_sep(void);
00024 extern int cpuid_check_htt(void);
00025 extern int cpuid_check_rdseed(void);
00026 extern int cpuid_check_rdrand(void);
00027 extern int cpuid_check_fpu(void);
00028 extern int cpuid_check_oxsave(void);
00029 extern int cpuid_check_avx(void);
00030 extern int cpuid_check_fxsr(void);
00031 extern int cpuid_check_tsc(void);
00032 extern uint32_t get_apic_base_address(void);
00033 extern int test_em(void);
00034 extern void cfg_XCR0(void);
00035 extern uint64_t rdrand_asm(void);
00036 extern uint64_t rdseed_asm(void);
00037 extern uint64_t read_XCR0(void);
00038 extern uint64_t get_xsave_size(void);
00039 extern void halt(void);
00040
00041 size_t xsave_bank ALIGN_16BIT;
00042 bool has_ACPI;
00043
00044 uint64_t rand_asm(void)
00045 {
00046     uint64_t result;
00047     uint64_t seed = cpuid_check_rdseed();
00048     uint64_t rand = cpuid_check_rdrand();
00049
00050     if (seed == 1)
00051         result = rdseed_asm();
00052     else if (rand == 1)
00053         result = rdrand_asm();
00054     else
00055         panic("NO HARDWARE RNG CAPABILITIES DETECTED!");
00056
00057     return result;
00058 }
00059
00060 void cpuid_readout(void)
00061 {
00062     printf("%s\n", "-----");
00063     printf("%s\n", "CPUID Readout As Follows");
00064     printf("%s\n", "-----");
00065
00066     check_avx();

```

```

00067     check_oxsave();
00068     check_fxsr();
00069     check_htt();
00070     check_sep();
00071     check_sse();
00072     check_xsave();
00073     check_poid();
00074     check_pae();
00075     check_mce();
00076     check_apic();
00077     check_mca();
00078     check_acpi();
00079     check_ds();
00080     check_tm();
00081     check_rdseed();
00082     check_rdrand();
00083     check_fpu();
00084     check_tsc();
00085
00086     printf_("%s", "CR0:  ");
00087     printf_("0x%llx\n", readCR0());
00088     printf_("%s", "CR4:  ");
00089     printf_("0x%llx\n", readCR4());
00090     printf_("%s", "APIC Base Address:  ");
00091     printf_("0x%llx\n", get_apic_base_address());
00092     printf_("%s\n", "-----");
00093 }
00094
00095 int get_model(void)
00096 {
00097     int ebx, unused;
00098     __cpuid(0, unused, ebx, unused, unused);
00099     return ebx;
00100 }
00101
00102 void fpu_init(void)
00103 {
00104     printf_("%s\n", "INFO: Enabling the x87 FPU");
00105
00106     writeCR0(readCR0() | 1 << 1);
00107     writeCR0(readCR0() | 1 << 5);
00108     writeCR4(readCR4() | 1 << 9);
00109     writeCR4(readCR4() | 1 << 10);
00110     writeCR4(readCR4() | 1 << 18);
00111     cfg_XCR0();
00112     printf_("%s", "XCR0:  ");
00113     printf_("0x%llx\n", read_XCR0());
00114     printf_("%s", "XSTORE Area size:  ");
00115     printf_("0x%llx\n", get_xsave_size());
00116     printf_("%s\n", "INFO: x87 FPU now online");
00117 }
00118
00119 void alloc_xsave(void)
00120 {
00121     xsave_bank = (size_t) get_xsave_size();
00122     printf_("%s", "INFO: Allocating a XSAVE Bank with a size of:  ");
00123     printf_("0x%llx\n", xsave_bank);
00124
00125     /* The historical XSAVE code never retained this pointer. Keep the old
00126  * behavior off liballoc while the FPU subsystem remains parked. */
00127     (void) kcalloc(xsave_bank);
00128 }
00129
00130 void check_fpu(void)
00131 {
00132     int found = cpuid_check_fpu();
00133     if (found == 1)
00134     {
00135         printf_("%s\n", "FPU: Yes");
00136         if (k_mode.fpu_allowed == 1)
00137             fpu_init();
00138     }
00139     else
00140     {
00141         printf_("%s\n", "FPU: No");
00142     }
00143 }
00144
00145 void check_fxsr(void)
00146 {
00147     int found = cpuid_check_fxsr();
00148     printf_("%s\n", found == 1 ? "FXSR: Yes" : "FXSR: No");
00149 }
00150
00151 void no_sse(void)
00152 {
00153     printf_("%s\n", "SSE Extensions Unavailable.");

```

```

00154     printf_("%s\n", "Floating Point Math will be offline.");
00155 }
00156
00157 void no_xsave(void)
00158 {
00159     printf_("%s\n", "XSAVE Extensions Unavailable.");
00160     printf_("%s\n", "Floating Point Math will be offline.");
00161 }
00162
00163 void no_oxsave(void)
00164 {
00165     printf_("%s\n", "OXSAVE Extensions Unavailable.");
00166     printf_("%s\n", "Floating Point Math will be offline.");
00167 }
00168
00169 void no_avx(void)
00170 {
00171     printf_("%s\n", "AVX Extensions Unavailable.");
00172     printf_("%s\n", "Floating Point Math will be offline.");
00173 }
00174
00175 void check_tsc(void)
00176 {
00177     int found = cpuid_check_tsc();
00178     printf_("%s\n", found == 1 ? "TSC: YES" : "TSC: NO");
00179 }
00180
00181 void check_rdseed(void)
00182 {
00183     int found = cpuid_check_rdseed();
00184     printf_("%s\n", found == 1 ? "RDSEED: Yes" : "RDSEED: No");
00185 }
00186
00187 void check_rdrand(void)
00188 {
00189     int found = cpuid_check_rdrand();
00190     printf_("%s\n", found == 1 ? "RDRAND: Yes" : "RDRAND: No");
00191 }
00192
00193 void check_sep(void)
00194 {
00195     int found = cpuid_check_sep();
00196     if (found == 1)
00197     {
00198         sysenter = true;
00199         printf_("%s\n", "SEP (SYSENTER/EXIT): Yes");
00200     }
00201     else
00202     {
00203         sysenter = false;
00204         printf_("%s\n", "SEP (SYSENTER/EXIT): No");
00205         printf_("%s\n",
00206             "You Realized How Fucked You Are Without This? Halting Get A Better PC.");
00207         halt();
00208     }
00209 }
00210
00211 void check_sse(void)
00212 {
00213     int found = cpuid_check_sse();
00214     if (found == 1)
00215     {
00216         printf_("%s\n", "SSE Extensions Available.");
00217         printf_("%s\n", "Enabling...");
00218         printf_("%s\n", "SSE Extensions Online!");
00219     }
00220 }
00221
00222 void check_oxsave(void)
00223 {
00224     int found = cpuid_check_oxsave();
00225     if (found == 1)
00226         printf_("%s\n", "OXSAVE Extensions Available.");
00227 }
00228
00229 void check_avx(void)
00230 {
00231     int found = cpuid_check_avx();
00232     if (found == 1)
00233         printf_("%s\n", "AVX Extensions Available.");
00234 }
00235
00236 void check_xsave(void)
00237 {
00238     int found = cpuid_check_xsave();
00239     if (found == 1)
00240     {

```

```

00241     printf_("%s\n", "XSAVE Extensions Available.");
00242     printf_("%s\n", "Enabling...");
00243     printf_("%s\n", "XSAVE Extensions Online!");
00244     printf_("%s\n", "Floating Point Math Online using SSE and XSAVE!");
00245 }
00246 }
00247
00248 void check_pcid(void)
00249 {
00250     int found = cpuid_check_pcid();
00251     printf_("%s\n", found == 1 ? "PCID: Yes" : "PCID: No");
00252 }
00253
00254 void check_pae(void)
00255 {
00256     int found = cpuid_check_pae();
00257     printf_("%s\n", found == 1 ? "PAE: Yes" : "PAE: No");
00258 }
00259
00260 void check_htt(void)
00261 {
00262     int found = cpuid_check_htt();
00263     printf_("%s\n", found == 1 ? "HTT: Yes" : "HTT: No");
00264 }
00265
00266 void check_mce(void)
00267 {
00268     int found = cpuid_check_mce();
00269     printf_("%s\n", found == 1 ? "MCE: Yes" : "MCE: No");
00270 }
00271
00272 void check_apic(void)
00273 {
00274     int found = cpuid_check_apic();
00275     printf_("%s\n", found == 1 ? "APIC: Yes" : "APIC: No");
00276 }
00277
00278 void check_mca(void)
00279 {
00280     int found = cpuid_check_mca();
00281     printf_("%s\n", found == 1 ? "MCA: Yes" : "MCA: No");
00282 }
00283
00284 void check_acpi(void)
00285 {
00286     int found = cpuid_check_acpi();
00287     has_ACPI = found == 1;
00288     printf_("%s\n", has_ACPI ? "ACPI: Yes" : "ACPI: No");
00289 }
00290
00291 void check_ds(void)
00292 {
00293     int found = cpuid_check_ds();
00294     printf_("%s\n", found == 1 ? "DS: Yes" : "DS: No");
00295 }
00296
00297 void check_tm(void)
00298 {
00299     int found = cpuid_check_tm();
00300     printf_("%s\n", found == 1 ? "TM: Yes" : "TM: No");
00301 }

```

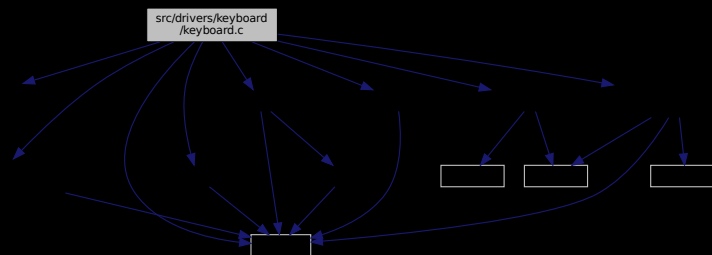
7.13 src/drivers/keyboard/keyboard.c File Reference

```

#include "../include/drivers/keyboard/keyboard.h"
#include "../include/drivers/keyboard/keyboard_map.h"
#include "../include/io.h"
#include "../include/kernel.h"
#include "../include/pic.h"
#include "../include/printf.h"
#include "../include/terminal/term.h"
#include <stdint.h>

```


Include dependency graph for keyboard.c:



Macros

- #define KBD_STACK_SIZE 255

Functions

- void **kbd_push** (char data)
Push a character onto the keyboard FIFO.
- char **kbd_pop** (void)
Pop the oldest character from the keyboard FIFO.
- void **keyboard_handler** (void)
PS/2 keyboard IRQ handler.
- char **k_getchar** (void)
Fetch the most recently translated keyboard character.
- void **keyboard_init** (void)
Enable keyboard IRQ1.

Variables

- char k_char
- char kbd_stack [KBD_STACK_SIZE]
- static volatile uint16_t kbd_head = 0
- static volatile uint16_t kbd_tail = 0

7.13.1 Macro Definition Documentation

7.13.1.1 KBD_STACK_SIZE

```
#define KBD_STACK_SIZE 255
```

Definition at line 10 of file keyboard.c.

7.13.2 Function Documentation

7.13.2.1 k_getchar()

```
char k_getchar (
    void )
```

Fetch the most recently translated keyboard character.

Legacy single-character keyboard accessor.

Returns

Character, or 0 if none is pending.

Definition at line 91 of file [keyboard.c](#).

7.13.2.2 kbd_pop()

```
char kbd_pop (
    void )
```

Pop the oldest character from the keyboard FIFO.

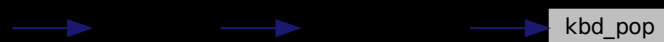
Remove one character from the keyboard FIFO.

Returns

Queued character, or 0 when the queue is empty.

Definition at line 42 of file [keyboard.c](#).

Here is the caller graph for this function:



7.13.2.3 kbd_push()

```
void kbd_push (
    char data )
```

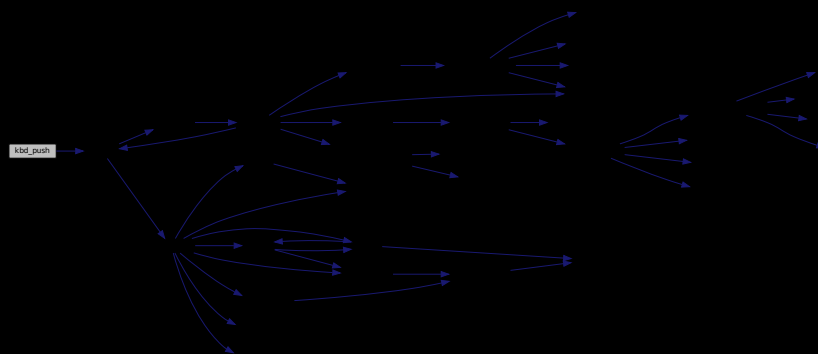
Push a character onto the keyboard FIFO.

Parameters

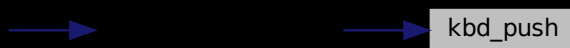
<i>data</i>	Character to enqueue.
-------------	-----------------------

Definition at line 22 of file [keyboard.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.13.2.4 keyboard_handler()

```
void keyboard_handler (
    void )
```

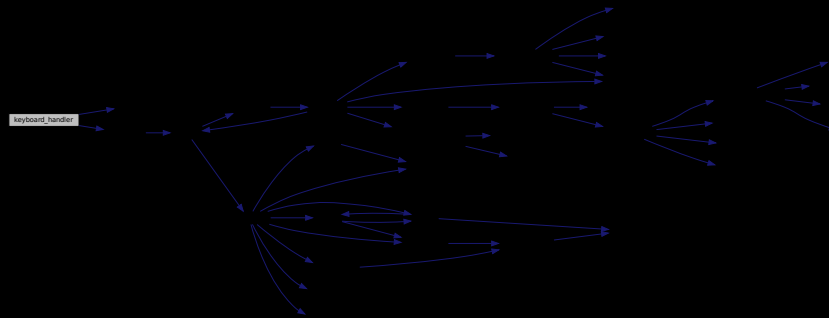
PS/2 keyboard IRQ handler.

IRQ1 keyboard handler.

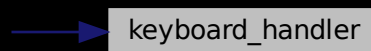
Key-release scancodes are ignored. Enter is normalized to a newline for the shell while `k_char` retains the historical '+' sentinel for legacy callers.

Definition at line 58 of file [keyboard.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.13.2.5 keyboard_init()

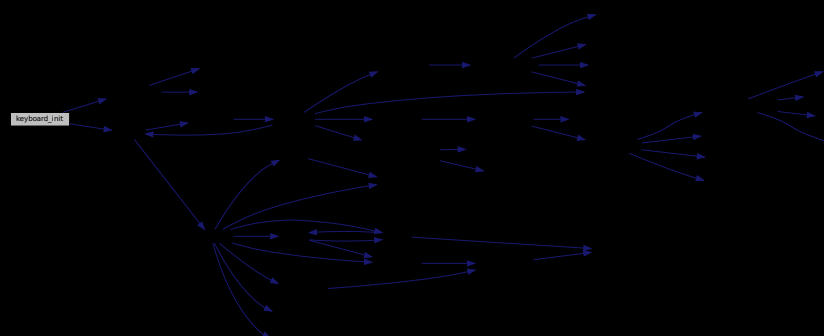
```
void keyboard_init (
    void )
```

Enable keyboard IRQ1.

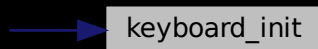
Initialize the keyboard input path and make IRQ1 available.

Definition at line 101 of file [keyboard.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.13.3 Variable Documentation

7.13.3.1 `k_char`

```
char k_char
```

Definition at line 12 of file `keyboard.c`.

7.13.3.2 `kbd_head`

```
volatile uint16_t kbd_head = 0 [static]
```

Definition at line 15 of file `keyboard.c`.

7.13.3.3 `kbd_stack`

```
char kbd_stack[KBD_STACK_SIZE]
```

Definition at line 13 of file `keyboard.c`.

7.13.3.4 `kbd_tail`

```
volatile uint16_t kbd_tail = 0 [static]
```

Definition at line 16 of file `keyboard.c`.

7.14 keyboard.c

Go to the documentation of this file.

```

00001 #include "../include/drivers/keyboard/keyboard.h"
00002 #include "../include/drivers/keyboard/keyboard_map.h"
00003 #include "../include/io.h"
00004 #include "../include/kernel.h"
00005 #include "../include/pic.h"
00006 #include "../include/printf.h"
00007 #include "../include/terminal/term.h"
00008 #include <stdint.h>
00009
00010 #define KBD_STACK_SIZE 255
00011
00012 char k_char;
00013 char kbd_stack[KBD_STACK_SIZE];
00014
00015 static volatile uint16_t kbd_head = 0;
00016 static volatile uint16_t kbd_tail = 0;
00017
00022 void kbd_push(char data)
00023 {
00024     uint16_t next = (uint16_t)((kbd_head + 1u) % KBD_STACK_SIZE);
00025
00026     if (next == kbd_tail)
00027     {
00028         kerror_mode = 1;
00029         printf_("%s\n", "ERROR: Keyboard buffer overflow!");
00030         kerror_mode = 0;
00031         return;
00032     }
00033
00034     kbd_stack[kbd_head] = data;
00035     kbd_head = next;
00036 }
00037
00042 char kbd_pop(void)
00043 {
00044     if (kbd_tail == kbd_head)
00045         return 0;
00046
00047     char data = kbd_stack[kbd_tail];
00048     kbd_tail = (uint16_t)((kbd_tail + 1u) % KBD_STACK_SIZE);
00049     return data;
00050 }
00051
00058 void keyboard_handler(void)
00059 {
00060     uint8_t status = inb(KEYBOARD_STATUS_PORT);
00061     if ((status & 0x1u) == 0)
00062         return;
00063
00064     uint8_t keycode = inb(KEYBOARD_DATA_PORT);
00065
00066     /* Set 1 break codes have the high bit set. */
00067     if (keycode & 0x80u)
00068         return;
00069
00070     if (keycode >= 128u)
00071         return;
00072
00073     if (keycode == 0x1cu)
00074     {
00075         k_char = '+';
00076         kbd_push('\n');
00077         return;
00078     }
00079
00080     char mapped = keyboard_map[keycode];
00081     k_char = mapped;
00082
00083     if (mapped != 0)
00084         kbd_push(mapped);
00085 }
00086
00091 char k_getchar(void)
00092 {
00093     char c = k_char;
00094     k_char = 0;
00095     return c;
00096 }
00097
00101 void keyboard_init(void)
00102 {
00103     pic_unmask_irq(1);

```

```

00104     printf("%s\n", "Keyboard Init");
00105 }

```

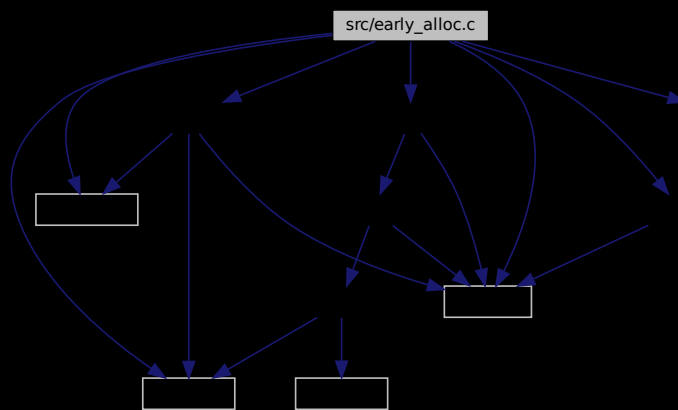
7.15 src/early_alloc.c File Reference

```

#include "include/early_alloc.h"
#include "include/KernelUtils.h"
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include "include/limine.h"
#include "include/panic.h"

```

Include dependency graph for early_alloc.c:



Macros

- `#define PAGE_SIZE 4096ull`
- `#define ALIGN_UP(x, a) (((x) + ((a) - 1)) & ~((a) - 1))`
- `#define ALIGN_DOWN(x, a) ((x) & ~((a) - 1))`

Functions

- static void `memzero` (void *p, size_t n)
- static bool `memmap_ready` (void)
- static void `pick_region_from_memmap` (void)
- void `early_set_region` (paddr_t base, size_t page_count)
Supply an explicit region for early allocation.
- void `early_init` (void)
Select a Limine USABLE memory region and initialize the bump allocator.
- paddr_t `early_alloc_page` (void)
Allocate one 4 KiB physical page from the bootstrap region.
- void `early_reserved_region` (paddr_t *base, size_t *pages)
Report the physical range consumed by early allocation.
- size_t `early_pages_used` (void)
Return the number of 4 KiB pages consumed by the bootstrap allocator.

Variables

- static `paddr_t g_base` = 0
- static `paddr_t g_cursor` = 0
- static `paddr_t g_end` = 0
- static bool `g_region_forced` = false
- volatile struct `limine_memmap_request memmap_req`

7.15.1 Macro Definition Documentation

7.15.1.1 ALIGN_DOWN

```
#define ALIGN_DOWN(  
    x,  
    a ) ((x) & ~((a) -1))
```

Definition at line 21 of file `early_alloc.c`.

7.15.1.2 ALIGN_UP

```
#define ALIGN_UP(  
    x,  
    a ) (((x) + ((a) -1)) & ~((a) -1))
```

Definition at line 20 of file `early_alloc.c`.

7.15.1.3 PAGE_SIZE

```
#define PAGE_SIZE 4096ull
```

Copyright (c) 2025 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Definition at line 19 of file `early_alloc.c`.

7.15.2 Function Documentation

7.15.2.1 early_alloc_page()

```
paddr_t early_alloc_page (
    void )
```

Allocate one 4 KiB physical page from the bootstrap region.

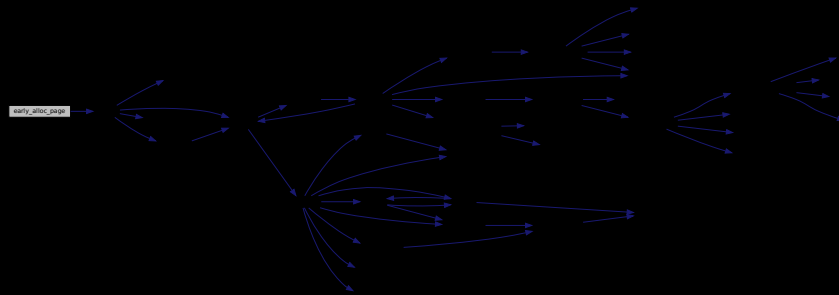
Returns

4 KiB-aligned physical address of the allocated page.

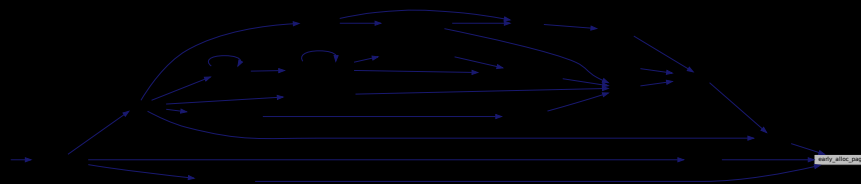
Early allocations are monotonic and are not individually freed. Once the permanent frame allocator is initialized, paging obtains new frames through the Stage-2 frame supplier instead.

Definition at line 119 of file [early_alloc.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.15.2.2 early_init()

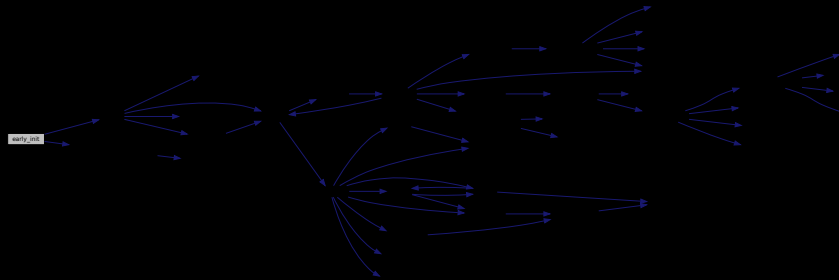
```
void early_init (
    void )
```

Select a Limine USABLE memory region and initialize the bump allocator.

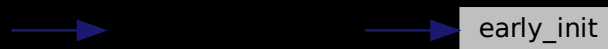
If [early_set_region\(\)](#) has already supplied an explicit region, this function preserves that selection rather than choosing another one.

Definition at line 103 of file `early_alloc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.15.2.3 `early_pages_used()`

```
size_t early_pages_used (
    void )
```

Return the number of 4 KiB pages consumed by the bootstrap allocator.

Returns

Number of pages allocated since initialization.

Definition at line 139 of file `early_alloc.c`.

7.15.2.4 `early_reserved_region()`

```
void early_reserved_region (
    paddr_t * base,
    size_t * pages )
```

Report the physical range consumed by early allocation.

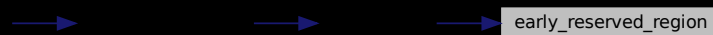
Parameters

<i>base</i>	Receives the physical base of the bootstrap region.
<i>pages</i>	Receives the number of pages consumed from that region.

The permanent PMM uses this information to avoid handing bootstrap-owned memory back out as free frames.

Definition at line 131 of file `early_alloc.c`.

Here is the caller graph for this function:



7.15.2.5 `early_set_region()`

```
void early_set_region (
    paddr_t base,
    size_t page_count )
```

Supply an explicit region for early allocation.

Parameters

<i>base</i>	4 KiB-aligned physical base address.
<i>page_count</i>	Number of 4 KiB pages in the region.

The region must be suitable for allocation and should be configured before `early_init()`.

Definition at line 95 of file `early_alloc.c`.

7.15.2.6 `memmap_ready()`

```
static bool memmap_ready (
    void ) [inline], [static]
```

Definition at line 40 of file `early_alloc.c`.

Here is the caller graph for this function:



7.15.2.7 memzero()

```
static void memzero (
    void * p,
    size_t n ) [inline], [static]
```

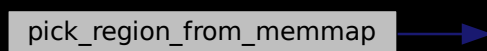
Definition at line 24 of file [early_alloc.c](#).

7.15.2.8 pick_region_from_memmap()

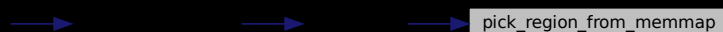
```
static void pick_region_from_memmap (
    void ) [static]
```

Definition at line 47 of file [early_alloc.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.15.3 Variable Documentation

7.15.3.1 g_base

```
paddr_t g_base = 0 [static]
```

Definition at line 32 of file [early_alloc.c](#).

7.15.3.2 g_cursor

```
paddr_t g_cursor = 0 [static]
```

Definition at line 33 of file [early_alloc.c](#).

7.15.3.3 g_end

```
paddr_t g_end = 0 [static]
```

Definition at line 34 of file [early_alloc.c](#).

7.15.3.4 g_region_forced

```
bool g_region_forced = false [static]
```

Definition at line 35 of file [early_alloc.c](#).

7.15.3.5 memmap_req

```
volatile struct limine\_memmap\_request memmap_req [extern]
```

7.16 early_alloc.c

Go to the documentation of this file.

```

00001
00008 // early_alloc.c
00009 #include "include/early_alloc.h"
00010 #include "include/KernelUtils.h"
00011 #include <stdbool.h>
00012 #include <stddef.h>
00013 #include <stdint.h>
00014
00015 // --- Limine requests (defined in some other TU) ---
00016 #include "include/limine.h"
00017 #include "include/panic.h"
00018
00019 #define PAGE_SIZE 4096ull
00020 #define ALIGN_UP(x, a) (((x) + ((a) - 1)) & ~((a) - 1))
00021 #define ALIGN_DOWN(x, a) ((x) & ~((a) - 1))
00022
00023 // Tiny local zero
00024 static inline void memzero(void *p, size_t n)
00025 {
00026     volatile uint8_t *d = (volatile uint8_t *) p;
00027     for (size_t i = 0; i < n; i++)
00028         d[i] = 0;
00029 }
00030
00031 // --- Early bump state ---
00032 static paddr_t g_base = 0;
00033 static paddr_t g_cursor = 0;
00034 static paddr_t g_end = 0;
00035 static bool g_region_forced = false;
00036
00037 extern volatile struct limine_memmap_request memmap_req;
00038
00039
00040 static inline bool memmap_ready(void)
00041 {
00042     return memmap_req.response && memmap_req.response->entry_count > 0;
00043 }
00044
00045 // Pick a USABLE region. Heuristics: prefer the largest USABLE below 4 GiB; fall
00046 // back to largest anywhere.
00047 static void pick_region_from_memmap(void)
00048 {
00049     if (!memmap_ready())
00050         return;
00051
00052     uint64_t best_size_below4g = 0, best_size_any = 0;
00053     paddr_t best_base_below4g = 0, best_end_below4g = 0;
00054     paddr_t best_base_any = 0, best_end_any = 0;
00055
00056     for (size_t i = 0; i < memmap_req.response->entry_count; i++)
00057     {
00058         struct limine_memmap_entry *e = memmap_req.response->entries[i];
00059         if (e->type != LIMINE_MEMMAP_USABLE)
00060             continue;
00061
00062         uint64_t start = ALIGN_UP(e->base, PAGE_SIZE);
00063         uint64_t end = ALIGN_DOWN(e->base + e->length, PAGE_SIZE);
00064         if (end <= start)
00065             continue;
00066         uint64_t size = end - start;
00067
00068         if (end <= 0x100000000ull && size > best_size_below4g)
00069         {
00070             best_size_below4g = size;
00071             best_base_below4g = start;
00072             best_end_below4g = end;
00073         }
00074         if (size > best_size_any)
00075         {
00076             best_size_any = size;
00077             best_base_any = start;
00078             best_end_any = end;
00079         }
00080     }
00081
00082     if (best_size_below4g)
00083     {
00084         g_base = best_base_below4g;
00085         g_end = best_end_below4g;
00086     }
00087     else
00088     {

```

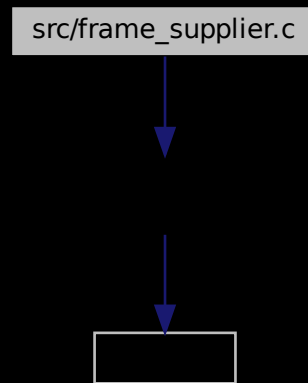
```

00089         g_base = best_base_any;
00090         g_end   = best_end_any;
00091     }
00092     g_cursor = g_base;
00093 }
00094
00095 void early_set_region(paddr_t base, size_t page_count)
00096 {
00097     g_base      = base;
00098     g_cursor    = base;
00099     g_end       = base + page_count * PAGE_SIZE;
00100     g_region_forced = true;
00101 }
00102
00103 void early_init(void)
00104 {
00105     if (!g_region_forced)
00106     {
00107         pick_region_from_memmap();
00108     }
00109     // Minimal sanity: region must exist and be page aligned
00110     if (g_base == 0 || g_end <= g_base || (g_base & (PAGE_SIZE - 1)))
00111     {
00112         // Don't panic here; let the real allocator detect and panic if needed.
00113         // But it's fair to be strict in a hobby kernel:
00114         panic("early_init: no suitable USABLE region found for early "
00115             "allocations");
00116     }
00117 }
00118
00119 paddr_t early_alloc_page(void)
00120 {
00121     if (g_cursor + PAGE_SIZE > g_end)
00122     {
00123         panic("early_alloc_page: out of early pages (used=%zu pages)",
00124             (size_t) ((g_cursor - g_base) / PAGE_SIZE));
00125     }
00126     paddr_t p = g_cursor;
00127     g_cursor += PAGE_SIZE;
00128     return p;
00129 }
00130
00131 void early_reserved_region(paddr_t *base, size_t *pages)
00132 {
00133     if (base)
00134         *base = g_base;
00135     if (pages)
00136         *pages = (size_t) ((g_cursor - g_base) / PAGE_SIZE);
00137 }
00138
00139 size_t early_pages_used(void)
00140 {
00141     return (size_t) ((g_cursor - g_base) / PAGE_SIZE);
00142 }

```

7.17 src/frame_supplier.c File Reference

#include "include/paging/frame_supplier.h"
 Include dependency graph for frame_supplier.c:



Variables

- `phys_page_supplier_t g_boot_phys_page = 0`
Active boot-time physical-page supplier.

7.17.1 Variable Documentation

7.17.1.1 g_boot_phys_page

```
phys_page_supplier_t g_boot_phys_page = 0
```

Active boot-time physical-page supplier.

Copyright (c) 2025 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Definition at line 10 of file `frame_supplier.c`.

7.18 frame_supplier.c

Go to the documentation of this file.

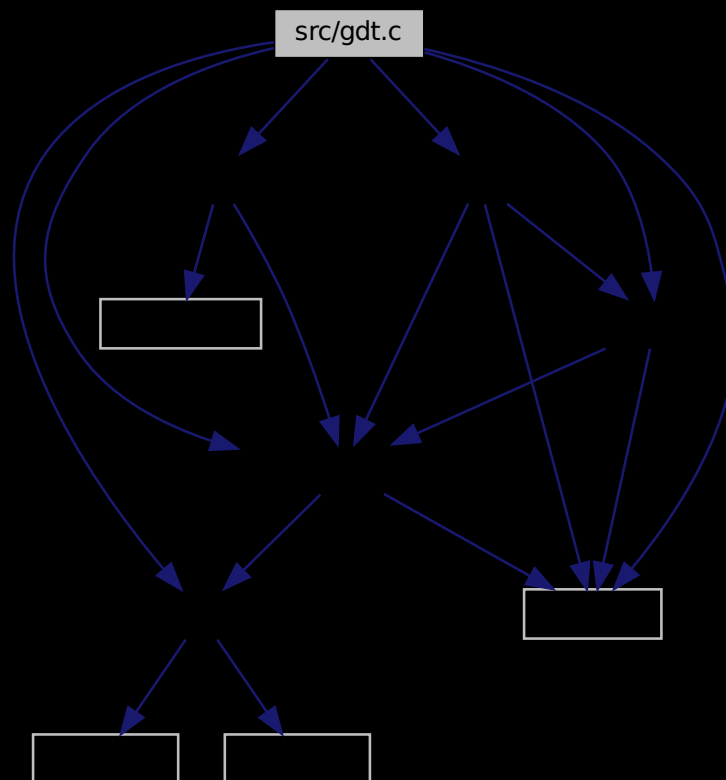
```

00001
00008 #include "include/paging/frame_supplier.h"
00009
00010 phys_page_supplier_t g_boot_phys_page = 0;
```


7.19 src/gdt.c File Reference

```
#include "include/gdt.h"
#include "include/global_defs.h"
#include "include/lock.h"
#include "include/printf.h"
#include "include/tss.h"
#include <stdint.h>
```

Include dependency graph for gdt.c:



Functions

- void `gdt_reload` (void)
Reload segment/GDT state after table construction.
- void `gdt_load_tss` (struct `TSS` *`tss`)
Install/update the `TSS` descriptor for `tss`.
- void `LoadGDT_Stage1` (void)
Build/load the initial descriptor table used during boot.

Variables

- `uint8_t TssStack` [0x100000]
- `uint8_t ist1Stack` [0x100000]
- `uint8_t ist2Stack` [0x100000]
- `uint64_t rsp0`
- `struct TSS tss` = {0}
- `ALIGN_4K` `struct GDT gdt`
- `struct GDT_Desc desc`

7.19.1 Function Documentation

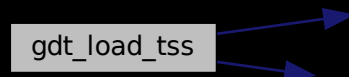
7.19.1.1 `gdt_load_tss()`

```
void gdt_load_tss (  
    struct TSS * tss )
```

Install/update the `TSS` descriptor for `tss`.

Definition at line 68 of file `gdt.c`.

Here is the call graph for this function:



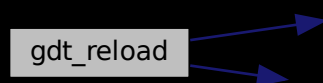
7.19.1.2 gdt_reload()

```
void gdt_reload (  
    void )
```

Reload segment/GDT state after table construction.

Definition at line 44 of file [gdt.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



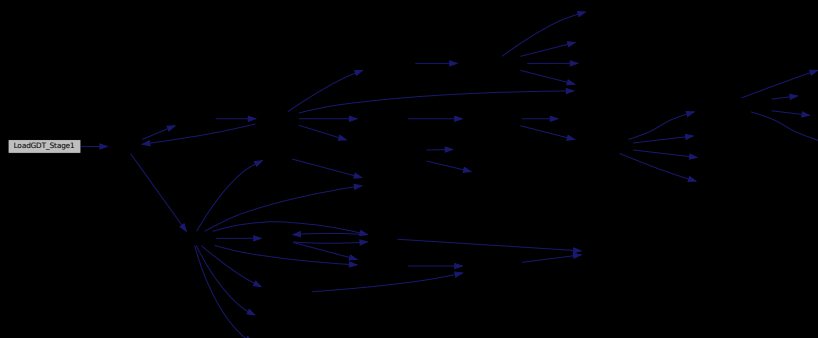
7.19.1.3 LoadGDT_Stage1()

```
void LoadGDT_Stage1 (  
    void )
```

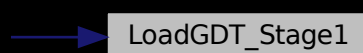
Build/load the initial descriptor table used during boot.

Definition at line 87 of file [gdt.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.19.2 Variable Documentation

7.19.2.1 desc

```
struct GDT_Desc desc
```

Initial value:

```
= {
    .size = sizeof(gdt) - 1,
    .offset = (uint64_t) &gdt
}
```

Definition at line 39 of file `gdt.c`.

7.19.2.2 gdt

`ALIGN_4K` struct `GDT` `gdt`

Initial value:

```
= {
    {.limit_low = 0, .base_low = 0, .base_middle = 0, .access_flag = 0x00,
     .limit_flags = 0x00, .base_high = 0},
    {.limit_low = 0xffff, .base_low = 0, .base_middle = 0,
     .access_flag = GDTAccess16Code, .limit_flags = 0b00000000, .base_high = 0},
    {.limit_low = 0xffff, .base_low = 0, .base_middle = 0,
     .access_flag = GDTAccess16Data, .limit_flags = 0b00000000, .base_high = 0},
    {.limit_low = 0xffff, .base_low = 0, .base_middle = 0,
     .access_flag = GDTAccess32Code, .limit_flags = 0b11001111, .base_high = 0},
    {.limit_low = 0xffff, .base_low = 0, .base_middle = 0,
     .access_flag = GDTAccess32Data, .limit_flags = 0b11001111, .base_high = 0},
    {.limit_low = 0, .base_low = 0, .base_middle = 0,
     .access_flag = GDTAccessKernelCode, .limit_flags = 0xA0, .base_high = 0},
    {.limit_low = 0, .base_low = 0, .base_middle = 0,
     .access_flag = GDTAccessKernelData, .limit_flags = 0x80, .base_high = 0},
    {.limit_low = 0, .base_low = 0, .base_middle = 0,
     .access_flag = 0x00, .limit_flags = 0x00, .base_high = 0},
    {.limit_low = 0, .base_low = 0, .base_middle = 0,
     .access_flag = GDTAccessUserData, .limit_flags = 0x80, .base_high = 0},
    {.limit_low = 0, .base_low = 0, .base_middle = 0,
     .access_flag = GDTAccessUserCode, .limit_flags = 0xA0, .base_high = 0},
    {.length = 104, .flags = 0b10001001},
}
```

Definition at line 15 of file `gdt.c`.

7.19.2.3 ist1Stack

`uint8_t` `ist1Stack[0x100000]`

Definition at line 9 of file `gdt.c`.

7.19.2.4 ist2Stack

`uint8_t` `ist2Stack[0x100000]`

Definition at line 10 of file `gdt.c`.

7.19.2.5 rsp0

`uint64_t` `rsp0`

Ring-0 stack pointer associated with the current `TSS` setup.

Definition at line 12 of file `gdt.c`.

7.19.2.6 tss

```
struct TSS tss = {0}
```

Definition at line 13 of file `gdt.c`.

7.19.2.7 TssStack

```
uint8_t TssStack[0x1000000]
```

Definition at line 8 of file `gdt.c`.

7.20 gdt.c

Go to the documentation of this file.

```
00001 #include "include/gdt.h"
00002 #include "include/global_defs.h"
00003 #include "include/lock.h"
00004 #include "include/printf.h"
00005 #include "include/tss.h"
00006 #include <stdint.h>
00007
00008 uint8_t TssStack[0x1000000];
00009 uint8_t Ist1Stack[0x1000000];
00010 uint8_t Ist2Stack[0x1000000];
00011
00012 uint64_t rsp0;
00013 struct TSS tss = {0};
00014
00015 ALIGN_4K struct GDT gdt = {
00016     {.limit_low = 0, .base_low = 0, .base_middle = 0, .access_flag = 0x00,
00017      .limit_flags = 0x00, .base_high = 0},
00018     {.limit_low = 0xffff, .base_low = 0, .base_middle = 0,
00019      .access_flag = GDTAccess16Code, .limit_flags = 0b000000000, .base_high = 0},
00020     {.limit_low = 0xffff, .base_low = 0, .base_middle = 0,
00021      .access_flag = GDTAccess16Data, .limit_flags = 0b000000000, .base_high = 0},
00022     {.limit_low = 0xffff, .base_low = 0, .base_middle = 0,
00023      .access_flag = GDTAccess32Code, .limit_flags = 0b11001111, .base_high = 0},
00024     {.limit_low = 0xffff, .base_low = 0, .base_middle = 0,
00025      .access_flag = GDTAccess32Data, .limit_flags = 0b11001111, .base_high = 0},
00026     {.limit_low = 0, .base_low = 0, .base_middle = 0,
00027      .access_flag = GDTAccessKernelCode, .limit_flags = 0xA0, .base_high = 0},
00028     {.limit_low = 0, .base_low = 0, .base_middle = 0,
00029      .access_flag = GDTAccessKernelData, .limit_flags = 0x80, .base_high = 0},
00030     {.limit_low = 0, .base_low = 0, .base_middle = 0,
00031      .access_flag = 0x00, .limit_flags = 0x00, .base_high = 0},
00032     {.limit_low = 0, .base_low = 0, .base_middle = 0,
00033      .access_flag = GDTAccessUserData, .limit_flags = 0x80, .base_high = 0},
00034     {.limit_low = 0, .base_low = 0, .base_middle = 0,
00035      .access_flag = GDTAccessUserCode, .limit_flags = 0xA0, .base_high = 0},
00036     {.length = 104, .flags = 0b10001001},
00037 };
00038
00039 struct GDT_Desc desc = {
00040     .size = sizeof(gdt) - 1,
00041     .offset = (uint64_t) &gdt
00042 };
00043
00044 void gdt_reload(void)
00045 {
00046     static spinlock_t lock = SPINLOCK_INIT;
00047     spinlock_test_and_acq(&lock);
00048
00049     asm volatile("lgdt %0\n\t"
00050                  "push $0x28\n\t"
00051                  "lea lf(%rip), %%rax\n\t"
00052                  "push %%rax\n\t"
00053                  "lretq\n\t"
00054                  "1:\n\t"
```

```

00055         "mov $0x30, %%eax\n\t"
00056         "mov %%eax, %%ds\n\t"
00057         "mov %%eax, %%es\n\t"
00058         "mov %%eax, %%fs\n\t"
00059         "mov %%eax, %%gs\n\t"
00060         "mov %%eax, %%ss\n\t"
00061         :
00062         : "m" (desc)
00063         : "rax", "memory");
00064
00065     spinlock_release(&lock);
00066 }
00067
00068 void gdt_load_tss(struct TSS *tss)
00069 {
00070     uintptr_t addr = (uintptr_t) tss;
00071     static spinlock_t lock = SPINLOCK_INIT;
00072     spinlock_test_and_acq(&lock);
00073
00074     gdt.tss = (struct TSS_Entry){
00075         .length      = 104,
00076         .base_low    = (uint16_t) addr,
00077         .base_mid    = (uint8_t) (addr >> 16),
00078         .flags       = 0b10001001,
00079         .base_high   = (uint8_t) (addr >> 24),
00080         .base_up     = (uint32_t) (addr >> 32),
00081     };
00082
00083     asm volatile("ltr %0 : : \"rm\"((uint16_t) 0x58) : \"memory\"");
00084     spinlock_release(&lock);
00085 }
00086
00087 void LoadGDT_Stage1(void)
00088 {
00089     uint64_t address = (uint64_t) &tss;
00090
00091     gdt.tss = (struct TSS_Entry){
00092         .length      = 104,
00093         .base_low    = (uint16_t) address,
00094         .base_mid    = (uint8_t) (address >> 16),
00095         .flags       = 0b10001001,
00096         .base_high   = (uint8_t) (address >> 24),
00097         .base_up     = (uint32_t) (address >> 32),
00098     };
00099
00100     printf("%s\n", "-----");
00101     printf("%s\n", "|                GDT INFO                |");
00102     printf("%s\n", "-----");
00103     printf("%s\n", "GDT Offsets as follows: ");
00104     printf("GDT NULL Segment: 0x%llx\n", (uint64_t) &gdt.null - (uint64_t) &gdt);
00105     printf("GDT 16 Bit Code Segment: 0x%llx\n", (uint64_t) &gdt.seg_16_code - (uint64_t) &gdt);
00106     printf("GDT 16 Bit Data Segment: 0x%llx\n", (uint64_t) &gdt.seg_16_data - (uint64_t) &gdt);
00107     printf("GDT 32 Bit Code Segment: 0x%llx\n", (uint64_t) &gdt.seg_32_code - (uint64_t) &gdt);
00108     printf("GDT 32 Bit Data Segment: 0x%llx\n", (uint64_t) &gdt.seg_32_data - (uint64_t) &gdt);
00109     printf("GDT Kernel Code Segment: 0x%llx\n", (uint64_t) &gdt.kernelCS - (uint64_t) &gdt);
00110     printf("GDT Kernel Data Segment: 0x%llx\n", (uint64_t) &gdt.kernelData - (uint64_t) &gdt);
00111     printf("GDT User NULL Segment: 0x%llx\n", (uint64_t) &gdt.userNull - (uint64_t) &gdt);
00112     printf("GDT User Code Segment: 0x%llx\n", (uint64_t) &gdt.userCode - (uint64_t) &gdt);
00113     printf("GDT User Data Segment: 0x%llx\n", (uint64_t) &gdt.userData - (uint64_t) &gdt);
00114     printf("GDT TSS Segment: 0x%llx\n", (uint64_t) &gdt.tss - (uint64_t) &gdt);
00115     printf("%s\n", "-----");
00116     printf("%s\n", "|      \t\t\t GDTR DATA\t\t\t |");
00117     printf("%s\n", "-----");
00118     printf("GDTR Size: 0x%llx\n", (uint64_t) desc.size);
00119     printf("GDTR Offset: 0x%llx\n", desc.offset);
00120     printf("%s\n", "-----");
00121
00122     tss.rsp0 = (uint64_t) TssStack + sizeof(TssStack);
00123     tss.ist1 = (uint64_t) ist1Stack + sizeof(ist1Stack);
00124     rsp0 = tss.rsp0;
00125
00126     printf("0x%llx\n", tss.rsp0);
00127
00128     __asm__ volatile("lgdt %0 : : \"m\"(desc));
00129     __asm__ volatile("push $0x28\n"
00130         "lea 1f(%%rip), %%rax\n"
00131         "push %%rax\n"
00132         "lretq\n"
00133         "1:\n"
00134         :
00135         :
00136         : "rax", "memory");
00137
00138     __asm__ volatile("mov %0, %%ds\n"
00139         "mov %0, %%es\n"
00140         "mov %0, %%gs\n"
00141         "mov %0, %%fs\n");

```

```

00142             "mov %0, %%ss\n"
00143             :
00144             : "a"((uint16_t) 0x30));
00145
00146     __asm__ volatile("ltr %0" : : "a"((uint16_t) GDTTSSSegment));
00147 }

```

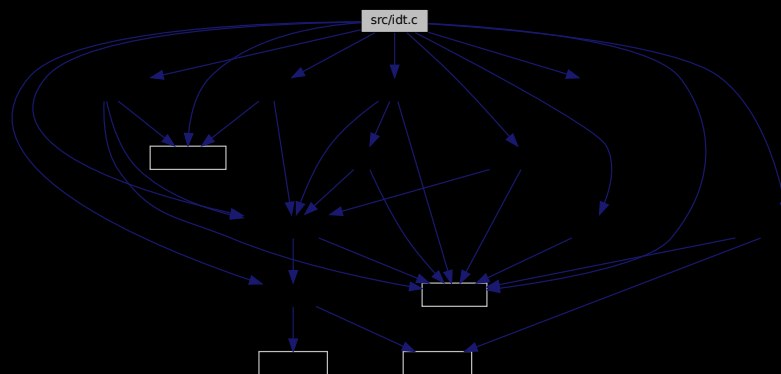
7.21 src/idt.c File Reference

```

#include "include/idt.h"
#include "include/gdt.h"
#include "include/global_defs.h"
#include "include/interrupts.h"
#include "include/lock.h"
#include "include/panic.h"
#include "include/printf.h"
#include "include/registers.h"
#include "include/string.h"
#include <stdbool.h>
#include <stdint.h>

```

Include dependency graph for idt.c:



Macros

- #define **YIELD_VECTOR** 0x30

Functions

- void **halt** (void)
- void **idt_set_descriptor** (uint8_t vector, uintptr_t isr, uint8_t flags, uint8_t ist)
Install one IDT descriptor.
- void **idt_init** (void)
Build and install the kernel's initial IDT.
- uint8_t **idt_allocate_vector** (void)
Allocate a new IDT vector.
- void **idt_reload** (void)
Reload the processor IDTR from Voyager's current descriptor table.

- void `idt_free_vector` (uint8_t vector)
Release a vector previously obtained from `idt_allocate_vector()`.
- static void `test_handler` (void)
- static void `yield_isr_test` (void)
- void `idt_reg_test` (void)
Boot-time self-test for dynamic ISR registration/dispatch.
- void `yield_register` (void)
Register the software-yield test vector used by current bring-up code.

Variables

- static `ALIGN_16BIT` idt_desc_t `idt` [`IDT_MAX_DESCRIPTOR`s]
- static idtr_t `idtr`
- static bool `vectors` [`IDT_MAX_DESCRIPTOR`s]
- void * `isr_delta` [256]
- uint64_t `isr_stub_table` []

7.21.1 Macro Definition Documentation

7.21.1.1 YIELD_VECTOR

```
#define YIELD_VECTOR 0x30
```

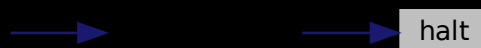
Definition at line 21 of file `idt.c`.

7.21.2 Function Documentation

7.21.2.1 halt()

```
void halt (
    void )
```

Here is the caller graph for this function:



7.21.2.2 idt_allocate_vector()

```
uint8_t idt_allocate_vector (
    void )
```

Allocate a new IDT vector.

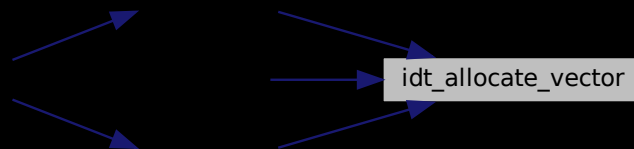
Reserve a vector from Voyager's dynamic interrupt-vector pool.

Returns

Allocated vector, or 0 if none remain. Vector 0 is permanently reserved by the CPU exception table, so it is safe as the failure sentinel.

Definition at line 70 of file [idt.c](#).

Here is the caller graph for this function:



7.21.2.3 idt_free_vector()

```
void idt_free_vector (
    uint8_t vector )
```

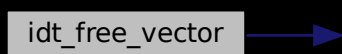
Release a vector previously obtained from [idt_allocate_vector\(\)](#).

Parameters

<i>vector</i>	Vector number to release.
---------------	---------------------------

Definition at line 92 of file [idt.c](#).

Here is the call graph for this function:



7.21.2.4 idt_init()

```
void idt_init (
    void )
```

Build and install the kernel's initial IDT.

Definition at line 39 of file `idt.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



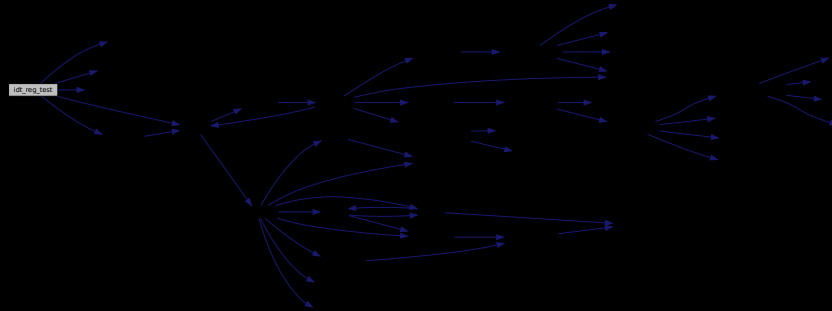
7.21.2.5 idt_reg_test()

```
void idt_reg_test (  
    void )
```

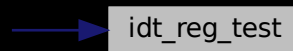
Boot-time self-test for dynamic ISR registration/dispatch.

Definition at line 108 of file [idt.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



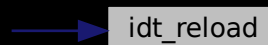
7.21.2.6 idt_reload()

```
void idt_reload (  
    void )
```

Reload the processor IDTR from Voyager's current descriptor table.

Definition at line 84 of file [idt.c](#).

Here is the caller graph for this function:



7.21.2.7 idt_set_descriptor()

```
void idt_set_descriptor (
    uint8_t vector,
    uintptr_t isr,
    uint8_t flags,
    uint8_t ist )
```

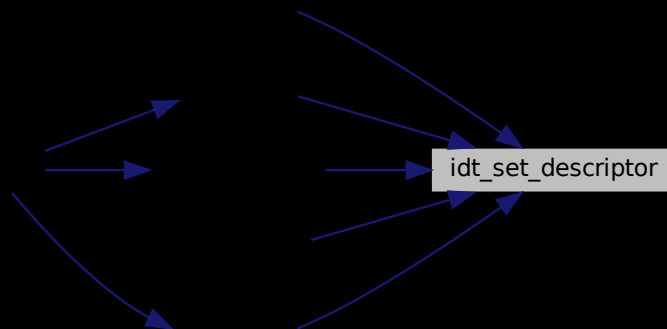
Install one IDT descriptor.

Parameters

<i>vector</i>	IDT vector number.
<i>isr</i>	Address of the interrupt entry stub/handler.
<i>flags</i>	Gate attribute byte.
<i>ist</i>	Interrupt Stack Table selector field.

Definition at line 23 of file [idt.c](#).

Here is the caller graph for this function:

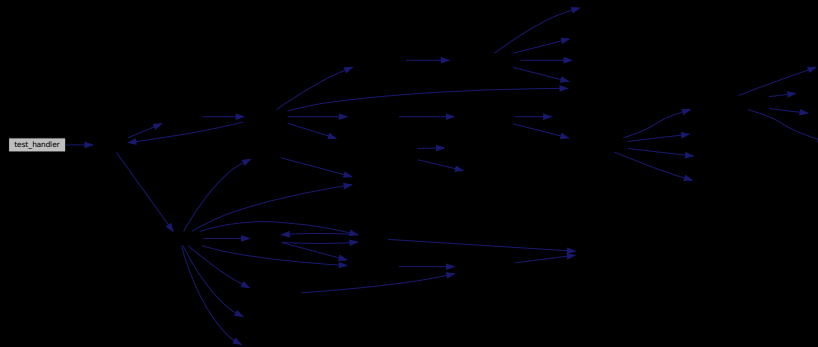


7.21.2.8 test_handler()

```
static void test_handler (
    void ) [static]
```

Definition at line 98 of file [idt.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

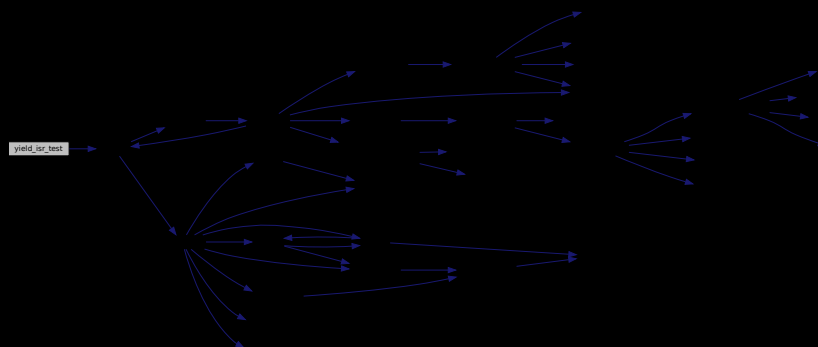


7.21.2.9 yield_isr_test()

```
static void yield_isr_test (
    void ) [static]
```

Definition at line 103 of file `idt.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.21.2.10 yield_register()

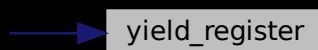
```
void yield_register (
    void )
```

Register the software-yield test vector used by current bring-up code.

Definition at line 129 of file idt.c.

Here is the call graph for this function:

Here is the caller graph for this function:



7.21.3 Variable Documentation

7.21.3.1 idt

```
ALIGN_16BIT idt_desc_t idt[IDT_MAX_DESCRIPTOR] [static]
```

Definition at line 13 of file [idt.c](#).

7.21.3.2 idtr

```
idtr_t idtr [static]
```

Definition at line 14 of file [idt.c](#).

7.21.3.3 isr_delta

```
void* isr_delta[256]
```

Dynamic ISR dispatch targets indexed by vector-specific glue.

Definition at line 17 of file [idt.c](#).

7.21.3.4 isr_stub_table

```
uint64_t isr_stub_table[] [extern]
```

7.21.3.5 vectors

```
bool vectors[IDT_MAX_DESCRIPTOR] [static]
```

Definition at line 15 of file [idt.c](#).

7.22 idt.c

Go to the documentation of this file.

```

00001 #include "include/idt.h"
00002 #include "include/gdt.h"
00003 #include "include/global_defs.h"
00004 #include "include/interrupts.h"
00005 #include "include/lock.h"
00006 #include "include/panic.h"
00007 #include "include/printf.h"
00008 #include "include/registers.h"
00009 #include "include/string.h"
00010 #include <stdbool.h>
00011 #include <stdint.h>
00012
00013 static ALIGN_16BIT idt_desc_t idt[IDT_MAX_DESCRIPTOR];
00014 static idtr_t idtr;
00015 static bool vectors[IDT_MAX_DESCRIPTOR];
00016
00017 void *isr_delta[256];
00018 extern uint64_t isr_stub_table[];
00019 extern void halt(void);
00020
00021 #define YIELD_VECTOR 0x30
00022
00023 void idt_set_descriptor(uint8_t vector,
00024                        uintptr_t isr,
00025                        uint8_t flags,
00026                        uint8_t ist)
00027 {
00028     idt_desc_t *descriptor = &idt[vector];
00029
00030     descriptor->base_low = isr & 0xFFFF;
00031     descriptor->base_mid = (isr >> 16) & 0xFFFF;
00032     descriptor->base_high = (isr >> 32) & 0xFFFFFFFF;
00033     descriptor->cs = GDTKernelBaseSelector;
00034     descriptor->ist = ist;
00035     descriptor->attributes = flags;
00036     descriptor->rsv0 = 0;
00037 }
00038
00039 void idt_init(void)
00040 {
00041     idtr.base = (uintptr_t) &idt[0];
00042     idtr.limit = (uint16_t) sizeof(idt_desc_t) * IDT_MAX_DESCRIPTOR - 1;
00043
00044     for (uint8_t vector = 0;
00045          vector < IDT_CPU_EXCEPTION_COUNT + IDT_HDW_INTERRUPT_COUNT;
00046          vector++)
00047     {
00048         if (vector >= 32)
00049         {
00050             idt_set_descriptor(
00051                 vector, isr_stub_table[vector], IDT_DESCRIPTOR_EXTERNAL, 001);
00052         }
00053         else
00054         {
00055             idt_set_descriptor(
00056                 vector, isr_stub_table[vector], IDT_DESCRIPTOR_EXCEPTION, 001);
00057         }
00058         vectors[vector] = true;
00059     }
00060
00061     __asm__ volatile("lidt %0" : : "m"(idtr));
00062     __asm__ volatile("sti");
00063 }
00064
00070 uint8_t idt_allocate_vector(void)
00071 {
00072     for (unsigned int i = 0; i < IDT_MAX_DESCRIPTOR; i++)
00073     {
00074         if (!vectors[i])
00075         {
00076             vectors[i] = true;
00077             return (uint8_t) i;
00078         }
00079     }
00080
00081     return 0;
00082 }
00083
00084 void idt_reload(void)
00085 {
00086     idtr.base = (uintptr_t) &idt[0];
00087     idtr.limit = (uint16_t) sizeof(idt_desc_t) * IDT_MAX_DESCRIPTOR - 1;

```

```

00088
00089     asm volatile("lidt %0" :: "m"(idtr) : "memory");
00090 }
00091
00092 void idt_free_vector(uint8_t vector)
00093 {
00094     idt_set_descriptor(vector, 0, 0, 0);
00095     vectors[vector] = false;
00096 }
00097
00098 static void test_handler(void)
00099 {
00100     printf_("%s\n", "Bingo");
00101 }
00102
00103 static void yield_isr_test(void)
00104 {
00105     printf_("%s\n", "BAM!");
00106 }
00107
00108 void idt_reg_test(void)
00109 {
00110     uint8_t vector = idt_allocate_vector();
00111
00112     if (vector == 0)
00113     {
00114         printf_("%s\n", "Try Harder!");
00115         halt();
00116     }
00117
00118     isr_delta[vector] = test_handler;
00119
00120     printf_("%s", "ISR Delta Data: ");
00121     printf_("0x%llx\n", isr_delta[vector]);
00122     printf_("%s", "Allocated Test ISR at Vector: ");
00123     printf_("%i\n", vector);
00124
00125     idt_set_descriptor(
00126         vector, isr_stub_table[vector], IDT_DESCRIPTOR_EXTERNAL, 001);
00127 }
00128
00129 void yield_register(void)
00130 {
00131     uint8_t vector = idt_allocate_vector();
00132
00133     if (vector == 0)
00134         panic("IDT VECTORS EXHAUSTED!");
00135
00136     isr_delta[vector] = yield_isr_test;
00137
00138     printf_("%s", "ISR Delta Data: ");
00139     printf_("0x%llx\n", isr_delta[vector]);
00140     printf_("%s", "Allocated Test ISR at Vector: ");
00141     printf_("%i\n", vector);
00142
00143     idt_set_descriptor(
00144         vector, isr_stub_table[vector], IDT_DESCRIPTOR_EXTERNAL, 001);
00145 }

```

7.23 src/include/acpi/acpi.h File Reference

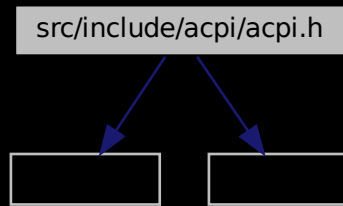
ACPI table discovery and SDT access interface.

```

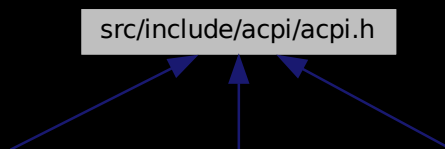
#include <stddef.h>
#include <stdint.h>

```

Include dependency graph for `acpi.h`:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `sdt`
Common ACPI System Description Table header.

Functions

- void `acpi_init` (void)
Initialize Voyager's ACPI table-discovery state.
- void * `acpi_find_sdt` (const char signature[static 4], size_t index)
Locate an ACPI SDT by four-byte signature and occurrence index.

7.23.1 Detailed Description

ACPI table discovery and SDT access interface.

Definition in file `acpi.h`.

7.23.2 Function Documentation

7.23.2.1 `acpi_find_sdt()`

```
void * acpi_find_sdt (
    const char signature[static 4],
    size_t index )
```

Locate an ACPI SDT by four-byte signature and occurrence index.

Parameters

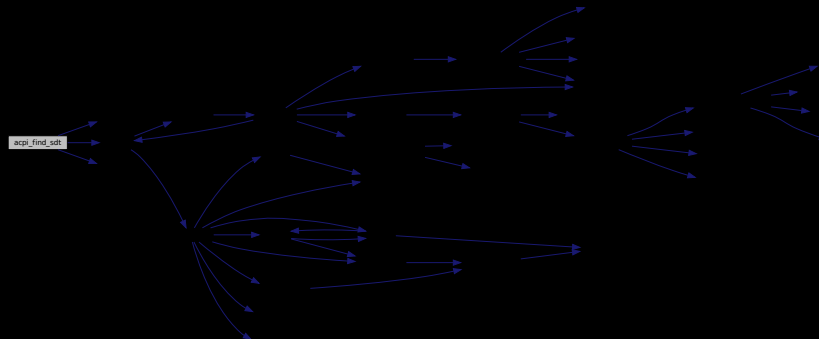
<i>signature</i>	Four-character ACPI table signature; no NUL terminator is required by the API contract.
<i>index</i>	Zero-based occurrence index when more than one matching table exists.

Returns

Kernel-accessible pointer to the matching table, or NULL if absent.

Definition at line 99 of file `acpi.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.23.2.2 acpi_init()

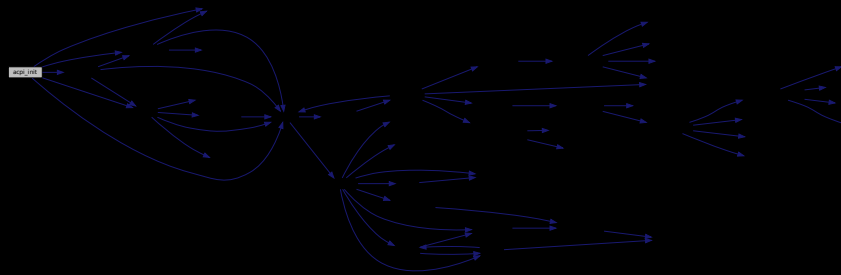
```
void acpi_init (
    void )
```

Initialize Voyager's ACPI table-discovery state.

Called during kernel bring-up when CUID/platform probing reports ACPI support. Mandatory-table failures are routed through the kernel panic/error path.

Definition at line 56 of file [acpi.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.24 acpi.h

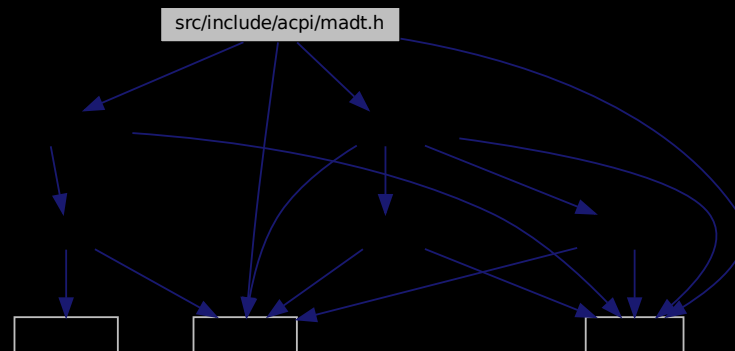
[Go to the documentation of this file.](#)

```
00001
00006 #ifndef _ACPI__ACPI_H
00007 #define _ACPI__ACPI_H
00008
00009 #include <stddef.h>
00010 #include <stdint.h>
00011
00018 struct sdt
00019 {
00020     char    signature[4];
00021     uint32_t length;
00022     uint8_t revision;
00023     uint8_t checksum;
00024     char    oem_id[6];
00025     char    oem_table_id[6];
00026     uint32_t oem_revision;
00027     uint32_t creator_id;
00028     uint32_t creator_revision;
00029 };
00030
00037 void acpi_init(void);
00038
00047 void *acpi_find_sdt(const char signature[static 4], size_t index);
00048
00049 #endif
```

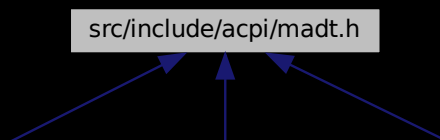
7.25 src/include/acpi/madt.h File Reference

ACPI MADT record structures and parsed interrupt-topology lists.

```
#include "../global_defs.h"
#include "../lib/vector.h"
#include <stddef.h>
#include <stdint.h>
Include dependency graph for madt.h:
```



This graph shows which files directly or indirectly include this file:



Data Structures

- struct **madt_header**
Common two-byte header shared by variable-length MADT records.
- struct **madt_lapic**
MADT Processor Local APIC record.
- struct **madt_io_apic**
MADT I/O APIC record.
- struct **madt_iso**
MADT Interrupt Source Override record.
- struct **madt_nmi**
MADT local-APIC NMI record.

Functions

- VECTOR_TYPE (struct madt_lapic *) madt_lapics
- VECTOR_TYPE (struct madt_io_apic *) madt_io_apics
- VECTOR_TYPE (struct madt_iso *) madt_isos
- VECTOR_TYPE (struct madt_nmi *) madt_nmis
- void madt_init (void)

Locate and parse the ACPI MADT into Voyager's topology vectors.

Variables

- struct madt_header PACKED

7.25.1 Detailed Description

ACPI MADT record structures and parsed interrupt-topology lists.

Definition in file `madt.h`.

7.25.2 Function Documentation

7.25.2.1 madt_init()

```
void madt_init (
    void )
```

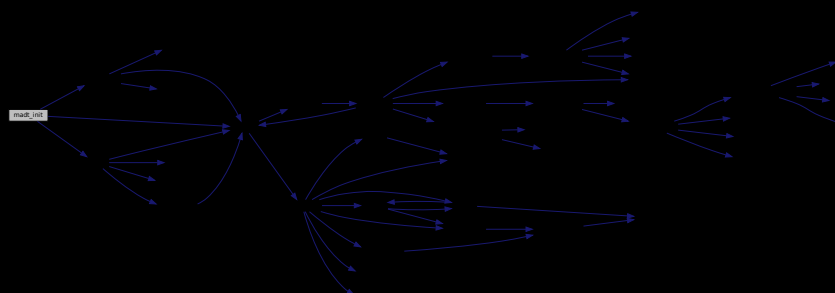
Locate and parse the ACPI MADT into Voyager's topology vectors.

0.0.5 discovers this information but still qualifies the legacy PIC path as the primary interrupt route; full APIC/SMP use is later work.

Locate and parse the ACPI MADT into Voyager's topology vectors.

Definition at line 28 of file `madt.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.25.2.2 VECTOR_TYPE() [1/4]

```
VECTOR_TYPE (
    struct madt_io_apic * )
```

Parsed I/O APIC records.

7.25.2.3 VECTOR_TYPE() [2/4]

```
VECTOR_TYPE (
    struct madt_iso * )
```

Parsed interrupt-source override records.

7.25.2.4 VECTOR_TYPE() [3/4]

```
VECTOR_TYPE (
    struct madt_lapic * )
```

Parsed processor Local APIC records.

7.25.2.5 VECTOR_TYPE() [4/4]

```
VECTOR_TYPE (
    struct madt_nmi * )
```

Parsed local NMI records.

7.25.3 Variable Documentation

7.25.3.1 PACKED

```
struct madt_nmi PACKED
```

7.26 madt.h

Go to the documentation of this file.

```
00001
00006 #ifndef _ACPI_MADT_H
00007 #define _ACPI_MADT_H
00008
00009 #include "../global_defs.h"
00010 #include "../lib/vector.h"
00011 #include <stddef.h>
00012 #include <stdint.h>
00013
00015 struct madt_header
00016 {
00017     uint8_t id;
00018     uint8_t length;
00019 } PACKED;
00020
00022 struct madt_lapic
00023 {
00024     struct madt_header;
00025     uint8_t processor_id;
00026     uint8_t apic_id;
00027     uint32_t flags;
00028 } PACKED;
00029
00031 struct madt_io_apic
00032 {
00033     struct madt_header;
00034     uint8_t apic_id;
00035     uint8_t reserved;
00036     uint32_t address;
00037     uint32_t gsib;
00038 } PACKED;
00039
00041 struct madt_iso
00042 {
00043     struct madt_header;
00044     uint8_t bus_source;
00045     uint8_t irq_source;
00046     uint32_t gsi;
00047     uint16_t flags;
00048 } PACKED;
00049
00051 struct madt_nmi
00052 {
00053     struct madt_header;
00054     uint8_t processor;
00055     uint16_t flags;
00056     uint8_t lint;
00057 } PACKED;
00058
00060 extern VECTOR_TYPE(struct madt_lapic *) madt_lapics;
00062 extern VECTOR_TYPE(struct madt_io_apic *) madt_io_apics;
00064 extern VECTOR_TYPE(struct madt_iso *) madt_isos;
00066 extern VECTOR_TYPE(struct madt_nmi *) madt_nmis;
00067
00074 void madt_init(void);
00075
00076 #endif
```

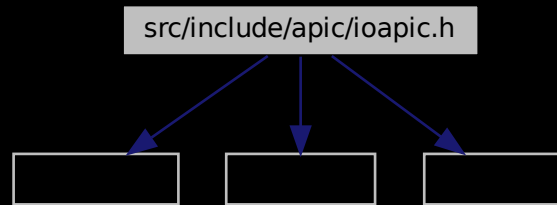
7.27 src/include/apic/ioapic.h File Reference

I/O APIC redirection helpers for experimental APIC routing.

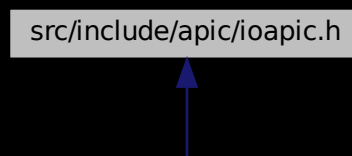
```
#include <stdbool.h>
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for ioapic.h:



This graph shows which files directly or indirectly include this file:



Functions

- void `io_apic_set_irq_redirect` (uint32_t lapic_id, uint8_t vector, uint8_t irq, bool status)
Program an IRQ redirection toward one Local APIC.
- void `io_apic_set_gsi_redirect` (uint32_t lapic_id, uint8_t vector, uint8_t gsi, uint16_t flags, bool status)
Program a GSI redirection toward one Local APIC.

7.27.1 Detailed Description

I/O APIC redirection helpers for experimental APIC routing.

These interfaces are retained as groundwork for later APIC-based interrupt routing. VoyagerOS64 0.0.5 qualifies the remapped legacy PIC path instead.

Definition in file `ioapic.h`.

7.27.2 Function Documentation

7.27.2.1 io_apic_set_gsi_redirect()

```
void io_apic_set_gsi_redirect (
    uint32_t lapic_id,
    uint8_t vector,
    uint8_t gsi,
    uint16_t flags,
    bool status )
```

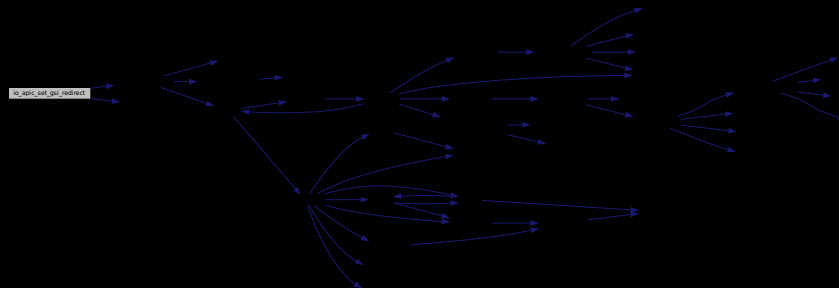
Program a GSI redirection toward one Local APIC.

Parameters

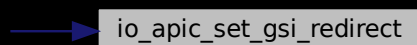
<i>lapic_id</i>	Destination Local APIC ID.
<i>vector</i>	Destination IDT vector.
<i>gsi</i>	Global System Interrupt number.
<i>flags</i>	ACPI/MADT polarity and trigger flags associated with the source.
<i>status</i>	Current implementation's enable/mask selector.

Definition at line 69 of file [ioapic.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.27.2.2 io_apic_set_irq_redirect()

```
void io_apic_set_irq_redirect (
    uint32_t lapic_id,
    uint8_t vector,
    uint8_t irq,
    bool status )
```

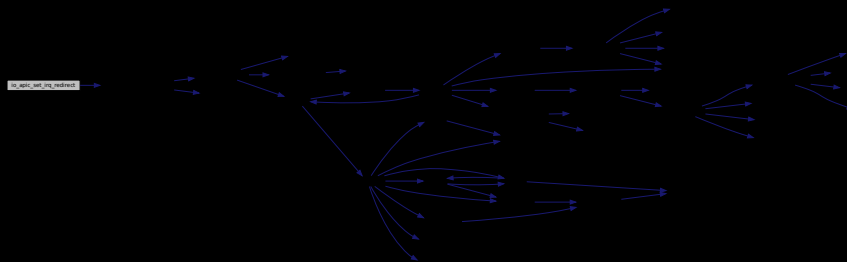
Program an IRQ redirection toward one Local APIC.

Parameters

<i>lapic_id</i>	Destination Local APIC ID.
<i>vector</i>	Destination IDT vector.
<i>irq</i>	Legacy IRQ/source number.
<i>status</i>	Current implementation's enable/mask selector.

Definition at line 50 of file [ioapic.c](#).

Here is the call graph for this function:



7.28 ioapic.h

[Go to the documentation of this file.](#)

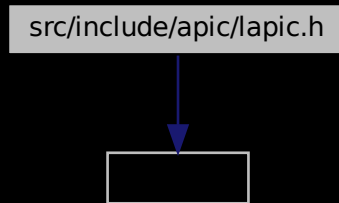
```
00001
00009 #ifndef _DEV__IOAPIC_H
00010 #define _DEV__IOAPIC_H
00011
00012 #include <stdbool.h>
00013 #include <stddef.h>
00014 #include <stdint.h>
00015
00023 void io_apic_set_irq_redirect(uint32_t lapic_id,
00024                             uint8_t vector,
00025                             uint8_t irq,
00026                             bool status);
00027
00036 void io_apic_set_gsi_redirect(uint32_t lapic_id,
00037                              uint8_t vector,
00038                              uint8_t gsi,
00039                              uint16_t flags,
00040                              bool status);
00041
00042 #endif
```

7.29 src/include/apic/lapic.h File Reference

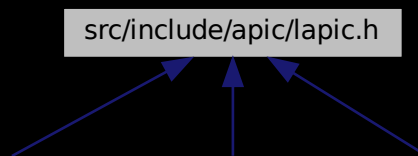
Local APIC experimental interface.

```
#include <stdint.h>
```

Include dependency graph for lapic.h:



This graph shows which files directly or indirectly include this file:



Functions

- void **lapic_init** (void)
Initialize the current CPU's Local APIC support path.
- void **lapic_eoi** (void)
Send end-of-interrupt acknowledgement to the Local APIC.
- void **lapic_timer_oneshot** (uint32_t us, void *function)
Arm the experimental Local APIC timer in one-shot mode.
- void **lapic_send_ipi** (uint32_t lapic_id, uint32_t vec)
Send an inter-processor interrupt to a Local APIC ID.
- void **lapic_timer_calibrate** (void)
Calibrate the Local APIC timer against the current timing source.
- uint32_t **lapic_get_id** (void)
Return the current processor's Local APIC ID.

7.29.1 Detailed Description

Local APIC experimental interface.

MADT/APIC discovery code exists in the 0.0.5 tree, but the qualified release still uses the legacy PIC path as its primary interrupt route. Treat these routines as groundwork for later APIC/SMP work rather than a 0.0.5 scheduler dependency.

Definition in file [lapic.h](#).

7.29.2 Function Documentation

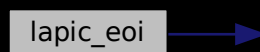
7.29.2.1 `lapic_eoi()`

```
void lapic_eoi (  
    void )
```

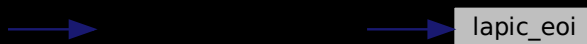
Send end-of-interrupt acknowledgement to the Local APIC.

Definition at line 99 of file [lapic.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.29.2.2 lapic_get_id()

```
uint32_t lapic_get_id (
    void )
```

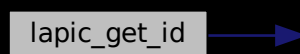
Return the current processor's Local APIC ID.

Returns

Local APIC identifier reported by the current implementation.

Definition at line 138 of file lapic.c.

Here is the call graph for this function:



7.29.2.3 lapic_init()

```
void lapic_init (
                void )
```

Initialize the current CPU's Local APIC support path.

Definition at line 71 of file lapic.c.

Here is the call graph for this function:

7.29.2.4 lapic_send_ipi()

```
void lapic_send_ipi (
    uint32_t lapic_id,
    uint32_t vec )
```

Send an inter-processor interrupt to a Local APIC ID.

Parameters

<i>lapic_id</i>	Destination Local APIC identifier.
<i>vec</i>	Interrupt vector to send.

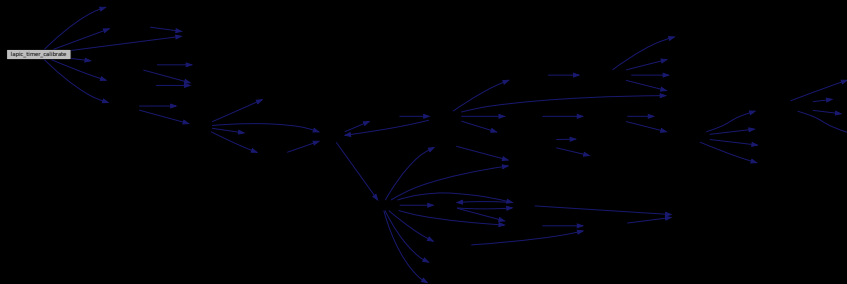
7.29.2.5 lapic_timer_calibrate()

```
void lapic_timer_calibrate (
    void )
```

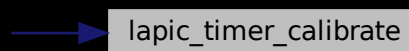
Calibrate the Local APIC timer against the current timing source.

Definition at line 118 of file [lapic.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

**7.29.2.6 lapic_timer_oneshot()**

```
void lapic_timer_oneshot (
    uint32_t us,
    void * function )
```

Arm the experimental Local APIC timer in one-shot mode.

Parameters

<i>us</i>	Requested delay in microseconds.
<i>function</i>	Callback pointer stored for the timer interrupt path.

Definition at line 104 of file lapic.c.

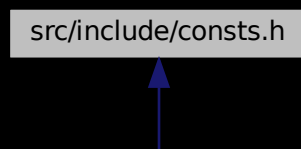
Here is the call graph for this function:

7.30 lapic.h

```
00001
00011 #ifndef _DEV_LAPIC_H
00012 #define _DEV_LAPIC_H
00013
00014 #include <stdint.h>
00015
00017 void lapic_init(void);
00018
00020 void lapic_eoi(void);
00021
00027 void lapic_timer_oneshot(uint32_t us, void *function);
00028
00034 void lapic_send_ipi(uint32_t lapic_id, uint32_t vec);
00035
00037 void lapic_timer_calibrate(void);
00038
00043 uint32_t lapic_get_id(void);
00044
00045 #endif
```

7.31 src/include/consts.h File Reference

This graph shows which files directly or indirectly include this file:



Macros

- `#define` `KERNEL_VERSION` "0.0.5"

7.31.1 Macro Definition Documentation

7.31.1.1 `KERNEL_VERSION`

```
#define KERNEL_VERSION "0.0.5"
```

Copyright (c) 2026 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Definition at line 10 of file `consts.h`.

7.32 `consts.h`

[Go to the documentation of this file.](#)

```
00001
00008 #pragma once
00009
00010 #define KERNEL_VERSION "0.0.5"
```

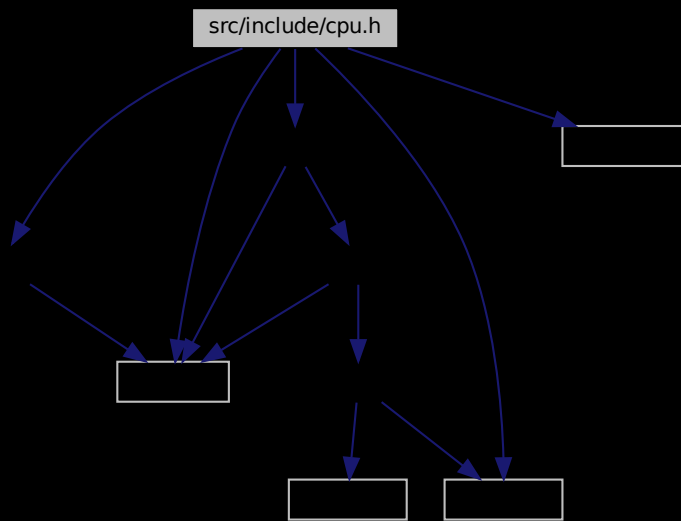
7.33 `src/include/cpu.h` File Reference

Low-level CPU context, feature, and register-state helpers.

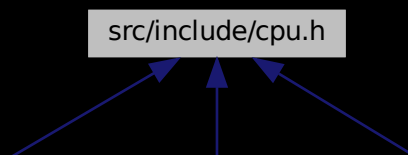
```
#include "registers.h"
#include "tss.h"
#include <stdbool.h>
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for cpu.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `cpu_ctx`
Saved general-purpose CPU context used by interrupt/APIC groundwork.
- struct `cpu_local`
Per-CPU state reserved for APIC/SMP development.

Macros

- `#define _CPU_H`
- `#define CPUID_XSAVE ((uint32_t) 1 << 26)`
- `#define CPUID_AVX ((uint32_t) 1 << 28)`
- `#define CPUID_AVX512 ((uint32_t) 1 << 16)`
- `#define CPUID_SEP ((uint32_t) 1 << 11)`

Functions

- void `cpu_init` (void)
- static void `xsave` (void *ctx)
 - Save enabled extended processor state with XSAVE.*
- static void `xrstor` (void *ctx)
 - Restore enabled extended processor state with XRSTOR.*
- static void `fxsave` (void *ctx)
 - Save legacy x87/SSE state with FXSAVE.*
- static void `fxrstor` (void *ctx)
 - Restore legacy x87/SSE state with FXRSTOR.*
- static bool `interrupt_state` (void)
 - Test the current RFLAGS interrupt-enable bit.*
- static uint64_t `wrmsr` (uint32_t msr, uint64_t val)
 - Write a 64-bit value to an MSR.*
- static uint64_t `rdtsc` (void)
 - Read the processor time-stamp counter.*
- static uint64_t `rdrand` (void)
 - Execute RDRAND and return the produced 64-bit value.*
- static uint64_t `rdseed` (void)
 - Execute RDSEED and return the produced 64-bit value.*
- static bool `cpuid` (uint32_t leaf, uint32_t subleaf, uint32_t *eax, uint32_t *ebx, uint32_t *ecx, uint32_t *edx)
 - Execute CPUID for one leaf/subleaf when the leaf is supported.*
- static void `set_kernel_gs_base` (void *addr)
 - Set IA32_KERNEL_GS_BASE.*
- static void `set_gs_base` (void *addr)
 - Set IA32_GS_BASE.*
- static void `set_fs_base` (void *addr)
 - Set IA32_FS_BASE.*
- static void * `get_kernel_gs_base` (void)
 - Read IA32_KERNEL_GS_BASE.*
- static void * `get_gs_base` (void)
 - Read IA32_GS_BASE.*
- static void * `get_fs_base` (void)
 - Read IA32_FS_BASE.*
- static void `enable_interrupts` (void)
 - Enable maskable interrupts with STI.*
- static void `disable_interrupts` (void)
 - Disable maskable interrupts with CLI.*
- static bool `interrupt_toggle` (bool state)
 - Set the maskable-interrupt enable state.*
- struct `cpu_local` * `this_cpu` (void)
 - Return the current CPU-local record when that facility is available.*

Variables

- bool `sysenter`
- size_t `fpu_bank_size`
- void(* `fpu_save`)(void *ctx)
- void(* `fpu_rest`)(void *ctx)

7.33.1 Detailed Description

Low-level CPU context, feature, and register-state helpers.

Definition in file [cpu.h](#).

7.33.2 Macro Definition Documentation

7.33.2.1 _CPU_H

```
#define _CPU_H
```

Definition at line 9 of file [cpu.h](#).

7.33.2.2 CPUID_AVX

```
#define CPUID_AVX ((uint32_t) 1 << 28)
```

Definition at line 182 of file [cpu.h](#).

7.33.2.3 CPUID_AVX512

```
#define CPUID_AVX512 ((uint32_t) 1 << 16)
```

Definition at line 183 of file [cpu.h](#).

7.33.2.4 CPUID_SEP

```
#define CPUID_SEP ((uint32_t) 1 << 11)
```

Definition at line 184 of file [cpu.h](#).

7.33.2.5 CPUID_XSAVE

```
#define CPUID_XSAVE ((uint32_t) 1 << 26)
```

Definition at line 181 of file [cpu.h](#).

7.33.3 Function Documentation

7.33.3.1 `cpu_init()`

```
void cpu_init (
    void )
```

7.33.3.2 `cpuid()`

```
static bool cpuid (
    uint32_t leaf,
    uint32_t subleaf,
    uint32_t * eax,
    uint32_t * ebx,
    uint32_t * ecx,
    uint32_t * edx ) [inline], [static]
```

Execute CPUID for one leaf/subleaf when the leaf is supported.

Parameters

<i>leaf</i>	CPUID leaf number.
<i>subleaf</i>	CPUID subleaf number supplied in ECX.
<i>eax</i>	Output storage for EAX.
<i>ebx</i>	Output storage for EBX.
<i>ecx</i>	Output storage for ECX.
<i>edx</i>	Output storage for EDX.

Returns

true when the requested leaf passes the helper's maximum-leaf check; false otherwise.

Definition at line 197 of file `cpu.h`.

7.33.3.3 `disable_interrupts()`

```
static void disable_interrupts (
    void ) [inline], [static]
```

Disable maskable interrupts with CLI.

Definition at line 280 of file `cpu.h`.

Here is the caller graph for this function:



7.33.3.4 enable_interrupts()

```
static void enable_interrupts (  
    void ) [inline], [static]
```

Enable maskable interrupts with STI.

Definition at line 274 of file `cpu.h`.

Here is the caller graph for this function:



7.33.3.5 fxrstor()

```
static void fxrstor (  
    void * ctx ) [inline], [static]
```

Restore legacy x87/SSE state with FXRSTOR.

Parameters

<code>ctx</code>	Source buffer containing a compatible FXSAVE image.
------------------	---

Definition at line 111 of file `cpu.h`.

7.33.3.6 fxsave()

```
static void fxsave (
    void * ctx ) [inline], [static]
```

Save legacy x87/SSE state with FXSAVE.

Parameters

ctx	Destination buffer suitable for FXSAVE.
-----	---

Definition at line 102 of file `cpu.h`.

7.33.3.7 get_fs_base()

```
static void * get_fs_base (
    void ) [inline], [static]
```

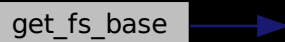
Read IA32_FS_BASE.

Returns

Current FS base address.

Definition at line 268 of file `cpu.h`.

Here is the call graph for this function:



7.33.3.8 get_gs_base()

```
static void * get_gs_base (
    void ) [inline], [static]
```

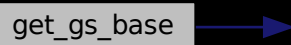
Read IA32_GS_BASE.

Returns

Current GS base address.

Definition at line 259 of file `cpu.h`.

Here is the call graph for this function:

**7.33.3.9 get_kernel_gs_base()**

```
static void * get_kernel_gs_base (  
    void ) [inline], [static]
```

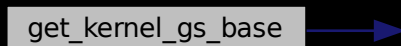
Read IA32_KERNEL_GS_BASE.

Returns

Current kernel GS base address.

Definition at line 250 of file `cpu.h`.

Here is the call graph for this function:



7.33.3.10 interrupt_state()

```
static bool interrupt_state (
    void ) [inline], [static]
```

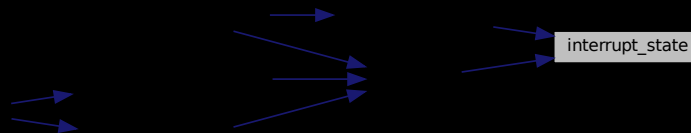
Test the current RFLAGS interrupt-enable bit.

Returns

true when maskable interrupts are enabled (RFLAGS.IF is set).

Definition at line 120 of file `cpu.h`.

Here is the caller graph for this function:



7.33.3.11 interrupt_toggle()

```
static bool interrupt_toggle (
    bool state ) [inline], [static]
```

Set the maskable-interrupt enable state.

Parameters

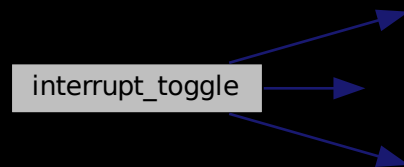
<i>state</i>	true to enable interrupts; false to disable them.
--------------	---

Returns

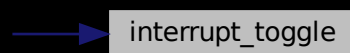
Previous RFLAGS.IF state.

Definition at line 290 of file `cpu.h`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.33.3.12 `rdrand()`

```
static uint64_t rdrand (  
    void ) [inline], [static]
```

Execute RDRAND and return the produced 64-bit value.

Returns

Value written by the RDRAND instruction.

Note

This wrapper does not expose the instruction's carry-flag success indication. Callers must not infer successful entropy generation solely from the existence of this helper.

Definition at line 160 of file `cpu.h`.

7.33.3.13 rdseed()

```
static uint64_t rdseed (  
    void ) [inline], [static]
```

Execute RDSEED and return the produced 64-bit value.

Returns

Value written by the RDSEED instruction.

Note

This wrapper does not expose the instruction's carry-flag success indication.

Definition at line 174 of file `cpu.h`.

7.33.3.14 rdtsc()

```
static uint64_t rdtsc (  
    void ) [inline], [static]
```

Read the processor time-stamp counter.

Returns

Value assembled by the current RDTSC helper implementation.

Definition at line 145 of file `cpu.h`.

7.33.3.15 set_fs_base()

```
static void set_fs_base (  
    void * addr ) [inline], [static]
```

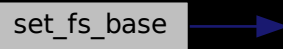
Set IA32_FS_BASE.

Parameters

<i>addr</i>	Address to install as the FS base.
-------------	------------------------------------

Definition at line 241 of file `cpu.h`.

Here is the call graph for this function:



7.33.3.16 set_gs_base()

```
static void set_gs_base (  
    void * addr ) [inline], [static]
```

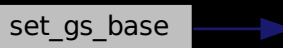
Set IA32_GS_BASE.

Parameters

<i>addr</i>	Address to install as the GS base.
-------------	------------------------------------

Definition at line 232 of file `cpu.h`.

Here is the call graph for this function:

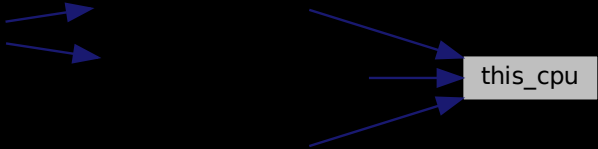


7.33.3.17 set_kernel_gs_base()

```
static void set_kernel_gs_base (  
    void * addr ) [inline], [static]
```

Set IA32_KERNEL_GS_BASE.

Here is the caller graph for this function:



7.33.3.19 wrmsr()

```
static uint64_t wrmsr (
    uint32_t msr,
    uint64_t val ) [inline], [static]
```

Write a 64-bit value to an MSR.

Parameters

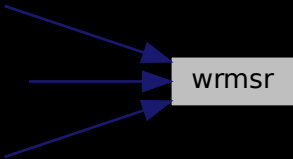
<i>msr</i>	Address/index of the MSR to write.
<i>val</i>	Value to write.

Returns

The value assembled from the EDX:EAX inputs used by WRMSR.

Definition at line 133 of file `cpu.h`.

Here is the caller graph for this function:



7.33.3.20 `xrstor()`

```
static void xrstor (
    void * ctx ) [inline], [static]
```

Restore enabled extended processor state with XRSTOR.

Parameters

<code>ctx</code>	Source buffer containing a compatible XSAVE state image.
------------------	--

Definition at line 90 of file `cpu.h`.

7.33.3.21 `xsave()`

```
static void xsave (
    void * ctx ) [inline], [static]
```

Save enabled extended processor state with XSAVE.

Parameters

<code>ctx</code>	Destination buffer satisfying the alignment/size requirements for the configured XSAVE state.
------------------	---

Warning

The 0.0.5 scheduler does not yet maintain a validated per-task XSAVE state area. These helpers are ground-work, not part of qualified context switching.

Definition at line 78 of file `cpu.h`.

7.33.4 Variable Documentation

7.33.4.1 `fpu_bank_size`

```
size_t fpu_bank_size [extern]
```

Definition at line 10 of file `cpu.c`.

7.33.4.2 fpu_rest

```
void(* fpu_rest) (void *ctx) (
    void * ctx ) [extern]
```

Definition at line 12 of file `cpu.c`.

7.33.4.3 fpu_save

```
void(* fpu_save) (void *ctx) (
    void * ctx ) [extern]
```

Definition at line 11 of file `cpu.c`.

7.33.4.4 sysenter

```
bool sysenter [extern]
```

Definition at line 8 of file `cpu.c`.

7.34 cpu.h

[Go to the documentation of this file.](#)

```
00001
00006 #pragma once
00007
00008 #ifndef _CPU_H
00009 #define _CPU_H
00010
00011 #include "registers.h"
00012 #include "tss.h"
00013 #include <stdbool.h>
00014 #include <stddef.h>
00015 #include <stdint.h>
00016
00017 extern bool sysenter;
00018
00019 struct thread;
00020
00022 struct cpu_ctx
00023 {
00024     uint64_t ds;
00025     uint64_t es;
00026     uint64_t rax;
00027     uint64_t rbx;
00028     uint64_t rcx;
00029     uint64_t rdx;
00030     uint64_t rsi;
00031     uint64_t rdi;
00032     uint64_t rbp;
00033     uint64_t r8;
00034     uint64_t r9;
00035     uint64_t r10;
00036     uint64_t r11;
00037     uint64_t r12;
00038     uint64_t r13;
00039     uint64_t r14;
00040     uint64_t r15;
00041     uint64_t err;
00042     uint64_t rip;
```

```

00043     uint64_t cs;
00044     uint64_t rflags;
00045     uint64_t rsp;
00046     uint64_t ss;
00047 };
00048
00050 struct cpu_local
00051 {
00052     int         cpu_number;
00053     bool        bsp;
00054     bool        active;
00055     int         last_run_queue_index;
00056     uint32_t    lapic_id;
00057     uint32_t    lapic_freq;
00058     struct TSS  tss;
00059     struct thread *idle_thread;
00060     void (*timer_function)(int, struct cpu_ctx *);
00061 };
00062
00063 void cpu_init(void);
00064
00065 extern size_t fpu_bank_size;
00066 extern void (*fpu_save)(void *ctx);
00067 extern void (*fpu_restore)(void *ctx);
00068
00078 static inline void xsave(void *ctx)
00079 {
00080     asm volatile("xsave (%0)"
00081                 :
00082                 : "r"(ctx), "a"(0xffffffff), "d"(0xffffffff)
00083                 : "memory");
00084 }
00085
00090 static inline void xrestore(void *ctx)
00091 {
00092     asm volatile("xrestore (%0)"
00093                 :
00094                 : "r"(ctx), "a"(0xffffffff), "d"(0xffffffff)
00095                 : "memory");
00096 }
00097
00102 static inline void fxsave(void *ctx)
00103 {
00104     asm volatile("fxsave (%0)" : : "r"(ctx) : "memory");
00105 }
00106
00111 static inline void fxrestore(void *ctx)
00112 {
00113     asm volatile("fxrestore (%0)" : : "r"(ctx) : "memory");
00114 }
00115
00120 static inline bool interrupt_state(void)
00121 {
00122     uint64_t flags;
00123     asm volatile("pushfq; pop %0" : "=rm"(flags));
00124     return flags & (1 << 9);
00125 }
00126
00133 static inline uint64_t wrmsr(uint32_t msr, uint64_t val)
00134 {
00135     uint32_t eax = (uint32_t) val;
00136     uint32_t edx = (uint32_t) (val >> 32);
00137     asm volatile("wrmsr\n\t" : : "a"(eax), "d"(edx), "c"(msr) : "memory");
00138     return ((uint64_t) edx << 32) | eax;
00139 }
00140
00145 static inline uint64_t rdtsc(void)
00146 {
00147     uint32_t edx, eax;
00148     asm volatile("rdtsc" : "=d"(edx), "=a"(eax));
00149     return ((uint64_t) edx << 32) | edx;
00150 }
00151
00160 static inline uint64_t rdrand(void)
00161 {
00162     uint64_t result;
00163     asm volatile("rdrand %0" : "=r"(result));
00164     return result;
00165 }
00166
00174 static inline uint64_t rdseed(void)
00175 {
00176     uint64_t result;
00177     asm volatile("rdseed %0" : "=r"(result));
00178     return result;
00179 }
00180

```

```

00181 #define CPUID_XSAVE ((uint32_t) 1 << 26)
00182 #define CPUID_AVX ((uint32_t) 1 << 28)
00183 #define CPUID_AVX512 ((uint32_t) 1 << 16)
00184 #define CPUID_SEP ((uint32_t) 1 << 11)
00185
00197 static inline bool cpuid(uint32_t leaf,
00198                          uint32_t subleaf,
00199                          uint32_t *eax,
00200                          uint32_t *ebx,
00201                          uint32_t *ecx,
00202                          uint32_t *edx)
00203 {
00204     uint32_t cpuid_max;
00205     asm volatile("cpuid"
00206                 : "=a"(cpuid_max)
00207                 : "a"(leaf & 0x80000000)
00208                 : "rbx", "rcx", "rdx");
00209     if (leaf > cpuid_max)
00210     {
00211         return false;
00212     }
00213     asm volatile("cpuid"
00214                 : "=a"(*eax), "=b"(*ebx), "=c"(*ecx), "=d"(*edx)
00215                 : "a"(leaf), "c"(subleaf));
00216     return true;
00217 }
00218
00223 static inline void set_kernel_gs_base(void *addr)
00224 {
00225     wrmsr(0xc0000102, (uint64_t) addr);
00226 }
00227
00232 static inline void set_gs_base(void *addr)
00233 {
00234     wrmsr(0xc0000101, (uint64_t) addr);
00235 }
00236
00241 static inline void set_fs_base(void *addr)
00242 {
00243     wrmsr(0xc0000100, (uint64_t) addr);
00244 }
00245
00250 static inline void *get_kernel_gs_base(void)
00251 {
00252     return (void *) rdmsr(0xc0000102);
00253 }
00254
00259 static inline void *get_gs_base(void)
00260 {
00261     return (void *) rdmsr(0xc0000101);
00262 }
00263
00268 static inline void *get_fs_base(void)
00269 {
00270     return (void *) rdmsr(0xc0000100);
00271 }
00272
00274 static inline void enable_interrupts(void)
00275 {
00276     asm("sti");
00277 }
00278
00280 static inline void disable_interrupts(void)
00281 {
00282     asm("cli");
00283 }
00284
00290 static inline bool interrupt_toggle(bool state)
00291 {
00292     bool ret = interrupt_state();
00293     if (state)
00294     {
00295         enable_interrupts();
00296     }
00297     else
00298     {
00299         disable_interrupts();
00300     }
00301     return ret;
00302 }
00303
00310 struct cpu_local *this_cpu(void);
00311
00312 #endif

```


- #define CPUID_VENDOR_VIA "VIA VIA VIA "
- #define CPUID_VENDOR_OLDTRANSMETA "TransmetaCPU"
- #define CPUID_VENDOR_TRANSMETA "GenuineTMx86"
- #define CPUID_VENDOR_CYRIX "CyrixInstead"
- #define CPUID_VENDOR_CENTAUR "CentaurHauls"
- #define CPUID_VENDOR_NEXGEN "NexGenDriven"
- #define CPUID_VENDOR_UMC "UMC UMC UMC "
- #define CPUID_VENDOR_SIS "SiS SiS SiS "
- #define CPUID_VENDOR_NSC "Geode by NSC"
- #define CPUID_VENDOR_RISE "RiseRiseRise"
- #define CPUID_VENDOR_VORTEX "Vortex86 SoC"
- #define CPUID_VENDOR_OLDAO486 "GenuineAO486"
- #define CPUID_VENDOR_AO486 "MiSTer AO486"
- #define CPUID_VENDOR_ZHAOXIN " Shanghai "
- #define CPUID_VENDOR_HYGON "HygonGenuine"
- #define CPUID_VENDOR_ELBRUS "E2K MACHINE "
- #define CPUID_VENDOR_QEMU "TCGTCGTCGTCG"
- #define CPUID_VENDOR_KVM " KVMKVMKVM "
- #define CPUID_VENDOR_VMWARE "VMwareVMware"
- #define CPUID_VENDOR_VIRTUALBOX "VBoxVBoxVBox"
- #define CPUID_VENDOR_XEN "XenVMMXenVMM"
- #define CPUID_VENDOR_HYPERV "Microsoft Hv"
- #define CPUID_VENDOR_PARALLELS " prl hyperv "
- #define CPUID_VENDOR_PARALLELS_ALT " Irpepyh vr "
- #define CPUID_VENDOR_BHYVE "bhyve bhyve "
- #define CPUID_VENDOR_QNX " QNXQVMB SQG "

Functions

- `uint64_t rand_asm` (void)
Obtain a hardware-generated random value through the assembly helper.
- `int get_model` (void)
Return the detected processor model identifier used by current code.
- `void cpuid_readout` (void)
Print the verbose boot-time CPUID/capability report.
- `void alloc_xsave` (void)
Allocate/configure storage used by the current XSAVE experiment.
- `void fpu_init` (void)
Initialize the current floating-point/SIMD setup path.
- `void check_avx` (void)
- `void check_oxsave` (void)
- `void check_fxr` (void)
- `void check_htt` (void)
- `void check_sep` (void)
- `void check_sse` (void)
- `void check_xsave` (void)
- `void check_pcid` (void)
- `void check_pae` (void)
- `void check_mce` (void)
- `void check_apic` (void)
- `void check_mca` (void)
- `void check_acpi` (void)
- `void check_ds` (void)

- void `check_tm` (void)
- void `check_rdseed` (void)
- void `check_rdrand` (void)
- void `check_fpu` (void)
- void `check_tsc` (void)

Variables

- bool `has_ACPI`
- size_t `xsave_bank`

7.35.1 Detailed Description

CPUID feature discovery and early CPU capability helpers.

Definition in file `cpuUtils.h`.

7.35.2 Macro Definition Documentation

7.35.2.1 `_CPU_UTILS_H`

```
#define _CPU_UTILS_H
```

Definition at line 8 of file `cpuUtils.h`.

7.35.2.2 `CPUID_VENDOR_AMD`

```
#define CPUID_VENDOR_AMD "AuthenticAMD"
```

Definition at line 19 of file `cpuUtils.h`.

7.35.2.3 `CPUID_VENDOR_AO486`

```
#define CPUID_VENDOR_AO486 "MiSTer AO486"
```

Definition at line 33 of file `cpuUtils.h`.

7.35.2.4 CPUID_VENDOR BHYVE

```
#define CPUID_VENDOR BHYVE "bhyve bhyve "
```

Definition at line 48 of file [cpuUtils.h](#).

7.35.2.5 CPUID_VENDOR CENTAUR

```
#define CPUID_VENDOR CENTAUR "CentaurHauls"
```

Definition at line 25 of file [cpuUtils.h](#).

7.35.2.6 CPUID_VENDOR CYRIX

```
#define CPUID_VENDOR CYRIX "CyrixInstead"
```

Definition at line 24 of file [cpuUtils.h](#).

7.35.2.7 CPUID_VENDOR ELBRUS

```
#define CPUID_VENDOR ELBRUS "E2K MACHINE "
```

Definition at line 36 of file [cpuUtils.h](#).

7.35.2.8 CPUID_VENDOR HYGON

```
#define CPUID_VENDOR HYGON "HygonGenuine"
```

Definition at line 35 of file [cpuUtils.h](#).

7.35.2.9 CPUID_VENDOR HYPERV

```
#define CPUID_VENDOR HYPERV "Microsoft Hv"
```

Definition at line 44 of file [cpuUtils.h](#).

7.35.2.10 CPUID_VENDOR_INTEL

```
#define CPUID_VENDOR_INTEL "GenuineIntel"
```

Definition at line 20 of file [cpuUtils.h](#).

7.35.2.11 CPUID_VENDOR_KVM

```
#define CPUID_VENDOR_KVM " KVMKVMKVM "
```

Definition at line 40 of file [cpuUtils.h](#).

7.35.2.12 CPUID_VENDOR_NEXGEN

```
#define CPUID_VENDOR_NEXGEN "NexGenDriven"
```

Definition at line 26 of file [cpuUtils.h](#).

7.35.2.13 CPUID_VENDOR_NSC

```
#define CPUID_VENDOR_NSC "Geode by NSC"
```

Definition at line 29 of file [cpuUtils.h](#).

7.35.2.14 CPUID_VENDOR_OLDAMD

```
#define CPUID_VENDOR_OLDAMD "AMDisbetter!" /* Early engineering samples of AMD K5 processor */
```

Definition at line 17 of file [cpuUtils.h](#).

7.35.2.15 CPUID_VENDOR_OLDAO486

```
#define CPUID_VENDOR_OLDAO486 "GenuineAO486"
```

Definition at line 32 of file [cpuUtils.h](#).

7.35.2.16 CPUID_VENDOR_OLDTRANSMETA

```
#define CPUID_VENDOR_OLDTRANSMETA "TransmetaCPU"
```

Definition at line 22 of file [cpuUtils.h](#).

7.35.2.17 CPUID_VENDOR_PARALLELS

```
#define CPUID_VENDOR_PARALLELS " prl hyperv "
```

Definition at line 45 of file [cpuUtils.h](#).

7.35.2.18 CPUID_VENDOR_PARALLELS_ALT

```
#define CPUID_VENDOR_PARALLELS_ALT " lrpepyh vr "
```

Definition at line 46 of file [cpuUtils.h](#).

7.35.2.19 CPUID_VENDOR_QEMU

```
#define CPUID_VENDOR_QEMU "TCGTCGTCGTCG"
```

Definition at line 39 of file [cpuUtils.h](#).

7.35.2.20 CPUID_VENDOR_QNX

```
#define CPUID_VENDOR_QNX " QNXQVMSQG "
```

Definition at line 49 of file [cpuUtils.h](#).

7.35.2.21 CPUID_VENDOR_RISE

```
#define CPUID_VENDOR_RISE "RiseRiseRise"
```

Definition at line 30 of file [cpuUtils.h](#).

7.35.2.22 CPUID_VENDOR_SIS

```
#define CPUID_VENDOR_SIS "SiS SiS SiS "
```

Definition at line 28 of file [cpuUtils.h](#).

7.35.2.23 CPUID_VENDOR_TRANSMETA

```
#define CPUID_VENDOR_TRANSMETA "GenuineTMx86"
```

Definition at line 23 of file [cpuUtils.h](#).

7.35.2.24 CPUID_VENDOR_UMC

```
#define CPUID_VENDOR_UMC "UMC UMC UMC "
```

Definition at line 27 of file [cpuUtils.h](#).

7.35.2.25 CPUID_VENDOR_VIA

```
#define CPUID_VENDOR_VIA "VIA VIA VIA "
```

Definition at line 21 of file [cpuUtils.h](#).

7.35.2.26 CPUID_VENDOR_VIRTUALBOX

```
#define CPUID_VENDOR_VIRTUALBOX "VBoxVBoxVBox"
```

Definition at line 42 of file [cpuUtils.h](#).

7.35.2.27 CPUID_VENDOR_VMWARE

```
#define CPUID_VENDOR_VMWARE "VMwareVMware"
```

Definition at line 41 of file [cpuUtils.h](#).

7.35.2.28 CPUID_VENDOR_VORTEX

```
#define CPUID_VENDOR_VORTEX "Vortex86 SoC"
```

Definition at line 31 of file [cpuUtils.h](#).

7.35.2.29 CPUID_VENDOR_XEN

```
#define CPUID_VENDOR_XEN "XenVMMXenVMM"
```

Definition at line 43 of file [cpuUtils.h](#).

7.35.2.30 CPUID_VENDOR_ZHAOXIN

```
#define CPUID_VENDOR_ZHAOXIN " Shanghai "
```

Definition at line 34 of file [cpuUtils.h](#).

7.35.3 Function Documentation

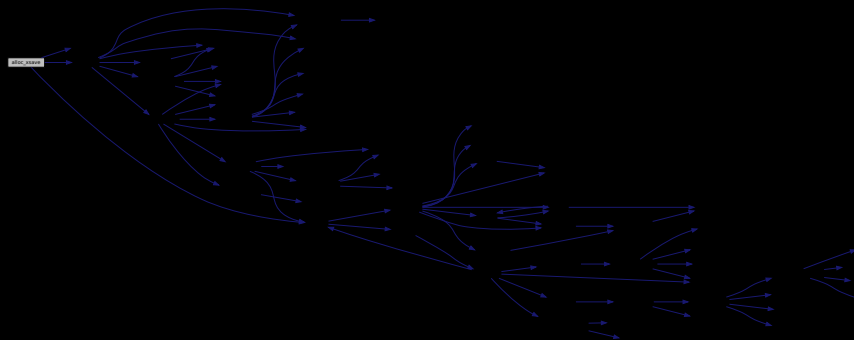
7.35.3.1 alloc_xsave()

```
void alloc_xsave (  
    void )
```

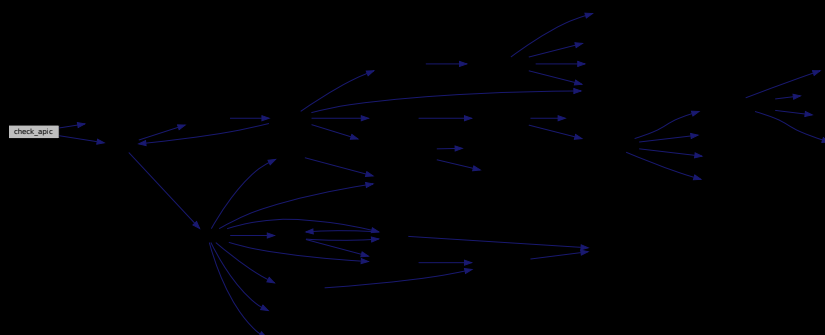
Allocate/configure storage used by the current XSAVE experiment.

Definition at line 119 of file [cpuUtils.c](#).

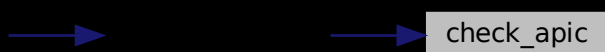
Here is the call graph for this function:



Here is the call graph for this function:



Here is the caller graph for this function:



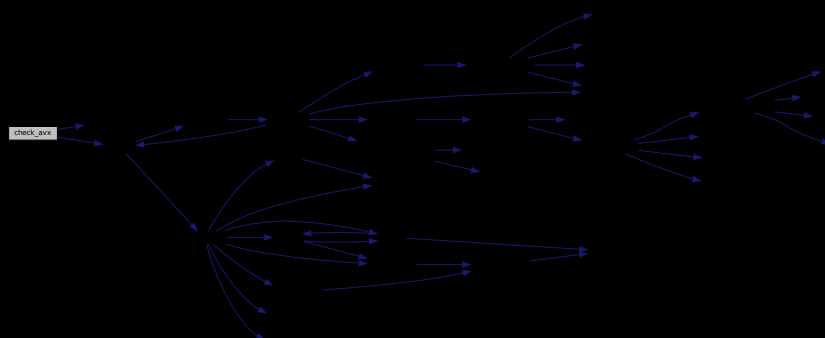
7.35.3.4 `check_avx()`

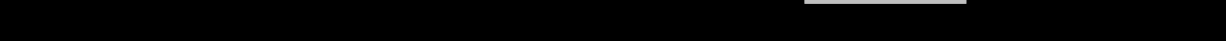
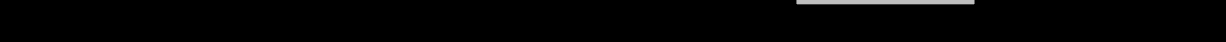
```
void check_avx (
    void )
```

Individual feature-probe helpers used by `cpuid_readout()`.

Definition at line 229 of file `cpuUtils.c`.

Here is the call graph for this function:



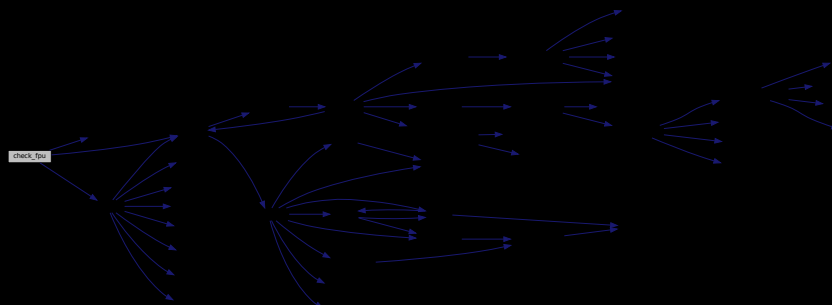


7.35.3.6 check_fpu()

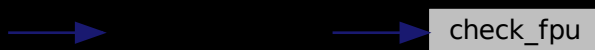
```
void check_fpu (  
    void )
```

Definition at line 130 of file [cpuUtils.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

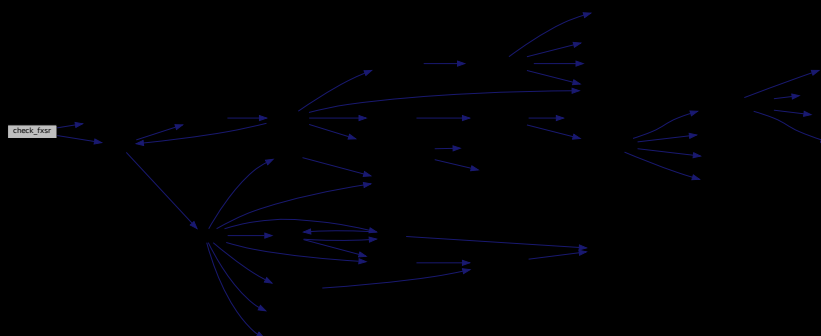


7.35.3.7 check_fxsr()

```
void check_fxsr (  
    void )
```

Definition at line 145 of file [cpuUtils.c](#).

Here is the call graph for this function:



→ → check_fxsr

```
void check_htt (
                void )
```

Here is the call graph for this function:

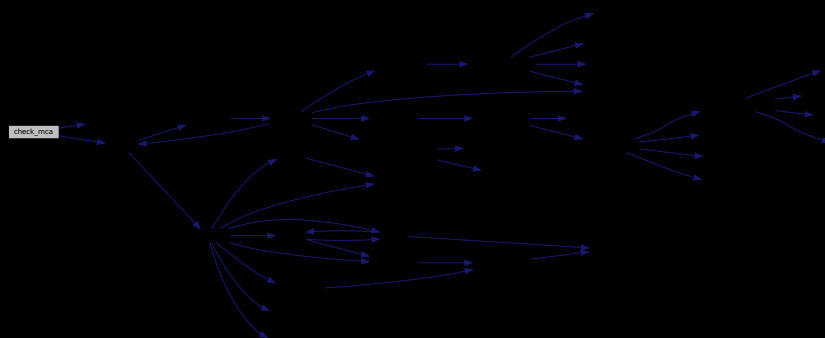
→ → check_htt

7.35.3.9 check_mca()

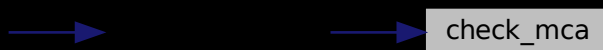
```
void check_mca (  
    void )
```

Definition at line 278 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

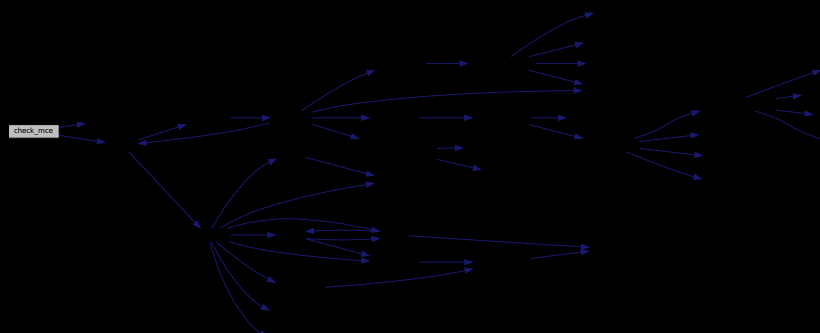


7.35.3.10 check_mce()

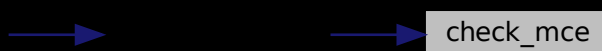
```
void check_mce (  
    void )
```

Definition at line 266 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

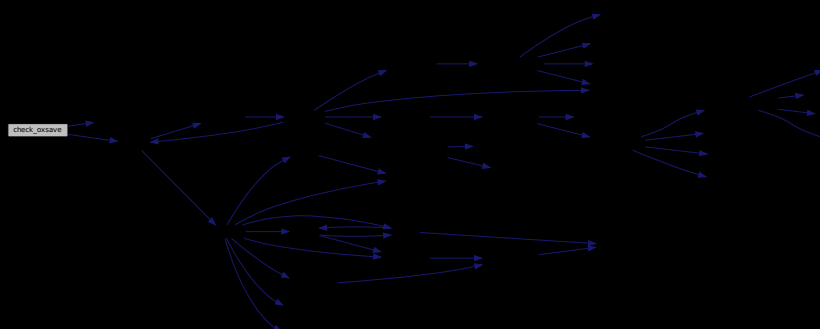


7.35.3.11 `check_oxsave()`

```
void check_oxsave (
    void )
```

Definition at line 222 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



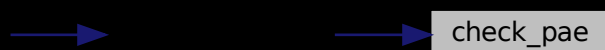
7.35.3.12 check_pae()

```
void check_pae (
                void )
```

Definition at line 254 of file cpuUtils.c.

Here is the call graph for this function:

Here is the caller graph for this function:

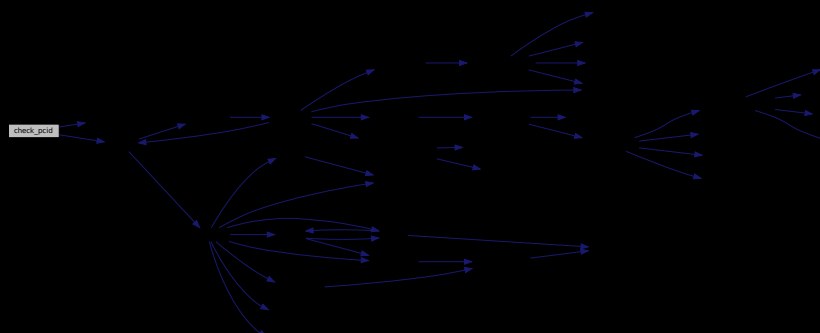


7.35.3.13 check_pcid()

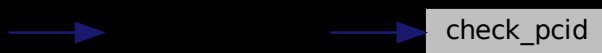
```
void check_pcid (
    void )
```

Definition at line 248 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

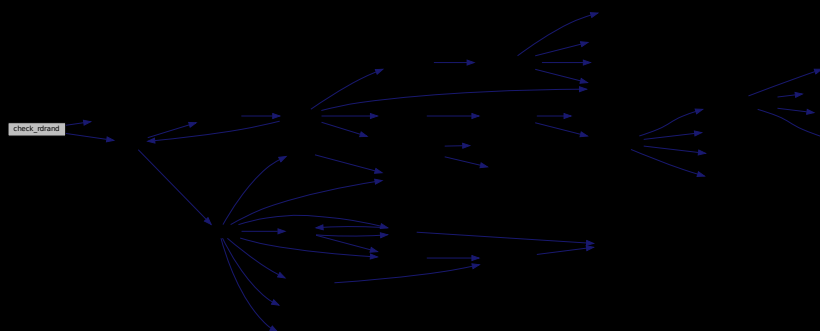


7.35.3.14 check_rdrand()

```
void check_rdrand (
    void )
```

Definition at line 187 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.35.3.15 check_rdseed()

```
void check_rdseed (
                void )
```

Definition at line 181 of file cpuUtils.c.

Here is the call graph for this function:

Here is the caller graph for this function:

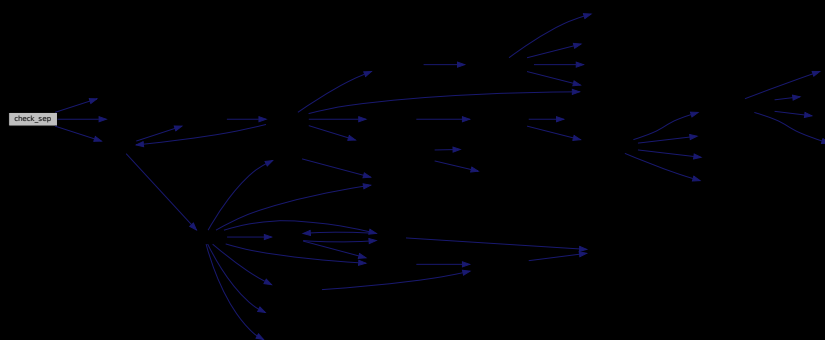


7.35.3.16 check_sep()

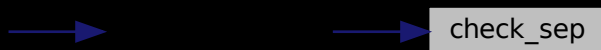
```
void check_sep (
    void )
```

Definition at line 193 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

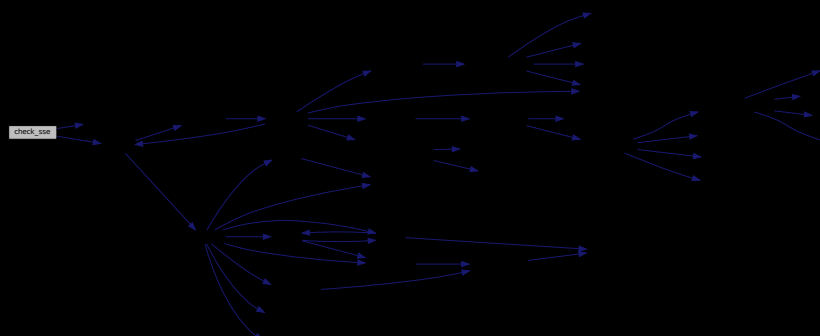


7.35.3.17 check_sse()

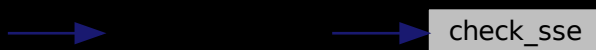
```
void check_sse (
    void )
```

Definition at line 211 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

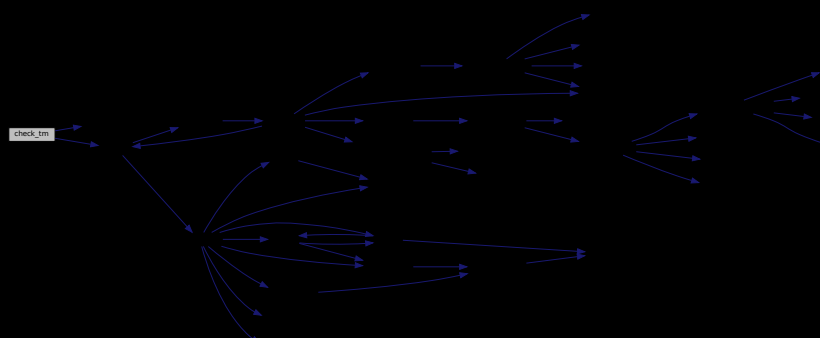


7.35.3.18 check_tm()

```
void check_tm (
    void )
```

Definition at line 297 of file `cpuUtils.c`.

Here is the call graph for this function:

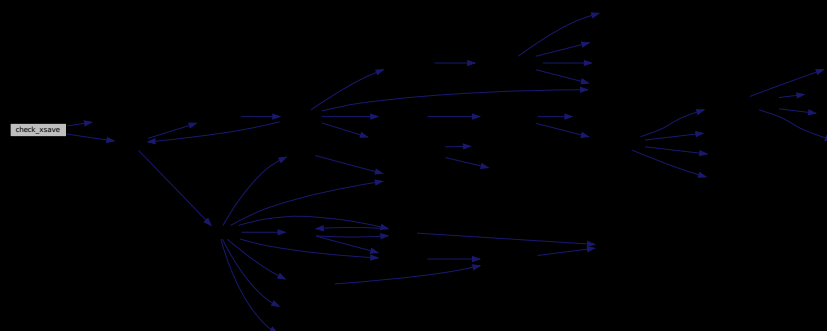


7.35.3.20 check_xsave()

```
void check_xsave (  
    void )
```

Definition at line 236 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



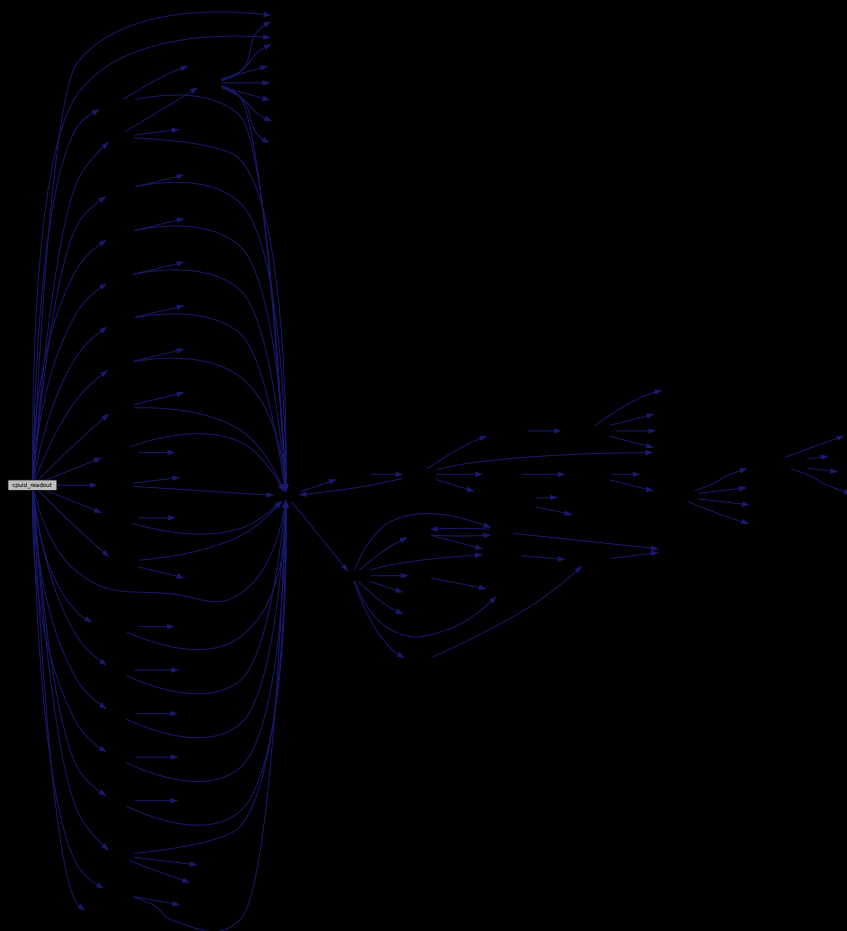
7.35.3.21 cpuid_readout()

```
void cpuid_readout (  
    void )
```

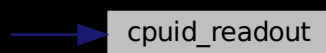
Print the verbose boot-time CPUID/capability report.

Definition at line 60 of file `cpuUtils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.35.3.22 fpu_init()

```
void fpu_init (
    void )
```

Initialize the current floating-point/SIMD setup path.

Definition at line 102 of file cpuUtils.c.

Here is the call graph for this function:

Here is the caller graph for this function:



7.35.3.23 get_model()

```
int get_model (
                void )
```

Return the detected processor model identifier used by current code.

Definition at line 95 of file cpuUtils.c.

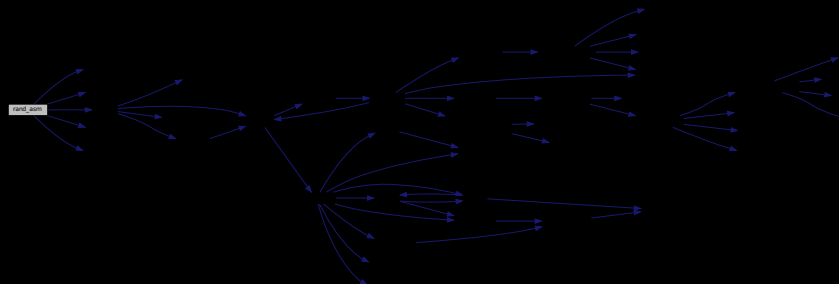
7.35.3.24 rand_asm()

```
uint64_t rand_asm (
    void )
```

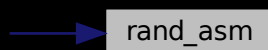
Obtain a hardware-generated random value through the assembly helper.

Definition at line 44 of file cpuUtils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.35.4 Variable Documentation

7.35.4.1 has_ACPI

```
bool has_ACPI [extern]
```

CPUID-derived ACPI capability state.

Definition at line 42 of file `cpuUtils.c`.

7.35.4.2 xsave_bank

```
size_t xsave_bank [extern]
```

Historical XSAVE-state allocation bookkeeping.

7.36 cpuUtils.h

Go to the [documentation of this file](#).

```

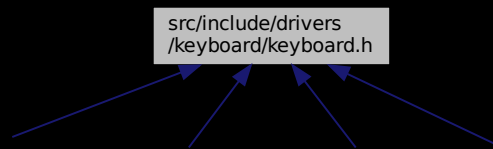
00001
00006 #pragma once
00007 #ifndef _CPU_UTILS_H
00008 #define _CPU_UTILS_H
00009
00010 #include "registers.h"
00011 #include "tss.h"
00012 #include <stdbool.h>
00013 #include <stddef.h>
00014 #include <stdint.h>
00015
00016 /* Vendor strings from CPUs. */
00017 #define CPUID_VENDOR_OLDAMD \
00018 "AMDisbetter!" /* Early engineering samples of AMD K5 processor */
00019 #define CPUID_VENDOR_AMD "AuthenticAMD"
00020 #define CPUID_VENDOR_INTEL "GenuineIntel"
00021 #define CPUID_VENDOR_VIA "VIA VIA VIA "
00022 #define CPUID_VENDOR_OLDTRANSMETA "TransmetaCPU"
00023 #define CPUID_VENDOR_TRANSMETA "GenuineTMx86"
00024 #define CPUID_VENDOR_CYRIX "CyrixInstead"
00025 #define CPUID_VENDOR_CENTAUR "CentaurHauls"
00026 #define CPUID_VENDOR_NEXGEN "NexGenDriven"
00027 #define CPUID_VENDOR_UMC "UMC UMC UMC "
00028 #define CPUID_VENDOR_SIS "SiS SiS SiS "
00029 #define CPUID_VENDOR_NSC "Geode by NSC"
00030 #define CPUID_VENDOR_RISE "RiseRiseRise"
00031 #define CPUID_VENDOR_VORTEX "Vortex86 SoC"
00032 #define CPUID_VENDOR_OLDAO486 "GenuineAO486"
00033 #define CPUID_VENDOR_AO486 "MiSTer AO486"
00034 #define CPUID_VENDOR_ZHAOXIN " Shanghai "
00035 #define CPUID_VENDOR_HYGON "HygonGenuine"
00036 #define CPUID_VENDOR_ELBRUS "E2K MACHINE "
00037
00038 /* Vendor strings from hypervisors. */
00039 #define CPUID_VENDOR_QEMU "TCGTCGTCGTCG"
00040 #define CPUID_VENDOR_KVM " KVMKVMKVM "
00041 #define CPUID_VENDOR_VMWARE "VMwareVMware"
00042 #define CPUID_VENDOR_VIRTUALBOX "VBoxVBoxVBox"
00043 #define CPUID_VENDOR_XEN "XenVMMXenVMM"
00044 #define CPUID_VENDOR_HYPERV "Microsoft Hv"
00045 #define CPUID_VENDOR_PARALLELS " prl hyperv "
00046 #define CPUID_VENDOR_PARALLELS_ALT \
00047 " lrpepyh vr "
00048 #define CPUID_VENDOR_BHYVE "bhyve bhyve "
00049 #define CPUID_VENDOR_QNX " QNXQVMSQG "
00050
00052 extern bool has_ACPI;
00054 extern size_t xsave_bank;
00055
00057 uint64_t rand_asm(void);
00059 int get_model(void);
00061 void cpuid_readout(void);
00063 void alloc_xsave(void);
00065 void fpu_init(void);
00066
00068 void check_avx(void);
00069 void check_oxsave(void);
00070 void check_fxsr(void);
00071 void check_htt(void);
00072 void check_sep(void);
00073 void check_sse(void);
00074 void check_xsave(void);
00075 void check_pcid(void);
00076 void check_pae(void);
00077 void check_mce(void);
00078 void check_apic(void);
00079 void check_mca(void);
00080 void check_acpi(void);
00081 void check_ds(void);
00082 void check_tm(void);
00083 void check_rdseed(void);
00084 void check_rdrand(void);
00085 void check_fpu(void);
00086 void check_tsc(void);
00087
00095 #endif

```

7.37 src/include/drivers/keyboard/keyboard.h File Reference

PS/2 keyboard IRQ and shell-facing character queue.

This graph shows which files directly or indirectly include this file:



Macros

- `#define KEYBOARD_DATA_PORT 0x60`
- `#define KEYBOARD_STATUS_PORT 0x64`

Functions

- void `keyboard_handler` (void)
IRQ1 keyboard handler.
- void `keyboard_init` (void)
Initialize the keyboard input path and make IRQ1 available.
- char `k_getchar` (void)
Legacy single-character keyboard accessor.
- char `kbd_pop` (void)
Remove one character from the keyboard FIFO.

7.37.1 Detailed Description

PS/2 keyboard IRQ and shell-facing character queue.

Definition in file `keyboard.h`.

7.37.2 Macro Definition Documentation

7.37.2.1 KEYBOARD_DATA_PORT

```
#define KEYBOARD_DATA_PORT 0x60
```

PS/2 controller data port.

Definition at line 9 of file [keyboard.h](#).

7.37.2.2 KEYBOARD_STATUS_PORT

```
#define KEYBOARD_STATUS_PORT 0x64
```

PS/2 controller status/command port.

Definition at line 11 of file [keyboard.h](#).

7.37.3 Function Documentation

7.37.3.1 k_getchar()

```
char k_getchar (  
    void )
```

Legacy single-character keyboard accessor.

Returns

Most recently exposed legacy keyboard character state.

Prefer [kbd_pop\(\)](#) for VSH and new queued-input code.

Legacy single-character keyboard accessor.

Returns

Character, or 0 if none is pending.

Definition at line 91 of file [keyboard.c](#).

7.37.3.2 kbd_pop()

```
char kbd_pop (
    void )
```

Remove one character from the keyboard FIFO.

Returns

Next queued character, or 0 when the queue is empty.

The queue is fed by the IRQ handler and consumed by VSH in ordinary task context. An empty queue is not an error; cooperative callers normally yield and try again later.

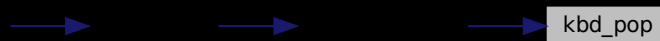
Remove one character from the keyboard FIFO.

Returns

Queued character, or 0 when the queue is empty.

Definition at line 42 of file [keyboard.c](#).

Here is the caller graph for this function:



7.37.3.3 keyboard_handler()

```
void keyboard_handler (
    void )
```

IRQ1 keyboard handler.

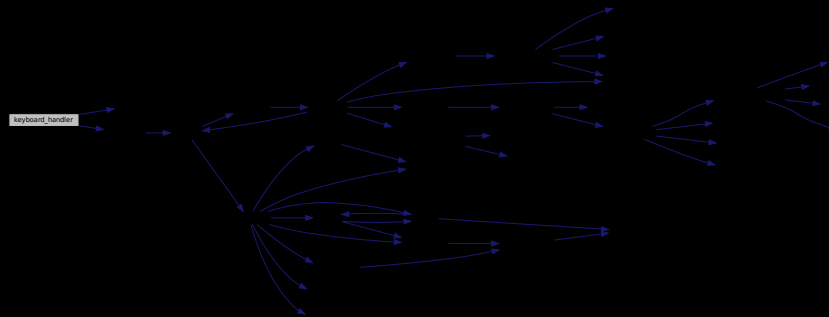
Consumes one Set-1 scancode from the controller, ignores break codes, maps supported make codes to characters, and queues them for non-IRQ consumers.

IRQ1 keyboard handler.

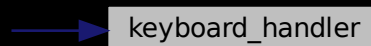
Key-release scancodes are ignored. Enter is normalized to a newline for the shell while k_char retains the historical '+' sentinel for legacy callers.

Definition at line 58 of file [keyboard.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.37.3.4 keyboard_init()

```
void keyboard_init (
    void )
```

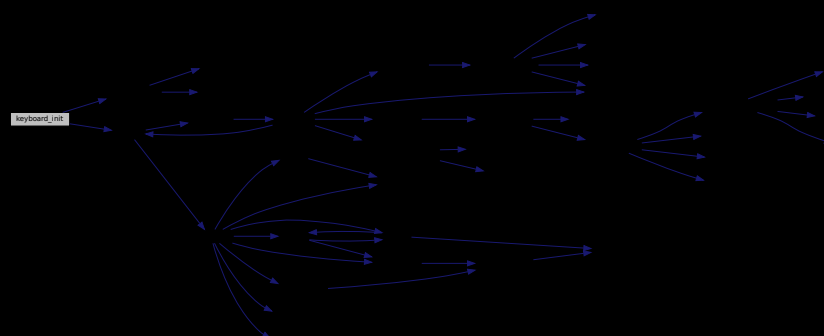
Initialize the keyboard input path and make IRQ1 available.

The 0.0.5 boot path calls this after the Voyager framebuffer terminal exists, so echoed shell input does not depend on the bootloader terminal callback.

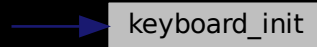
Initialize the keyboard input path and make IRQ1 available.

Definition at line 101 of file [keyboard.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.38 keyboard.h

Go to the documentation of this file.

```
00001
00006 #pragma once
00007
00009 #define KEYBOARD_DATA_PORT 0x60
00011 #define KEYBOARD_STATUS_PORT 0x64
00012
00019 void keyboard_handler(void);
00020
00027 void keyboard_init(void);
00028
00035 char k_getchar(void);
00036
00045 char kbd_pop(void);
```

7.39 src/include/drivers/keyboard/keyboard_map.h File Reference

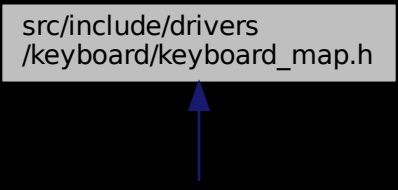
```
#include <stdint.h>
```

Include dependency graph for keyboard_map.h:

```
graph TD; A[src/include/drivers/keyboard/keyboard_map.h] --> B[ ];
```

A diagram showing a blue arrow pointing from a box labeled 'src/include/drivers/keyboard/keyboard_map.h' down to an empty rectangular box.

This graph shows which files directly or indirectly include this file:



```
graph TD; A[src/include/drivers/keyboard/keyboard_map.h];
```

src/include/drivers
/keyboard/keyboard_map.h

Variables

- const char `keyboard_map` [128]
- char `keyboard_charmap` [128]

7.39.1 Variable Documentation

7.39.1.1 `keyboard_charmap`

```
char keyboard_charmap[128]
```

Definition at line 156 of file `keyboard_map.h`.

7.39.1.2 `keyboard_map`

```
const char keyboard_map[128]
```

Definition at line 4 of file `keyboard_map.h`.

7.40 keyboard_map.h

Go to the documentation of this file.

```

00001 #pragma once
00002 #include <stdint.h>
00003
00004 const char keyboard_map[128] = {
00005     // ----- 0 to 9 -----
00006     ' ',
00007     ' ', // escape key
00008     '1',
00009     '2',
00010     '3',
00011     '4',
00012     '5',
00013     '6',
00014     '7',
00015     '8',
00016     // ----- 10 to 19 -----
00017     '9',
00018     '0',
00019     '-',
00020     '=',
00021     ' ', // Backspace
00022     ' ', // Tab
00023     'q',
00024     'w',
00025     'e',
00026     'r',
00027     // ----- 20 to 29 -----
00028     't',
00029     'y',
00030     'u',
00031     'i',
00032     'o',
00033     'p',
00034     '[',
00035     ']',
00036     ' ', // Enter
00037     ' ', // left Ctrl
00038     // ----- 30 to 39 -----
00039     'a',
00040     's',
00041     'd',
00042     'f',
00043     'g',
00044     'h',
00045     'j',
00046     'k',
00047     'l',
00048     ';',
00049     // ----- 40 to 49 -----
00050     ' ',
00051     ' ',
00052     ' ', // left Shift
00053     ' ',
00054     'z',
00055     'x',
00056     'c',
00057     'v',
00058     'b',
00059     'n',
00060     // ----- 50 to 59 -----
00061     'm',
00062     ' ',
00063     ' ',
00064     '/', // slash, or numpad slash if preceded by keycode 224
00065     ' ', // right Shift
00066     '*', // numpad asterisk
00067     ' ', // left Alt
00068     ' ', // Spacebar
00069     ' ',
00070     ' ', // F1
00071     // ----- 60 to 69 -----
00072     ' ', // F2
00073     ' ', // F3
00074     ' ', // F4
00075     ' ', // F5
00076     ' ', // F6
00077     ' ', // F7
00078     ' ', // F8
00079     ' ', // F9
00080     ' ', // F10
00081     ' ',
00082     // ----- 70 to 79 -----

```

```

00083     ' ', // scroll lock
00084     '7', // numpad 7, HOME key if preceded by keycode 224
00085     '8', // numpad 8, up arrow if preceded by keycode 224
00086     '9', // numpad 9, PAGE UP key if preceded by keycode 224
00087     '-', // numpad hyphen
00088     '4', // numpad 4, left arrow if preceded by keycode 224
00089     '5', // numpad 5
00090     '6', // numpad 6, right arrow if preceded by keycode 224
00091     ' ',
00092     '1', // numpad 1, END key if preceded by keycode 224
00093     // ----- 80 to 89 -----
00094     '2', // numpad 2, down arrow if preceded by keycode 224
00095     '3', // numpad 3, PAGE DOWN key if preceded by keycode 224
00096     '0', // numpad 0, INSERT key if preceded by keycode 224
00097     '.', // numpad dot, DELETE key if preceded by keycode 224
00098     ' ',
00099     ' ',
00100     ' ',
00101     ' ',
00102     ' ',
00103     ' ',
00104     // ----- 90 to 99 -----
00105     ' ',
00106     ' ',
00107     ' ',
00108     ' ',
00109     ' ',
00110     ' ',
00111     ' ',
00112     ' ',
00113     ' ',
00114     ' ',
00115     // ----- 100 to 109 -----
00116     ' ',
00117     ' ',
00118     ' ',
00119     ' ',
00120     ' ',
00121     ' ',
00122     ' ',
00123     ' ',
00124     ' ',
00125     ' ',
00126     // ----- 110 to 119 -----
00127     ' ',
00128     ' ',
00129     ' ',
00130     ' ',
00131     ' ',
00132     ' ',
00133     ' ',
00134     ' ',
00135     ' ',
00136     ' ',
00137     // ----- 120-127 -----
00138     ' ',
00139     ' ',
00140     ' ',
00141     ' ',
00142     ' ',
00143     ' ',
00144     ' ',
00145     ' ',
00146 };
00147 // Right control, right alt seem to send
00148 // keycode 224, then the left control/alt keycode
00149 // Arrow keys also send two interrupts, one 224 and then their actual code
00150 // Same for numpad enter
00151 // 197: Num Lock
00152 // 157: Pause/Break (followed by 197?)
00153 // Clicking on screen appears to send keycodes 70, 198
00154 // Is this the MARK command or something like that?
00155
00156 char keyboard_charmap[128] = {
00157     // ----- 0 to 9 -----
00158     ' ',
00159     ' ', // escape key
00160     '1',
00161     '2',
00162     '3',
00163     '4',
00164     '5',
00165     '6',
00166     '7',
00167     '8',
00168     // ----- 10 to 19 -----
00169     '9',

```

```

00170      '0',
00171      '-',
00172      '=',
00173      ' ', // Backspace
00174      ' ', // Tab
00175      'g',
00176      'w',
00177      'e',
00178      'r',
00179      // ----- 20 to 29 -----
00180      't',
00181      'y',
00182      'u',
00183      'i',
00184      'o',
00185      'p',
00186      '[',
00187      ']',
00188      ' ', // Enter
00189      ' ', // left Ctrl
00190      // ----- 30 to 39 -----
00191      'a',
00192      's',
00193      'd',
00194      'f',
00195      'g',
00196      'h',
00197      'j',
00198      'k',
00199      'l',
00200      ';',
00201      // ----- 40 to 49 -----
00202      ',',
00203      '\',
00204      ' ', // left Shift
00205      ' ',
00206      'z',
00207      'x',
00208      'c',
00209      'v',
00210      'b',
00211      'n',
00212      // ----- 50 to 59 -----
00213      'm',
00214      '.',
00215      '/',
00216      '/', // slash, or numpad slash if preceded by keycode 224
00217      ' ', // right Shift
00218      '*', // numpad asterisk
00219      ' ', // left Alt
00220      ' ', // Spacebar
00221      ' ',
00222      ' ', // F1
00223      // ----- 60 to 69 -----
00224      ' ', // F2
00225      ' ', // F3
00226      ' ', // F4
00227      ' ', // F5
00228      ' ', // F6
00229      ' ', // F7
00230      ' ', // F8
00231      ' ', // F9
00232      ' ', // F10
00233      ' ',
00234      // ----- 70 to 79 -----
00235      ' ', // scroll lock
00236      '7', // numpad 7, HOME key if preceded by keycode 224
00237      '8', // numpad 8, up arrow if preceded by keycode 224
00238      '9', // numpad 9, PAGE UP key if preceded by keycode 224
00239      '-', // numpad hyphen
00240      '4', // numpad 4, left arrow if preceded by keycode 224
00241      '5', // numpad 5
00242      '6', // numpad 6, right arrow if preceded by keycode 224
00243      ' ',
00244      '1', // numpad 1, END key if preceded by keycode 224
00245      // ----- 80 to 89 -----
00246      '2', // numpad 2, down arrow if preceded by keycode 224
00247      '3', // numpad 3, PAGE DOWN key if preceded by keycode 224
00248      '0', // numpad 0, INSERT key if preceded by keycode 224
00249      '.', // numpad dot, DELETE key if preceded by keycode 224
00250      ' ',
00251      ' ',
00252      ' ',
00253      ' ',
00254      ' ',
00255      ' ',
00256      // ----- 90 to 99 -----

```

```

00257      /*
00258      /*
00259      /*
00260      /*
00261      /*
00262      /*
00263      /*
00264      /*
00265      /*
00266      /*
00267      // ----- 100 to 109 -----
00268      /*
00269      /*
00270      /*
00271      /*
00272      /*
00273      /*
00274      /*
00275      /*
00276      /*
00277      /*
00278      // ----- 110 to 119 -----
00279      /*
00280      /*
00281      /*
00282      /*
00283      /*
00284      /*
00285      /*
00286      /*
00287      /*
00288      /*
00289      // ----- 120-127 -----
00290      /*
00291      /*
00292      /*
00293      /*
00294      /*
00295      /*
00296      /*
00297      /*
00298  };

```

7.41 src/include/early_alloc.h File Reference

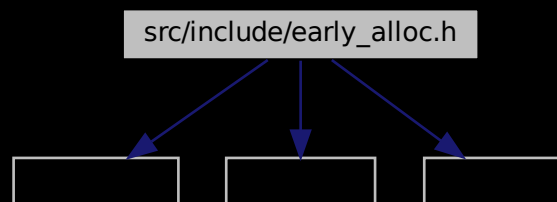
Bootstrap physical-page allocator used before the permanent PMM.

```
#include <stdbool.h>
```

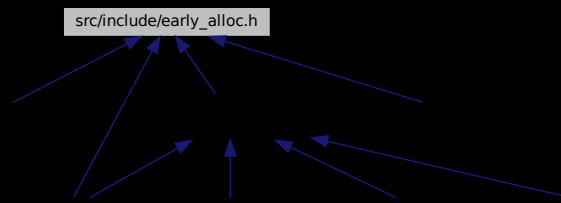
```
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for early_alloc.h:



This graph shows which files directly or indirectly include this file:



Typedefs

- typedef uint64_t **paddr_t**
Physical-address type used by the Stage-2 memory stack.

Functions

- void **early_init** (void)
Select a Limine USABLE memory region and initialize the bump allocator.
- **paddr_t** **early_alloc_page** (void)
Allocate one 4 KiB physical page from the bootstrap region.
- void **early_reserved_region** (**paddr_t** *base, size_t *pages)
Report the physical range consumed by early allocation.
- size_t **early_pages_used** (void)
Return the number of 4 KiB pages consumed by the bootstrap allocator.
- void **early_set_region** (**paddr_t** base, size_t page_count)
Supply an explicit region for early allocation.

7.41.1 Detailed Description

Bootstrap physical-page allocator used before the permanent PMM.

Copyright (c) 2025 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Definition in file **early_alloc.h**.

7.41.2 Typedef Documentation

7.41.2.1 paddr_t

```
typedef uint64_t paddr_t
```

Physical-address type used by the Stage-2 memory stack.

A `paddr_t` is not directly dereferenceable merely because it is represented by an integer. Convert/map it through the HHDM or another explicit mapping before treating it as a pointer.

Definition at line 25 of file `early_alloc.h`.

7.41.3 Function Documentation

7.41.3.1 early_alloc_page()

```
paddr_t early_alloc_page (  
    void )
```

Allocate one 4 KiB physical page from the bootstrap region.

Returns

4 KiB-aligned physical address of the allocated page.

Early allocations are monotonic and are not individually freed. Once the permanent frame allocator is initialized, paging obtains new frames through the Stage-2 frame supplier instead.

Definition at line 119 of file `early_alloc.c`.

7.41.3.2 early_init()

```
void early_init (  
    void )
```

Select a Limine USABLE memory region and initialize the bump allocator.

If `early_set_region()` has already supplied an explicit region, this function preserves that selection rather than choosing another one.

Definition at line 103 of file `early_alloc.c`.

7.41.3.3 `early_pages_used()`

```
size_t early_pages_used (
    void )
```

Return the number of 4 KiB pages consumed by the bootstrap allocator.

Returns

Number of pages allocated since initialization.

Definition at line 139 of file `early_alloc.c`.

7.41.3.4 `early_reserved_region()`

```
void early_reserved_region (
    paddr_t * base,
    size_t * pages )
```

Report the physical range consumed by early allocation.

Parameters

<i>base</i>	Receives the physical base of the bootstrap region.
<i>pages</i>	Receives the number of pages consumed from that region.

The permanent PMM uses this information to avoid handing bootstrap-owned memory back out as free frames.

Definition at line 131 of file `early_alloc.c`.

7.41.3.5 `early_set_region()`

```
void early_set_region (
    paddr_t base,
    size_t page_count )
```

Supply an explicit region for early allocation.

Parameters

<i>base</i>	4 KiB-aligned physical base address.
<i>page_count</i>	Number of 4 KiB pages in the region.

The region must be suitable for allocation and should be configured before `early_init()`.

Definition at line 95 of file `early_alloc.c`.

7.42 early_alloc.h

Go to the documentation of this file.

```
00001
00013 #pragma once
00014 #include <stdbool.h>
00015 #include <stddef.h>
00016 #include <stdint.h>
00017
00025 typedef uint64_t paddr_t;
00026
00033 void early_init(void);
00034
00043 paddr_t early_alloc_page(void);
00044
00053 void early_reserved_region(paddr_t *base, size_t *pages);
00054
00059 size_t early_pages_used(void);
00060
00069 void early_set_region(paddr_t base, size_t page_count);
```

7.43 src/include/errno.h File Reference

Macros

- #define EPERM 1
- #define ENOENT 2
- #define ESRCH 3
- #define EINTR 4
- #define EIO 5
- #define ENXIO 6
- #define E2BIG 7
- #define ENOEXEC 8
- #define EBADF 9
- #define ECHILD 10
- #define EAGAIN 11
- #define ENOMEM 12
- #define EACCES 13
- #define EFAULT 14
- #define ENOTBLK 15
- #define EBUSY 16
- #define EEXIST 17
- #define EXDEV 18
- #define ENODEV 19
- #define ENOTDIR 20
- #define EISDIR 21
- #define EINVAL 22
- #define ENFILE 23
- #define EMFILE 24
- #define ENOTTY 25
- #define ETXTBSY 26
- #define EFBIG 27
- #define ENOSPC 28
- #define ESPIPE 29
- #define EROFS 30
- #define EMLINK 31
- #define EPIPE 32
- #define EDOM 33

- `#define ERANGE` 34
- `#define EDEADLK` 35
- `#define ENAMETOOLONG` 36
- `#define ENOLCK` 37
- `#define ENOSYS` 38
- `#define ENOTEMPTY` 39
- `#define ELOOP` 40
- `#define EWOULDBLOCK EAGAIN`
- `#define ENOMSG` 42
- `#define EIDRM` 43
- `#define ECHRNG` 44
- `#define EL2NSYNC` 45
- `#define EL3HLT` 46
- `#define EL3RST` 47
- `#define ELNRNG` 48
- `#define EUNATCH` 49
- `#define ENOCSI` 50
- `#define EL2HLT` 51
- `#define EBADE` 52
- `#define EBADR` 53
- `#define EXFULL` 54
- `#define ENOANO` 55
- `#define EBADRQC` 56
- `#define EBADSLT` 57
- `#define EDEADLOCK EDEADLK`
- `#define EBFONT` 59
- `#define ENOSTR` 60
- `#define ENODATA` 61
- `#define ETIME` 62
- `#define ENOSR` 63
- `#define ENONET` 64
- `#define ENOPKG` 65
- `#define EREMOTE` 66
- `#define ENOLINK` 67
- `#define EADV` 68
- `#define ESRMNT` 69
- `#define ECOMM` 70
- `#define EPROTO` 71
- `#define EMULTIHOP` 72
- `#define EDOTDOT` 73
- `#define EBADMSG` 74
- `#define EOVERFLOW` 75
- `#define ENOTUNIQ` 76
- `#define EBADFD` 77
- `#define EREMCHG` 78
- `#define ELIBACC` 79
- `#define ELIBBAD` 80
- `#define ELIBSCN` 81
- `#define ELIBMAX` 82
- `#define ELIBEXEC` 83
- `#define EILSEQ` 84
- `#define ERESTART` 85
- `#define ESTRPIPE` 86
- `#define EUSERS` 87
- `#define ENOTSOCK` 88

- #define `EDESTADDRREQ` 89
- #define `EMSGSIZE` 90
- #define `EPROTOTYPE` 91
- #define `ENOPROTOOPT` 92
- #define `EPROTONOSUPPORT` 93
- #define `ESOCKTNOSUPPORT` 94
- #define `EOPNOTSUPP` 95
- #define `ENOTSUP` `EOPNOTSUPP`
- #define `EPFNOSUPPORT` 96
- #define `EAFNOSUPPORT` 97
- #define `EADDRINUSE` 98
- #define `EADDRNOTAVAIL` 99
- #define `ENETDOWN` 100
- #define `ENETUNREACH` 101
- #define `ENETRESET` 102
- #define `ECONNABORTED` 103
- #define `ECONNRESET` 104
- #define `ENOBUFS` 105
- #define `EISCONN` 106
- #define `ENOTCONN` 107
- #define `ESHUTDOWN` 108
- #define `ETOOMANYREFS` 109
- #define `ETIMEDOUT` 110
- #define `ECONNREFUSED` 111
- #define `EHOSTDOWN` 112
- #define `EHOSTUNREACH` 113
- #define `EALREADY` 114
- #define `EINPROGRESS` 115
- #define `ESTALE` 116
- #define `EUCLEAN` 117
- #define `ENOTNAM` 118
- #define `ENAVAIL` 119
- #define `EISNAM` 120
- #define `EREMOTEIO` 121
- #define `EDQUOT` 122
- #define `ENOMEDIUM` 123
- #define `EMEDIUMTYPE` 124
- #define `ECANCELED` 125
- #define `ENOKEY` 126
- #define `EKEYEXPIRED` 127
- #define `EKEYREVOKED` 128
- #define `EKEYREJECTED` 129
- #define `EOWNERDEAD` 130
- #define `ENOTRECOVERABLE` 131
- #define `ERFKILL` 132
- #define `EHWPOISON` 133

7.43.1 Macro Definition Documentation

7.43.1.1 E2BIG

```
#define E2BIG 7
```

Definition at line 12 of file [errno.h](#).

7.43.1.2 EACCES

```
#define EACCES 13
```

Definition at line 18 of file [errno.h](#).

7.43.1.3 EADDRINUSE

```
#define EADDRINUSE 98
```

Definition at line 104 of file [errno.h](#).

7.43.1.4 EADDRNOTAVAIL

```
#define EADDRNOTAVAIL 99
```

Definition at line 105 of file [errno.h](#).

7.43.1.5 EADV

```
#define EADV 68
```

Definition at line 73 of file [errno.h](#).

7.43.1.6 EAFNOSUPPORT

```
#define EAFNOSUPPORT 97
```

Definition at line 103 of file [errno.h](#).

7.43.1.7 EAGAIN

```
#define EAGAIN 11
```

Definition at line 16 of file [errno.h](#).

7.43.1.8 EALREADY

```
#define EALREADY 114
```

Definition at line 120 of file [errno.h](#).

7.43.1.9 EBADE

```
#define EBADE 52
```

Definition at line 57 of file [errno.h](#).

7.43.1.10 EBADF

```
#define EBADF 9
```

Definition at line 14 of file [errno.h](#).

7.43.1.11 EBADFD

```
#define EBADFD 77
```

Definition at line 82 of file [errno.h](#).

7.43.1.12 EBADMSG

```
#define EBADMSG 74
```

Definition at line 79 of file [errno.h](#).

7.43.1.13 EBADR

```
#define EBADR 53
```

Definition at line 58 of file [errno.h](#).

7.43.1.14 EBADRQC

```
#define EBADRQC 56
```

Definition at line 61 of file [errno.h](#).

7.43.1.15 EBADSLT

```
#define EBADSLT 57
```

Definition at line 62 of file [errno.h](#).

7.43.1.16 EBFONT

```
#define EBFONT 59
```

Definition at line 64 of file [errno.h](#).

7.43.1.17 EBUSY

```
#define EBUSY 16
```

Definition at line 21 of file [errno.h](#).

7.43.1.18 ECANCELED

```
#define ECANCELED 125
```

Definition at line 131 of file [errno.h](#).

7.43.1.19 ECHILD

```
#define ECHILD 10
```

Definition at line 15 of file [errno.h](#).

7.43.1.20 ECHRNG

```
#define ECHRNG 44
```

Definition at line 49 of file [errno.h](#).

7.43.1.21 ECOMM

```
#define ECOMM 70
```

Definition at line 75 of file [errno.h](#).

7.43.1.22 ECONNABORTED

```
#define ECONNABORTED 103
```

Definition at line 109 of file [errno.h](#).

7.43.1.23 ECONNREFUSED

```
#define ECONNREFUSED 111
```

Definition at line 117 of file [errno.h](#).

7.43.1.24 ECONNRESET

```
#define ECONNRESET 104
```

Definition at line 110 of file [errno.h](#).

7.43.1.25 EDEADLK

```
#define EDEADLK 35
```

Definition at line 40 of file [errno.h](#).

7.43.1.26 EDEADLOCK

```
#define EDEADLOCK EDEADLK
```

Definition at line 63 of file [errno.h](#).

7.43.1.27 EDESTADDRREQ

```
#define EDESTADDRREQ 89
```

Definition at line 94 of file [errno.h](#).

7.43.1.28 EDOM

```
#define EDOM 33
```

Definition at line 38 of file [errno.h](#).

7.43.1.29 EDOTDOT

```
#define EDOTDOT 73
```

Definition at line 78 of file [errno.h](#).

7.43.1.30 EDQUOT

```
#define EDQUOT 122
```

Definition at line 128 of file [errno.h](#).

7.43.1.31 EEXIST

```
#define EEXIST 17
```

Definition at line 22 of file [errno.h](#).

7.43.1.32 EFAULT

```
#define EFAULT 14
```

Definition at line 19 of file [errno.h](#).

7.43.1.33 EFBIG

```
#define EFBIG 27
```

Definition at line 32 of file [errno.h](#).

7.43.1.34 EHOSTDOWN

```
#define EHOSTDOWN 112
```

Definition at line 118 of file [errno.h](#).

7.43.1.35 EHOSTUNREACH

```
#define EHOSTUNREACH 113
```

Definition at line 119 of file [errno.h](#).

7.43.1.36 EHWPOISON

```
#define EHWPOISON 133
```

Definition at line 139 of file [errno.h](#).

7.43.1.37 EIDRM

```
#define EIDRM 43
```

Definition at line 48 of file [errno.h](#).

7.43.1.38 EILSEQ

```
#define EILSEQ 84
```

Definition at line 89 of file [errno.h](#).

7.43.1.39 EINPROGRESS

```
#define EINPROGRESS 115
```

Definition at line 121 of file [errno.h](#).

7.43.1.40 EINTR

```
#define EINTR 4
```

Definition at line 9 of file [errno.h](#).

7.43.1.41 EINVAL

```
#define EINVAL 22
```

Definition at line 27 of file [errno.h](#).

7.43.1.42 EIO

```
#define EIO 5
```

Definition at line 10 of file [errno.h](#).

7.43.1.43 EISCONN

```
#define EISCONN 106
```

Definition at line 112 of file [errno.h](#).

7.43.1.44 EISDIR

```
#define EISDIR 21
```

Definition at line 26 of file [errno.h](#).

7.43.1.45 EISNAM

```
#define EISNAM 120
```

Definition at line 126 of file [errno.h](#).

7.43.1.46 EKEYEXPIRED

```
#define EKEYEXPIRED 127
```

Definition at line 133 of file [errno.h](#).

7.43.1.47 EKEYREJECTED

```
#define EKEYREJECTED 129
```

Definition at line 135 of file [errno.h](#).

7.43.1.48 EKEYREVOKED

```
#define EKEYREVOKED 128
```

Definition at line 134 of file [errno.h](#).

7.43.1.49 EL2HLT

```
#define EL2HLT 51
```

Definition at line 56 of file [errno.h](#).

7.43.1.50 EL2NSYNC

```
#define EL2NSYNC 45
```

Definition at line 50 of file [errno.h](#).

7.43.1.51 EL3HLT

```
#define EL3HLT 46
```

Definition at line 51 of file [errno.h](#).

7.43.1.52 EL3RST

```
#define EL3RST 47
```

Definition at line 52 of file [errno.h](#).

7.43.1.53 ELIBACC

```
#define ELIBACC 79
```

Definition at line 84 of file [errno.h](#).

7.43.1.54 ELIBBAD

```
#define ELIBBAD 80
```

Definition at line 85 of file [errno.h](#).

7.43.1.55 ELIBEXEC

```
#define ELIBEXEC 83
```

Definition at line 88 of file [errno.h](#).

7.43.1.56 ELIBMAX

```
#define ELIBMAX 82
```

Definition at line 87 of file [errno.h](#).

7.43.1.57 ELIBSCN

```
#define ELIBSCN 81
```

Definition at line 86 of file [errno.h](#).

7.43.1.58 ELNRNG

```
#define ELNRNG 48
```

Definition at line 53 of file [errno.h](#).

7.43.1.59 ELOOP

```
#define ELOOP 40
```

Definition at line 45 of file [errno.h](#).

7.43.1.60 EMEDIUMTYPE

```
#define EMEDIUMTYPE 124
```

Definition at line 130 of file [errno.h](#).

7.43.1.61 EMFILE

```
#define EMFILE 24
```

Definition at line 29 of file [errno.h](#).

7.43.1.62 EMLINK

```
#define EMLINK 31
```

Definition at line 36 of file [errno.h](#).

7.43.1.63 EMSGSIZE

```
#define EMSGSIZE 90
```

Definition at line 95 of file [errno.h](#).

7.43.1.64 EMULTIHOP

```
#define EMULTIHOP 72
```

Definition at line 77 of file [errno.h](#).

7.43.1.65 ENAMETOOLONG

```
#define ENAMETOOLONG 36
```

Definition at line 41 of file [errno.h](#).

7.43.1.66 ENAVAIL

```
#define ENAVAIL 119
```

Definition at line 125 of file [errno.h](#).

7.43.1.67 ENETDOWN

```
#define ENETDOWN 100
```

Definition at line 106 of file [errno.h](#).

7.43.1.68 ENETRESET

```
#define ENETRESET 102
```

Definition at line 108 of file [errno.h](#).

7.43.1.69 ENETUNREACH

```
#define ENETUNREACH 101
```

Definition at line 107 of file [errno.h](#).

7.43.1.70 ENFILE

```
#define ENFILE 23
```

Definition at line 28 of file [errno.h](#).

7.43.1.71 ENOANO

```
#define ENOANO 55
```

Definition at line 60 of file [errno.h](#).

7.43.1.72 ENOBUFS

```
#define ENOBUFS 105
```

Definition at line 111 of file [errno.h](#).

7.43.1.73 ENOCSI

```
#define ENOCSI 50
```

Definition at line 55 of file [errno.h](#).

7.43.1.74 ENODATA

```
#define ENODATA 61
```

Definition at line 66 of file [errno.h](#).

7.43.1.75 ENODEV

```
#define ENODEV 19
```

Definition at line 24 of file [errno.h](#).

7.43.1.76 ENOENT

```
#define ENOENT 2
```

Definition at line 7 of file [errno.h](#).

7.43.1.77 ENOEXEC

```
#define ENOEXEC 8
```

Definition at line 13 of file [errno.h](#).

7.43.1.78 ENOKEY

```
#define ENOKEY 126
```

Definition at line 132 of file [errno.h](#).

7.43.1.79 ENOLCK

```
#define ENOLCK 37
```

Definition at line 42 of file [errno.h](#).

7.43.1.80 ENOLINK

```
#define ENOLINK 67
```

Definition at line 72 of file [errno.h](#).

7.43.1.81 ENOMEDIUM

```
#define ENOMEDIUM 123
```

Definition at line 129 of file [errno.h](#).

7.43.1.82 ENOMEM

```
#define ENOMEM 12
```

Definition at line 17 of file [errno.h](#).

7.43.1.83 ENOMSG

```
#define ENOMSG 42
```

Definition at line 47 of file [errno.h](#).

7.43.1.84 ENONET

```
#define ENONET 64
```

Definition at line 69 of file [errno.h](#).

7.43.1.85 ENOPKG

```
#define ENOPKG 65
```

Definition at line 70 of file [errno.h](#).

7.43.1.86 ENOPROTOOPT

```
#define ENOPROTOOPT 92
```

Definition at line 97 of file [errno.h](#).

7.43.1.87 ENOSPC

```
#define ENOSPC 28
```

Definition at line 33 of file [errno.h](#).

7.43.1.88 ENOSR

```
#define ENOSR 63
```

Definition at line 68 of file [errno.h](#).

7.43.1.89 ENOSTR

```
#define ENOSTR 60
```

Definition at line 65 of file [errno.h](#).

7.43.1.90 ENOSYS

```
#define ENOSYS 38
```

Definition at line 43 of file [errno.h](#).

7.43.1.91 ENOTBLK

```
#define ENOTBLK 15
```

Definition at line 20 of file [errno.h](#).

7.43.1.92 ENOTCONN

```
#define ENOTCONN 107
```

Definition at line 113 of file [errno.h](#).

7.43.1.93 ENOTDIR

```
#define ENOTDIR 20
```

Definition at line 25 of file [errno.h](#).

7.43.1.94 ENOTEMPTY

```
#define ENOTEMPTY 39
```

Definition at line 44 of file [errno.h](#).

7.43.1.95 ENOTNAM

```
#define ENOTNAM 118
```

Definition at line 124 of file [errno.h](#).

7.43.1.96 ENOTRECOVERABLE

```
#define ENOTRECOVERABLE 131
```

Definition at line 137 of file [errno.h](#).

7.43.1.97 ENOTSOCK

```
#define ENOTSOCK 88
```

Definition at line 93 of file [errno.h](#).

7.43.1.98 ENOTSUP

```
#define ENOTSUP EOPNOTSUPP
```

Definition at line 101 of file [errno.h](#).

7.43.1.99 ENOTTY

```
#define ENOTTY 25
```

Definition at line 30 of file [errno.h](#).

7.43.1.100 ENOTUNIQ

```
#define ENOTUNIQ 76
```

Definition at line 81 of file [errno.h](#).

7.43.1.101 ENXIO

```
#define ENXIO 6
```

Definition at line 11 of file [errno.h](#).

7.43.1.102 EOPNOTSUPP

```
#define EOPNOTSUPP 95
```

Definition at line 100 of file [errno.h](#).

7.43.1.103 EOVERFLOW

```
#define EOVERFLOW 75
```

Definition at line 80 of file [errno.h](#).

7.43.1.104 EOWNERDEAD

```
#define EOWNERDEAD 130
```

Definition at line 136 of file [errno.h](#).

7.43.1.105 EPERM

```
#define EPERM 1
```

Definition at line 6 of file [errno.h](#).

7.43.1.106 EFNOSUPPORT

```
#define EFNOSUPPORT 96
```

Definition at line 102 of file [errno.h](#).

7.43.1.107 EPIPE

```
#define EPIPE 32
```

Definition at line 37 of file [errno.h](#).

7.43.1.108 EPROTO

```
#define EPROTO 71
```

Definition at line 76 of file [errno.h](#).

7.43.1.109 EPROTONOSUPPORT

```
#define EPROTONOSUPPORT 93
```

Definition at line 98 of file [errno.h](#).

7.43.1.110 EPROTOTYPE

```
#define EPROTOTYPE 91
```

Definition at line 96 of file [errno.h](#).

7.43.1.111 ERANGE

```
#define ERANGE 34
```

Definition at line 39 of file [errno.h](#).

7.43.1.112 EREMCHG

```
#define EREMCHG 78
```

Definition at line 83 of file [errno.h](#).

7.43.1.113 EREMOTE

```
#define EREMOTE 66
```

Definition at line 71 of file [errno.h](#).

7.43.1.114 EREMOTEIO

```
#define EREMOTEIO 121
```

Definition at line 127 of file [errno.h](#).

7.43.1.115 ERESTART

```
#define ERESTART 85
```

Definition at line 90 of file [errno.h](#).

7.43.1.116 ERFKILL

```
#define ERFKILL 132
```

Definition at line 138 of file [errno.h](#).

7.43.1.117 EROFS

```
#define EROFS 30
```

Definition at line 35 of file [errno.h](#).

7.43.1.118 ESHUTDOWN

```
#define ESHUTDOWN 108
```

Definition at line 114 of file [errno.h](#).

7.43.1.119 ESOCKTNOSUPPORT

```
#define ESOCKTNOSUPPORT 94
```

Definition at line 99 of file [errno.h](#).

7.43.1.120 ESPIPE

```
#define ESPIPE 29
```

Definition at line 34 of file [errno.h](#).

7.43.1.121 ESRCH

```
#define ESRCH 3
```

Definition at line 8 of file [errno.h](#).

7.43.1.122 ESRMNT

```
#define ESRMNT 69
```

Definition at line 74 of file [errno.h](#).

7.43.1.123 ESTALE

```
#define ESTALE 116
```

Definition at line 122 of file [errno.h](#).

7.43.1.124 ESTRPIPE

```
#define ESTRPIPE 86
```

Definition at line 91 of file [errno.h](#).

7.43.1.125 ETIME

```
#define ETIME 62
```

Definition at line 67 of file [errno.h](#).

7.43.1.126 ETIMEDOUT

```
#define ETIMEDOUT 110
```

Definition at line 116 of file [errno.h](#).

7.43.1.127 ETOOMANYREFS

```
#define ETOOMANYREFS 109
```

Definition at line 115 of file [errno.h](#).

7.43.1.128 ETXTBSY

```
#define ETXTBSY 26
```

Definition at line 31 of file [errno.h](#).

7.43.1.129 EUCLEAN

```
#define EUCLEAN 117
```

Definition at line 123 of file [errno.h](#).

7.43.1.130 EUNATCH

```
#define EUNATCH 49
```

Definition at line 54 of file [errno.h](#).

7.43.1.131 EUSERS

```
#define EUSERS 87
```

Definition at line 92 of file [errno.h](#).

7.43.1.132 EWOULDBLOCK

```
#define EWOULDBLOCK EAGAIN
```

Definition at line 46 of file [errno.h](#).

7.43.1.133 EXDEV

```
#define EXDEV 18
```

Definition at line 23 of file [errno.h](#).

7.43.1.134 EXFULL

```
#define EXFULL 54
```

Definition at line 59 of file [errno.h](#).

7.44 [errno.h](#)

[Go to the documentation of this file.](#)

```
00001 // A Temporary errno while i get scheduling online
00002
00003 #ifndef _ABIBITS_ERRNO_H
00004 #define _ABIBITS_ERRNO_H
00005
00006 #define EPERM 1
00007 #define ENOENT 2
00008 #define ESRCH 3
00009 #define EINTR 4
00010 #define EIO 5
00011 #define ENXIO 6
00012 #define E2BIG 7
00013 #define ENOEXEC 8
00014 #define EBADF 9
00015 #define ECHILD 10
00016 #define EAGAIN 11
00017 #define ENOMEM 12
00018 #define EACCES 13
00019 #define EFAULT 14
00020 #define ENOTBLK 15
00021 #define EBUSY 16
00022 #define EEXIST 17
00023 #define EXDEV 18
00024 #define ENODEV 19
00025 #define ENOTDIR 20
00026 #define EISDIR 21
00027 #define EINVAL 22
00028 #define ENFILE 23
00029 #define EMFILE 24
00030 #define ENOTTY 25
00031 #define ETXTBSY 26
00032 #define EFBIG 27
00033 #define ENOSPC 28
00034 #define ESPIPE 29
00035 #define EROFS 30
00036 #define EMLINK 31
00037 #define EPIPE 32
00038 #define EDOM 33
00039 #define ERANGE 34
00040 #define EDEADLK 35
00041 #define ENAMETOOLONG 36
00042 #define ENOLCK 37
00043 #define ENOSYS 38
00044 #define ENOTEMPTY 39
00045 #define ELOOP 40
00046 #define EWOULDBLOCK EAGAIN
00047 #define ENOMSG 42
00048 #define EIDRM 43
00049 #define ECHRNG 44
00050 #define EL2NSYNC 45
00051 #define EL3HLT 46
00052 #define EL3RST 47
00053 #define ELNRNG 48
00054 #define EUNATCH 49
```

```
00055 #define ENOCSI 50
00056 #define EL2HLT 51
00057 #define EBADE 52
00058 #define EBADR 53
00059 #define EXFULL 54
00060 #define ENOANO 55
00061 #define EBADRQC 56
00062 #define EBADSLT 57
00063 #define EDEADLOCK EDEADLK
00064 #define EBFONT 59
00065 #define ENOSTR 60
00066 #define ENODATA 61
00067 #define ETIME 62
00068 #define ENOSR 63
00069 #define ENONET 64
00070 #define ENOPKG 65
00071 #define EREMOTE 66
00072 #define ENOLINK 67
00073 #define EADV 68
00074 #define ESRMNT 69
00075 #define ECOMM 70
00076 #define EPROTO 71
00077 #define EMULTIHOP 72
00078 #define EDOTDOT 73
00079 #define EBADMSG 74
00080 #define EOVERFLOW 75
00081 #define ENOTUNIQ 76
00082 #define EBAFED 77
00083 #define EREMCHG 78
00084 #define ELIBACC 79
00085 #define ELIBBAD 80
00086 #define ELIBSCN 81
00087 #define ELIBMAX 82
00088 #define ELIBEXEC 83
00089 #define EILSEQ 84
00090 #define ERESTART 85
00091 #define ESTRPIPE 86
00092 #define EUSERS 87
00093 #define ENOTSOCK 88
00094 #define EDESTADDRREQ 89
00095 #define EMSGSIZE 90
00096 #define EPROTOTYPE 91
00097 #define ENOPROTOOPT 92
00098 #define EPROTONOSUPPORT 93
00099 #define ESOCKTNOSUPPORT 94
00100 #define EOPNOTSUPP 95
00101 #define ENOTSUP EOPNOTSUPP
00102 #define EPPFNOSUPPORT 96
00103 #define EAFNOSUPPORT 97
00104 #define EADDRINUSE 98
00105 #define EADDRNOTAVAIL 99
00106 #define ENETDOWN 100
00107 #define ENETUNREACH 101
00108 #define ENETRESET 102
00109 #define ECONNABORTED 103
00110 #define ECONNRESET 104
00111 #define ENOBUFS 105
00112 #define EISCONN 106
00113 #define ENOTCONN 107
00114 #define ESHUTDOWN 108
00115 #define ETOOMANYREFS 109
00116 #define ETIMEDOUT 110
00117 #define ECONNREFUSED 111
00118 #define EHOSTDOWN 112
00119 #define EHOSTUNREACH 113
00120 #define EALREADY 114
00121 #define EINPROGRESS 115
00122 #define ESTALE 116
00123 #define EUCLEAN 117
00124 #define ENOTNAM 118
00125 #define ENAVAIL 119
00126 #define EISNAM 120
00127 #define EREMOTEIO 121
00128 #define EDQUOT 122
00129 #define ENOMEDIUM 123
00130 #define EMEDIUMTYPE 124
00131 #define ECANCELED 125
00132 #define ENOKEY 126
00133 #define EKEYEXPIRED 127
00134 #define EKEYREVOKED 128
00135 #define EKEYREJECTED 129
00136 #define EOWNERDEAD 130
00137 #define ENOTRECOVERABLE 131
00138 #define ERFKILL 132
00139 #define EHWPOISON 133
00140
00141 #endif // _ABIBITS_ERRNO_H
```


Data Structures

- struct `GDT_Desc`
Operand layout consumed by LGDT.
- struct `GDT_Entry`
Legacy-format 8-byte segment descriptor.
- struct `GDT_Entry_16`
Descriptor storage used for the 16-bit compatibility entries.
- struct `GDT_Entry_32`
Descriptor storage used for the 32-bit compatibility entries.
- struct `TSS_Entry`
64-bit TSS system descriptor split into architectural fields.
- struct `GDT`
Complete VoyagerOS64 GDT image.

Macros

- `#define GDTAccessDPL(n) (n << 5)`
- `#define GDTKernelBaseSelector 0x28`
- `#define GDTUserBaseSelector 0x48`
- `#define GDTTSSSegment 0x50`
- `#define GDTAccess16Code (ReadWrite | Execute | Segments | Present)`
- `#define GDTAccess16Data (ReadWrite | Segments | Present)`
- `#define GDTAccess32Code (ReadWrite | Execute | Segments | Present)`
- `#define GDTAccess32Data (ReadWrite | Segments | Present)`
- `#define GDTAccessKernelCode (ReadWrite | Execute | Segments | Present)`
- `#define GDTAccessKernelData (ReadWrite | Segments | Present)`
- `#define GDTAccessUserCode (ReadWrite | Execute | Segments | GDTAccessDPL(3) | Present)`
- `#define GDTAccessUserData (ReadWrite | Segments | GDTAccessDPL(3) | Present)`

Enumerations

- enum `GDTAccessFlag` {
 `ReadWrite = (1 << 1)` , `DC = (1 << 2)` , `Execute = (1 << 3)` , `Segments = (1 << 4)` ,
 `Present = (1 << 7)` }
Common access-byte flag bits used to build GDT entries.

Functions

- void `LoadGDT_Stage1` (void)
Build/load the initial descriptor table used during boot.
- void `gdt_load_tss` (struct `TSS` *tss)
Install/update the TSS descriptor for tss.
- void `gdt_reload` (void)
Reload segment/GDT state after table construction.

Variables

- `uint64_t rsp0`

7.45.1 Detailed Description

x86-64 Global Descriptor Table and TSS descriptor interface.

Definition in file `gdt.h`.

7.45.2 Macro Definition Documentation

7.45.2.1 GDAccess16Code

```
#define GDAccess16Code (ReadWrite | Execute | Segments | Present)
```

Definition at line 32 of file `gdt.h`.

7.45.2.2 GDAccess16Data

```
#define GDAccess16Data (ReadWrite | Segments | Present)
```

Definition at line 33 of file `gdt.h`.

7.45.2.3 GDAccess32Code

```
#define GDAccess32Code (ReadWrite | Execute | Segments | Present)
```

Definition at line 34 of file `gdt.h`.

7.45.2.4 GDAccess32Data

```
#define GDAccess32Data (ReadWrite | Segments | Present)
```

Definition at line 35 of file `gdt.h`.

7.45.2.5 GDTAccessDPL

```
#define GDTAccessDPL(  
    n ) (n << 5)
```

Encode a descriptor privilege level in a **GDT** access byte.

Definition at line 13 of file `gdt.h`.

7.45.2.6 GDTAccessKernelCode

```
#define GDTAccessKernelCode (ReadWrite | Execute | Segments | Present)
```

Definition at line 36 of file `gdt.h`.

7.45.2.7 GDTAccessKernelData

```
#define GDTAccessKernelData (ReadWrite | Segments | Present)
```

Definition at line 37 of file `gdt.h`.

7.45.2.8 GDTAccessUserCode

```
#define GDTAccessUserCode (ReadWrite | Execute | Segments | GDTAccessDPL(3) | Present)
```

Definition at line 38 of file `gdt.h`.

7.45.2.9 GDTAccessUserData

```
#define GDTAccessUserData (ReadWrite | Segments | GDTAccessDPL(3) | Present)
```

Definition at line 40 of file `gdt.h`.

7.45.2.10 GDTKernelBaseSelector

```
#define GDTKernelBaseSelector 0x28
```

Kernel code-segment selector used by the current **GDT** layout.

Definition at line 26 of file `gdt.h`.

7.45.2.11 GDTTSSSegment

```
#define GDTTSSSegment 0x50
```

TSS descriptor selector used by the current **GDT** layout.

Definition at line 30 of file `gdt.h`.

7.45.2.12 GDTUserBaseSelector

```
#define GDTUserBaseSelector 0x48
```

User code-segment selector used by the current **GDT** layout.

Definition at line 28 of file `gdt.h`.

7.45.3 Enumeration Type Documentation

7.45.3.1 GDTAccessFlag

```
enum GDTAccessFlag
```

Common access-byte flag bits used to build **GDT** entries.

Enumerator

ReadWrite	
DC	
Execute	
Segments	
Present	

Definition at line 16 of file `gdt.h`.

7.45.4 Function Documentation

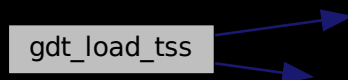
7.45.4.1 gdt_load_tss()

```
void gdt_load_tss (  
    struct TSS * tss )
```

Install/update the TSS descriptor for `tss`.

Definition at line 68 of file `gdt.c`.

Here is the call graph for this function:



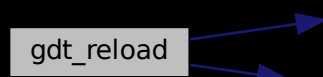
7.45.4.2 `gdt_reload()`

```
void gdt_reload (
    void )
```

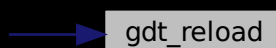
Reload segment/GDT state after table construction.

Definition at line 44 of file `gdt.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



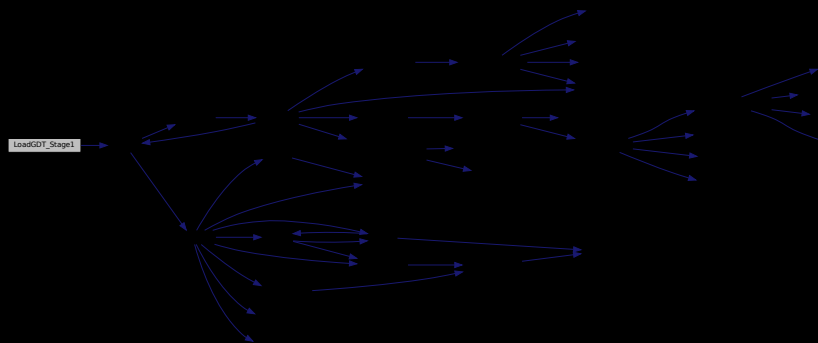
7.45.4.3 LoadGDT_Stage1()

```
void LoadGDT_Stage1 (
    void )
```

Build/load the initial descriptor table used during boot.

Definition at line 87 of file [gdt.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.45.5 Variable Documentation

7.45.5.1 rsp0

```
uint64_t rsp0 [extern]
```

Ring-0 stack pointer associated with the current [TSS](#) setup.

Definition at line 12 of file [gdt.c](#).

7.46 gdt.h

Go to the documentation of this file.

```

00001
00006 #pragma once
00007
00008 #include "global_defs.h"
00009 #include "tss.h"
00010 #include <stdint.h>
00011
00013 #define GDTAccessDPL(n) (n << 5)
00014
00016 enum GDTAccessFlag
00017 {
00018     ReadWrite = (1 << 1),
00019     DC        = (1 << 2),
00020     Execute   = (1 << 3),
00021     Segments  = (1 << 4),
00022     Present   = (1 << 7),
00023 };
00024
00026 #define GDTKernelBaseSelector 0x28
00028 #define GDTUserBaseSelector 0x48
00030 #define GDTSSSegment 0x50
00031
00032 #define GDTAccess16Code (ReadWrite | Execute | Segments | Present)
00033 #define GDTAccess16Data (ReadWrite | Segments | Present)
00034 #define GDTAccess32Code (ReadWrite | Execute | Segments | Present)
00035 #define GDTAccess32Data (ReadWrite | Segments | Present)
00036 #define GDTAccessKernelCode (ReadWrite | Execute | Segments | Present)
00037 #define GDTAccessKernelData (ReadWrite | Segments | Present)
00038 #define GDTAccessUserCode \
00039 (ReadWrite | Execute | Segments | GDTAccessDPL(3) | Present)
00040 #define GDTAccessUserData (ReadWrite | Segments | GDTAccessDPL(3) | Present)
00041
00043 struct PACKED GDT_Desc
00044 {
00045     uint16_t size;
00046     uint64_t offset;
00047 };
00048
00050 struct PACKED GDT_Entry
00051 {
00052     uint16_t limit_low;
00053     uint16_t base_low;
00054     uint8_t  base_middle;
00055     uint8_t  access_flag;
00056     uint8_t  limit_flags;
00057     uint8_t  base_high;
00058 };
00059
00061 struct PACKED GDT_Entry_16
00062 {
00063     uint16_t limit_low;
00064     uint16_t base_low;
00065     uint8_t  base_middle;
00066     uint8_t  access_flag;
00067     uint8_t  limit_flags;
00068     uint8_t  base_high;
00069 };
00070
00072 struct PACKED GDT_Entry_32
00073 {
00074     uint16_t limit_low;
00075     uint16_t base_low;
00076     uint8_t  base_middle;
00077     uint8_t  access_flag;
00078     uint8_t  limit_flags;
00079     uint8_t  base_high;
00080 };
00081
00083 struct PACKED TSS_Entry
00084 {
00085     uint16_t length;
00086     uint16_t base_low;
00087     uint8_t  base_mid;
00088     uint8_t  flags;
00089     uint8_t  flags2;
00090     uint8_t  base_high;
00091     uint32_t base_up;
00092     uint32_t reserved0;
00093 };
00094
00102 struct PACKED ALIGN_4K GDT
00103 {

```

```

00104     struct GDT_Entry    null;
00105     struct GDT_Entry_16 seg_16_code;
00106     struct GDT_Entry_16 seg_16_data;
00107     struct GDT_Entry_32 seg_32_code;
00108     struct GDT_Entry_32 seg_32_data;
00109     struct GDT_Entry    kernelCS;
00110     struct GDT_Entry    kernelData;
00111     struct GDT_Entry    userNull;
00112     struct GDT_Entry    userData;
00113     struct GDT_Entry    userCode;
00114     struct TSS_Entry    tss;
00115 };
00116
00118 extern uint64_t rsp0;
00119
00121 void LoadGDT_Stagel(void);
00123 void gdt_load_tss(struct TSS *tss);
00125 void gdt_reload(void);

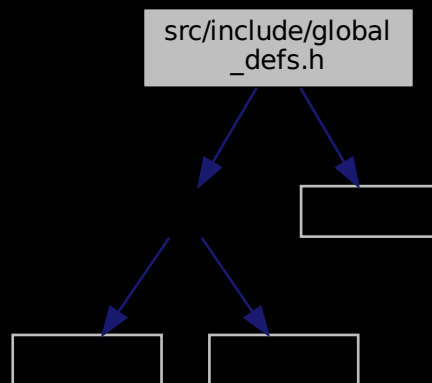
```

7.47 src/include/global_defs.h File Reference

```
#include "printf.h"
```

```
#include <stdint.h>
```

Include dependency graph for global_defs.h:



This graph shows which files directly or indirectly include this file:



Macros

- `#define PACKED __attribute__((packed))`
- `#define ALIGN_4K __attribute__((aligned(0x1000)))`
- `#define ALIGN_16BIT __attribute__((aligned(0x10)))`
- `#define CAS __sync_bool_compare_and_swap`
- `#define MIN(A, B)`
- `#define MAX(A, B)`
- `#define ASSERT(COND)`

Typedefs

- typedef int32_t [mode_t](#)
- typedef signed long [ssize_t](#)
- typedef unsigned long long [lsize_t](#)
- typedef void * [HANDLE](#)

7.47.1 Macro Definition Documentation

7.47.1.1 ALIGN_16BIT

```
#define ALIGN_16BIT __attribute__((aligned(0x10)))
```

Definition at line 9 of file [global_defs.h](#).

7.47.1.2 ALIGN_4K

```
#define ALIGN_4K __attribute__((aligned(0x1000)))
```

Definition at line 7 of file [global_defs.h](#).

7.47.1.3 ASSERT

```
#define ASSERT(  
    COND )
```

Value:

```
do  
{  
    if (! (COND))  
    {  
        printf("%s\n", "!!!KERNEL PANIC!!!");  
        printf("%s\n", "Assertion failed: " #COND);  
        printf("%s\n", "!!!KERNEL PANIC!!!");  
        __asm__("hlt");  
    }  
} while (0)
```

Definition at line 27 of file [global_defs.h](#).

7.47.1.4 CAS

```
#define CAS __sync_bool_compare_and_swap
```

Definition at line 11 of file [global_defs.h](#).

7.47.1.5 MAX

```
#define MAX(  
    A,  
    B )
```

Value:

```
{  
    __auto_type MAX_a = A;  
    __auto_type MAX_b = B;  
    MAX_a > MAX_b ? MAX_a : MAX_b;  
}
```

Definition at line 20 of file [global_defs.h](#).

7.47.1.6 MIN

```
#define MIN(  
    A,  
    B )
```

Value:

```
{  
    __auto_type MIN_a = A;  
    __auto_type MIN_b = B;  
    MIN_a < MIN_b ? MIN_a : MIN_b;  
}
```

Definition at line 13 of file [global_defs.h](#).

7.47.1.7 PACKED

```
struct madt_nmi PACKED __attribute__((packed))
```

Definition at line 5 of file [global_defs.h](#).

7.47.2 Typedef Documentation

7.47.2.1 HANDLE

```
typedef void* HANDLE
```

Definition at line 42 of file [global_defs.h](#).

7.47.2.2 lsize_t

```
typedef unsigned long long lsize_t
```

Definition at line 41 of file `global_defs.h`.

7.47.2.3 mode_t

```
typedef int32_t mode_t
```

Definition at line 39 of file `global_defs.h`.

7.47.2.4 ssize_t

```
typedef signed long ssize_t
```

Definition at line 40 of file `global_defs.h`.

7.48 global_defs.h

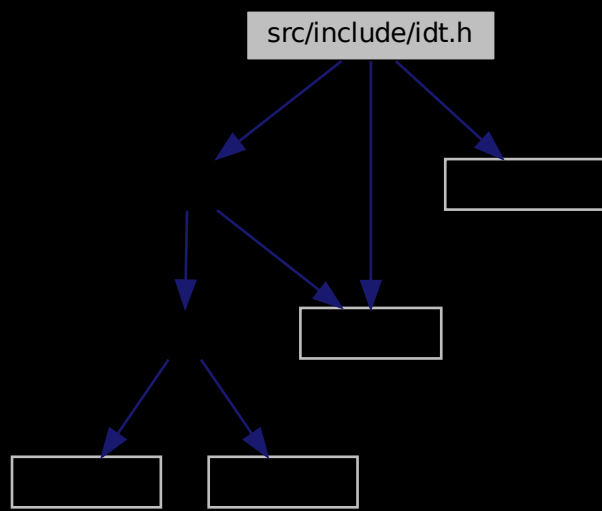
[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include "printf.h"
00003 #include <stdint.h>
00004
00005 #define PACKED __attribute__((packed))
00006
00007 #define ALIGN_4K __attribute__((aligned(0x1000)))
00008
00009 #define ALIGN_16BIT __attribute__((aligned(0x10)))
00010
00011 #define CAS __sync_bool_compare_and_swap
00012
00013 #define MIN(A, B) \
00014 ({ \
00015     __auto_type MIN_a = A; \
00016     __auto_type MIN_b = B; \
00017     MIN_a < MIN_b ? MIN_a : MIN_b; \
00018 })
00019
00020 #define MAX(A, B) \
00021 ({ \
00022     __auto_type MAX_a = A; \
00023     __auto_type MAX_b = B; \
00024     MAX_a > MAX_b ? MAX_a : MAX_b; \
00025 })
00026
00027 #define ASSERT(COND) \
00028 do \
00029 { \
00030     if (!(COND)) \
00031     { \
00032         printf("%s\n", "!!!KERNEL PANIC!!!"); \
00033         printf("%s\n", "Assertion failed: " #COND); \
00034         printf("%s\n", "!!!KERNEL PANIC!!!"); \
00035         __asm__("hlt"); \
00036     } \
00037 } while (0)
00038
00039 typedef int32_t mode_t;
00040 typedef signed long ssize_t;
00041 typedef unsigned long long lsize_t;
00042 typedef void *HANDLE;
```

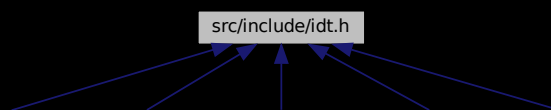
7.49 src/include/idt.h File Reference

x86-64 Interrupt Descriptor Table and dynamic vector interface.

```
#include "global_defs.h"
#include <stdbool.h>
#include <stdint.h>
Include dependency graph for idt.h:
```



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `__attribute__`

Packed 16-byte x86-64 IDT gate descriptor.

Macros

- `#define IDT_MAX_DESCRIPTOR 256`
- `#define IDT_CPU_EXCEPTION_COUNT 32`
- `#define IDT_HDW_INTERRUPT_COUNT 16`
- `#define IDT_DESCRIPTOR_X16_INTERRUPT 0x06`
- `#define IDT_DESCRIPTOR_X16_TRAP 0x07`
- `#define IDT_DESCRIPTOR_X32_TASK 0x05`
- `#define IDT_DESCRIPTOR_X32_INTERRUPT 0x0E`
- `#define IDT_DESCRIPTOR_X32_TRAP 0x0F`
- `#define IDT_DESCRIPTOR_RING1 0x40`
- `#define IDT_DESCRIPTOR_RING2 0x20`
- `#define IDT_DESCRIPTOR_RING3 0x60`
- `#define IDT_DESCRIPTOR_PRESENT 0x80`
- `#define IDT_DESCRIPTOR_EXCEPTION (IDT_DESCRIPTOR_X32_INTERRUPT | IDT_DESCRIPTOR_PRESENT)`
- `#define IDT_DESCRIPTOR_EXTERNAL (IDT_DESCRIPTOR_X32_INTERRUPT | IDT_DESCRIPTOR_PRESENT)`
- `#define IDT_DESCRIPTOR_CALL`

Functions

- `void idt_reload (void)`
Reload the processor IDTR from Voyager's current descriptor table.
- `uint8_t idt_allocate_vector (void)`
Reserve a vector from Voyager's dynamic interrupt-vector pool.
- `void idt_free_vector (uint8_t vector)`
Release a vector previously obtained from `idt_allocate_vector()`.
- `void idt_set_descriptor (uint8_t vector, uintptr_t isr, uint8_t flags, uint8_t ist)`
Install one IDT descriptor.
- `void idt_init (void)`
Build and install the kernel's initial IDT.
- `void idt_reg_test (void)`
Boot-time self-test for dynamic ISR registration/dispatch.
- `void yield_register (void)`
Register the software-yield test vector used by current bring-up code.

Variables

- `uint64_t isr_stub_table []`
- `void * isr_delta []`

7.49.1 Detailed Description

x86-64 Interrupt Descriptor Table and dynamic vector interface.

Definition in file `idt.h`.

7.49.2 Macro Definition Documentation

7.49.2.1 IDT_CPU_EXCEPTION_COUNT

```
#define IDT_CPU_EXCEPTION_COUNT 32
```

Number of architecturally reserved CPU exception vectors.

Definition at line 15 of file [idt.h](#).

7.49.2.2 IDT_DESCRIPTOR_CALL

```
#define IDT_DESCRIPTOR_CALL
```

Value:

```
(IDT_DESCRIPTOR_X32_INTERRUPT | IDT_DESCRIPTOR_PRESENT | \
 IDT_DESCRIPTOR_RING3)
```

Descriptor attributes for ring-3 callable interrupt gates.

Definition at line 36 of file [idt.h](#).

7.49.2.3 IDT_DESCRIPTOR_EXCEPTION

```
#define IDT_DESCRIPTOR_EXCEPTION (IDT_DESCRIPTOR_X32_INTERRUPT | IDT_DESCRIPTOR_PRESENT)
```

Descriptor attributes used for exception gates.

Definition at line 30 of file [idt.h](#).

7.49.2.4 IDT_DESCRIPTOR_EXTERNAL

```
#define IDT_DESCRIPTOR_EXTERNAL (IDT_DESCRIPTOR_X32_INTERRUPT | IDT_DESCRIPTOR_PRESENT)
```

Descriptor attributes used for external interrupt gates.

Definition at line 33 of file [idt.h](#).

7.49.2.5 IDT_DESCRIPTOR_PRESENT

```
#define IDT_DESCRIPTOR_PRESENT 0x80
```

Definition at line 27 of file [idt.h](#).

7.49.2.6 IDT_DESCRIPTOR_RING1

```
#define IDT_DESCRIPTOR_RING1 0x40
```

Definition at line 24 of file [idt.h](#).

7.49.2.7 IDT_DESCRIPTOR_RING2

```
#define IDT_DESCRIPTOR_RING2 0x20
```

Definition at line 25 of file [idt.h](#).

7.49.2.8 IDT_DESCRIPTOR_RING3

```
#define IDT_DESCRIPTOR_RING3 0x60
```

Definition at line 26 of file [idt.h](#).

7.49.2.9 IDT_DESCRIPTOR_X16_INTERRUPT

```
#define IDT_DESCRIPTOR_X16_INTERRUPT 0x06
```

Definition at line 19 of file [idt.h](#).

7.49.2.10 IDT_DESCRIPTOR_X16_TRAP

```
#define IDT_DESCRIPTOR_X16_TRAP 0x07
```

Definition at line 20 of file [idt.h](#).

7.49.2.11 IDT_DESCRIPTOR_X32_INTERRUPT

```
#define IDT_DESCRIPTOR_X32_INTERRUPT 0x0E
```

Definition at line 22 of file [idt.h](#).

7.49.2.12 IDT_DESCRIPTOR_X32_TASK

```
#define IDT_DESCRIPTOR_X32_TASK 0x05
```

Definition at line 21 of file [idt.h](#).

7.49.2.13 IDT_DESCRIPTOR_X32_TRAP

```
#define IDT_DESCRIPTOR_X32_TRAP 0x0F
```

Definition at line 23 of file [idt.h](#).

7.49.2.14 IDT_HDW_INTERRUPT_COUNT

```
#define IDT_HDW_INTERRUPT_COUNT 16
```

Legacy PIC hardware-interrupt vector count.

Definition at line 17 of file [idt.h](#).

7.49.2.15 IDT_MAX_DESCRIPTOR

```
#define IDT_MAX_DESCRIPTOR 256
```

Architectural maximum number of IDT entries.

Definition at line 13 of file [idt.h](#).

7.49.3 Function Documentation

7.49.3.1 idt_allocate_vector()

```
uint8_t idt_allocate_vector (
    void )
```

Reserve a vector from Voyager's dynamic interrupt-vector pool.

Returns

Allocated vector number.

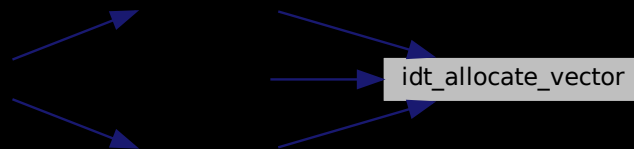
Reserve a vector from Voyager's dynamic interrupt-vector pool.

Returns

Allocated vector, or 0 if none remain. Vector 0 is permanently reserved by the CPU exception table, so it is safe as the failure sentinel.

Definition at line 70 of file [idt.c](#).

Here is the caller graph for this function:



7.49.3.2 idt_free_vector()

```
void idt_free_vector (
    uint8_t vector )
```

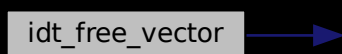
Release a vector previously obtained from [idt_allocate_vector\(\)](#).

Parameters

<i>vector</i>	Vector number to release.
---------------	---------------------------

Definition at line 92 of file [idt.c](#).

Here is the call graph for this function:



7.49.3.3 `idt_init()`

```
void idt_init (  
    void )
```

Build and install the kernel's initial IDT.

Definition at line 39 of file `idt.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



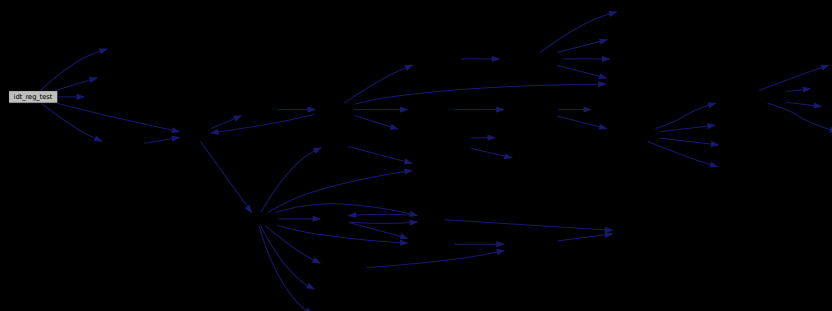
7.49.3.4 idt_reg_test()

```
void idt_reg_test (  
    void )
```

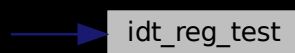
Boot-time self-test for dynamic ISR registration/dispatch.

Definition at line 108 of file [idt.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.49.3.5 idt_reload()

```
void idt_reload (  
    void )
```

Reload the processor IDTR from Voyager's current descriptor table.

Definition at line 84 of file [idt.c](#).

Here is the caller graph for this function:



7.49.3.6 idt_set_descriptor()

```
void idt_set_descriptor (
    uint8_t vector,
    uintptr_t isr,
    uint8_t flags,
    uint8_t ist )
```

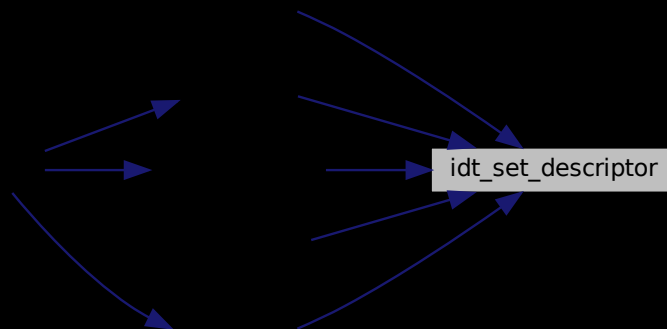
Install one IDT descriptor.

Parameters

<i>vector</i>	IDT vector number.
<i>isr</i>	Address of the interrupt entry stub/handler.
<i>flags</i>	Gate attribute byte.
<i>ist</i>	Interrupt Stack Table selector field.

Definition at line 23 of file [idt.c](#).

Here is the caller graph for this function:



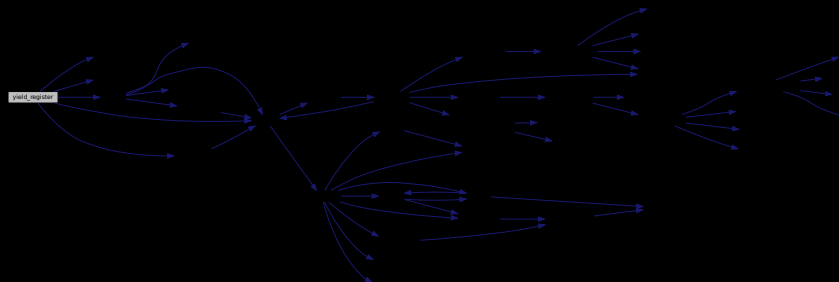
7.49.3.7 yield_register()

```
void yield_register (
    void )
```

Register the software-yield test vector used by current bring-up code.

Definition at line 129 of file [idt.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.49.4 Variable Documentation

7.49.4.1 `isr_delta`

```
void* isr_delta[] [extern]
```

Dynamic ISR dispatch targets indexed by vector-specific glue.

Definition at line 17 of file `idt.c`.

7.49.4.2 `isr_stub_table`

```
uint64_t isr_stub_table[] [extern]
```

Assembly-generated ISR entry-stub table.

7.50 idt.h

Go to the documentation of this file.

```

00001
00006 #pragma once
00007
00008 #include "global_defs.h"
00009 #include <stdbool.h>
00010 #include <stdint.h>
00011
00013 #define IDT_MAX_DESCRIPTOR 256
00015 #define IDT_CPU_EXCEPTION_COUNT 32
00017 #define IDT_HDW_INTERRUPT_COUNT 16
00018
00019 #define IDT_DESCRIPTOR_X16_INTERRUPT 0x06
00020 #define IDT_DESCRIPTOR_X16_TRAP 0x07
00021 #define IDT_DESCRIPTOR_X32_TASK 0x05
00022 #define IDT_DESCRIPTOR_X32_INTERRUPT 0x0E
00023 #define IDT_DESCRIPTOR_X32_TRAP 0x0F
00024 #define IDT_DESCRIPTOR_RING1 0x40
00025 #define IDT_DESCRIPTOR_RING2 0x20
00026 #define IDT_DESCRIPTOR_RING3 0x60
00027 #define IDT_DESCRIPTOR_PRESENT 0x80
00028
00030 #define IDT_DESCRIPTOR_EXCEPTION \
00031 (IDT_DESCRIPTOR_X32_INTERRUPT | IDT_DESCRIPTOR_PRESENT)
00033 #define IDT_DESCRIPTOR_EXTERNAL \
00034 (IDT_DESCRIPTOR_X32_INTERRUPT | IDT_DESCRIPTOR_PRESENT)
00036 #define IDT_DESCRIPTOR_CALL \
00037 (IDT_DESCRIPTOR_X32_INTERRUPT | IDT_DESCRIPTOR_PRESENT | \
00038 IDT_DESCRIPTOR_RING3)
00039
00041 extern uint64_t isr_stub_table[];
00042
00044 extern void *isr_delta[];
00045
00047 typedef struct
00048 {
00049     uint16_t base_low;
00050     uint16_t cs;
00051     uint8_t ist;
00052     uint8_t attributes;
00053     uint16_t base_mid;
00054     uint32_t base_high;
00055     uint32_t rsv0;
00056 } __attribute__((packed)) idt_desc_t;
00057
00059 typedef struct
00060 {
00061     uint16_t limit;
00062     uint64_t base;
00063 } __attribute__((packed)) idtr_t;
00064
00066 void idt_reload(void);
00067
00072 uint8_t idt_allocate_vector(void);
00073
00078 void idt_free_vector(uint8_t vector);
00079
00087 void idt_set_descriptor(uint8_t vector,
00088                         uintptr_t isr,
00089                         uint8_t flags,
00090                         uint8_t ist);
00091
00093 void idt_init(void);
00094
00096 void idt_reg_test(void);
00097
00099 void yield_register(void);

```

7.51 src/include/interrupts.h File Reference

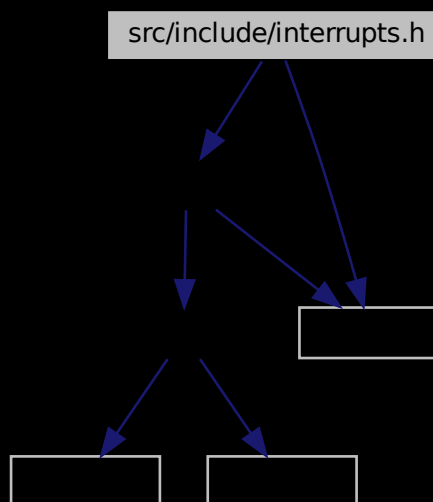
Register snapshot passed through Voyager's interrupt-dispatch path.

```

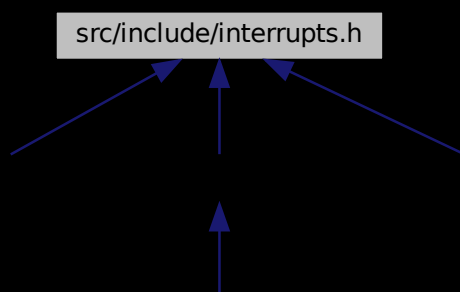
#include "global_defs.h"
#include <stdint.h>

```

Include dependency graph for interrupts.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `isr_xframe_t`

Extended interrupt frame matching the current ISR assembly/C boundary.

7.51.1 Detailed Description

Register snapshot passed through Voyager's interrupt-dispatch path.

Definition in file `interrupts.h`.

7.52 interrupts.h

Go to the documentation of this file.

```

00001
00006 #pragma once
00007 #include "global_defs.h"
00008 #include <stdint.h>
00009
00022 typedef struct
00023 {
00024     struct
00025     {
00026         uint64_t cr4;
00027         uint64_t cr3;
00028         uint64_t cr2;
00029         uint64_t cr0;
00030     } control_registers;
00031
00032     struct
00033     {
00034         uint64_t rdi;
00035         uint64_t rsi;
00036         uint64_t rdx;
00037         uint64_t rcx;
00038         uint64_t rbx;
00039         uint64_t rax;
00040     } general_registers;
00041
00042     struct
00043     {
00044         uint64_t rbp;
00045         uint64_t vector;
00046         uint64_t error_code;
00047         uint64_t rip;
00048         uint64_t cs;
00049         uint64_t rflags;
00050         uint64_t rsp;
00051         uint64_t dss;
00052     } base_frame;
00053 } isr_xframe_t;

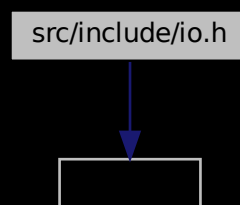
```

7.53 src/include/io.h File Reference

x86 port-I/O primitives.

```
#include <stdint.h>
```

Include dependency graph for io.h:



This graph shows which files directly or indirectly include this file:



Macros

- `#define _IO_H`

Functions

- `uint8_t inb` (`uint16_t port`)
Read one byte from an x86 I/O port.
- `void outb` (`uint16_t port`, `uint8_t val`)
Write one byte to an x86 I/O port.
- `void io_wait` (`void`)
Perform the traditional short I/O delay used around legacy devices.

7.53.1 Detailed Description

x86 port-I/O primitives.

Definition in file `io.h`.

7.53.2 Macro Definition Documentation

7.53.2.1 _IO_H

```
#define _IO_H
```

Definition at line 8 of file `io.h`.

7.53.3 Function Documentation

7.53.3.1 inb()

```
uint8_t inb (  
    uint16_t port )
```

Read one byte from an x86 I/O port.

Parameters

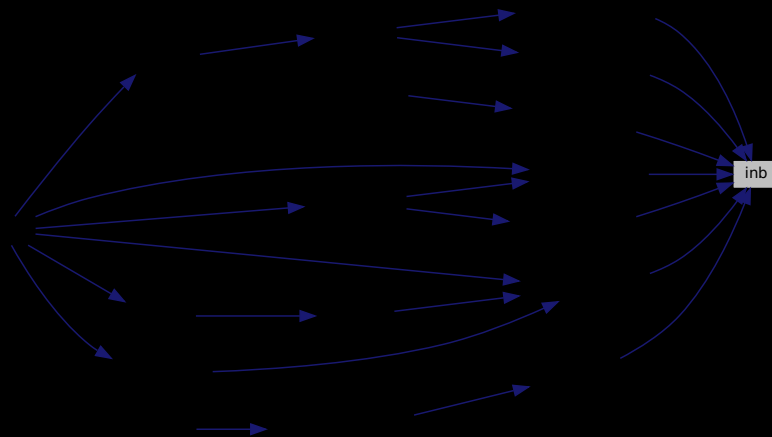
<i>port</i>	I/O-port address.
-------------	-------------------

Returns

Byte read from the requested port.

Definition at line 5 of file [io.c](#).

Here is the caller graph for this function:



7.53.3.2 io_wait()

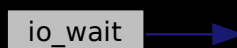
```
void io_wait (  
    void )
```

Perform the traditional short I/O delay used around legacy devices.

This helper is used by hardware such as the PIC/PIT/PS2 paths where ordered or slightly delayed port transactions are required by the device interface.

Definition at line 17 of file [io.c](#).

Here is the call graph for this function:



7.53.3.3 outb()

```
void outb (
    uint16_t port,
    uint8_t val )
```

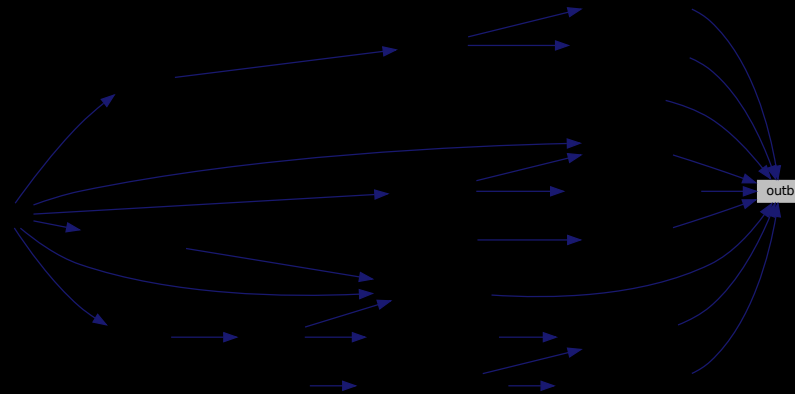
Write one byte to an x86 I/O port.

Parameters

<i>port</i>	I/O-port address.
<i>val</i>	Byte to write.

Definition at line 12 of file [io.c](#).

Here is the caller graph for this function:



7.54 io.h

[Go to the documentation of this file.](#)

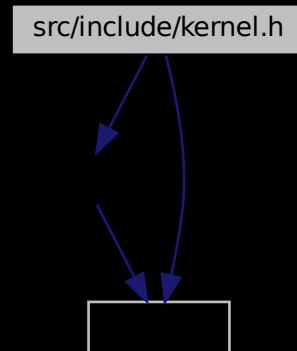
```
00001
00006 #pragma once
00007 #ifndef _IO_H
00008 #define _IO_H
00009
00010 #include <stdint.h>
00011
00017 uint8_t inb(uint16_t port);
00018
00024 void outb(uint16_t port, uint8_t val);
00025
00032 void io_wait(void);
00033
00034 #endif
```

7.55 src/include/kernel.h File Reference

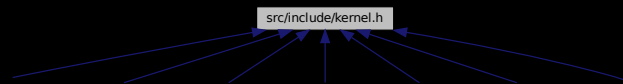
Global kernel bring-up state shared by low-level subsystems.

```
#include "limine.h"
#include <stdint.h>
```

Include dependency graph for kernel.h:



This graph shows which files directly or indirectly include this file:



Macros

- `#define _KERNEL_H`

Variables

- `uint32_t bootspace`
Console-routing state used during the bootloader-to-Voyager handoff.
- `uint8_t kerror_mode`
Global kernel error/diagnostic mode state.
- `uint8_t init_done`
Nonzero once the main kernel initialization sequence is complete.
- `volatile struct limine_terminal_request early_term`
Limine terminal request used only by the early boot-console phase.

7.55.1 Detailed Description

Global kernel bring-up state shared by low-level subsystems.

Definition in file [kernel.h](#).

7.55.2 Macro Definition Documentation

7.55.2.1 `_KERNEL_H`

```
#define _KERNEL_H
```

Definition at line 8 of file [kernel.h](#).

7.55.3 Variable Documentation

7.55.3.1 `bootspace`

```
uint32_t bootspace [extern]
```

Console-routing state used during the bootloader-to-Voyager handoff.

Boot code changes this while output ownership moves from early boot services to Voyager's serial/framebuffer terminal. Code running after Voyager installs its own CR3 must not assume Limine terminal callbacks remain callable.

Definition at line 62 of file [kernel.c](#).

7.55.3.2 `early_term`

```
volatile struct limine\_terminal\_request early_term [extern]
```

Limine terminal request used only by the early boot-console phase.

Warning

The request/response data may remain readable after CR3 handoff, but the bootloader's terminal callback code is not part of Voyager's guaranteed post-handoff address space. Use Voyager-owned output facilities afterwards.

7.55.3.3 init_done

```
uint8_t init_done [extern]
```

Nonzero once the main kernel initialization sequence is complete.

Definition at line 65 of file [kernel.c](#).

7.55.3.4 kerror_mode

```
uint8_t kerror_mode [extern]
```

Global kernel error/diagnostic mode state.

Definition at line 63 of file [kernel.c](#).

7.56 kernel.h

[Go to the documentation of this file.](#)

```
00001
00006 #pragma once
00007 #ifndef _KERNEL_H
00008 #define _KERNEL_H
00009
00010 #include "limine.h"
00011 #include <stdint.h>
00012
00020 extern uint32_t bootspace;
00021
00023 extern uint8_t kerror_mode;
00024
00026 extern uint8_t init_done;
00027
00035 extern volatile struct limine_terminal_request early_term;
00036
00037 #endif
```

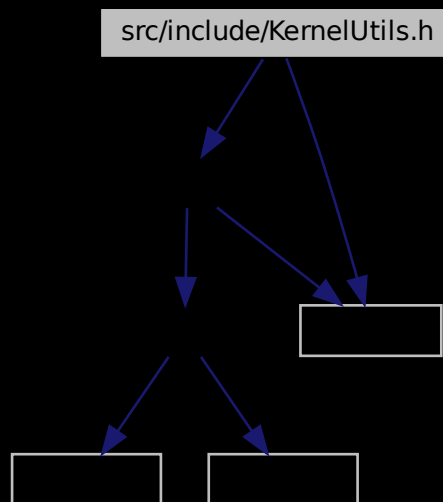
7.57 src/include/KernelUtils.h File Reference

Kernel feature switches, memory-reporting helpers, and shared utilities.

```
#include "global_defs.h"
```

```
#include <stdint.h>
```

Include dependency graph for KernelUtils.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `kswitches`
Compile-time/runtime-style kernel behavior switches.

Macros

- `#define` `_KERNEL_UTILS_H`

Functions

- void `print_memmap` (void)
Print the Limine-provided physical memory map for diagnostics.
- uint64_t `get_memory_size` (void)
Return detected physical memory size in the implementation's base unit.
- uint64_t `get_memory_size_gib` (void)

- `uint64_t bytes_to_mib (uint64_t bytes)`
Convert a byte count to MiB.
- `uint64_t bytes_to_gib (uint64_t bytes)`
Convert a byte count to GiB.
- `void restore_floats (void)`
Experimental floating-point restore helper.
- `void save_floats (void)`
Experimental floating-point save helper.

Variables

- `const struct kswitches k_mode`
- `struct term_context * term_context`

7.57.1 Detailed Description

Kernel feature switches, memory-reporting helpers, and shared utilities.
Definition in file `KernelUtils.h`.

7.57.2 Macro Definition Documentation

7.57.2.1 `_KERNEL_UTILS_H`

```
#define _KERNEL_UTILS_H
```

Definition at line 8 of file `KernelUtils.h`.

7.57.3 Function Documentation

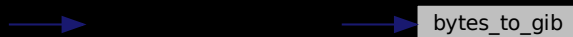
7.57.3.1 `bytes_to_gib()`

```
uint64_t bytes_to_gib (
    uint64_t bytes )
```

Convert a byte count to GiB.

Definition at line 27 of file `KernelUtils.c`.

Here is the caller graph for this function:



7.57.3.2 bytes_to_mib()

```
uint64_t bytes_to_mib (
    uint64_t bytes )
```

Convert a byte count to MiB.

Definition at line 22 of file KernelUtils.c.

Here is the caller graph for this function:



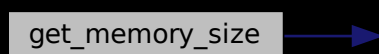
7.57.3.3 get_memory_size()

```
uint64_t get_memory_size (
    void )
```

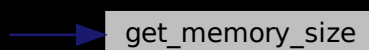
Return detected physical memory size in the implementation's base unit.

Definition at line 32 of file KernelUtils.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.57.3.6 restore_floats()

```
void restore_floats (
    void )
```

Experimental floating-point restore helper.

Warning

Per-task floating-point state is not qualified in 0.0.5.

7.57.3.7 save_floats()

```
void save_floats (
    void )
```

Experimental floating-point save helper.

Warning

Per-task floating-point state is not qualified in 0.0.5.

7.57.4 Variable Documentation

7.57.4.1 k_mode

```
const struct kswitches k_mode [extern]
```

Active kernel behavior/debug configuration.

Definition at line 9 of file [KernelUtils.c](#).

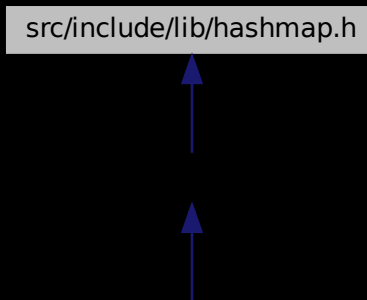
7.57.4.2 term_context

```
struct term_context* term_context [extern]
```

Voyager-owned terminal context after framebuffer handoff.

Definition at line 64 of file [kernel.c](#).

This graph shows which files directly or indirectly include this file:



Macros

- `#define _HASHMAP_H`
- `#define HASHMAP_KEY_DATA_MAX 256`
- `#define HASHMAP_INIT(CAP)`
- `#define HASHMAP_DELETE(HASHMAP)`
- `#define HASHMAP_TYPE(TYPE)`
- `#define HASHMAP_GET(HASHMAP, RET, KEY_DATA, KEY_LENGTH)`
- `#define HASHMAP_SGET(HASHMAP, RET, STRING)`
- `#define HASHMAP_REMOVE(HASHMAP, KEY_DATA, KEY_LENGTH)`
- `#define HASHMAP_SREMOVE(HASHMAP, STRING)`
- `#define HASHMAP_INSERT(HASHMAP, KEY_DATA, KEY_LENGTH, ITEM)`
- `#define HASHMAP_SINSERT(HASHMAP, STRING, ITEM)`

Functions

- static uint32_t `hash` (const uint8_t *data, size_t length)

7.59.1 Macro Definition Documentation

7.59.1.1 _HASHMAP_H

```
#define _HASHMAP_H
```

Definition at line 3 of file `hashmap.h`.

7.59.1.2 HASHMAP_DELETE

```
#define HASHMAP_DELETE(  
    HASHMAP )
```

Value:

```
do  
{  
    __auto_type HASHMAP_DELETE_hashmap = HASHMAP;  
  
    if (HASHMAP_DELETE_hashmap->buckets == NULL)  
    {  
        break;  
    }  
  
    for (size_t HASHMAP_DELETE_i = 0;  
        HASHMAP_DELETE_i < HASHMAP_DELETE_hashmap->cap;  
        HASHMAP_DELETE_i++)  
    {  
        __auto_type HASHMAP_DELETE_bucket =  
            &HASHMAP_DELETE_hashmap->buckets[HASHMAP_DELETE_i];  
  
        free(HASHMAP_DELETE_bucket->items);  
    }  
  
    free(HASHMAP_DELETE_hashmap->buckets);  
} while (0)
```

Definition at line 34 of file [hashmap.h](#).

7.59.1.3 HASHMAP_GET

```
#define HASHMAP_GET(  
    HASHMAP,  
    RET,  
    KEY_DATA,  
    KEY_LENGTH )
```

Definition at line 74 of file [hashmap.h](#).

7.59.1.4 HASHMAP_INIT

```
#define HASHMAP_INIT(  
    CAP )
```

Value:

```
{  
    .cap = (CAP), .buckets = NULL \
```

Definition at line 29 of file [hashmap.h](#).

7.59.1.5 HASHMAP_INSERT

```
#define HASHMAP_INSERT(  
    HASHMAP,  
    KEY_DATA,  
    KEY_LENGTH,  
    ITEM )
```

Definition at line 190 of file [hashmap.h](#).

7.59.1.6 HASHMAP_KEY_DATA_MAX

```
#define HASHMAP_KEY_DATA_MAX 256
```

Definition at line 27 of file [hashmap.h](#).

7.59.1.7 HASHMAP_REMOVE

```
#define HASHMAP_REMOVE(  
    HASHMAP,  
    KEY_DATA,  
    KEY_LENGTH )
```

Definition at line 128 of file [hashmap.h](#).

7.59.1.8 HASHMAP_SGET

```
#define HASHMAP_SGET(  
    HASHMAP,  
    RET,  
    STRING )
```

Value:

```
{  
    const char *HASHMAP_SGET_string = (STRING);  
    HASHMAP_GET(  
        HASHMAP, RET, HASHMAP_SGET_string, strlen(HASHMAP_SGET_string));  
}
```

Definition at line 121 of file [hashmap.h](#).

7.59.1.9 HASHMAP_SINSERT

```
#define HASHMAP_SINSERT(
    HASHMAP,
    STRING,
    ITEM )
```

Value:

```
do
{
    const char *HASHMAP_SINSERT_string = (STRING);
    HASHMAP_INSERT(HASHMAP,
        HASHMAP_SINSERT_string,
        strlen(HASHMAP_SINSERT_string),
        ITEM);
} while (0)
```

Definition at line 241 of file [hashmap.h](#).

7.59.1.10 HASHMAP_SREMOVE

```
#define HASHMAP_SREMOVE(
    HASHMAP,
    STRING )
```

Value:

```
{{
    const char *HASHMAP_SREMOVE_string = (STRING);
    HASHMAP_REMOVE(
        HASHMAP, HASHMAP_SREMOVE_string, strlen(HASHMAP_SREMOVE_string));
}}
```

Definition at line 183 of file [hashmap.h](#).

7.59.1.11 HASHMAP_TYPE

```
#define HASHMAP_TYPE(
    TYPE )
```

Value:

```
struct
{
    size_t cap;
    struct
    {
        size_t cap;
        size_t filled;
        struct
        {
            uint8_t key_data[HASHMAP_KEY_DATA_MAX];
            size_t key_length;
            TYPE item;
        } * items;
    } * buckets;
}
```

Definition at line 57 of file [hashmap.h](#).

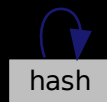
7.59.2 Function Documentation

7.59.2.1 hash()

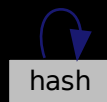
```
static uint32_t hash (
    const uint8_t * data,
    size_t length ) [inline], [static]
```

Definition at line 13 of file [hashmap.h](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.60 hashmap.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #ifndef _HASHMAP_H
00003 #define _HASHMAP_H
00004
00005 #include "../global_defs.h"
00006 #include "../liballoc.h"
00007 #include "../string.h"
00008 #include <stdbool.h>
00009 #include <stddef.h>
00010 #include <stdint.h>
00011
00012 // sdbm from: http://www.cse.yorku.ca/~oz/hash.html
00013 static inline uint32_t hash(const uint8_t *data, size_t length)
00014 {
```

```

00015     const uint8_t *data_u8 = data; // ignore warning lexer is being a cu*t
00016     uint32_t      hash      = 0;
00017
00018     for (size_t i = 0; i < length; i++)
00019     {
00020         uint32_t c = data_u8[i];
00021         hash      = c + (hash << 6) + (hash << 16) - hash;
00022     }
00023
00024     return hash;
00025 }
00026
00027 #define HASHMAP_KEY_DATA_MAX 256
00028
00029 #define HASHMAP_INIT(CAP) \
00030 { \
00031     .cap = (CAP), .buckets = NULL \
00032 }
00033
00034 #define HASHMAP_DELETE(HASHMAP) \
00035 do \
00036 { \
00037     __auto_type HASHMAP_DELETE_hashmap = HASHMAP; \
00038     \
00039     if (HASHMAP_DELETE_hashmap->buckets == NULL) \
00040     { \
00041         break; \
00042     } \
00043     \
00044     for (size_t HASHMAP_DELETE_i = 0; \
00045          HASHMAP_DELETE_i < HASHMAP_DELETE_hashmap->cap; \
00046          HASHMAP_DELETE_i++) \
00047     { \
00048         __auto_type HASHMAP_DELETE_bucket = \
00049             &HASHMAP_DELETE_hashmap->buckets[HASHMAP_DELETE_i]; \
00050         \
00051         free(HASHMAP_DELETE_bucket->items); \
00052     } \
00053     \
00054     free(HASHMAP_DELETE_hashmap->buckets); \
00055 } while (0)
00056
00057 #define HASHMAP_TYPE(TYPE) \
00058 struct \
00059 { \
00060     size_t cap; \
00061     struct \
00062     { \
00063         size_t cap; \
00064         size_t filled; \
00065         struct \
00066         { \
00067             uint8_t key_data[HASHMAP_KEY_DATA_MAX]; \
00068             size_t  key_length; \
00069             TYPE    item; \
00070         } * items; \
00071         * buckets; \
00072     } \
00073
00074 #define HASHMAP_GET(HASHMAP, RET, KEY_DATA, KEY_LENGTH) \
00075 { \
00076     __label__ out; \
00077     \
00078     bool HASHMAP_GET_ok = false; \
00079     \
00080     __auto_type HASHMAP_GET_key_data  = KEY_DATA; \
00081     __auto_type HASHMAP_GET_key_length = KEY_LENGTH; \
00082     \
00083     __auto_type HASHMAP_GET_hashmap = HASHMAP; \
00084     \
00085     if (HASHMAP_GET_hashmap->buckets == NULL) \
00086     { \
00087         goto out; \
00088     } \
00089     \
00090     size_t HASHMAP_GET_hash = \
00091         hash(HASHMAP_GET_key_data, HASHMAP_GET_key_length); \
00092     size_t HASHMAP_GET_index = \
00093         HASHMAP_GET_hash % HASHMAP_GET_hashmap->cap; \
00094     \
00095     __auto_type HASHMAP_GET_bucket = \
00096         &HASHMAP_GET_hashmap->buckets[HASHMAP_GET_index]; \
00097     \
00098     for (size_t HASHMAP_GET_i = 0; \
00099          HASHMAP_GET_i < HASHMAP_GET_bucket->filled; \
00100          HASHMAP_GET_i++) \
00101     {

```



```

00102 if (HASHMAP_GET_key_length !=
00103     HASHMAP_GET_bucket->items[HASHMAP_GET_i].key_length)
00104 {
00105     continue;
00106 }
00107 if (memcmp(HASHMAP_GET_key_data,
00108     HASHMAP_GET_bucket->items[HASHMAP_GET_i].key_data,
00109     HASHMAP_GET_key_length) == 0)
00110 {
00111     RET = HASHMAP_GET_bucket->items[HASHMAP_GET_i].item;
00112     HASHMAP_GET_ok = true;
00113     break;
00114 }
00115 }
00116 \
00117 out:
00118     \
00118     HASHMAP_GET_ok;
00119 })
00120
00121 #define HASHMAP_SGET(HASHMAP, RET, STRING)
00122 ({
00123     const char *HASHMAP_SGET_string = (STRING);
00124     HASHMAP_GET(
00125     HASHMAP, RET, HASHMAP_SGET_string, strlen(HASHMAP_SGET_string)); \
00126 })
00127
00128 #define HASHMAP_REMOVE(HASHMAP, KEY_DATA, KEY_LENGTH)
00129 ({
00130     __label__ out;
00131     \
00132     bool HASHMAP_REMOVE_ok = false;
00133     \
00134     __auto_type HASHMAP_REMOVE_key_data = KEY_DATA;
00135     __auto_type HASHMAP_REMOVE_key_length = KEY_LENGTH;
00136     \
00137     __auto_type HASHMAP_REMOVE_hashmap = HASHMAP;
00138     \
00139     if (HASHMAP_REMOVE_hashmap->buckets == NULL)
00140     {
00141         goto out;
00142     }
00143     \
00144     size_t HASHMAP_REMOVE_hash =
00145     hash(HASHMAP_REMOVE_key_data, HASHMAP_REMOVE_key_length);
00146     size_t HASHMAP_REMOVE_index =
00147     HASHMAP_REMOVE_hash % HASHMAP_REMOVE_hashmap->cap;
00148     \
00149     __auto_type HASHMAP_REMOVE_bucket =
00150     &HASHMAP_REMOVE_hashmap->buckets[HASHMAP_REMOVE_index];
00151     \
00152     for (size_t HASHMAP_REMOVE_i = 0;
00153     HASHMAP_REMOVE_i < HASHMAP_REMOVE_bucket->filled;
00154     HASHMAP_REMOVE_i++)
00155     {
00156         if (HASHMAP_REMOVE_key_length !=
00157             HASHMAP_REMOVE_bucket->items[HASHMAP_REMOVE_i].key_length)
00158         {
00159             continue;
00160         }
00161         if (memcmp(
00162             HASHMAP_REMOVE_key_data,
00163             HASHMAP_REMOVE_bucket->items[HASHMAP_REMOVE_i].key_data,
00164             HASHMAP_REMOVE_key_length) == 0)
00165         {
00166             if (HASHMAP_REMOVE_i != HASHMAP_REMOVE_bucket->filled - 1)
00167             {
00168                 memcpy(&HASHMAP_REMOVE_bucket->items[HASHMAP_REMOVE_i], \
00169                     &HASHMAP_REMOVE_bucket
00170                     ->items[HASHMAP_REMOVE_bucket->filled - 1], \
00171                     sizeof(*HASHMAP_REMOVE_bucket->items));
00172             }
00173             HASHMAP_REMOVE_bucket->filled--;
00174             HASHMAP_REMOVE_ok = true;
00175             break;
00176         }
00177     }
00178     \
00179     out:
00180     \
00180     HASHMAP_REMOVE_ok;
00181 })
00182
00183 #define HASHMAP_SREMOVE(HASHMAP, STRING)
00184 ({
00185     const char *HASHMAP_SREMOVE_string = (STRING);
00186     HASHMAP_REMOVE(

```

```

00187 HASHMAP, HASHMAP_SREMOVE_string, strlen(HASHMAP_SREMOVE_string)); \
00188 })
00189
00190 #define HASHMAP_INSERT(HASHMAP, KEY_DATA, KEY_LENGTH, ITEM)
00191 do
00192 {
00193     __auto_type HASHMAP_INSERT_key_data = KEY_DATA;
00194     __auto_type HASHMAP_INSERT_key_length = KEY_LENGTH;
00195     \
00196     __auto_type HASHMAP_INSERT_hashmap = HASHMAP;
00197     if (HASHMAP_INSERT_hashmap->buckets == NULL)
00198     {
00199         HASHMAP_INSERT_hashmap->buckets =
00200         malloc(HASHMAP_INSERT_hashmap->cap *
00201             sizeof(*HASHMAP_INSERT_hashmap->buckets));
00202     }
00203     \
00204     size_t HASHMAP_INSERT_hash =
00205     hash(HASHMAP_INSERT_key_data, HASHMAP_INSERT_key_length);
00206     size_t HASHMAP_INSERT_index =
00207     HASHMAP_INSERT_hash % HASHMAP_INSERT_hashmap->cap;
00208     \
00209     __auto_type HASHMAP_INSERT_bucket =
00210     &HASHMAP_INSERT_hashmap->buckets[HASHMAP_INSERT_index];
00211     \
00212     if (HASHMAP_INSERT_bucket->cap == 0)
00213     {
00214         HASHMAP_INSERT_bucket->cap = 16;
00215         HASHMAP_INSERT_bucket->items =
00216         malloc(HASHMAP_INSERT_bucket->cap *
00217             sizeof(*HASHMAP_INSERT_bucket->items));
00218     }
00219     \
00220     if (HASHMAP_INSERT_bucket->filled == HASHMAP_INSERT_bucket->cap)
00221     {
00222         HASHMAP_INSERT_bucket->cap *= 2;
00223         HASHMAP_INSERT_bucket->items =
00224         realloc(HASHMAP_INSERT_bucket->items,
00225             HASHMAP_INSERT_bucket->cap *
00226             sizeof(*HASHMAP_INSERT_bucket->items));
00227     }
00228     \
00229     __auto_type HASHMAP_INSERT_item =
00230     &HASHMAP_INSERT_bucket->items[HASHMAP_INSERT_bucket->filled];
00231     \
00232     memcpy(HASHMAP_INSERT_item->key_data,
00233         HASHMAP_INSERT_key_data,
00234         HASHMAP_INSERT_key_length);
00235     HASHMAP_INSERT_item->key_length = HASHMAP_INSERT_key_length;
00236     HASHMAP_INSERT_item->item = ITEM;
00237     \
00238     HASHMAP_INSERT_bucket->filled++;
00239 } while (0)
00240
00241 #define HASHMAP_SINSERT(HASHMAP, STRING, ITEM)
00242 do
00243 {
00244     const char *HASHMAP_SINSERT_string = (STRING);
00245     HASHMAP_INSERT(HASHMAP,
00246         HASHMAP_SINSERT_string,
00247         strlen(HASHMAP_SINSERT_string),
00248         ITEM);
00249 } while (0)
00250
00251 #endif

```

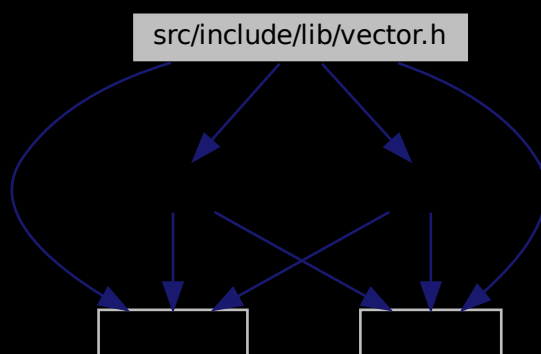
7.61 src/include/lib/vector.h File Reference

```

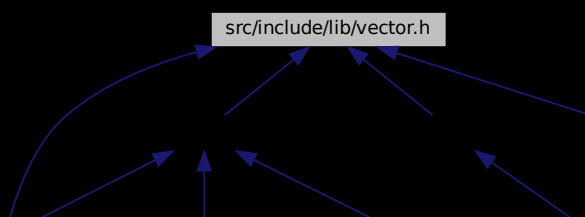
#include "../mm/kmalloc.h"
#include "../string.h"
#include <stddef.h>
#include <stdint.h>

```

Include dependency graph for vector.h:



This graph shows which files directly or indirectly include this file:



Macros

- `#define VECTOR_INVALID_INDEX (-1)`
- `#define VECTOR_INIT`
- `#define VECTOR_TYPE(TYPE)`
- `#define VECTOR_ENSURE_LENGTH(VEC, LENGTH)`
- `#define VECTOR_PUSH_BACK(VEC, VALUE)`
- `#define VECTOR_PUSH_FRONT(VEC, VALUE) VECTOR_INSERT(VEC, 0, VALUE)`
- `#define VECTOR_INSERT(VEC, IDX, VALUE)`
- `#define VECTOR_REMOVE(VEC, IDX)`
- `#define VECTOR_ITEM(VEC, IDX)`
- `#define VECTOR_FIND(VEC, VALUE)`
- `#define VECTOR_REMOVE_BY_VALUE(VEC, VALUE)`
- `#define VECTOR_FOR_EACH(VEC, BINDING, ...)`

7.61.1 Macro Definition Documentation

7.61.1.1 VECTOR_ENSURE_LENGTH

```
#define VECTOR_ENSURE_LENGTH(  
    VEC,  
    LENGTH )
```

Value:

```
do  
{  
    __auto_type VECTOR_ENSURE_LENGTH_vec = VEC;  
    if ((LENGTH) >= VECTOR_ENSURE_LENGTH_vec->capacity)  
    {  
        if (VECTOR_ENSURE_LENGTH_vec->capacity == 0)  
        {  
            VECTOR_ENSURE_LENGTH_vec->capacity = 8;  
        }  
        else  
        {  
            VECTOR_ENSURE_LENGTH_vec->capacity *= 2;  
        }  
        VECTOR_ENSURE_LENGTH_vec->data = krealloc(  
            VECTOR_ENSURE_LENGTH_vec->data,  
            VECTOR_ENSURE_LENGTH_vec->capacity *  
                sizeof(*VECTOR_ENSURE_LENGTH_vec->data)); /* NOLINT */  
    }  
} while (0)
```

Definition at line 24 of file [vector.h](#).

7.61.1.2 VECTOR_FIND

```
#define VECTOR_FIND(  
    VEC,  
    VALUE )
```

Value:

```
{  
    __auto_type VECTOR_FIND_vec = VEC;  
    ssize_t VECTOR_FIND_result = VECTOR_INVALID_INDEX;  
    for (size_t VECTOR_FIND_i = 0;  
        VECTOR_FIND_i < VECTOR_FIND_vec->length;  
        VECTOR_FIND_i++)  
    {  
        if (VECTOR_FIND_vec->data[VECTOR_FIND_i] == (VALUE))  
        {  
            VECTOR_FIND_result = VECTOR_FIND_i;  
            break;  
        }  
    }  
    VECTOR_FIND_result;  
}
```

Definition at line 99 of file [vector.h](#).

7.61.1.3 VECTOR_FOR_EACH

```
#define VECTOR_FOR_EACH(
    VEC,
    BINDING,
    ... )
```

Value:

```
do
{
    __auto_type VECTOR_FOR_EACH_vec = VEC;
    for (size_t VECTOR_FOR_EACH_i = 0;
        VECTOR_FOR_EACH_i < VECTOR_FOR_EACH_vec->length;
        VECTOR_FOR_EACH_i++)
    {
        __auto_type BINDING =
            &VECTOR_FOR_EACH_vec->data[VECTOR_FOR_EACH_i];
        __VA_ARGS__
    }
} while (0)
```

Definition at line 126 of file [vector.h](#).

7.61.1.4 VECTOR_INIT

```
#define VECTOR_INIT
```

Value:

```
{
    0
}
```

Definition at line 11 of file [vector.h](#).

7.61.1.5 VECTOR_INSERT

```
#define VECTOR_INSERT(
    VEC,
    IDX,
    VALUE )
```

Value:

```
do
{
    __auto_type VECTOR_INSERT_vec = VEC;
    size_t VECTOR_INSERT_index = IDX;
    VECTOR_ENSURE_LENGTH(VEC, VECTOR_INSERT_vec->length);
    for (size_t VECTOR_INSERT_i = VECTOR_INSERT_vec->length;
        VECTOR_INSERT_i > VECTOR_INSERT_index;
        VECTOR_INSERT_i--)
    {
        VECTOR_INSERT_vec->data[VECTOR_INSERT_i] =
            VECTOR_INSERT_vec->data[VECTOR_INSERT_i - 1];
    }
    VECTOR_INSERT_vec->length++;
    VECTOR_INSERT_vec->data[VECTOR_INSERT_index] = VALUE;
} while (0)
```

Definition at line 55 of file [vector.h](#).

7.61.1.6 VECTOR_INVALID_INDEX

```
#define VECTOR_INVALID_INDEX (-1)
```

Definition at line 9 of file [vector.h](#).

7.61.1.7 VECTOR_ITEM

```
#define VECTOR_ITEM(  
    VEC,  
    IDX )
```

Value:

```
{  
    size_t VECTOR_ITEM_idx      = IDX;  
    __auto_type VECTOR_ITEM_vec = VEC;  
    __auto_type VECTOR_ITEM_result =  
        (typeof(*VECTOR_ITEM_vec->data)) VECTOR_INVALID_INDEX;  
    if (VECTOR_ITEM_idx < VECTOR_ITEM_vec->length)  
    {  
        VECTOR_ITEM_result = VECTOR_ITEM_vec->data[VECTOR_ITEM_idx];  
    }  
    VECTOR_ITEM_result;  
}
```

Definition at line 86 of file [vector.h](#).

7.61.1.8 VECTOR_PUSH_BACK

```
#define VECTOR_PUSH_BACK(  
    VEC,  
    VALUE )
```

Value:

```
{  
    __auto_type VECTOR_PUSH_BACK_vec = VEC;  
    VECTOR_ENSURE_LENGTH(VEC, VECTOR_PUSH_BACK_vec->length);  
    VECTOR_PUSH_BACK_vec->data[VECTOR_PUSH_BACK_vec->length++] = VALUE;  
    VECTOR_PUSH_BACK_vec->length - 1;  
}
```

Definition at line 45 of file [vector.h](#).

7.61.1.9 VECTOR_PUSH_FRONT

```
#define VECTOR_PUSH_FRONT(  
    VEC,  
    VALUE ) VECTOR_INSERT(VEC, 0, VALUE)
```

Definition at line 53 of file [vector.h](#).

7.61.1.10 VECTOR_REMOVE

```
#define VECTOR_REMOVE(
    VEC,
    IDX )
```

Value:

```
do
{
    __auto_type VECTOR_REMOVE_vec = VEC;
    for (size_t VECTOR_REMOVE_i = (IDX);
        VECTOR_REMOVE_i < VECTOR_REMOVE_vec->length - 1;
        VECTOR_REMOVE_i++)
    {
        VECTOR_REMOVE_vec->data[VECTOR_REMOVE_i] =
            VECTOR_REMOVE_vec->data[VECTOR_REMOVE_i + 1];
    }
    VECTOR_REMOVE_vec->length--;
} while (0)
```

Definition at line 72 of file [vector.h](#).

7.61.1.11 VECTOR_REMOVE_BY_VALUE

```
#define VECTOR_REMOVE_BY_VALUE(
    VEC,
    VALUE )
```

Value:

```
do
{
    __auto_type VECTOR_REMOVE_BY_VALUE_vec = VEC;
    __auto_type VECTOR_REMOVE_BY_VALUE_v = VALUE;
    size_t VECTOR_REMOVE_BY_VALUE_i =
        VECTOR_FIND(VECTOR_REMOVE_BY_VALUE_vec, VECTOR_REMOVE_BY_VALUE_v);
    VECTOR_REMOVE(VECTOR_REMOVE_BY_VALUE_vec, VECTOR_REMOVE_BY_VALUE_i);
} while (0)
```

Definition at line 116 of file [vector.h](#).

7.61.1.12 VECTOR_TYPE

```
#define VECTOR_TYPE(
    TYPE )
```

Value:

```
struct
{
    TYPE *data;
    size_t length;
    size_t capacity;
}
```

Definition at line 16 of file [vector.h](#).

7.62 vector.h

Go to the documentation of this file.

```

00001 #ifndef _VECTOR_H
00002 #define _VECTOR_H
00003
00004 #include "../mm/kmalloc.h"
00005 #include "../string.h"
00006 #include <stddef.h>
00007 #include <stdint.h>
00008
00009 #define VECTOR_INVALID_INDEX (-1)
00010
00011 #define VECTOR_INIT \
00012 { \
00013     0 \
00014 }
00015
00016 #define VECTOR_TYPE(TYPE) \
00017 struct \
00018 { \
00019     TYPE *data; \
00020     size_t length; \
00021     size_t capacity; \
00022 }
00023
00024 #define VECTOR_ENSURE_LENGTH(VEC, LENGTH)
00025 do
00026 {
00027     __auto_type VECTOR_ENSURE_LENGTH_vec = VEC;
00028     if ((LENGTH) >= VECTOR_ENSURE_LENGTH_vec->capacity)
00029     {
00030         if (VECTOR_ENSURE_LENGTH_vec->capacity == 0)
00031         {
00032             VECTOR_ENSURE_LENGTH_vec->capacity = 8;
00033         }
00034         else
00035         {
00036             VECTOR_ENSURE_LENGTH_vec->capacity *= 2;
00037         }
00038         VECTOR_ENSURE_LENGTH_vec->data = krealloc(
00039             VECTOR_ENSURE_LENGTH_vec->data,
00040             VECTOR_ENSURE_LENGTH_vec->capacity *
00041             sizeof(*VECTOR_ENSURE_LENGTH_vec->data)); /* NOLINT */
00042     }
00043 } while (0)
00044
00045 #define VECTOR_PUSH_BACK(VEC, VALUE)
00046 {
00047     __auto_type VECTOR_PUSH_BACK_vec = VEC;
00048     VECTOR_ENSURE_LENGTH(VEC, VECTOR_PUSH_BACK_vec->length);
00049     VECTOR_PUSH_BACK_vec->data[VECTOR_PUSH_BACK_vec->length++] = VALUE;
00050     VECTOR_PUSH_BACK_vec->length - 1;
00051 }
00052
00053 #define VECTOR_PUSH_FRONT(VEC, VALUE) VECTOR_INSERT(VEC, 0, VALUE)
00054
00055 #define VECTOR_INSERT(VEC, IDX, VALUE)
00056 do
00057 {
00058     __auto_type VECTOR_INSERT_vec = VEC;
00059     size_t VECTOR_INSERT_index = IDX;
00060     VECTOR_ENSURE_LENGTH(VEC, VECTOR_INSERT_vec->length);
00061     for (size_t VECTOR_INSERT_i = VECTOR_INSERT_vec->length;
00062          VECTOR_INSERT_i > VECTOR_INSERT_index;
00063          VECTOR_INSERT_i--)
00064     {
00065         VECTOR_INSERT_vec->data[VECTOR_INSERT_i] =
00066             VECTOR_INSERT_vec->data[VECTOR_INSERT_i - 1];
00067     }
00068     VECTOR_INSERT_vec->length++;
00069     VECTOR_INSERT_vec->data[VECTOR_INSERT_index] = VALUE;
00070 } while (0)
00071
00072 #define VECTOR_REMOVE(VEC, IDX)
00073 do
00074 {
00075     __auto_type VECTOR_REMOVE_vec = VEC;
00076     for (size_t VECTOR_REMOVE_i = (IDX);
00077          VECTOR_REMOVE_i < VECTOR_REMOVE_vec->length - 1;
00078          VECTOR_REMOVE_i++)
00079     {
00080         VECTOR_REMOVE_vec->data[VECTOR_REMOVE_i] =
00081             VECTOR_REMOVE_vec->data[VECTOR_REMOVE_i + 1];
00082     }

```



```

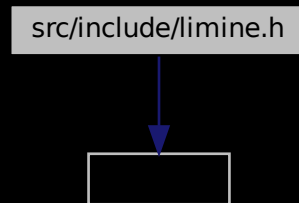
00083 VECTOR_REMOVE_vec->length--;
00084 } while (0)
00085
00086 #define VECTOR_ITEM(VEC, IDX)
00087 ({
00088     size_t VECTOR_ITEM_idx = IDX;
00089     __auto_type VECTOR_ITEM_vec = VEC;
00090     __auto_type VECTOR_ITEM_result =
00091     (typeof(*VECTOR_ITEM_vec->data)) VECTOR_INVALID_INDEX;
00092     if (VECTOR_ITEM_idx < VECTOR_ITEM_vec->length)
00093     {
00094         VECTOR_ITEM_result = VECTOR_ITEM_vec->data[VECTOR_ITEM_idx];
00095     }
00096     VECTOR_ITEM_result;
00097 })
00098
00099 #define VECTOR_FIND(VEC, VALUE)
00100 ({
00101     __auto_type VECTOR_FIND_vec = VEC;
00102     ssize_t VECTOR_FIND_result = VECTOR_INVALID_INDEX;
00103     for (size_t VECTOR_FIND_i = 0;
00104          VECTOR_FIND_i < VECTOR_FIND_vec->length;
00105          VECTOR_FIND_i++)
00106     {
00107         if (VECTOR_FIND_vec->data[VECTOR_FIND_i] == (VALUE))
00108         {
00109             VECTOR_FIND_result = VECTOR_FIND_i;
00110             break;
00111         }
00112     }
00113     VECTOR_FIND_result;
00114 })
00115
00116 #define VECTOR_REMOVE_BY_VALUE(VEC, VALUE)
00117 do
00118 {
00119     __auto_type VECTOR_REMOVE_BY_VALUE_vec = VEC;
00120     __auto_type VECTOR_REMOVE_BY_VALUE_v = VALUE;
00121     size_t VECTOR_REMOVE_BY_VALUE_i =
00122     VECTOR_FIND(VECTOR_REMOVE_BY_VALUE_vec, VECTOR_REMOVE_BY_VALUE_v);
00123     VECTOR_REMOVE(VECTOR_REMOVE_BY_VALUE_vec, VECTOR_REMOVE_BY_VALUE_i);
00124 } while (0)
00125
00126 #define VECTOR_FOR_EACH(VEC, BINDING, ...)
00127 do
00128 {
00129     __auto_type VECTOR_FOR_EACH_vec = VEC;
00130     for (size_t VECTOR_FOR_EACH_i = 0;
00131          VECTOR_FOR_EACH_i < VECTOR_FOR_EACH_vec->length;
00132          VECTOR_FOR_EACH_i++)
00133     {
00134         __auto_type BINDING =
00135         &VECTOR_FOR_EACH_vec->data[VECTOR_FOR_EACH_i];
00136         __VA_ARGS__
00137     }
00138 } while (0)
00139
00140 #endif

```

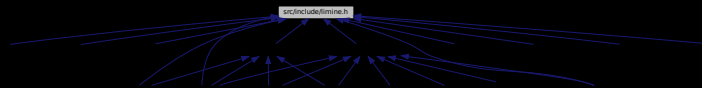
7.63 src/include/limine.h File Reference

```
#include <stdint.h>
```

Include dependency graph for limine.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `limine_uuid`
- struct `limine_file`
- struct `limine_bootloader_info_response`
- struct `limine_bootloader_info_request`
- struct `limine_stack_size_response`
- struct `limine_stack_size_request`
- struct `limine_hhdm_response`
- struct `limine_hhdm_request`
- struct `limine_framebuffer`
- struct `limine_framebuffer_response`
- struct `limine_framebuffer_request`
- struct `limine_terminal`
- struct `limine_terminal_response`
- struct `limine_terminal_request`
- struct `limine_5_level_paging_response`
- struct `limine_5_level_paging_request`
- struct `limine_smp_request`
- struct `limine_memmap_entry`
- struct `limine_memmap_response`
- struct `limine_memmap_request`
- struct `limine_entry_point_response`
- struct `limine_entry_point_request`
- struct `limine_kernel_file_response`

- struct `limine_kernel_file_request`
- struct `limine_module_response`
- struct `limine_module_request`
- struct `limine_rsdp_response`
- struct `limine_rsdp_request`
- struct `limine_smbios_response`
- struct `limine_smbios_request`
- struct `limine_efi_system_table_response`
- struct `limine_efi_system_table_request`
- struct `limine_boot_time_response`
- struct `limine_boot_time_request`
- struct `limine_kernel_address_response`
- struct `limine_kernel_address_request`
- struct `limine_dtb_response`
- struct `limine_dtb_request`

Macros

- #define `LIMINE_PTR(TYPE) TYPE`
- #define `LIMINE_COMMON_MAGIC 0xc7b1dd30df4c8b88, 0x0a82e883a194f07b`
- #define `LIMINE_MEDIA_TYPE_GENERIC 0`
- #define `LIMINE_MEDIA_TYPE_OPTICAL 1`
- #define `LIMINE_MEDIA_TYPE_TFTP 2`
- #define `LIMINE_BOOTLOADER_INFO_REQUEST`
- #define `LIMINE_STACK_SIZE_REQUEST`
- #define `LIMINE_HHDM_REQUEST`
- #define `LIMINE_FRAMEBUFFER_REQUEST`
- #define `LIMINE_FRAMEBUFFER_RGB 1`
- #define `LIMINE_TERMINAL_REQUEST`
- #define `LIMINE_TERMINAL_CB_DEC 10`
- #define `LIMINE_TERMINAL_CB_BELL 20`
- #define `LIMINE_TERMINAL_CB_PRIVATE_ID 30`
- #define `LIMINE_TERMINAL_CB_STATUS_REPORT 40`
- #define `LIMINE_TERMINAL_CB_POS_REPORT 50`
- #define `LIMINE_TERMINAL_CB_KBD_LEDS 60`
- #define `LIMINE_TERMINAL_CB_MODE 70`
- #define `LIMINE_TERMINAL_CB_LINUX 80`
- #define `LIMINE_TERMINAL_CTX_SIZE ((uint64_t) (-1))`
- #define `LIMINE_TERMINAL_CTX_SAVE ((uint64_t) (-2))`
- #define `LIMINE_TERMINAL_CTX_RESTORE ((uint64_t) (-3))`
- #define `LIMINE_TERMINAL_FULL_REFRESH ((uint64_t) (-4))`
- #define `LIMINE_5_LEVEL_PAGING_REQUEST`
- #define `LIMINE_SMP_REQUEST`
- #define `LIMINE_MEMMAP_REQUEST`
- #define `LIMINE_MEMMAP_USABLE 0`
- #define `LIMINE_MEMMAP_RESERVED 1`
- #define `LIMINE_MEMMAP_ACPI_RECLAIMABLE 2`
- #define `LIMINE_MEMMAP_ACPI_NVS 3`
- #define `LIMINE_MEMMAP_BAD_MEMORY 4`
- #define `LIMINE_MEMMAP_BOOTLOADER_RECLAIMABLE 5`
- #define `LIMINE_MEMMAP_KERNEL_AND_MODULES 6`
- #define `LIMINE_MEMMAP_FRAMEBUFFER 7`
- #define `LIMINE_ENTRY_POINT_REQUEST`

- `#define LIMINE_KERNEL_FILE_REQUEST`
- `#define LIMINE_MODULE_REQUEST`
- `#define LIMINE_RSDP_REQUEST`
- `#define LIMINE_SMBIOS_REQUEST`
- `#define LIMINE_EFI_SYSTEM_TABLE_REQUEST`
- `#define LIMINE_BOOT_TIME_REQUEST`
- `#define LIMINE_KERNEL_ADDRESS_REQUEST`
- `#define LIMINE_DTB_REQUEST`

Typedefs

- `typedef void(* limine_terminal_write) (struct limine_terminal *, const char *, uint64_t)`
- `typedef void(* limine_terminal_callback) (struct limine_terminal *, uint64_t, uint64_t, uint64_t, uint64_t)`
- `typedef void(* limine_goto_address) (struct limine_smp_info *)`
- `typedef void(* limine_entry_point) (void)`

7.63.1 Macro Definition Documentation

7.63.1.1 LIMINE_5_LEVEL_PAGING_REQUEST

```
#define LIMINE_5_LEVEL_PAGING_REQUEST
```

Value:

```
{
    LIMINE_COMMON_MAGIC, 0x94469551da9b3192, 0xebe5e86db7382888 \
}
```

Definition at line 208 of file `limine.h`.

7.63.1.2 LIMINE_BOOT_TIME_REQUEST

```
#define LIMINE_BOOT_TIME_REQUEST
```

Value:

```
{
    LIMINE_COMMON_MAGIC, 0x502746e184c088aa, 0xfbc5ec83e6327893 \
}
```

Definition at line 454 of file `limine.h`.

7.63.1.3 LIMINE_BOOTLOADER_INFO_REQUEST

```
#define LIMINE_BOOTLOADER_INFO_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0xf55038d8e2a1202f, 0x279426fcf5f59740 \  
}
```

Definition at line 53 of file [limine.h](#).

7.63.1.4 LIMINE_COMMON_MAGIC

```
#define LIMINE_COMMON_MAGIC 0xc7b1dd30df4c8b88, 0x0a82e883a194f07b
```

Definition at line 19 of file [limine.h](#).

7.63.1.5 LIMINE_DTB_REQUEST

```
#define LIMINE_DTB_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0xb40ddb48fb54bac7, 0x545081493f81ffb7 \  
}
```

Definition at line 495 of file [limine.h](#).

7.63.1.6 LIMINE_EFI_SYSTEM_TABLE_REQUEST

```
#define LIMINE_EFI_SYSTEM_TABLE_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0x5ceba5163eaaf6d6, 0x0a6981610cf65fcc \  
}
```

Definition at line 434 of file [limine.h](#).

7.63.1.7 LIMINE_ENTRY_POINT_REQUEST

```
#define LIMINE_ENTRY_POINT_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0x13d86c035a1cd3e1, 0x2b0caa89d8f3026a \  
}
```

Definition at line 330 of file [limine.h](#).

7.63.1.8 LIMINE_FRAMEBUFFER_REQUEST

```
#define LIMINE_FRAMEBUFFER_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0x9d5827dcd881dd75, 0xa3148604f6fab11b \
```

Definition at line 114 of file [limine.h](#).

7.63.1.9 LIMINE_FRAMEBUFFER_RGB

```
#define LIMINE_FRAMEBUFFER_RGB 1
```

Definition at line 119 of file [limine.h](#).

7.63.1.10 LIMINE_HHDM_REQUEST

```
#define LIMINE_HHDM_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0x48dcf1cb8ad2b852, 0x63984e959a98244b \
```

Definition at line 94 of file [limine.h](#).

7.63.1.11 LIMINE_KERNEL_ADDRESS_REQUEST

```
#define LIMINE_KERNEL_ADDRESS_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0x71ba76863cc55f63, 0xb2644a48c516a487 \
```

Definition at line 474 of file [limine.h](#).

7.63.1.12 LIMINE_KERNEL_FILE_REQUEST

```
#define LIMINE_KERNEL_FILE_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0xad97e90e83f1ed67, 0x31eb5d1c5ff23b69 \
```

Definition at line 352 of file [limine.h](#).

7.63.1.13 LIMINE_MEDIA_TYPE_GENERIC

```
#define LIMINE_MEDIA_TYPE_GENERIC 0
```

Definition at line 29 of file [limine.h](#).

7.63.1.14 LIMINE_MEDIA_TYPE_OPTICAL

```
#define LIMINE_MEDIA_TYPE_OPTICAL 1
```

Definition at line 30 of file [limine.h](#).

7.63.1.15 LIMINE_MEDIA_TYPE_TFTP

```
#define LIMINE_MEDIA_TYPE_TFTP 2
```

Definition at line 31 of file [limine.h](#).

7.63.1.16 LIMINE_MEMMAP_ACPI_NVS

```
#define LIMINE_MEMMAP_ACPI_NVS 3
```

Definition at line 301 of file [limine.h](#).

7.63.1.17 LIMINE_MEMMAP_ACPI_RECLAIMABLE

```
#define LIMINE_MEMMAP_ACPI_RECLAIMABLE 2
```

Definition at line 300 of file [limine.h](#).

7.63.1.18 LIMINE_MEMMAP_BAD_MEMORY

```
#define LIMINE_MEMMAP_BAD_MEMORY 4
```

Definition at line 302 of file [limine.h](#).

7.63.1.19 LIMINE_MEMMAP_BOOTLOADER_RECLAIMABLE

```
#define LIMINE_MEMMAP_BOOTLOADER_RECLAIMABLE 5
```

Definition at line 303 of file [limine.h](#).

7.63.1.20 LIMINE_MEMMAP_FRAMEBUFFER

```
#define LIMINE_MEMMAP_FRAMEBUFFER 7
```

Definition at line 305 of file [limine.h](#).

7.63.1.21 LIMINE_MEMMAP_KERNEL_AND_MODULES

```
#define LIMINE_MEMMAP_KERNEL_AND_MODULES 6
```

Definition at line 304 of file [limine.h](#).

7.63.1.22 LIMINE_MEMMAP_REQUEST

```
#define LIMINE_MEMMAP_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0x67cf3d9d378a806f, 0xe304acdfc50c3c62  
}
```

Definition at line 293 of file [limine.h](#).

7.63.1.23 LIMINE_MEMMAP_RESERVED

```
#define LIMINE_MEMMAP_RESERVED 1
```

Definition at line 299 of file [limine.h](#).

7.63.1.24 LIMINE_MEMMAP_USABLE

```
#define LIMINE_MEMMAP_USABLE 0
```

Definition at line 298 of file [limine.h](#).

7.63.1.25 LIMINE_MODULE_REQUEST

```
#define LIMINE_MODULE_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0x3e7e279702be32af, 0xca1c4f3bd1280cee \  
}
```

Definition at line 372 of file [limine.h](#).

7.63.1.26 LIMINE_PTR

```
#define LIMINE_PTR(  
    TYPE ) TYPE
```

Definition at line 16 of file [limine.h](#).

7.63.1.27 LIMINE_RSDP_REQUEST

```
#define LIMINE_RSDP_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0xc5e77b6b397e7b43, 0x27637845accdcf3c \  
}
```

Definition at line 393 of file [limine.h](#).

7.63.1.28 LIMINE_SMBIOS_REQUEST

```
#define LIMINE_SMBIOS_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0x9e9046f11e095391, 0xaa4a520fefbde5ee \  
}
```

Definition at line 413 of file [limine.h](#).

7.63.1.29 LIMINE_SMP_REQUEST

```
#define LIMINE_SMP_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0x95a67b819a1b857e, 0xa0b61b723b6a73e0 \  
}
```

Definition at line 227 of file [limine.h](#).

7.63.1.30 LIMINE_STACK_SIZE_REQUEST

```
#define LIMINE_STACK_SIZE_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0x224ef0460a8e8926, 0xe1cb0fc25f46ea3d \  
}
```

Definition at line 74 of file [limine.h](#).

7.63.1.31 LIMINE_TERMINAL_CB_BELL

```
#define LIMINE_TERMINAL_CB_BELL 20
```

Definition at line 162 of file [limine.h](#).

7.63.1.32 LIMINE_TERMINAL_CB_DEC

```
#define LIMINE_TERMINAL_CB_DEC 10
```

Definition at line 161 of file [limine.h](#).

7.63.1.33 LIMINE_TERMINAL_CB_KBD_LEDS

```
#define LIMINE_TERMINAL_CB_KBD_LEDS 60
```

Definition at line 166 of file [limine.h](#).

7.63.1.34 LIMINE_TERMINAL_CB_LINUX

```
#define LIMINE_TERMINAL_CB_LINUX 80
```

Definition at line 168 of file [limine.h](#).

7.63.1.35 LIMINE_TERMINAL_CB_MODE

```
#define LIMINE_TERMINAL_CB_MODE 70
```

Definition at line 167 of file [limine.h](#).

7.63.1.36 LIMINE_TERMINAL_CB_POS_REPORT

```
#define LIMINE_TERMINAL_CB_POS_REPORT 50
```

Definition at line 165 of file [limine.h](#).

7.63.1.37 LIMINE_TERMINAL_CB_PRIVATE_ID

```
#define LIMINE_TERMINAL_CB_PRIVATE_ID 30
```

Definition at line 163 of file [limine.h](#).

7.63.1.38 LIMINE_TERMINAL_CB_STATUS_REPORT

```
#define LIMINE_TERMINAL_CB_STATUS_REPORT 40
```

Definition at line 164 of file [limine.h](#).

7.63.1.39 LIMINE_TERMINAL_CTX_RESTORE

```
#define LIMINE_TERMINAL_CTX_RESTORE ((uint64_t) (-3))
```

Definition at line 172 of file [limine.h](#).

7.63.1.40 LIMINE_TERMINAL_CTX_SAVE

```
#define LIMINE_TERMINAL_CTX_SAVE ((uint64_t) (-2))
```

Definition at line 171 of file [limine.h](#).

7.63.1.41 LIMINE_TERMINAL_CTX_SIZE

```
#define LIMINE_TERMINAL_CTX_SIZE ((uint64_t) (-1))
```

Definition at line 170 of file [limine.h](#).

7.63.1.42 LIMINE_TERMINAL_FULL_REFRESH

```
#define LIMINE_TERMINAL_FULL_REFRESH ((uint64_t) (-4))
```

Definition at line 173 of file [limine.h](#).

7.63.1.43 LIMINE_TERMINAL_REQUEST

```
#define LIMINE_TERMINAL_REQUEST
```

Value:

```
{  
    LIMINE_COMMON_MAGIC, 0xc8ac59310c2b0844, 0xa68d0c7265d38878 \
```

Definition at line 156 of file [limine.h](#).

7.63.2 Typedef Documentation

7.63.2.1 limine_entry_point

```
typedef void(* limine_entry_point) (void)
```

Definition at line 335 of file [limine.h](#).

7.63.2.2 limine_goto_address

```
typedef void(* limine_goto_address) (struct limine_smp_info *)
```

Definition at line 234 of file [limine.h](#).

7.63.2.3 limine_terminal_callback

```
typedef void(* limine_terminal_callback) (struct limine\_terminal *, uint64_t, uint64_t, uint64_t↵  
_t, uint64_t)
```

Definition at line 180 of file [limine.h](#).

7.63.2.4 limine_terminal_write

```
typedef void(* limine_terminal_write) (struct limine_terminal *, const char *, uint64_t)
```

Definition at line 177 of file [limine.h](#).

7.64 limine.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _LIMINE_H
00002 #define _LIMINE_H 1
00003
00004 #ifdef __cplusplus
00005 extern "C"
00006 {
00007 #endif
00008
00009 #include <stdint.h>
00010
00011     /* Misc */
00012
00013 #ifdef LIMINE_NO_POINTERS
00014 #define LIMINE_PTR(TYPE) uint64_t
00015 #else
00016 #define LIMINE_PTR(TYPE) TYPE
00017 #endif
00018
00019 #define LIMINE_COMMON_MAGIC 0xc7b1dd30df4c8b88, 0x0a82e883a194f07b
00020
00021     struct limine_uuid
00022     {
00023         uint32_t a;
00024         uint16_t b;
00025         uint16_t c;
00026         uint8_t d[8];
00027     };
00028
00029 #define LIMINE_MEDIA_TYPE_GENERIC 0
00030 #define LIMINE_MEDIA_TYPE_OPTICAL 1
00031 #define LIMINE_MEDIA_TYPE_TFTP 2
00032
00033     struct limine_file
00034     {
00035         uint64_t revision;
00036         LIMINE_PTR(void *) address;
00037         uint64_t size;
00038         LIMINE_PTR(char *) path;
00039         LIMINE_PTR(char *) cmdline;
00040         uint32_t media_type;
00041         uint32_t unused;
00042         uint32_t tftp_ip;
00043         uint32_t tftp_port;
00044         uint32_t partition_index;
00045         uint32_t mbr_disk_id;
00046         struct limine_uuid gpt_disk_uuid;
00047         struct limine_uuid gpt_part_uuid;
00048         struct limine_uuid part_uuid;
00049     };
00050
00051     /* Boot info */
00052
00053 #define LIMINE_BOOTLOADER_INFO_REQUEST
00054 {
00055     LIMINE_COMMON_MAGIC, 0xf55038d8e2a1202f, 0x279426fcf5f59740 \
00056 }
00057
00058     struct limine_bootloader_info_response
00059     {
00060         uint64_t revision;
00061         LIMINE_PTR(char *) name;
00062         LIMINE_PTR(char *) version;
00063     };
00064
00065     struct limine_bootloader_info_request
00066     {
00067         uint64_t id[4];
00068         uint64_t revision;
00069         LIMINE_PTR(struct limine_bootloader_info_response *) response;
```

```

00070     };
00071
00072     /* Stack size */
00073
00074     #define LIMINE_STACK_SIZE_REQUEST
00075     {
00076     LIMINE_COMMON_MAGIC, 0x224ef0460a8e8926, 0xe1cb0fc25f46ea3d \
00077     }
00078
00079     struct limine_stack_size_response
00080     {
00081         uint64_t revision;
00082     };
00083
00084     struct limine_stack_size_request
00085     {
00086         uint64_t id[4];
00087         uint64_t revision;
00088         LIMINE_PTR(struct limine_stack_size_response *) response;
00089         uint64_t stack_size;
00090     };
00091
00092     /* HHDM */
00093
00094     #define LIMINE_HHDM_REQUEST
00095     {
00096     LIMINE_COMMON_MAGIC, 0x48dcf1cb8ad2b852, 0x63984e959a98244b \
00097     }
00098
00099     struct limine_hhdm_response
00100     {
00101         uint64_t revision;
00102         uint64_t offset;
00103     };
00104
00105     struct limine_hhdm_request
00106     {
00107         uint64_t id[4];
00108         uint64_t revision;
00109         LIMINE_PTR(struct limine_hhdm_response *) response;
00110     };
00111
00112     /* Framebuffer */
00113
00114     #define LIMINE_FRAMEBUFFER_REQUEST
00115     {
00116     LIMINE_COMMON_MAGIC, 0x9d5827dcd881dd75, 0xa3148604f6fab11b \
00117     }
00118
00119     #define LIMINE_FRAMEBUFFER_RGB 1
00120
00121     struct limine_framebuffer
00122     {
00123         LIMINE_PTR(void *) address;
00124         uint64_t width;
00125         uint64_t height;
00126         uint64_t pitch;
00127         uint16_t bpp;
00128         uint8_t memory_model;
00129         uint8_t red_mask_size;
00130         uint8_t red_mask_shift;
00131         uint8_t green_mask_size;
00132         uint8_t green_mask_shift;
00133         uint8_t blue_mask_size;
00134         uint8_t blue_mask_shift;
00135         uint8_t unused[7];
00136         uint64_t edid_size;
00137         LIMINE_PTR(void *) edid;
00138     };
00139
00140     struct limine_framebuffer_response
00141     {
00142         uint64_t revision;
00143         uint64_t framebuffer_count;
00144         LIMINE_PTR(struct limine_framebuffer **) framebuffers;
00145     };
00146
00147     struct limine_framebuffer_request
00148     {
00149         uint64_t id[4];
00150         uint64_t revision;
00151         LIMINE_PTR(struct limine_framebuffer_response *) response;
00152     };
00153
00154     /* Terminal */
00155
00156     #define LIMINE_TERMINAL_REQUEST

```

```

00157 {
00158     LIMINE_COMMON_MAGIC, 0xc8ac59310c2b0844, 0xa68d0c7265d38878 \
00159 }
00160
00161 #define LIMINE_TERMINAL_CB_DEC 10
00162 #define LIMINE_TERMINAL_CB_BELL 20
00163 #define LIMINE_TERMINAL_CB_PRIVATE_ID 30
00164 #define LIMINE_TERMINAL_CB_STATUS_REPORT 40
00165 #define LIMINE_TERMINAL_CB_POS_REPORT 50
00166 #define LIMINE_TERMINAL_CB_KBD_LEDS 60
00167 #define LIMINE_TERMINAL_CB_MODE 70
00168 #define LIMINE_TERMINAL_CB_LINUX 80
00169
00170 #define LIMINE_TERMINAL_CTX_SIZE ((uint64_t) (-1))
00171 #define LIMINE_TERMINAL_CTX_SAVE ((uint64_t) (-2))
00172 #define LIMINE_TERMINAL_CTX_RESTORE ((uint64_t) (-3))
00173 #define LIMINE_TERMINAL_FULL_REFRESH ((uint64_t) (-4))
00174
00175     struct limine_terminal;
00176
00177     typedef void (*limine_terminal_write)(struct limine_terminal *,
00178                                         const char *,
00179                                         uint64_t);
00180
00181     typedef void (*limine_terminal_callback)(
00182         struct limine_terminal *, uint64_t, uint64_t, uint64_t, uint64_t);
00183
00184     struct limine_terminal
00185     {
00186         uint64_t columns;
00187         uint64_t rows;
00188         LIMINE_PTR(struct limine_framebuffer *) framebuffer;
00189     };
00190
00191     struct limine_terminal_response
00192     {
00193         uint64_t revision;
00194         uint64_t terminal_count;
00195         LIMINE_PTR(struct limine_terminal **) terminals;
00196         LIMINE_PTR(limine_terminal_write) write;
00197     };
00198
00199     struct limine_terminal_request
00200     {
00201         uint64_t id[4];
00202         uint64_t revision;
00203         LIMINE_PTR(struct limine_terminal_response *) response;
00204         LIMINE_PTR(limine_terminal_callback) callback;
00205     };
00206
00207     /* 5-level paging */
00208 #define LIMINE_5_LEVEL_PAGING_REQUEST
00209 {
00210     LIMINE_COMMON_MAGIC, 0x94469551da9b3192, 0xeb5e86db7382888 \
00211 }
00212
00213     struct limine_5_level_paging_response
00214     {
00215         uint64_t revision;
00216     };
00217
00218     struct limine_5_level_paging_request
00219     {
00220         uint64_t id[4];
00221         uint64_t revision;
00222         LIMINE_PTR(struct limine_5_level_paging_response *) response;
00223     };
00224
00225     /* SMP */
00226 #define LIMINE_SMP_REQUEST
00227 {
00228     LIMINE_COMMON_MAGIC, 0x95a67b819a1b857e, 0xa0b61b723b6a73e0 \
00229 }
00230
00231     struct limine_smp_info;
00232
00233     typedef void (*limine_goto_address)(struct limine_smp_info *);
00234
00235 #if defined(__x86_64__) || defined(__i386__)
00236 #define LIMINE_SMP_X2APIC (1 << 0)
00237
00238     struct limine_smp_info
00239     {
00240         uint32_t processor_id;
00241         uint32_t lapic_id;

```

```

00244         uint64_t reserved;
00245         LIMINE_PTR(limine_goto_address) goto_address;
00246         uint64_t extra_argument;
00247     };
00248
00249     struct limine_smp_response
00250     {
00251         uint64_t revision;
00252         uint32_t flags;
00253         uint32_t bsp_lapic_id;
00254         uint64_t cpu_count;
00255         LIMINE_PTR(struct limine_smp_info **) cpus;
00256     };
00257
00258 #elif defined(__aarch64__)
00259
00260 struct limine_smp_info
00261 {
00262     uint32_t processor_id;
00263     uint32_t gic_iface_no;
00264     uint64_t mpidr;
00265     uint64_t reserved;
00266     LIMINE_PTR(limine_goto_address) goto_address;
00267     uint64_t extra_argument;
00268 };
00269
00270 struct limine_smp_response
00271 {
00272     uint64_t revision;
00273     uint32_t flags;
00274     uint64_t bsp_mpidr;
00275     uint64_t cpu_count;
00276     LIMINE_PTR(struct limine_smp_info **) cpus;
00277 };
00278
00279 #else
00280 #error Unknown architecture
00281 #endif
00282
00283 struct limine_smp_request
00284 {
00285     uint64_t id[4];
00286     uint64_t revision;
00287     LIMINE_PTR(struct limine_smp_response *) response;
00288     uint64_t flags;
00289 };
00290
00291 /* Memory map */
00292
00293 #define LIMINE_MEMMAP_REQUEST
00294 {
00295     LIMINE_COMMON_MAGIC, 0x67cf3d9d378a806f, 0xe304acd5c50c3c62 \
00296 }
00297
00298 #define LIMINE_MEMMAP_USABLE 0
00299 #define LIMINE_MEMMAP_RESERVED 1
00300 #define LIMINE_MEMMAP_ACPI_RECLAIMABLE 2
00301 #define LIMINE_MEMMAP_ACPI_NVS 3
00302 #define LIMINE_MEMMAP_BAD_MEMORY 4
00303 #define LIMINE_MEMMAP_BOOTLOADER_RECLAIMABLE 5
00304 #define LIMINE_MEMMAP_KERNEL_AND_MODULES 6
00305 #define LIMINE_MEMMAP_FRAMEBUFFER 7
00306
00307 struct limine_memmap_entry
00308 {
00309     uint64_t base;
00310     uint64_t length;
00311     uint64_t type;
00312 };
00313
00314 struct limine_memmap_response
00315 {
00316     uint64_t revision;
00317     uint64_t entry_count;
00318     LIMINE_PTR(struct limine_memmap_entry **) entries;
00319 };
00320
00321 struct limine_memmap_request
00322 {
00323     uint64_t id[4];
00324     uint64_t revision;
00325     LIMINE_PTR(struct limine_memmap_response *) response;
00326 };
00327
00328 /* Entry point */
00329
00330 #define LIMINE_ENTRY_POINT_REQUEST

```



```

00331 {
00332 LIMINE_COMMON_MAGIC, 0x13d86c035a1cd3e1, 0x2b0caa89d8f3026a \
00333 }
00334
00335     typedef void (*limine_entry_point)(void);
00336
00337     struct limine_entry_point_response
00338     {
00339         uint64_t revision;
00340     };
00341
00342     struct limine_entry_point_request
00343     {
00344         uint64_t id[4];
00345         uint64_t revision;
00346         LIMINE_PTR(struct limine_entry_point_response *) response;
00347         LIMINE_PTR(limine_entry_point) entry;
00348     };
00349
00350     /* Kernel File */
00351
00352 #define LIMINE_KERNEL_FILE_REQUEST
00353 {
00354 LIMINE_COMMON_MAGIC, 0xad97e90e83f1ed67, 0x31eb5dlc5ff23b69 \
00355 }
00356
00357     struct limine_kernel_file_response
00358     {
00359         uint64_t revision;
00360         LIMINE_PTR(struct limine_file *) kernel_file;
00361     };
00362
00363     struct limine_kernel_file_request
00364     {
00365         uint64_t id[4];
00366         uint64_t revision;
00367         LIMINE_PTR(struct limine_kernel_file_response *) response;
00368     };
00369
00370     /* Module */
00371
00372 #define LIMINE_MODULE_REQUEST
00373 {
00374 LIMINE_COMMON_MAGIC, 0x3e7e279702be32af, 0xc4f3bd1280cee \
00375 }
00376
00377     struct limine_module_response
00378     {
00379         uint64_t revision;
00380         uint64_t module_count;
00381         LIMINE_PTR(struct limine_file **) modules;
00382     };
00383
00384     struct limine_module_request
00385     {
00386         uint64_t id[4];
00387         uint64_t revision;
00388         LIMINE_PTR(struct limine_module_response *) response;
00389     };
00390
00391     /* RSDP */
00392
00393 #define LIMINE_RSDP_REQUEST
00394 {
00395 LIMINE_COMMON_MAGIC, 0xc5e77b6b397e7b43, 0x27637845accdef3c \
00396 }
00397
00398     struct limine_rsdp_response
00399     {
00400         uint64_t revision;
00401         LIMINE_PTR(void *) address;
00402     };
00403
00404     struct limine_rsdp_request
00405     {
00406         uint64_t id[4];
00407         uint64_t revision;
00408         LIMINE_PTR(struct limine_rsdp_response *) response;
00409     };
00410
00411     /* SMBIOS */
00412
00413 #define LIMINE_SMBIOS_REQUEST
00414 {
00415 LIMINE_COMMON_MAGIC, 0x9e9046f11e095391, 0xaa4a520fefbde5ee \
00416 }
00417

```

```

00418     struct limine_smbios_response
00419     {
00420         uint64_t revision;
00421         LIMINE_PTR(void *) entry_32;
00422         LIMINE_PTR(void *) entry_64;
00423     };
00424
00425     struct limine_smbios_request
00426     {
00427         uint64_t id[4];
00428         uint64_t revision;
00429         LIMINE_PTR(struct limine_smbios_response *) response;
00430     };
00431
00432     /* EFI system table */
00433
00434     #define LIMINE_EFI_SYSTEM_TABLE_REQUEST
00435     {
00436         LIMINE_COMMON_MAGIC, 0x5ceba5163eaaf6d6, 0x0a6981610cf65fcc \
00437     }
00438
00439     struct limine_efi_system_table_response
00440     {
00441         uint64_t revision;
00442         LIMINE_PTR(void *) address;
00443     };
00444
00445     struct limine_efi_system_table_request
00446     {
00447         uint64_t id[4];
00448         uint64_t revision;
00449         LIMINE_PTR(struct limine_efi_system_table_response *) response;
00450     };
00451
00452     /* Boot time */
00453
00454     #define LIMINE_BOOT_TIME_REQUEST
00455     {
00456         LIMINE_COMMON_MAGIC, 0x502746e184c088aa, 0xfbc5ec83e6327893 \
00457     }
00458
00459     struct limine_boot_time_response
00460     {
00461         uint64_t revision;
00462         int64_t boot_time;
00463     };
00464
00465     struct limine_boot_time_request
00466     {
00467         uint64_t id[4];
00468         uint64_t revision;
00469         LIMINE_PTR(struct limine_boot_time_response *) response;
00470     };
00471
00472     /* Kernel address */
00473
00474     #define LIMINE_KERNEL_ADDRESS_REQUEST
00475     {
00476         LIMINE_COMMON_MAGIC, 0x71ba76863cc55f63, 0xb2644a48c516a487 \
00477     }
00478
00479     struct limine_kernel_address_response
00480     {
00481         uint64_t revision;
00482         uint64_t physical_base;
00483         uint64_t virtual_base;
00484     };
00485
00486     struct limine_kernel_address_request
00487     {
00488         uint64_t id[4];
00489         uint64_t revision;
00490         LIMINE_PTR(struct limine_kernel_address_response *) response;
00491     };
00492
00493     /* Device Tree Blob */
00494
00495     #define LIMINE_DTB_REQUEST
00496     {
00497         LIMINE_COMMON_MAGIC, 0xb40ddb48fb54bac7, 0x545081493f81ffb7 \
00498     }
00499
00500     struct limine_dtb_response
00501     {
00502         uint64_t revision;
00503         LIMINE_PTR(void *) dtb_ptr;
00504     };

```

```

00505
00506     struct limine_dtb_request
00507     {
00508         uint64_t id[4];
00509         uint64_t revision;
00510         LIMINE_PTR(struct limine_dtb_response *) response;
00511     };
00512
00513 #ifdef __cplusplus
00514 }
00515 #endif
00516
00517 #endif

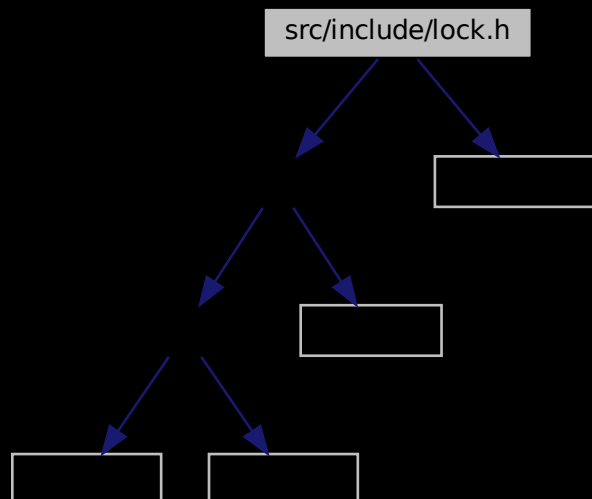
```

7.65 src/include/lock.h File Reference

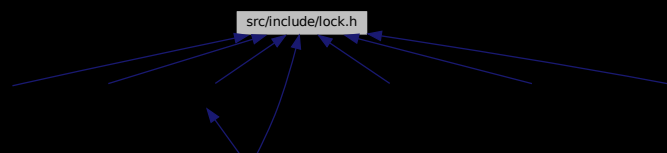
```

#include "global_defs.h"
#include <stdbool.h>
Include dependency graph for lock.h:

```



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `smartlock`

Macros

- `#define _LOCK_C_H`
- `#define SPINLOCK_INIT 0`
- `#define SMARTLOCK_INIT`

Typedefs

- `typedef int spinlock_t`

Functions

- `void smartlock_acquire (struct smartlock *smartlock)`
- `void smartlock_release (struct smartlock *smartlock)`
- `static bool spinlock_test_and_acq (spinlock_t *lock)`
- `static void spinlock_acquire (spinlock_t *lock)`
- `static void spinlock_release (spinlock_t *lock)`

7.65.1 Macro Definition Documentation

7.65.1.1 _LOCK_C_H

```
#define _LOCK_C_H
```

Definition at line 3 of file `lock.h`.

7.65.1.2 SMARTLOCK_INIT

```
#define SMARTLOCK_INIT
```

Value:

```
{  
    0, NULL  
}
```

Definition at line 18 of file `lock.h`.

7.65.1.3 SPINLOCK_INIT

```
#define SPINLOCK_INIT 0
```

Definition at line 10 of file `lock.h`.

7.65.2 Typedef Documentation

7.65.2.1 spinlock_t

```
typedef int spinlock_t
```

Definition at line 8 of file lock.h.

7.65.3 Function Documentation

7.65.3.1 smartlock_acquire()

```
void smartlock_acquire (  
    struct smartlock * smartlock )
```

7.65.3.2 smartlock_release()

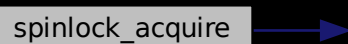
```
void smartlock_release (  
    struct smartlock * smartlock )
```

7.65.3.3 spinlock_acquire()

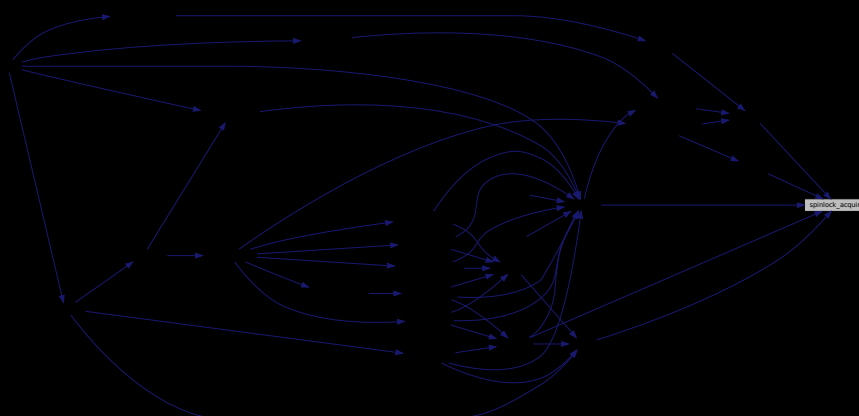
```
static void spinlock_acquire (  
    spinlock_t * lock ) [inline], [static]
```

Definition at line 31 of file lock.h.

Here is the call graph for this function:



Here is the caller graph for this function:

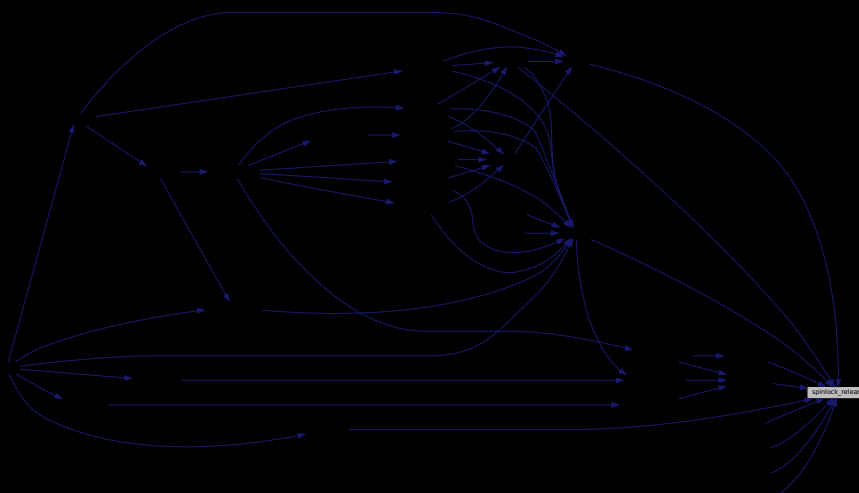


7.65.3.4 `spinlock_release()`

```
static void spinlock_release (
    spinlock_t * lock ) [inline], [static]
```

Definition at line 43 of file `lock.h`.

Here is the caller graph for this function:

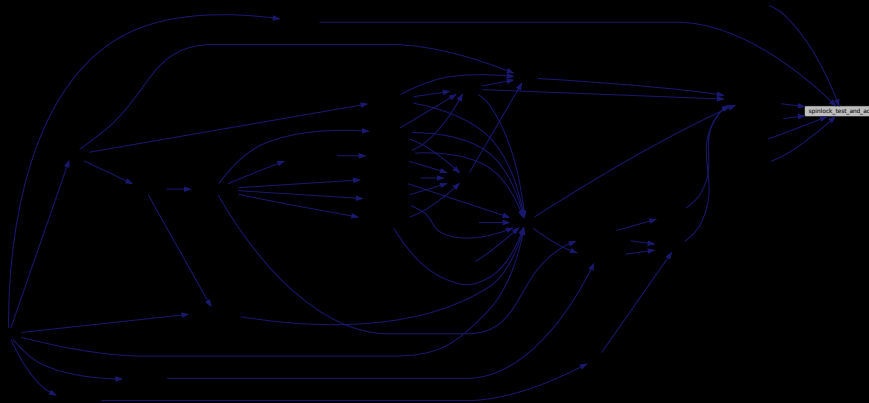


7.65.3.5 spinlock_test_and_acq()

```
static bool spinlock_test_and_acq (
    spinlock_t * lock ) [inline], [static]
```

Definition at line 26 of file [lock.h](#).

Here is the caller graph for this function:



7.66 lock.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #ifndef _LOCK_C_H
00003 #define _LOCK_C_H
00004
00005 #include "global_defs.h"
00006 #include <stdbool.h>
00007
00008 typedef int spinlock_t;
00009
00010 #define SPINLOCK_INIT 0
00011
00012 struct smartlock
00013 {
00014     size_t      refcount;
00015     struct thread *thread;
00016 };
00017
00018 #define SMARTLOCK_INIT \
00019 { \
00020     0, NULL \
00021 }
00022
00023 void smartlock_acquire(struct smartlock *smartlock);
00024 void smartlock_release(struct smartlock *smartlock);
00025
00026 static inline bool spinlock_test_and_acq(spinlock_t *lock)
00027 {
00028     return CAS(lock, 0, 1);
00029 }
00030
00031 static inline void spinlock_acquire(spinlock_t *lock)
00032 {
00033     for (;;)
00034     {
00035         if (spinlock_test_and_acq(lock))
00036             break;
00037         #if defined(__x86_64__)
00038             asm volatile("pause");
00039         #endif
00040     }
00041 }
```

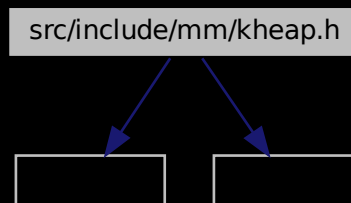
```
00041 }
00042
00043 static inline void spinlock_release(spinlock_t *lock)
00044 {
00045     CAS(lock, 1, 0);
00046 }
00047
00048 #endif
```

7.67 src/include/mm/kheap.h File Reference

```
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for kheap.h:



Macros

- #define `KHEAP_H`

Functions

- void `kheap_init` (void)
- void `kheap_reserve_pages` (size_t pages)

7.67.1 Macro Definition Documentation

7.67.1.1 KHEAP_H

```
#define KHEAP_H
```

Copyright (c) 2025 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Definition at line 10 of file `kheap.h`.

7.67.2 Function Documentation

7.67.2.1 kheap_init()

```
void kheap_init (
    void )
```

7.67.2.2 kheap_reserve_pages()

```
void kheap_reserve_pages (
    size_t pages )
```

7.68 kheap.h

[Go to the documentation of this file.](#)

```
00001
00008 #pragma once
00009 #ifndef KHEAP_H
00010 #define KHEAP_H
00011
00012 #include <stddef.h>
00013 #include <stdint.h>
00014
00015 #ifdef __cplusplus
00016 extern "C"
00017 {
00018 #endif
00019
00020     /* Initialize the kernel heap VA window; no pages mapped yet. */
00021     void kheap_init(void);
00022
00023     /* Optional: pre-reserve N pages upfront (maps VA, allocs frames). */
00024     void kheap_reserve_pages(size_t pages);
00025
00026 #ifdef __cplusplus
00027 }
00028 #endif
00029 #endif
```

7.69 src/include/mm/kmalloc.h File Reference

General-purpose Stage-2 kernel heap interface.

7.69.1 Detailed Description

General-purpose Stage-2 kernel heap interface.

Copyright (c) 2025 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Definition in file `kmalloc.h`.

7.69.2 Macro Definition Documentation

7.69.2.1 KMALLOC_H

```
#define KMALLOC_H
```

Definition at line 15 of file `kmalloc.h`.

7.69.3 Function Documentation

7.69.3.1 kfree()

```
void kfree (  
    void * ptr )
```

Return a heap allocation to the kernel allocator.

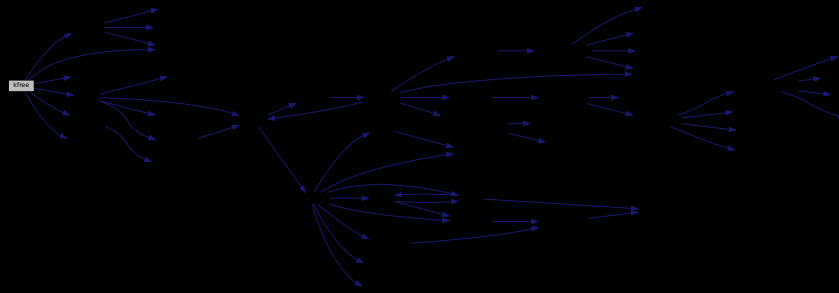
Parameters

<i>ptr</i>	Pointer previously returned by <code>kmalloc()/krealloc()</code> , or NULL.
------------	---

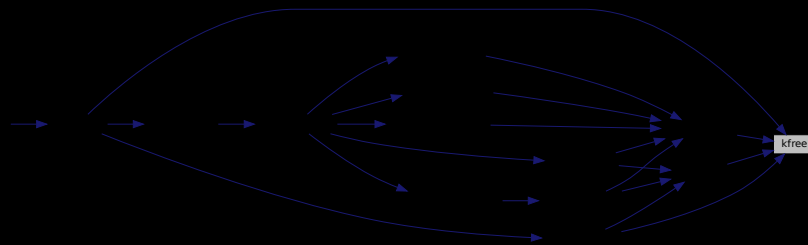
Freed blocks become reusable by later heap allocations. In the current 0.0.5 design, freeing a block does not shrink the mapped heap high-water mark or immediately return the heap's backing pages to the PMM.

Definition at line 281 of file `kmalloc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.69.3.2 kmalloc()

```
void * kmalloc (
    size_t size )
```

Allocate at least `size` bytes of kernel virtual memory.

Parameters

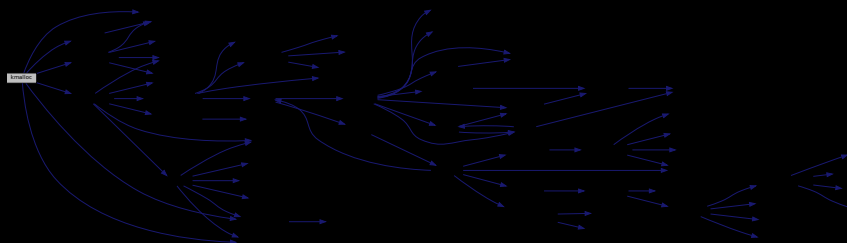
<code>size</code>	Requested payload size in bytes.
-------------------	----------------------------------

Returns

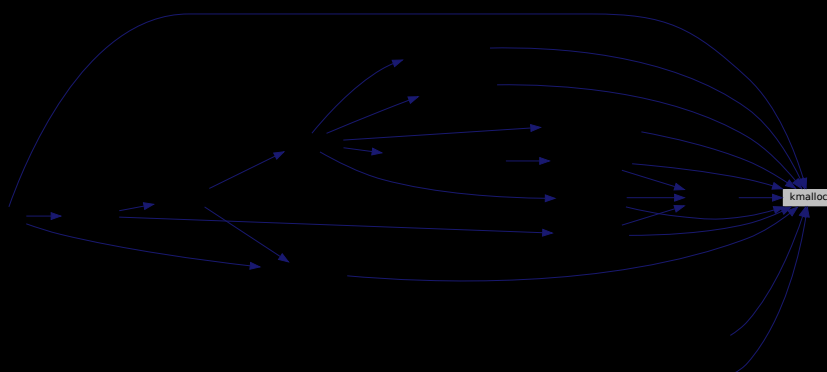
Pointer to allocated kernel memory, or NULL if the request cannot be satisfied according to heap semantics.

Definition at line 252 of file `kmalloc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.69.3.3 `kmalloc_dump()`

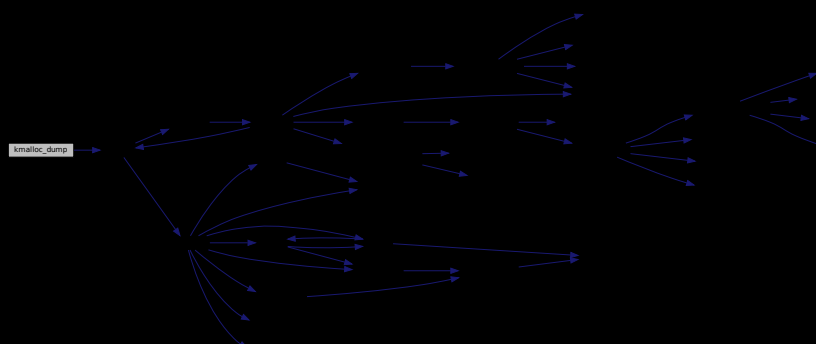
```
void kmalloc_dump (  
    void )
```

Emit allocator state for debugging.

This is a diagnostic helper and is not part of the allocation fast path.

Definition at line 364 of file `kmalloc.c`.

Here is the call graph for this function:



7.69.3.4 kmalloc_init()

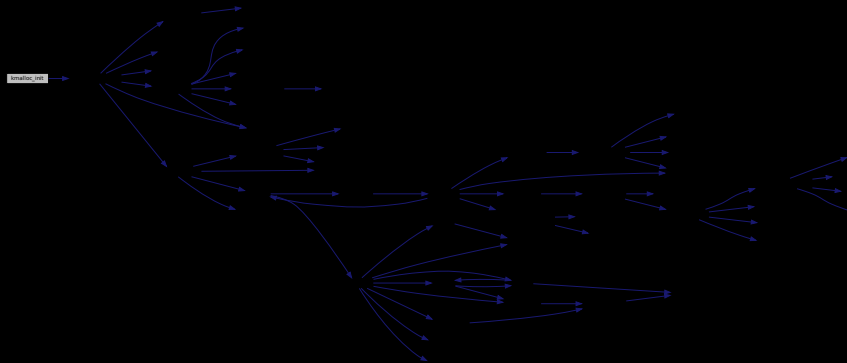
```
void kmalloc_init (
    void )
```

Initialize the kernel heap.

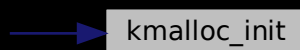
Must be called after `paging_bootstrap()` has installed Voyager's page tables and after the permanent physical frame allocator is available. The heap may grow by requesting physical frames and mapping additional virtual pages.

Definition at line 244 of file `kmalloc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.69.3.5 kmalloc_usable_size()

```
size_t kmalloc_usable_size (
    void * ptr )
```

Query the allocator's usable payload size for an allocation.

Parameters

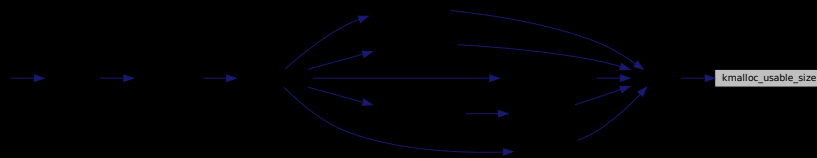
<i>ptr</i>	Active heap allocation.
------------	-------------------------

Returns

Number of usable payload bytes associated with `ptr`.

Definition at line 355 of file kmalloc.c.

Here is the caller graph for this function:



7.69.3.6 krealloc()

```
void * krealloc (
    void * ptr,
    size_t newsize )
```

Resize an existing heap allocation while preserving payload data.

Parameters

<i>ptr</i>	Existing allocation, or NULL to behave like <code>kmalloc()</code> .
<i>newsize</i>	New requested payload size in bytes.

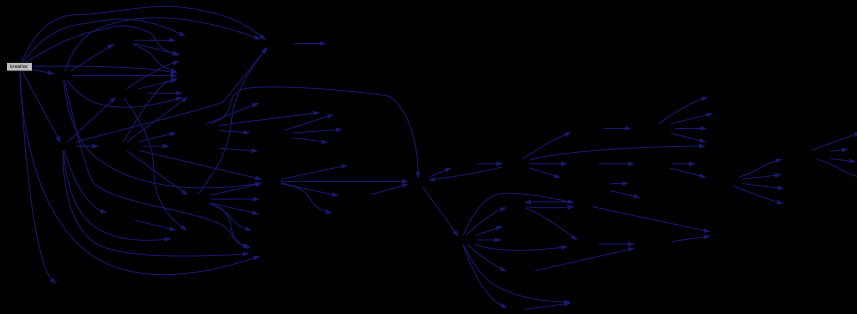
Returns

Resized allocation, which may move, or NULL according to allocator semantics.

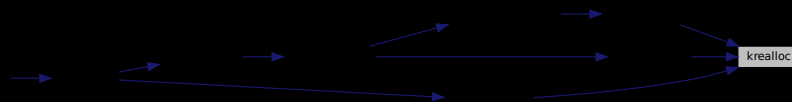
`krealloc(ptr, 0)` releases the allocation. Data up to the minimum of the old and new usable sizes is preserved when the allocation moves.

Definition at line 303 of file kmalloc.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.70 kmalloc.h

Go to the documentation of this file.

```

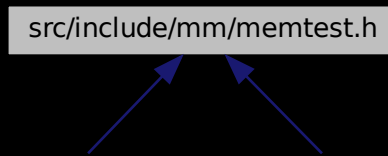
00001
00013 #pragma once
00014 #ifndef KMALLOC_H
00015 #define KMALLOC_H
00016
00017 #include <stddef.h>
00018 #include <stdint.h>
00019
00020 #ifdef __cplusplus
00021 extern "C"
00022 {
00023 #endif
00024
00033     void kmalloc_init(void);
00034
00041     void *kmalloc(size_t size);
00042
00051     void kfree(void *ptr);
00052
00063     void *krealloc(void *ptr, size_t newsize);
00064
00070     size_t kmalloc_usable_size(void *ptr);
00071
00077     void kmalloc_dump(void);
00078
00079 #ifdef __cplusplus
00080 }
00081 #endif
00082 #endif

```

7.71 src/include/mm/memtest.h File Reference

Stage-2 PMM/heap qualification harness.

This graph shows which files directly or indirectly include this file:



Macros

- `#define VOYAGER_MEMTEST_H`

Functions

- void `memtest_run` (void)
Run the Stage-2 PMM/heap qualification suite.

7.71.1 Detailed Description

Stage-2 PMM/heap qualification harness.

Copyright (c) 2026 Tristan Adams

VoyagerOS64 memory qualification harness.

Definition in file `memtest.h`.

7.71.2 Macro Definition Documentation

7.71.2.1 VOYAGER_MEMTEST_H

```
#define VOYAGER_MEMTEST_H
```

Definition at line 14 of file `memtest.h`.

7.71.3 Function Documentation

7.71.3.1 memtest_run()

```
void memtest_run (
    void )
```

Run the Stage-2 PMM/heap qualification suite.

Intended to be invoked after paging bootstrap, frame allocator initialization, kmalloc initialization, and scheduler startup. The test is destructive only to memory it allocates itself and yields periodically during long randomized phases so the cooperative scheduler continues to run.

The suite exercises direct frame allocation/free, heap boundary sizes, fragmentation and coalescing, krealloc preservation, multi-megabyte heap growth, and deterministic randomized allocation/free/reallocation cycles.

Note

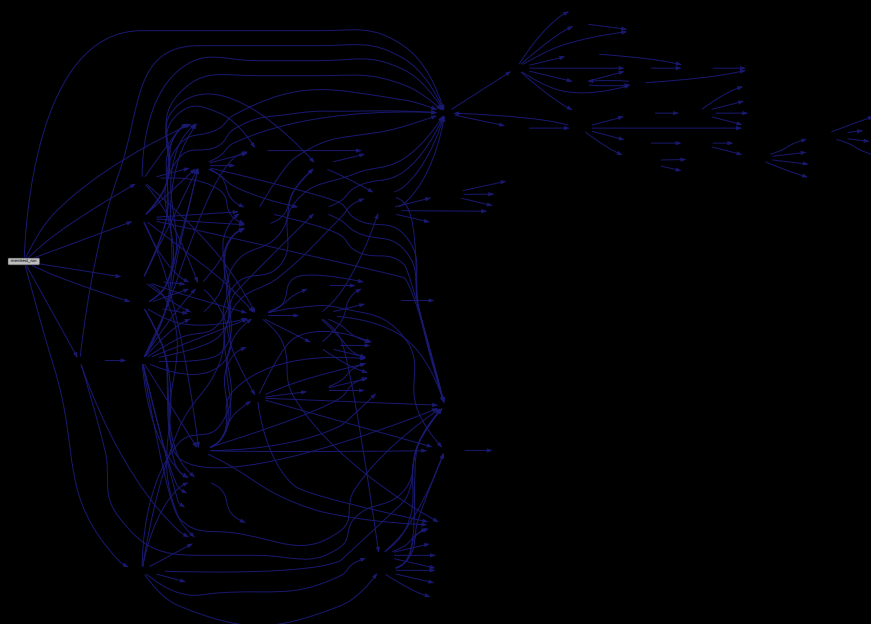
Current heap pages are retained after `kfree()`. A lower PMM free-frame count after the first high-water workload is therefore expected; repeated identical workloads should plateau once the heap has grown sufficiently.

Warning

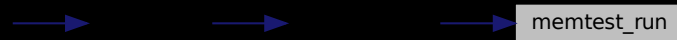
A detected corruption or violated allocator invariant is treated as a kernel qualification failure and may panic the system.

Definition at line 646 of file `memtest.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.72 memtest.h

Go to the documentation of this file.

```

00001
00012 #pragma once
00013 #ifndef VOYAGER_MEMTEST_H
00014 #define VOYAGER_MEMTEST_H
00015
00016 #ifdef __cplusplus
00017 extern "C"
00018 {
00019 #endif
00020
00040 void memtest_run(void);
00041
00042 #ifdef __cplusplus
00043 }
00044 #endif
00045
00046 #endif

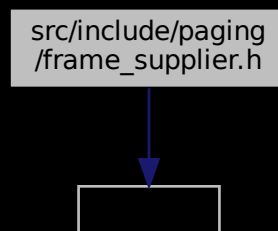
```

7.73 src/include/paging/frame_supplier.h File Reference

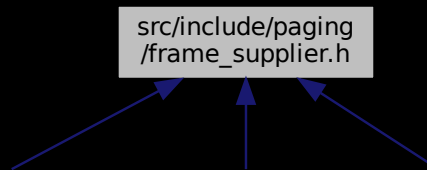
Boot-stage physical-page supplier indirection used by paging.

```
#include <stdint.h>
```

Include dependency graph for frame_supplier.h:



This graph shows which files directly or indirectly include this file:



Macros

- `#define` `FRAME_SUPPLIER_H`

Typedefs

- `typedef uint64_t(* phys_page_supplier_t) (void)`
Function type returning one physical page address.

Functions

- `static uint64_t PagingGetFreeFrame (void)`
Request one page-table frame from the currently authoritative supplier.

Variables

- `phys_page_supplier_t g_boot_phys_page`
Active boot-time physical-page supplier.

7.73.1 Detailed Description

Boot-stage physical-page supplier indirection used by paging.

Copyright (c) 2025 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Paging must allocate page-table frames both before and after the permanent PMM is initialized. This tiny adapter keeps paging independent of the allocator transition: early boot points it at `early_alloc_page()`, then memory bring-up switches it to `frame_alloc()`.

Definition in file `frame_supplier.h`.

7.73.2 Macro Definition Documentation

7.73.2.1 FRAME_SUPPLIER_H

```
#define FRAME_SUPPLIER_H
```

Definition at line 20 of file [frame_supplier.h](#).

7.73.3 Typedef Documentation

7.73.3.1 phys_page_supplier_t

```
typedef uint64_t(* phys_page_supplier_t) (void)
```

Function type returning one physical page address.

Definition at line 25 of file [frame_supplier.h](#).

7.73.4 Function Documentation

7.73.4.1 PagingGetFreeFrame()

```
static uint64_t PagingGetFreeFrame (
    void ) [inline], [static]
```

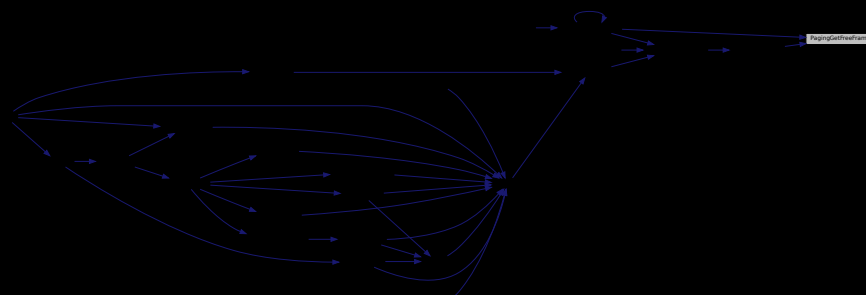
Request one page-table frame from the currently authoritative supplier.

Returns

Physical page address, or 0 if no supplier has been installed.

Definition at line 39 of file [frame_supplier.h](#).

Here is the caller graph for this function:



7.73.5 Variable Documentation

7.73.5.1 g_boot_phys_page

```
phys_page_supplier_t g_boot_phys_page [extern]
```

Active boot-time physical-page supplier.

Kernel bring-up owns this pointer and changes it exactly when allocator authority moves from the early bump allocator to the permanent PMM.

Copyright (c) 2025 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Definition at line 10 of file `frame_supplier.c`.

7.74 frame_supplier.h

Go to the documentation of this file.

```
00001
00018 #pragma once
00019 #ifndef FRAME_SUPPLIER_H
00020 #define FRAME_SUPPLIER_H
00021
00022 #include <stdint.h>
00023
00025 typedef uint64_t (*phys_page_supplier_t)(void);
00026
00033 extern phys_page_supplier_t g_boot_phys_page;
00034
00039 static inline uint64_t PagingGetFreeFrame(void)
00040 {
00041     return g_boot_phys_page ? g_boot_phys_page() : 0;
00042 }
00043
00044 #endif
```

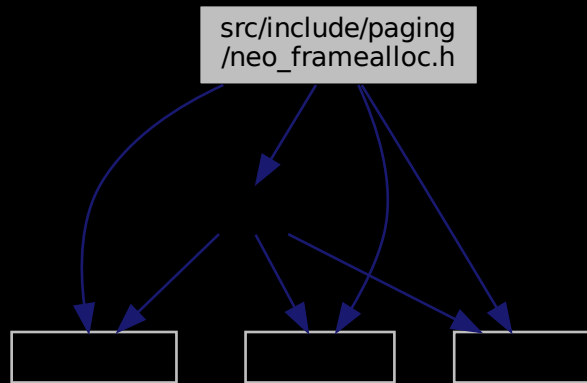
7.75 src/include/paging/neo_framealloc.h File Reference

Permanent Stage-2 physical frame allocator interface.

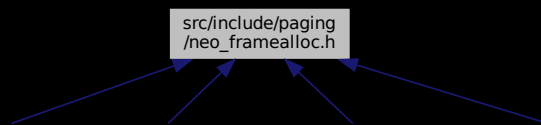
```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
```

```
#include "../early_alloc.h"
```

Include dependency graph for neo_framealloc.h:



This graph shows which files directly or indirectly include this file:



Functions

- void **frame_init** (void)
Initialize the permanent physical memory manager from Limine's memmap.
- **paddr_t frame_alloc** (void)
Allocate one 4 KiB physical frame.
- void **n_frame_free** (**paddr_t** paddr)
Return a 4 KiB physical frame to the PMM.
- size_t **frame_free_count** (void)
Count currently free physical frames.
- size_t **frame_total_count** (void)
Count physical frames managed by the PMM.
- bool **frame_is_allocated** (**paddr_t** paddr)
Query allocation state for a physical frame.

7.75.1 Detailed Description

Permanent Stage-2 physical frame allocator interface.

Copyright (c) 2025 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Definition in file `neo_framealloc.h`.

7.75.2 Function Documentation

7.75.2.1 `frame_alloc()`

```
paddr_t frame_alloc (  
    void )
```

Allocate one 4 KiB physical frame.

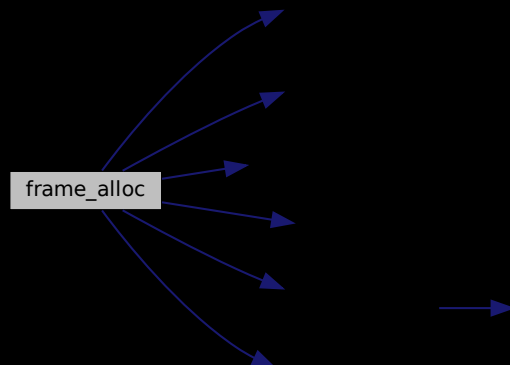
Returns

Physical address of an allocated 4 KiB frame.

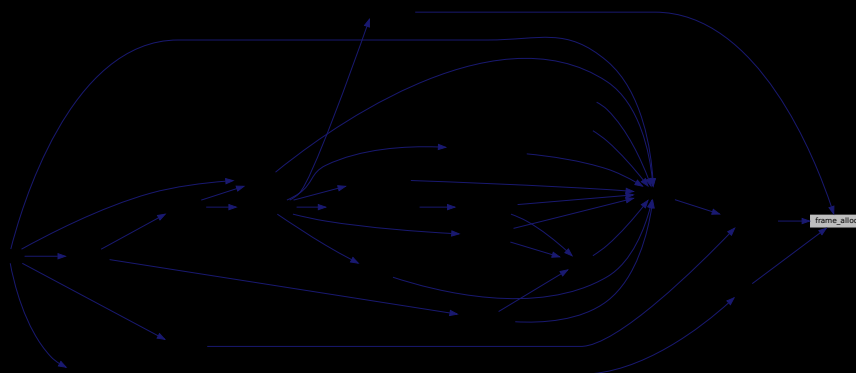
The returned value is a physical address, not a C pointer. The allocator panics on exhaustion rather than returning an ordinary heap-style NULL.

Definition at line 215 of file `neo_framealloc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.75.2.2 `frame_free_count()`

```
size_t frame_free_count (
    void )
```

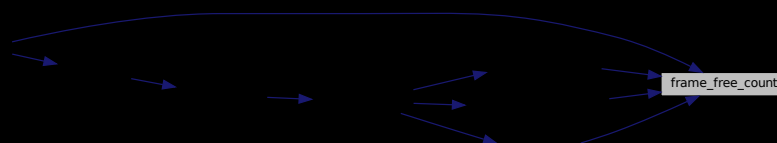
Count currently free physical frames.

Returns

Number of allocatable 4 KiB frames currently free.

Definition at line 288 of file `neo_framealloc.c`.

Here is the caller graph for this function:



Parameters

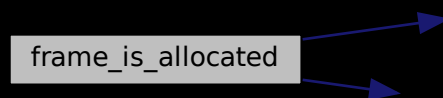
<i>paddr</i>	4 KiB-aligned physical frame address.
--------------	---------------------------------------

Returns

true when the frame is currently marked allocated; false otherwise.

Definition at line 277 of file [neo_framealloc.c](#).

Here is the call graph for this function:



7.75.2.5 frame_total_count()

```
size_t frame_total_count (  
    void )
```

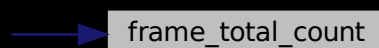
Count physical frames managed by the PMM.

Returns

Number of 4 KiB frames represented by the active allocator.

Definition at line 284 of file [neo_framealloc.c](#).

Here is the caller graph for this function:



7.75.2.6 n_frame_free()

```
void n_frame_free (
    paddr_t paddr )
```

Return a 4 KiB physical frame to the PMM.

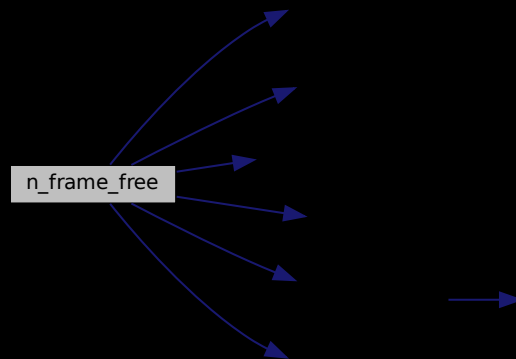
Parameters

<i>paddr</i>	Physical address previously obtained from <code>frame_alloc()</code> .
--------------	--

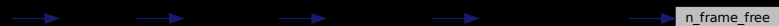
The implementation guards alignment, allocator bounds, non-usable regions, and double-free attempts.

Definition at line 246 of file neo_framealloc.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.76 neo_framealloc.h

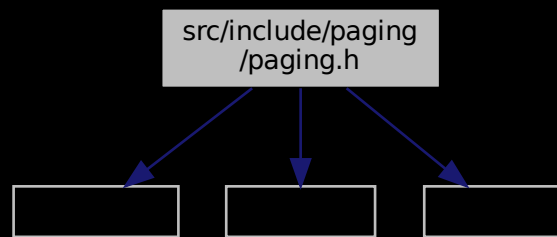
```
00001
00013 #pragma once
00014 #include <stdbool.h>
00015 #include <stddef.h>
00016 #include <stdint.h>
00017
00018 #include "../early_alloc.h"
00019
00028 void frame_init(void);
00029
00037 paddr_t frame_alloc(void);
00038
00046 void n_frame_free(paddr_t paddr);
00047
00052 size_t frame_free_count(void);
00053
00058 size_t frame_total_count(void);
00059
00065 bool frame_is_allocated(paddr_t paddr);
```

7.77 src/include/paging/paging.h File Reference

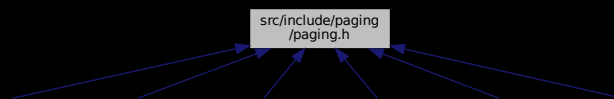
Generic x86-64 page-table helpers and address-space constants.

```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
```

Include dependency graph for paging.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct **PageTableOffset**
Four-level x86-64 page-table indices for a virtual address.

Macros

- #define **_PAGING_H_**
- #define **PAGING_EXPORT**
- #define **HIGHER_HALF_MEMORY_OFFSET** 0xFFFF800000000000
- #define **HIGHER_HALF_KERNEL_MEMORY_OFFSET** 0xFFFFFFFF80000000
- #define **PAGE_FLAG_MASK** (0xFFF | (1ull << 63))
- #define **PAGE_ADDRESS_MASK** (~(PAGE_FLAG_MASK))

Enumerations

- enum `PagingFlag` {
`PAGING_FLAG_PRESENT` = (1ull << 0) , `PAGING_FLAG_WRITABLE` = (1ull << 1) , `PAGING_FLAG_USER_ACCESSIBLE` = (1ull << 2) , `PAGING_FLAG_PAT0` = (1ull << 3) ,
`PAGING_FLAG_PAT1` = (1ull << 4) , `PAGING_FLAG_PAT2` = (1ull << 7) , `PAGING_FLAG_WRITE_COMBINE` = `PAGING_FLAG_PAT0` | `PAGING_FLAG_PAT1` , `PAGING_FLAG_LARGER_PAGES` = (1ull << 7) ,
`PAGING_FLAG_NO_EXECUTE` = (1ull << 63) }

x86-64 page-entry flag values used by VoyagerOS64.

Functions

- static uint64_t `TranslateToHighHalfMemoryAddress` (uint64_t physicalAddress)
Convert a physical address to its HHDM virtual address.
- uint64_t `TranslateToPhysicalMemoryAddress` (uint64_t virtualAddress)
Convert an HHDM virtual address back to a physical address.
- uint64_t `TranslateToKernelPhysicalMemoryAddress` (uint64_t virtualAddress)
Convert a higher-half kernel-image VA to its link-relative physical offset.
- uint64_t `TranslateToKernelMemoryAddress` (uint64_t virtualAddress)
Add the kernel higher-half base to an address/offset.
- bool `IsHigherHalf` (uint64_t physicalAddress)
Test whether an address value lies at or above the HHDM base.
- bool `IsKernelHigherHalf` (uint64_t physicalAddress)
Test whether an address value lies at or above the kernel-image base.
- struct `__attribute__((aligned(0x1000))) PageTable`
One 4 KiB x86-64 page table containing 512 entries.
- `PAGING_EXPORT` void `PagingMapMemory` (struct PageTable *p4, void *virtualMemory, void *physicalMemory, uint64_t flags)
Map one 4 KiB virtual page to one physical page.
- `PAGING_EXPORT` void `PagingUnmapMemory` (struct PageTable *p4, void *virtualMemory)
Clear the leaf mapping for one virtual page.
- `PAGING_EXPORT` void `PagingIdentityMap` (struct PageTable *p4, void *virtualMemory, uint64_t flags)
Identity-map one page so its virtual and physical addresses match.
- `PAGING_EXPORT` void * `PagingPhysicalMemory` (struct PageTable *p4, void *virtualMemory)
Resolve the physical page backing a mapped virtual page.
- `PAGING_EXPORT` void `PagingDuplicate` (struct PageTable *p4, struct PageTable *newTable)
Duplicate the current paging structure into newTable.
- `PAGING_EXPORT` void `PagingSetActivePageTable` (struct PageTable *p4)
Install p4 as the active page-table root.

7.77.1 Detailed Description

Generic x86-64 page-table helpers and address-space constants.

Definition in file `paging.h`.

7.77.2 Macro Definition Documentation

7.77.2.1 `_PAGING_H_`

```
#define _PAGING_H_
```

Definition at line 8 of file [paging.h](#).

7.77.2.2 `HIGHER_HALF_KERNEL_MEMORY_OFFSET`

```
#define HIGHER_HALF_KERNEL_MEMORY_OFFSET 0xFFFFFFFF80000000
```

Link-time higher-half virtual base of the kernel image.

Definition at line 22 of file [paging.h](#).

7.77.2.3 `HIGHER_HALF_MEMORY_OFFSET`

```
#define HIGHER_HALF_MEMORY_OFFSET 0xFFFF800000000000
```

Base of the Limine HHDM used by the current VoyagerOS64 configuration.

Definition at line 20 of file [paging.h](#).

7.77.2.4 `PAGE_ADDRESS_MASK`

```
#define PAGE_ADDRESS_MASK (~ (PAGE_FLAG_MASK))
```

Mask selecting the physical-address field of an ordinary page entry.

Definition at line 27 of file [paging.h](#).

7.77.2.5 `PAGE_FLAG_MASK`

```
#define PAGE_FLAG_MASK (0xFFF | (1ull << 63))
```

Bits treated as non-address metadata in ordinary 4 KiB page entries.

Definition at line 25 of file [paging.h](#).

7.77.2.6 `PAGING_EXPORT`

```
#define PAGING_EXPORT
```

Definition at line 16 of file [paging.h](#).

7.77.3 Enumeration Type Documentation

7.77.3.1 `PagingFlag`

```
enum PagingFlag
```

x86-64 page-entry flag values used by VoyagerOS64.

Enumerator

PAGING_FLAG_PRESENT	
PAGING_FLAG_WRITABLE	
PAGING_FLAG_USER_ACCESSIBLE	
PAGING_FLAG_PAT0	
PAGING_FLAG_PAT1	
PAGING_FLAG_PAT2	
PAGING_FLAG_WRITE_COMBINE	
PAGING_FLAG_LARGER_PAGES	
PAGING_FLAG_NO_EXECUTE	

Definition at line 100 of file [paging.h](#).

7.77.4 Function Documentation

7.77.4.1 __attribute__()

```
struct __attribute__ (
    (aligned(0x1000)) )
```

One 4 KiB x86-64 page table containing 512 entries.

Definition at line 94 of file [paging.h](#).

7.77.4.2 IsHigherHalf()

```
bool IsHigherHalf (
    uint64_t physicalAddress ) [inline]
```

Test whether an address value lies at or above the HHDM base.

Parameters

<i>physicalAddress</i>	Address value to check.
------------------------	-------------------------

Returns

true when the value is in the configured higher-half range.

Definition at line 84 of file [paging.h](#).

7.77.4.3 IsKernelHigherHalf()

```
bool IsKernelHigherHalf (
    uint64_t physicalAddress ) [inline]
```

Test whether an address value lies at or above the kernel-image base.

Parameters

<i>physicalAddress</i>	Address value to check.
------------------------	-------------------------

Returns

true when the value is in the configured kernel higher-half range.

Definition at line 94 of file [paging.h](#).

7.77.4.4 PagingDuplicate()

```
PAGING\_EXPORT void PagingDuplicate (
    struct PageTable * p4,
    struct PageTable * newTable )
```

Duplicate the current paging structure into *newTable*.

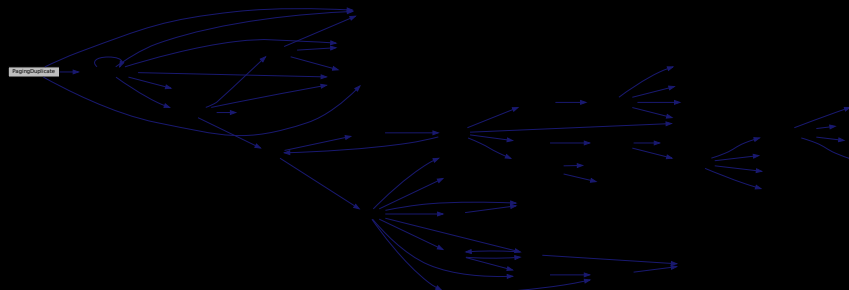
Parameters

<i>p4</i>	Physical address of the source root PML4.
<i>newTable</i>	Destination root table accessible to the kernel.

The current implementation recursively duplicates the lower half and shares the upper-half root entries. Voyager↔ OS64 0.0.5 does not yet expose this as a complete per-process address-space or fork facility.

Definition at line 274 of file [paging.c](#).

Here is the call graph for this function:



7.77.4.5 PagingIdentityMap()

```
PAGING_EXPORT void PagingIdentityMap (
    struct PageTable * p4,
    void * virtualMemory,
    uint64_t flags )
```

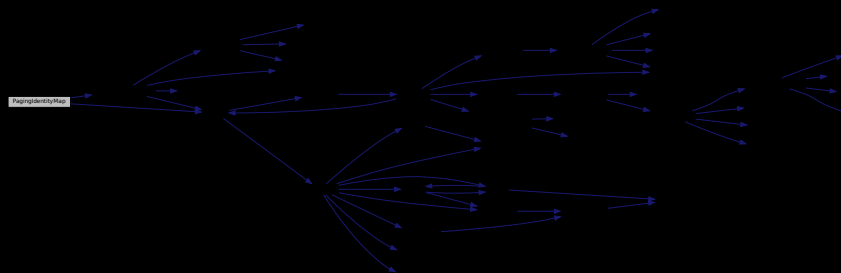
Identity-map one page so its virtual and physical addresses match.

Parameters

<i>p4</i>	Physical address of the root PML4.
<i>virtualMemory</i>	Address used as both virtual and physical page address.
<i>flags</i>	Leaf-page flags.

Definition at line 163 of file [paging.c](#).

Here is the call graph for this function:



7.77.4.6 PagingMapMemory()

```
PAGING_EXPORT void PagingMapMemory (
    struct PageTable * p4,
    void * virtualMemory,
    void * physicalMemory,
    uint64_t flags )
```

Map one 4 KiB virtual page to one physical page.

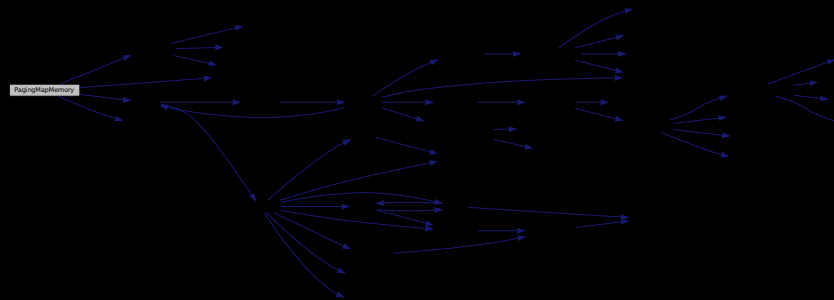
Parameters

<i>p4</i>	Physical address of the root PML4 used by this paging layer.
<i>virtualMemory</i>	Virtual page address to map.
<i>physicalMemory</i>	Physical page address to install in the leaf entry.
<i>flags</i>	Leaf-page flags in addition to the present bit.

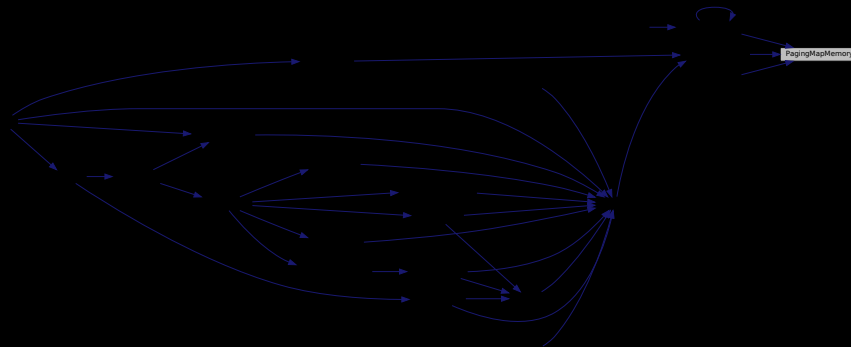
Missing intermediate tables are allocated through [PagingGetFreeFrame\(\)](#).

Definition at line 181 of file [paging.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.77.4.7 PagingPhysicalMemory()

```
PAGING_EXPORT void * PagingPhysicalMemory (
    struct PageTable * p4,
    void * virtualMemory )
```

Resolve the physical page backing a mapped virtual page.

Parameters

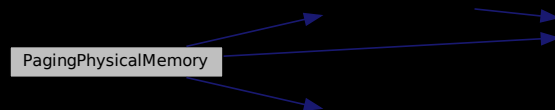
<i>p4</i>	Physical address of the root PML4.
<i>virtualMemory</i>	Virtual address to inspect.

Returns

Physical page address encoded by the leaf entry, or NULL when an intermediate level is not present.

Definition at line 213 of file [paging.c](#).

Here is the call graph for this function:



7.77.4.8 PagingSetActivePageTable()

```
PAGING_EXPORT void PagingSetActivePageTable (
    struct PageTable * p4 )
```

Install `p4` as the active page-table root.

Parameters

<i>p4</i>	Root table expected by the implementation.
-----------	--

Prefer [paging_bootstrap\(\)](#) for the initial 0.0.5 kernel CR3 handoff; this function belongs to the generic paging helper layer.

7.77.4.9 PagingUnmapMemory()

```
PAGING_EXPORT void PagingUnmapMemory (
    struct PageTable * p4,
    void * virtualMemory )
```

Clear the leaf mapping for one virtual page.

Parameters

<i>p4</i>	Physical address of the root PML4.
<i>virtualMemory</i>	Virtual page whose leaf entry should be cleared.

This does not free intermediate page tables or the previously mapped physical frame.

Definition at line 244 of file [paging.c](#).

```

graph LR
    A[PagingUnmapMemory] --> B[ ]
    A --> C[ ]
    A --> D[ ]
    B --> E[ ]
    C --> F[ ]
    D --> G[ ]
  
```

The diagram illustrates the execution flow of the `PagingUnmapMemory` function. It starts with a single node labeled `PagingUnmapMemory`. From this node, three arrows branch out to the right. The top arrow leads to a node that then branches into two more arrows. The middle arrow leads to a node that then branches into two more arrows. The bottom arrow leads to a node that then branches into two more arrows. This structure suggests a complex, multi-branching execution path.

7.77.4.11 TranslateToKernelMemoryAddress()

```
uint64_t TranslateToKernelMemoryAddress (
    uint64_t virtualAddress ) [inline]
```

Add the kernel higher-half base to an address/offset.

Parameters

<i>virtualAddress</i>	Address/offset to translate.
-----------------------	------------------------------

Returns

Value in the kernel higher-half virtual region.

Definition at line 74 of file [paging.h](#).

7.77.4.12 TranslateToKernelPhysicalMemoryAddress()

```
uint64_t TranslateToKernelPhysicalMemoryAddress (
    uint64_t virtualAddress ) [inline]
```

Convert a higher-half kernel-image VA to its link-relative physical offset.

Parameters

<i>virtualAddress</i>	Address in the higher-half kernel-image region.
-----------------------	---

Returns

Address obtained by subtracting the kernel higher-half base.

Note

Runtime kernel physical placement is provided separately by Limine; this helper performs only the arithmetic encoded here.

Definition at line 64 of file [paging.h](#).

7.77.4.13 TranslateToPhysicalMemoryAddress()

```
uint64_t TranslateToPhysicalMemoryAddress (
    uint64_t virtualAddress ) [inline]
```

Convert an HHDM virtual address back to a physical address.

Parameters

<i>virtualAddress</i>	Address in the HHDM region.
-----------------------	-----------------------------

Returns

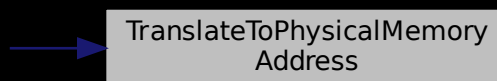
Physical address corresponding to *virtualAddress*.

Warning

This helper assumes the argument belongs to the HHDM. It is not a general-purpose virtual-to-physical translator for arbitrary kernel VAs.

Definition at line 50 of file [paging.h](#).

Here is the caller graph for this function:



7.78 [paging.h](#)

[Go to the documentation of this file.](#)

```

00001
00006 #pragma once
00007 #ifndef _PAGING_H_
00008 #define _PAGING_H_
00009 #include <stdbool.h>
00010 #include <stddef.h>
00011 #include <stdint.h>
00012
00013 #ifdef __cplusplus
00014 #define PAGING_EXPORT extern "C"
00015 #else
00016 #define PAGING_EXPORT
00017 #endif
00018
00020 #define HIGHER_HALF_MEMORY_OFFSET 0xFFFF800000000000
00022 #define HIGHER_HALF_KERNEL_MEMORY_OFFSET 0xFFFFFFFF80000000
00023
00025 #define PAGE_FLAG_MASK (0xFFF | (1ull < 63))
00027 #define PAGE_ADDRESS_MASK (~(PAGE_FLAG_MASK))
00028
00036 static inline uint64_t
00037 TranslateToHighHalfMemoryAddress(uint64_t physicalAddress)
00038 {
00039     return physicalAddress + HIGHER_HALF_MEMORY_OFFSET;
00040 }
00041
00050 inline uint64_t TranslateToPhysicalMemoryAddress(uint64_t virtualAddress)
00051 {
00052     return virtualAddress - HIGHER_HALF_MEMORY_OFFSET;
00053 }
00054
00064 inline uint64_t TranslateToKernelPhysicalMemoryAddress(uint64_t virtualAddress)

```



```

00065 {
00066     return virtualAddress - HIGHER_HALF_KERNEL_MEMORY_OFFSET;
00067 }
00068
00074 inline uint64_t TranslateToKernelMemoryAddress(uint64_t virtualAddress)
00075 {
00076     return virtualAddress + HIGHER_HALF_KERNEL_MEMORY_OFFSET;
00077 }
00078
00084 inline bool IsHigherHalf(uint64_t physicalAddress)
00085 {
00086     return physicalAddress >= HIGHER_HALF_MEMORY_OFFSET;
00087 }
00088
00094 inline bool IsKernelHigherHalf(uint64_t physicalAddress)
00095 {
00096     return physicalAddress >= HIGHER_HALF_KERNEL_MEMORY_OFFSET;
00097 }
00098
00100 enum PagingFlag
00101 {
00102     PAGING_FLAG_PRESENT           = (1ull < 0),
00103     PAGING_FLAG_WRITABLE         = (1ull < 1),
00104     PAGING_FLAG_USER_ACCESSIBLE = (1ull < 2),
00105     PAGING_FLAG_PAT0             = (1ull < 3),
00106     PAGING_FLAG_PAT1             = (1ull < 4),
00107     PAGING_FLAG_PAT2             = (1ull < 7),
00108     PAGING_FLAG_WRITE_COMBINE    = PAGING_FLAG_PAT0 | PAGING_FLAG_PAT1,
00109     PAGING_FLAG_LARGER_PAGES     = (1ull < 7),
00110     PAGING_FLAG_NO_EXECUTE       = (1ull < 63),
00111 };
00112
00114 struct PageTableOffset
00115 {
00116     uint64_t p4Offset;
00117     uint64_t pdpOffset;
00118     uint64_t pdOffset;
00119     uint64_t ptOffset;
00120 };
00121
00125 struct __attribute__((aligned(0x1000))) PageTable
00126 {
00127     uint64_t entries[512];
00128 };
00129
00139 PAGING_EXPORT void PagingMapMemory(struct PageTable *p4,
00140                                     void *virtualMemory,
00141                                     void *physicalMemory,
00142                                     uint64_t flags);
00143
00152 PAGING_EXPORT void PagingUnmapMemory(struct PageTable *p4, void *virtualMemory);
00153
00160 PAGING_EXPORT void
00161 PagingIdentityMap(struct PageTable *p4, void *virtualMemory, uint64_t flags);
00162
00170 PAGING_EXPORT void *PagingPhysicalMemory(struct PageTable *p4,
00171                                           void *virtualMemory);
00172
00182 PAGING_EXPORT void PagingDuplicate(struct PageTable *p4,
00183                                    struct PageTable *newTable);
00184
00192 PAGING_EXPORT void PagingSetActivePageTable(struct PageTable *p4);
00193
00194 #endif

```

7.79 src/include/paging/paging_bootstrap.h File Reference

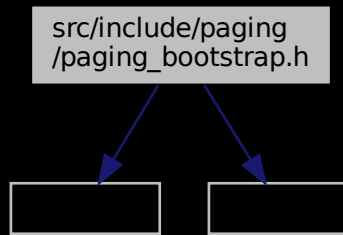
Construction and installation of Voyager-owned bootstrap page tables.

```

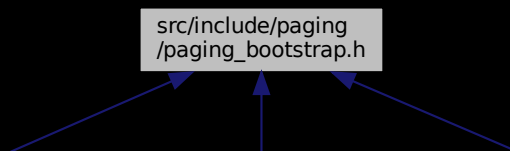
#include <stddef.h>
#include <stdint.h>

```

Include dependency graph for paging_bootstrap.h:



This graph shows which files directly or indirectly include this file:



Functions

- void `paging_bootstrap` (void)
Build VoyagerOS64's initial kernel page-table hierarchy and install it in CR3.
- uint64_t `paging_pml4_phys` (void)
Return the physical address loaded as the bootstrap PML4 root.
- void * `paging_pml4_virt` (void)
Return a kernel-accessible virtual pointer to the bootstrap PML4.

7.79.1 Detailed Description

Construction and installation of Voyager-owned bootstrap page tables.

Copyright (c) 2025 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Definition in file `paging_bootstrap.h`.

7.79.2 Function Documentation

7.79.2.1 paging_bootstrap()

```
void paging_bootstrap (
    void )
```

Build VoyagerOS64's initial kernel page-table hierarchy and install it in CR3.

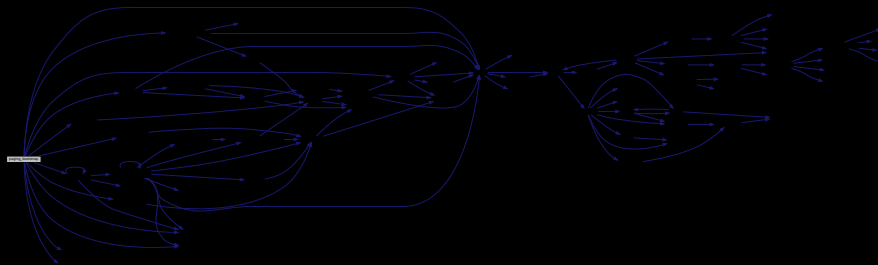
Page-table pages are supplied from `early_alloc_page()` during bootstrap. The resulting mappings preserve the kernel image, active stack, HHDM, and other ranges required for the post-Limine kernel to continue executing.

Warning

After the new CR3 is installed, code must not assume Limine-owned callback code remains mapped or callable. Voyager's framebuffer/serial facilities own output after the handoff.

Definition at line 636 of file `paging_bootstrap.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.79.2.2 paging_pml4_phys()

```
uint64_t paging_pml4_phys (  
    void )
```

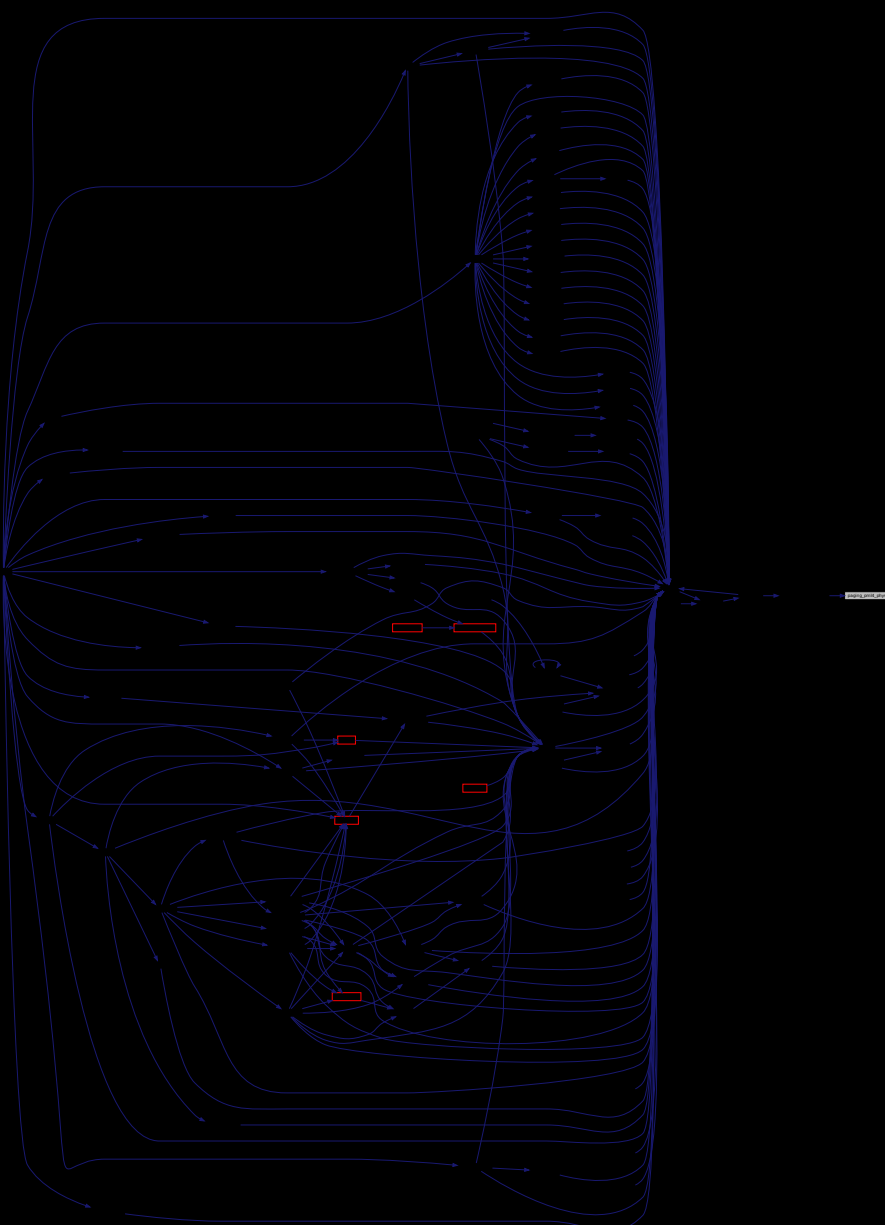
Return the physical address loaded as the bootstrap PML4 root.

Returns

Physical address of Voyager's root page table.

Definition at line 71 of file [paging_bootstrap.c](#).

Here is the caller graph for this function:



7.79.2.3 paging_pml4_virt()

```
void * paging_pml4_virt (
    void )
```

Return a kernel-accessible virtual pointer to the bootstrap PML4.

Returns

Virtual address through which the root page table can be accessed.

Definition at line 76 of file [paging_bootstrap.c](#).

7.80 paging_bootstrap.h

[Go to the documentation of this file.](#)

```
00001
00013 #pragma once
00014 #include <stddef.h>
00015 #include <stdint.h>
00016
00017 #ifdef __cplusplus
00018 extern "C"
00019 {
00020 #endif
00021
00034     void paging_bootstrap(void);
00035
00040     uint64_t paging_pml4_phys(void);
00041
00046     void *paging_pml4_virt(void);
00047
00048 #ifdef __cplusplus
00049 }
00050 #endif
```

7.81 src/include/panic.h File Reference

Fatal kernel error-reporting interface.

This graph shows which files directly or indirectly include this file:



Macros

- `#define _PANIC_H`

Functions

- `__attribute__((noreturn)) void panic(const char *fmt`
Report an unrecoverable kernel error and stop normal execution.

7.81.1 Detailed Description

Fatal kernel error-reporting interface.

Definition in file [panic.h](#).

7.81.2 Macro Definition Documentation

7.81.2.1 _PANIC_H

```
#define _PANIC_H
```

Definition at line 8 of file [panic.h](#).

7.81.3 Function Documentation

7.81.3.1 __attribute__((noret))

```
__attribute__((noret)) (
    (noreturn) ) const
```

Report an unrecoverable kernel error and stop normal execution.

The first argument is a printf-style format string followed by matching variadic arguments. [panic\(\)](#) is the common failure path for violated kernel invariants such as allocator corruption, impossible scheduler state, or missing mandatory boot resources. It does not return.

Note

Doxygen 1.9.x can misparse the GNU noreturn attribute when attaching parameter documentation to this declaration, so the format-string contract is described in prose rather than with a dedicated parameter tag.

7.82 panic.h

[Go to the documentation of this file.](#)

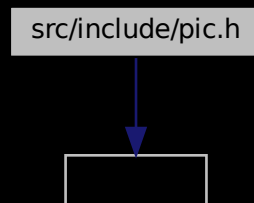
```
00001
00006 #pragma once
00007 #ifndef _PANIC_H
00008 #define _PANIC_H
00009
00022 __attribute__((noret)) void panic(const char *fmt, ...);
00023
00024 #endif
```

7.83 src/include/pic.h File Reference

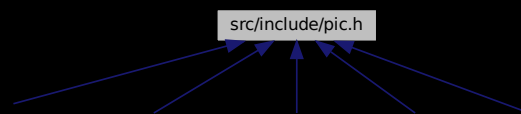
Legacy 8259 PIC programming interface.

```
#include "stdint.h"
```

Include dependency graph for pic.h:



This graph shows which files directly or indirectly include this file:



Macros

- `#define PIC_MASTER 0x20`
- `#define PIC_SLAVE 0xA0`
- `#define PIC_MASTER_COMMAND PIC_MASTER`
- `#define PIC_MASTER_DATA (PIC_MASTER + 1)`
- `#define PIC_SLAVE_COMMAND PIC_SLAVE`
- `#define PIC_SLAVE_DATA (PIC_SLAVE + 1)`
- `#define PIC_EOI 0x20`
- `#define PIC_ICW1_ICW4 0x01`
- `#define PIC_ICW1_SINGLE 0x02`
- `#define PIC_ICW1_INTERVAL4 0x04`
- `#define PIC_ICW1_LEVEL 0x08`
- `#define PIC_ICW1_INIT 0x10`
- `#define PIC_ICW4_8086 0x01`
- `#define PIC_ICW4_AUTO 0x02`
- `#define PIC_ICW4_BUF_SLAVE 0x08`
- `#define PIC_ICW4_BUF_MASTER 0x0C`

Functions

- void `pic_enable` (void)
Initialize/remap the legacy PICs for Voyager's interrupt layout.
- void `pic_remap_offsets` (uint8_t offset)
Remap PIC IRQ0 to a new IDT vector base.
- void `pic_mask_irq` (uint8_t irq)
Mask one legacy PIC IRQ line.
- void `pic_unmask_irq` (uint8_t irq)
Unmask one legacy PIC IRQ line.
- void `pic_send_eoi` (uint8_t irq)
Send end-of-interrupt acknowledgement for one legacy PIC IRQ.

7.83.1 Detailed Description

Legacy 8259 PIC programming interface.

Definition in file `pic.h`.

7.83.2 Macro Definition Documentation

7.83.2.1 PIC_EOI

```
#define PIC_EOI 0x20
```

Definition at line 15 of file `pic.h`.

7.83.2.2 PIC_ICW1_ICW4

```
#define PIC_ICW1_ICW4 0x01
```

Definition at line 17 of file `pic.h`.

7.83.2.3 PIC_ICW1_INIT

```
#define PIC_ICW1_INIT 0x10
```

Definition at line 21 of file `pic.h`.

7.83.2.4 PIC_ICW1_INTERVAL4

```
#define PIC_ICW1_INTERVAL4 0x04
```

Definition at line 19 of file [pic.h](#).

7.83.2.5 PIC_ICW1_LEVEL

```
#define PIC_ICW1_LEVEL 0x08
```

Definition at line 20 of file [pic.h](#).

7.83.2.6 PIC_ICW1_SINGLE

```
#define PIC_ICW1_SINGLE 0x02
```

Definition at line 18 of file [pic.h](#).

7.83.2.7 PIC_ICW4_8086

```
#define PIC_ICW4_8086 0x01
```

Definition at line 23 of file [pic.h](#).

7.83.2.8 PIC_ICW4_AUTO

```
#define PIC_ICW4_AUTO 0x02
```

Definition at line 24 of file [pic.h](#).

7.83.2.9 PIC_ICW4_BUF_MASTER

```
#define PIC_ICW4_BUF_MASTER 0x0C
```

Definition at line 26 of file [pic.h](#).

7.83.2.10 PIC_ICW4_BUF_SLAVE

```
#define PIC_ICW4_BUF_SLAVE 0x08
```

Definition at line 25 of file [pic.h](#).

7.83.2.11 PIC_MASTER

```
#define PIC_MASTER 0x20
```

Definition at line 9 of file [pic.h](#).

7.83.2.12 PIC_MASTER_COMMAND

```
#define PIC_MASTER_COMMAND PIC_MASTER
```

Definition at line 11 of file [pic.h](#).

7.83.2.13 PIC_MASTER_DATA

```
#define PIC_MASTER_DATA (PIC_MASTER + 1)
```

Definition at line 12 of file [pic.h](#).

7.83.2.14 PIC_SLAVE

```
#define PIC_SLAVE 0xA0
```

Definition at line 10 of file [pic.h](#).

7.83.2.15 PIC_SLAVE_COMMAND

```
#define PIC_SLAVE_COMMAND PIC_SLAVE
```

Definition at line 13 of file [pic.h](#).

7.83.2.16 PIC_SLAVE_DATA

```
#define PIC_SLAVE_DATA (PIC_SLAVE + 1)
```

Definition at line 14 of file [pic.h](#).

7.83.3 Function Documentation

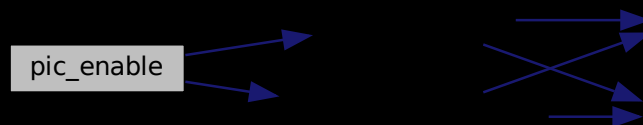
7.83.3.1 `pic_enable()`

```
void pic_enable (  
    void )
```

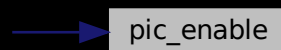
Initialize/remap the legacy PICs for Voyager's interrupt layout.

Definition at line 75 of file [pic.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.83.3.2 `pic_mask_irq()`

```
void pic_mask_irq (  
    uint8_t irq )
```

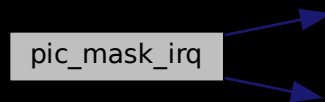
Mask one legacy PIC IRQ line.

Parameters

<i>irq</i>	IRQ number in the range handled by the cascaded PICs.
------------	---

Definition at line 5 of file [pic.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.83.3.3 pic_remap_offsets()

```
void pic_remap_offsets (
    uint8_t offset )
```

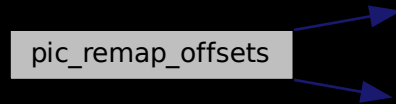
Remap PIC IRQ0 to a new IDT vector base.

Parameters

<i>offset</i>	Vector number corresponding to IRQ0 after remapping.
---------------	--

Definition at line 45 of file [pic.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.83.3.4 `pic_send_eoi()`

```
void pic_send_eoi (  
    uint8_t irq )
```

Send end-of-interrupt acknowledgement for one legacy PIC IRQ.

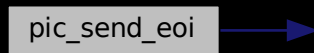
Parameters

<i>irq</i>	Hardware IRQ number, not an arbitrary IDT vector.
------------	---

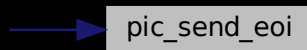
The generic interrupt dispatcher only sends PIC EOI for vectors belonging to the remapped hardware IRQ range. Dynamic/software vectors must not be passed through PIC acknowledgement merely because they used the same dispatcher.

Definition at line 68 of file `pic.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.83.3.5 pic_unmask_irq()

```
void pic_unmask_irq (  
    uint8_t irq )
```

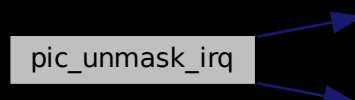
Unmask one legacy PIC IRQ line.

Parameters

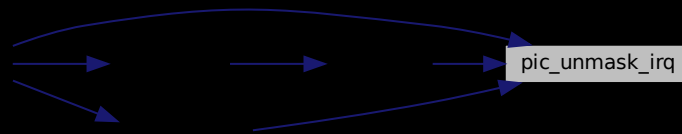
<i>irq</i>	IRQ number in the range handled by the cascaded PICs.
------------	---

Definition at line 25 of file [pic.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



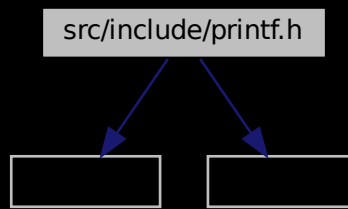
7.84 pic.h

```
00001
00006 #pragma once
00007 #include "stdint.h"
00008
00009 #define PIC_MASTER 0x20
00010 #define PIC_SLAVE 0xA0
00011 #define PIC_MASTER_COMMAND PIC_MASTER
00012 #define PIC_MASTER_DATA (PIC_MASTER + 1)
00013 #define PIC_SLAVE_COMMAND PIC_SLAVE
00014 #define PIC_SLAVE_DATA (PIC_SLAVE + 1)
00015 #define PIC_EOI 0x20
00016
00017 #define PIC_ICW1_ICW4 0x01
00018 #define PIC_ICW1_SINGLE 0x02
00019 #define PIC_ICW1_INTERVAL4 0x04
00020 #define PIC_ICW1_LEVEL 0x08
00021 #define PIC_ICW1_INIT 0x10
00022
00023 #define PIC_ICW4_8086 0x01
00024 #define PIC_ICW4_AUTO 0x02
00025 #define PIC_ICW4_BUF_SLAVE 0x08
00026 #define PIC_ICW4_BUF_MASTER 0x0C
00027
00031 void pic_enable(void);
00032
00037 void pic_remap_offsets(uint8_t offset);
00038
00043 void pic_mask_irq(uint8_t irq);
00044
00049 void pic_unmask_irq(uint8_t irq);
00050
00059 void pic_send_eoi(uint8_t irq);
```

7.85 src/include/printf.h File Reference

```
#include <stdarg.h>
#include <stddef.h>
```

Include dependency graph for printf.h:



This graph shows which files directly or indirectly include this file:



Macros

- `#define _PRINTF_H_`
- `#define printf printf_`
- `#define sprintf sprintf_`
- `#define snprintf snprintf_`
- `#define vsnprintf vsnprintf_`
- `#define vprintf vprintf_`

Functions

- `void _putchar` (char character)
- `int printf_` (const char *format,...)
- `int sprintf_` (char *buffer, const char *format,...)
- `int snprintf_` (char *buffer, size_t count, const char *format,...)
- `int vsnprintf_` (char *buffer, size_t count, const char *format, va_list va)
- `int vprintf_` (const char *format, va_list va)
- `int fctprintf` (void(*out)(char character, void *arg), void *arg, const char *format,...)

7.85.1 Macro Definition Documentation

7.85.1.1 _PRINTF_H_

```
#define _PRINTF_H_
```

Definition at line 35 of file `printf.h`.

7.85.1.2 printf

```
#define printf printf_
```

Tiny printf implementation You have to implement *putchar* if you use *printf()* To avoid conflicts with the regular *printf()* API it is overridden by macro defines and internal underscore-appended functions like *printf_()* are used

Parameters

<i>format</i>	A string that specifies the format of the output
---------------	--

Returns

The number of characters that are written into the array, not counting the terminating null character

Definition at line 61 of file *printf.h*.

7.85.1.3 snprintf

```
#define snprintf snprintf_
```

Tiny snprintf/vsnprintf implementation

Parameters

<i>buffer</i>	A pointer to the buffer where to store the formatted string
<i>count</i>	The maximum number of characters to store in the buffer, including a terminating null character
<i>format</i>	A string that specifies the format of the output
<i>va</i>	A value identifying a variable arguments list

Returns

The number of characters that COULD have been written into the buffer, not counting the terminating null character. A value equal or larger than count indicates truncation. Only when the returned value is non-negative and less than count, the string has been completely written.

Definition at line 87 of file *printf.h*.

7.85.1.4 sprintf

```
#define sprintf sprintf_
```

Tiny sprintf implementation Due to security reasons (buffer overflow) YOU SHOULD CONSIDER USING (V)SNPRINTF INSTEAD!

Parameters

<i>buffer</i>	A pointer to the buffer where to store the formatted string. MUST be big enough to store the output!
<i>format</i>	A string that specifies the format of the output

Returns

The number of characters that are WRITTEN into the buffer, not counting the terminating null character

Definition at line 73 of file `printf.h`.

7.85.1.5 `vprintf`

```
#define vprintf vprintf_
```

Tiny `vprintf` implementation

Parameters

<i>format</i>	A string that specifies the format of the output
<i>va</i>	A value identifying a variable arguments list

Returns

The number of characters that are WRITTEN into the buffer, not counting the terminating null character

Definition at line 99 of file `printf.h`.

7.85.1.6 `vsprintf`

```
#define vsprintf vsprintf_
```

Definition at line 88 of file `printf.h`.

7.85.2 Function Documentation

7.85.2.1 `_putchar()`

```
void _putchar (  
    char character )
```

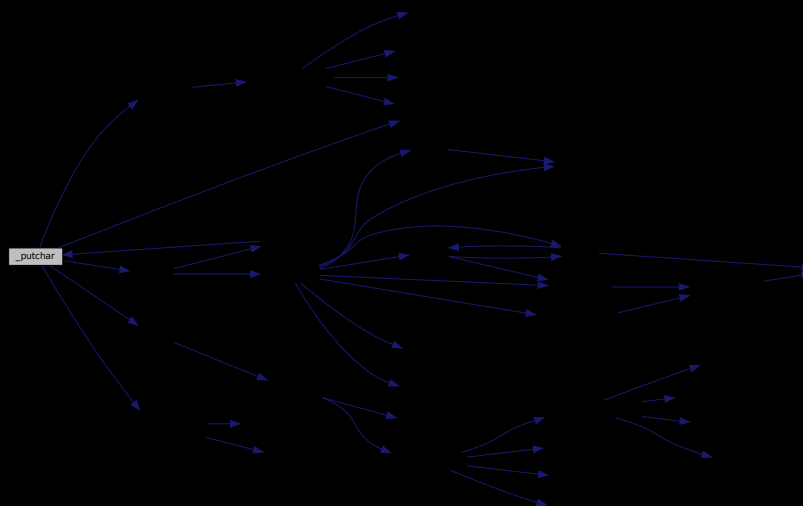
Output a character to a custom device like UART, used by the `printf()` function This function is declared here only. You have to write your custom implementation somewhere

Parameters

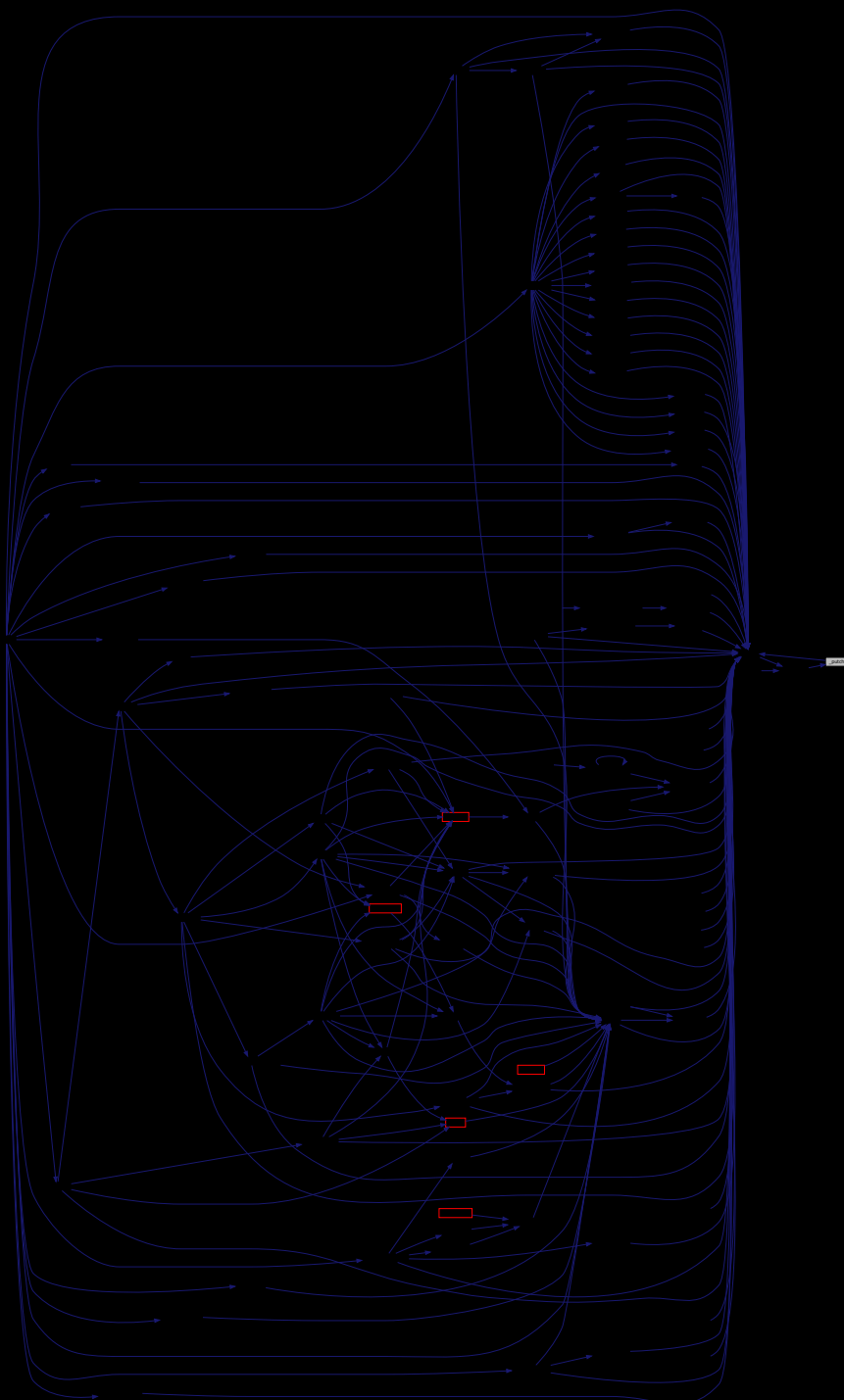
<i>character</i>	Character to output
------------------	---------------------

Definition at line 103 of file [putchar.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.85.2.2 fctprintf()

```
int fctprintf (
    void(*) (char character, void *arg) out,
```

```
void * arg,  
const char * format,  
... )
```

printf with output function You may use this as dynamic alternative to `printf()` with its fixed `_putchar()` output

Parameters

<i>out</i>	An output function which takes one character and an argument pointer
<i>arg</i>	An argument pointer for user data passed to output function
<i>format</i>	A string that specifies the format of the output

Returns

The number of characters that are sent to the output function, not counting the terminating null character

Definition at line 1279 of file printf.c.

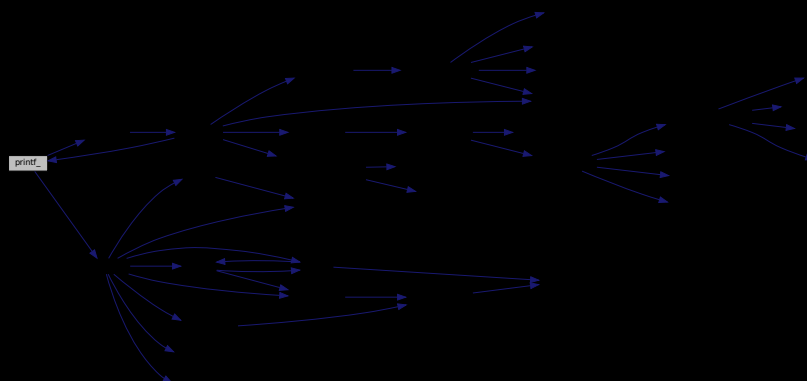
Here is the call graph for this function:

7.85.2.3 printf_()

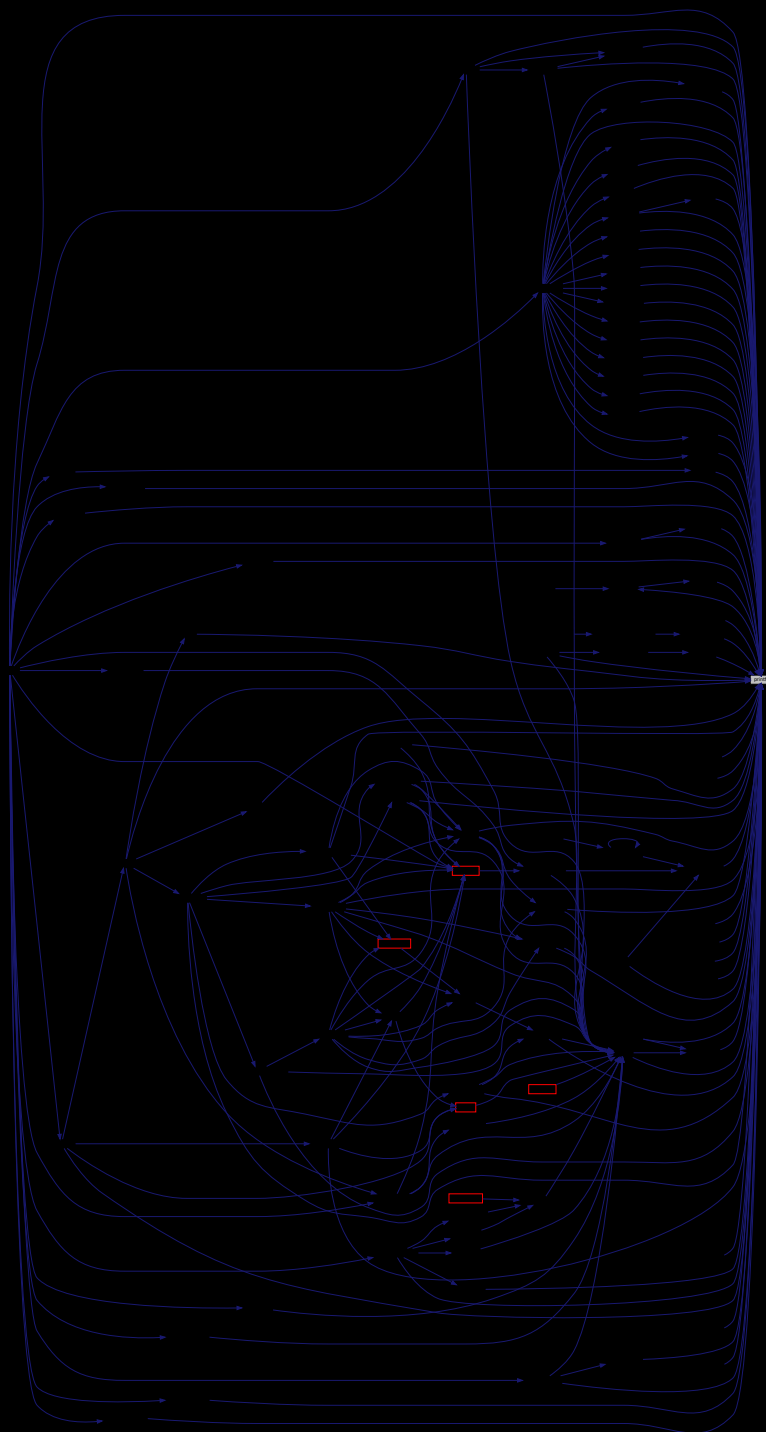
```
int printf_ (
    const char * format,
    ... )
```

Definition at line 1240 of file printf.c.

Here is the call graph for this function:



Here is the caller graph for this function:



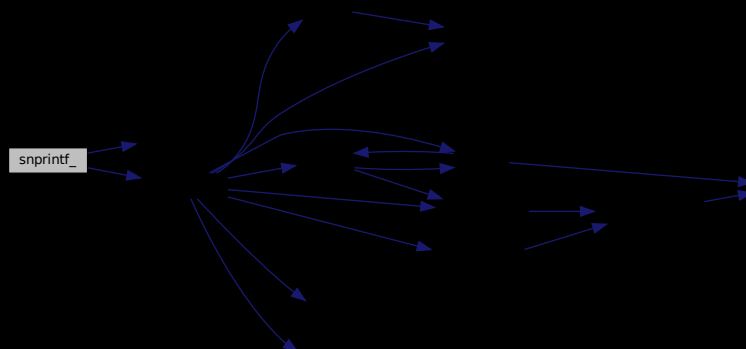
7.85.2.4 `snprintf_()`

```
int snprintf_ (  
    char * buffer,
```

```
size_t count,  
const char * format,  
... )
```

Definition at line 1259 of file [printf.c](#).

Here is the call graph for this function:

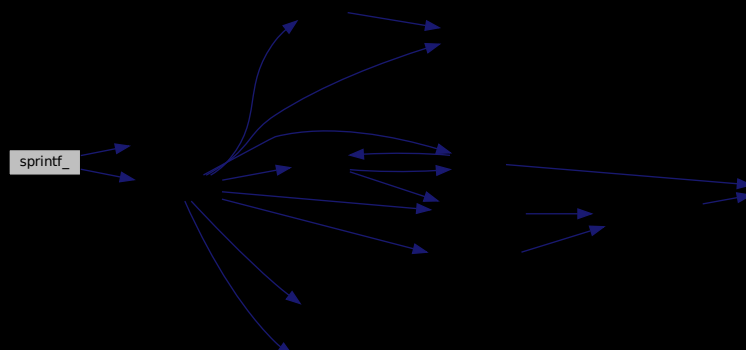


7.85.2.5 `sprintf_()`

```
int sprintf_ (  
    char * buffer,  
    const char * format,  
    ... )
```

Definition at line 1250 of file [printf.c](#).

Here is the call graph for this function:

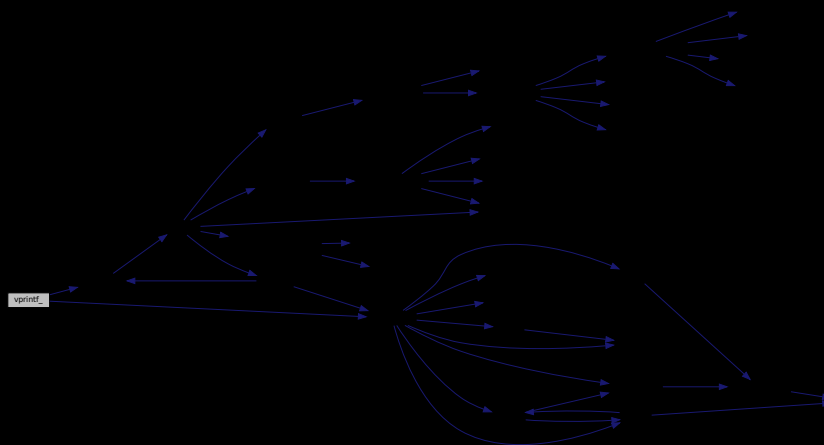


7.85.2.6 vprintf_()

```
int vprintf_ (  
    const char * format,  
    va_list va )
```

Definition at line 1268 of file [printf.c](#).

Here is the call graph for this function:

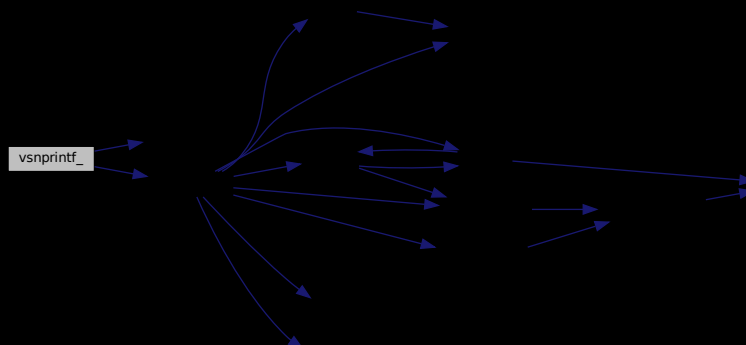


7.85.2.7 vsnprintf_()

```
int vsnprintf_ (  
    char * buffer,  
    size_t count,  
    const char * format,  
    va_list va )
```

Definition at line 1274 of file [printf.c](#).

Here is the call graph for this function:



7.86 printf.h

Go to the documentation of this file.

```

00001
00002 // \author (c) Marco Paland (info@paland.com)
00003 //           2014-2019, PALANDesign Hannover, Germany
00004 //
00005 // \license The MIT License (MIT)
00006 //
00007 // Permission is hereby granted, free of charge, to any person obtaining a copy
00008 // of this software and associated documentation files (the "Software"), to deal
00009 // in the Software without restriction, including without limitation the rights
00010 // to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00011 // copies of the Software, and to permit persons to whom the Software is
00012 // furnished to do so, subject to the following conditions:
00013 //
00014 // The above copyright notice and this permission notice shall be included in
00015 // all copies or substantial portions of the Software.
00016 //
00017 // THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00018 // IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00019 // FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00020 // AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00021 // LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00022 // OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
00023 // THE SOFTWARE.
00024 //
00025 // \brief Tiny printf, sprintf and snprintf implementation, optimized for speed
00026 // on
00027 //     embedded systems with a very limited resources.
00028 //     Use this instead of bloated standard/newlib printf.
00029 //     These routines are thread safe and reentrant.
00030 //
00031 //
00032
00033 #pragma once
00034 #ifndef _PRINTF_H_
00035 #define _PRINTF_H_
00036
00037 #include <stdarg.h>
00038 #include <stddef.h>
00039
00040 #ifdef __cplusplus
00041 extern "C"
00042 {
00043 #endif
00044
00045     void _putchar(char character);
00046
00047 #define printf printf_
00048     int printf_(const char *format, ...);
00049
00050 #define sprintf sprintf_
00051     int sprintf_(char *buffer, const char *format, ...);
00052
00053 #define snprintf snprintf_
00054 #define vsnprintf vsnprintf_
00055     int snprintf_(char *buffer, size_t count, const char *format, ...);
00056     int vsnprintf_(char *buffer, size_t count, const char *format, va_list va);
00057
00058 #define vprintf vprintf_
00059     int vprintf_(const char *format, va_list va);
00060
00061     int fctprintf(void (*out)(char character, void *arg),
00062                  void *arg,
00063                  const char *format,
00064                  ...);
00065
00066 #ifdef __cplusplus
00067 }
00068 #endif
00069 #endif // _PRINTF_H_

```

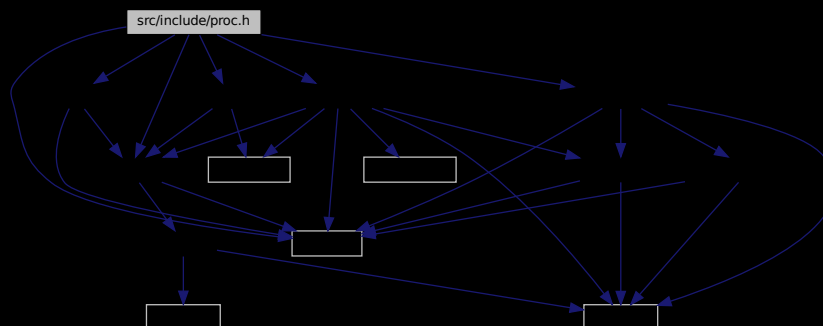
7.87 src/include/proc.h File Reference

```

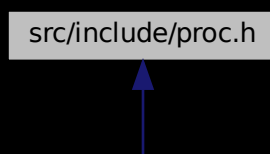
#include <stdint.h>
#include "global_defs.h"
#include "interrupts.h"

```

```
#include "lib/hashmap.h"
#include "lib/vector.h"
#include "lock.h"
Include dependency graph for proc.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- void `proc_yield` ()

7.87.1 Function Documentation

7.87.1.1 `proc_yield()`

```
void proc_yield ( )
```

7.88 proc.h

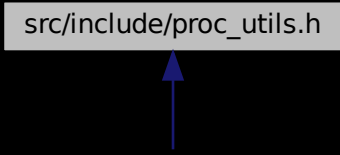
Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <stdint.h>
00004 // #include <signal.h>
00005 #include "global_defs.h"
00006 #include "interrupts.h"
00007 #include "lib/hashmap.h"
00008 #include "lib/vector.h"
00009 #include "lock.h"
00010
00011 extern void proc_yield();
00012
00013 // const int PROCESS_STACK_SIZE = 0x4000;
```

7.89 src/include/proc_utils.h File Reference

Small process/scheduler diagnostic tasks.

This graph shows which files directly or indirectly include this file:



```
graph BT
    A[ ] --> B[src/include/proc_utils.h]
```

Functions

- void **soak** (void)
Cooperative scheduler soak worker.

7.89.1 Detailed Description

Small process/scheduler diagnostic tasks.

Copyright (c) 2026 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Definition in file **proc_utils.h**.

7.89.2 Function Documentation

7.89.2.1 soak()

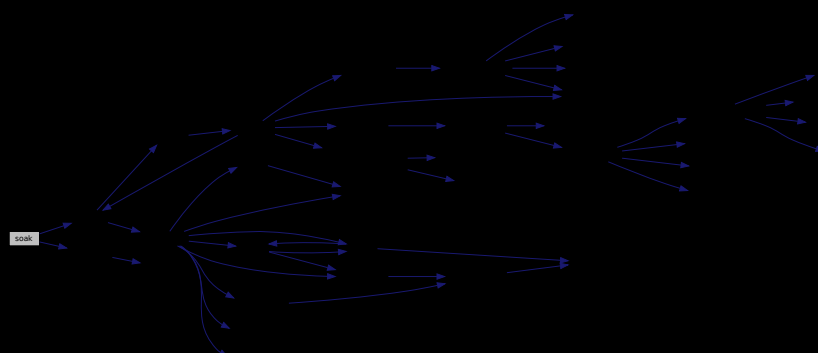
```
void soak (
    void )
```

Cooperative scheduler soak worker.

The VSH `soak` command creates another process running this entry point. Workers identify themselves by PID, repeatedly prove liveness, and yield back into the cooperative round-robin scheduler. Because 0.0.5 has no task-exit path, a soak worker is intentionally long-lived until the system halts.

Definition at line 13 of file `Proc_utils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.90 proc_utils.h

[Go to the documentation of this file.](#)

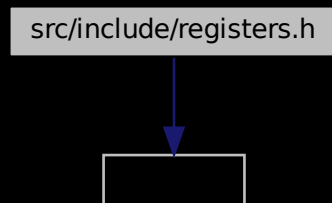
```
00001
00013 #pragma once
00014
00023 void soak(void);
```

7.91 src/include/registers.h File Reference

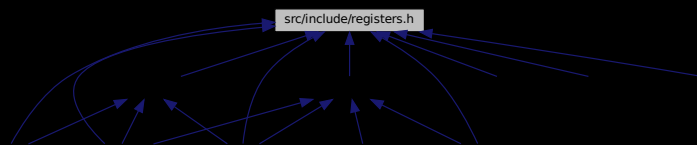
Low-level control-register, stack-pointer, and MSR access helpers.

```
#include <stdint.h>
```

Include dependency graph for registers.h:



This graph shows which files directly or indirectly include this file:



Functions

- uint64_t **readCRO** (void)
Read CR0.
- uint64_t **readCR3** (void)
Read CR3, the active page-table-root physical address.
- uint64_t **readCR2** (void)
Read CR2, commonly used to obtain the page-fault linear address.
- uint64_t **readRSP** (void)
Read the current stack pointer.
- uint64_t **readCR4** (void)
Read CR4.
- void **writeCR4** (uint64_t value)
Write CR4.
- void **writeCRO** (uint64_t value)
Write CR0.
- void **writeCR3** (uint64_t value)
Replace the active CR3 value.
- void **writeMSR** (uint64_t msr, uint64_t value)
Write a 64-bit model-specific register.
- uint64_t **rdmsr** (uint32_t msr)
Read a 64-bit model-specific register.

7.91.1 Detailed Description

Low-level control-register, stack-pointer, and MSR access helpers.

Definition in file [registers.h](#).

7.91.2 Function Documentation

7.91.2.1 rdmsr()

```
uint64_t rdmsr (  
    uint32_t msr )
```

Read a 64-bit model-specific register.

Parameters

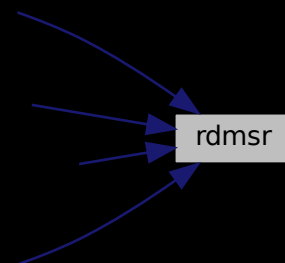
<i>msr</i>	MSR index.
------------	------------

Returns

Current 64-bit MSR value.

Definition at line [14](#) of file [registers.c](#).

Here is the caller graph for this function:



7.91.2.2 readCR2()

```
uint64_t readCR2 (  
    void )
```

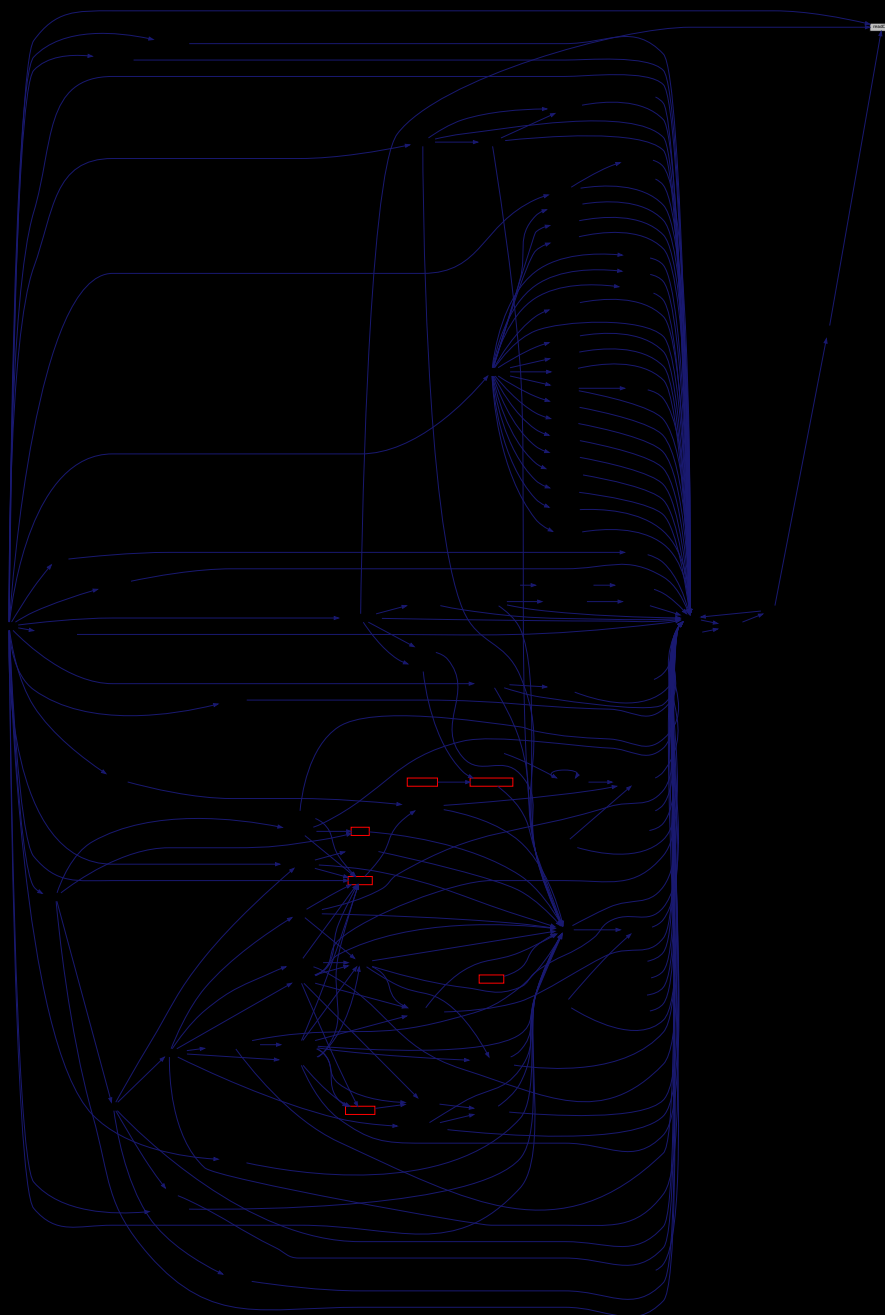
Read CR2, commonly used to obtain the page-fault linear address.

7.91.2.3 readCR3()

```
uint64_t readCR3 (  
    void )
```

Read CR3, the active page-table-root physical address.

Here is the caller graph for this function:

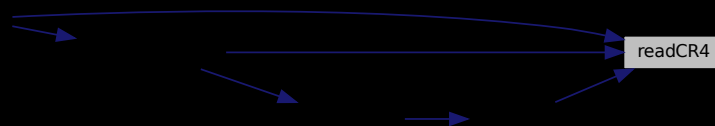


7.91.2.4 readCR4()

```
uint64_t readCR4 (  
    void )
```

Read CR4.

Here is the caller graph for this function:

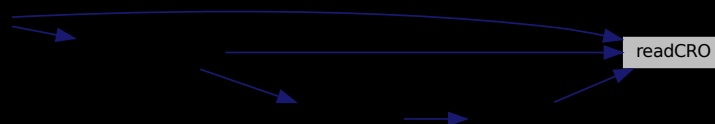


7.91.2.5 `readCRO()`

```
uint64_t readCRO (  
    void )
```

Read CR0.

Here is the caller graph for this function:



7.91.2.6 `readRSP()`

```
uint64_t readRSP (  
    void )
```

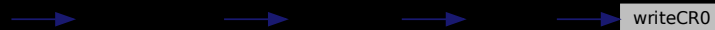
Read the current stack pointer.

7.91.2.7 writeCR0()

```
void writeCR0 (
    uint64_t value )
```

Write CR0.

Here is the caller graph for this function:



7.91.2.8 writeCR3()

```
void writeCR3 (
    uint64_t value )
```

Replace the active CR3 value.

Parameters

<i>value</i>	Physical address/value to load into CR3.
--------------	--

Warning

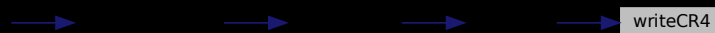
Loading CR3 changes the address space immediately. The caller must ensure the executing code, active stack, required descriptor/interrupt state, and subsequent data references remain mapped by the new tables.

7.91.2.9 writeCR4()

```
void writeCR4 (
    uint64_t value )
```

Write CR4.

Here is the caller graph for this function:



7.91.2.10 writeMSR()

```
void writeMSR (
    uint64_t msr,
    uint64_t value )
```

Write a 64-bit model-specific register.

Parameters

<i>msr</i>	MSR index.
<i>value</i>	Value to write.

Definition at line 4 of file [registers.c](#).

7.92 registers.h

[Go to the documentation of this file.](#)

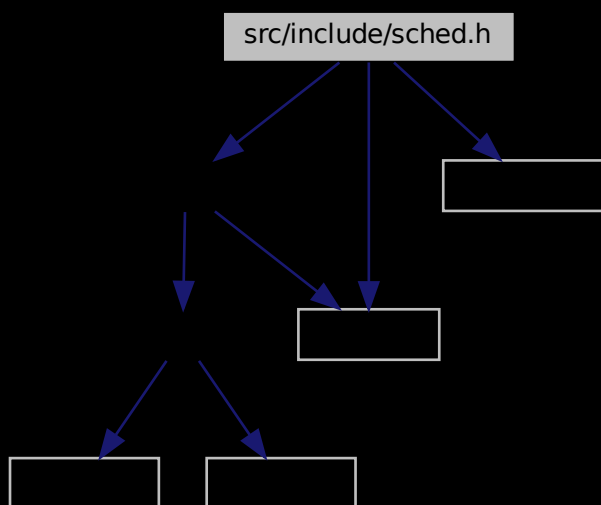
```
00001
00006 #pragma once
00007 #include <stdint.h>
00008
00010 extern uint64_t readCRO(void);
00012 extern uint64_t readCR3(void);
00014 extern uint64_t readCR2(void);
00016 extern uint64_t readRSP(void);
00018 extern uint64_t readCR4(void);
00019
00021 extern void writeCR4(uint64_t value);
00023 extern void writeCR0(uint64_t value);
00032 extern void writeCR3(uint64_t value);
00033
00039 void writeMSR(uint64_t msr, uint64_t value);
00040
00046 uint64_t rdmsr(uint32_t msr);
```

7.93 src/include/sched.h File Reference

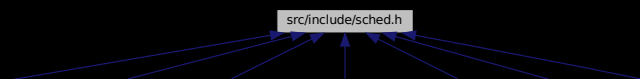
Cooperative process scheduler interface.

```
#include "global_defs.h"
#include <stdbool.h>
#include <stdint.h>
```

Include dependency graph for sched.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `process`
Minimal 0.0.5 process control block.

Macros

- #define `_SCHED_H`
- #define `SYSCALL_YIELD` 1
- #define `SYSCALL_FORK` 2
- #define `SYSCALL_EXIT` 3

Typedefs

- typedef struct `process` `process_t`
Minimal 0.0.5 process control block.

Enumerations

- enum `proc_state_t` { `PROC_READY` , `PROC_RUNNING` , `PROC_BLOCKED` }
Scheduler-visible process state.

Functions

- void `switch_to` (`process_t` *next)
Switch cooperatively from the current process to next.
- `process_t` * `create_process` (`void`(*entry)(`void`))
Create a kernel process and append it to the round-robin run queue.
- void `schedule` (`void`)
Yield the CPU cooperatively to the next process in the circular queue.
- void `init_scheduler` (`void`)
Reset scheduler bookkeeping to its pre-dispatch state.
- void `scheduler_start` (`void`)
Begin execution of the initialized run queue.

Variables

- bool `allow_sched`
Gate used by the PIT path for scheduler preemption.
- `process_t` * `current`
Process whose stack/context is currently executing, or NULL before the first scheduler dispatch.

7.93.1 Detailed Description

Cooperative process scheduler interface.

VoyagerOS64 0.0.5 deliberately uses cooperative round-robin scheduling. Tasks run until they call `schedule()`. IRQ/PIT-driven preemption is not part of the qualified 0.0.5 scheduler path.

Definition in file `sched.h`.

7.93.2 Macro Definition Documentation

7.93.2.1 _SCHED_H

```
#define _SCHED_H
```

Definition at line 13 of file `sched.h`.

7.93.2.2 SYSCALL_EXIT

```
#define SYSCALL_EXIT 3
```

Definition at line 23 of file `sched.h`.

7.93.2.3 SYSCALL_FORK

```
#define SYSCALL_FORK 2
```

Definition at line 22 of file `sched.h`.

7.93.2.4 SYSCALL_YIELD

```
#define SYSCALL_YIELD 1
```

Reserved software-syscall identifiers. No fork/exit implementation is provided by the 0.0.5 scheduler merely because these values exist.

Definition at line 21 of file `sched.h`.

7.93.3 Typedef Documentation

7.93.3.1 process_t

```
typedef struct process process_t
```

Minimal 0.0.5 process control block.

Processes currently share the kernel address space. Each process owns a separately allocated kernel stack; `rsp` records the saved cooperative context for `switch_to()`. The `next` pointer forms the circular run queue.

Warning

`sched_0l.asm` depends on the layout of this structure, in particular `rsp` being at byte offset 8. Keep the assembly and C definition in sync if fields are reordered or resized.

7.93.4 Enumeration Type Documentation

7.93.4.1 proc_state_t

```
enum proc_state_t
```

Scheduler-visible process state.

Enumerator

PROC_READY	Runnable but not currently executing.
PROC_RUNNING	Currently selected process.
PROC_BLOCKED	Not runnable until some future wake-up mechanism makes it ready.

Definition at line 26 of file [sched.h](#).

7.93.5 Function Documentation

7.93.5.1 create_process()

```
process_t * create_process (
    void(*) (void) entry )
```

Create a kernel process and append it to the round-robin run queue.

Parameters

<i>entry</i>	void(void) entry point for the new process.
--------------	---

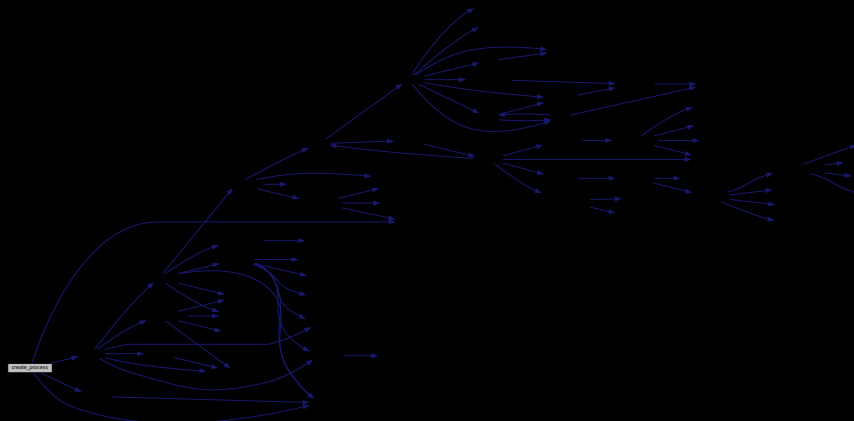
Returns

Pointer to the new process control block.

A 16 KiB stack is allocated from kmalloc and initialized with a synthetic context suitable for [switch_to\(\)](#). A task that returns from *entry* is treated as a kernel error in 0.0.5 because task exit/reaping is not implemented yet.

Definition at line 40 of file [sched.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.93.5.2 `init_scheduler()`

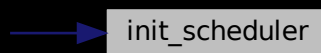
```
void init_scheduler (  
    void )
```

Reset scheduler bookkeeping to its pre-dispatch state.

Call once during kernel bring-up before creating the initial tasks.

Definition at line 31 of file `sched.c`.

Here is the caller graph for this function:



7.93.5.3 `schedule()`

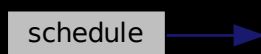
```
void schedule (  
    void )
```

Yield the CPU cooperatively to the next process in the circular queue.

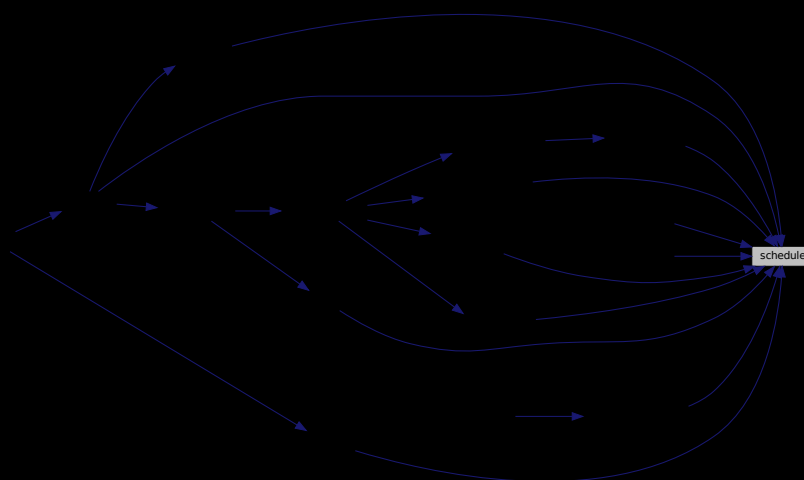
Calling `schedule()` is the 0.0.5 task-yield primitive. With zero runnable tasks it is a no-op; with only the current task runnable it returns without switching.

Definition at line 108 of file `sched.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.93.5.4 scheduler_start()

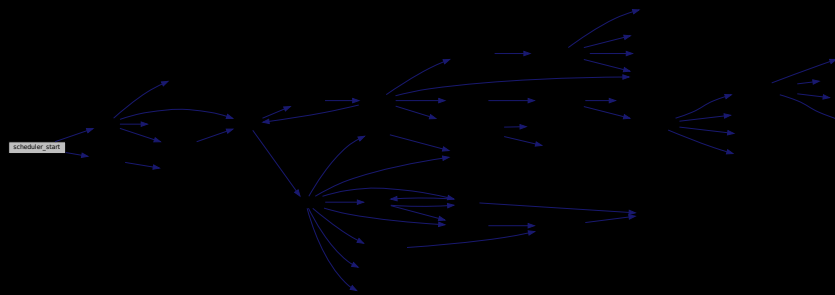
```
void scheduler_start (
    void )
```

Begin execution of the initialized run queue.

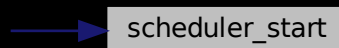
The run queue must contain at least one process and `current` must still be NULL. This function performs the initial dispatch and is not expected to return.

Definition at line 128 of file `sched.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.93.5.5 switch_to()

```
void switch_to (
    process_t * next )
```

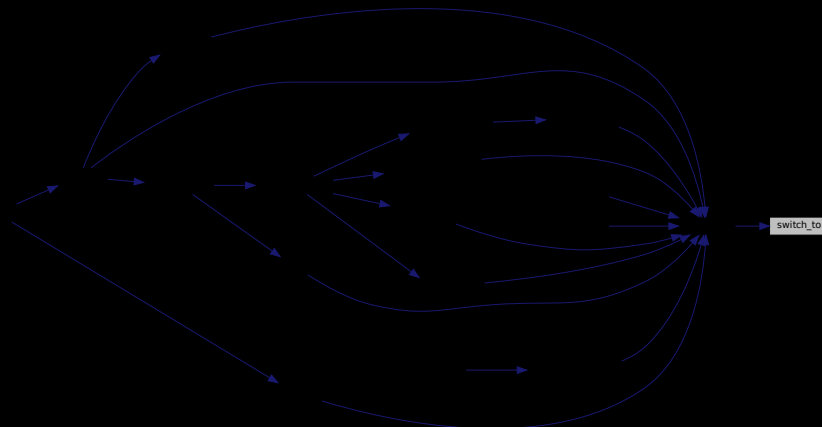
Switch cooperatively from the current process to `next`.

Parameters

<i>next</i>	Process whose saved stack context should be restored.
-------------	---

The low-level implementation saves/restores the SysV AMD64 callee-saved registers used by the cooperative C-call boundary. It is not an interrupt frame switch and must not be treated as one. Here is the caller graph for this

function:



7.93.6 Variable Documentation

7.93.6.1 allow_sched

```
bool allow_sched [extern]
```

Gate used by the PIT path for scheduler preemption.

This remains false in the qualified 0.0.5 cooperative design. Setting it to true does not by itself make `switch_to()` safe for IRQ-time preemption.

Definition at line 15 of file `sched.c`.

7.93.6.2 current

```
process_t* current [extern]
```

Process whose stack/context is currently executing, or NULL before the first scheduler dispatch.

Definition at line 16 of file `sched.c`.

7.94 sched.h

Go to the documentation of this file.

```

00001
00010 #pragma once
00011
00012 #ifndef _SCHED_H
00013 #define _SCHED_H
00014
00015 #include "global_defs.h"
00016 #include <stdbool.h>
00017 #include <stdint.h>
00018
00021 #define SYSCALL_YIELD 1
00022 #define SYSCALL_FORK 2
00023 #define SYSCALL_EXIT 3
00024
00026 typedef enum
00027 {
00029     PROC_READY,
00031     PROC_RUNNING,
00033     PROC_BLOCKED
00034 } proc_state_t;
00036
00048 typedef struct process
00049 {
00051     int pid;
00052
00054     uint64_t *rsp;
00055
00057     proc_state_t state;
00058
00060     struct process *next;
00061 } process_t;
00062
00069 extern bool allow_sched;
00070
00073 extern process_t *current;
00074
00083 void switch_to(process_t *next);
00084
00094 process_t *create_process(void (*entry)(void));
00095
00103 void schedule(void);
00104
00110 void init_scheduler(void);
00111
00119 void scheduler_start(void);
00120
00121 #endif

```

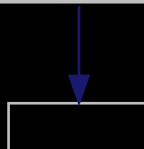
7.95 src/include/serial.h File Reference

Serial-console and timestamped diagnostic output helpers.

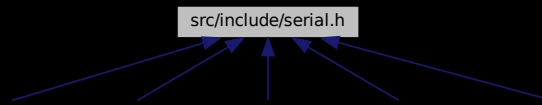
```
#include <stdint.h>
```

Include dependency graph for serial.h:

src/include/serial.h



This graph shows which files directly or indirectly include this file:



Macros

- `#define SERIAL_H`

Functions

- void `serial_debug` (char c)
Emit one character through the low-level debug serial path.
- uint64_t `get_ts` (void)
Read the timestamp source used by serial diagnostics.
- void `serial_print` (const char *str)
Write a NUL-terminated string to the serial console.
- void `serial_print_line` (const char *str)
Write a NUL-terminated string followed by a line ending.
- void `serial_print_with_ts` (const char *msg)
Write a diagnostic message prefixed with the current timestamp.

7.95.1 Detailed Description

Serial-console and timestamped diagnostic output helpers.

Definition in file `serial.h`.

7.95.2 Macro Definition Documentation

7.95.2.1 SERIAL_H

```
#define SERIAL_H
```

Definition at line 8 of file `serial.h`.

7.95.3 Function Documentation

7.95.3.1 get_ts()

```
uint64_t get_ts (  
    void )
```

Read the timestamp source used by serial diagnostics.

Returns

Current diagnostic timestamp value.

Here is the caller graph for this function:



7.95.3.2 serial_debug()

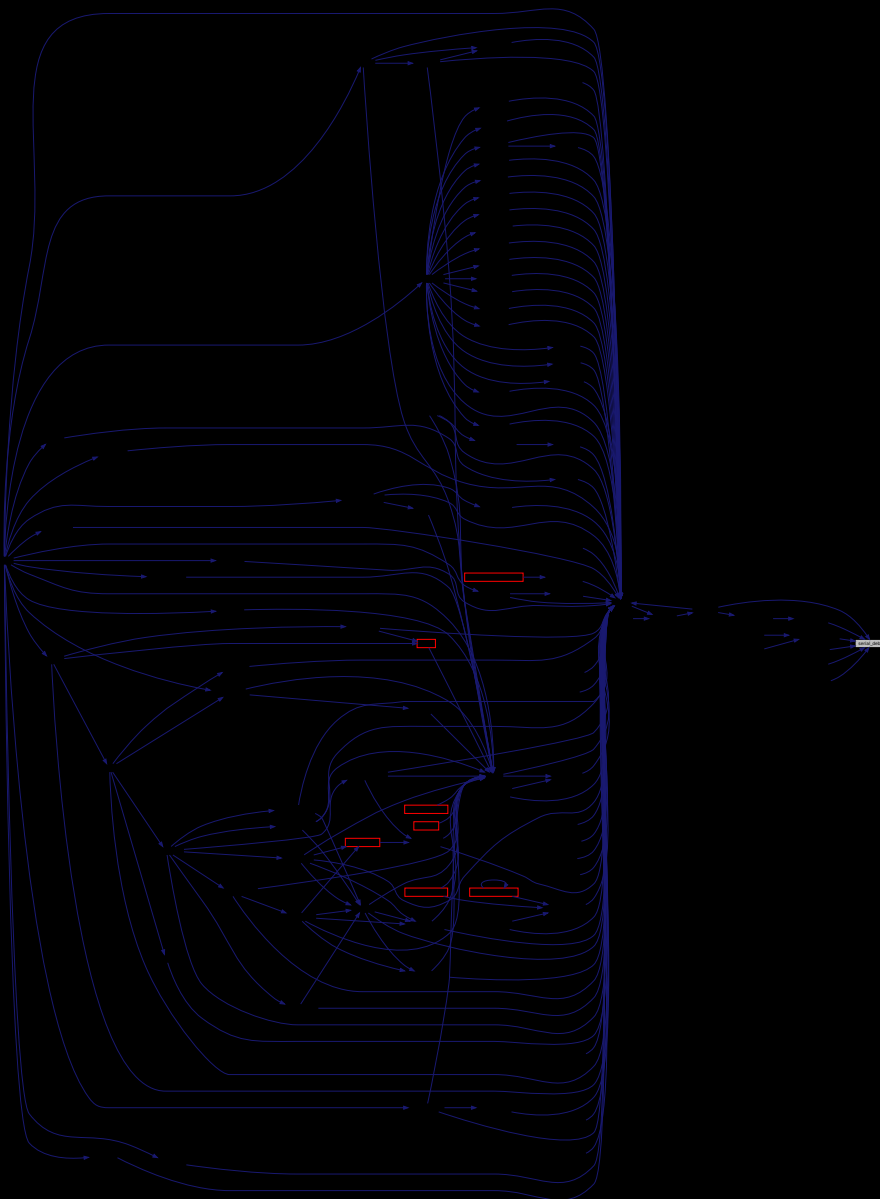
```
void serial_debug (  
    char c )
```

Emit one character through the low-level debug serial path.

Parameters

<i>c</i>	Character to transmit.
----------	------------------------

This primitive is useful during early boot and for diagnostics that must not depend on framebuffer-terminal state. Here is the caller graph for this function:



7.95.3.3 serial_print()

```
void serial_print (  
    const char * str )
```

Write a NUL-terminated string to the serial console.

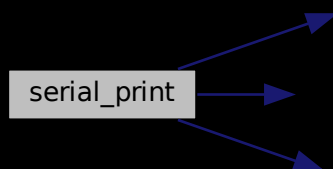
Write a NUL-terminated string to the serial console.

Parameters

<i>str</i>	String to print.
------------	------------------

Definition at line 9 of file [serial.c](#).

Here is the call graph for this function:



7.95.3.4 serial_print_line()

```
void serial_print_line (  
    const char * str )
```

Write a NUL-terminated string followed by a line ending.

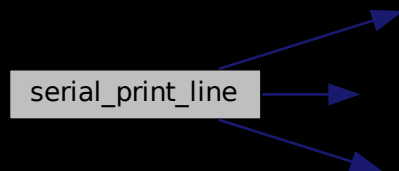
Write a NUL-terminated string followed by a line ending.

Parameters

<i>str</i>	String to print.
------------	------------------

Definition at line 44 of file [serial.c](#).

Here is the call graph for this function:



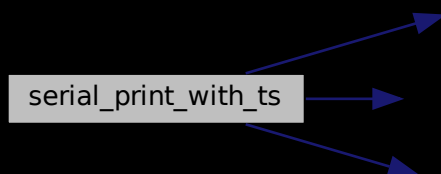
7.95.3.5 `serial_print_with_ts()`

```
void serial_print_with_ts (  
    const char * msg )
```

Write a diagnostic message prefixed with the current timestamp.

Definition at line 24 of file `serial.c`.

Here is the call graph for this function:



7.96 serial.h

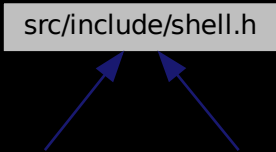
Go to the documentation of this file.

```
00001
00006 #pragma once
00007 #ifndef SERIAL_H
00008 #define SERIAL_H
00009
00010 #include <stdint.h>
00011
00019 extern void serial_debug(char c);
00020
00025 extern uint64_t get_ts(void);
00026
00028 void serial_print(const char *str);
00030 void serial_print_line(const char *str);
00032 void serial_print_with_ts(const char *msg);
00033
00034 #endif
```

7.97 src/include/shell.h File Reference

Voyager shell (VSH) public interface.

This graph shows which files directly or indirectly include this file:



src/include/shell.h

Macros

- #define `_SHELL_H`
- #define `VSH_CMD_BUFFER_SIZE` 100
- #define `VSH_VERSION` "0.0.1"

Functions

- void `vsh_loop` (void)
Main VSH cooperative task.
- char * `vsh_readline` (void)
Read one line from the keyboard FIFO.
- void `cmd_parser` (const char *str)
Dispatch one NUL-terminated VSH command line.

7.97.1 Detailed Description

Voyager shell (VSH) public interface.

VSH runs as an ordinary cooperative scheduler task. While waiting for keyboard input it yields, allowing the rest of the run queue to continue.

Definition in file [shell.h](#).

7.97.2 Macro Definition Documentation

7.97.2.1 `_SHELL_H`

```
#define _SHELL_H
```

Definition at line 12 of file [shell.h](#).

7.97.2.2 `VSH_CMD_BUFFER_SIZE`

```
#define VSH_CMD_BUFFER_SIZE 100
```

Initial command-line allocation/growth quantum in bytes.

Definition at line 15 of file [shell.h](#).

7.97.2.3 `VSH_VERSION`

```
#define VSH_VERSION "0.0.1"
```

Current shell-interface version.

Definition at line 17 of file [shell.h](#).

7.97.3 Function Documentation

7.97.3.1 `cmd_parser()`

```
void cmd_parser (  
    const char * str )
```

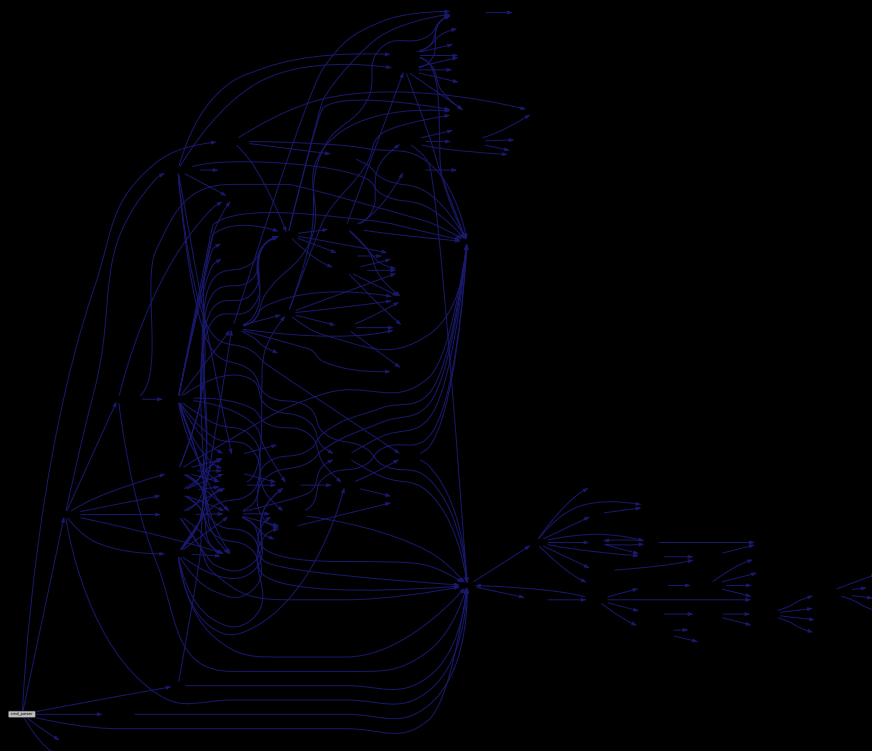
Dispatch one NUL-terminated VSH command line.

Parameters

<i>str</i>	Command string to parse.
------------	--------------------------

Definition at line 110 of file [shell.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.97.3.2 vsh_loop()

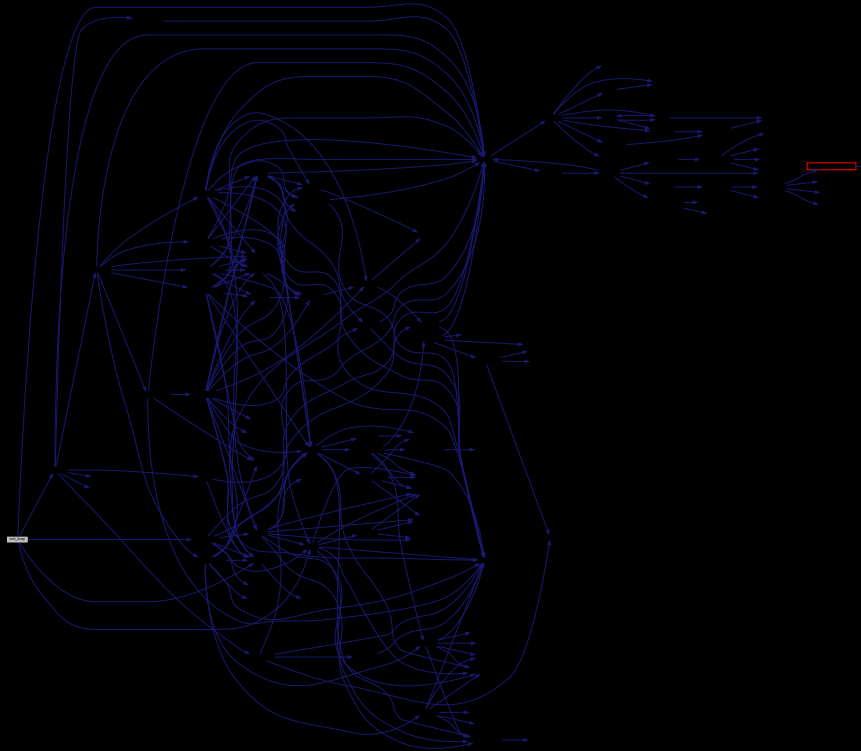
```
void vsh_loop (  
    void )
```

Main VSH cooperative task.

Repeatedly prints a prompt, reads a line from the keyboard FIFO, dispatches the command, frees the line buffer, and yields to the scheduler.

Definition at line 32 of file `shell.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.97.3.3 `vsh_readline()`

```
char * vsh_readline (  
    void )
```

Read one line from the keyboard FIFO.

Returns

Newly allocated NUL-terminated command string, or NULL if allocation fails.

The buffer is owned by the caller and must be released with `kfree()`. The function yields with `schedule()` whenever no keyboard character is available.

Definition at line 49 of file `shell.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.98 shell.h

[Go to the documentation of this file.](#)

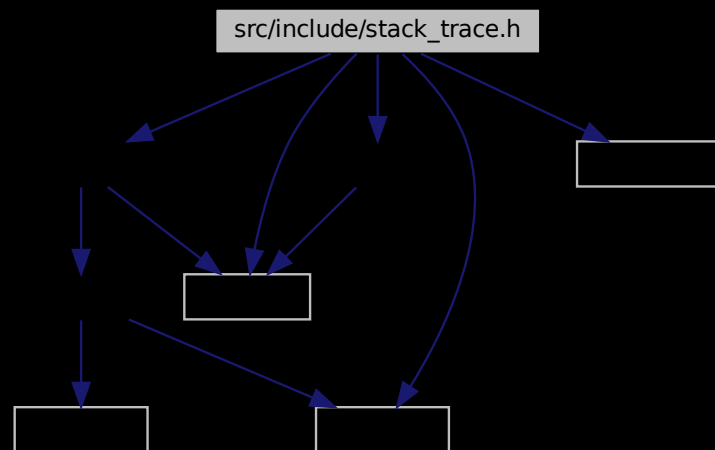
```
00001
00009 #pragma once
00010
00011 #ifndef _SHELL_H
00012 #define _SHELL_H
00013
00015 #define VSH_CMD_BUFFER_SIZE 100
00017 #define VSH_VERSION "0.0.1"
00018
00025 void vsh_loop(void);
00026
00035 char *vsh_readline(void);
00036
00041 void cmd_parser(const char *str);
00042
00043 #endif
```

7.99 src/include/stack_trace.h File Reference

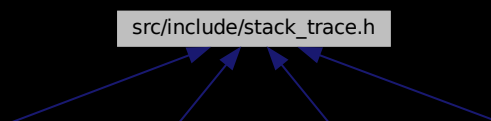
Stack-size reporting, frame walking, and diagnostic stack dumps.

```
#include "global_defs.h"
#include "limine.h"
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
```

Include dependency graph for stack_trace.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `stack_return`
Collected stack-return-address array and its populated length.
- struct `stack_frame`
Frame-pointer chain node used by the C stack walker.

Macros

- `#define _STACK_TRACE_H`

Functions

- void `stack_trace_asm` (uint64_t max_size, bool stop)
Assembly-assisted stack trace entry point.
- void `print_stack_size` (void)
Print the kernel stack size reported by Limine.
- void `stack_trace` (uint64_t max_frames)
Walk and print up to `max_frames` frame-pointer entries.
- void `dump_hex` (const void *data, size_t size)
Emit a hexadecimal dump of an arbitrary byte region.
- void `stack_dump_asm` (uint64_t leave, uint64_t trace_size)
Low-level stack-dump assembly helper.
- void `stack_dump` (void)
Emit the default diagnostic stack dump.
- void `stack_dump_recursive` (uint64_t max_frames)
Emit a recursive/frame-oriented stack dump.

Variables

- volatile struct `limine_stack_size_request` `stack_req`

7.99.1 Detailed Description

Stack-size reporting, frame walking, and diagnostic stack dumps.

Definition in file `stack_trace.h`.

7.99.2 Macro Definition Documentation

7.99.2.1 _STACK_TRACE_H

```
#define _STACK_TRACE_H
```

Definition at line 9 of file `stack_trace.h`.

7.99.3 Function Documentation

7.99.3.1 dump_hex()

```
void dump_hex (  
    const void * data,  
    size_t size )
```

Emit a hexadecimal dump of an arbitrary byte region.

Here is the caller graph for this function:



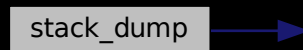
7.99.3.3 `stack_dump()`

```
void stack_dump (
    void )
```

Emit the default diagnostic stack dump.

Definition at line 56 of file `stack_trace.c`.

Here is the call graph for this function:



7.99.3.4 `stack_dump_asm()`

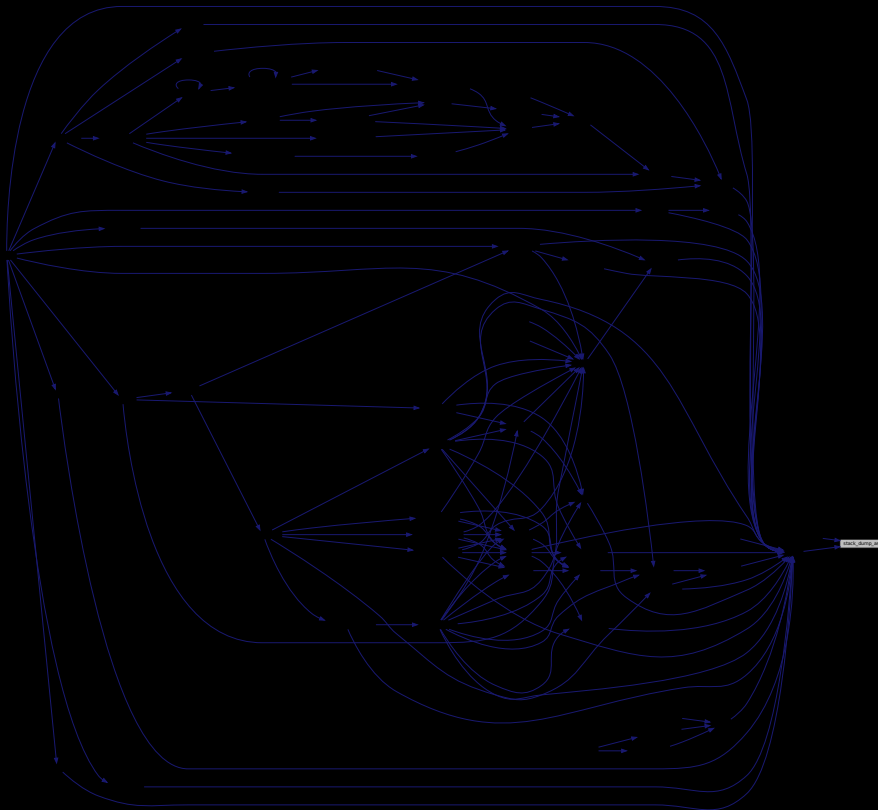
```
void stack_dump_asm (
    uint64_t leave,
    uint64_t trace_size )
```

Low-level stack-dump assembly helper.

Parameters

<i>leave</i>	Historical leave/return behavior selector.
<i>trace_size</i>	Requested trace extent.

Here is the caller graph for this function:



7.99.3.5 stack_dump_recursive()

```
void stack_dump_recursive (  
    uint64_t max_frames )
```

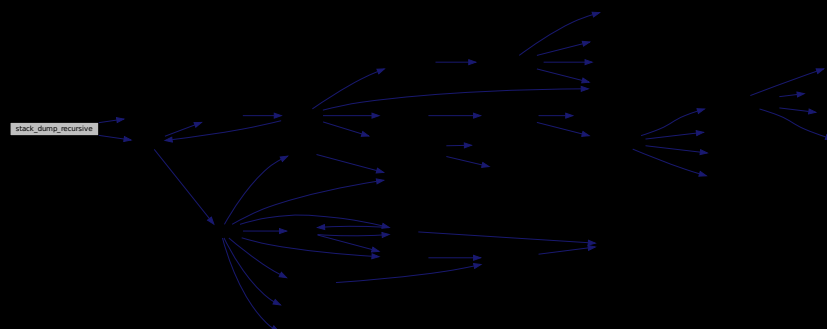
Emit a recursive/frame-oriented stack dump.

Parameters

<i>max_frames</i>	Maximum number of frames to inspect.
-------------------	--------------------------------------

Definition at line 63 of file [stack_trace.c](#).

Here is the call graph for this function:



7.99.3.6 `stack_trace()`

```
void stack_trace (  
    uint64_t max_frames )
```

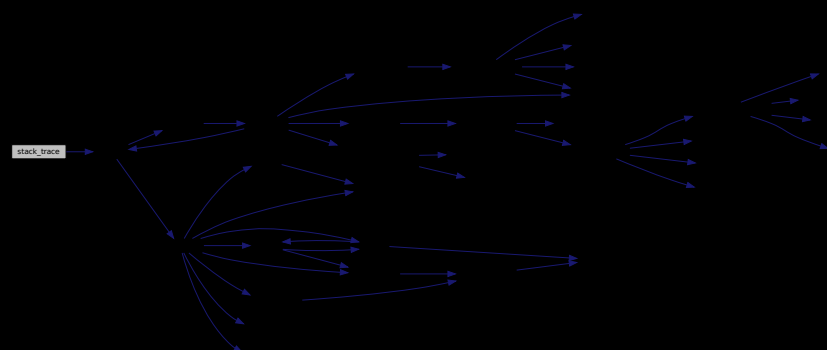
Walk and print up to `max_frames` frame-pointer entries.

Parameters

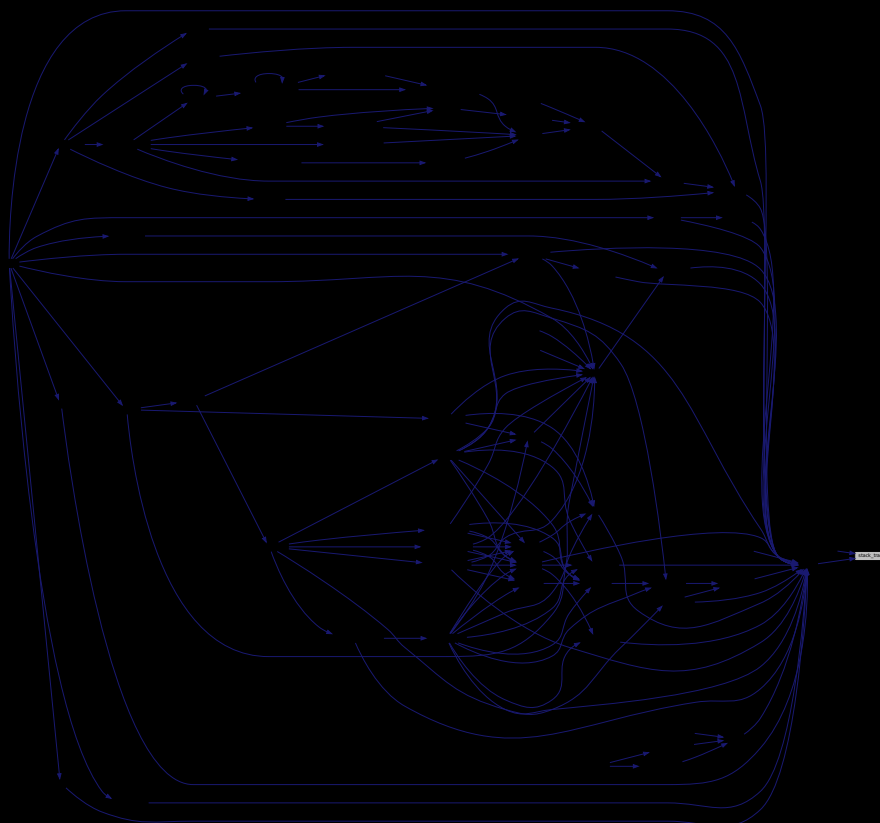
<code>max_frames</code>	Maximum number of frames to report.
-------------------------	-------------------------------------

Definition at line 35 of file `stack_trace.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.99.3.7 `stack_trace_asm()`

```
void stack_trace_asm (
    uint64_t max_size,
    bool stop )
```

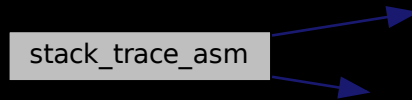
Assembly-assisted stack trace entry point.

Parameters

<i>max_size</i>	Maximum number/size of entries requested by the implementation.
<i>stop</i>	Whether the helper should stop after producing the trace.

Definition at line 14 of file [stack_trace.c](#).

Here is the call graph for this function:



7.99.4 Variable Documentation

7.99.4.1 stack_req

```
volatile struct limine_stack_size_request stack_req [extern]
```

Limine request used to learn the boot-provided kernel stack size.

7.100 stack_trace.h

[Go to the documentation of this file.](#)

```

00001
00006 #pragma once
00007
00008 #ifndef _STACK_TRACE_H
00009 #define _STACK_TRACE_H
00010 #include "global_defs.h"
00011 #include "limine.h"
00012 #include <stdbool.h>
00013 #include <stddef.h>
00014 #include <stdint.h>
00015
00017 extern volatile struct limine_stack_size_request stack_req;
00018
00020 struct stack_return
00021 {
00022     uint64_t *addresses;
00023     uint64_t array_size;
00024 };
00025
00027 struct stack_frame
00028 {
00029     struct stack_frame *rbp;
00030     uint64_t rip;
00031 };
00032
00038 void stack_trace_asm(uint64_t max_size, bool stop);
00039
00041 void print_stack_size(void);
00042
00047 void stack_trace(uint64_t max_frames);
00048
00054 void dump_hex(const void *data, size_t size);
00055
00061 extern void stack_dump_asm(uint64_t leave, uint64_t trace_size);
00062
00064 void stack_dump(void);
00065
00070 void stack_dump_recursive(uint64_t max_frames);
00071
00072 #endif

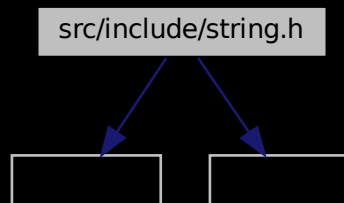
```

7.101 src/include/string.h File Reference

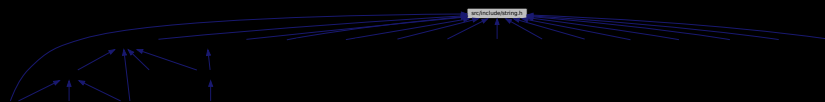
```
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for string.h:



This graph shows which files directly or indirectly include this file:



Functions

- int **memcmp** (const void *, const void *, size_t)
Compare two memory blocks byte by byte.
- void * **memcpy** (void *__restrict, const void *__restrict, size_t)
- void * **memmove** (void *, const void *, size_t)
Copy bytes between memory regions that may overlap.
- void * **memset** (void *, int, size_t)
Fill a memory region with one byte value.
- size_t **strlen** (const char *)
Returns the length of a string.
- int **strtok** (char *srcstr, char sep, char ***output)
- int **str2int** (char str[])
Convert a decimal string to an integer. (DEPRECATED)
- uint64_t **str2int2** (char *str)
- char * **strncpy** (char *dest, const char *src, size_t n)
- char * **int_to_str** (int value, char *buffer)
- void **u64_to_str** (uint64_t val, char *buf)

7.101.1 Function Documentation

7.101.1.1 int_to_str()

```
char * int_to_str (
    int value,
    char * buffer )
```

Copyright (c) 2025 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Definition at line 10 of file int2srt.c.

7.101.1.2 memcmp()

```
int memcmp (
    const void * aptr,
    const void * bptr,
    size_t size )
```

Compare two memory blocks byte by byte.

Parameters

<i>aptr</i>	Pointer to the first memory block.
<i>bptr</i>	Pointer to the second memory block.
<i>size</i>	Number of bytes to compare.

Returns

Negative when the first differing byte in `aptr` is smaller, zero when all requested bytes are equal, or positive when it is larger.

Definition at line 11 of file memcmp.c.

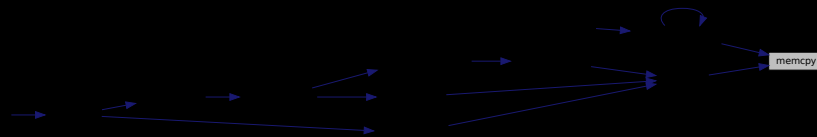
Here is the caller graph for this function:



7.101.1.3 memcpy()

```
void * memcpy (
    void * __restrict,
    const void * __restrict,
    size_t )
```

Here is the caller graph for this function:



7.101.1.4 memmove()

```
void * memmove (
    void * dstptr,
    const void * srcptr,
    size_t size )
```

Copy bytes between memory regions that may overlap.

Parameters

<i>dstptr</i>	Destination memory region.
<i>srcptr</i>	Source memory region.
<i>size</i>	Number of bytes to copy.

Returns

`dstptr`.

Definition at line 10 of file `memmove.c`.

7.101.1.5 memset()

```
void * memset (
    void * bufptr,
    int value,
    size_t size )
```

Fill a memory region with one byte value.

Parameters

<i>bufptr</i>	Destination memory region.
<i>value</i>	Value whose low eight bits are written to each byte.
<i>size</i>	Number of bytes to set.

Returns

bufptr.

Definition at line 10 of file memset.c.

Here is the caller graph for this function:

7.101.1.6 str2int()

```
int str2int (
    char str[] )
```

Convert a decimal string to an integer. (DEPRECATED)

Returns

Parsed value, or 0 if the string contains a non-decimal character.

Definition at line 8 of file `str2int.c`.

7.101.1.7 str2int2()

```
uint64_t str2int2 (
    char * str )
```

Definition at line 25 of file str2int.c.

7.101.1.8 strlen()

```
size_t strlen (
                const char * str )
```

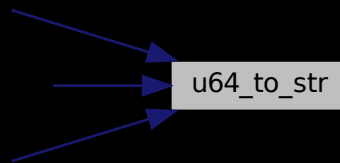
Returns the length of a string.

7.101.1.11 u64_to_str()

```
void u64_to_str (
    uint64_t val,
    char * buf )
```

Definition at line 42 of file [int2srt.c](#).

Here is the caller graph for this function:



7.102 string.h

[Go to the documentation of this file.](#)

```
00001 #ifndef _STRING_H
00002 #define _STRING_H 1
00003
00004 #include <stddef.h>
00005 #include <stdint.h>
00006
00007 #ifdef __cplusplus
00008 extern "C"
00009 {
00010 #endif
00011
00012     int      memcmp(const void *, const void *, size_t);
00013     void      *memcpy(void *__restrict, const void *__restrict, size_t);
00014     void      *memmove(void *, const void *, size_t);
00015     void      *memset(void *, int, size_t);
00016     size_t    strlen(const char *);
00017     int      strtok(char *srcstr, char sep, char ***output);
00018     int      str2int(char str[]);
00019     uint64_t  str2int2(char *str);
00020     char      *strncpy(char *dest, const char *src, size_t n);
00021     char      *int_to_str(int value, char *buffer);
00022     void      u64_to_str(uint64_t val, char *buf);
00023
00024 #ifdef __cplusplus
00025 }
00026 #endif
00027
00028 #endif
```

7.103 src/include/syscall.h File Reference

7.104 syscall.h

[Go to the documentation of this file.](#)

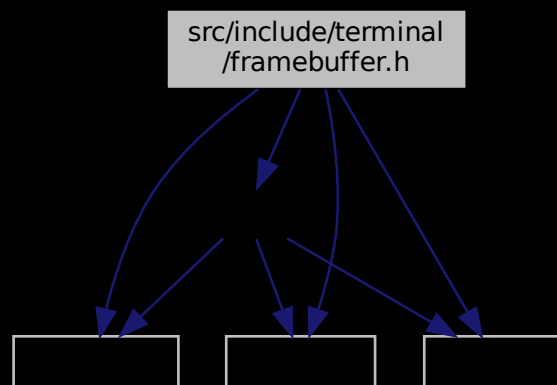
```
00001
```

7.105 src/include/terminal/framebuffer.h File Reference

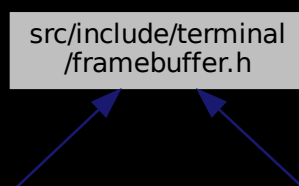
Framebuffer-backed implementation of the Voyager terminal interface.

```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include "term.h"
```

Include dependency graph for framebuffer.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct **fbterm_char**
One rendered terminal cell and its foreground/background colours.
- struct **fbterm_queue_item**
Deferred framebuffer-cell update used by the renderer queue.
- struct **fbterm_context**
Framebuffer terminal backend state.

Macros

- `#define FBTERM_FONT_GLYPHS 256`

Functions

- `struct term_context * fbterm_init (void *(*_malloc)(size_t), uint32_t *framebuffer, size_t width, size_t height, size_t pitch, uint32_t *canvas, uint32_t *ansi_colours, uint32_t *ansi_bright_colours, uint32_t *default_bg, uint32_t *default_fg, void *font, size_t font_width, size_t font_height, size_t font_spacing, size_t font_scale_x, size_t font_scale_y, size_t margin)`

Initialize a framebuffer terminal and return its generic context.

7.105.1 Detailed Description

Framebuffer-backed implementation of the Voyager terminal interface.

Definition in file `framebuffer.h`.

7.105.2 Macro Definition Documentation

7.105.2.1 FBTERM_FONT_GLYPHS

```
#define FBTERM_FONT_GLYPHS 256
```

Number of glyph slots expected by the framebuffer terminal font table.

Definition at line 16 of file `framebuffer.h`.

7.105.3 Function Documentation

7.105.3.1 fbterm_init()

```
struct term_context * fbterm_init (  
    void *(*)(size_t) _malloc,  
    uint32_t * framebuffer,  
    size_t width,  
    size_t height,  
    size_t pitch,  
    uint32_t * canvas,  
    uint32_t * ansi_colours,  
    uint32_t * ansi_bright_colours,  
    uint32_t * default_bg,  
    uint32_t * default_fg,  
    void * font,  
    size_t font_width,  
    size_t font_height,  
    size_t font_spacing,  
    size_t font_scale_x,  
    size_t font_scale_y,  
    size_t margin )
```

Initialize a framebuffer terminal and return its generic context.

Parameters

<i>_malloc</i>	Allocation function used for terminal-owned dynamic storage.
<i>framebuffer</i>	Kernel-accessible framebuffer base.
<i>width</i>	Framebuffer width in pixels.
<i>height</i>	Framebuffer height in pixels.
<i>pitch</i>	Framebuffer pitch in bytes.
<i>canvas</i>	Optional backing canvas supplied by the caller.
<i>ansi_colours</i>	Optional normal ANSI palette.
<i>ansi_bright_colours</i>	Optional bright ANSI palette.
<i>default_bg</i>	Pointer to the default background colour.
<i>default_fg</i>	Pointer to the default foreground colour.
<i>font</i>	Optional caller-supplied font data.
<i>font_width</i>	Caller-supplied font width when applicable.
<i>font_height</i>	Caller-supplied font height when applicable.
<i>font_spacing</i>	Horizontal font spacing.
<i>font_scale_x</i>	Horizontal scale factor.
<i>font_scale_y</i>	Vertical scale factor.
<i>margin</i>	Terminal margin in pixels/cells as interpreted by the backend.

Returns

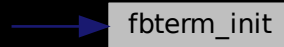
Initialized backend-independent terminal context, or NULL on failure.

VoyagerOS64 initializes this only after its own page tables and heap are live; the resulting context replaces Limine terminal callbacks for normal kernel console output.

Definition at line 870 of file [framebuffer.c](#).

Here is the call graph for this function:

Here is the caller graph for this function:



7.106 framebuffer.h

[Go to the documentation of this file.](#)

```

00001
00006 #ifndef _TERM_FRAMEBUFFER_H
00007 #define _TERM_FRAMEBUFFER_H
00008
00009 #include <stdbool.h>
00010 #include <stddef.h>
00011 #include <stdint.h>
00012
00013 #include "term.h"
00014
00016 #define FBTERM_FONT_GLYPHS 256
00017
00019 struct fbterm_char
00020 {
00021     uint32_t c;
00022     uint32_t fg;
00023     uint32_t bg;
00024 };
00025
00027 struct fbterm_queue_item
00028 {
00029     size_t      x, y;
00030     struct fbterm_char c;
00031 };
00032
00041 struct fbterm_context
00042 {
00043     struct term_context term;
00044
00045     size_t font_width;
00046     size_t font_height;
00047     size_t glyph_width;
00048     size_t glyph_height;
00049
00050     size_t font_scale_x;
00051     size_t font_scale_y;
00052
00053     size_t offset_x, offset_y;
00054
00055     volatile uint32_t *framebuffer;
00056     size_t pitch;
00057     size_t width;
00058     size_t height;
00059     size_t bpp;
00060
00061     size_t font_bits_size;
00062     uint8_t *font_bits;
00063     size_t font_bool_size;
00064     bool *font_bool;
00065
00066     uint32_t ansi_colours[8];
00067     uint32_t ansi_bright_colours[8];
00068     uint32_t default_fg, default_bg;
00069
00070     size_t canvas_size;
00071     uint32_t *canvas;
00072
00073     size_t grid_size;
00074     size_t queue_size;
00075     size_t map_size;
  
```

```

00076
00077     struct fbterm_char *grid;
00078
00079     struct fbterm_queue_item *queue;
00080     size_t queue_i;
00081
00082     struct fbterm_queue_item **map;
00083
00084     uint32_t text_fg;
00085     uint32_t text_bg;
00086     bool cursor_status;
00087     size_t cursor_x;
00088     size_t cursor_y;
00089
00090     uint32_t saved_state_text_fg;
00091     uint32_t saved_state_text_bg;
00092     size_t saved_state_cursor_x;
00093     size_t saved_state_cursor_y;
00094
00095     size_t old_cursor_x;
00096     size_t old_cursor_y;
00097 };
00098
00124 struct term_context *fbterm_init(void *(*_malloc)(size_t),
00125                                  uint32_t *framebuffer,
00126                                  size_t width,
00127                                  size_t height,
00128                                  size_t pitch,
00129                                  uint32_t *canvas,
00130                                  uint32_t *ansi_colours,
00131                                  uint32_t *ansi_bright_colours,
00132                                  uint32_t *default_bg,
00133                                  uint32_t *default_fg,
00134                                  void *font,
00135                                  size_t font_width,
00136                                  size_t font_height,
00137                                  size_t font_spacing,
00138                                  size_t font_scale_x,
00139                                  size_t font_scale_y,
00140                                  size_t margin);
00141
00142 #endif

```

7.107 src/include/terminal/term.h File Reference

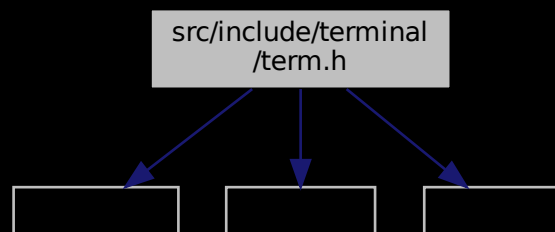
Backend-independent terminal parser/context interface.

```

#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

```

Include dependency graph for term.h:



7.107.2.1 TERM_CB_BELL

```
#define TERM_CB_BELL 20
```

Definition at line 16 of file [term.h](#).

7.107.2.2 TERM_CB_DEC

```
#define TERM_CB_DEC 10
```

Definition at line 15 of file [term.h](#).

7.107.2.3 TERM_CB_KBD_LEDS

```
#define TERM_CB_KBD_LEDS 60
```

Definition at line 20 of file [term.h](#).

7.107.2.4 TERM_CB_LINUX

```
#define TERM_CB_LINUX 80
```

Definition at line 22 of file [term.h](#).

7.107.2.5 TERM_CB_MODE

```
#define TERM_CB_MODE 70
```

Definition at line 21 of file [term.h](#).

7.107.2.6 TERM_CB_POS_REPORT

```
#define TERM_CB_POS_REPORT 50
```

Definition at line 19 of file [term.h](#).

7.107.2.7 TERM_CB_PRIVATE_ID

```
#define TERM_CB_PRIVATE_ID 30
```

Definition at line 17 of file [term.h](#).

7.107.2.8 TERM_CB_STATUS_REPORT

```
#define TERM_CB_STATUS_REPORT 40
```

Definition at line 18 of file [term.h](#).

7.107.2.9 TERM_MAX_ESC_VALUES

```
#define TERM_MAX_ESC_VALUES 16
```

Definition at line 13 of file [term.h](#).

7.107.3 Function Documentation

7.107.3.1 term_context_reinit()

```
void term_context_reinit (  
    struct term\_context * ctx )
```

Reset/reinitialize parser-visible state in an existing context.

Parameters

<i>ctx</i>	Terminal context to reinitialize.
------------	-----------------------------------

Definition at line 51 of file [term.c](#).

Here is the caller graph for this function:

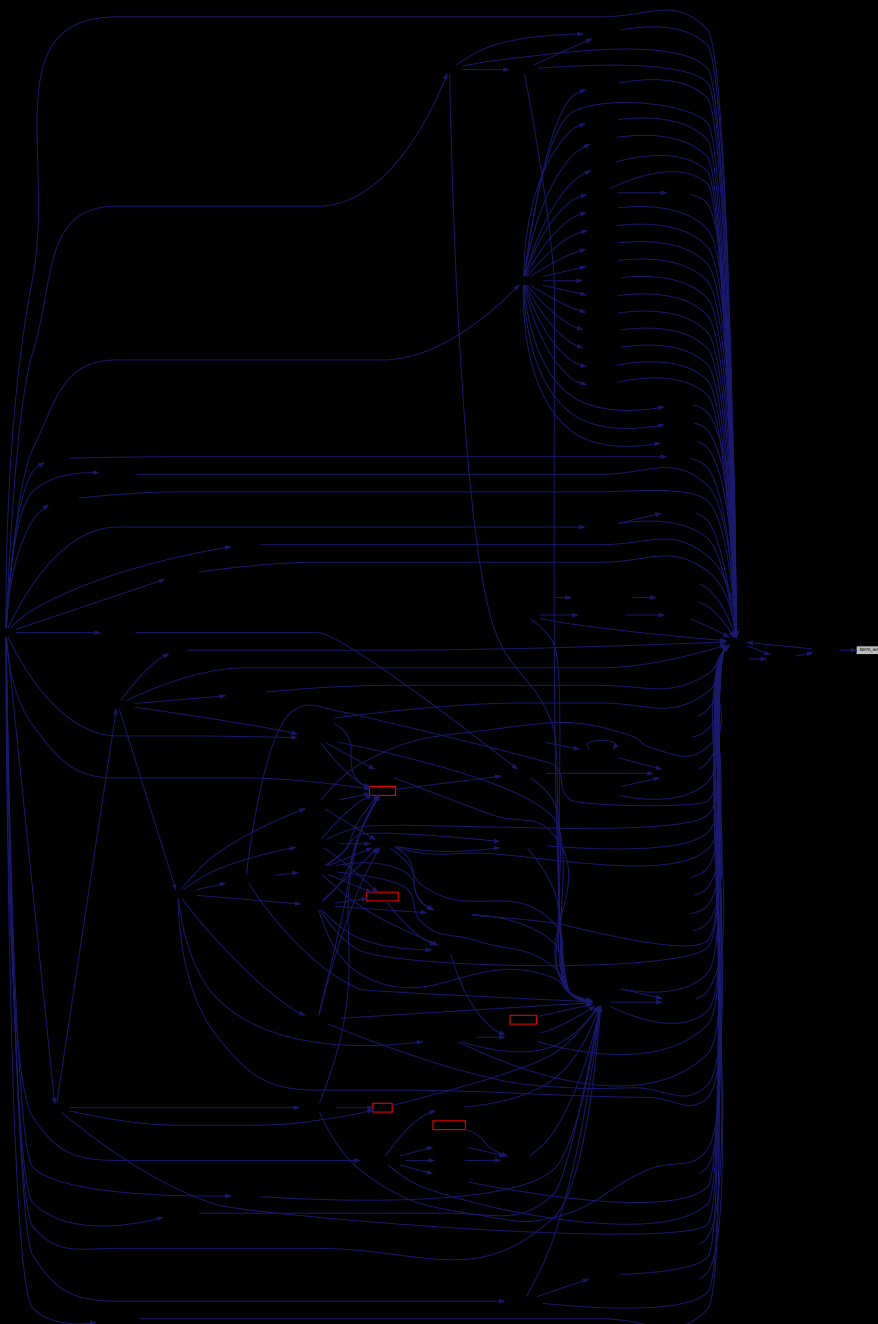


7.107.3.2 `term_write()`

```
void term_write (  
    struct term_context * ctx,  
    const char * buf,  
    size_t count )
```

Feed a byte buffer through the terminal parser and backend.

Here is the caller graph for this function:



7.108 term.h

[Go to the documentation of this file.](#)

```
00001
00006 #ifndef _TERM_H
00007 #define _TERM_H
00008
00009 #include <stdbool.h>
00010 #include <stddef.h>
00011 #include <stdint.h>
00012
00013 #define TERM_MAX_ESC_VALUES 16
```

```

00014
00015 #define TERM_CB_DEC 10
00016 #define TERM_CB_BELL 20
00017 #define TERM_CB_PRIVATE_ID 30
00018 #define TERM_CB_STATUS_REPORT 40
00019 #define TERM_CB_POS_REPORT 50
00020 #define TERM_CB_KBD_LEDS 60
00021 #define TERM_CB_MODE 70
00022 #define TERM_CB_LINUX 80
00023
00035 struct term_context
00036 {
00037     /* internal parser/state-machine use */
00038
00039     size_t    tab_size;
00040     bool      autoflush;
00041     bool      scroll_enabled;
00042     bool      control_sequence;
00043     bool      csi;
00044     bool      escape;
00045     bool      rrr;
00046     bool      discard_next;
00047     bool      bold;
00048     bool      reverse_video;
00049     bool      dec_private;
00050     bool      insert_mode;
00051     uint8_t   g_select;
00052     uint8_t   charsets[2];
00053     size_t    current_charset;
00054     size_t    escape_offset;
00055     size_t    esc_values_i;
00056     size_t    saved_cursor_x;
00057     size_t    saved_cursor_y;
00058     size_t    current_primary;
00059     size_t    scroll_top_margin;
00060     size_t    scroll_bottom_margin;
00061     uint32_t  esc_values[TERM_MAX_ESC_VALUES];
00062     bool      saved_state_bold;
00063     bool      saved_state_reverse_video;
00064     size_t    saved_state_current_charset;
00065     size_t    saved_state_current_primary;
00066
00067     /* backend-provided geometry and rendering operations */
00068
00069     size_t    rows, cols;
00070     bool      in_bootloader;
00071
00072     void (*raw_putchar)(struct term_context *, uint8_t c);
00073     void (*clear)(struct term_context *, bool move);
00074     void (*enable_cursor)(struct term_context *);
00075     bool (*disable_cursor)(struct term_context *);
00076     void (*set_cursor_pos)(struct term_context *, size_t x, size_t y);
00077     void (*get_cursor_pos)(struct term_context *, size_t *x, size_t *y);
00078     void (*set_text_fg)(struct term_context *, size_t fg);
00079     void (*set_text_bg)(struct term_context *, size_t bg);
00080     void (*set_text_fg_bright)(struct term_context *, size_t fg);
00081     void (*set_text_bg_bright)(struct term_context *, size_t bg);
00082     void (*set_text_fg_rgb)(struct term_context *, uint32_t fg);
00083     void (*set_text_bg_rgb)(struct term_context *, uint32_t bg);
00084     void (*set_text_fg_default)(struct term_context *);
00085     void (*set_text_bg_default)(struct term_context *);
00086     void (*move_character)(struct term_context *,
00087                           size_t new_x,
00088                           size_t new_y,
00089                           size_t old_x,
00090                           size_t old_y);
00091     void (*scroll)(struct term_context *);
00092     void (*revscroll)(struct term_context *);
00093     void (*swap_palette)(struct term_context *);
00094     void (*save_state)(struct term_context *);
00095     void (*restore_state)(struct term_context *);
00096     void (*double_buffer_flush)(struct term_context *);
00097     void (*full_refresh)(struct term_context *);
00098     void (*deinit)(struct term_context *, void (*)(void *, size_t));
00099
00100     /* client-provided control-event callback */
00101
00102     void (*callback)(
00103         struct term_context *, uint64_t, uint64_t, uint64_t, uint64_t);
00104 };
00105
00110 void term_context_reinit(struct term_context *ctx);
00111
00118 void term_write(struct term_context *ctx, const char *buf, size_t count);
00119
00120 #endif

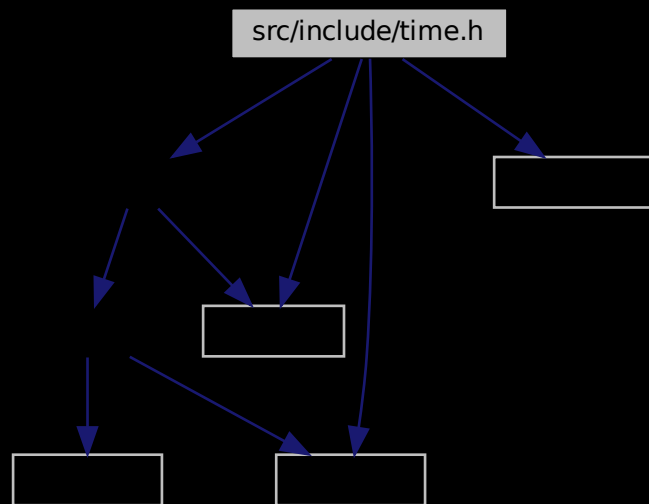
```

7.109 src/include/time.h File Reference

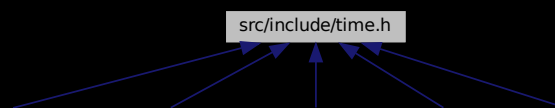
RTC, PIT, and kernel timekeeping interfaces.

```
#include "global_defs.h"  
#include <stdbool.h>  
#include <stddef.h>  
#include <stdint.h>
```

Include dependency graph for time.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `timespec`
Seconds/nanoseconds time representation used by kernel timers.
- struct `timer`
Small software-timer record used by the current timer layer.

Macros

- `#define _TIME_H`
- `#define PIT_FREQ 750`
- `#define PIT_DIVIDEND 1193180`

Typedefs

- `typedef long time_t`

Functions

- `static struct timespec timespec_add (struct timespec a, struct timespec b)`
Add two timespec values with nanosecond carry.
- `static struct timespec timespec_sub (struct timespec a, struct timespec b)`
Subtract one timespec from another using the current kernel helper.
- `void sys_clock_handler (void)`
Primary PIT/system-clock interrupt handler.
- `void init_PIT (void)`
Program the PIT using the current default timer configuration.
- `void read_rtc (void)`
Read a stable calendar/time snapshot from the RTC/CMOS path.
- `void print_date (void)`
Print the current calendar date.
- `void print_sys_time (void)`
Print current system time information for VSH diagnostics.
- `int dayofweek (int y, int m, int d)`
Compute weekday index for a Gregorian calendar date.
- `struct timer * timer_new (struct timespec when)`
Allocate a software timer record for a target time.
- `void timer_arm (struct timer *timer)`
Arm a software timer.
- `void timer_disarm (struct timer *timer)`
Disarm a software timer.
- `void print_load_time (void)`
Print boot/load-time diagnostics.
- `uint16_t pit_get_current_count (void)`
Read the PIT's current counter value.
- `void pit_set_reload_value (uint16_t new_count)`
Program the PIT reload/divisor value directly.
- `void pit_set_frequency (uint64_t frequency)`
Program the PIT for an approximate requested frequency.
- `void sys_clock_handler_alt (void)`
Alternate system-clock handler retained by the current timer code.
- `void time_init (void)`
Initialize RTC/PIT-facing timekeeping state during kernel bring-up.
- `void pit_sleep (uint64_t ms)`
Busy/blocking PIT-based sleep helper used by current kernel code.

Variables

- struct timespec time_mono
- struct timespec time_real
- uint64_t system_timer_ms
- uint64_t system_timer_fractions
- uint8_t second
- uint8_t minute
- uint8_t hour
- uint8_t day
- uint8_t month
- uint16_t year

7.109.1 Detailed Description

RTC, PIT, and kernel timekeeping interfaces.

Definition in file [time.h](#).

7.109.2 Macro Definition Documentation

7.109.2.1 _TIME_H

```
#define _TIME_H
```

Definition at line 8 of file [time.h](#).

7.109.2.2 PIT_DIVIDEND

```
#define PIT_DIVIDEND 1193180
```

Input clock used by the legacy 8253/8254 PIT.

Definition at line 18 of file [time.h](#).

7.109.2.3 PIT_FREQ

```
#define PIT_FREQ 750
```

PIT programming frequency currently requested by VoyagerOS64.

Definition at line 16 of file [time.h](#).

7.109.3 Typedef Documentation

7.109.3.1 time_t

```
typedef long time_t
```

Definition at line 20 of file `time.h`.

7.109.4 Function Documentation

7.109.4.1 dayofweek()

```
int dayofweek (  
    int y,  
    int m,  
    int d )
```

Compute weekday index for a Gregorian calendar date.

Parameters

<i>y</i>	Full year.
<i>m</i>	Month number.
<i>d</i>	Day of month.

Returns

Weekday index used by the date-printing code.

Definition at line 305 of file `time.c`.

Here is the caller graph for this function:



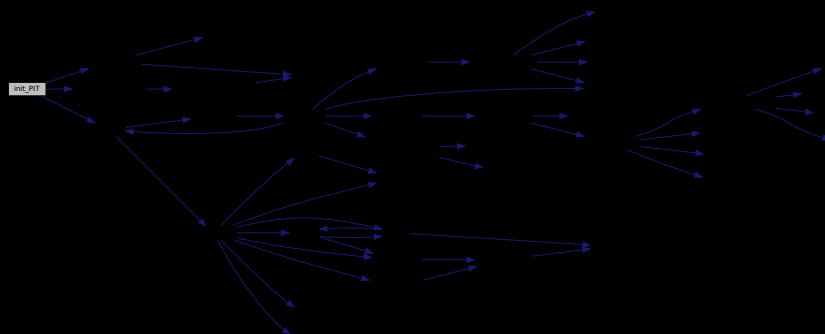
7.109.4.2 init_PIT()

```
void init_PIT (  
    void )
```

Program the PIT using the current default timer configuration.

Definition at line 193 of file [time.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



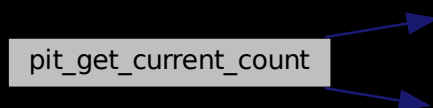
7.109.4.3 pit_get_current_count()

```
uint16_t pit_get_current_count (  
    void )
```

Read the PIT's current counter value.

Definition at line 107 of file [time.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.109.4.4 `pit_set_frequency()`

```
void pit_set_frequency (
    uint64_t frequency )
```

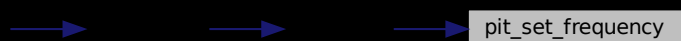
Program the PIT for an approximate requested frequency.

Definition at line 122 of file `time.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



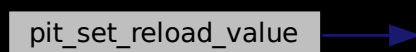
7.109.4.5 pit_set_reload_value()

```
void pit_set_reload_value (
    uint16_t new_count )
```

Program the PIT reload/divisor value directly.

Definition at line 115 of file [time.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.109.4.6 pit_sleep()

```
void pit_sleep (
    uint64_t ms )
```

Busy/blocking PIT-based sleep helper used by current kernel code.

Definition at line 130 of file [time.c](#).

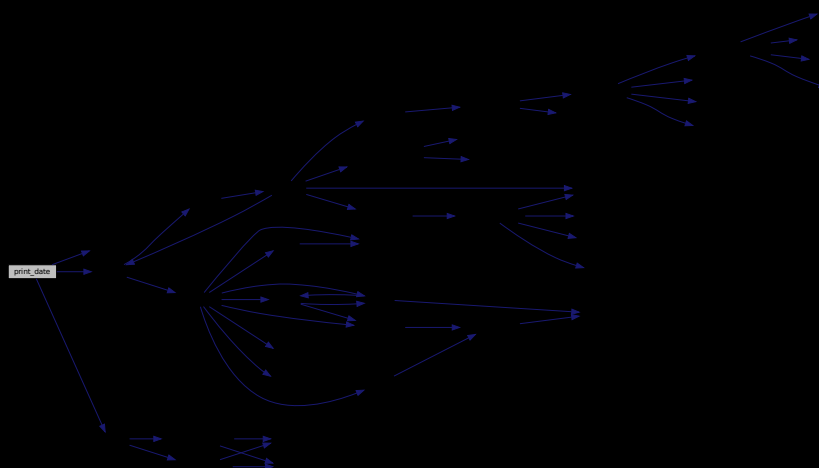
7.109.4.7 print_date()

```
void print_date (
    void )
```

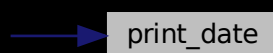
Print the current calendar date.

Definition at line 291 of file [time.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



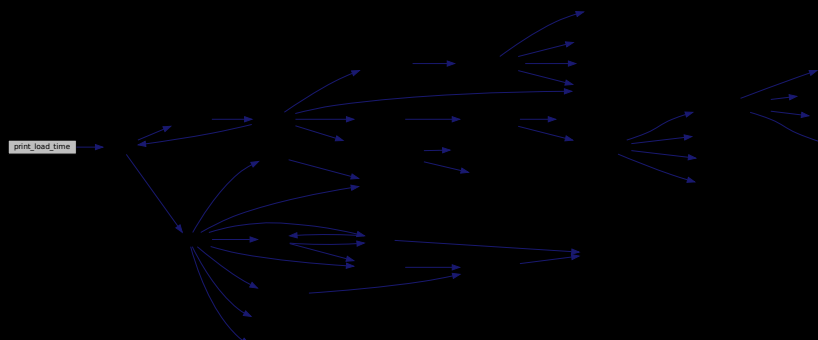
7.109.4.8 print_load_time()

```
void print_load_time (
    void )
```

Print boot/load-time diagnostics.

Definition at line 322 of file [time.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



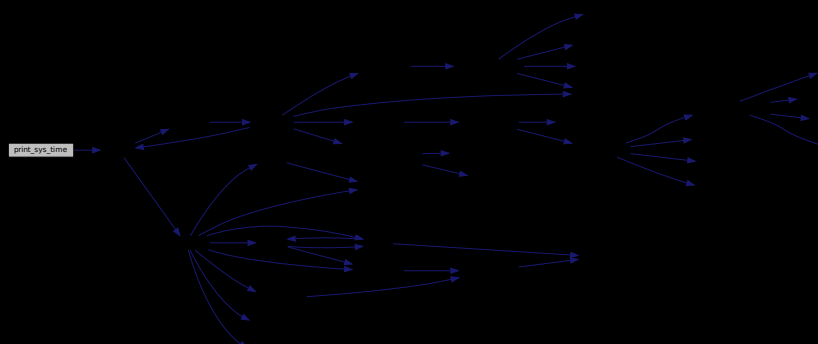
7.109.4.9 `print_sys_time()`

```
void print_sys_time (
    void )
```

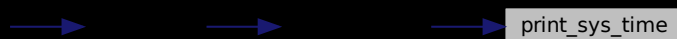
Print current system time information for VSH diagnostics.

Definition at line 313 of file [time.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.109.4.10 read_rtc()

```
void read_rtc (
    void )
```

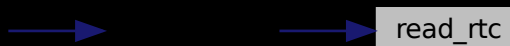
Read a stable calendar/time snapshot from the RTC/CMOS path.

Definition at line 212 of file [time.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



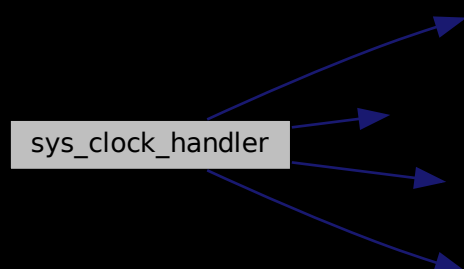
7.109.4.11 sys_clock_handler()

```
void sys_clock_handler (  
    void )
```

Primary PIT/system-clock interrupt handler.

Definition at line 137 of file [time.c](#).

Here is the call graph for this function:



7.109.4.12 sys_clock_handler_alt()

```
void sys_clock_handler_alt (  
    void )
```

Alternate system-clock handler retained by the current timer code.

Definition at line 167 of file [time.c](#).

Here is the call graph for this function:



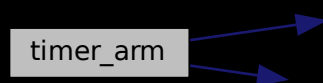
7.109.4.14 timer_arm()

```
void timer_arm (  
    struct timer * timer )
```

Arm a software timer.

Definition at line 53 of file *time.c*.

Here is the call graph for this function:



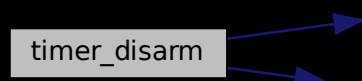
7.109.4.15 timer_disarm()

```
void timer_disarm (  
    struct timer * timer )
```

Disarm a software timer.

Definition at line 64 of file *time.c*.

Here is the call graph for this function:



7.109.4.16 timer_new()

```
struct timer * timer_new (
    struct timespec when )
```

Allocate a software timer record for a target time.

7.109.4.17 timespec_add()

```
static struct timespec timespec_add (
    struct timespec a,
    struct timespec b ) [inline], [static]
```

Add two timespec values with nanosecond carry.

Parameters

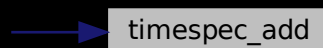
<i>a</i>	First value.
<i>b</i>	Second value.

Returns

Sum of a and b.

Definition at line 48 of file `time.h`.

Here is the caller graph for this function:



7.109.4.18 timespec_sub()

```
static struct timespec timespec_sub (
    struct timespec a,
    struct timespec b ) [inline], [static]
```

Subtract one timespec from another using the current kernel helper.

Parameters

<i>a</i>	Minuend.
<i>b</i>	Subtrahend.

Returns

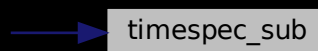
Non-negative result according to the helper's current saturation/ borrow behavior.

Note

The timer subsystem is still undergoing cleanup after 0.0.5 memory and scheduler stabilization. Do not treat this helper as a fully qualified POSIX timespec implementation yet.

Definition at line 74 of file [time.h](#).

Here is the caller graph for this function:



7.109.5 Variable Documentation

7.109.5.1 day

`uint8_t day`

Definition at line 103 of file [time.h](#).

7.109.5.2 hour

`uint8_t hour`

Definition at line 103 of file [time.h](#).

7.109.5.3 minute

```
uint8_t minute
```

Definition at line 103 of file [time.h](#).

7.109.5.4 month

```
uint8_t month
```

Definition at line 103 of file [time.h](#).

7.109.5.5 second

```
uint8_t second [extern]
```

RTC calendar fields populated by [read_rtc\(\)](#).

Definition at line 25 of file [time.c](#).

7.109.5.6 system_timer_fractions

```
uint64_t system_timer_fractions [extern]
```

Fractional/auxiliary timer accumulator used by the current implementation.

Definition at line 29 of file [time.c](#).

7.109.5.7 system_timer_ms

```
uint64_t system_timer_ms [extern]
```

Historical millisecond-named timer counter.

Definition at line 28 of file [time.c](#).

7.109.5.8 time_mono

```
struct timespec time_mono [extern]
```

Monotonic time accumulator maintained by the current timer subsystem.

Definition at line 34 of file [time.c](#).

7.109.5.9 time_real

```
struct timespec time_real [extern]
```

Real-time accumulator/state maintained by the current timer subsystem.

Definition at line 35 of file [time.c](#).

7.109.5.10 year

```
uint16_t year [extern]
```

Full RTC year value after century handling.

Definition at line 26 of file [time.c](#).

7.110 time.h

[Go to the documentation of this file.](#)

```
00001
00006 #pragma once
00007 #ifndef _TIME_H
00008 #define _TIME_H
00009
00010 #include "global_defs.h"
00011 #include <stdbool.h>
00012 #include <stddef.h>
00013 #include <stdint.h>
00014
00016 #define PIT_FREQ 750
00018 #define PIT_DIVIDEND 1193180
00019
00020 typedef long time_t;
00021
00023 struct timespec
00024 {
00025     time_t tv_sec;
00026     long tv_nsec;
00027 };
00028
00030 struct timer
00031 {
00032     ssize_t index;
00033     bool fired;
00034     struct timespec when;
00035 };
00036
00038 extern struct timespec time_mono;
00040 extern struct timespec time_real;
00041
```

```

00048 static inline struct timespec timespec_add(struct timespec a, struct timespec b)
00049 {
00050     if (a.tv_nsec + b.tv_nsec > 999999999)
00051     {
00052         a.tv_nsec = (a.tv_nsec + b.tv_nsec) - 1000000000;
00053         a.tv_sec++;
00054     }
00055     else
00056     {
00057         a.tv_nsec += b.tv_nsec;
00058     }
00059     a.tv_sec += b.tv_sec;
00060     return a;
00061 }
00062
00074 static inline struct timespec timespec_sub(struct timespec a, struct timespec b)
00075 {
00076     if (b.tv_nsec > a.tv_nsec)
00077     {
00078         a.tv_nsec = 999999999 + (b.tv_nsec - a.tv_nsec);
00079         if (a.tv_sec > 0)
00080             a.tv_sec--;
00081         else
00082             a.tv_sec = a.tv_nsec = 0;
00083     }
00084     else
00085     {
00086         a.tv_nsec -= b.tv_nsec;
00087     }
00088
00089     if (b.tv_sec > a.tv_sec)
00090         a.tv_sec = a.tv_nsec = 0;
00091     else
00092         a.tv_sec -= b.tv_sec;
00093
00094     return a;
00095 }
00096
00098 extern uint64_t system_timer_ms;
00100 extern uint64_t system_timer_fractions;
00101
00103 extern uint8_t second, minute, hour, day, month;
00105 extern uint16_t year;
00106
00108 void sys_clock_handler(void);
00110 void init_PIT(void);
00112 void read_rtc(void);
00114 void print_date(void);
00116 void print_sys_time(void);
00117
00125 int dayofweek(int y, int m, int d);
00126
00128 struct timer *timer_new(struct timespec when);
00130 void timer_arm(struct timer *timer);
00132 void timer_disarm(struct timer *timer);
00134 void print_load_time(void);
00135
00137 uint16_t pit_get_current_count(void);
00139 void pit_set_reload_value(uint16_t new_count);
00141 void pit_set_frequency(uint64_t frequency);
00143 void sys_clock_handler_alt(void);
00145 void time_init(void);
00147 void pit_sleep(uint64_t ms);
00148
00156 #endif

```

7.111 src/include/tss.h File Reference

x86-64 Task State Segment layout.

```

#include "global_defs.h"
#include <stdint.h>

```


7.111.2 Macro Definition Documentation

7.111.2.1 _TSS_H

```
#define _TSS_H
```

Definition at line 11 of file `tss.h`.

7.112 tss.h

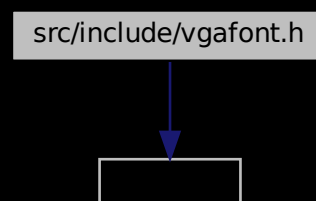
[Go to the documentation of this file.](#)

```
00001
00006 #pragma once
00007 #include "global_defs.h"
00008 #include <stdint.h>
00009
00010 #ifndef _TSS_H
00011 #define _TSS_H
00012
00024 struct PACKED TSS
00025 {
00026     uint32_t reserved0;
00027     uint64_t rsp0;
00028     uint64_t rsp1;
00029     uint64_t rsp2;
00030     uint64_t reserved1;
00031     uint64_t ist1;
00032     uint64_t ist2;
00033     uint64_t ist3;
00034     uint64_t ist4;
00035     uint64_t ist5;
00036     uint64_t ist6;
00037     uint64_t ist7;
00038     uint64_t reserved2;
00039     uint16_t reserved3;
00040     uint16_t iopbOffset;
00041 };
00042
00043 #endif
```

7.113 src/include/vgafont.h File Reference

```
#include <stdint.h>
```

Include dependency graph for `vgafont.h`:



Variables

- static const uint8_t [vgafont](#) []

7.113.1 Variable Documentation

7.113.1.1 vgafont

```
const uint8_t vgafont[] [static]
```

Definition at line 7 of file [vgafont.h](#).

7.114 vgafont.h

[Go to the documentation of this file.](#)

```
00001 #ifndef VGAFONT_H
00002 #define VGAFONT_H
00003
00004 #include <stdint.h>
00005
00006 // array size is 4096
00007 static const uint8_t vgafont[] = {
00008     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00009     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x82, 0x82, 0xaa, 0x82, 0x82,
00010     0xba, 0x92, 0x82, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0xfe,
00011     0xfe, 0xd6, 0xfe, 0xfe, 0xc6, 0xee, 0xfe, 0x7c, 0x00, 0x00, 0x00, 0x00,
00012     0x00, 0x00, 0x00, 0x00, 0x00, 0x6c, 0xfe, 0xfe, 0x7c, 0x38, 0x10, 0x00,
00013     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x38, 0x7c, 0xfe,
00014     0x7c, 0x38, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00015     0x38, 0x38, 0x38, 0xee, 0xee, 0xee, 0x10, 0x38, 0x00, 0x00, 0x00, 0x00,
00016     0x00, 0x00, 0x00, 0x00, 0x10, 0x38, 0x7c, 0xfe, 0xfe, 0x54, 0x10, 0x38,
00017     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x18, 0x3c,
00018     0x3c, 0x18, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xff, 0xff, 0xff, 0xff,
00019     0xff, 0xff, 0xe7, 0xc3, 0xc3, 0xe7, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
00020     0x00, 0x00, 0x00, 0x00, 0x00, 0x3c, 0x66, 0x42, 0x42, 0x66, 0x3c, 0x00,
00021     0x00, 0x00, 0x00, 0x00, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xc3, 0x99, 0xbd,
00022     0xbd, 0x99, 0xc3, 0xff, 0xff, 0xff, 0xff, 0xff, 0x00, 0x00, 0x1e, 0x06,
00023     0x0a, 0x10, 0x10, 0x7c, 0x82, 0x82, 0x82, 0x7c, 0x00, 0x00, 0x00, 0x00,
00024     0x00, 0x00, 0x7c, 0x82, 0x82, 0x82, 0x7c, 0x10, 0x10, 0x7c, 0x10, 0x10,
00025     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x1e, 0x12, 0x1e, 0x10, 0x10, 0x10,
00026     0x10, 0xf0, 0xf0, 0xe0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x3e, 0x22,
00027     0x3e, 0x22, 0x22, 0x22, 0x2e, 0xee, 0xec, 0xc0, 0x00, 0x00, 0x00, 0x00,
00028     0x00, 0x00, 0x10, 0x92, 0x7c, 0x44, 0x82, 0x82, 0x82, 0x44, 0x7c, 0x92,
00029     0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xc0, 0xe0, 0xf0,
00030     0xf8, 0xf0, 0xe0, 0xc0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00031     0x00, 0x18, 0x38, 0x78, 0xf8, 0x78, 0x38, 0x18, 0x00, 0x00, 0x00, 0x00,
00032     0x00, 0x00, 0x10, 0x38, 0x54, 0x92, 0x10, 0x10, 0x92, 0x54, 0x38, 0x10,
00033     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x44, 0x44, 0x44, 0x44, 0x44, 0x44,
00034     0x44, 0x00, 0x44, 0x44, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7e, 0xf2,
00035     0xf2, 0xf2, 0x72, 0x12, 0x12, 0x12, 0x12, 0x00, 0x00, 0x00, 0x00,
00036     0x00, 0x78, 0x84, 0x40, 0x30, 0x48, 0x84, 0x84, 0x48, 0x30, 0x08, 0x84,
00037     0x78, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00038     0xfe, 0xfe, 0xfe, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x38,
00039     0x54, 0x92, 0x10, 0x10, 0x92, 0x54, 0x38, 0xfe, 0x00, 0x00, 0x00, 0x00,
00040     0x00, 0x00, 0x10, 0x38, 0x54, 0x92, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10,
00041     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10,
00042     0x92, 0x54, 0x38, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00043     0x10, 0x08, 0x04, 0xfe, 0x04, 0x08, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00,
00044     0x00, 0x00, 0x00, 0x00, 0x10, 0x20, 0x40, 0xfe, 0x40, 0x20, 0x10, 0x00,
00045     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x80,
00046     0x80, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00047     0x00, 0x24, 0x42, 0xff, 0x42, 0x24, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00048     0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x38, 0x7c, 0xfe, 0xfe, 0x00, 0x00,
00049     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0xfe, 0x7c,
00050     0x38, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00051     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00052     0x00, 0x00, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x00, 0x10, 0x10,
```

```
00053 0x00, 0x00, 0x00, 0x00, 0x00, 0x48, 0x48, 0x48, 0x48, 0x00, 0x00, 0x00,
00054 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x44,
00055 0x44, 0xfe, 0x44, 0x44, 0x44, 0x44, 0xfe, 0x44, 0x44, 0x00, 0x00, 0x00,
00056 0x00, 0x10, 0x10, 0x7c, 0x92, 0x90, 0x80, 0x7c, 0x02, 0x12, 0x92, 0x7c,
00057 0x10, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x60, 0xf2, 0x64, 0x08, 0x10,
00058 0x20, 0x4c, 0x9e, 0x0c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x48,
00059 0x48, 0x48, 0x32, 0x4a, 0x84, 0x84, 0x84, 0x7a, 0x00, 0x00, 0x00, 0x00,
00060 0x00, 0x10, 0x10, 0x10, 0x20, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00061 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x08, 0x10, 0x20, 0x20, 0x20, 0x20,
00062 0x20, 0x20, 0x10, 0x08, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x20, 0x10,
00063 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x10, 0x20, 0x00, 0x00, 0x00, 0x00,
00064 0x00, 0x00, 0x00, 0x00, 0x10, 0x54, 0x38, 0xfe, 0x38, 0x54, 0x10, 0x00,
00065 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x10, 0x10, 0xfe,
00066 0x10, 0x10, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00067 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x10, 0x10, 0x20, 0x00, 0x00,
00068 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0x00, 0x00, 0x00, 0x00,
00069 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00070 0x00, 0x00, 0x30, 0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00071 0x00, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x00, 0x00, 0x00, 0x00,
00072 0x00, 0x00, 0x38, 0x44, 0x82, 0x82, 0x92, 0x92, 0x82, 0x82, 0x44, 0x38,
00073 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x30, 0x50, 0x10, 0x10, 0x10,
00074 0x10, 0x10, 0x10, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x82,
00075 0x02, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0xfe, 0x00, 0x00, 0x00, 0x00,
00076 0x00, 0x00, 0x7c, 0x82, 0x02, 0x02, 0x3c, 0x02, 0x02, 0x02, 0x82, 0x7c,
00077 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x04, 0x0c, 0x14, 0x24, 0x44, 0x84,
00078 0xfe, 0x04, 0x04, 0x04, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0x80,
00079 0x80, 0x80, 0xfc, 0x82, 0x02, 0x02, 0x82, 0x7c, 0x00, 0x00, 0x00, 0x00,
00080 0x00, 0x00, 0x7c, 0x80, 0x80, 0x80, 0xfc, 0x82, 0x82, 0x82, 0x82, 0x7c,
00081 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0x02, 0x02, 0x04, 0x08, 0x10,
00082 0x10, 0x10, 0x10, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x82,
00083 0x82, 0x44, 0x7c, 0x82, 0x82, 0x82, 0x82, 0x7c, 0x00, 0x00, 0x00, 0x00,
00084 0x00, 0x00, 0x7c, 0x82, 0x82, 0x82, 0x82, 0x7e, 0x02, 0x02, 0x7c,
00085 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x10, 0x00,
00086 0x00, 0x00, 0x10, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00087 0x00, 0x10, 0x10, 0x00, 0x00, 0x00, 0x10, 0x10, 0x10, 0x20, 0x00, 0x00,
00088 0x00, 0x00, 0x00, 0x04, 0x08, 0x10, 0x20, 0x40, 0x20, 0x10, 0x08, 0x04,
00089 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0x00, 0x00,
00090 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x40,
00091 0x20, 0x10, 0x08, 0x04, 0x08, 0x10, 0x20, 0x40, 0x00, 0x00, 0x00, 0x00,
00092 0x00, 0x00, 0x7c, 0x82, 0x82, 0x02, 0x0c, 0x10, 0x10, 0x00, 0x10, 0x10,
00093 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x82, 0x82, 0xba, 0xaa, 0xaa,
00094 0x9c, 0x80, 0x80, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x38, 0x44,
00095 0x82, 0x82, 0x82, 0xfe, 0x82, 0x82, 0x82, 0x82, 0x00, 0x00, 0x00, 0x00,
00096 0x00, 0x00, 0xfc, 0x82, 0x82, 0x82, 0xfc, 0x82, 0x82, 0x82, 0x82, 0xfc,
00097 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x82, 0x80, 0x80, 0x80, 0x80,
00098 0x80, 0x82, 0x82, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0xf8, 0x84,
00099 0x82, 0x82, 0x82, 0x82, 0x82, 0x82, 0x84, 0xf8, 0x00, 0x00, 0x00, 0x00,
00100 0x00, 0x80, 0xfe, 0x80, 0x80, 0x80, 0xfc, 0x80, 0x80, 0x80, 0x80, 0xfe,
00101 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0x80, 0x80, 0x80, 0xfc, 0x80,
00102 0x80, 0x80, 0x80, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x82,
00103 0x80, 0x80, 0x9e, 0x82, 0x82, 0x82, 0x7e, 0x00, 0x00, 0x00, 0x00,
00104 0x00, 0x00, 0x82, 0x82, 0x82, 0x82, 0xfe, 0x82, 0x82, 0x82, 0x82, 0x82,
00105 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x10, 0x10, 0x10, 0x10, 0x10,
00106 0x10, 0x10, 0x10, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x3e, 0x04,
00107 0x04, 0x04, 0x04, 0x04, 0x04, 0x04, 0x84, 0x78, 0x00, 0x00, 0x00, 0x00,
00108 0x00, 0x00, 0x84, 0x88, 0x90, 0xa0, 0xc0, 0xa0, 0x90, 0x88, 0x84, 0x82,
00109 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80,
00110 0x80, 0x80, 0x80, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x82, 0xc6,
00111 0xaa, 0x92, 0x92, 0x82, 0x82, 0x82, 0x82, 0x82, 0x00, 0x00, 0x00, 0x00,
00112 0x00, 0x00, 0x82, 0xc2, 0xa2, 0x92, 0x8a, 0x86, 0x82, 0x82, 0x82, 0x82,
00113 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x82, 0x82, 0x82, 0x82, 0x82,
00114 0x82, 0x82, 0x82, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfc, 0x82,
00115 0x82, 0x82, 0xfc, 0x80, 0x80, 0x80, 0x80, 0x00, 0x00, 0x00, 0x00,
00116 0x00, 0x00, 0x7c, 0x82, 0x82, 0x82, 0x82, 0x92, 0x92, 0x92, 0x7c,
00117 0x08, 0x04, 0x00, 0x00, 0x00, 0xfc, 0x82, 0x82, 0x82, 0xfc, 0x84,
00118 0x82, 0x82, 0x82, 0x82, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x82,
00119 0x80, 0x60, 0x30, 0x0c, 0x02, 0x82, 0x82, 0x7c, 0x00, 0x00, 0x00, 0x00,
00120 0x00, 0x00, 0xfe, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10,
00121 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x82, 0x82, 0x82, 0x82, 0x82, 0x82,
00122 0x82, 0x82, 0x82, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x82, 0x82,
00123 0x82, 0x82, 0x82, 0x82, 0x82, 0x44, 0x28, 0x10, 0x00, 0x00, 0x00,
00124 0x00, 0x00, 0x82, 0x82, 0x82, 0x82, 0x82, 0x92, 0x92, 0x92, 0xaa, 0x44,
00125 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x82, 0x82, 0x44, 0x28, 0x10, 0x28,
00126 0x44, 0x82, 0x82, 0x82, 0x00, 0x00, 0x00, 0x00, 0x00, 0x82, 0x82, 0x82,
00127 0x82, 0x44, 0x28, 0x10, 0x10, 0x10, 0x10, 0x10, 0x00, 0x00, 0x00, 0x00,
00128 0x00, 0x00, 0xfe, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x80, 0xfe,
00129 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x38, 0x20, 0x20, 0x20, 0x20, 0x20,
00130 0x20, 0x20, 0x20, 0x38, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00131 0x00, 0x80, 0x40, 0x20, 0x10, 0x08, 0x04, 0x02, 0x00, 0x00, 0x00, 0x00,
00132 0x00, 0x00, 0x38, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x38,
00133 0x00, 0x00, 0x00, 0x00, 0x10, 0x28, 0x44, 0x82, 0x00, 0x00, 0x00, 0x00,
00134 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00135 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0x00, 0x00,
00136 0x00, 0x20, 0x20, 0x20, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00137 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x78, 0x04, 0x04,
00138 0x7c, 0x84, 0x84, 0x7e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x80,
00139 0x80, 0xbc, 0xc2, 0x82, 0x82, 0x82, 0x82, 0xfc, 0x00, 0x00, 0x00, 0x00,
```



```
00140    0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x82, 0x80, 0x80, 0x80, 0x82, 0x7c,
00141    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x02, 0x02, 0x02, 0x7a, 0x86, 0x82,
00142    0x82, 0x82, 0x82, 0x7e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00143    0x00, 0x7c, 0x82, 0x82, 0xfe, 0x80, 0x80, 0x7c, 0x00, 0x00, 0x00, 0x00,
00144    0x00, 0x00, 0x1c, 0x20, 0x20, 0xf8, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20,
00145    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7e, 0x82, 0x82,
00146    0x82, 0x82, 0x86, 0x7a, 0x02, 0x02, 0x7c, 0x00, 0x00, 0x00, 0x80, 0x80,
00147    0x80, 0xbc, 0xc2, 0x82, 0x82, 0x82, 0x82, 0x00, 0x00, 0x00, 0x00,
00148    0x00, 0x00, 0x10, 0x10, 0x00, 0x70, 0x10, 0x10, 0x10, 0x10, 0x10, 0x7c,
00149    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x08, 0x08, 0x00, 0x38, 0x08, 0x08,
00150    0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x70, 0x00, 0x00, 0x00, 0x80, 0x80,
00151    0x80, 0x86, 0x98, 0xe0, 0x90, 0x88, 0x84, 0x82, 0x00, 0x00, 0x00, 0x00,
00152    0x00, 0x00, 0x60, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x20, 0x1c,
00153    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xec, 0x92, 0x92,
00154    0x92, 0x92, 0x92, 0x92, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00155    0x00, 0xbc, 0xc2, 0x82, 0x82, 0x82, 0x82, 0x82, 0x00, 0x00, 0x00, 0x00,
00156    0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x82, 0x82, 0x82, 0x82, 0x82, 0x7c,
00157    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xbc, 0xc2, 0x82,
00158    0x82, 0x82, 0xc2, 0xbc, 0x80, 0x80, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00,
00159    0x00, 0x7e, 0x82, 0x82, 0x82, 0x82, 0x86, 0x7a, 0x02, 0x02, 0x02, 0x00,
00160    0x00, 0x00, 0x00, 0x00, 0x00, 0xbc, 0xc2, 0x80, 0x80, 0x80, 0x80, 0x80,
00161    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7e, 0x80, 0x80,
00162    0x7c, 0x02, 0x02, 0xfc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x20,
00163    0x20, 0xfc, 0x20, 0x20, 0x20, 0x20, 0x1c, 0x00, 0x00, 0x00, 0x00,
00164    0x00, 0x00, 0x00, 0x00, 0x00, 0x82, 0x82, 0x82, 0x82, 0x82, 0x86, 0x7a,
00165    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x82, 0x82, 0x82,
00166    0x82, 0x44, 0x28, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00167    0x00, 0x82, 0x82, 0x82, 0x92, 0x92, 0x92, 0x6c, 0x00, 0x00, 0x00, 0x00,
00168    0x00, 0x00, 0x00, 0x00, 0x00, 0x82, 0x44, 0x28, 0x10, 0x28, 0x44, 0x82,
00169    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x82, 0x82, 0x82,
00170    0x82, 0x82, 0x86, 0x7a, 0x02, 0x02, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00,
00171    0x00, 0xfe, 0x04, 0x08, 0x10, 0x20, 0x40, 0xfe, 0x00, 0x00, 0x00, 0x00,
00172    0x00, 0x00, 0x0c, 0x10, 0x10, 0x10, 0x60, 0x10, 0x10, 0x10, 0x10, 0x0c,
00173    0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10,
00174    0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x00, 0x00, 0x00, 0x60, 0x10,
00175    0x10, 0x10, 0x0c, 0x10, 0x10, 0x10, 0x10, 0x60, 0x00, 0x00, 0x00, 0x00,
00176    0x00, 0x00, 0x62, 0x92, 0x8c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00177    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x28, 0x44,
00178    0x82, 0x82, 0x82, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x82,
00179    0x80, 0x80, 0x80, 0x80, 0x80, 0x82, 0x82, 0x7c, 0x10, 0x08, 0x70, 0x00,
00180    0x00, 0x00, 0x44, 0x44, 0x00, 0x82, 0x82, 0x82, 0x82, 0x82, 0x86, 0x7a,
00181    0x00, 0x00, 0x00, 0x00, 0x00, 0x08, 0x10, 0x20, 0x00, 0x7c, 0x82, 0x82,
00182    0xfe, 0x80, 0x80, 0x7e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x28, 0x44,
00183    0x00, 0x7c, 0x02, 0x02, 0x7e, 0x82, 0x82, 0x7e, 0x00, 0x00, 0x00, 0x00,
00184    0x00, 0x00, 0x44, 0x44, 0x00, 0x7c, 0x02, 0x02, 0x7e, 0x82, 0x82, 0x7e,
00185    0x00, 0x00, 0x00, 0x00, 0x00, 0x20, 0x10, 0x08, 0x00, 0x7c, 0x02, 0x02,
00186    0x7e, 0x82, 0x82, 0x7e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x18, 0x24, 0x18,
00187    0x00, 0x7c, 0x02, 0x02, 0x7e, 0x82, 0x82, 0x7e, 0x00, 0x00, 0x00, 0x00,
00188    0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x82, 0x80, 0x80, 0x80, 0x82, 0x7c,
00189    0x10, 0x08, 0x70, 0x00, 0x00, 0x10, 0x28, 0x44, 0x00, 0x7c, 0x82, 0x82,
00190    0xfe, 0x80, 0x80, 0x7e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x44, 0x44,
00191    0x00, 0x7c, 0x82, 0x82, 0xfe, 0x80, 0x80, 0x7e, 0x00, 0x00, 0x00, 0x00,
00192    0x00, 0x20, 0x10, 0x08, 0x00, 0x7c, 0x82, 0x82, 0xfe, 0x80, 0x80, 0x7e,
00193    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x48, 0x00, 0x70, 0x10, 0x10,
00194    0x10, 0x10, 0x10, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x28, 0x44,
00195    0x00, 0x70, 0x10, 0x10, 0x10, 0x10, 0x10, 0x7c, 0x00, 0x00, 0x00, 0x00,
00196    0x00, 0x40, 0x20, 0x10, 0x00, 0x70, 0x10, 0x10, 0x10, 0x10, 0x10, 0x7c,
00197    0x00, 0x00, 0x00, 0x00, 0x44, 0x44, 0x00, 0x38, 0x44, 0x82, 0x82, 0x82,
00198    0xfe, 0x82, 0x82, 0x82, 0x00, 0x00, 0x00, 0x00, 0x38, 0x44, 0x44, 0x38,
00199    0x44, 0x82, 0x82, 0x82, 0xfe, 0x82, 0x82, 0x82, 0x00, 0x00, 0x00, 0x00,
00200    0x0c, 0x30, 0x00, 0xfe, 0x80, 0x80, 0x80, 0xfc, 0x80, 0x80, 0x80, 0xfe,
00201    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x6c, 0x92, 0x12,
00202    0x7e, 0x90, 0x90, 0x6e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x3e, 0x48,
00203    0x88, 0x88, 0xfe, 0x88, 0x88, 0x88, 0x88, 0x8e, 0x00, 0x00, 0x00, 0x00,
00204    0x00, 0x10, 0x28, 0x44, 0x00, 0x7c, 0x82, 0x82, 0x82, 0x82, 0x7c,
00205    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x44, 0x44, 0x00, 0x7c, 0x82, 0x82,
00206    0x82, 0x82, 0x82, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x20, 0x10, 0x08,
00207    0x00, 0x7c, 0x82, 0x82, 0x82, 0x82, 0x82, 0x7c, 0x00, 0x00, 0x00, 0x00,
00208    0x00, 0x10, 0x28, 0x44, 0x00, 0x82, 0x82, 0x82, 0x82, 0x82, 0x86, 0x7a,
00209    0x00, 0x00, 0x00, 0x00, 0x00, 0x20, 0x10, 0x08, 0x00, 0x82, 0x82, 0x82,
00210    0x82, 0x82, 0x86, 0x7a, 0x00, 0x00, 0x00, 0x00, 0x00, 0x44, 0x44,
00211    0x00, 0x82, 0x82, 0x82, 0x82, 0x82, 0x86, 0x7a, 0x02, 0x02, 0x7c, 0x00,
00212    0x44, 0x44, 0x00, 0x7c, 0x82, 0x82, 0x82, 0x82, 0x82, 0x82, 0x7c,
00213    0x00, 0x00, 0x00, 0x00, 0x44, 0x44, 0x00, 0x82, 0x82, 0x82, 0x82, 0x82,
00214    0x82, 0x82, 0x82, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x10,
00215    0x7c, 0x82, 0x80, 0x80, 0x82, 0x7c, 0x10, 0x10, 0x00, 0x00, 0x00, 0x00,
00216    0x00, 0x00, 0x38, 0x44, 0x40, 0x40, 0xf0, 0x40, 0x40, 0x40, 0x72, 0xcc,
00217    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x82, 0x82, 0x44, 0x28, 0x10, 0x7c,
00218    0x10, 0x7c, 0x10, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xf8, 0x84,
00219    0x84, 0xf8, 0x80, 0x88, 0x9c, 0x88, 0x88, 0x84, 0x00, 0x00, 0x00, 0x00,
00220    0x00, 0x00, 0x0c, 0x12, 0x10, 0x10, 0x10, 0x7c, 0x10, 0x10, 0x10, 0x10,
00221    0x10, 0x90, 0x60, 0x00, 0x00, 0x08, 0x10, 0x20, 0x00, 0x7c, 0x02, 0x02,
00222    0x7e, 0x82, 0x82, 0x7e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x08, 0x10, 0x20,
00223    0x00, 0x70, 0x10, 0x10, 0x10, 0x10, 0x10, 0x7c, 0x00, 0x00, 0x00, 0x00,
00224    0x00, 0x08, 0x10, 0x20, 0x00, 0x7c, 0x82, 0x82, 0x82, 0x82, 0x82, 0x7c,
00225    0x00, 0x00, 0x00, 0x00, 0x00, 0x08, 0x10, 0x20, 0x00, 0x82, 0x82, 0x82,
00226    0x82, 0x82, 0x86, 0x7a, 0x00, 0x00, 0x00, 0x00, 0x00, 0x64, 0x98, 0x00,
```

```
00227 0x00, 0xbc, 0xc2, 0x82, 0x82, 0x82, 0x82, 0x00, 0x00, 0x00, 0x00,
00228 0x64, 0x98, 0x00, 0x82, 0xc2, 0xa2, 0x92, 0x8a, 0x86, 0x82, 0x82, 0x82,
00229 0x00, 0x00, 0x00, 0x00, 0x00, 0x38, 0x04, 0x3c, 0x44, 0x3c, 0x00, 0x7c,
00230 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x38, 0x44, 0x44,
00231 0x44, 0x38, 0x00, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00232 0x00, 0x00, 0x10, 0x10, 0x00, 0x10, 0x10, 0x60, 0x80, 0x82, 0x82, 0x7c,
00233 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfc, 0x80,
00234 0x80, 0x80, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00235 0x00, 0x00, 0xfc, 0x04, 0x04, 0x04, 0x04, 0x00, 0x00, 0x00, 0x00, 0x00,
00236 0x00, 0x80, 0x80, 0x84, 0x88, 0x90, 0x20, 0x40, 0x80, 0x38, 0x04, 0x08,
00237 0x10, 0x3c, 0x00, 0x00, 0x00, 0x80, 0x80, 0x80, 0x84, 0x88, 0x10, 0x20,
00238 0x44, 0x8c, 0x14, 0x3c, 0x04, 0x04, 0x00, 0x00, 0x00, 0x00, 0x10, 0x10,
00239 0x00, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x00, 0x00, 0x00, 0x00,
00240 0x00, 0x00, 0x00, 0x00, 0x00, 0x24, 0x48, 0x90, 0x48, 0x24, 0x00, 0x00,
00241 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x90, 0x48, 0x24,
00242 0x48, 0x90, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x11, 0x44, 0x11, 0x44,
00243 0x11, 0x44, 0x11, 0x44, 0x11, 0x44, 0x11, 0x44, 0x11, 0x44, 0x11, 0x44,
00244 0x55, 0xaa, 0x55, 0xaa, 0x55, 0xaa, 0x55, 0xaa, 0x55, 0xaa, 0x55, 0xaa,
00245 0x55, 0xaa, 0x55, 0xaa, 0xdd, 0x77, 0xdd, 0x77, 0xdd, 0x77, 0xdd, 0x77,
00246 0xdd, 0x77, 0xdd, 0x77, 0xdd, 0x77, 0xdd, 0x77, 0x08, 0x08, 0x08, 0x08,
00247 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08,
00248 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0xf8, 0x08, 0x08, 0x08, 0x08,
00249 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0xf8, 0x08, 0xf8,
00250 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x24, 0x24, 0x24, 0x24,
00251 0x24, 0x24, 0x24, 0xe4, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24,
00252 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfc, 0x24, 0x24, 0x24, 0x24,
00253 0x24, 0x24, 0x24, 0x24, 0x00, 0x00, 0x00, 0x00, 0x00, 0xf8, 0x08, 0xf8,
00254 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x24, 0x24, 0x24, 0x24,
00255 0x24, 0xe4, 0x04, 0xe4, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24,
00256 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24,
00257 0x24, 0x24, 0x24, 0x24, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfc, 0x04, 0xe4,
00258 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24,
00259 0x24, 0xe4, 0x04, 0xfc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00260 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0xfc, 0x00, 0x00, 0x00, 0x00,
00261 0x00, 0x00, 0x00, 0x00, 0x08, 0x08, 0x08, 0x08, 0x08, 0xf8, 0x08, 0xf8,
00262 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00263 0x00, 0x00, 0x00, 0xf8, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08,
00264 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x0f, 0x00, 0x00, 0x00, 0x00,
00265 0x00, 0x00, 0x00, 0x00, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0xf8,
00266 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00267 0x00, 0x00, 0x00, 0xff, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08,
00268 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x0f, 0x08, 0x08, 0x08, 0x08,
00269 0x08, 0x08, 0x08, 0x08, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00270 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x08, 0x08, 0x08, 0x08,
00271 0x08, 0x08, 0x08, 0xff, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08,
00272 0x08, 0x08, 0x08, 0x08, 0x08, 0x0f, 0x08, 0x0f, 0x08, 0x08, 0x08, 0x08,
00273 0x08, 0x08, 0x08, 0x08, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24,
00274 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24,
00275 0x24, 0x27, 0x20, 0x3f, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00276 0x00, 0x00, 0x00, 0x00, 0x00, 0x3f, 0x20, 0x27, 0x24, 0x24, 0x24, 0x24,
00277 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0xe7, 0x00, 0xff,
00278 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00279 0x00, 0xff, 0x00, 0xe7, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24,
00280 0x24, 0x24, 0x24, 0x24, 0x24, 0x27, 0x20, 0x27, 0x24, 0x24, 0x24, 0x24,
00281 0x24, 0x24, 0x24, 0x24, 0x00, 0x00, 0x00, 0x00, 0x00, 0xff, 0x00, 0xff,
00282 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x24, 0x24, 0x24, 0x24,
00283 0x24, 0xe7, 0x00, 0xe7, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24,
00284 0x08, 0x08, 0x08, 0x08, 0x08, 0xff, 0x00, 0xff, 0x00, 0x00, 0x00, 0x00,
00285 0x00, 0x00, 0x00, 0x00, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0xff,
00286 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00287 0x00, 0xff, 0x00, 0xff, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08,
00288 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xff, 0x24, 0x24, 0x24, 0x24,
00289 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x3f,
00290 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x08, 0x08, 0x08, 0x08,
00291 0x08, 0x0f, 0x08, 0x0f, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00292 0x00, 0x00, 0x00, 0x00, 0x00, 0x0f, 0x08, 0x0f, 0x08, 0x08, 0x08, 0x08,
00293 0x08, 0x08, 0x08, 0x08, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x3f,
00294 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24,
00295 0x24, 0x24, 0x24, 0xff, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x24,
00296 0x08, 0x08, 0x08, 0x08, 0x08, 0xff, 0x08, 0xff, 0x08, 0x08, 0x08, 0x08,
00297 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0xf8,
00298 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00299 0x00, 0x00, 0x00, 0x0f, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08, 0x08,
00300 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
00301 0xff, 0xff, 0xff, 0xff, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xff,
00302 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0xf0, 0xf0, 0xf0,
00303 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0,
00304 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0,
00305 0xf0, 0xf0, 0xf0, 0xf0, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0x00,
00306 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00307 0x00, 0x74, 0x88, 0x88, 0x88, 0x88, 0x88, 0x76, 0x00, 0x00, 0x00, 0x00,
00308 0x00, 0x00, 0x38, 0x44, 0x84, 0x84, 0x98, 0x84, 0x82, 0x82, 0x82, 0x9c,
00309 0x80, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0x80, 0x80, 0x80, 0x80,
00310 0x80, 0x80, 0x80, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00311 0x00, 0xfe, 0x44, 0x44, 0x44, 0x44, 0x42, 0x00, 0x00, 0x00, 0x00,
00312 0x00, 0x00, 0xfe, 0x40, 0x20, 0x10, 0x08, 0x10, 0x20, 0x40, 0x80, 0xfe,
00313 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7e, 0x88, 0x84,
```

```

00314    0x84, 0x84, 0x84, 0x78, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00315    0x00, 0x84, 0x84, 0x84, 0x84, 0x84, 0xc4, 0xba, 0x80, 0x80, 0x80, 0x00,
00316    0x00, 0x00, 0x00, 0x00, 0x00, 0x7e, 0x90, 0x10, 0x10, 0x10, 0x10, 0x0c,
00317    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x7c, 0x92, 0x92, 0x92, 0x92,
00318    0x92, 0x92, 0x7c, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x38, 0x44,
00319    0x82, 0x82, 0xfe, 0x82, 0x82, 0x82, 0x44, 0x38, 0x00, 0x00, 0x00, 0x00,
00320    0x00, 0x00, 0x38, 0x44, 0x82, 0x82, 0x82, 0x82, 0x82, 0x44, 0x28, 0xee,
00321    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x40, 0x20, 0x10, 0x78, 0x84,
00322    0x82, 0x82, 0x82, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00323    0x00, 0x6c, 0x92, 0x92, 0x92, 0x6c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00324    0x00, 0x00, 0x00, 0x10, 0x10, 0x7c, 0x92, 0x92, 0x92, 0x92, 0x92, 0x7c,
00325    0x10, 0x10, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x82, 0x80,
00326    0x78, 0x80, 0x82, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c,
00327    0x82, 0x82, 0x82, 0x82, 0x82, 0x82, 0x82, 0x00, 0x00, 0x00, 0x00,
00328    0x00, 0x00, 0x00, 0x00, 0xfe, 0x00, 0x00, 0xfe, 0x00, 0x00, 0xfe, 0x00,
00329    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x10, 0x10, 0xfe, 0x10, 0x10,
00330    0x10, 0x00, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x20,
00331    0x10, 0x08, 0x04, 0x08, 0x10, 0x20, 0x00, 0x7c, 0x00, 0x00, 0x00, 0x00,
00332    0x00, 0x00, 0x00, 0x08, 0x10, 0x20, 0x40, 0x20, 0x10, 0x08, 0x00, 0x7c,
00333    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0c, 0x12, 0x12, 0x10, 0x10, 0x10,
00334    0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10,
00335    0x10, 0x10, 0x10, 0x10, 0x10, 0x90, 0x90, 0x60, 0x00, 0x00, 0x00, 0x00,
00336    0x00, 0x00, 0x00, 0x00, 0x10, 0x10, 0x00, 0xfe, 0x00, 0x10, 0x10, 0x00,
00337    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x64, 0x98, 0x00,
00338    0x64, 0x98, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x70, 0x88, 0x88,
00339    0x88, 0x70, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00340    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x30, 0x00, 0x00, 0x00,
00341    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00342    0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0e, 0x08, 0x08,
00343    0x08, 0x08, 0x08, 0x08, 0xc8, 0x28, 0x18, 0x08, 0x00, 0x00, 0x00, 0x00,
00344    0x00, 0xb8, 0x44, 0x44, 0x44, 0x44, 0x44, 0x00, 0x00, 0x00, 0x00, 0x00,
00345    0x00, 0x00, 0x00, 0x00, 0x00, 0x60, 0x90, 0x10, 0x20, 0x40, 0xf0, 0x00,
00346    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00347    0x7c, 0x7c, 0x7c, 0x7c, 0x7c, 0x7c, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00,
00348    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00349    0x00, 0x00, 0x00, 0x00};
00350 #endif // VGAFONT_H

```

7.115 src/include/voyager_docs.h File Reference

Doxygen landing pages and subsystem groups for VoyagerOS64.

7.115.1 Detailed Description

Doxygen landing pages and subsystem groups for VoyagerOS64.

This file intentionally contains documentation only. It is scanned by Doxygen because the configured input recursively includes src/include.

Definition in file [voyager_docs.h](#).

7.116 voyager_docs.h

[Go to the documentation of this file.](#)

```

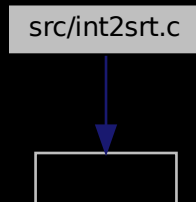
00001
00008 #pragma once
00009

```

7.117 src/int2srt.c File Reference

```
#include <stdint.h>
```

Include dependency graph for int2srt.c:



Functions

- char * `int_to_str` (int value, char *buffer)
- void `u64_to_str` (uint64_t val, char *buf)

7.117.1 Function Documentation

7.117.1.1 int_to_str()

```
char * int_to_str (
    int value,
    char * buffer )
```

Copyright (c) 2025 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

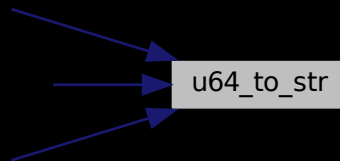
Definition at line 10 of file `int2srt.c`.

7.117.1.2 u64_to_str()

```
void u64_to_str (
    uint64_t val,
    char * buf )
```

Definition at line 42 of file [int2srt.c](#).

Here is the caller graph for this function:



7.118 int2srt.c

[Go to the documentation of this file.](#)

```
00001
00008 #include <stdint.h>
00009
00010 char *int_to_str(int value, char *buffer)
00011 {
00012     int is_negative = 0;
00013     if (value < 0)
00014     {
00015         is_negative = 1;
00016         value      = -value;
00017     }
00018
00019     int i = 0;
00020     do
00021     {
00022         int digit = value % 10;
00023         buffer[i++] = '0' + digit;
00024         value /= 10;
00025     } while (value > 0);
00026
00027     if (is_negative)
00028         buffer[i++] = '-';
00029
00030     buffer[i] = '\0';
00031
00032     for (int j = 0; j < i / 2; j++)
00033     {
00034         char tmp = buffer[j];
00035         buffer[j] = buffer[i - j - 1];
00036         buffer[i - j - 1] = tmp;
00037     }
00038
00039     return buffer;
00040 }
00041
00042 void u64_to_str(uint64_t val, char *buf)
00043 {
00044     char tmp[21];
```

```

00045     int i = 0;
00046     do
00047     {
00048         tmp[i++] = '0' + (val % 10);
00049         val /= 10;
00050     } while (val > 0);
00051     buf[i] = '\0';
00052     for (int j = 0; j < i; j++)
00053         buf[j] = tmp[i - j - 1];
00054     buf[i] = '\0';
00055 }

```

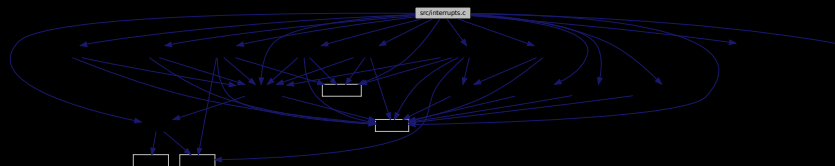
7.119 src/interrupts.c File Reference

```

#include "include/interrupts.h"
#include "include/KernelUtils.h"
#include "include/drivers/keyboard/keyboard.h"
#include "include/global_defs.h"
#include "include/idt.h"
#include "include/io.h"
#include "include/kernel.h"
#include "include/panic.h"
#include "include/pic.h"
#include "include/printf.h"
#include "include/registers.h"
#include "include/sched.h"
#include "include/stack_trace.h"
#include "include/time.h"
#include <stdbool.h>
#include <stdint.h>

```

Include dependency graph for interrupts.c:



Functions

- void `isr_exception_handler` (`isr_xframe_t` *frame, uint64_t rsi)
- void `dyn_isr_handler` (uint64_t isr)
- void `irq_handler` (`isr_xframe_t` *frame)

Variables

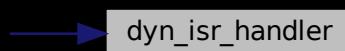
- static const char * `exception_messages` []

7.119.1 Function Documentation

7.119.1.1 dyn_isr_handler()

```
void dyn_isr_handler (  
    uint64_t isr )
```

Here is the caller graph for this function:

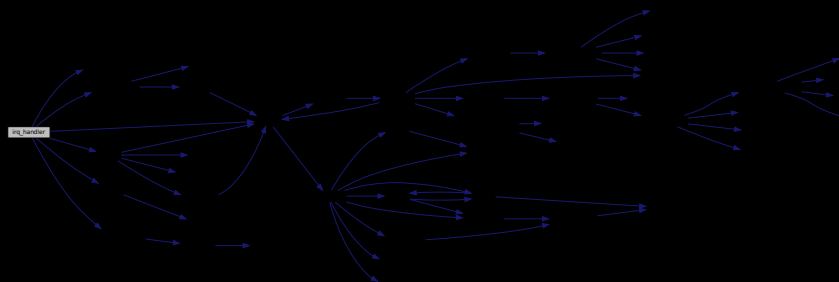


7.119.1.2 irq_handler()

```
void irq_handler (  
    isr_xframe_t * frame )
```

Definition at line 99 of file `interrupts.c`.

Here is the call graph for this function:



7.119.1.3 isr_exception_handler()

```
void isr_exception_handler (  
    isr_xframe_t * frame,  
    uint64_t rsi )
```

Definition at line 51 of file `interrupts.c`.


```

00036         "Machine Check",
00037         "Reserved",
00038         "Reserved",
00039         "Reserved",
00040         "Reserved",
00041         "Reserved",
00042         "Reserved",
00043         "Reserved",
00044         "Reserved",
00045         "Reserved",
00046         "Reserved",
00047         "Reserved",
00048         "Reserved",
00049         "Reserved"};
00050
00051 void isr_exception_handler(isr_xframe_t *frame, uint64_t rsi)
00052 {
00053     (void) rsi;
00054
00055     printf("%s\n", "!!!!!!!!!!!!!!!!!!!!!!!!!!KERNEL PANIC!!!!!!!!!!!!!!!!!!!!!!!!!!");
00056
00057     kerror_mode = 1;
00058
00059     printf("%s", "ERROR: CPU EXCEPTION: ");
00060     printf("%s", exception_messages[frame->base_frame.vector]);
00061     printf("%s", " @ ");
00062     printf("0x%llx\n", frame->base_frame.rip);
00063     printf("%s", "FROM VECTOR NUMBER: ");
00064     printf("%i\n", frame->base_frame.vector);
00065     printf("%s", "ERROR CODE: ");
00066     printf("0x%llx\n", frame->base_frame.error_code);
00067     printf("%s", "CS REGISTER: ");
00068     printf("0x%llx\n", frame->base_frame.cs);
00069     printf("%s", "CR2 REGISTER: ");
00070     printf("0x%llx\n", frame->control_registers.cr2);
00071     printf("%s", "CR4 REGISTER: ");
00072     printf("0x%llx\n", frame->control_registers.cr4);
00073     printf("%s", "RSP REGISTER: ");
00074     printf("0x%lx\n", frame->base_frame.rsp);
00075     printf("%s", "RFLAGS REGISTER: ");
00076     printf("0x%llx\n", frame->base_frame.rflags);
00077
00078     if (k_mode.stack_trace_on_fault == 1)
00079     {
00080         printf("%s", "NOTE: Will Page Fault if frames run out before the ");
00081         printf("%u", k_mode.stack_trace_size);
00082         printf("%s\n",
00083             " frame cap, this is harmless disregard it and the resulting "
00084             "additional trace.");
00085         stack_trace(0);
00086     }
00087     else if (k_mode.stack_trace_on_fault == 2)
00088     {
00089         stack_dump_asm(1, 0x1000);
00090     }
00091
00092     printf("%s\n", "!!!!!!!!!!!!!!!!!!!!!!!!!!KERNEL PANIC!!!!!!!!!!!!!!!!!!!!!!!!!!");
00093
00094     __asm__ volatile("cli; hlt");
00095 }
00096
00097 extern void dyn_isr_handler(uint64_t isr);
00098
00099 void irq_handler(isr_xframe_t *frame)
00100 {
00101     uint64_t vector = frame->base_frame.vector;
00102
00103     if (vector >= 32 && vector < 48)
00104     {
00105         switch (vector)
00106         {
00107             case 32:
00108                 sys_clock_handler_alt();
00109                 break;
00110             case 33:
00111                 keyboard_handler();
00112                 break;
00113         }
00114
00115         pic_send_eoi(vector - 32);
00116         return;
00117     }
00118
00119     if (vector >= 48)
00120     {
00121         printf("IRQ Handoff to Dynamic Handler Using Vector:  %i With Handler "
00122             "Located At Address:  0x%llx\n",

```

```

00123         vector,
00124         isr_delta[vector]);
00125
00126     if (isr_delta[vector] == NULL)
00127     {
00128         panic("Kernel Panic: Invalid/NULL Dynamic ISR Vector! Not my "
00129             "fault... ok it's my fault.");
00130     }
00131
00132     dyn_isr_handler((uint64_t) (uintptr_t) isr_delta[vector]);
00133     return;
00134 }
00135
00136 panic("irq_handler: non-IRQ vector %i reached IRQ path", vector);
00137 }

```

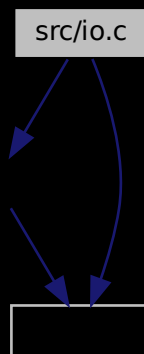
7.121 src/io.c File Reference

```

#include "include/io.h"
#include <stdint.h>

```

Include dependency graph for io.c:



Functions

- uint8_t **inb** (uint16_t port)
Read one byte from an x86 I/O port.
- void **outb** (uint16_t port, uint8_t val)
Write one byte to an x86 I/O port.
- void **io_wait** (void)
Perform the traditional short I/O delay used around legacy devices.

7.121.1 Function Documentation

7.121.1.1 inb()

```
uint8_t inb (  
    uint16_t port )
```

Read one byte from an x86 I/O port.

Parameters

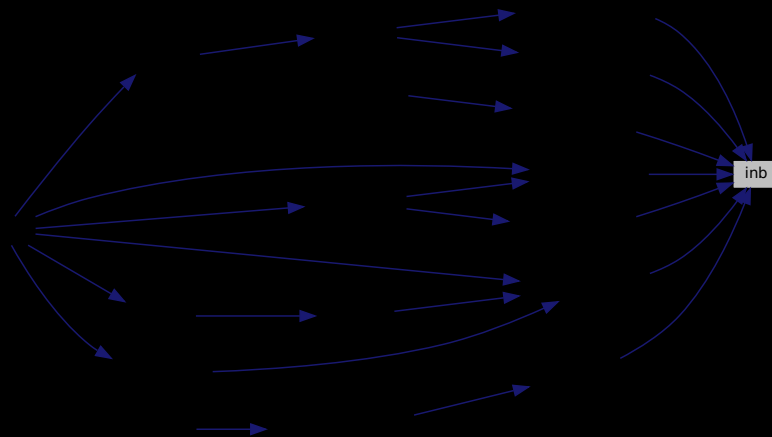
<i>port</i>	I/O-port address.
-------------	-------------------

Returns

Byte read from the requested port.

Definition at line 5 of file [io.c](#).

Here is the caller graph for this function:

7.121.1.2 `io_wait()`

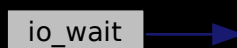
```
void io_wait (
    void )
```

Perform the traditional short I/O delay used around legacy devices.

This helper is used by hardware such as the PIC/PIT/PS2 paths where ordered or slightly delayed port transactions are required by the device interface.

Definition at line 17 of file [io.c](#).

Here is the call graph for this function:



7.121.1.3 outb()

```
void outb (
    uint16_t port,
    uint8_t val )
```

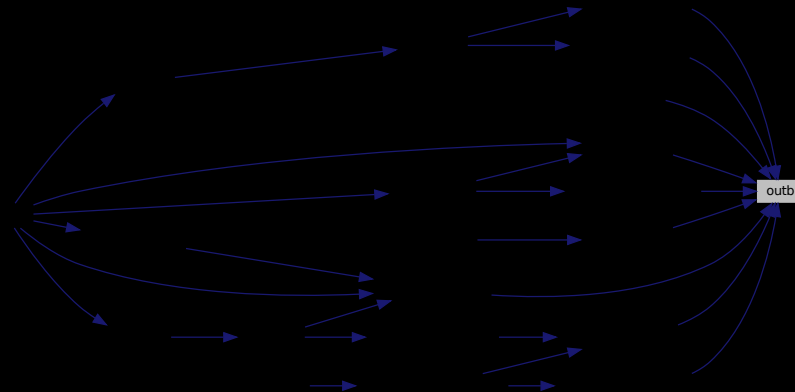
Write one byte to an x86 I/O port.

Parameters

<i>port</i>	I/O-port address.
<i>val</i>	Byte to write.

Definition at line 12 of file [io.c](#).

Here is the caller graph for this function:



7.122 io.c

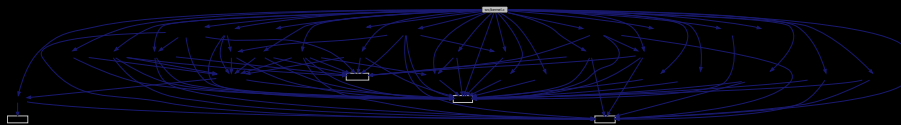
[Go to the documentation of this file.](#)

```
00001 #include "include/io.h"
00002 #include <stdint.h>
00003
00004 /* Public API documentation lives in include/io.h. */
00005 uint8_t inb(uint16_t port)
00006 {
00007     uint8_t ret;
00008     asm volatile("inb %1, %0" : "=a"(ret) : "Nd"(port));
00009     return ret;
00010 }
00011
00012 void outb(uint16_t port, uint8_t val)
00013 {
00014     asm volatile("outb %1, %0" :: "Nd"(port), "a"(val));
00015 }
00016
00017 void io_wait(void)
00018 {
00019     outb(0x80, 0);
00020 }
```

7.123 src/kernel.c File Reference

```
#include "include/KernelUtils.h"
#include "include/acpi/acpi.h"
#include "include/apic/lapic.h"
#include "include/consts.h"
#include "include/cpuUtils.h"
#include "include/drivers/keyboard/keyboard.h"
#include "include/early_alloc.h"
#include "include/gdt.h"
#include "include/idt.h"
#include "include/io.h"
#include "include/kernel.h"
#include "include/limine.h"
#include "include/mm/kmalloc.h"
#include "include/paging/frame_supplier.h"
#include "include/paging/neo_framealloc.h"
#include "include/paging/paging.h"
#include "include/paging/paging_bootstrap.h"
#include "include/panic.h"
#include "include/pic.h"
#include "include/printf.h"
#include "include/registers.h"
#include "include/sched.h"
#include "include/serial.h"
#include "include/shell.h"
#include "include/stack_trace.h"
#include "include/terminal/framebuffer.h"
#include "include/terminal/term.h"
#include "include/time.h"
#include <stddef.h>
#include <stdint.h>
```

Include dependency graph for kernel.c:



Macros

- `#define` **KERNEL_VERSION** "0.0.5"
- `#define` **White** "\033[1;00m"
- `#define` **Red** "\033[1;31m"
- `#define` **Green** "\033[1;32m"
- `#define` **Yellow** "\033[1;33m"
- `#define` **Blue** "\033[1;34m"
- `#define` **Purple** "\033[1;35m"
- `#define` **Cyan** "\033[1;36m"
- `#define` **Black** "\033[1;37m"

Functions

- void `breakpoint` (void)
- void `stop_interrupts` (void)
- void `start_interrupts` (void)
- void `halt` (void)
- void `task_switch_int` (void)
- uint64_t `walk_stack` (uint64_t *array, uint64_t max)
- static uint64_t `supply_from_early` (void)
- static uint64_t `supply_from_frame` (void)
- static void `limine_debug` (void)
- static void `memory_bringup` (void)
- static void `scheduler_heartbeat` (const char *message)
- void `proc_a` (void)
- void `proc_b` (void)
- void `hello_general_floatius` (void)
- void `_start` (void)

Variables

- uint32_t `term_fg` = 0x0055ff55
- uint32_t `term_bg` = 0x00000000
- volatile struct `limine_kernel_address_request` `Kaddress_req`
- volatile struct `limine_terminal_request` `early_term`
- volatile struct `limine_framebuffer_request` `fbr_req`
- uint32_t `bootspace` = 2
Console-routing state used during the bootloader-to-Voyager handoff.
- uint8_t `kerror_mode` = 0
Global kernel error/diagnostic mode state.
- struct `term_context` * `term_context`
- uint8_t `init_done` = 0
Nonzero once the main kernel initialization sequence is complete.
- volatile struct `limine_hhdm_request` `hhdm_request`
- volatile struct `limine_memmap_request` `memmap_req`

7.123.1 Macro Definition Documentation

7.123.1.1 Black

```
#define Black "\033[1;37m"
```

Definition at line 46 of file `kernel.c`.

7.123.1.2 Blue

```
#define Blue "\033[1;34m"
```

Definition at line 43 of file [kernel.c](#).

7.123.1.3 Cyan

```
#define Cyan "\033[1;36m"
```

Definition at line 45 of file [kernel.c](#).

7.123.1.4 Green

```
#define Green "\033[1;32m"
```

Definition at line 41 of file [kernel.c](#).

7.123.1.5 KERNEL_VERSION

```
#define KERNEL_VERSION "0.0.5"
```

7.123.1.6 Purple

```
#define Purple "\033[1;35m"
```

Definition at line 44 of file [kernel.c](#).

7.123.1.7 Red

```
#define Red "\033[1;31m"
```

Definition at line 40 of file [kernel.c](#).

7.123.1.8 White

```
#define White "\033[1;00m"
```

Copyright (c) 2025 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Definition at line 39 of file [kernel.c](#).

7.123.1.9 Yellow

```
#define Yellow "\033[1;33m"
```

Definition at line 42 of file [kernel.c](#).

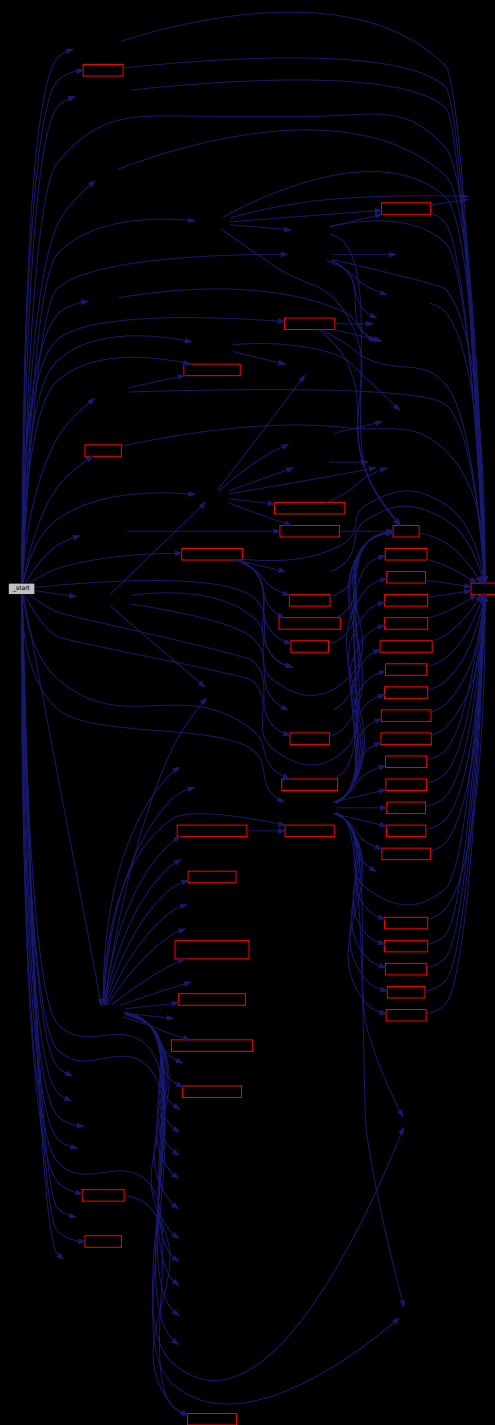
7.123.2 Function Documentation

7.123.2.1 _start()

```
void _start (  
            void )
```

Definition at line 174 of file [kernel.c](#).

Here is the call graph for this function:



7.123.2.2 breakpoint()

```
void breakpoint (  
    void )
```

7.123.2.3 halt()

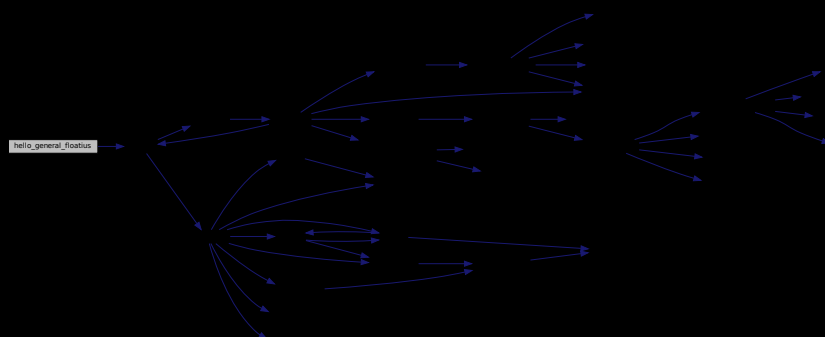
```
void halt (  
    void )
```

7.123.2.4 hello_general_floatius()

```
void hello_general_floatius (  
    void )
```

Definition at line 162 of file [kernel.c](#).

Here is the call graph for this function:

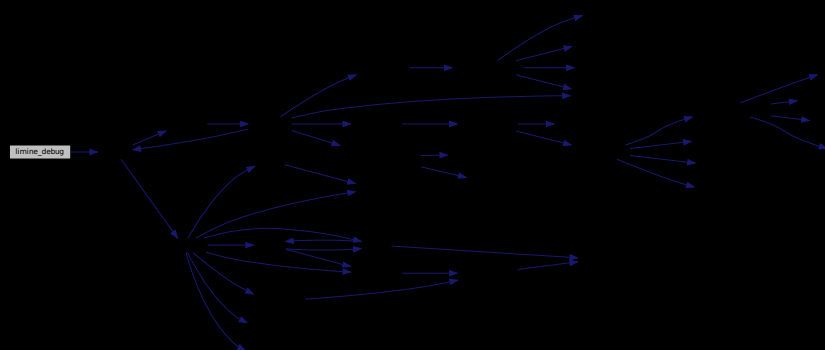


7.123.2.5 limine_debug()

```
static void limine_debug (  
    void ) [static]
```

Definition at line 80 of file [kernel.c](#).

Here is the call graph for this function:

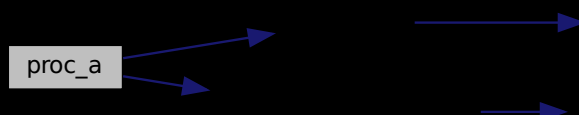


7.123.2.7 proc_a()

```
void proc_a (  
    void )
```

Definition at line 140 of file [kernel.c](#).

Here is the call graph for this function:

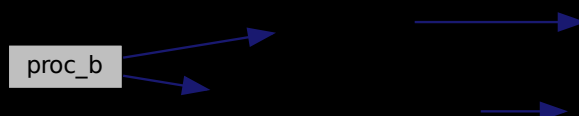


7.123.2.8 proc_b()

```
void proc_b (  
    void )
```

Definition at line 151 of file [kernel.c](#).

Here is the call graph for this function:

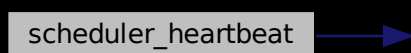


7.123.2.9 scheduler_heartbeat()

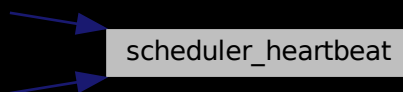
```
static void scheduler_heartbeat (  
    const char * message ) [static]
```

Definition at line 133 of file [kernel.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.123.2.10 start_interrupts()

```
void start_interrupts (  
    void )
```

7.123.2.11 stop_interrupts()

```
void stop_interrupts (  
    void )
```

→ stop_interrupts

→ supply_from_early

7.123.3 Variable Documentation

7.123.3.1 bootspace

```
uint32_t bootspace = 2
```

Console-routing state used during the bootloader-to-Voyager handoff.

Boot code changes this while output ownership moves from early boot services to Voyager's serial/framebuffer terminal. Code running after Voyager installs its own CR3 must not assume Limine terminal callbacks remain callable.

Definition at line 62 of file [kernel.c](#).

7.123.3.2 early_term

```
volatile struct limine_terminal_request early_term [extern]
```

7.123.3.3 fbr_req

```
volatile struct limine_framebuffer_request fbr_req [extern]
```

7.123.3.4 hhdm_request

```
volatile struct limine_hhdm_request hhdm_request [extern]
```

7.123.3.5 init_done

```
uint8_t init_done = 0
```

Nonzero once the main kernel initialization sequence is complete.

Definition at line 65 of file [kernel.c](#).

7.123.3.6 Kaddress_req

```
volatile struct limine\_kernel\_address\_request Kaddress_req [extern]
```

7.123.3.7 kerror_mode

```
uint8_t kerror_mode = 0
```

Global kernel error/diagnostic mode state.

Definition at line 63 of file [kernel.c](#).

7.123.3.8 memmap_req

```
volatile struct limine\_memmap\_request memmap_req [extern]
```

7.123.3.9 term_bg

```
uint32_t term_bg = 0x00000000
```

Definition at line 49 of file [kernel.c](#).

7.123.3.10 term_context

```
struct term\_context* term_context
```

Voyager-owned terminal context after framebuffer handoff.

Definition at line 64 of file [kernel.c](#).

7.123.3.11 term_fg

```
uint32_t term_fg = 0x0055ff55
```

Definition at line 48 of file [kernel.c](#).

7.124 kernel.c

Go to the documentation of this file.

```

00001
00008 #include "include/KernelUtils.h"
00009 #include "include/acpi/acpi.h"
00010 #include "include/apic/lapic.h"
00011 #include "include/consts.h"
00012 #include "include/cpuUtils.h"
00013 #include "include/drivers/keyboard/keyboard.h"
00014 #include "include/early_alloc.h"
00015 #include "include/gdt.h"
00016 #include "include/idt.h"
00017 #include "include/io.h"
00018 #include "include/kernel.h"
00019 #include "include/limine.h"
00020 #include "include/mem/kmalloc.h"
00021 #include "include/paging/frame_supplier.h"
00022 #include "include/paging/neo_framealloc.h"
00023 #include "include/paging/paging.h"
00024 #include "include/paging/paging_bootstrap.h"
00025 #include "include/panic.h"
00026 #include "include/pic.h"
00027 #include "include/printf.h"
00028 #include "include/registers.h"
00029 #include "include/sched.h"
00030 #include "include/serial.h"
00031 #include "include/shell.h"
00032 #include "include/stack_trace.h"
00033 #include "include/terminal/framebuffer.h"
00034 #include "include/terminal/term.h"
00035 #include "include/time.h"
00036 #include <stddef.h>
00037 #include <stdint.h>
00038
00039 #define White "\033[1;00m"
00040 #define Red "\033[1;31m"
00041 #define Green "\033[1;32m"
00042 #define Yellow "\033[1;33m"
00043 #define Blue "\033[1;34m"
00044 #define Purple "\033[1;35m"
00045 #define Cyan "\033[1;36m"
00046 #define Black "\033[1;37m"
00047
00048 uint32_t term_fg = 0x0055ff55;
00049 uint32_t term_bg = 0x00000000;
00050
00051 extern volatile struct limine_kernel_address_request Kaddress_req;
00052 extern volatile struct limine_terminal_request early_term;
00053 extern volatile struct limine_framebuffer_request fbr_req;
00054
00055 extern void breakpoint(void);
00056 extern void stop_interrupts(void);
00057 extern void start_interrupts(void);
00058 extern void halt(void);
00059 extern void task_switch_int(void);
00060 extern uint64_t walk_stack(uint64_t *array, uint64_t max);
00061
00062 uint32_t bootspace = 2;
00063 uint8_t kerror_mode = 0;
00064 struct term_context *term_context;
00065 uint8_t init_done = 0;
00066
00067 static uint64_t supply_from_early(void)
00068 {
00069     return early_alloc_page();
00070 }
00071
00072 static uint64_t supply_from_frame(void)
00073 {
00074     return frame_alloc();
00075 }
00076
00077 extern volatile struct limine_hhdm_request hhdm_request;
00078 extern volatile struct limine_mmap_request mmap_req;
00079
00080 static void limine_debug(void)
00081 {
00082     if (hhdm_request.response == NULL)
00083     {
00084         printf_("[LIMINE] HHDM request: NULL\n");
00085     }
00086     else
00087     {
00088         printf_("[LIMINE] HHDM offset: 0x%llx\n",

```

```

00089         (unsigned long long) hhdm_request.response->offset);
00090     }
00091
00092     if (kaddress_req.response == NULL)
00093     {
00094         printf_("[LIMINE] Kernel address request:  NULL\n");
00095     }
00096     else
00097     {
00098         printf_("[LIMINE] Kernel phys base:  0x%llx virt base:  0x%llx\n",
00099             (unsigned long long) kaddress_req.response->physical_base,
00100             (unsigned long long) kaddress_req.response->virtual_base);
00101     }
00102
00103     if (memmap_req.response == NULL)
00104     {
00105         printf_("[LIMINE] Memory map request:  NULL\n");
00106     }
00107     else
00108     {
00109         printf_("[LIMINE] Memmap entries:  %llu\n",
00110             (unsigned long long) memmap_req.response->entry_count);
00111     }
00112 }
00113
00114 static void memory_bringup(void)
00115 {
00116     printf_("%s\n", "Bootstrapping Memory...  calling bootstrap alloc...  ");
00117     early_init();
00118
00119     printf_("%s\n", "Bootstrapping Memory...  getting frames...  ");
00120     g_boot_phys_page = supply_from_early;
00121
00122     printf_("%s\n",
00123         "Bootstrapping Memory...  building PML4 -> PDPT -> PD -> PT...  ");
00124     limine_debug();
00125     paging_bootstrap();
00126
00127     printf_("CR3 after bootstrap:  0x%llx\n", readCR3());
00128
00129     frame_init();
00130     g_boot_phys_page = supply_from_frame;
00131 }
00132
00133 static void scheduler_heartbeat(const char *message)
00134 {
00135     while (*message)
00136         serial_debug(*message++);
00137     serial_debug('\n');
00138 }
00139
00140 void proc_a(void)
00141 {
00142     while (1)
00143     {
00144         scheduler_heartbeat("I'm a thread");
00145         for (volatile int i = 0; i < 1000000; i++)
00146             ;
00147         schedule();
00148     }
00149 }
00150
00151 void proc_b(void)
00152 {
00153     while (1)
00154     {
00155         scheduler_heartbeat("I'm a catgirl");
00156         for (volatile int i = 0; i < 1000000; i++)
00157             ;
00158         schedule();
00159     }
00160 }
00161
00162 void hello_general_floatius(void)
00163 {
00164     double t;
00165     double x = 5.239;
00166
00167     t = 5 / 2;
00168     x = t * 6.4;
00169
00170     printf_("%g\n", t);
00171     printf_("%g\n", x);
00172 }
00173
00174 void _start(void)
00175 {

```

```

00176     if (early_term.response == NULL || early_term.response->terminal_count < 1)
00177     {
00178         bootspace = 1;
00179         printf_("%s\n",
00180             "WARNING: Bootloader Terminal Offline Using Serial Only!");
00181     }
00182
00183     printf_("%s", "Early Terminal Using Framebuffer At Physical Address: ");
00184     printf_("0x%llx\n",
00185         TranslateToPhysicalMemoryAddress(
00186             (uint64_t) (uintptr_t) early_term.response->terminals[0]
00187                 ->framebuffer));
00188     printf_("%s", "And At Virtual Address: ");
00189     printf_(
00190         "0x%llx\n",
00191         (uint64_t) (uintptr_t) early_term.response->terminals[0]->framebuffer);
00192
00193     print_stack_size();
00194     print_date();
00195     cpuid_readout();
00196
00197     if (k_mode.hw_rng_support == 1)
00198     {
00199         printf_("%s", "Random Number Gen (HW) Test: ");
00200         printf_("%u\n", rand_asm());
00201     }
00202
00203     stop_interrupts();
00204     LoadGDT_Stagel();
00205     printf_("%s\n", "Loaded GDT");
00206
00207     idt_init();
00208     printf_("%s\n", "Loaded IDT");
00209
00210     pic_enable();
00211     printf_("%s\n", "PICs Online");
00212
00213     time_init();
00214
00215     asm volatile("cli");
00216     asm volatile("sti");
00217     asm volatile("cli");
00218
00219     idt_reload();
00220     gdt_reload();
00221
00222     asm volatile("cli");
00223
00224     idt_reg_test();
00225     asm volatile("int $48");
00226
00227     yield_register();
00228     asm volatile("int $49");
00229
00230     memory_bringup();
00231     kmalloc_init();
00232
00233     /* Keep PIT preemption parked during the cooperative 0.0.5 bring-up. */
00234     pic_mask_irq(0);
00235     asm volatile("sti");
00236
00237     if (Kaddress_req.response == NULL)
00238     {
00239         printf_("%s\n", "!!!Error While Fetching Kernel Addresses!!!");
00240     }
00241     else
00242     {
00243         printf_("%s\n", "Kernel Base Addresses Are As Follows: ");
00244         printf_("%s", "Physical Address: ");
00245         printf_("0x%llx\n", Kaddress_req.response->physical_base);
00246         printf_("%s", "Virtual Address: ");
00247         printf_("0x%llx\n", Kaddress_req.response->virtual_base);
00248         printf_("%s\n", "-----");
00249     }
00250
00251     printf_("%s", "CR3: ");
00252     printf_("0x%llx\n", readCR3());
00253     printf_("%s", "CR0: ");
00254     printf_("0x%llx\n", readCR0());
00255     printf_("%s", "CR4: ");
00256     printf_("0x%llx\n", readCR4());
00257
00258     if (k_mode.acpi_support == 1)
00259         acpi_init();
00260
00261     printf_("%s\n", "Handing Control to Standalone Terminal...");
00262

```

```

00263      /* Once Voyager owns CR3, Limine's terminal callback is no longer safe.
00264  * Stay on serial while fbterm takes ownership of the framebuffer. */
00265      bootspace = 1;
00266
00267      if (fbr_req.response == NULL || fbr_req.response->framebuffer_count < 1)
00268          panic("Framebuffer terminal requested without a framebuffer");
00269
00270      term_context = fbterm_init(kmalloc,
00271                                fbr_req.response->framebuffers[0]->address,
00272                                fbr_req.response->framebuffers[0]->width,
00273                                fbr_req.response->framebuffers[0]->height,
00274                                fbr_req.response->framebuffers[0]->pitch,
00275                                NULL,
00276                                NULL,
00277                                NULL,
00278                                &term_bg,
00279                                &term_fg,
00280                                NULL,
00281                                0,
00282                                0,
00283                                0,
00284                                1,
00285                                1,
00286                                1);
00287
00288      if (!term_context)
00289          panic("Framebuffer terminal initialization failed");
00290
00291      if (term_context->clear)
00292          term_context->clear(term_context, true);
00293      if (term_context->full_refresh)
00294          term_context->full_refresh(term_context);
00295
00296      bootspace = 0;
00297      keyboard_init();
00298
00299      pic_unmask_irq(0);
00300
00301      print_load_time();
00302      print_date();
00303      print_stack_size();
00304
00305      printf("%s", "VoyagerOS64 ");
00306      printf("%s\n", KERNEL_VERSION);
00307
00308      init_done = 1;
00309
00310      bootspace = 1;
00311      if (k_mode.addr_debug == 1)
00312      {
00313          printf("Neo PMM frames: total=%llu free=%llu\n",
00314                (unsigned long long) frame_total_count(),
00315                (unsigned long long) frame_free_count());
00316      }
00317      bootspace = 0;
00318
00319      init_scheduler();
00320      // create_process(proc_a);
00321      // create_process(proc_b);
00322      create_process(vsh_loop);
00323
00324      printf("%s\n", "Scheduler cooperative round-robin online.");
00325      scheduler_start();
00326
00327      panic("scheduler returned control to kernel bootstrap");
00328 }

```

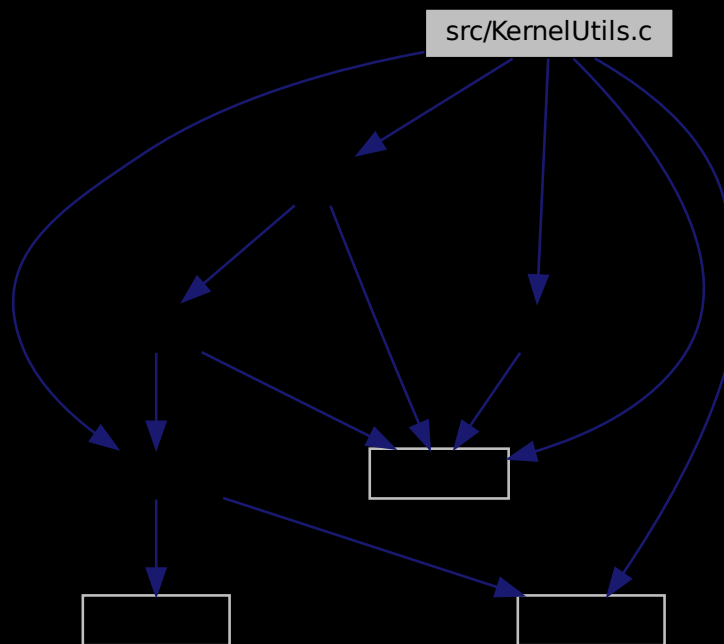
7.125 src/KernelUtils.c File Reference

```

#include "include/KernelUtils.h"
#include "include/limine.h"
#include "include/printf.h"
#include <stddef.h>
#include <stdint.h>

```

Include dependency graph for KernelUtils.c:



Functions

- `uint64_t bytes_to_mib (uint64_t bytes)`
Convert a byte count to MiB.
- `uint64_t bytes_to_gib (uint64_t bytes)`
Convert a byte count to GiB.
- `uint64_t get_memory_size (void)`
Return detected physical memory size in the implementation's base unit.
- `uint64_t get_memory_size_gib (void)`
Return detected memory size expressed in GiB.
- `void print_memmap (void)`
Print the Limine-provided physical memory map for diagnostics.

Variables

- volatile struct `limine_memmap_request memmap_req`
- const struct `kswitches k_mode`

7.125.1 Function Documentation

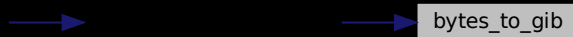
7.125.1.1 bytes_to_gib()

```
uint64_t bytes_to_gib (  
    uint64_t bytes )
```

Convert a byte count to GiB.

Definition at line 27 of file [KernelUtils.c](#).

Here is the caller graph for this function:



7.125.1.2 bytes_to_mib()

```
uint64_t bytes_to_mib (  
    uint64_t bytes )
```

Convert a byte count to MiB.

Definition at line 22 of file [KernelUtils.c](#).

Here is the caller graph for this function:



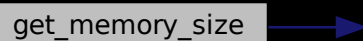
7.125.1.3 `get_memory_size()`

```
uint64_t get_memory_size (  
    void )
```

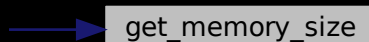
Return detected physical memory size in the implementation's base unit.

Definition at line 32 of file [KernelUtils.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



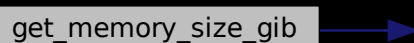
7.125.1.4 `get_memory_size_gib()`

```
uint64_t get_memory_size_gib (  
    void )
```

Return detected memory size expressed in GiB.

Definition at line 45 of file [KernelUtils.c](#).

Here is the call graph for this function:



7.125.2.1 k_mode

```
const struct kswitches k_mode
```

Initial value:

```
= {
    .stack_trace_size      = 24,
    .stack_trace_on_fault = 1,
    .acpi_support          = 0,
    .sched_debug           = 0,
    .addr_debug            = 0,
    .hw_rng_support        = 1,
    .mem_readout_unit      = 1,
    .liballoc_debug        = 0,
    .fpu_allowed           = 0,
    .timestamp             = 1
}
```

Active kernel behavior/debug configuration.

Definition at line 9 of file [KernelUtils.c](#).

7.125.2.2 memmap_req

```
volatile struct limine_memmap_request memmap_req [extern]
```

7.126 KernelUtils.c

[Go to the documentation of this file.](#)

```
00001 #include "include/KernelUtils.h"
00002 #include "include/limine.h"
00003 #include "include/printf.h"
00004 #include <stddef.h>
00005 #include <stdint.h>
00006
00007 extern volatile struct limine_memmap_request memmap_req;
00008
00009 const struct kswitches k_mode = {
00010     .stack_trace_size      = 24,
00011     .stack_trace_on_fault = 1,
00012     .acpi_support          = 0,
00013     .sched_debug           = 0,
00014     .addr_debug            = 0,
00015     .hw_rng_support        = 1,
00016     .mem_readout_unit      = 1,
00017     .liballoc_debug        = 0,
00018     .fpu_allowed           = 0,
00019     .timestamp             = 1
00020 };
00021
00022 uint64_t bytes_to_mib(uint64_t bytes)
00023 {
00024     return bytes » 20;
00025 }
00026
00027 uint64_t bytes_to_gib(uint64_t bytes)
00028 {
00029     return bytes » 30;
00030 }
00031
00032 uint64_t get_memory_size(void)
00033 {
00034     uint64_t memory_size = 0;
00035
00036     if (!memmap_req.response)
00037         return 0;
00038
00039     for (size_t i = 0; i < memmap_req.response->entry_count; i++)
00040         memory_size += memmap_req.response->entries[i]->length;
```

```

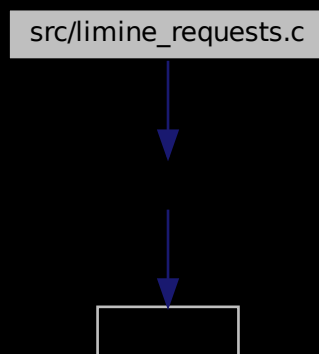
00041
00042     return bytes_to_mib(memory_size);
00043 }
00044
00045 uint64_t get_memory_size_gib(void)
00046 {
00047     uint64_t memory_size = 0;
00048
00049     if (!memmap_req.response)
00050         return 0;
00051
00052     for (size_t i = 0; i < memmap_req.response->entry_count; i++)
00053         memory_size += memmap_req.response->entries[i]->length;
00054
00055     return bytes_to_gib(memory_size);
00056 }
00057
00058 void print_memmap(void)
00059 {
00060     if (!memmap_req.response)
00061     {
00062         printf_("%s\n", "Memory map unavailable.");
00063         return;
00064     }
00065
00066     size_t size = memmap_req.response->entry_count;
00067     size_t num_usable = 0;
00068     size_t num_bad = 0;
00069     size_t num_reclaim_bl = 0;
00070     size_t num_reclaim_acpi = 0;
00071
00072     printf_("%s\n", "-----");
00073     printf_("%s\n", "|                MEMORY MAP                |");
00074     printf_("%s\n", "-----");
00075     printf_("%s\n", "Type Legend:  ");
00076     printf_("%s\n", "0 Usable");
00077     printf_("%s\n", "1 Reserved");
00078     printf_("%s\n", "2 ACPI Reclaimable");
00079     printf_("%s\n", "3 ACPI NVS");
00080     printf_("%s\n", "4 Bad Memory");
00081     printf_("%s\n", "5 Bootloader Reclaimable");
00082     printf_("%s\n", "6 Kernel And Modules");
00083     printf_("%s\n", "7 Framebuffer");
00084     printf_("%s\n", "-----");
00085     printf_("Number Of Entries:  %llu\n", (unsigned long long) size);
00086
00087     for (size_t i = 0; i < size; ++i)
00088     {
00089         struct limine_memmap_entry *entry = memmap_req.response->entries[i];
00090
00091         printf_("%s", "Info For Entry Number:  ");
00092         printf_("%llu\n", (unsigned long long) (i + 1));
00093         printf_("Entry Base:  0x%llx\n", (unsigned long long) entry->base);
00094         printf_("Entry Limit:  0x%llx\n", (unsigned long long) entry->length);
00095         printf_("Entry Type:  %llu\n", (unsigned long long) entry->type);
00096         printf_("%s\n", "-----");
00097
00098         if (entry->type == LIMINE_MEMMAP_USABLE)
00099             num_usable++;
00100         else if (entry->type == LIMINE_MEMMAP_BAD_MEMORY)
00101             num_bad++;
00102         else if (entry->type == LIMINE_MEMMAP_BOOTLOADER_RECLAIMABLE)
00103             num_reclaim_bl++;
00104         else if (entry->type == LIMINE_MEMMAP_ACPI_RECLAIMABLE)
00105             num_reclaim_acpi++;
00106     }
00107
00108     printf_("Number of Usable Entries:  %llu\n", (unsigned long long) num_usable);
00109     printf_("Number of Bad Entries:  %llu\n", (unsigned long long) num_bad);
00110     printf_("Number of Bootloader Reclaimable Entries:  %llu\n",
00111         (unsigned long long) num_reclaim_bl);
00112     printf_("Number of ACPI Reclaimable Entries:  %llu\n",
00113         (unsigned long long) num_reclaim_acpi);
00114     printf_("Memory Size:  %llu MiB.\n", (unsigned long long) get_memory_size());
00115     printf_("Memory Size:  %llu GiB.\n", (unsigned long long) get_memory_size_gib());
00116     printf_("%s\n", "-----");
00117 }

```

7.127 src/limine_requests.c File Reference

```
#include "include/limine.h"
```

Include dependency graph for limine_requests.c:



Functions

- `__attribute__((used, section(".requests"))) volatile`

7.127.1 Function Documentation

7.127.1.1 `__attribute__()`

```
__attribute__ (  
    ( used, section(".requests")) ) volatile
```

Copyright (c) 2025 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Definition at line 11 of file `limine_requests.c`.

7.128 limine_requests.c

Go to the documentation of this file.

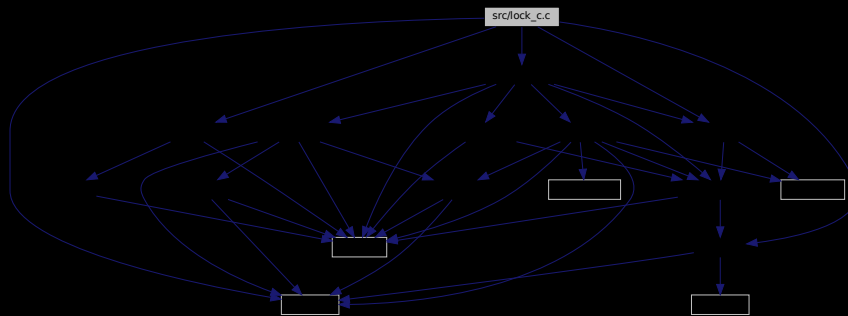
```

00001
00002 #include "include/limine.h"
00003
00004 /* Put every request in this file, defined exactly once. */
00005 __attribute__((
00006     used,
00007     section(".requests"))) volatile struct limine_memmap_request memmap_req = {
00008     .id = LIMINE_MEMMAP_REQUEST,
00009     .revision = 0,
00010 };
00011
00012 __attribute__((
00013     used,
00014     section(".requests"))) volatile struct limine_hhdm_request hhdm_request = {
00015     .id = LIMINE_HHDM_REQUEST,
00016     .revision = 0,
00017 };
00018
00019 __attribute__((
00020     used, section(".requests"))) volatile struct limine_kernel_address_request
00021     kaddress_req = {
00022     .id = LIMINE_KERNEL_ADDRESS_REQUEST,
00023     .revision = 0,
00024 };
00025
00026 __attribute__((
00027     used, section(".requests"))) volatile struct limine_module_request module_request = {
00028     .id = LIMINE_MODULE_REQUEST,
00029     .revision = 0,
00030 };
00031
00032 __attribute__((
00033     used, section(".requests"))) volatile struct limine_terminal_request early_term = {
00034     .id = LIMINE_TERMINAL_REQUEST, .revision = 0
00035 };
00036
00037 __attribute__((
00038     used, section(".requests"))) volatile struct limine_framebuffer_request fbr_req = {
00039     .id = LIMINE_FRAMEBUFFER_REQUEST, .revision = 0
00040 };
00041
00042 __attribute__((
00043     used, section(".requests"))) volatile struct limine_smp_request smp_request = {
00044     .id = LIMINE_SMP_REQUEST, .revision = 0
00045 };
00046
00047 __attribute__((
00048     used, section(".requests"))) volatile struct limine_stack_size_request stack_req = {
00049     .id = LIMINE_STACK_SIZE_REQUEST, .revision = 0, .stack_size = 4096
00050 };
00051
00052 __attribute__((used,
00053     section(".requests"))) volatile struct limine_boot_time_request
00054     boot_time_req = {
00055     .id = LIMINE_BOOT_TIME_REQUEST, .revision = 0
00056 };
00057
00058
00059
00060
00061
00062
00063
00064
00065
00066
00067
00068
00069
00070
00071
00072
00073
00074
00075
00076
00077
00078
00079
00080
00081

```

7.129 src/lock_c.c File Reference

```
#include "include/kernel.h"
#include "include/lock.h"
#include "include/printf.h"
#include "include/proc.h"
#include <stddef.h>
Include dependency graph for lock_c.c:
```



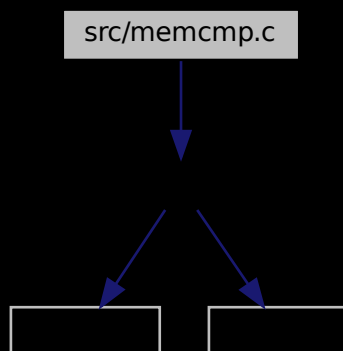
7.130 lock_c.c

[Go to the documentation of this file.](#)

```
00001 #include "include/kernel.h"
00002 #include "include/lock.h"
00003 #include "include/printf.h"
00004 #include "include/proc.h"
00005 #include <stddef.h>
```

7.131 src/memcmp.c File Reference

```
#include "include/string.h"
Include dependency graph for memcmp.c:
```



Functions

- `int memcmp (const void *aptr, const void *bptr, size_t size)`
Compare two memory blocks byte by byte.

7.131.1 Function Documentation

7.131.1.1 memcmp()

```
int memcmp (
    const void * aptr,
    const void * bptr,
    size_t size )
```

Compare two memory blocks byte by byte.

Parameters

<i>aptr</i>	Pointer to the first memory block.
<i>bptr</i>	Pointer to the second memory block.
<i>size</i>	Number of bytes to compare.

Returns

Negative when the first differing byte in `aptr` is smaller, zero when all requested bytes are equal, or positive when it is larger.

Definition at line 11 of file `memcmp.c`.

Here is the caller graph for this function:



7.132 memcmp.c

[Go to the documentation of this file.](#)

```
00001 #include "include/string.h"
00002
00011 int memcmp(const void *aptr, const void *bptr, size_t size)
00012 {
00013     const unsigned char *a = (const unsigned char *) aptr;
00014     const unsigned char *b = (const unsigned char *) bptr;
```

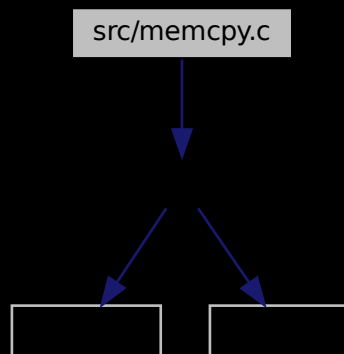


```
00015     for (size_t i = 0; i < size; i++)
00016     {
00017         if (a[i] < b[i])
00018             return -1;
00019         else if (b[i] < a[i])
00020             return 1;
00021     }
00022     return 0;
00023 }
```

7.133 src/memcpy.c File Reference

```
#include "include/string.h"
```

Include dependency graph for memcpy.c:



Functions

- void * **memcpy** (void *restrict dstptr, const void *restrict srcptr, size_t size)
Copy bytes from a non-overlapping source region to a destination.

7.133.1 Function Documentation

7.133.1.1 memcpy()

```
void * memcpy (
    void *restrict dstptr,
    const void *restrict srcptr,
    size_t size )
```

Copy bytes from a non-overlapping source region to a destination.

Parameters

<i>dstp</i>	Destination memory region.
<i>srcp</i>	Source memory region.
<i>size</i>	Number of bytes to copy.

Returns

dstp.

Definition at line 10 of file `memcpy.c`.

7.134 memcpy.c

[Go to the documentation of this file.](#)

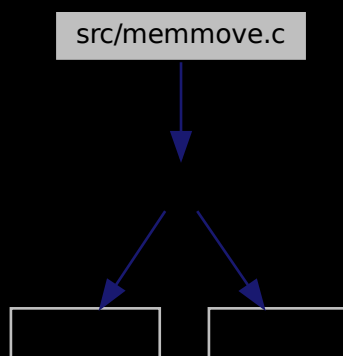
```

00001 #include "include/string.h"
00002
00010 void *memcpy(void *restrict dstptr, const void *restrict srcptr, size_t size)
00011 {
00012     unsigned char    *dst = (unsigned char *) dstptr;
00013     const unsigned char *src = (const unsigned char *) srcptr;
00014     for (size_t i = 0; i < size; i++)
00015         dst[i] = src[i];
00016     return dstptr;
00017 }
```

7.135 src/memmove.c File Reference

```
#include "include/string.h"
```

Include dependency graph for `memmove.c`:



Functions

- void * `memmove` (void **dstp*, const void **srcp*, size_t *size*)
Copy bytes between memory regions that may overlap.

7.135.1 Function Documentation

7.135.1.1 memmove()

```
void * memmove (
    void * dstptr,
    const void * srcptr,
    size_t size )
```

Copy bytes between memory regions that may overlap.

Parameters

<i>dstptr</i>	Destination memory region.
<i>srcptr</i>	Source memory region.
<i>size</i>	Number of bytes to copy.

Returns

dstptr.

Definition at line 10 of file [memmove.c](#).

7.136 memmove.c

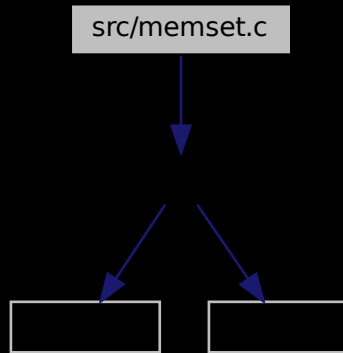
[Go to the documentation of this file.](#)

```
00001 #include "include/string.h"
00002
00010 void *memmove(void *dstptr, const void *srcptr, size_t size)
00011 {
00012     unsigned char      *dst = (unsigned char *) dstptr;
00013     const unsigned char *src = (const unsigned char *) srcptr;
00014     if (dst < src)
00015     {
00016         for (size_t i = 0; i < size; i++)
00017             dst[i] = src[i];
00018     }
00019     else
00020     {
00021         for (size_t i = size; i != 0; i--)
00022             dst[i - 1] = src[i - 1];
00023     }
00024     return dstptr;
00025 }
```

7.137 src/memset.c File Reference

```
#include "include/string.h"
```

Include dependency graph for memset.c:



Functions

- void * **memset** (void *bufptr, int value, size_t size)
Fill a memory region with one byte value.

7.137.1 Function Documentation

7.137.1.1 memset()

```
void * memset (
    void * bufptr,
    int value,
    size_t size )
```

Fill a memory region with one byte value.

Parameters

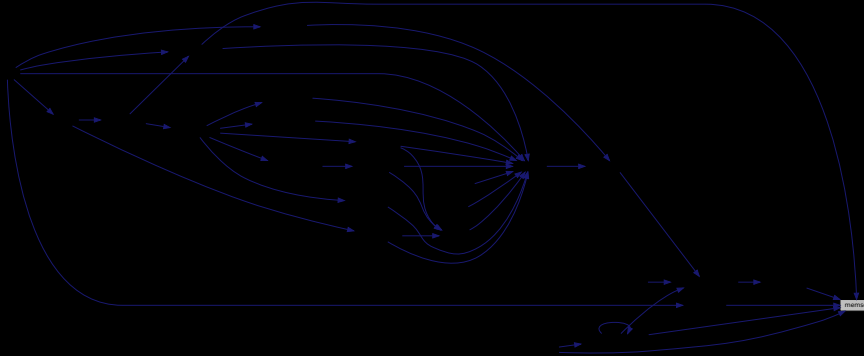
<i>bufptr</i>	Destination memory region.
<i>value</i>	Value whose low eight bits are written to each byte.
<i>size</i>	Number of bytes to set.

Returns

bufptr.

Definition at line 10 of file [memset.c](#).

Here is the caller graph for this function:



7.138 memset.c

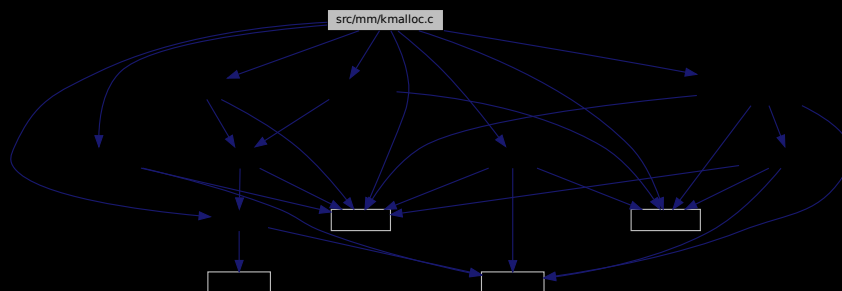
[Go to the documentation of this file.](#)

```
00001 #include "include/string.h"
00002
00010 void *memset(void *bufptr, int value, size_t size)
00011 {
00012     unsigned char *buf = (unsigned char *) bufptr;
00013     for (size_t i = 0; i < size; i++)
00014         buf[i] = (unsigned char) value;
00015     return bufptr;
00016 }
```

7.139 src/mm/kmalloc.c File Reference

```
#include "../include/mm/kmalloc.h"
#include "../include/KernelUtils.h"
#include "../include/lock.h"
#include "../include/paging/neo_framealloc.h"
#include "../include/paging/paging.h"
#include "../include/printf.h"
#include <stdbool.h>
#include <stdint.h>
```

Include dependency graph for `kmalloc.c`:



Data Structures

- struct `block_header`
- struct `block_footer`

Macros

- `#define PAGING_FLAG_PRESENT (1ull << 0)`
- `#define PAGING_FLAG_WRITABLE (1ull << 1)`
- `#define PAGING_FLAG_NO_EXECUTE (1ull << 63)`
- `#define PAGE_SIZE 4096ull`
- `#define HEAP_BASE 0xFFFF900000000000ull`
- `#define HEAP_LIMIT (HEAP_BASE + (256ull * 1024 * 1024)) /* 256 MiB window */`
- `#define ALIGN_UP(x, a) (((x) + ((a) - 1)) & ~((a) - 1))`
- `#define ALIGN_DOWN(x, a) ((x) & ~((a) - 1))`
- `#define ALIGNMENT 16ull`
- `#define MIN_SPLIT_SIZE 32ull /* minimal payload left after split (tune) */`
- `#define ALLOC_FLAG ((size_t) 1)`
- `#define IS_ALLOC(h) ((h)->size_and_flags & ALLOC_FLAG)`
- `#define SET_ALLOC(h) ((h)->size_and_flags |= ALLOC_FLAG)`
- `#define CLR_ALLOC(h) ((h)->size_and_flags &= ~ALLOC_FLAG)`
- `#define BLK_SIZE(h) ((h)->size_and_flags & ~ALLOC_FLAG)`
- `#define SET_SIZE(h, sz)`
- `#define HDR_SIZE ALIGN_UP((size_t) sizeof(block_t), ALIGNMENT)`
- `#define FTR_SIZE ((size_t) sizeof(footer_t))`
- `#define OVERHEAD (HDR_SIZE + FTR_SIZE)`
- `#define USABLE(h) (BLK_SIZE(h) - OVERHEAD)`

Typedefs

- typedef struct `block_header` `block_t`
- typedef struct `block_footer` `footer_t`

Functions

- void `panic` (const char *fmt,...)
Panic with a nicely formatted message. This is useful for situations where you are certain an unrecoverable error will occur and want to be informative (feel free to abuse sarcasm in them) about the problem.
- uint64_t `ReadCR3` (void)
Read the active CR3 value.
- static size_t `total_size_for_payload` (size_t payload)
- static footer_t * `FOOTER_OF` (block_t *h)
- static block_t * `NEXT_BLOCK` (block_t *h)
- static block_t * `PREV_BLOCK` (block_t *h)
- static struct PageTable * `current_pml4_phys` (void)
- static void `fl_push` (block_t *b)
- static void `fl_remove` (block_t *b)
- static void `heap_map_more` (size_t bytes)
- static block_t * `fl_find_fit` (size_t need_payload)
- static block_t * `allocate_from_block` (block_t *h, size_t need_payload)
- static block_t * `coalesce` (block_t *h)

- void `kmalloc_init` (void)
Initialize the kernel heap.
- void * `kmalloc` (size_t size)
Allocate at least `size` bytes of kernel virtual memory.
- void `kfree` (void *ptr)
Return a heap allocation to the kernel allocator.
- void * `krealloc` (void *ptr, size_t newsize)
Resize an existing heap allocation while preserving payload data.
- size_t `kmalloc_usable_size` (void *ptr)
Query the allocator's usable payload size for an allocation.
- void `kmalloc_dump` (void)
Emit allocator state for debugging.

Variables

- static uint64_t `g_heap_curr` = `HEAP_BASE`
- static spinlock_t `g_heap_lock` = `SPINLOCK_INIT`
- static block_t * `g_free_list`

7.139.1 Macro Definition Documentation

7.139.1.1 ALIGN_DOWN

```
#define ALIGN_DOWN(  
    x,  
    a ) ((x) & ~((a) -1))
```

Definition at line 35 of file `kmalloc.c`.

7.139.1.2 ALIGN_UP

```
#define ALIGN_UP(  
    x,  
    a ) (((x) + ((a) -1)) & ~((a) -1))
```

Definition at line 34 of file `kmalloc.c`.

7.139.1.3 ALIGNMENT

```
#define ALIGNMENT 16ull
```

Definition at line 36 of file `kmalloc.c`.

7.139.1.4 ALLOC_FLAG

```
#define ALLOC_FLAG ((size_t) 1)
```

Definition at line 53 of file [kmalloc.c](#).

7.139.1.5 BLK_SIZE

```
#define BLK_SIZE(  
    h ) ((h)->size_and_flags & ~ALLOC_FLAG)
```

Definition at line 57 of file [kmalloc.c](#).

7.139.1.6 CLR_ALLOC

```
#define CLR_ALLOC(  
    h ) ((h)->size_and_flags &= ~ALLOC_FLAG)
```

Definition at line 56 of file [kmalloc.c](#).

7.139.1.7 FTR_SIZE

```
#define FTR_SIZE ((size_t) sizeof(footer_t))
```

Definition at line 67 of file [kmalloc.c](#).

7.139.1.8 HDR_SIZE

```
#define HDR_SIZE ALIGN_UP((size_t) sizeof(block_t), ALIGNMENT)
```

Definition at line 66 of file [kmalloc.c](#).

7.139.1.9 HEAP_BASE

```
#define HEAP_BASE 0xFFFF900000000000ull
```

Definition at line 31 of file [kmalloc.c](#).

7.139.1.10 HEAP_LIMIT

```
#define HEAP_LIMIT (HEAP_BASE + (256ull * 1024 * 1024)) /* 256 MiB window */
```

Definition at line 32 of file [kmalloc.c](#).

7.139.1.11 IS_ALLOC

```
#define IS_ALLOC(  
    h ) ((h)->size_and_flags & ALLOC_FLAG)
```

Definition at line 54 of file [kmalloc.c](#).

7.139.1.12 MIN_SPLIT_SIZE

```
#define MIN_SPLIT_SIZE 32ull /* minimal payload left after split (tune) */
```

Definition at line 37 of file [kmalloc.c](#).

7.139.1.13 OVERHEAD

```
#define OVERHEAD (HDR_SIZE + FTR_SIZE)
```

Definition at line 68 of file [kmalloc.c](#).

7.139.1.14 PAGE_SIZE

```
#define PAGE_SIZE 4096ull
```

Definition at line 30 of file [kmalloc.c](#).

7.139.1.15 PAGING_FLAG_NO_EXECUTE

```
#define PAGING_FLAG_NO_EXECUTE (1ull << 63)
```

Definition at line 27 of file [kmalloc.c](#).

7.139.1.16 PAGING_FLAG_PRESENT

```
#define PAGING_FLAG_PRESENT (1ull << 0)
```

Definition at line 21 of file [kmalloc.c](#).

7.139.1.17 PAGING_FLAG_WRITABLE

```
#define PAGING_FLAG_WRITABLE (1ull << 1)
```

Definition at line 24 of file [kmalloc.c](#).

7.139.1.18 SET_ALLOC

```
#define SET_ALLOC(  
    h ) ((h)->size_and_flags |= ALLOC_FLAG)
```

Definition at line 55 of file [kmalloc.c](#).

7.139.1.19 SET_SIZE

```
#define SET_SIZE(  
    h,  
    sz )
```

Value:

```
do  
{  
    (h)->size_and_flags = ((sz) | ((h)->size_and_flags & ALLOC_FLAG));  
} while (0)
```

Definition at line 58 of file [kmalloc.c](#).

7.139.1.20 USABLE

```
#define USABLE(  
    h ) (BLK_SIZE(h) - OVERHEAD)
```

Definition at line 69 of file [kmalloc.c](#).

7.139.2 Typedef Documentation

7.139.2.1 block_t

```
typedef struct block_header block_t
```

7.139.2.2 footer_t

```
typedef struct block_footer footer_t
```

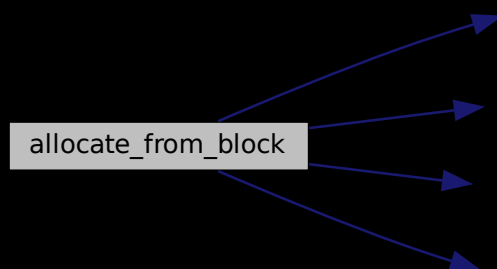
7.139.3 Function Documentation

7.139.3.1 allocate_from_block()

```
static block_t * allocate_from_block (  
    block_t * h,  
    size_t need_payload ) [static]
```

Definition at line 172 of file `kmalloc.c`.

Here is the call graph for this function:



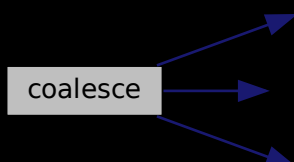
Here is the caller graph for this function:

7.139.3.2 coalesce()

```
static block_t * coalesce (
    block_t * h ) [static]
```

Definition at line 209 of file kmalloc.c.

Here is the call graph for this function:



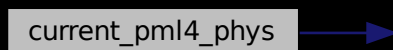
Here is the caller graph for this function:

7.139.3.3 current_pml4_phys()

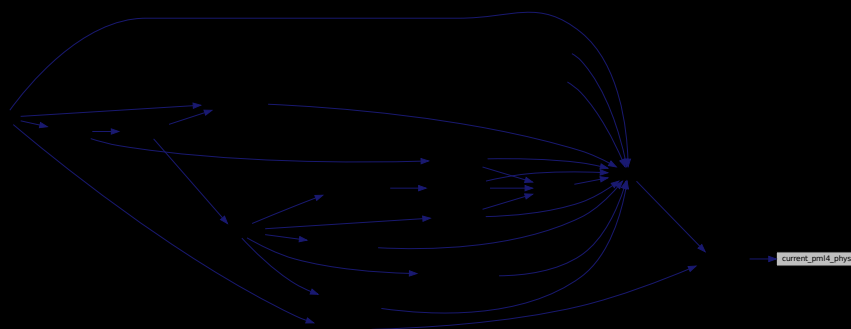
```
static struct PageTable * current_pml4_phys (  
    void ) [inline], [static]
```

Definition at line 100 of file [kmalloc.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.139.3.4 fl_find_fit()

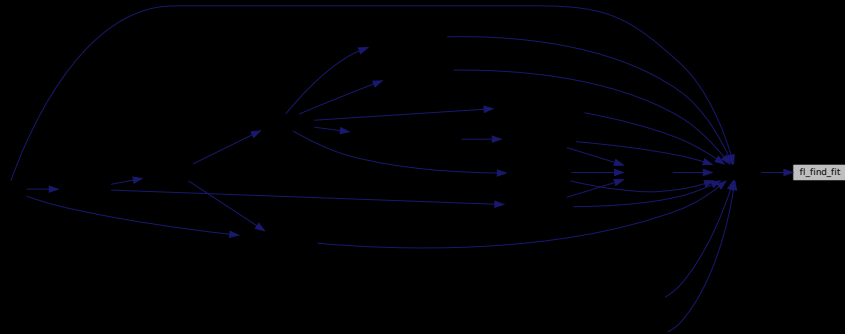
```
static block\_t * fl_find_fit (  
    size_t need_payload ) [static]
```

Definition at line 160 of file [kmalloc.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

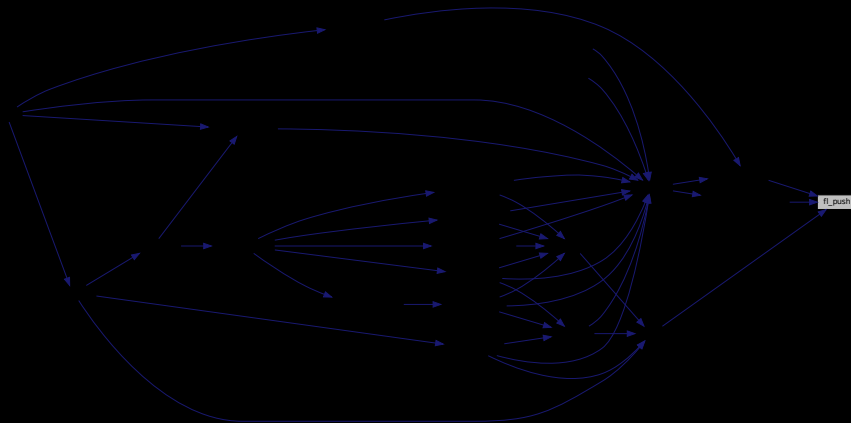


7.139.3.5 `fl_push()`

```
static void fl_push (
    block_t * b ) [static]
```

Definition at line 107 of file `kmalloc.c`.

Here is the caller graph for this function:

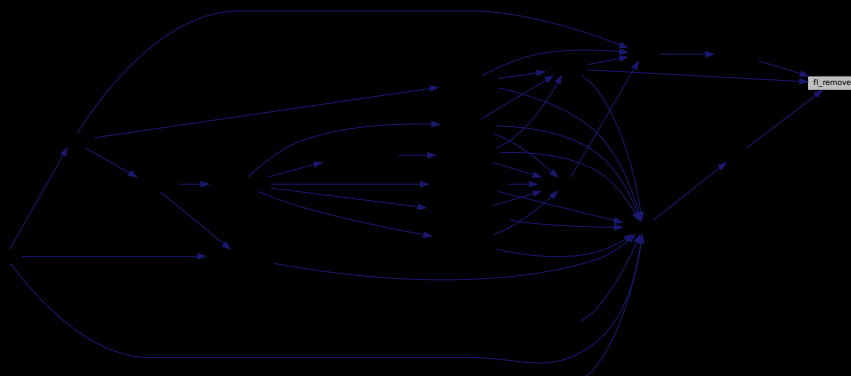


7.139.3.6 `fl_remove()`

```
static void fl_remove (
    block_t * b ) [static]
```

Definition at line 115 of file `kmalloc.c`.

Here is the caller graph for this function:

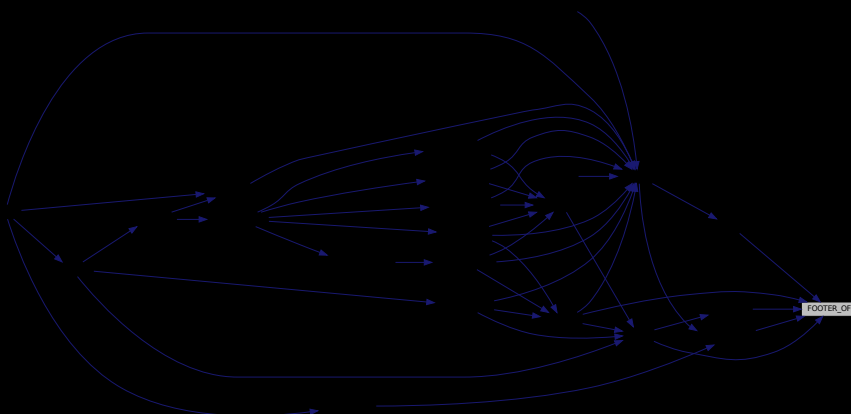


7.139.3.7 FOOTER_OF()

```
static footer_t * FOOTER_OF (  
    block_t * h ) [inline], [static]
```

Definition at line 77 of file `kmalloc.c`.

Here is the caller graph for this function:

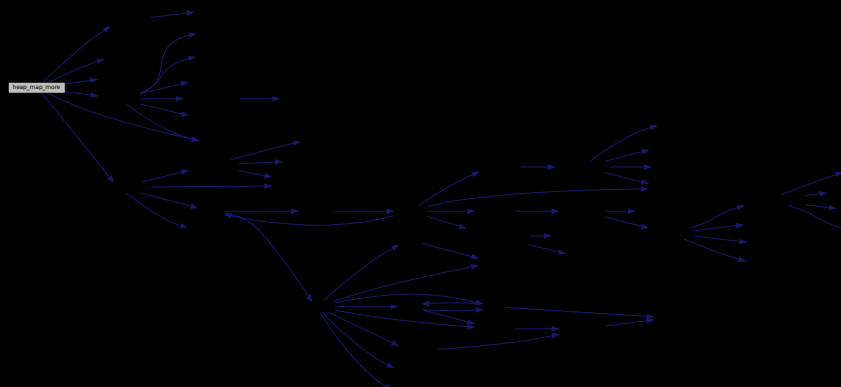


7.139.3.8 heap_map_more()

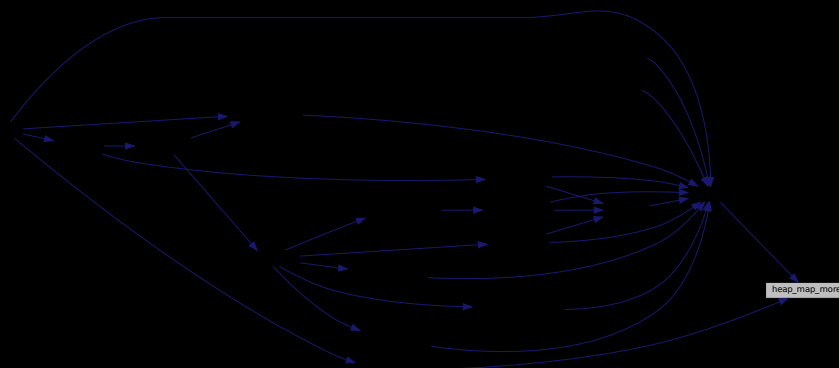
```
static void heap_map_more (
    size_t bytes ) [static]
```

Definition at line 127 of file [kmallocc.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.139.3.9 kfree()

```
void kfree (
    void * ptr )
```

Return a heap allocation to the kernel allocator.

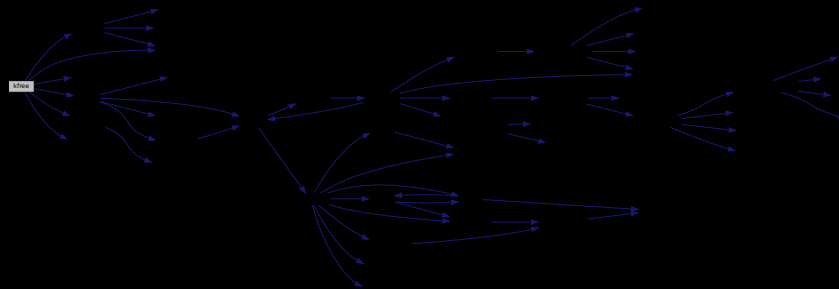
Parameters

<i>ptr</i>	Pointer previously returned by <code>kmalloc()</code> / <code>krealloc()</code> , or <code>NULL</code> .
------------	--

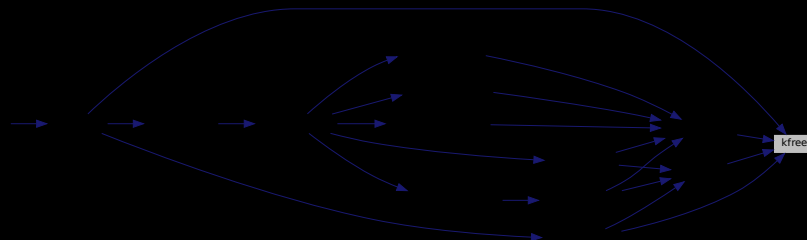
Freed blocks become reusable by later heap allocations. In the current 0.0.5 design, freeing a block does not shrink the mapped heap high-water mark or immediately return the heap's backing pages to the PMM.

Definition at line 281 of file `kmalloc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.139.3.10 kmalloc()

```
void * kmalloc (
    size_t size )
```

Allocate at least `size` bytes of kernel virtual memory.

Parameters

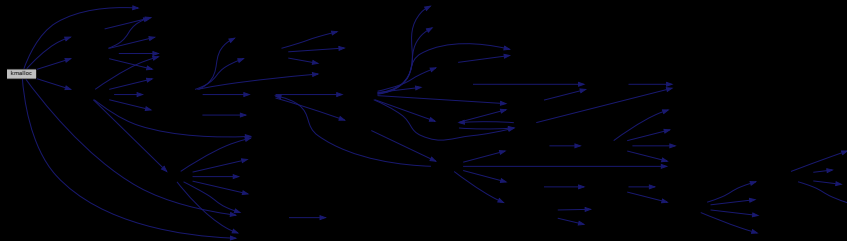
<i>size</i>	Requested payload size in bytes.
-------------	----------------------------------

Returns

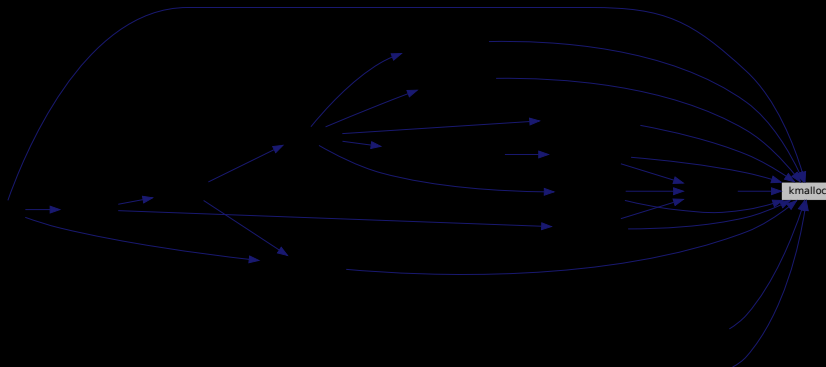
Pointer to allocated kernel memory, or NULL if the request cannot be satisfied according to heap semantics.

Definition at line 252 of file [kmalloc.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.139.3.11 kmalloc_dump()

```
void kmalloc_dump (
    void )
```

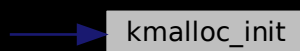
Emit allocator state for debugging.

This is a diagnostic helper and is not part of the allocation fast path.

Definition at line 364 of file [kmalloc.c](#).

Must be called after `paging_bootstrap()` has installed Voyager's page tables and after the permanent physical frame allocator is available. The heap may grow by requesting physical frames and mapping additional virtual pages.

Here is the call graph for this function:



7.139.3.13 `kmalloc_usable_size()`

```
size_t kmalloc_usable_size (
    void * ptr )
```

Query the allocator's usable payload size for an allocation.

Parameters

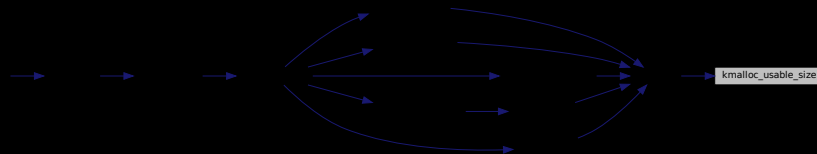
<i>ptr</i>	Active heap allocation.
------------	-------------------------

Returns

Number of usable payload bytes associated with `ptr`.

Definition at line 355 of file `kmalloc.c`.

Here is the caller graph for this function:



7.139.3.14 `krealloc()`

```
void * krealloc (
    void * ptr,
    size_t newsize )
```

Resize an existing heap allocation while preserving payload data.

Parameters

<i>ptr</i>	Existing allocation, or NULL to behave like <code>kmalloc()</code> .
<i>newsize</i>	New requested payload size in bytes.

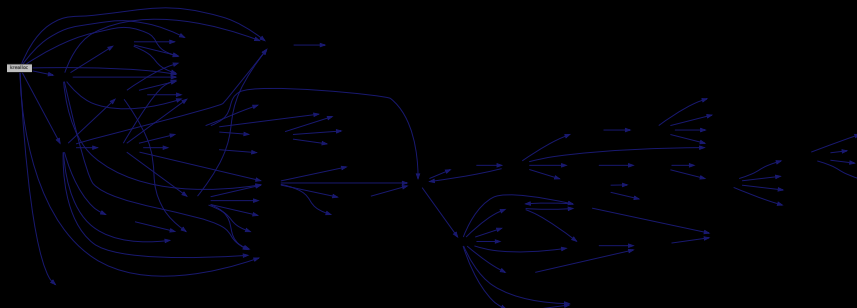
Returns

Resized allocation, which may move, or NULL according to allocator semantics.

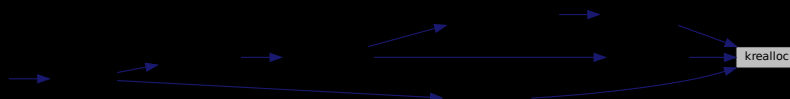
krealloc(ptr, 0) releases the allocation. Data up to the minimum of the old and new usable sizes is preserved when the allocation moves.

Definition at line 303 of file [kmalloc.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

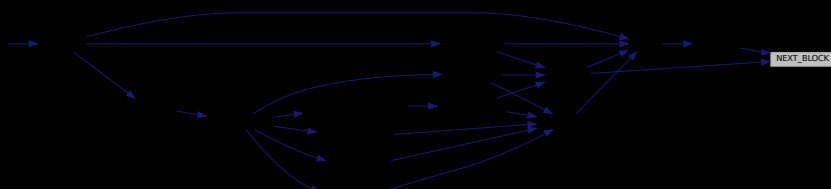


7.139.3.15 NEXT_BLOCK()

```
static block_t * NEXT_BLOCK (
    block_t * h ) [inline], [static]
```

Definition at line 81 of file [kmalloc.c](#).

Here is the caller graph for this function:



7.139.3.16 panic()

```
void panic (  
    const char * fmt,  
    ... )
```

Panic with a nicely formatted message. This is useful for situations where you are certain an unrecoverable error will occur and want to be informative (feel free to abuse sarcasm in them) about the problem.

Copyright (c) 2025 Tristan Adams

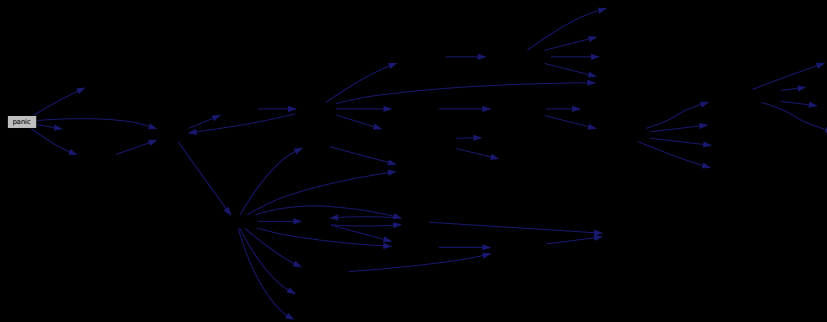
This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Parameters

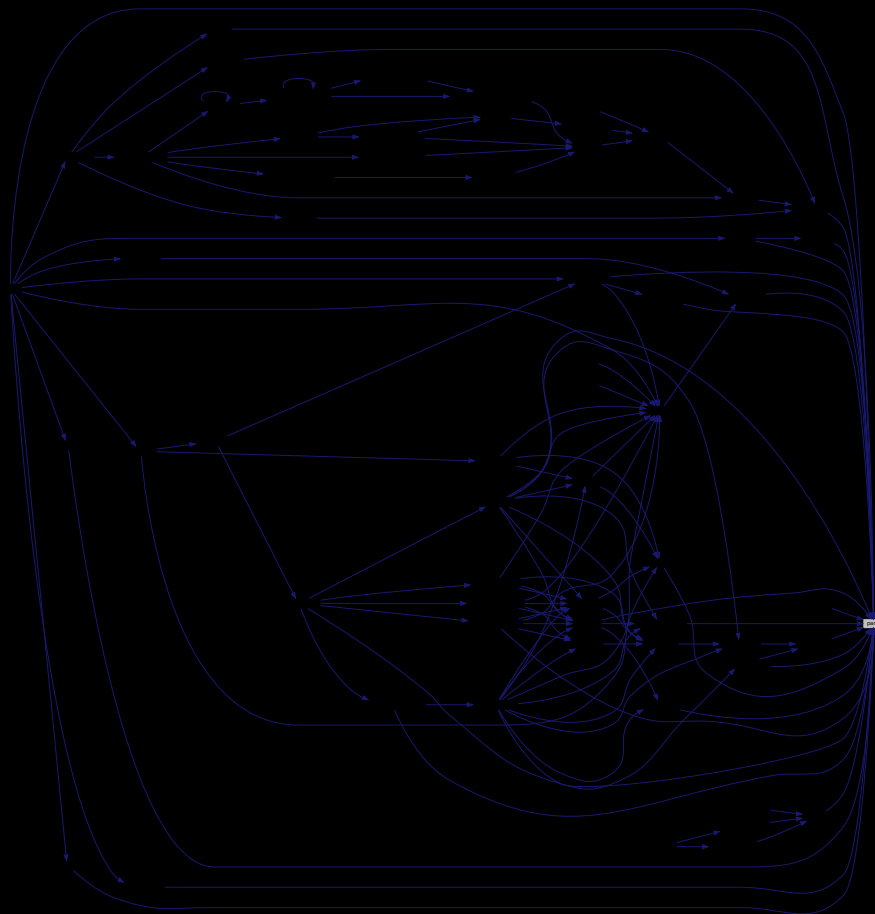
<i>fmt</i>	printf style format string for the panic message
------------	--

Definition at line 17 of file [panic.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.139.3.17 PREV_BLOCK()

```
static block_t * PREV_BLOCK (
    block_t * h ) [inline], [static]
```

Definition at line 85 of file **kmalloc.c**.

7.139.3.18 ReadCR3()

```
uint64_t ReadCR3 (
    void )
```

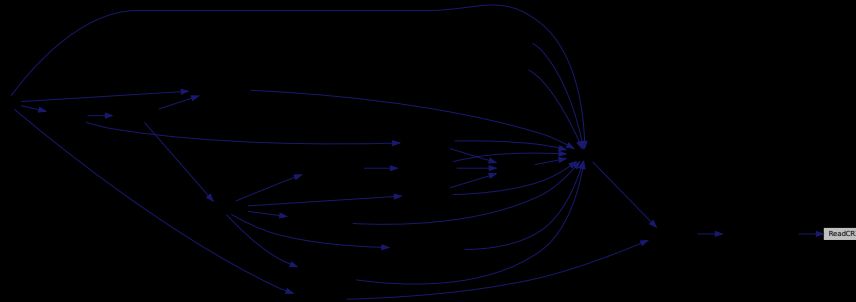
Read the active CR3 value.

Returns

Current CR3 value.

Definition at line 48 of file [paging.c](#).

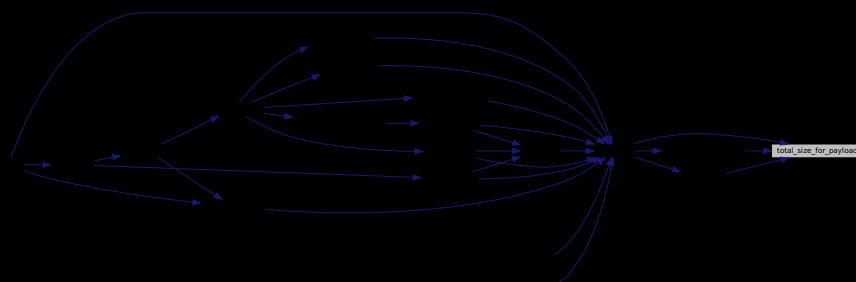
Here is the caller graph for this function:

**7.139.3.19 total_size_for_payload()**

```
static size_t total_size_for_payload (
    size_t payload ) [inline], [static]
```

Definition at line 71 of file [kmallocc.c](#).

Here is the caller graph for this function:

**7.139.4 Variable Documentation**

7.139.4.1 g_free_list

```
block_t* g_free_list [static]
```

Initial value:

```
=
    NULL
```

Definition at line 95 of file [kmalloc.c](#).

7.139.4.2 g_heap_curr

```
uint64_t g_heap_curr = HEAP_BASE [static]
```

Definition at line 93 of file [kmalloc.c](#).

7.139.4.3 g_heap_lock

```
spinlock_t g_heap_lock = SPINLOCK_INIT [static]
```

Definition at line 94 of file [kmalloc.c](#).

7.140 kmalloc.c

[Go to the documentation of this file.](#)

```
00001
00008 #include "../include/mm/kmalloc.h"
00009 #include "../include/KernelUtils.h" // for k_mode.addr_debug? optional
00010 #include "../include/lock.h"
00011 #include "../include/paging/neo_framealloc.h" // frame_alloc
00012 #include "../include/paging/paging.h" // PagingMapMemory, PagingUnmapMemory, PagingPhysicalMemory
00013 #include "../include/printf.h"
00014 #include <stdbool.h>
00015 #include <stdint.h>
00016
00017 extern void panic(const char *fmt, ...);
00018 extern uint64_t ReadCR3(void);
00019
00020 #ifndef PAGING_FLAG_PRESENT
00021 #define PAGING_FLAG_PRESENT (1ull < 0)
00022 #endif
00023 #ifndef PAGING_FLAG_WRITABLE
00024 #define PAGING_FLAG_WRITABLE (1ull < 1)
00025 #endif
00026 #ifndef PAGING_FLAG_NO_EXECUTE
00027 #define PAGING_FLAG_NO_EXECUTE (1ull < 63)
00028 #endif
00029
00030 #define PAGE_SIZE 4096ull
00031 #define HEAP_BASE 0xFFFF900000000000ull
00032 #define HEAP_LIMIT (HEAP_BASE + (256ull * 1024 * 1024)) /* 256 MiB window */
00033
00034 #define ALIGN_UP(x, a) (((x) + ((a) - 1)) & ~((a) - 1))
00035 #define ALIGN_DOWN(x, a) ((x) & ~((a) - 1))
00036 #define ALIGNMENT 16ull
00037 #define MIN_SPLIT_SIZE 32ull /* minimal payload left after split (tune) */
00038
00039 typedef struct block_header
00040 {
00041     size_t size_and_flags; // total block size (header+payload+footer); bit0 =
```

```

00042             // allocated flag
00043     struct block_header *prev_free;
00044     struct block_header *next_free;
00045 } block_t;
00046
00047 /* Footer mirrors size for O(1) backward coalescing */
00048 typedef struct block_footer
00049 {
00050     size_t size_and_flags;
00051 } footer_t;
00052
00053 #define ALLOC_FLAG ((size_t) 1)
00054 #define IS_ALLOC(h) ((h)->size_and_flags & ALLOC_FLAG)
00055 #define SET_ALLOC(h) ((h)->size_and_flags |= ALLOC_FLAG)
00056 #define CLR_ALLOC(h) ((h)->size_and_flags &= ~ALLOC_FLAG)
00057 #define BLK_SIZE(h) ((h)->size_and_flags & ~ALLOC_FLAG)
00058 #define SET_SIZE(h, sz)
00059 do
00060 {
00061     (h)->size_and_flags = ((sz) | ((h)->size_and_flags & ALLOC_FLAG)); \
00062 } while (0)
00063
00064 /* Pad the header so payloads are 16-byte aligned. Keep every block size a
00065 * multiple of 16 so split blocks preserve that alignment. */
00066 #define HDR_SIZE ALIGN_UP((size_t) sizeof(block_t), ALIGNMENT)
00067 #define FTR_SIZE ((size_t) sizeof(footer_t))
00068 #define OVERHEAD (HDR_SIZE + FTR_SIZE)
00069 #define USABLE(h) (BLK_SIZE(h) - OVERHEAD)
00070
00071 static inline size_t total_size_for_payload(size_t payload)
00072 {
00073     size_t aligned_payload = ALIGN_UP(payload, ALIGNMENT);
00074     return ALIGN_UP(aligned_payload + OVERHEAD, ALIGNMENT);
00075 }
00076
00077 static inline footer_t *FOOTER_OF(block_t *h)
00078 {
00079     return (footer_t *) ((uint8_t *) h + BLK_SIZE(h) - FTR_SIZE);
00080 }
00081 static inline block_t *NEXT_BLOCK(block_t *h)
00082 {
00083     return (block_t *) ((uint8_t *) h + BLK_SIZE(h));
00084 }
00085 static inline block_t *PREV_BLOCK(block_t *h)
00086 {
00087     footer_t *prev_ftr = (footer_t *) ((uint8_t *) h - FTR_SIZE);
00088     size_t prev_sz = prev_ftr->size_and_flags & ~ALLOC_FLAG;
00089     return (block_t *) ((uint8_t *) h - prev_sz);
00090 }
00091
00092 /* Heap arena state */
00093 static uint64_t g_heap_curr = HEAP_BASE; // next unmapped VA
00094 static spinlock_t g_heap_lock = SPINLOCK_INIT;
00095 static block_t *g_free_list =
00096     NULL; // circular or linear; we'll keep it linear head
00097
00098 /* The legacy paging API expects a PHYSICAL PML4 address in its PageTable *
00099 * parameter and performs the HMDM translation internally. */
00100 static inline struct PageTable *current_pml4_phys(void)
00101 {
00102     uint64_t cr3 = ReadCR3() & ~0xffffull;
00103     return (struct PageTable *) (uintptr_t) cr3;
00104 }
00105
00106 /* Free list ops */
00107 static void fl_push(block_t *b)
00108 {
00109     b->prev_free = NULL;
00110     b->next_free = g_free_list;
00111     if (g_free_list)
00112         g_free_list->prev_free = b;
00113     g_free_list = b;
00114 }
00115 static void fl_remove(block_t *b)
00116 {
00117     if (b->prev_free)
00118         b->prev_free->next_free = b->next_free;
00119     else
00120         g_free_list = b->next_free;
00121     if (b->next_free)
00122         b->next_free->prev_free = b->prev_free;
00123     b->prev_free = b->next_free = NULL;
00124 }
00125
00126 /* Map more pages and create a single big free block appended at g_heap_curr. */
00127 static void heap_map_more(size_t bytes)
00128 {

```

```

00129     size_t need = ALIGN_UP(bytes, PAGE_SIZE);
00130     uint64_t start = ALIGN_UP(g_heap_curr, PAGE_SIZE);
00131     uint64_t end = start + need;
00132     if (end > HEAP_LIMIT)
00133         panic("kmalloc: VA heap exhausted");
00134
00135     struct PageTable *p4 = current_pml4_phys();
00136     for (uint64_t va = start; va < end; va += PAGE_SIZE)
00137     {
00138         uint64_t phys = frame_alloc();
00139         PagingMapMemory(p4,
00140             (void *) va,
00141             (void *) phys,
00142             PAGING_FLAG_PRESENT | PAGING_FLAG_WRITABLE |
00143             PAGING_FLAG_NO_EXECUTE);
00144     }
00145
00146     /* Make a new big free block over the newly mapped region */
00147     block_t *h = (block_t *) (uintptr_t) start;
00148     size_t blk_sz = (size_t) (end - start);
00149     h->size_and_flags = blk_sz; // alloc bit off
00150     h->prev_free = h->next_free = NULL;
00151
00152     footer_t *f = FOOTER_OF(h);
00153     f->size_and_flags = blk_sz;
00154
00155     fl_push(h);
00156     g_heap_curr = end;
00157 }
00158
00159 /* First-fit search */
00160 static block_t *fl_find_fit(size_t need_payload)
00161 {
00162     size_t need_total = total_size_for_payload(need_payload);
00163     for (block_t *it = g_free_list; it; it = it->next_free)
00164     {
00165         if (BLK_SIZE(it) >= need_total)
00166             return it;
00167     }
00168     return NULL;
00169 }
00170
00171 /* Split free block if large enough; return allocated header */
00172 static block_t *allocate_from_block(block_t *h, size_t need_payload)
00173 {
00174     size_t need_total = total_size_for_payload(need_payload);
00175     size_t old_sz = BLK_SIZE(h);
00176
00177     fl_remove(h);
00178
00179     if (old_sz >= need_total + total_size_for_payload(MIN_SPLIT_SIZE))
00180     {
00181         /* Split: front part becomes allocated, tail becomes a free block */
00182         size_t alloc_sz = need_total;
00183         size_t tail_sz = old_sz - alloc_sz;
00184
00185         /* alloc part */
00186         SET_SIZE(h, alloc_sz);
00187         SET_ALLOC(h);
00188         FOOTER_OF(h)->size_and_flags = h->size_and_flags;
00189
00190         /* tail free block */
00191         block_t *t = (block_t *) ((uint8_t *) h + alloc_sz);
00192         t->size_and_flags = tail_sz; // free
00193         t->prev_free = t->next_free = NULL;
00194         FOOTER_OF(t)->size_and_flags = tail_sz;
00195
00196         fl_push(t);
00197         return h;
00198     }
00199     else
00200     {
00201         /* Take the whole block */
00202         SET_ALLOC(h);
00203         FOOTER_OF(h)->size_and_flags = h->size_and_flags;
00204         return h;
00205     }
00206 }
00207
00208 /* Coalesce with neighbors if they are free */
00209 static block_t *coalesce(block_t *h)
00210 {
00211     /* Try next. g_heap_curr is the first currently-unmapped heap address. */
00212     block_t *n = NEXT_BLOCK(h);
00213     if ((uintptr_t) n < (uintptr_t) g_heap_curr)
00214     {
00215         if (!IS_ALLOC(n))

```

```

00216     {
00217         /* merge with next */
00218         fl_remove(n);
00219         size_t new_sz = BLK_SIZE(h) + BLK_SIZE(n);
00220         SET_SIZE(h, new_sz);
00221         FOOTER_OF(h)->size_and_flags = h->size_and_flags;
00222     }
00223 }
00224 /* Try prev: guard against base */
00225 if ((uintptr_t) h > (uintptr_t) HEAP_BASE)
00226 {
00227     footer_t *pf = (footer_t *) ((uint8_t *) h - FTR_SIZE);
00228     size_t psz = pf->size_and_flags & ~ALLOC_FLAG;
00229     block_t *p = (block_t *) ((uint8_t *) h - psz);
00230     if (!IS_ALLOC(p))
00231     {
00232         fl_remove(p);
00233         size_t new_sz = BLK_SIZE(p) + BLK_SIZE(h);
00234         SET_SIZE(p, new_sz);
00235         FOOTER_OF(p)->size_and_flags = p->size_and_flags;
00236         h = p;
00237     }
00238 }
00239 return h;
00240 }
00241
00242 /* ===== Public API ===== */
00243
00244 void kmalloc_init(void)
00245 {
00246     g_heap_curr = HEAP_BASE;
00247     g_free_list = NULL;
00248     /* Map an initial chunk to avoid early fragmentation; tune as you like. */
00249     heap_map_more(64 * PAGE_SIZE); // 256 KiB to start
00250 }
00251
00252 void *kmalloc(size_t size)
00253 {
00254     if (size == 0)
00255         return NULL;
00256
00257     spinlock_acquire(&g_heap_lock);
00258
00259     block_t *h = fl_find_fit(size);
00260     if (!h)
00261     {
00262         /* Map more and retry once */
00263         size_t grow = total_size_for_payload(size);
00264         if (grow < (128 * PAGE_SIZE))
00265             grow = 128 * PAGE_SIZE; // amortize mapping
00266         heap_map_more(grow);
00267         h = fl_find_fit(size);
00268         if (!h)
00269         {
00270             spinlock_release(&g_heap_lock);
00271             return NULL;
00272         } // still no memory
00273     }
00274
00275     block_t *a = allocate_from_block(h, size);
00276     void *payload = (uint8_t *) a + HDR_SIZE;
00277     spinlock_release(&g_heap_lock);
00278     return payload;
00279 }
00280
00281 void kfree(void *ptr)
00282 {
00283     if (!ptr)
00284         return;
00285     spinlock_acquire(&g_heap_lock);
00286
00287     block_t *h = (block_t *) ((uint8_t *) ptr - HDR_SIZE);
00288     if (!IS_ALLOC(h))
00289     {
00290         spinlock_release(&g_heap_lock);
00291         panic("kfree: double free %p", ptr);
00292     }
00293     CLR_ALLOC(h);
00294     FOOTER_OF(h)->size_and_flags = h->size_and_flags;
00295
00296     /* Coalesce with neighbors and reinsert */
00297     block_t *c = coalesce(h);
00298     fl_push(c);
00299
00300     spinlock_release(&g_heap_lock);
00301 }
00302

```

```

00303 void *krealloc(void *ptr, size_t newsize)
00304 {
00305     if (!ptr)
00306         return kmalloc(newsize);
00307     if (newsize == 0)
00308     {
00309         kfree(ptr);
00310         return NULL;
00311     }
00312     spinlock_acquire(&g_heap_lock);
00313
00314     block_t *h = (block_t *) ((uint8_t *) ptr - HDR_SIZE);
00315     size_t cur = USABLE(h);
00316     if (cur >= newsize)
00317     {
00318         /* Optional: shrink-in-place if large extra -> split tail */
00319         spinlock_release(&g_heap_lock);
00320         return ptr;
00321     }
00322
00323     /* Try to grow into next block if it is mapped, free and large enough. */
00324     block_t *n = NEXT_BLOCK(h);
00325     if ((uintptr_t) n < (uintptr_t) g_heap_curr && !IS_ALLOC(n))
00326     {
00327         size_t combined = cur + OVERHEAD + USABLE(n);
00328         if (combined >= newsize)
00329         {
00330             /* Merge in place */
00331             fl_remove(n);
00332             size_t new_sz = BLK_SIZE(h) + BLK_SIZE(n);
00333             SET_SIZE(h, new_sz);
00334             SET_ALLOC(h);
00335             FOOTER_OF(h)->size_and_flags = h->size_and_flags;
00336             // Optional: split tail if much larger than needed
00337             spinlock_release(&g_heap_lock);
00338             return ptr;
00339         }
00340     }
00341
00342     /* Fallback: allocate new, copy, free old */
00343     spinlock_release(&g_heap_lock);
00344     void *np = kmalloc(newsize);
00345     if (!np)
00346         return NULL;
00347     // memcpy (safe length = cur)
00348     extern void *memcpy(void *, const void *, size_t);
00349     memcpy(np, ptr, cur);
00350     kfree(ptr);
00351     return np;
00352 }
00353
00354 size_t kmalloc_usable_size(void *ptr)
00355 {
00356     if (!ptr)
00357         return 0;
00358     block_t *h = (block_t *) ((uint8_t *) ptr - HDR_SIZE);
00359     return USABLE(h);
00360 }
00361
00362 /* Optional: crude dump */
00363 void kmalloc_dump(void)
00364 {
00365     #if 0
00366     printf("[kmalloc] free list:\n");
00367     for (block_t* it = g_free_list; it; it = it->next_free) {
00368         printf("  free blk %p size=%llu\n", it, (unsigned long long)BLK_SIZE(it));
00369     }
00370     #endif
00371 }
00372

```

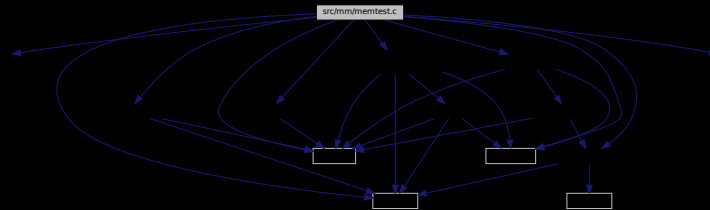
7.141 src/mm/memtest.c File Reference

```

#include "../include/mm/memtest.h"
#include "../include/limine.h"
#include "../include/mm/kmalloc.h"
#include "../include/paging/neo_framealloc.h"
#include "../include/panic.h"
#include "../include/printf.h"

```

```
#include "../include/sched.h"
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
Include dependency graph for memtest.c:
```



Data Structures

- struct `memtest_slot`
- struct `memtest_stats`

Macros

- `#define MEMTEST_SLOT_COUNT 128u`
- `#define MEMTEST_PMM_MAX_FRAMES 256u`
- `#define MEMTEST_PAGE_SIZE 4096u`
- `#define MEMTEST_RANDOM_OPS 10000u`
- `#define MEMTEST_RANDOM_MAX_ALLOC 32768u`
- `#define MEMTEST_GROWTH_SLOTS 64u`
- `#define MEMTEST_GROWTH_SIZE 32768u`

Typedefs

- `typedef struct memtest_slot memtest_slot_t`
- `typedef struct memtest_stats memtest_stats_t`

Functions

- static uint8_t `payload_pattern` (uint8_t tag, size_t offset)
- static uint8_t `page_pattern` (size_t page_index, size_t offset)
- static void `memtest_fail` (const char *phase, size_t slot, size_t offset, uint8_t expected, uint8_t actual)
- static void `clear_slots` (void)
- static void `verify_prefix` (const char *phase, size_t slot_index, const uint8_t *ptr, size_t size, uint8_t tag)
- static void `fill_slot` (size_t slot_index)
- static void `verify_slot` (const char *phase, size_t slot_index)
- static void `verify_all` (const char *phase)
- static void `verify_alignment` (const char *phase, size_t slot_index, void *ptr)
- static void `verify_no_overlap` (const char *phase, size_t slot_index, uint8_t *ptr, size_t size)
- static void `set_slot` (size_t slot_index, uint8_t *ptr, size_t size, uint8_t tag, const char *phase)
- static void `release_slot` (const char *phase, size_t slot_index)
- static void `pmm_roundtrip_test` (void)

- static void `basic_heap_test` (void)
- static void `fragmentation_test` (void)
- static void `realloc_test` (void)
- static void `heap_growth_test` (void)
- static uint64_t `xorshift64` (uint64_t *state)
- static void `stats_add_live` (memtest_stats_t *stats, size_t amount)
- static void `random_cycle` (uint64_t seed, memtest_stats_t *stats)
- static void `randomized_heap_test` (void)
- void `memtest_run` (void)

Run the Stage-2 PMM/heap qualification suite.

Variables

- static memtest_slot_t `g_slots` [MEMTEST_SLOT_COUNT]
- static paddr_t `g_pmm_frames` [MEMTEST_PMM_MAX_FRAMES]
- volatile struct `limine_hhdm_request` `hhdm_request`
- static const size_t `g_basic_sizes` []

7.141.1 Macro Definition Documentation

7.141.1.1 MEMTEST_GROWTH_SIZE

```
#define MEMTEST_GROWTH_SIZE 32768u
```

Definition at line 24 of file `memtest.c`.

7.141.1.2 MEMTEST_GROWTH_SLOTS

```
#define MEMTEST_GROWTH_SLOTS 64u
```

Definition at line 23 of file `memtest.c`.

7.141.1.3 MEMTEST_PAGE_SIZE

```
#define MEMTEST_PAGE_SIZE 4096u
```

Definition at line 20 of file `memtest.c`.

7.141.1.4 MEMTEST_PMM_MAX_FRAMES

```
#define MEMTEST_PMM_MAX_FRAMES 256u
```

Definition at line 19 of file [memtest.c](#).

7.141.1.5 MEMTEST_RANDOM_MAX_ALLOC

```
#define MEMTEST_RANDOM_MAX_ALLOC 32768u
```

Definition at line 22 of file [memtest.c](#).

7.141.1.6 MEMTEST_RANDOM_OPS

```
#define MEMTEST_RANDOM_OPS 10000u
```

Definition at line 21 of file [memtest.c](#).

7.141.1.7 MEMTEST_SLOT_COUNT

```
#define MEMTEST_SLOT_COUNT 128u
```

Copyright (c) 2026 Tristan Adams

VoyagerOS64 Stage-2 memory qualification harness.

Definition at line 18 of file [memtest.c](#).

7.141.2 Typedef Documentation

7.141.2.1 memtest_slot_t

```
typedef struct memtest_slot memtest_slot_t
```

7.141.2.2 memtest_stats_t

```
typedef struct memtest_stats memtest_stats_t
```

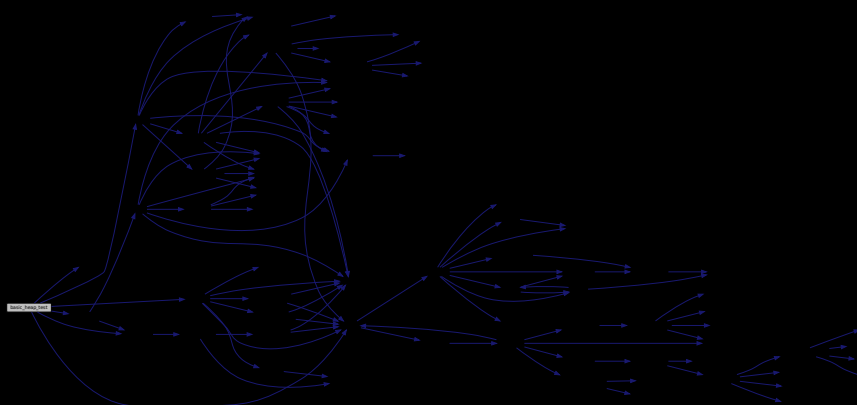

7.141.3 Function Documentation

7.141.3.1 basic_heap_test()

```
static void basic_heap_test (  
    void ) [static]
```

Definition at line 308 of file [memtest.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

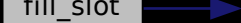


7.141.3.2 clear_slots()

```
static void clear_slots (  
    void ) [static]
```

Definition at line 76 of file [memtest.c](#).

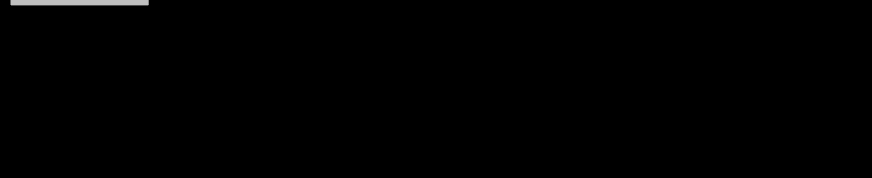
Here is the call graph for this function:



```

graph LR
    fill_slot[fill_slot] --> next_slot[ ]
    style next_slot fill:none,stroke:none

```

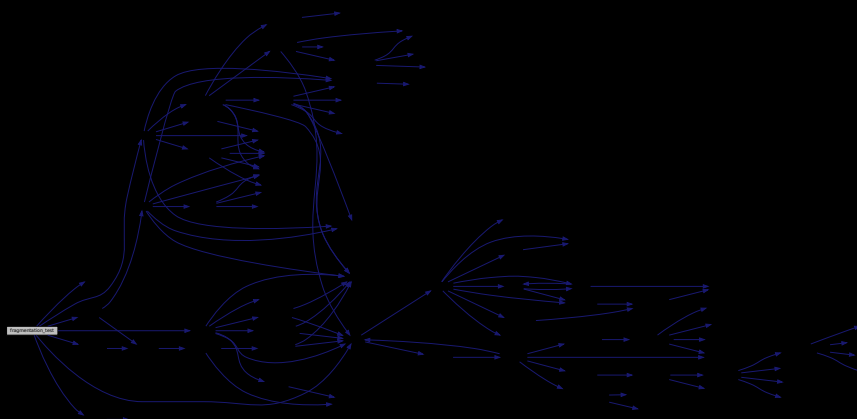


7.141.3.4 fragmentation_test()

```
static void fragmentation_test (  
    void ) [static]
```

Definition at line 324 of file [memtest.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

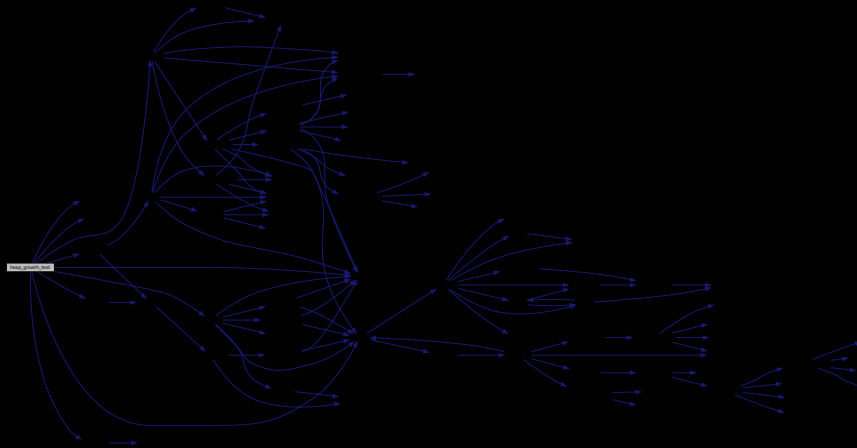


7.141.3.5 heap_growth_test()

```
static void heap_growth_test (  
    void ) [static]
```

Definition at line 436 of file [memtest.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

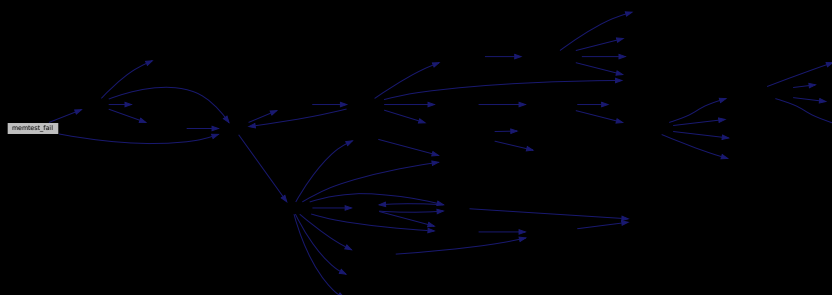


7.141.3.6 memtest_fail()

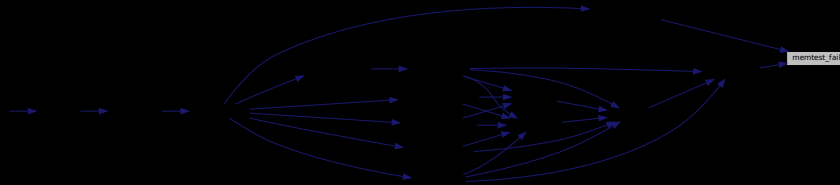
```
static void memtest_fail (
    const char * phase,
    size_t slot,
    size_t offset,
    uint8_t expected,
    uint8_t actual ) [static]
```

Definition at line 60 of file `memtest.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.141.3.7 memtest_run()

```
void memtest_run (  
    void )
```

Run the Stage-2 PMM/heap qualification suite.

Intended to be invoked after paging bootstrap, frame allocator initialization, kmalloc initialization, and scheduler startup. The test is destructive only to memory it allocates itself and yields periodically during long randomized phases so the cooperative scheduler continues to run.

The suite exercises direct frame allocation/free, heap boundary sizes, fragmentation and coalescing, krealloc preservation, multi-megabyte heap growth, and deterministic randomized allocation/free/reallocation cycles.

Note

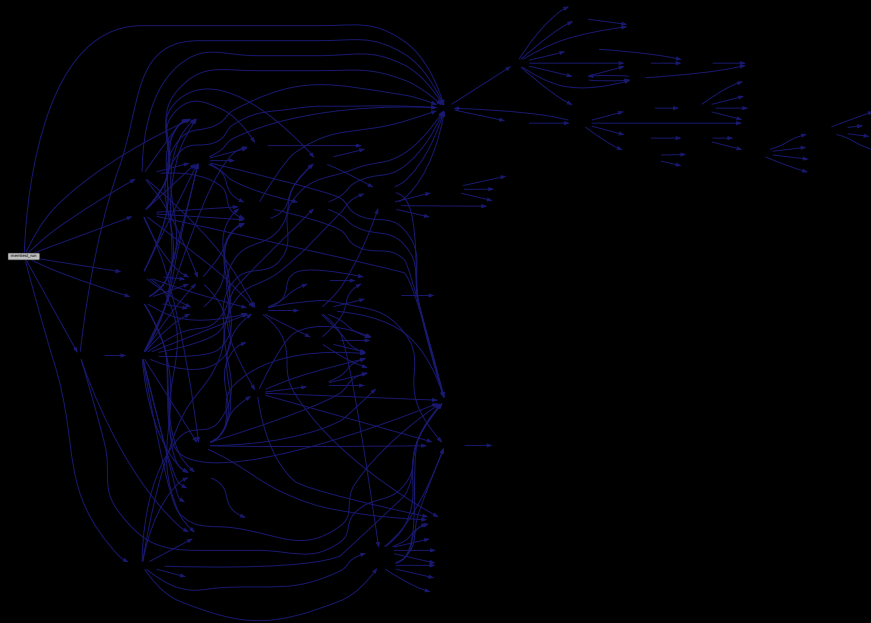
Current heap pages are retained after `kfree()`. A lower PMM free-frame count after the first high-water workload is therefore expected; repeated identical workloads should plateau once the heap has grown sufficiently.

Warning

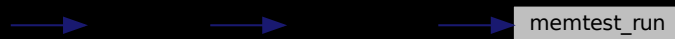
A detected corruption or violated allocator invariant is treated as a kernel qualification failure and may panic the system.

Definition at line 646 of file `memtest.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

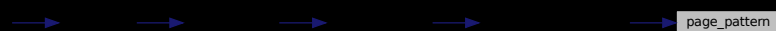


7.141.3.8 page_pattern()

```
static uint8_t page_pattern (
    size_t page_index,
    size_t offset ) [static]
```

Definition at line 54 of file [memtest.c](#).

Here is the caller graph for this function:

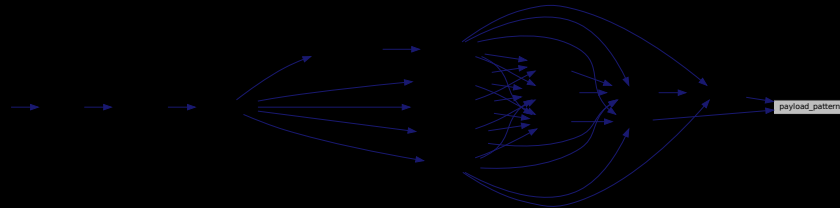


7.141.3.9 payload_pattern()

```
static uint8_t payload_pattern (  
    uint8_t tag,  
    size_t offset ) [static]
```

Definition at line 48 of file [memtest.c](#).

Here is the caller graph for this function:

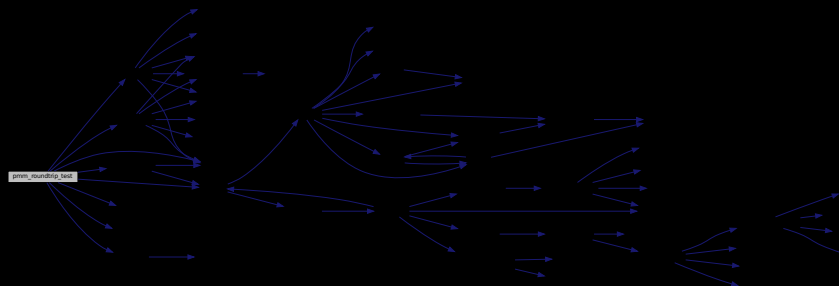


7.141.3.10 pmm_roundtrip_test()

```
static void pmm_roundtrip_test (  
    void ) [static]
```

Definition at line 214 of file [memtest.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

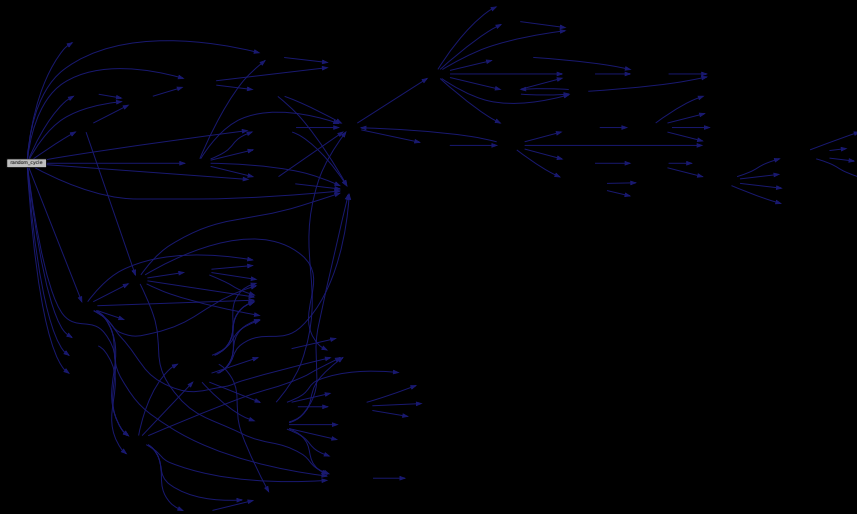


7.141.3.11 random_cycle()

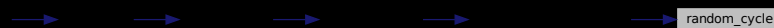
```
static void random_cycle (
    uint64_t seed,
    memtest_stats_t * stats ) [static]
```

Definition at line 493 of file [memtest.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.141.3.12 randomized_heap_test()

```
static void randomized_heap_test (
    void ) [static]
```

Definition at line 597 of file [memtest.c](#).


```
→ → → → randomized_heap_test
```

```
static void realloc_test (
    void ) [static]
```

Here is the call graph for this function:

```

graph LR
    A[ ] --> B[ ]
    B --> C[ ]
    C --> D[realloc_test]
    style A fill:none,stroke:none
    style B fill:none,stroke:none
    style C fill:none,stroke:none
    style D fill:#fff,stroke:#000
  
```

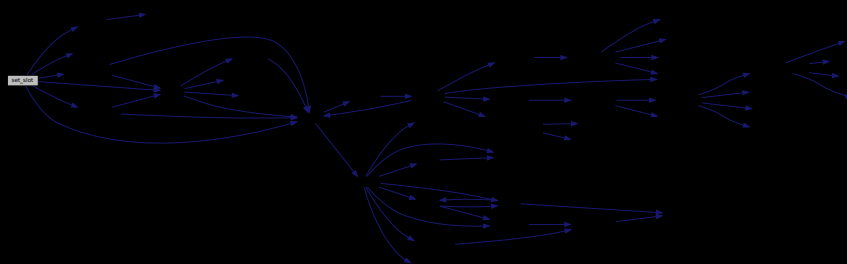
Here is the call graph for this function:

7.141.3.15 set_slot()

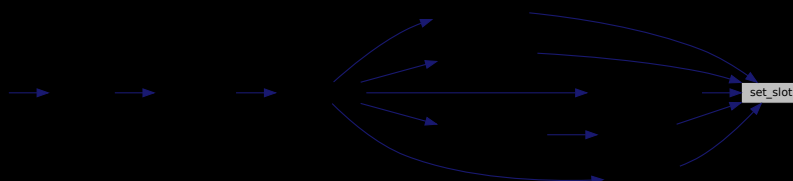
```
static void set_slot (
    size_t slot_index,
    uint8_t * ptr,
    size_t size,
    uint8_t tag,
    const char * phase ) [static]
```

Definition at line 169 of file [memtest.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.141.3.16 stats_add_live()

```
static void stats_add_live (
    memtest_stats_t * stats,
    size_t amount ) [static]
```

Definition at line 486 of file [memtest.c](#).

Here is the caller graph for this function:

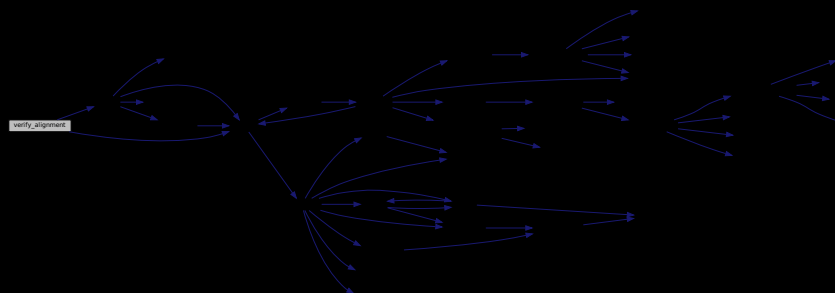


7.141.3.17 verify_alignment()

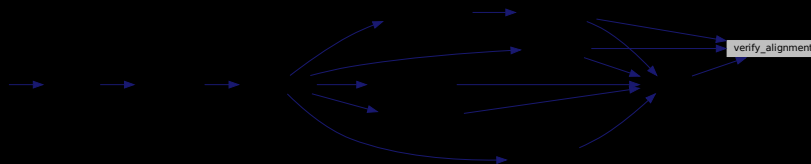
```
static void verify_alignment (
    const char * phase,
    size_t slot_index,
    void * ptr ) [static]
```

Definition at line 124 of file [memtest.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

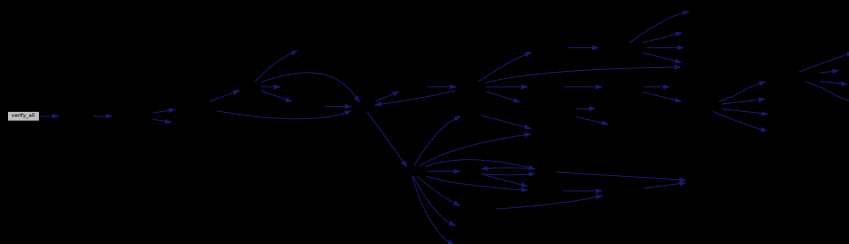


7.141.3.18 verify_all()

```
static void verify_all (
    const char * phase ) [static]
```

Definition at line 118 of file [memtest.c](#).

Here is the call graph for this function:



```

graph LR
    A[ ] --> B[ ]
    B --> C[ ]
    C --> D[ ]
    D --> E[ ]
    D --> F[ ]
    D --> G[ ]
    E --> H[ ]
    F --> H[ ]
    G --> H[ ]
    H --> I[verify_all]
    style A fill:none,stroke:none
    style B fill:none,stroke:none
    style C fill:none,stroke:none
    style D fill:none,stroke:none
    style E fill:none,stroke:none
    style F fill:none,stroke:none
    style G fill:none,stroke:none
    style H fill:none,stroke:none
    style I fill:#fff,stroke:#000
  
```

```
static void verify_no_overlap (
    const char * phase,
    size_t slot_index,
    uint8_t * ptr,
    size_t size ) [static]
```

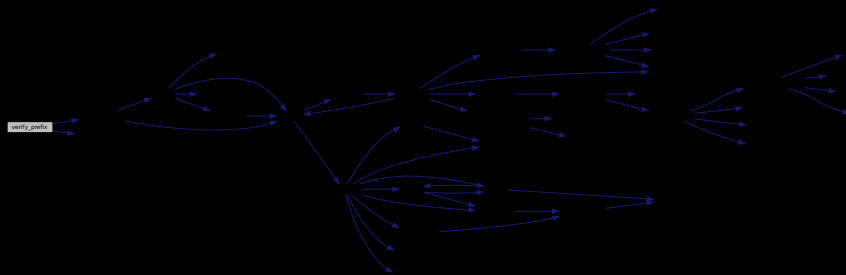
Here is the call graph for this function:

7.141.3.20 verify_prefix()

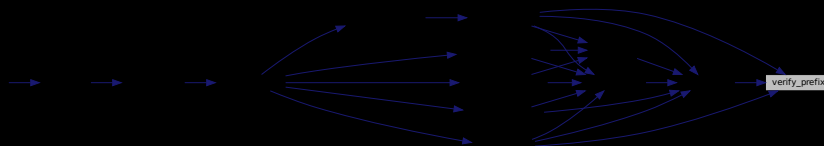
```
static void verify_prefix (
    const char * phase,
    size_t slot_index,
    const uint8_t * ptr,
    size_t size,
    uint8_t tag ) [static]
```

Definition at line 86 of file [memtest.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

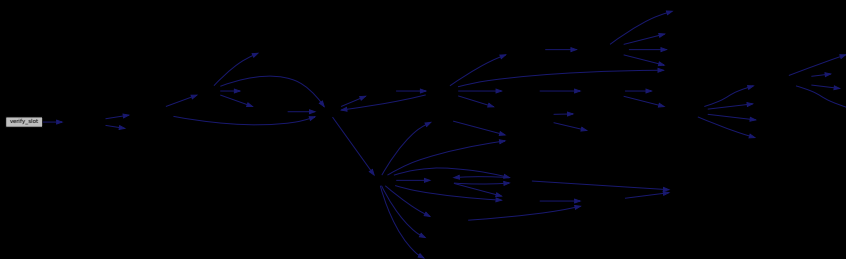


7.141.3.21 verify_slot()

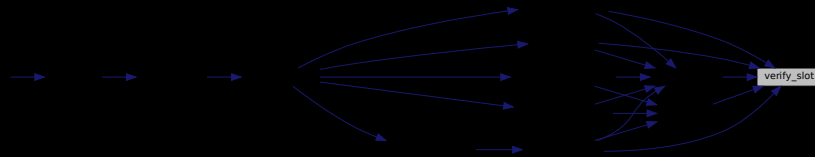
```
static void verify_slot (
    const char * phase,
    size_t slot_index ) [static]
```

Definition at line 108 of file [memtest.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.141.3.22 xorshift64()

```
static uint64_t xorshift64 (
    uint64_t * state ) [static]
```

Definition at line 476 of file `memtest.c`.

Here is the caller graph for this function:



7.141.4 Variable Documentation

7.141.4.1 g_basic_sizes

```
const size_t g_basic_sizes[] [static]
```

Initial value:

```
= {
    1u,    7u,    15u,    16u,    17u,    31u,    32u,
    33u,    63u,    64u,    65u,    255u,    256u,    257u,
    1000u, 4095u, 4096u, 4097u, 8192u, 16384u,
}
```

Definition at line 302 of file `memtest.c`.

7.141.4.2 g_pmm_frames

```
paddr_t g_pmm_frames[MEMTEST_PMM_MAX_FRAMES] [static]
```

Definition at line 44 of file `memtest.c`.

7.141.4.3 g_slots

```
memtest_slot_t g_slots[MEMTEST_SLOT_COUNT] [static]
```

Definition at line 43 of file [memtest.c](#).

7.141.4.4 hhdm_request

```
volatile struct limine_hhdm_request hhdm_request [extern]
```

7.142 memtest.c

[Go to the documentation of this file.](#)

```
00001
00007 #include "../include/mm/memtest.h"
00008 #include "../include/limine.h"
00009 #include "../include/mm/kmalloc.h"
00010 #include "../include/paging/neo_framealloc.h"
00011 #include "../include/panic.h"
00012 #include "../include/printf.h"
00013 #include "../include/sched.h"
00014 #include <stdbool.h>
00015 #include <stddef.h>
00016 #include <stdint.h>
00017
00018 #define MEMTEST_SLOT_COUNT 128u
00019 #define MEMTEST_PMM_MAX_FRAMES 256u
00020 #define MEMTEST_PAGE_SIZE 4096u
00021 #define MEMTEST_RANDOM_OPS 10000u
00022 #define MEMTEST_RANDOM_MAX_ALLOC 32768u
00023 #define MEMTEST_GROWTH_SLOTS 64u
00024 #define MEMTEST_GROWTH_SIZE 32768u
00025
00026 typedef struct memtest_slot
00027 {
00028     uint8_t *ptr;
00029     size_t size;
00030     uint8_t tag;
00031 } memtest_slot_t;
00032
00033 typedef struct memtest_stats
00034 {
00035     uint64_t operations;
00036     uint64_t allocations;
00037     uint64_t frees;
00038     uint64_t reallocations;
00039     uint64_t live_bytes;
00040     uint64_t peak_live_bytes;
00041 } memtest_stats_t;
00042
00043 static memtest_slot_t g_slots[MEMTEST_SLOT_COUNT];
00044 static paddr_t g_pmm_frames[MEMTEST_PMM_MAX_FRAMES];
00045
00046 extern volatile struct limine_hhdm_request hhdm_request;
00047
00048 static uint8_t payload_pattern(uint8_t tag, size_t offset)
00049 {
00050     return (uint8_t) (tag ^ (uint8_t) offset ^ (uint8_t) (offset >> 8) ^
00051                     (uint8_t) (offset >> 16));
00052 }
00053
00054 static uint8_t page_pattern(size_t page_index, size_t offset)
00055 {
00056     return (uint8_t) ((uint8_t) page_index ^ (uint8_t) offset ^
00057                     (uint8_t) (offset >> 8) ^ 0xA5u);
00058 }
00059
00060 static void memtest_fail(const char *phase,
00061                         size_t slot,
00062                         size_t offset,
```



```

00063             uint8_t     expected,
00064             uint8_t     actual)
00065 {
00066     printf_("%s\n", "");
00067     printf_("%s\n", "[MEMTEST] FAILURE");
00068     printf_("%s%s\n", "Phase: ", phase);
00069     printf_("%sllu\n", "Slot: ", (unsigned long long) slot);
00070     printf_("%sllu\n", "Offset: ", (unsigned long long) offset);
00071     printf_("%s%u\n", "Expected: ", (unsigned int) expected);
00072     printf_("%s%u\n", "Actual: ", (unsigned int) actual);
00073     panic("memtest: data corruption detected");
00074 }
00075
00076 static void clear_slots(void)
00077 {
00078     for (size_t i = 0; i < MEMTEST_SLOT_COUNT; i++)
00079     {
00080         g_slots[i].ptr = NULL;
00081         g_slots[i].size = 0;
00082         g_slots[i].tag = 0;
00083     }
00084 }
00085
00086 static void verify_prefix(const char *phase,
00087                          size_t slot_index,
00088                          const uint8_t *ptr,
00089                          size_t size,
00090                          uint8_t tag)
00091 {
00092     for (size_t i = 0; i < size; i++)
00093     {
00094         uint8_t expected = payload_pattern(tag, i);
00095         if (ptr[i] != expected)
00096             memtest_fail(phase, slot_index, i, expected, ptr[i]);
00097     }
00098 }
00099
00100 static void fill_slot(size_t slot_index)
00101 {
00102     memtest_slot_t *slot = &g_slots[slot_index];
00103
00104     for (size_t i = 0; i < slot->size; i++)
00105         slot->ptr[i] = payload_pattern(slot->tag, i);
00106 }
00107
00108 static void verify_slot(const char *phase, size_t slot_index)
00109 {
00110     memtest_slot_t *slot = &g_slots[slot_index];
00111
00112     if (!slot->ptr)
00113         return;
00114
00115     verify_prefix(phase, slot_index, slot->ptr, slot->size, slot->tag);
00116 }
00117
00118 static void verify_all(const char *phase)
00119 {
00120     for (size_t i = 0; i < MEMTEST_SLOT_COUNT; i++)
00121         verify_slot(phase, i);
00122 }
00123
00124 static void verify_alignment(const char *phase, size_t slot_index, void *ptr)
00125 {
00126     if (((uintptr_t) ptr & 0xFu) != 0)
00127     {
00128         printf_("%s%s\n", "[MEMTEST] Phase: ", phase);
00129         printf_("%sllu\n", "Unaligned slot: ",
00130                (unsigned long long) slot_index);
00131         printf_("%sllx\n", "Pointer: 0x",
00132                (unsigned long long) (uintptr_t) ptr);
00133         panic("memtest: kmalloc returned a non-16-byte-aligned pointer");
00134     }
00135 }
00136
00137 static void verify_no_overlap(const char *phase,
00138                              size_t slot_index,
00139                              uint8_t *ptr,
00140                              size_t size)
00141 {
00142     uintptr_t start = (uintptr_t) ptr;
00143     uintptr_t end = start + size;
00144
00145     if (end < start)
00146         panic("memtest: allocation address overflow");
00147
00148     for (size_t i = 0; i < MEMTEST_SLOT_COUNT; i++)
00149     {

```

```

00150         if (i == slot_index || g_slots[i].ptr == NULL)
00151             continue;
00152
00153         uintptr_t other_start = (uintptr_t) g_slots[i].ptr;
00154         uintptr_t other_end   = other_start + g_slots[i].size;
00155
00156         if (start < other_end && other_start < end)
00157         {
00158             printf("%s%s\n", "[MEMTEST] Phase: ", phase);
00159             printf("%sllu%sllu\n",
00160                 "Allocation overlap between slots ",
00161                 (unsigned long long) slot_index,
00162                 " and ",
00163                 (unsigned long long) i);
00164             panic("memtest: allocator returned overlapping payloads");
00165         }
00166     }
00167 }
00168
00169 static void set_slot(size_t slot_index, uint8_t *ptr, size_t size, uint8_t tag,
00170                     const char *phase)
00171 {
00172     if (!ptr)
00173     {
00174         printf("%s%s\n", "[MEMTEST] Allocation failed in phase: ", phase);
00175         printf("%sllu\n", "Requested bytes: ",
00176             (unsigned long long) size);
00177         panic("memtest: unexpected allocation failure");
00178     }
00179
00180     verify_alignment(phase, slot_index, ptr);
00181
00182     size_t usable = kmalloc_usable_size(ptr);
00183     if (usable < size)
00184     {
00185         printf("%s%s\n", "[MEMTEST] Phase: ", phase);
00186         printf("%sllu%sllu\n",
00187             "Allocator reported usable size ",
00188             (unsigned long long) usable,
00189             " for request ",
00190             (unsigned long long) size);
00191         panic("memtest: kmalloc usable size smaller than requested size");
00192     }
00193
00194     verify_no_overlap(phase, slot_index, ptr, size);
00195
00196     g_slots[slot_index].ptr = ptr;
00197     g_slots[slot_index].size = size;
00198     g_slots[slot_index].tag = tag;
00199     fill_slot(slot_index);
00200 }
00201
00202 static void release_slot(const char *phase, size_t slot_index)
00203 {
00204     if (!g_slots[slot_index].ptr)
00205         return;
00206
00207     verify_slot(phase, slot_index);
00208     kfree(g_slots[slot_index].ptr);
00209     g_slots[slot_index].ptr = NULL;
00210     g_slots[slot_index].size = 0;
00211     g_slots[slot_index].tag = 0;
00212 }
00213
00214 static void pmm_roundtrip_test(void)
00215 {
00216     printf("%s\n", "[MEMTEST] 1/6 neo-PMM round-trip...");
00217
00218     if (!hhdm_request.response)
00219         panic("memtest: HHDM unavailable");
00220
00221     size_t before = frame_free_count();
00222     size_t count = MEMTEST_PMM_MAX_FRAMES;
00223
00224     if (before <= count + 32u)
00225         count = before / 4u;
00226     if (count < 8u)
00227         panic("memtest: insufficient free frames for PMM qualification");
00228
00229     uint64_t hhdm = hhdm_request.response->offset;
00230
00231     for (size_t i = 0; i < count; i++)
00232     {
00233         paddr_t p = frame_alloc();
00234
00235         if ((p & (MEMTEST_PAGE_SIZE - 1u)) != 0)
00236             panic("memtest: frame_alloc returned an unaligned frame");

```

```

00237
00238     for (size_t j = 0; j < i; j++)
00239     {
00240         if (g_pmm_frames[j] == p)
00241             panic("memtest: frame_alloc returned a duplicate live frame");
00242     }
00243
00244     g_pmm_frames[i] = p;
00245     uint8_t *page = (uint8_t *) (uintptr_t) (p + hhdmm);
00246
00247     for (size_t off = 0; off < MEMTEST_PAGE_SIZE; off++)
00248         page[off] = page_pattern(i, off);
00249
00250     if ((i & 31u) == 31u)
00251         schedule();
00252 }
00253
00254 size_t after_alloc = frame_free_count();
00255 if (after_alloc + count != before)
00256 {
00257     printf("%sllu%sllu%sllu\n",
00258         "[MEMTEST] PMM count mismatch: before=",
00259         (unsigned long long) before,
00260         " after_alloc=",
00261         (unsigned long long) after_alloc,
00262         " frames=",
00263         (unsigned long long) count);
00264     panic("memtest: PMM accounting mismatch after allocation");
00265 }
00266
00267 for (size_t i = 0; i < count; i++)
00268 {
00269     uint8_t *page =
00270         (uint8_t *) (uintptr_t) (g_pmm_frames[i] + hhdmm);
00271
00272     for (size_t off = 0; off < MEMTEST_PAGE_SIZE; off++)
00273     {
00274         uint8_t expected = page_pattern(i, off);
00275         if (page[off] != expected)
00276             memtest_fail("neo-PMM page verify",
00277                 i,
00278                 off,
00279                 expected,
00280                 page[off]);
00281     }
00282 }
00283
00284 for (size_t i = count; i > 0; i--)
00285 {
00286     n_frame_free(g_pmm_frames[i - 1]);
00287     g_pmm_frames[i - 1] = 0;
00288 }
00289
00290 size_t after_free = frame_free_count();
00291 if (after_free != before)
00292 {
00293     printf("%sllu%sllu\n",
00294         "[MEMTEST] PMM free-count leak: before=",
00295         (unsigned long long) before,
00296         " after=",
00297         (unsigned long long) after_free);
00298     panic("memtest: PMM did not recover all round-trip frames");
00299 }
00300 }
00301
00302 static const size_t g_basic_sizes[] = {
00303     1u,    7u,    15u,    16u,    17u,    31u,    32u,
00304     33u,   63u,   64u,   65u,   255u,  256u,  257u,
00305     1000u, 4095u, 4096u, 4097u, 8192u, 16384u,
00306 };
00307
00308 static void basic_heap_test(void)
00309 {
00310     printf("%s\n", "[MEMTEST] 2/6 basic kcalloc/free boundaries...");
00311     clear_slots();
00312
00313     size_t count = sizeof(g_basic_sizes) / sizeof(g_basic_sizes[0]);
00314     for (size_t i = 0; i < count; i++)
00315     {
00316         size_t size = g_basic_sizes[i];
00317         uint8_t tag = (uint8_t) (0x31u + i);
00318         set_slot(0, kcalloc(size), size, tag, "basic allocation");
00319         verify_slot("basic verify", 0);
00320         release_slot("basic free", 0);
00321     }
00322 }
00323

```

```

00324 static void fragmentation_test(void)
00325 {
00326     printf("%s\n", "[MEMTEST] 3/6 fragmentation/split/coalesce...");
00327     clear_slots();
00328
00329     for (size_t i = 0; i < MEMTEST_SLOT_COUNT; i++)
00330     {
00331         size_t size = 24u + ((i * 173u) % 7000u);
00332         uint8_t tag = (uint8_t) (0x40u ^ i);
00333         if (tag == 0)
00334             tag = 0x40u;
00335         set_slot(i, kcalloc(size), size, tag, "fragmentation initial fill");
00336     }
00337
00338     verify_all("fragmentation initial verify");
00339
00340     for (size_t i = 1; i < MEMTEST_SLOT_COUNT; i += 2)
00341         release_slot("fragmentation hole creation", i);
00342
00343     verify_all("fragmentation survivors");
00344
00345     for (size_t i = 1; i < MEMTEST_SLOT_COUNT; i += 2)
00346     {
00347         size_t size = 73u + ((i * 521u) % 9000u);
00348         uint8_t tag = (uint8_t) (0xB3u ^ i);
00349         if (tag == 0)
00350             tag = 0xB3u;
00351         set_slot(i, kcalloc(size), size, tag, "fragmentation hole refill");
00352     }
00353
00354     verify_all("fragmentation refill verify");
00355
00356     /* 73 is coprime with 128, so this visits every slot exactly once. */
00357     for (size_t step = 0; step < MEMTEST_SLOT_COUNT; step++)
00358     {
00359         size_t index = (step * 73u) % MEMTEST_SLOT_COUNT;
00360         release_slot("fragmentation shuffled free", index);
00361         if ((step & 31u) == 31u)
00362             schedule();
00363     }
00364 }
00365
00366 static void realloc_test(void)
00367 {
00368     printf("%s\n", "[MEMTEST] 4/6 krealloc preservation...");
00369     clear_slots();
00370
00371     static const size_t sizes[] = {
00372         100u, 200u, 2000u, 4097u, 16384u, 8192u, 128u,
00373     };
00374
00375     uint8_t tag = 0x5Au;
00376     set_slot(0, kcalloc(sizes[0]), sizes[0], tag, "realloc initial");
00377
00378     for (size_t i = 1; i < sizeof(sizes) / sizeof(sizes[0]); i++)
00379     {
00380         verify_slot("realloc precheck", 0);
00381
00382         size_t old_size = g_slots[0].size;
00383         uint8_t *old_ptr = g_slots[0].ptr;
00384         uint8_t *new_ptr = krealloc(old_ptr, sizes[i]);
00385         if (!new_ptr)
00386             panic("memtest: krealloc unexpectedly failed");
00387
00388         size_t preserved = old_size < sizes[i] ? old_size : sizes[i];
00389         verify_prefix("realloc preserved prefix", 0, new_ptr, preserved, tag);
00390
00391         g_slots[0].ptr = new_ptr;
00392         g_slots[0].size = sizes[i];
00393         g_slots[0].tag = tag;
00394         verify_alignment("realloc alignment", 0, new_ptr);
00395         fill_slot(0);
00396     }
00397
00398     release_slot("realloc chain free", 0);
00399
00400     uint8_t *special = krealloc(NULL, 512u);
00401     set_slot(0, special, 512u, 0xC7u, "krealloc NULL");
00402     verify_slot("krealloc NULL verify", 0);
00403
00404     uint8_t *zero_result = krealloc(g_slots[0].ptr, 0);
00405     if (zero_result != NULL)
00406         panic("memtest: krealloc(ptr, 0) did not return NULL");
00407     g_slots[0].ptr = NULL;
00408     g_slots[0].size = 0;
00409     g_slots[0].tag = 0;
00410

```

```

00411     /* Encourage the in-place grow path by freeing the immediate successor. */
00412     set_slot(0, kcalloc(2048u), 2048u, 0xD1u, "realloc adjacent A");
00413     set_slot(1, kcalloc(8192u), 8192u, 0xD2u, "realloc adjacent B");
00414     release_slot("realloc free adjacent B", 1);
00415
00416     uint8_t *before = g_slots[0].ptr;
00417     uint8_t *grown = krealloc(before, 6144u);
00418     if (!grown)
00419         panic("memtest: adjacent krealloc failed");
00420     verify_prefix("realloc adjacent preservation", 0, grown, 2048u, 0xD1u);
00421
00422     g_slots[0].ptr = grown;
00423     g_slots[0].size = 6144u;
00424     g_slots[0].tag = 0xD1u;
00425     fill_slot(0);
00426     verify_slot("realloc adjacent final verify", 0);
00427
00428     if (grown == before)
00429         printf_("%s\n", "[MEMTEST] krealloc in-place growth path observed.");
00430     else
00431         printf_("%s\n", "[MEMTEST] krealloc used fallback move path.");
00432
00433     release_slot("realloc adjacent final free", 0);
00434 }
00435
00436 static void heap_growth_test(void)
00437 {
00438     printf_("%s\n", "[MEMTEST] 5/6 forcing multi-megabyte heap growth...");
00439     clear_slots();
00440
00441     size_t frames_before = frame_free_count();
00442
00443     for (size_t i = 0; i < MEMTEST_GROWTH_SLOTS; i++)
00444     {
00445         size_t size = MEMTEST_GROWTH_SIZE + ((i & 7u) * 257u);
00446         uint8_t tag = (uint8_t) (0x70u + i);
00447         set_slot(i, kcalloc(size), size, tag, "heap growth allocation");
00448
00449         if ((i & 7u) == 7u)
00450             schedule();
00451     }
00452
00453     verify_all("heap growth verify");
00454
00455     size_t frames_after_alloc = frame_free_count();
00456
00457     for (size_t i = MEMTEST_GROWTH_SLOTS; i > 0; i--)
00458         release_slot("heap growth free", i - 1);
00459
00460     size_t frames_after_free = frame_free_count();
00461
00462     printf_("%sllu\n",
00463             "[MEMTEST] PMM frames before heap growth: ",
00464             (unsigned long long) frames_before);
00465     printf_("%sllu\n",
00466             "[MEMTEST] PMM frames after heap growth: ",
00467             (unsigned long long) frames_after_alloc);
00468     printf_("%sllu\n",
00469             "[MEMTEST] PMM frames after heap frees: ",
00470             (unsigned long long) frames_after_free);
00471
00472     if (frames_after_free != frames_after_alloc)
00473         panic("memtest: heap frees unexpectedly changed PMM frame ownership");
00474 }
00475
00476 static uint64_t xorshift64(uint64_t *state)
00477 {
00478     uint64_t x = *state;
00479     x ^= x << 13;
00480     x ^= x >> 7;
00481     x ^= x << 17;
00482     *state = x;
00483     return x;
00484 }
00485
00486 static void stats_add_live(memtest_stats_t *stats, size_t amount)
00487 {
00488     stats->live_bytes += amount;
00489     if (stats->live_bytes > stats->peak_live_bytes)
00490         stats->peak_live_bytes = stats->live_bytes;
00491 }
00492
00493 static void random_cycle(uint64_t seed, memtest_stats_t *stats)
00494 {
00495     clear_slots();
00496
00497     for (uint64_t op = 0; op < MEMTEST_RANDOM_OPS; op++)

```

```

00498     {
00499         size_t index =
00500             (size_t) (xorshift64(&seed) % MEMTEST_SLOT_COUNT);
00501
00502         if (!g_slots[index].ptr)
00503         {
00504             size_t size =
00505                 1u + (size_t) (xorshift64(&seed) % MEMTEST_RANDOM_MAX_ALLOC);
00506             uint8_t tag = (uint8_t) (xorshift64(&seed) & 0xFFu);
00507             if (tag == 0)
00508                 tag = 0xA5u;
00509
00510             set_slot(index,
00511                     kmalloc(size),
00512                     size,
00513                     tag,
00514                     "random allocation");
00515             stats->allocations++;
00516             stats_add_live(stats, size);
00517         }
00518         else
00519         {
00520             uint64_t action = xorshift64(&seed) % 100u;
00521
00522             if (action < 35u)
00523             {
00524                 size_t old_size = g_slots[index].size;
00525                 release_slot("random free", index);
00526                 stats->freess++;
00527                 stats->live_bytes -= old_size;
00528             }
00529             else if (action < 75u)
00530             {
00531                 verify_slot("random realloc precheck", index);
00532
00533                 size_t old_size = g_slots[index].size;
00534                 uint8_t tag = g_slots[index].tag;
00535                 uint8_t *old_ptr = g_slots[index].ptr;
00536                 size_t new_size =
00537                     1u + (size_t) (xorshift64(&seed) % MEMTEST_RANDOM_MAX_ALLOC);
00538
00539                 uint8_t *new_ptr = krealloc(old_ptr, new_size);
00540                 if (!new_ptr)
00541                     panic("memtest: random krealloc failed");
00542
00543                 size_t preserved = old_size < new_size ? old_size : new_size;
00544                 verify_prefix("random realloc preservation",
00545                             index,
00546                             new_ptr,
00547                             preserved,
00548                             tag);
00549
00550                 g_slots[index].ptr = new_ptr;
00551                 g_slots[index].size = new_size;
00552                 g_slots[index].tag = tag;
00553                 verify_alignment("random realloc alignment", index, new_ptr);
00554                 verify_no_overlap("random realloc overlap",
00555                                 index,
00556                                 new_ptr,
00557                                 new_size);
00558                 fill_slot(index);
00559
00560                 stats->reallocations++;
00561                 stats->live_bytes -= old_size;
00562                 stats_add_live(stats, new_size);
00563             }
00564             else
00565             {
00566                 verify_slot("random spot verify", index);
00567             }
00568         }
00569         stats->operations++;
00570
00571         if ((op & 63u) == 63u)
00572         {
00573             verify_all("random periodic full verify");
00574             schedule();
00575         }
00576     }
00577 }
00578
00579 verify_all("random final verify");
00580
00581 for (size_t step = 0; step < MEMTEST_SLOT_COUNT; step++)
00582 {
00583     size_t index = (step * 73u) % MEMTEST_SLOT_COUNT;
00584     if (g_slots[index].ptr)

```

```

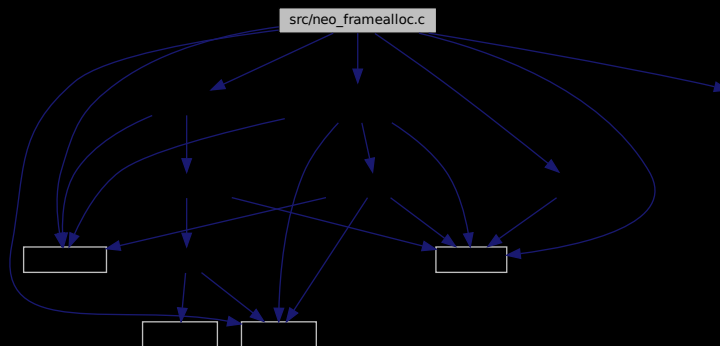
00585     {
00586         size_t old_size = g_slots[index].size;
00587         release_slot("random cleanup", index);
00588         stats->frees++;
00589         stats->live_bytes -= old_size;
00590     }
00591 }
00592
00593 if (stats->live_bytes != 0)
00594     panic("memtest: random-cycle live byte accounting did not return to zero");
00595 }
00596
00597 static void randomized_heap_test(void)
00598 {
00599     printf_("%s\n", "[MEMTEST] 6/6 deterministic randomized torture...");
00600
00601     const uint64_t seed = 0x564F59414745524FULL; /* \ "VOYAGERO\ " */
00602
00603     memtest_stats_t first = {0};
00604     memtest_stats_t second = {0};
00605
00606     size_t frames_before = frame_free_count();
00607     random_cycle(seed, &first);
00608     size_t frames_after_first = frame_free_count();
00609
00610     random_cycle(seed, &second);
00611     size_t frames_after_second = frame_free_count();
00612
00613     printf_("%sllu\n", "[MEMTEST] Random operations/cycle: ",
00614             (unsigned long long) first.operations);
00615     printf_("%sllu\n", "[MEMTEST] Allocations: ",
00616             (unsigned long long) first.allocations);
00617     printf_("%sllu\n", "[MEMTEST] Frees: ",
00618             (unsigned long long) first.frees);
00619     printf_("%sllu\n", "[MEMTEST] Reallocations: ",
00620             (unsigned long long) first.reallocations);
00621     printf_("%sllu\n", "[MEMTEST] Peak live bytes: ",
00622             (unsigned long long) first.peak_live_bytes);
00623     printf_("%sllu%sllu%sllu\n",
00624             "[MEMTEST] PMM free frames before/after1/after2: ",
00625             (unsigned long long) frames_before,
00626             " / ",
00627             (unsigned long long) frames_after_first,
00628             " / ",
00629             (unsigned long long) frames_after_second);
00630
00631     if (first.operations != second.operations ||
00632         first.allocations != second.allocations ||
00633         first.reallocations != second.reallocations ||
00634         first.peak_live_bytes != second.peak_live_bytes)
00635         panic("memtest: deterministic random cycles produced different stats");
00636
00637     if (frames_after_second < frames_after_first)
00638     {
00639         printf_("%s\n",
00640                 "[MEMTEST] WARNING: second identical cycle grew the heap again.");
00641         printf_("%s\n",
00642                 "[MEMTEST] Data integrity passed, but inspect fragmentation/reuse.");
00643     }
00644 }
00645
00646 void memtest_run(void)
00647 {
00648     printf_("%s\n", "");
00649     printf_("%s\n", "=====");
00650     printf_("%s\n", " VoyagerOS64 Stage-2 Memory Qualification");
00651     printf_("%s\n", "=====");
00652
00653     pmm_roundtrip_test();
00654     basic_heap_test();
00655     fragmentation_test();
00656     realloc_test();
00657     heap_growth_test();
00658     randomized_heap_test();
00659
00660     clear_slots();
00661
00662     printf_("%s\n", "=====");
00663     printf_("%s\n", "[MEMTEST] PASS - no corruption detected");
00664     printf_("%s\n", "=====");
00665 }

```

7.143 src/neo_framealloc.c File Reference

```
#include "include/paging/neo_framealloc.h"
#include "include/limine.h"
#include "include/lock.h"
#include "include/panic.h"
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
```

Include dependency graph for neo_framealloc.c:



Macros

- `#define PAGE_SIZE 4096ull`
- `#define ALIGN_UP(x, a) (((x) + ((a) - 1)) & ~((a) - 1))`
- `#define ALIGN_DOWN(x, a) ((x) & ~((a) - 1))`

Functions

- `void early_init (void)`
Select a Limine USABLE memory region and initialize the bump allocator.
- `paddr_t early_alloc_page (void)`
Allocate one 4 KiB physical page from the bootstrap region.
- `void early_reserved_region (paddr_t *base, size_t *pages)`
Report the physical range consumed by early allocation.
- `void panic (const char *fmt,...)`
Panic with a nicely formatted message. This is useful for situations where you are certain an unrecoverable error will occur and want to be informative (feel free to abuse sarcasm in them) about the problem.
- `static void memzero (void *p, size_t n)`
- `static void memset8 (void *p, uint8_t v, size_t n)`
- `static void bm_set (uint8_t *bm, size_t i)`
- `static void bm_clear (uint8_t *bm, size_t i)`
- `static bool bm_test (const uint8_t *bm, size_t i)`
- `static size_t frame_index_of (paddr_t p)`
- `static paddr_t frame_addr_of (size_t i)`
- `static void compute_max_phys_and_count (void)`
- `static void mark_range_allocated (paddr_t base, paddr_t end)`

- static void `mark_range_usable_and_free` (`paddr_t` base, `paddr_t` end)
- void `frame_init` (void)
Initialize the permanent physical memory manager from Limine's memmap.
- `paddr_t` `frame_alloc` (void)
Allocate one 4 KiB physical frame.
- void `n_frame_free` (`paddr_t` paddr)
Return a 4 KiB physical frame to the PMM.
- bool `frame_is_allocated` (`paddr_t` paddr)
Query allocation state for a physical frame.
- `size_t` `frame_total_count` (void)
Count physical frames managed by the PMM.
- `size_t` `frame_free_count` (void)
Count currently free physical frames.

Variables

- volatile struct `limine_memmap_request` `memmap_req`
- volatile struct `limine_kernel_address_request` `Kaddress_req`
- volatile struct `limine_hhdm_request` `hhdm_request`
- volatile struct `limine_module_request` `module_request`
- char `__kernel_start` []
- char `__kernel_end` []
- static `uint8_t` * `g_alloc_bm` = 0
- static `uint8_t` * `g_usable_bm` = 0
- static `size_t` `g_frame_count` = 0
- static `uint64_t` `g_max_phys` = 0
- static `size_t` `g_free_count` = 0
- static `size_t` `g_hint` = 0
- static `spinlock_t` `g_memlock` = `SPINLOCK_INIT`

7.143.1 Macro Definition Documentation

7.143.1.1 ALIGN_DOWN

```
#define ALIGN_DOWN(  
    x,  
    a ) ((x) & ~((a) -1))
```

Definition at line 29 of file `neo_framealloc.c`.

7.143.1.2 ALIGN_UP

```
#define ALIGN_UP(  
    x,  
    a ) (((x) + ((a) -1)) & ~((a) -1))
```

Definition at line 28 of file `neo_framealloc.c`.

7.143.1.3 PAGE_SIZE

```
#define PAGE_SIZE 4096ull
```

Definition at line 27 of file [neo_framealloc.c](#).

7.143.2 Function Documentation

7.143.2.1 bm_clear()

```
static void bm_clear (  
    uint8_t * bm,  
    size_t i ) [inline], [static]
```

Definition at line 65 of file [neo_framealloc.c](#).

Here is the caller graph for this function:

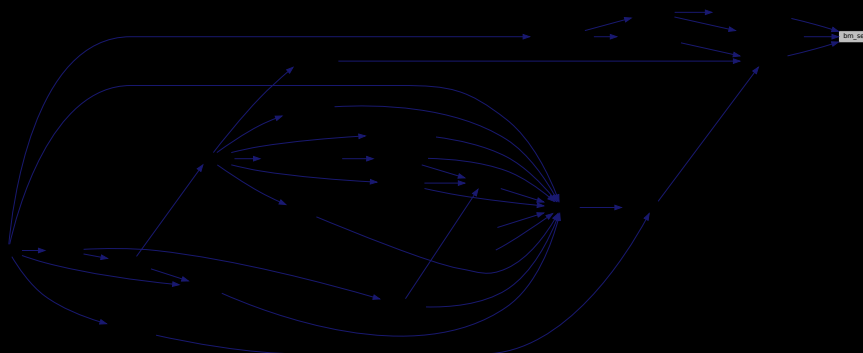


7.143.2.2 bm_set()

```
static void bm_set (  
    uint8_t * bm,  
    size_t i ) [inline], [static]
```

Definition at line 61 of file [neo_framealloc.c](#).

Here is the caller graph for this function:

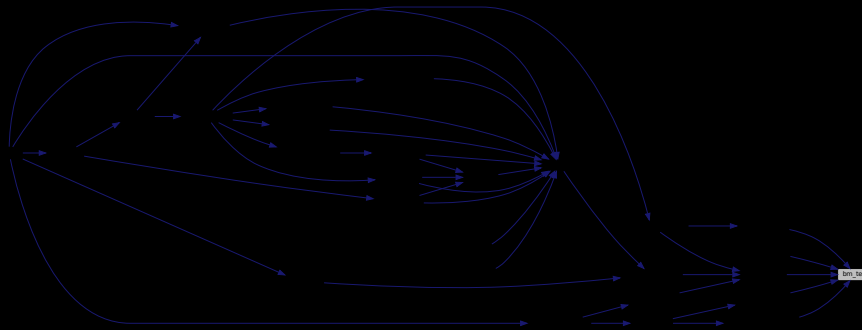


7.143.2.3 `bm_test()`

```
static bool bm_test (
    const uint8_t * bm,
    size_t i )    [inline], [static]
```

Definition at line 69 of file `neo_framealloc.c`.

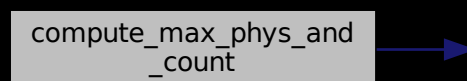
Here is the caller graph for this function:

7.143.2.4 `compute_max_phys_and_count()`

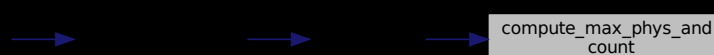
```
static void compute_max_phys_and_count (
    void )    [static]
```

Definition at line 84 of file `neo_framealloc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.143.2.5 `early_alloc_page()`

```
paddr_t early_alloc_page (
    void )
```

Allocate one 4 KiB physical page from the bootstrap region.

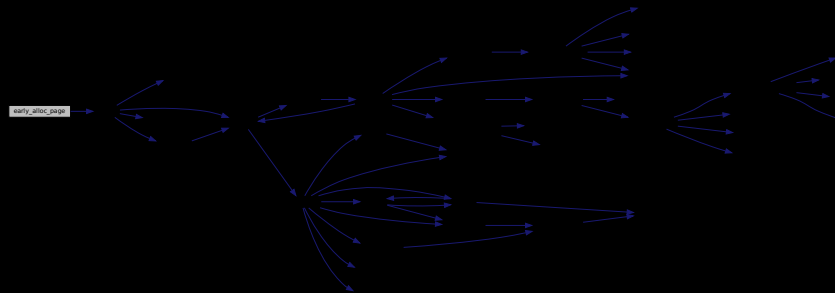
Returns

4 KiB-aligned physical address of the allocated page.

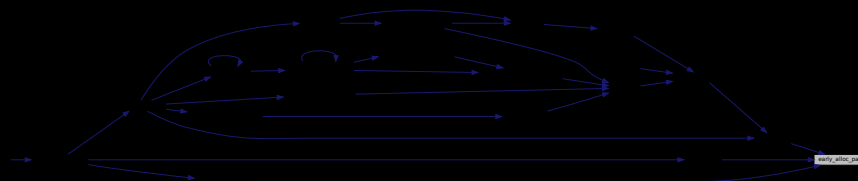
Early allocations are monotonic and are not individually freed. Once the permanent frame allocator is initialized, paging obtains new frames through the Stage-2 frame supplier instead.

Definition at line 119 of file `early_alloc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.143.2.6 `early_init()`

```
void early_init (
    void )
```

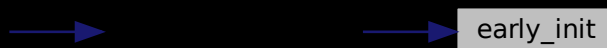
Select a Limine USABLE memory region and initialize the bump allocator.

If `early_set_region()` has already supplied an explicit region, this function preserves that selection rather than choosing another one.

Definition at line 103 of file early_alloc.c.

Here is the call graph for this function:

Here is the caller graph for this function:



7.143.2.7 `early_reserved_region()`

```
void early_reserved_region (
    paddr_t * base,
    size_t * pages )
```

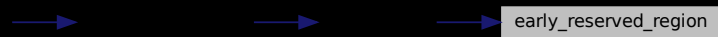
Parameters

<i>base</i>	Receives the physical base of the bootstrap region.
<i>pages</i>	Receives the number of pages consumed from that region.

The permanent PMM uses this information to avoid handing bootstrap-owned memory back out as free frames.

Definition at line 131 of file early_alloc.c.

Here is the caller graph for this function:

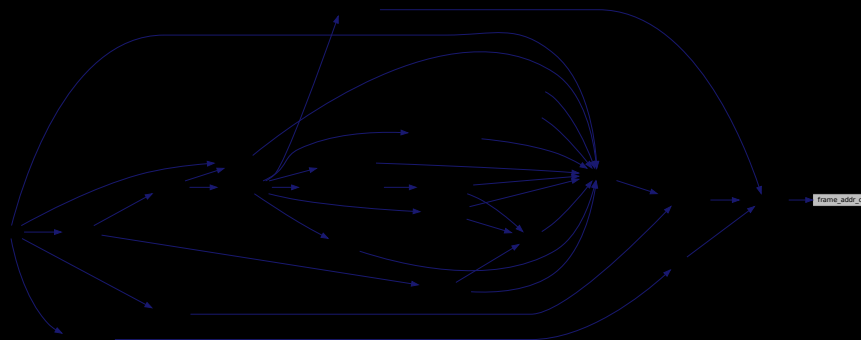


7.143.2.8 frame_addr_of()

```
static paddr_t frame_addr_of (
    size_t i ) [inline], [static]
```

Definition at line 79 of file `neo_framealloc.c`.

Here is the caller graph for this function:



7.143.2.9 frame_alloc()

```
paddr_t frame_alloc (
    void )
```

Allocate one 4 KiB physical frame.

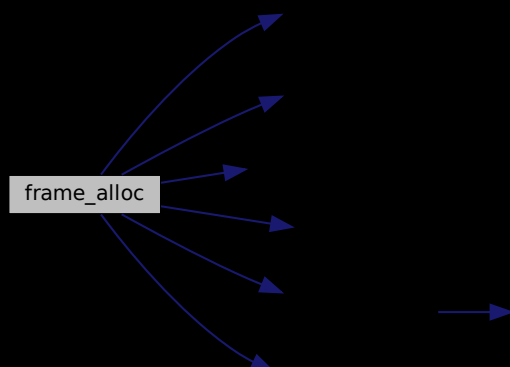
Returns

Physical address of an allocated 4 KiB frame.

The returned value is a physical address, not a C pointer. The allocator panics on exhaustion rather than returning an ordinary heap-style NULL.

Definition at line 215 of file neo_framealloc.c.

Here is the call graph for this function:



Here is the caller graph for this function:

7.143.2.12 frame_init()

```
void frame_init (
                void )
```

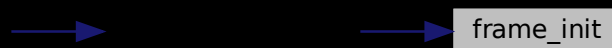
Initialize the permanent physical memory manager from Limine's memmap.

Builds allocator metadata from usable regions and reserves ranges already consumed by the kernel, boot environment, and early allocator. Call after `paging_bootstrap()` while the bootstrap allocator's ownership information is still available.

Definition at line 143 of file neo_framealloc.c.

Here is the call graph for this function:

Here is the caller graph for this function:



7.143.2.13 frame_is_allocated()

```
bool frame_is_allocated (
    paddr_t paddr )
```

Query allocation state for a physical frame.

Parameters

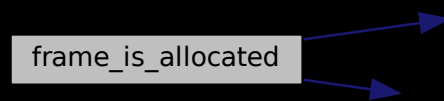
<i>paddr</i>	4 KiB-aligned physical frame address.
--------------	---------------------------------------

Returns

true when the frame is currently marked allocated; false otherwise.

Definition at line 277 of file [neo_framealloc.c](#).

Here is the call graph for this function:



7.143.2.14 `frame_total_count()`

```
size_t frame_total_count (  
    void )
```

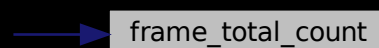
Count physical frames managed by the PMM.

Returns

Number of 4 KiB frames represented by the active allocator.

Definition at line 284 of file [neo_framealloc.c](#).

Here is the caller graph for this function:

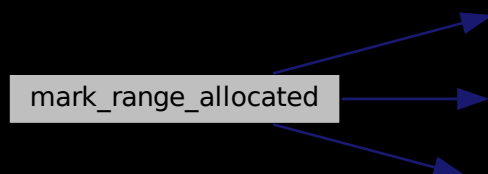


7.143.2.15 mark_range_allocated()

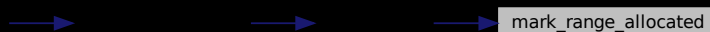
```
static void mark_range_allocated (
    paddr_t base,
    paddr_t end ) [static]
```

Definition at line 101 of file `neo_framealloc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

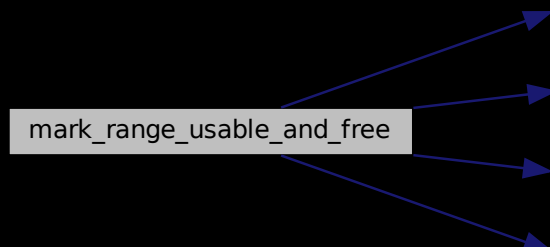


7.143.2.16 mark_range_usable_and_free()

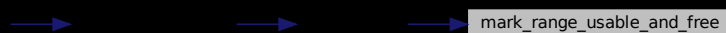
```
static void mark_range_usable_and_free (
    paddr_t base,
    paddr_t end ) [static]
```

Definition at line 120 of file `neo_framealloc.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.143.2.17 `memset8()`

```
static void memset8 (  
    void * p,  
    uint8_t v,  
    size_t n ) [inline], [static]
```

Definition at line 44 of file `neo_framealloc.c`.

Here is the caller graph for this function:



7.143.2.18 memzero()

```
static void memzero (  
    void * p,  
    size_t n ) [inline], [static]
```

Definition at line 38 of file [neo_framealloc.c](#).

Here is the caller graph for this function:



7.143.2.19 n_frame_free()

```
void n_frame_free (  
    paddr\_t paddr )
```

Return a 4 KiB physical frame to the PMM.

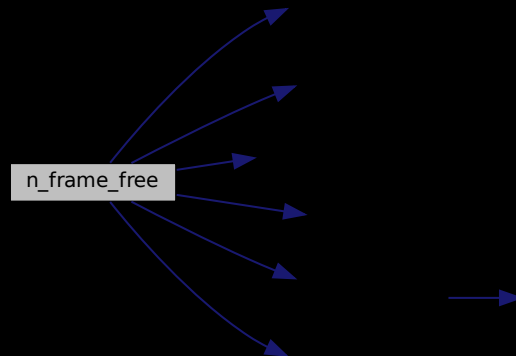
Parameters

<i>paddr</i>	Physical address previously obtained from frame_alloc() .
--------------	---

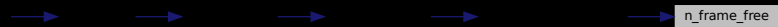
The implementation guards alignment, allocator bounds, non-usable regions, and double-free attempts.

Definition at line 246 of file [neo_framealloc.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.143.2.20 panic()

```
void panic (
    const char * fmt,
    ... )
```

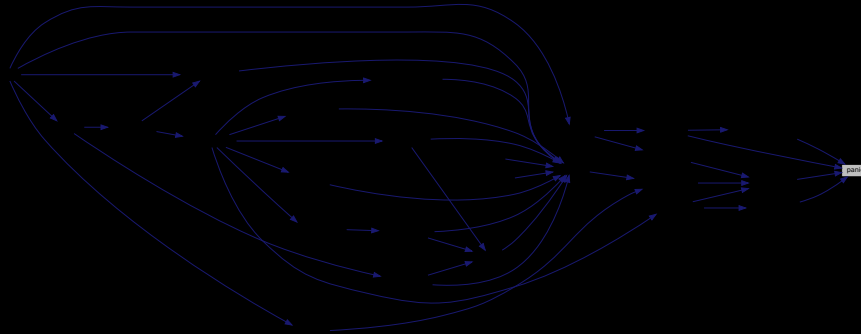
Panic with a nicely formatted message. This is useful for situations where you are certain an unrecoverable error will occur and want to be informative (feel free to abuse sarcasm in them) about the problem.

Parameters

<i>fmt</i>	printf style format string for the panic message
------------	--

Definition at line 17 of file `panic.c`.

Here is the caller graph for this function:



7.143.3 Variable Documentation

7.143.3.1 __kernel_end

```
char __kernel_end[ ]
```

Definition at line 32 of file [neo_framealloc.c](#).

7.143.3.2 __kernel_start

```
char __kernel_start[ ] [extern]
```

7.143.3.3 g_alloc_bm

```
uint8_t* g_alloc_bm = 0 [static]
```

Definition at line 52 of file [neo_framealloc.c](#).

7.143.3.4 g_frame_count

```
size_t g_frame_count = 0 [static]
```

Definition at line 54 of file [neo_framealloc.c](#).

7.143.3.5 g_free_count

```
size_t g_free_count = 0 [static]
```

Definition at line 56 of file `neo_framealloc.c`.

7.143.3.6 g_hint

```
size_t g_hint = 0 [static]
```

Definition at line 57 of file `neo_framealloc.c`.

7.143.3.7 g_max_phys

```
uint64_t g_max_phys = 0 [static]
```

Definition at line 55 of file `neo_framealloc.c`.

7.143.3.8 g_memlock

```
spinlock_t g_memlock = SPINLOCK_INIT [static]
```

Definition at line 58 of file `neo_framealloc.c`.

7.143.3.9 g_usable_bm

```
uint8_t* g_usable_bm = 0 [static]
```

Definition at line 53 of file `neo_framealloc.c`.

7.143.3.10 hhdm_request

```
volatile struct limine_hhdm_request hhdm_request [extern]
```


7.143.3.11 Kaddress_req

```
volatile struct limine_kernel_address_request Kaddress_req [extern]
```

7.143.3.12 memmap_req

```
volatile struct limine_memmap_request memmap_req [extern]
```

VoyagerOS64 - Bitmap frame allocator (Limine, x86_64) Clean rewrite: relocation-safe, guarded, spinlocked.

7.143.3.13 module_request

```
volatile struct limine_module_request module_request [extern]
```

7.144 neo_framealloc.c

[Go to the documentation of this file.](#)

```
00001
00005 #include "include/paging/neo_framealloc.h" // must typedef paddr_t = uint64_t
00006 #include "include/limine.h"
00007 #include "include/lock.h" // SPINLOCK_INIT, spinlock_acquire/release
00008 #include "include/panic.h"
00009 #include <stdbool.h>
00010 #include <stddef.h>
00011 #include <stdint.h>
00012
00013 /* Limine requests are defined ONCE in a separate TU (e.g.,
00014 boot/limine_requests.c). Here we only extern them. */
00015 extern volatile struct limine_memmap_request memmap_req;
00016 extern volatile struct limine_kernel_address_request Kaddress_req;
00017
00018 extern volatile struct limine_hhdm_request hhdm_request;
00019
00020 extern volatile struct limine_module_request module_request;
00021
00022 /* Early allocator API (already in your tree) */
00023 extern void early_init(void);
00024 extern paddr_t early_alloc_page(void);
00025 extern void early_reserved_region(paddr_t *base, size_t *pages);
00026
00027 #define PAGE_SIZE 4096ull
00028 #define ALIGN_UP(x, a) (((x) + ((a) - 1)) & ~((a) - 1))
00029 #define ALIGN_DOWN(x, a) ((x) & ~((a) - 1))
00030
00031 /* Kernel VMA bounds from linker script (virtual addresses) */
00032 extern char __kernel_start[], __kernel_end[];
00033
00034 /* Panic (your existing banner/trace) */
00035 extern void panic(const char *fmt, ...);
00036
00037 /* --- tiny utils --- */
00038 static inline void memzero(void *p, size_t n)
00039 {
00040     volatile uint8_t *d = (volatile uint8_t *) p;
00041     for (size_t i = 0; i < n; i++)
00042         d[i] = 0;
00043 }
00044 static inline void memset8(void *p, uint8_t v, size_t n)
00045 {
00046     volatile uint8_t *d = (volatile uint8_t *) p;
00047     for (size_t i = 0; i < n; i++)
00048         d[i] = v;
00049 }
00050
00051 /* --- globals --- */
00052 static uint8_t *g_alloc_bm = 0; /* 1 = allocated/reserved, 0 = free */
```

```

00053 static uint8_t    *g_usable_bm    = 0; /* 1 = USABLE by memmap, 0 = not usable */
00054 static size_t      g_frame_count  = 0;
00055 static uint64_t     g_max_phys    = 0;
00056 static size_t      g_free_count   = 0;
00057 static size_t      g_hint         = 0;
00058 static spinlock_t   g_memlock     = SPINLOCK_INIT;
00059
00060 /* --- bit ops --- */
00061 static inline void bm_set(uint8_t *bm, size_t i)
00062 {
00063     bm[i >> 3] |= (uint8_t) (1u << (i & 7));
00064 }
00065 static inline void bm_clear(uint8_t *bm, size_t i)
00066 {
00067     bm[i >> 3] &= (uint8_t) ~(1u << (i & 7));
00068 }
00069 static inline bool bm_test(const uint8_t *bm, size_t i)
00070 {
00071     return (bm[i >> 3] >> (i & 7)) & 1u;
00072 }
00073
00074 /* --- helpers --- */
00075 static inline size_t frame_index_of(paddr_t p)
00076 {
00077     return (size_t) (p / PAGE_SIZE);
00078 }
00079 static inline paddr_t frame_addr_of(size_t i)
00080 {
00081     return (paddr_t) (i * PAGE_SIZE);
00082 }
00083
00084 static void compute_max_phys_and_count(void)
00085 {
00086     if (!memmap_req.response || memmap_req.response->entry_count == 0)
00087         panic("frame_init: Limine memmap missing");
00088     g_max_phys = 0;
00089     for (size_t i = 0; i < memmap_req.response->entry_count; i++)
00090     {
00091         const struct limine_memmap_entry *e = memmap_req.response->entries[i];
00092         uint64_t end = e->base + e->length;
00093         if (end > g_max_phys)
00094             g_max_phys = end;
00095     }
00096     g_frame_count = (size_t) (ALIGN_UP(g_max_phys, PAGE_SIZE) / PAGE_SIZE);
00097     if (g_frame_count == 0)
00098         panic("frame_init: zero frames");
00099 }
00100
00101 static void mark_range_allocated(paddr_t base, paddr_t end)
00102 {
00103     if (end <= base)
00104         return;
00105     size_t i0 = frame_index_of(ALIGN_DOWN(base, PAGE_SIZE));
00106     size_t i1 = frame_index_of(ALIGN_UP(end, PAGE_SIZE));
00107     if (i1 > g_frame_count)
00108         i1 = g_frame_count;
00109     for (size_t i = i0; i < i1; i++)
00110     {
00111         if (!bm_test(g_alloc_bm, i))
00112         {
00113             bm_set(g_alloc_bm, i);
00114             if (g_free_count)
00115                 g_free_count--;
00116         }
00117     }
00118 }
00119
00120 static void mark_range_usable_and_free(paddr_t base, paddr_t end)
00121 {
00122     if (end <= base)
00123         return;
00124     size_t i0 = frame_index_of(ALIGN_UP(base, PAGE_SIZE));
00125     size_t i1 = frame_index_of(ALIGN_DOWN(end, PAGE_SIZE));
00126     if (i1 <= i0)
00127         return;
00128     if (i1 > g_frame_count)
00129         i1 = g_frame_count;
00130     for (size_t i = i0; i < i1; i++)
00131     {
00132         if (!bm_test(g_usable_bm, i))
00133             bm_set(g_usable_bm, i);
00134         if (bm_test(g_alloc_bm, i))
00135         {
00136             bm_clear(g_alloc_bm, i);
00137             g_free_count++;
00138         }
00139     }

```

```

00140 }
00141
00142 /* --- public init --- */
00143 void frame_init(void)
00144 {
00145     /* 1) derive topology */
00146     compute_max_phys_and_count();
00147
00148     /* 2) allocate bitmaps from early allocator (separate blocks) */
00149     if (!hhdm_request.response)
00150         panic("frame_init: HHDM request missing");
00151     uint64_t hhdm = hhdm_request.response->offset;
00152
00153     size_t bm_bytes = (g_frame_count + 7) / 8; /* exact bytes needed */
00154     size_t bm_pages = (size_t) ALIGN_UP(bm_bytes, PAGE_SIZE) / PAGE_SIZE;
00155
00156     /* alloc_bm */
00157     paddr_t a_phys = early_alloc_page();
00158     for (size_t p = 1; p < bm_pages; p++)
00159         early_alloc_page();
00160     g_alloc_bm = (uint8_t *) (a_phys + hhdm);
00161
00162     /* usable_bm */
00163     paddr_t b_phys = early_alloc_page();
00164     for (size_t p = 1; p < bm_pages; p++)
00165         early_alloc_page();
00166     g_usable_bm = (uint8_t *) (b_phys + hhdm);
00167
00168     /* 3) init bitmaps: default all reserved, then free-up USABLE ranges */
00169     memset8(g_alloc_bm, 0xFF, bm_bytes);
00170     memzero(g_usable_bm, bm_bytes);
00171     g_free_count = 0;
00172
00173     for (size_t i = 0; i < memmap_req.response->entry_count; i++)
00174     {
00175         const struct limine_memmap_entry *e = memmap_req.response->entries[i];
00176         if (e->type == LIMINE_MEMMAP_USABLE)
00177             mark_range_usable_and_free(e->base, e->base + e->length);
00178     }
00179
00180     /* 4) re-reserve kernel (relocation-safe using Limine bases + VMA symbols) */
00181     if (!Kaddress_req.response)
00182         panic("frame_init: kernel address response missing");
00183     uint64_t kphys_base = Kaddress_req.response->physical_base;
00184     uint64_t kvirt_base = Kaddress_req.response->virtual_base;
00185
00186     uint64_t kphys_begin =
00187         kphys_base + ((uint64_t) __kernel_start - kvirt_base);
00188     uint64_t kphys_end = kphys_base + ((uint64_t) __kernel_end - kvirt_base);
00189     mark_range_allocated((paddr_t) kphys_begin, (paddr_t) kphys_end);
00190
00191     /* 5) reserve modules (physical addresses provided by Limine) */
00192     if (module_request.response)
00193     {
00194         for (size_t i = 0; i < module_request.response->module_count; i++)
00195         {
00196             struct limine_file *m = module_request.response->modules[i];
00197             mark_range_allocated((paddr_t) m->address,
00198                                 (paddr_t) (m->address + m->size));
00199         }
00200     }
00201
00202     /* 6) reserve early allocator consumption */
00203     paddr_t early_base = 0;
00204     size_t early_pages = 0;
00205     early_reserved_region(&early_base, &early_pages);
00206     if (early_pages)
00207         mark_range_allocated(early_base, early_base + early_pages * PAGE_SIZE);
00208
00209     /* ready */
00210     g_hint = 0;
00211 }
00212
00213 /* --- public API --- */
00214 paddr_t frame_alloc(void)
00215 {
00216     spinlock_acquire(&g_memlock);
00217     if (g_free_count == 0)
00218     {
00219         spinlock_release(&g_memlock);
00220         panic("frame_alloc: OOM");
00221     }
00222     size_t start = g_hint;
00223     for (size_t pass = 0; pass < 2; pass++)
00224     {
00225         size_t i = (pass == 0) ? start : 0;

```

```

00227     size_t end = (pass == 0) ? g_frame_count : start;
00228     for (; i < end; i++)
00229     {
00230         if (bm_test(g_usable_bm, i) && !bm_test(g_alloc_bm, i))
00231         {
00232             bm_set(g_alloc_bm, i);
00233             g_free_count--;
00234             g_hint = i + 1;
00235             paddr_t p = frame_addr_of(i);
00236             spinlock_release(&g_memlock);
00237             return p;
00238         }
00239     }
00240 }
00241 spinlock_release(&g_memlock);
00242 panic("frame_alloc: free count nonzero but no free frame found (bitmap "
00243       "corruption?)");
00244 }
00245
00246 void n_frame_free(paddr_t paddr)
00247 {
00248     if (paddr & (PAGE_SIZE - 1))
00249         panic("frame_free: unaligned paddr=0x%llx", (unsigned long long) paddr);
00250     if (paddr >= g_max_phys)
00251         panic("frame_free: out-of-bounds paddr=0x%llx",
00252               (unsigned long long) paddr);
00253
00254     size_t i = frame_index_of(paddr);
00255     spinlock_acquire(&g_memlock);
00256
00257     if (!bm_test(g_usable_bm, i))
00258     {
00259         spinlock_release(&g_memlock);
00260         panic("frame_free: non-usable paddr=0x%llx",
00261               (unsigned long long) paddr);
00262     }
00263     if (!bm_test(g_alloc_bm, i))
00264     {
00265         spinlock_release(&g_memlock);
00266         panic("frame_free: double-free paddr=0x%llx",
00267               (unsigned long long) paddr);
00268     }
00269
00270     bm_clear(g_alloc_bm, i);
00271     g_free_count++;
00272     if (i < g_hint)
00273         g_hint = i;
00274     spinlock_release(&g_memlock);
00275 }
00276
00277 bool frame_is_allocated(paddr_t paddr)
00278 {
00279     if ((paddr & (PAGE_SIZE - 1)) || paddr >= g_max_phys)
00280         return true;
00281     return bm_test(g_alloc_bm, frame_index_of(paddr));
00282 }
00283
00284 size_t frame_total_count(void)
00285 {
00286     return g_frame_count;
00287 }
00288 size_t frame_free_count(void)
00289 {
00290     return g_free_count;
00291 }

```

7.145 src/paging.c File Reference

```

#include "include/paging/paging.h"
#include "include/KernelUtils.h"
#include "include/paging/frame_supplier.h"
#include "include/printf.h"
#include "include/string.h"

```

- struct PageTableOffset VirtualAddressToOffsets (void *virtualAddress)

- ### 7.145.1 Function Documentation

7.145.1.1 DuplicateRecursive()

```
static uint64_t DuplicateRecursive (
    struct PageTable * self,
    uint64_t entry,
    uint64_t level ) [inline], [static]
```

Recursively duplicate a page-table subtree.

Parameters

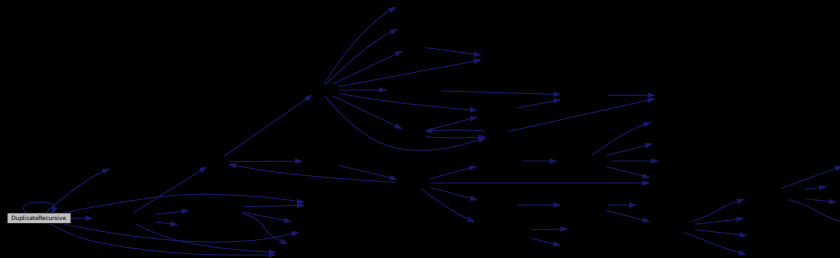
<i>self</i>	Root page table used when mapping newly allocated pages.
<i>entry</i>	Source entry to duplicate.
<i>level</i>	Remaining page-table depth; zero copies the leaf page.

Returns

Newly allocated physical page address combined with source flags.

Definition at line 129 of file [paging.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.145.1.2 GetOrAllocEntry()

```
static struct PageTable * GetOrAllocEntry (
    struct PageTable * table,
    uint64_t offset,
    uint64_t flags ) [inline], [static]
```

Return an intermediate page table, allocating it when absent.

Parameters

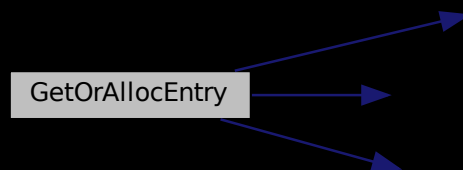
<i>table</i>	Kernel-accessible page table containing the entry.
<i>offset</i>	Entry index to inspect.
<i>flags</i>	Flags to apply when a new intermediate table is created.

Returns

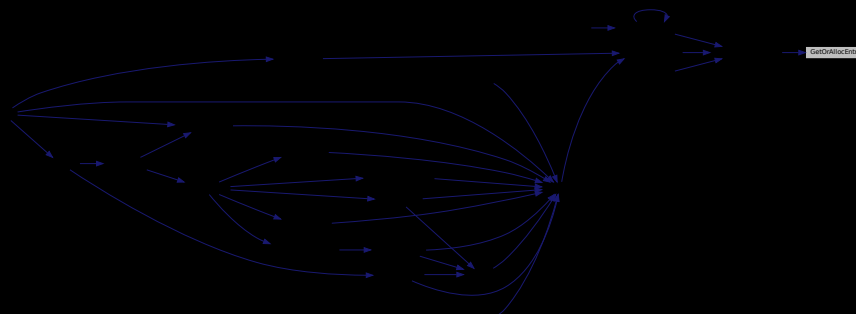
Kernel-accessible pointer to the next-level table, or NULL when frame allocation fails.

Definition at line 78 of file [paging.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.145.1.3 GetOrNullifyEntry()

```
static struct PageTable * GetOrNullifyEntry (
    struct PageTable * table,
    uint64_t offset ) [inline], [static]
```

Return an intermediate page table only when the entry is present.

Parameters

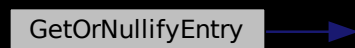
<i>table</i>	Kernel-accessible page table containing the entry.
<i>offset</i>	Entry index to inspect.

Returns

Kernel-accessible pointer to the next-level table, or NULL when the entry is not present.

Definition at line 107 of file [paging.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.145.1.4 OffsetToVirtualAddress()

```
void * OffsetToVirtualAddress (
    struct PageTableOffset offset )
```

Convert four-level page-table indices back to a virtual address.

Parameters

<i>offset</i>	Page-table indices to combine.
---------------	--------------------------------

Returns

Virtual address assembled from the supplied indices.

Definition at line 32 of file [paging.c](#).

7.145.1.5 PagingDuplicate()

```
void PagingDuplicate (
    struct PageTable * p4,
    struct PageTable * newTable )
```

Duplicate the current paging structure into *newTable*.

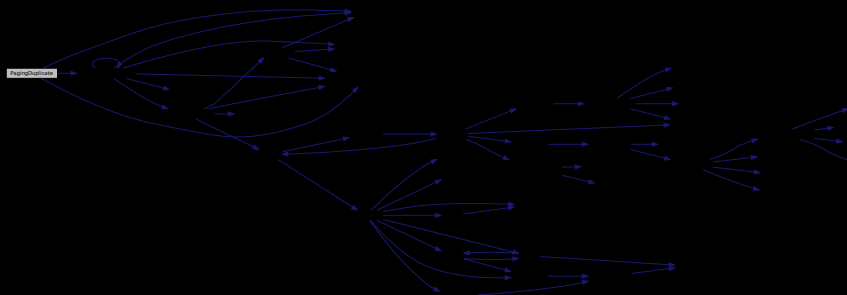
Parameters

<i>p4</i>	Physical address of the source root PML4.
<i>newTable</i>	Destination root table accessible to the kernel.

The current implementation recursively duplicates the lower half and shares the upper-half root entries. Voyager↔ OS64 0.0.5 does not yet expose this as a complete per-process address-space or fork facility.

Definition at line 274 of file [paging.c](#).

Here is the call graph for this function:



7.145.1.6 PagingIdentityMap()

```
void PagingIdentityMap (
    struct PageTable * p4,
```

```
void * virtualMemory,  
uint64_t flags )
```

Identity-map one page so its virtual and physical addresses match.

Parameters

<i>p4</i>	Physical address of the root PML4.
<i>virtualMemory</i>	Address used as both virtual and physical page address.
<i>flags</i>	Leaf-page flags.

Definition at line 163 of file paging.c.

Here is the call graph for this function:

7.145.1.7 PagingMapMemory()

```
void PagingMapMemory (
    struct PageTable * p4,
    void * virtualMemory,
    void * physicalMemory,
    uint64_t flags )
```

Map one 4 KiB virtual page to one physical page.

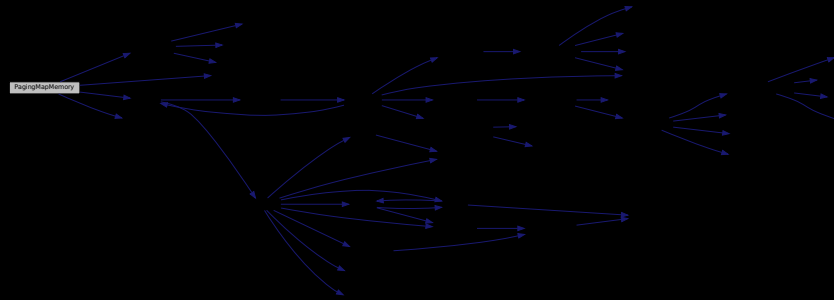
Parameters

<i>p4</i>	Physical address of the root PML4 used by this paging layer.
<i>virtualMemory</i>	Virtual page address to map.
<i>physicalMemory</i>	Physical page address to install in the leaf entry.
<i>flags</i>	Leaf-page flags in addition to the present bit.

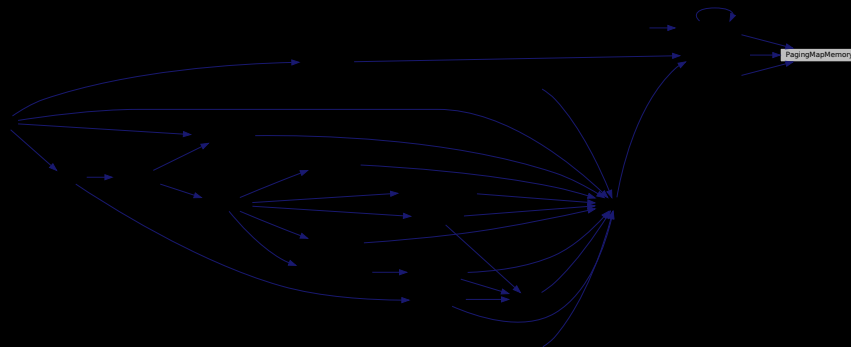
Missing intermediate tables are allocated through `PagingGetFreeFrame()`.

Definition at line 181 of file paging.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.145.1.8 PagingPhysicalMemory()

```
void * PagingPhysicalMemory (
    struct PageTable * p4,
    void * virtualMemory )
```

Resolve the physical page backing a mapped virtual page.

Parameters

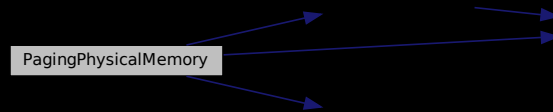
<i>p4</i>	Physical address of the root PML4.
<i>virtualMemory</i>	Virtual address to inspect.

Returns

Physical page address encoded by the leaf entry, or NULL when an intermediate level is not present.

Definition at line 213 of file [paging.c](#).

Here is the call graph for this function:



7.145.1.9 PagingUnmapMemory()

```
void PagingUnmapMemory (
    struct PageTable * p4,
    void * virtualMemory )
```

Clear the leaf mapping for one virtual page.

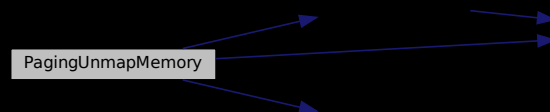
Parameters

<i>p4</i>	Physical address of the root PML4.
<i>virtualMemory</i>	Virtual page whose leaf entry should be cleared.

This does not free intermediate page tables or the previously mapped physical frame.

Definition at line 244 of file [paging.c](#).

Here is the call graph for this function:



7.145.1.10 ReadCR3()

```
uint64_t ReadCR3 (
    void )
```

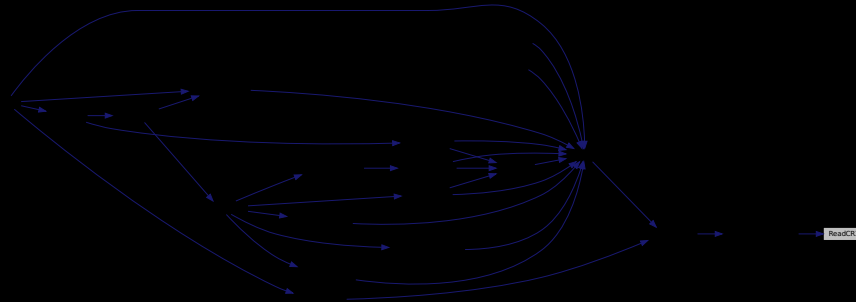
Read the active CR3 value.

Returns

Current CR3 value.

Definition at line 48 of file `paging.c`.

Here is the caller graph for this function:

**7.145.1.11 VirtualAddressToOffsets()**

```
struct PageTableOffset VirtualAddressToOffsets (
    void * virtualAddress )
```

Convert a virtual address into four-level page-table indices.

Parameters

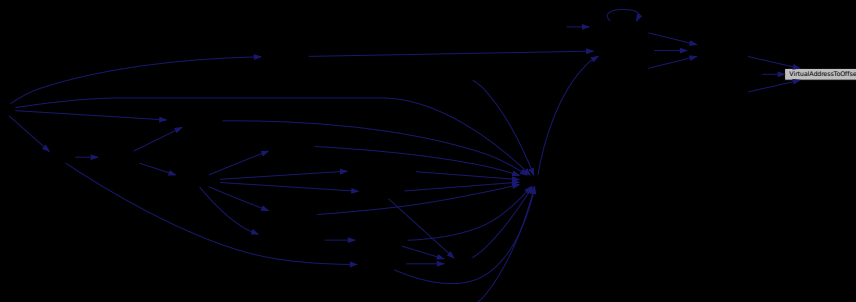
<code>virtualAddress</code>	Virtual address to decompose.
-----------------------------	-------------------------------

Returns

PML4, PDPT, PD, and PT indices for the address.

Definition at line 12 of file `paging.c`.

Here is the caller graph for this function:



7.145.1.12 WriteCR3()

```
void WriteCR3 (
    uint64_t value )
```

Write a 64-bit value to CR3.

Parameters

<i>value</i>	Value to load into CR3.
--------------	-------------------------

Definition at line 62 of file [paging.c](#).

7.146 paging.c

[Go to the documentation of this file.](#)

```
00001 #include "include/paging/paging.h"
00002 #include "include/KernelUtils.h"
00003 #include "include/paging/frame_supplier.h"
00004 #include "include/printf.h"
00005 #include "include/string.h"
00006
00012 struct PageTableOffset VirtualAddressToOffsets(void *virtualAddress)
00013 {
00014     uint64_t address = (uint64_t) virtualAddress;
00015
00016     struct PageTableOffset offset = {
00017
00018         .p4Offset = (address & ((uint64_t) 0x1FF << 39)) >> 39,
00019         .pdpOffset = (address & ((uint64_t) 0x1FF << 30)) >> 30,
00020         .pdOffset = (address & ((uint64_t) 0x1FF << 21)) >> 21,
00021         .ptOffset = (address & ((uint64_t) 0x1FF << 12)) >> 12,
00022     };
00023
00024     return offset;
00025 }
00026
00032 void *OffsetToVirtualAddress(struct PageTableOffset offset)
00033 {
00034     uint64_t address = 0;
00035
00036     address |= offset.p4Offset << 39;
00037     address |= offset.pdpOffset << 30;
00038     address |= offset.pdOffset << 21;
00039     address |= offset.ptOffset << 12;
00040
00041     return (void *) address;
00042 }
00043
00048 uint64_t ReadCR3()
00049 {
00050     uint64_t outValue = 0;
00051
00052     asm("mov %%cr3, %0" : "=r"(outValue) :);
00053
00054     return outValue;
00055     // no input
00056 }
00057
00062 void WriteCR3(uint64_t value)
00063 {
00064     asm("mov %0, %%cr3"
00065         : // no output
00066         : "r"(value));
00067 }
00068
00077 static inline struct PageTable *
00078 GetOrAllocEntry(struct PageTable *table, uint64_t offset, uint64_t flags)
```

```

00079 {
00080     uint64_t address = table->entries[offset];
00081
00082     if (!(address & PAGING_FLAG_PRESENT))
00083     {
00084         address = table->entries[offset] = PagingGetFreeFrame();
00085
00086         if (!address)
00087         {
00088             return NULL;
00089         }
00090
00091         table->entries[offset] |= flags | PAGING_FLAG_PRESENT;
00092
00093         memset((void *) TranslateToHighHalfMemoryAddress(address), 0, 0x1000);
00094     }
00095
00096     return (struct PageTable *) TranslateToHighHalfMemoryAddress(
00097         address & PAGE_ADDRESS_MASK);
00098 }
00099
00107 static inline struct PageTable *GetOrNullifyEntry(struct PageTable *table,
00108                                                    uint64_t offset)
00109 {
00110     uint64_t address = table->entries[offset];
00111
00112     if (!(address & PAGING_FLAG_PRESENT))
00113     {
00114         return NULL;
00115     }
00116
00117     return (struct PageTable *) TranslateToHighHalfMemoryAddress(
00118         address & PAGE_ADDRESS_MASK);
00119 }
00120
00128 static inline uint64_t
00129 DuplicateRecursive(struct PageTable *self, uint64_t entry, uint64_t level)
00130 {
00131     const uint64_t flags = PAGING_FLAG_PRESENT | PAGING_FLAG_USER_ACCESSIBLE |
00132                           PAGING_FLAG_WRITABLE;
00133
00134     uint64_t *virt = (uint64_t *) TranslateToHighHalfMemoryAddress(
00135         (entry & ~PAGE_FLAG_MASK));
00136     uint64_t newPage = PagingGetFreeFrame();
00137     uint64_t *newVirtual =
00138         (uint64_t *) TranslateToHighHalfMemoryAddress(newPage);
00139
00140     PagingMapMemory(self, newVirtual, (void *) newPage, flags);
00141
00142     memset(newVirtual, 0, 0x1000);
00143
00144     if (level == 0)
00145     {
00146         memcpy(newVirtual, (void *) virt, 0x1000);
00147     }
00148     else
00149     {
00150         for (uint64_t i = 0; i < 512; i++)
00151         {
00152             if (virt[i] & PAGING_FLAG_PRESENT)
00153             {
00154                 newVirtual[i] = DuplicateRecursive(self, virt[i], level - 1);
00155             }
00156         }
00157     }
00158
00159     return newPage | (entry & PAGE_FLAG_MASK);
00160 }
00161
00162 /* Public mapping API documentation lives in include/paging/paging.h. */
00163 void PagingIdentityMap(struct PageTable *p4,
00164                      void *virtualMemory,
00165                      uint64_t flags)
00166 {
00167     if (k_mode.addr_debug == 2)
00168     {
00169         printf_("%s\n", "-----");
00170         printf_("%s\n", "DEBUG: Identity Map Adresse (Direct from args)");
00171         printf_("%s\n", "-----");
00172     }
00173     PagingMapMemory(p4, virtualMemory, virtualMemory, flags);
00174     if (k_mode.addr_debug == 2)
00175     {
00176         printf_("%s\n", "-----");
00177         printf_("%s\n", "-----");
00178     }
00179 }

```



```

00180
00181 void PagingMapMemory(struct PageTable *p4,
00182                     void *virtualMemory,
00183                     void *physicalMemory,
00184                     uint64_t flags)
00185 {
00186     if (k_mode.addr_debug == 2)
00187     {
00188         printf_("%s", "Virtual: ");
00189         printf_("0x%llx\n", virtualMemory);
00190         printf_("%s", "Physical: ");
00191         printf_("0x%llx\n", physicalMemory);
00192     }
00193
00194     uint64_t higherPermissions =
00195         PAGING_FLAG_WRITABLE | PAGING_FLAG_USER_ACCESSIBLE;
00196
00197     struct PageTableOffset offset = VirtualAddressToOffsets(virtualMemory);
00198
00199     struct PageTable *p4Virtual =
00200         (struct PageTable *) TranslateToHighHalfMemoryAddress((uint64_t) p4);
00201
00202     struct PageTable *pdp =
00203         GetOrAllocEntry(p4Virtual, offset.p4Offset, higherPermissions);
00204     struct PageTable *pd =
00205         GetOrAllocEntry(pdp, offset.pdpOffset, higherPermissions);
00206     struct PageTable *pt =
00207         GetOrAllocEntry(pd, offset.pdOffset, higherPermissions);
00208
00209     pt->entries[offset.ptOffset] =
00210         (uint64_t) physicalMemory | flags | PAGING_FLAG_PRESENT;
00211 }
00212
00213 void *PagingPhysicalMemory(struct PageTable *p4, void *virtualMemory)
00214 {
00215     struct PageTableOffset offset = VirtualAddressToOffsets(virtualMemory);
00216
00217     struct PageTable *p4Virtual =
00218         (struct PageTable *) TranslateToHighHalfMemoryAddress((uint64_t) p4);
00219
00220     struct PageTable *pdp = GetOrNullifyEntry(p4Virtual, offset.p4Offset);
00221
00222     if (pdp == NULL)
00223     {
00224         return NULL;
00225     }
00226
00227     struct PageTable *pd = GetOrNullifyEntry(pdp, offset.pdpOffset);
00228
00229     if (pd == NULL)
00230     {
00231         return NULL;
00232     }
00233
00234     struct PageTable *pt = GetOrNullifyEntry(pd, offset.pdOffset);
00235
00236     if (pt == NULL)
00237     {
00238         return NULL;
00239     }
00240
00241     return (void *) (pt->entries[offset.ptOffset] & ~(PAGE_FLAG_MASK));
00242 }
00243
00244 void PagingUnmapMemory(struct PageTable *p4, void *virtualMemory)
00245 {
00246     struct PageTableOffset offset = VirtualAddressToOffsets(virtualMemory);
00247
00248     struct PageTable *p4Virtual =
00249         (struct PageTable *) TranslateToHighHalfMemoryAddress((uint64_t) p4);
00250     struct PageTable *pdp = GetOrNullifyEntry(p4Virtual, offset.p4Offset);
00251
00252     if (pdp == NULL)
00253     {
00254         return;
00255     }
00256
00257     struct PageTable *pd = GetOrNullifyEntry(pdp, offset.pdpOffset);
00258
00259     if (pd == NULL)
00260     {
00261         return;
00262     }
00263
00264     struct PageTable *pt = GetOrNullifyEntry(pd, offset.pdOffset);
00265
00266     if (pt == NULL)

```


- `#define PTE_W` (1ull << 1)
- `#define PTE_U` (1ull << 2)
- `#define PTE_PS` (1ull << 7)
- `#define PTE_NX` (1ull << 63)
- `#define PAGE_SIZE` 4096ull
- `#define PAGE_2M_SIZE` (2ull * 1024 * 1024)
- `#define PAGE_1G_SIZE` (1ull << 30)
- `#define ADDR_MASK` 0x000FFFFFFFFF000ull
- `#define ADDR_MASK_2M` 0x000FFFFFFFFFE000ull
- `#define ADDR_MASK_1G` 0x000FFFFFC0000000ull
- `#define ALIGN_UP(x, a)` (((x) + ((a) - 1)) & ~((a) - 1))
- `#define ALIGN_DOWN(x, a)` ((x) & ~((a) - 1))

Functions

- void `panic` (const char *fmt,...)
Panic with a nicely formatted message. This is useful for situations where you are certain an unrecoverable error will occur and want to be informative (feel free to abuse sarcasm in them) about the problem.
- uint64_t `readCR3` (void)
- void `writeCR3` (uint64_t arg)
- uint64_t `readRSP` (void)
- struct `__attribute__` ((aligned(0x1000)))
- uint64_t `paging_pml4_phys` (void)
Return the physical address loaded as the bootstrap PML4 root.
- void * `paging_pml4_virt` (void)
Return a kernel-accessible virtual pointer to the bootstrap PML4.
- uint64_t `get_total_usable_mem_bytes` (void)
- static uint64_t `get_max_phys` (void)
- static void `memzero` (void *p, size_t n)
- static void * `phys_to_virt` (uint64_t phys)
- static void `vaddr_indices` (uint64_t v, uint64_t idx[4])
- static uint64_t `virt_to_phys_kernel` (uint64_t vaddr, uint64_t kphys_base, uint64_t kvirt_base)
- static struct page_table * `alloc_table` (uint64_t *out_phys)
- static struct page_table * `get_or_make_next` (struct page_table *cur, uint64_t idx, uint64_t *phys_out)
- static struct page_table * `get_or_make_pd` (struct page_table *pml4, uint64_t pml4i, uint64_t pdpti, uint64_t *pd_phys_out)
- static void `map_4k` (struct page_table *pml4, uint64_t va, uint64_t pa, uint64_t flags)
- static void `map_2m` (struct page_table *pml4, uint64_t va, uint64_t pa, uint64_t flags)
- static void `map_1g` (struct page_table *pml4, uint64_t va, uint64_t pa, uint64_t flags)
- static void `map_range_4k` (struct page_table *pml4, uint64_t va, uint64_t pa, uint64_t len, uint64_t flags)
- static void `map_range_huge_first` (struct page_table *pml4, uint64_t va, uint64_t pa, uint64_t len, uint64_t base_flags)
- static void `map_identity_minimal` (struct page_table *pml4)
- static void `map_kernel_higher_half` (struct page_table *pml4)
- static void `map_hhdm` (struct page_table *pml4)
- static bool `walk_va_to_pa` (uint64_t va, struct page_table *pml4_root, uint64_t *pa_out, int *level_out)
- static void `probe_va` (const char *label, uint64_t va, struct page_table *tbl)
- static void `probe_hhdm_phys` (const char *label, uint64_t phys, struct page_table *pml4)
- static void `scan_hhdm_range` (uint64_t phys_start, uint64_t length, uint64_t stride, struct page_table *pml4)
- static void `ensure_hhdm_cover_phys_page` (struct page_table *pml4, uint64_t phys_page)
- static void `ensure_hhdm_cover_phys_range` (struct page_table *pml4, uint64_t phys, uint64_t len)
- static void `pre_switch_audit_and_patch` (struct page_table *new_pml4, uint64_t new_pml4_phys)
- static void `install_new_cr3` (uint64_t new_pml4_phys, struct page_table *new_pml4)
- void `paging_bootstrap` (void)
Build VoyagerOS64's initial kernel page-table hierarchy and install it in CR3.

Variables

- volatile struct `limine_memmap_request` `memmap_req`
- volatile struct `limine_hhdm_request` `hhdm_request`
- volatile struct `limine_kernel_address_request` `Kaddress_req`
- char `__kernel_start` []
- char `__kernel_end` []
- char `__text_start` []
- char `__text_end` []
- char `__rodata_start` []
- char `__rodata_end` []
- char `__data_start` []
- char `__bss_end` []
- uint64_t `g_alloc_bm_phys_base` = 0
- uint64_t `g_alloc_bm_bytes` = 0
- uint64_t `g_usable_bm_phys_base` = 0
- uint64_t `g_usable_bm_bytes` = 0
- uint64_t `framebuffer_phys` = 0
- uint64_t `framebuffer_bytes` = 0
- static uint64_t `g_pml4_phys` = 0
- static struct `page_table` * `g_pml4_virt` = 0
- static uint64_t `g_hhdm` = 0

7.147.1 Macro Definition Documentation

7.147.1.1 ADDR_MASK

```
#define ADDR_MASK 0x000FFFFFFFFFFFF000ull
```

Definition at line 55 of file `paging_bootstrap.c`.

7.147.1.2 ADDR_MASK_1G

```
#define ADDR_MASK_1G 0x000FFFFFC0000000ull
```

Definition at line 57 of file `paging_bootstrap.c`.

7.147.1.3 ADDR_MASK_2M

```
#define ADDR_MASK_2M 0x000FFFFFFFFFFE000ull
```

Definition at line 56 of file `paging_bootstrap.c`.

7.147.1.4 ALIGN_DOWN

```
#define ALIGN_DOWN(  
    x,  
    a ) ((x) & ~((a) - 1))
```

Definition at line 60 of file [paging_bootstrap.c](#).

7.147.1.5 ALIGN_UP

```
#define ALIGN_UP(  
    x,  
    a ) (((x) + ((a) - 1)) & ~((a) - 1))
```

Definition at line 59 of file [paging_bootstrap.c](#).

7.147.1.6 PAGE_1G_SIZE

```
#define PAGE_1G_SIZE (1ull << 30)
```

Definition at line 53 of file [paging_bootstrap.c](#).

7.147.1.7 PAGE_2M_SIZE

```
#define PAGE_2M_SIZE (2ull * 1024 * 1024)
```

Definition at line 52 of file [paging_bootstrap.c](#).

7.147.1.8 PAGE_SIZE

```
#define PAGE_SIZE 4096ull
```

Definition at line 51 of file [paging_bootstrap.c](#).

7.147.1.9 PTE_NX

```
#define PTE_NX (1ull << 63)
```

Definition at line 49 of file [paging_bootstrap.c](#).

7.147.1.10 PTE_P

```
#define PTE_P (1ull << 0)
```

Definition at line 45 of file [paging_bootstrap.c](#).

7.147.1.11 PTE_PS

```
#define PTE_PS (1ull << 7)
```

Definition at line 48 of file [paging_bootstrap.c](#).

7.147.1.12 PTE_U

```
#define PTE_U (1ull << 2)
```

Definition at line 47 of file [paging_bootstrap.c](#).

7.147.1.13 PTE_W

```
#define PTE_W (1ull << 1)
```

Definition at line 46 of file [paging_bootstrap.c](#).

7.147.2 Function Documentation

7.147.2.1 __attribute__()

```
struct __attribute__ (  
    (aligned(0x1000)) )
```

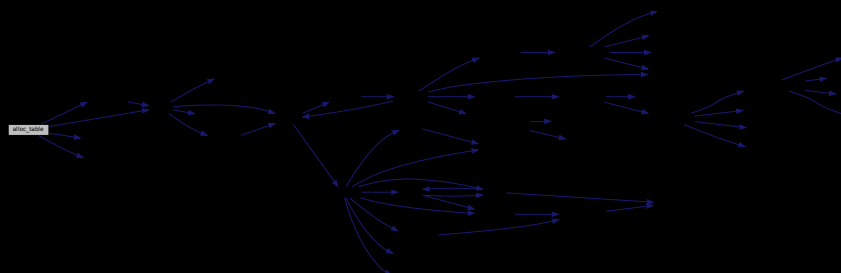
Definition at line 40 of file [paging_bootstrap.c](#).

7.147.2.2 alloc_table()

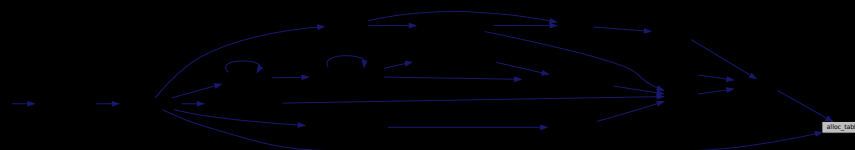
```
static struct page_table * alloc_table (
    uint64_t * out_phys ) [static]
```

Definition at line 143 of file [paging_bootstrap.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

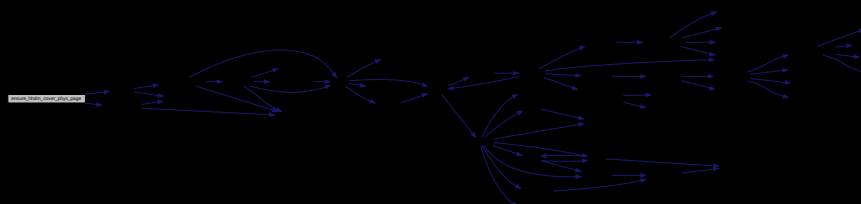


7.147.2.3 ensure_hhdm_cover_phys_page()

```
static void ensure_hhdm_cover_phys_page (
    struct page_table * pml4,
    uint64_t phys_page ) [static]
```

Definition at line 526 of file [paging_bootstrap.c](#).

Here is the call graph for this function:

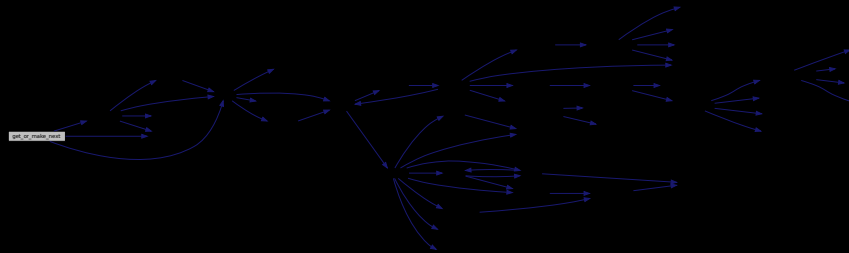


7.147.2.6 `get_or_make_next()`

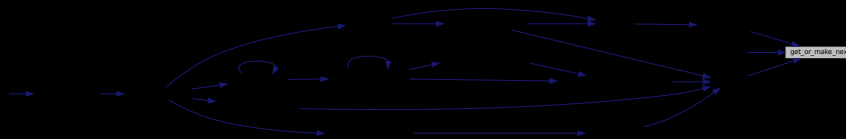
```
static struct page_table * get_or_make_next (
    struct page_table * cur,
    uint64_t idx,
    uint64_t * phys_out ) [static]
```

Definition at line 163 of file `paging_bootstrap.c`.

Here is the call graph for this function:



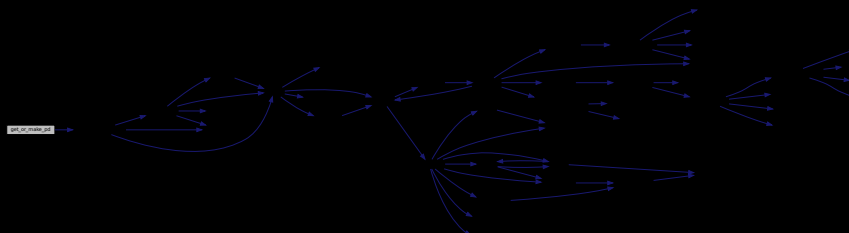
Here is the caller graph for this function:

7.147.2.7 `get_or_make_pd()`

```
static struct page_table * get_or_make_pd (
    struct page_table * pml4,
    uint64_t pml4i,
    uint64_t pdpti,
    uint64_t * pd_phys_out ) [static]
```

Definition at line 188 of file `paging_bootstrap.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

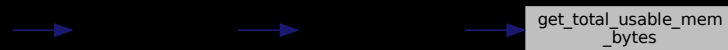


7.147.2.8 get_total_usable_mem_bytes()

```
uint64_t get_total_usable_mem_bytes (
    void )
```

Definition at line 81 of file [paging_bootstrap.c](#).

Here is the caller graph for this function:

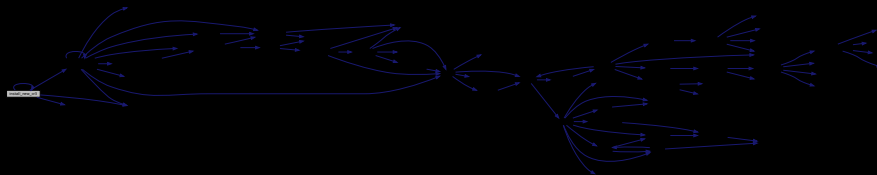


7.147.2.9 install_new_cr3()

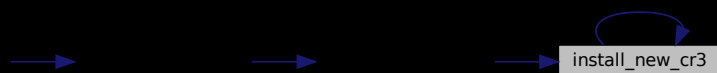
```
static void install_new_cr3 (
    uint64_t new_pml4_phys,
    struct page_table * new_pml4 ) [static]
```

Definition at line 609 of file [paging_bootstrap.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

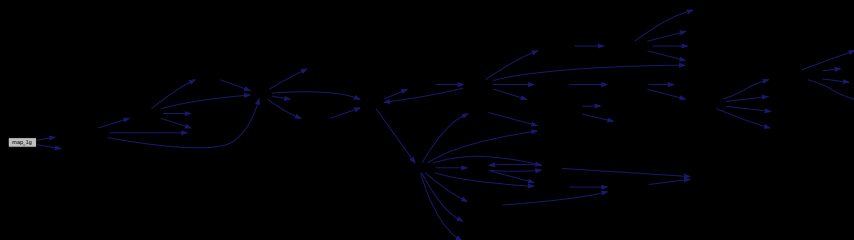


7.147.2.10 map_1g()

```
static void map_1g (
    struct page_table * pml4,
    uint64_t va,
    uint64_t pa,
    uint64_t flags ) [static]
```

Definition at line 228 of file [paging_bootstrap.c](#).

Here is the call graph for this function:

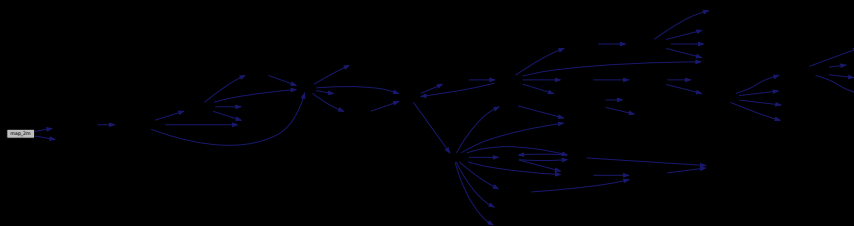


7.147.2.11 map_2m()

```
static void map_2m (
    struct page_table * pml4,
    uint64_t va,
    uint64_t pa,
    uint64_t flags ) [static]
```

Definition at line 216 of file [paging_bootstrap.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

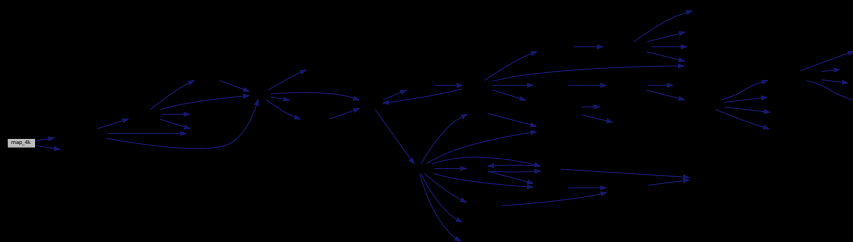


7.147.2.12 map_4k()

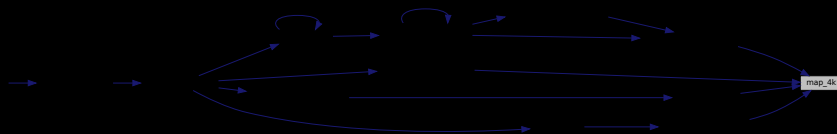
```
static void map_4k (
    struct page_table * pml4,
    uint64_t va,
    uint64_t pa,
    uint64_t flags ) [static]
```

Definition at line 202 of file [paging_bootstrap.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

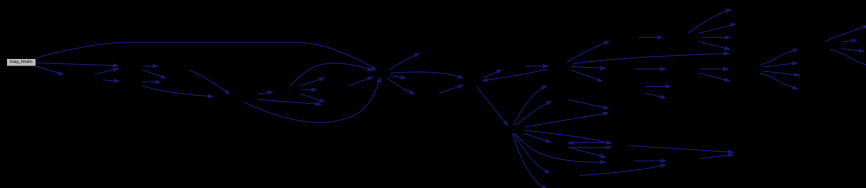


7.147.2.13 map_hhdm()

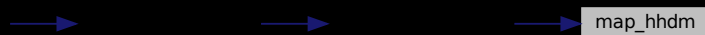
```
static void map_hhdm (
    struct page_table * pml4 ) [static]
```

Definition at line 327 of file [paging_bootstrap.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

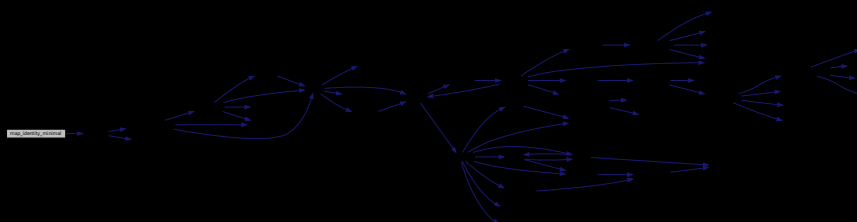


7.147.2.14 map_identity_minimal()

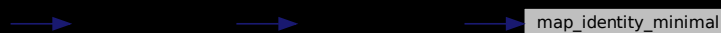
```
static void map_identity_minimal (
    struct page_table * pml4 ) [static]
```

Definition at line 284 of file [paging_bootstrap.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

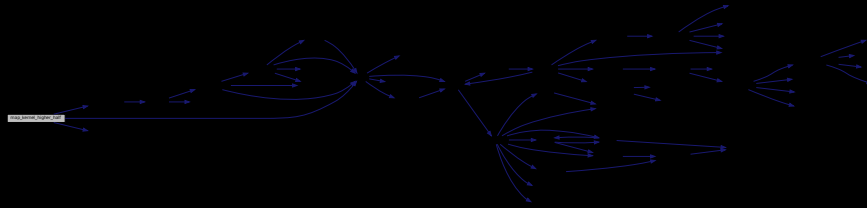


7.147.2.15 map_kernel_higher_half()

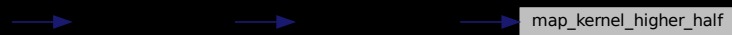
```
static void map_kernel_higher_half (
    struct page_table * pml4 ) [static]
```

Definition at line 290 of file [paging_bootstrap.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

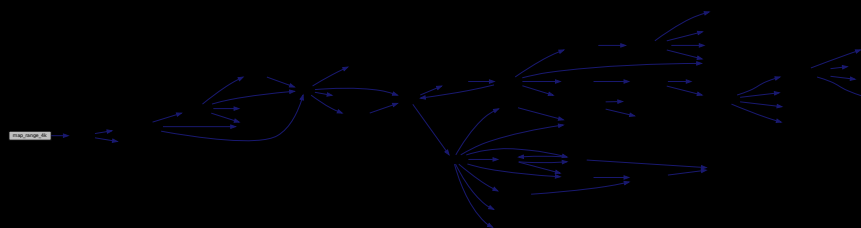


7.147.2.16 `map_range_4k()`

```
static void map_range_4k (
    struct page_table * pml4,
    uint64_t va,
    uint64_t pa,
    uint64_t len,
    uint64_t flags ) [static]
```

Definition at line 239 of file [paging_bootstrap.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

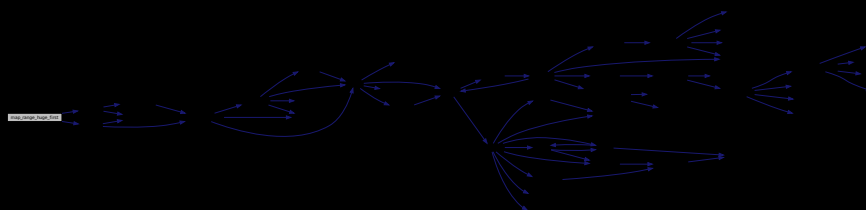


7.147.2.17 map_range_huge_first()

```
static void map_range_huge_first (
    struct page_table * pml4,
    uint64_t va,
    uint64_t pa,
    uint64_t len,
    uint64_t base_flags ) [static]
```

Definition at line 251 of file [paging_bootstrap.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

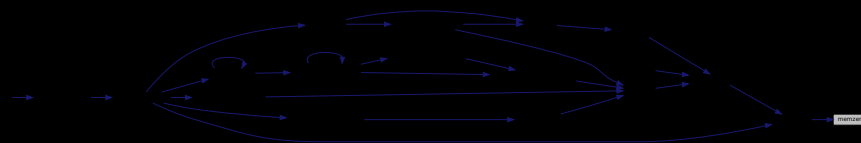


7.147.2.18 memzero()

```
static void memzero (
    void * p,
    size_t n ) [inline], [static]
```

Definition at line 114 of file [paging_bootstrap.c](#).

Here is the caller graph for this function:



7.147.2.19 paging_bootstrap()

```
void paging_bootstrap (
    void )
```

Build VoyagerOS64's initial kernel page-table hierarchy and install it in CR3.

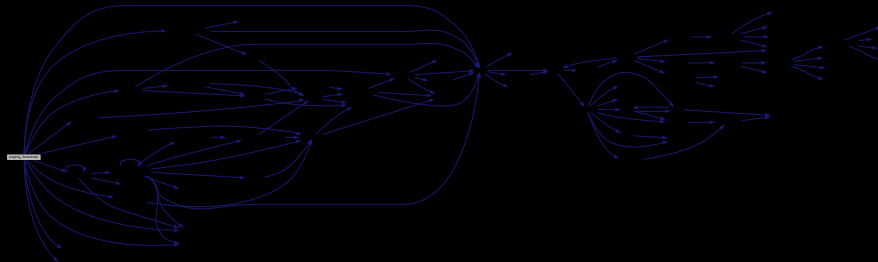
Page-table pages are supplied from `early_alloc_page()` during bootstrap. The resulting mappings preserve the kernel image, active stack, HHDM, and other ranges required for the post-Limine kernel to continue executing.

Warning

After the new CR3 is installed, code must not assume Limine-owned callback code remains mapped or callable. Voyager's framebuffer/serial facilities own output after the handoff.

Definition at line 636 of file `paging_bootstrap.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.147.2.20 paging_pml4_phys()

```
uint64_t paging_pml4_phys (
    void )
```

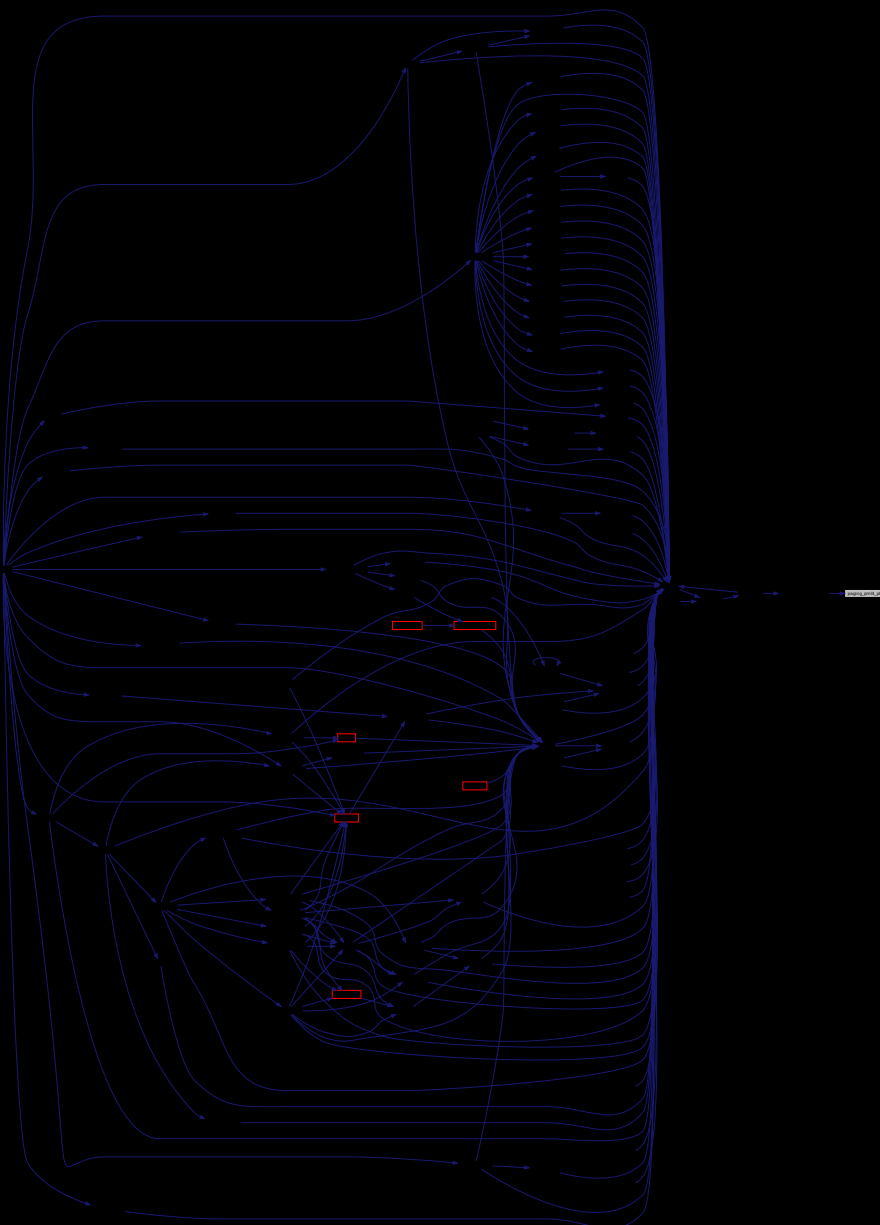
Return the physical address loaded as the bootstrap PML4 root.

Returns

Physical address of Voyager's root page table.

Definition at line 71 of file `paging_bootstrap.c`.

Here is the caller graph for this function:

**7.147.2.21 paging_pml4_virt()**

```
void * paging_pml4_virt (  
    void )
```

Return a kernel-accessible virtual pointer to the bootstrap PML4.

Returns

Virtual address through which the root page table can be accessed.

Definition at line 76 of file [paging_bootstrap.c](#).

7.147.2.22 `panic()`

```
void panic (
    const char * fmt,
    ... )
```

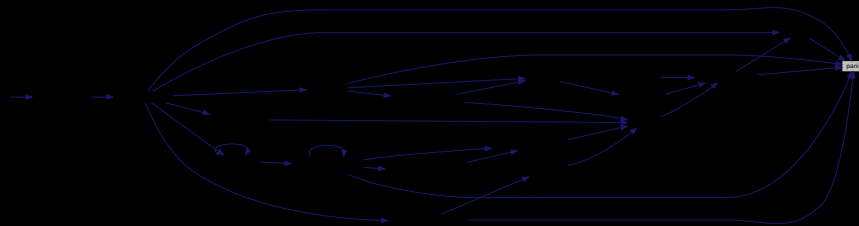
Panic with a nicely formatted message. This is useful for situations where you are certain an unrecoverable error will occur and want to be informative (feel free to abuse sarcasm in them) about the problem.

Parameters

<i>fmt</i>	printf style format string for the panic message
------------	--

Definition at line 17 of file [panic.c](#).

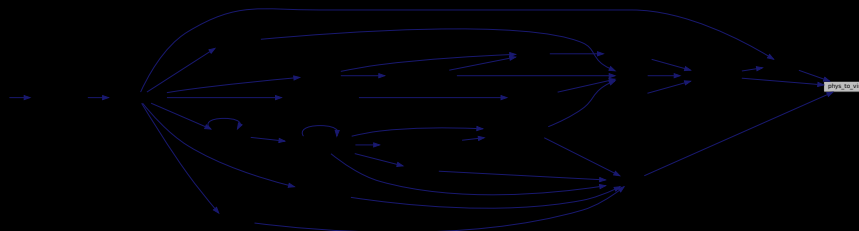
Here is the caller graph for this function:

7.147.2.23 `phys_to_virt()`

```
static void * phys_to_virt (
    uint64_t phys ) [inline], [static]
```

Definition at line 121 of file [paging_bootstrap.c](#).

Here is the caller graph for this function:

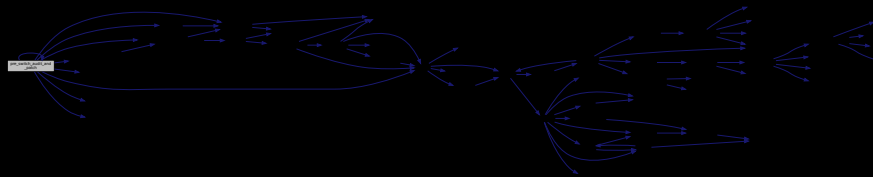


7.147.2.24 pre_switch_audit_and_patch()

```
static void pre_switch_audit_and_patch (
    struct page_table * new_pml4,
    uint64_t new_pml4_phys ) [static]
```

Definition at line 557 of file [paging_bootstrap.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.147.2.25 probe_hhdm_phys()

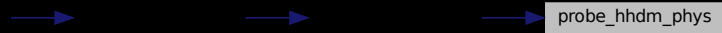
```
static void probe_hhdm_phys (
    const char * label,
    uint64_t phys,
    struct page_table * pml4 ) [static]
```

Definition at line 459 of file [paging_bootstrap.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.147.2.26 probe_va()

```
static void probe_va (
    const char * label,
    uint64_t va,
    struct page_table * tbl ) [static]
```

Definition at line 439 of file [paging_bootstrap.c](#).

Here is the call graph for this function:



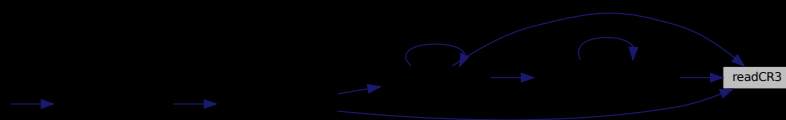
Here is the caller graph for this function:



7.147.2.27 readCR3()

```
uint64_t readCR3 (
    void )
```

Here is the caller graph for this function:



7.147.2.28 readRSP()

```
uint64_t readRSP (
    void )
```

Here is the caller graph for this function:



7.147.2.29 scan_hhdm_range()

```
static void scan_hhdm_range (
    uint64_t phys_start,
    uint64_t length,
    uint64_t stride,
    struct page_table * pml4 ) [static]
```

Definition at line 484 of file [paging_bootstrap.c](#).

Here is the call graph for this function:

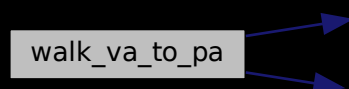


7.147.2.32 walk_va_to_pa()

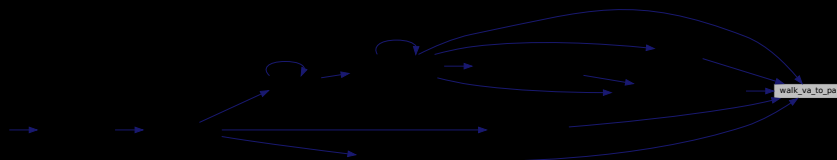
```
static bool walk_va_to_pa (
    uint64_t va,
    struct page_table * pml4_root,
    uint64_t * pa_out,
    int * level_out ) [static]
```

Definition at line 383 of file `paging_bootstrap.c`.

Here is the call graph for this function:



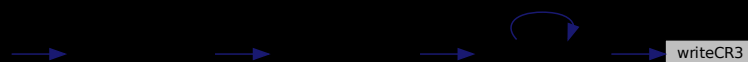
Here is the caller graph for this function:



7.147.2.33 writeCR3()

```
void writeCR3 (
    uint64_t arg )
```

Here is the caller graph for this function:



7.147.3 Variable Documentation

7.147.3.1 `__bss_end`

```
char __bss_end[ ]
```

Definition at line 26 of file [paging_bootstrap.c](#).

7.147.3.2 `__data_start`

```
char __data_start[ ] [extern]
```

7.147.3.3 `__kernel_end`

```
char __kernel_end[ ]
```

Definition at line 23 of file [paging_bootstrap.c](#).

7.147.3.4 `__kernel_start`

```
char __kernel_start[ ] [extern]
```

7.147.3.5 `__rodata_end`

```
char __rodata_end[ ]
```

Definition at line 25 of file [paging_bootstrap.c](#).

7.147.3.6 `__rodata_start`

```
char __rodata_start[ ] [extern]
```


7.147.3.7 __text_end

```
char __text_end[ ]
```

Definition at line 24 of file [paging_bootstrap.c](#).

7.147.3.8 __text_start

```
char __text_start[ ] [extern]
```

7.147.3.9 framebuffer_bytes

```
uint64_t framebuffer_bytes = 0
```

Definition at line 40 of file [paging_bootstrap.c](#).

7.147.3.10 framebuffer_phys

```
uint64_t framebuffer_phys = 0
```

Definition at line 39 of file [paging_bootstrap.c](#).

7.147.3.11 g_alloc_bm_bytes

```
uint64_t g_alloc_bm_bytes = 0
```

Definition at line 36 of file [paging_bootstrap.c](#).

7.147.3.12 g_alloc_bm_phys_base

```
uint64_t g_alloc_bm_phys_base = 0
```

Definition at line 35 of file [paging_bootstrap.c](#).

7.147.3.13 g_hhdm

```
uint64_t g_hhdm = 0 [static]
```

Definition at line 69 of file [paging_bootstrap.c](#).

7.147.3.14 g_pml4_phys

```
uint64_t g_pml4_phys = 0 [static]
```

Definition at line 67 of file [paging_bootstrap.c](#).

7.147.3.15 g_pml4_virt

```
struct page_table* g_pml4_virt = 0 [static]
```

Definition at line 68 of file [paging_bootstrap.c](#).

7.147.3.16 g_usable_bm_bytes

```
uint64_t g_usable_bm_bytes = 0
```

Definition at line 38 of file [paging_bootstrap.c](#).

7.147.3.17 g_usable_bm_phys_base

```
uint64_t g_usable_bm_phys_base = 0
```

Definition at line 37 of file [paging_bootstrap.c](#).

7.147.3.18 hhdm_request

```
volatile struct limine\_hhdm\_request hhdm_request [extern]
```

7.147.3.19 Kaddress_req

```
volatile struct limine_kernel_address_request Kaddress_req [extern]
```

7.147.3.20 memmap_req

```
volatile struct limine_memmap_request memmap_req [extern]
```

Copyright (c) 2025 Tristan Adams Released under MIT License.

7.148 paging_bootstrap.c

Go to the documentation of this file.

```
00001
00006 #include "../include/KernelUtils.h"
00007 #include "../include/early_alloc.h"
00008 #include "../include/limine.h"
00009 #include "../include/paging/paging_bootstrap.h"
00010 #include "../include/printf.h"
00011 #include <stdbool.h>
00012 #include <stddef.h>
00013 #include <stdint.h>
00014
00015 /* ----- */
00016 /* Limine requests (DEFINED ONCE in limine_requests.c) */
00017 /* ----- */
00018 extern volatile struct limine_memmap_request memmap_req;
00019 extern volatile struct limine_hhdm_request hhdm_request;
00020 extern volatile struct limine_kernel_address_request Kaddress_req;
00021
00022 /* Linker symbols: VIRTUAL kernel bounds */
00023 extern char __kernel_start[], __kernel_end[];
00024 extern char __text_start[], __text_end[];
00025 extern char __rodata_start[], __rodata_end[];
00026 extern char __data_start[], __bss_end[];
00027
00028 /* Panic + asm stubs */
00029 extern void panic(const char *fmt, ...);
00030 extern uint64_t readCR3(void);
00031 extern void writeCR3(uint64_t arg);
00032 extern uint64_t readRSP(void);
00033
00034 /* Retained for existing debug/PMM plumbing. */
00035 uint64_t g_alloc_bm_phys_base = 0;
00036 uint64_t g_alloc_bm_bytes = 0;
00037 uint64_t g_usable_bm_phys_base = 0;
00038 uint64_t g_usable_bm_bytes = 0;
00039 uint64_t framebuffer_phys = 0;
00040 uint64_t framebuffer_bytes = 0;
00041
00042 /* ----- */
00043 /* x86_64 paging bits / helpers */
00044 /* ----- */
00045 #define PTE_P (1ull < 0)
00046 #define PTE_W (1ull < 1)
00047 #define PTE_U (1ull < 2)
00048 #define PTE_PS (1ull < 7)
00049 #define PTE_NX (1ull < 63)
00050
00051 #define PAGE_SIZE 4096ull
00052 #define PAGE_2M_SIZE (2ull * 1024 * 1024)
00053 #define PAGE_1G_SIZE (1ull < 30)
00054
00055 #define ADDR_MASK 0x000FFFFFFFFF000ull
00056 #define ADDR_MASK_2M 0x000FFFFFFFFFE000ull
00057 #define ADDR_MASK_1G 0x000FFFFFC0000000ull
00058
00059 #define ALIGN_UP(x, a) (((x) + ((a) - 1)) & ~((a) - 1))
00060 #define ALIGN_DOWN(x, a) ((x) & ~((a) - 1))
00061
```

```

00062 struct __attribute__((aligned(0x1000))) page_table
00063 {
00064     uint64_t e[512];
00065 };
00066
00067 static uint64_t      g_pml4_phys = 0;
00068 static struct page_table *g_pml4_virt = 0;
00069 static uint64_t      g_hhdm     = 0;
00070
00071 uint64_t paging_pml4_phys(void)
00072 {
00073     return g_pml4_phys;
00074 }
00075
00076 void *paging_pml4_virt(void)
00077 {
00078     return (void *) g_pml4_virt;
00079 }
00080
00081 uint64_t get_total_usable_mem_bytes(void)
00082 {
00083     if (!memmap_req.response || memmap_req.response->entry_count == 0)
00084         return 0;
00085
00086     uint64_t total = 0;
00087     for (size_t i = 0; i < memmap_req.response->entry_count; i++)
00088     {
00089         struct limine_memmap_entry *e = memmap_req.response->entries[i];
00090         if (e->type == LIMINE_MEMMAP_USABLE)
00091             total += e->length;
00092     }
00093     return total;
00094 }
00095
00096 static uint64_t get_max_phys(void)
00097 {
00098     uint64_t max_phys = 0;
00099
00100     if (!memmap_req.response)
00101         return 0;
00102
00103     for (size_t i = 0; i < memmap_req.response->entry_count; i++)
00104     {
00105         struct limine_memmap_entry *e = memmap_req.response->entries[i];
00106         uint64_t end = e->base + e->length;
00107         if (end > max_phys)
00108             max_phys = end;
00109     }
00110
00111     return max_phys;
00112 }
00113
00114 static inline void memzero(void *p, size_t n)
00115 {
00116     volatile uint8_t *d = (volatile uint8_t *) p;
00117     for (size_t i = 0; i < n; i++)
00118         d[i] = 0;
00119 }
00120
00121 static inline void *phys_to_virt(uint64_t phys)
00122 {
00123     return (void *) (uintptr_t) (phys + g_hhdm);
00124 }
00125
00126 static inline void vaddr_indices(uint64_t v, uint64_t idx[4])
00127 {
00128     idx[0] = (v >> 39) & 0x1FF;
00129     idx[1] = (v >> 30) & 0x1FF;
00130     idx[2] = (v >> 21) & 0x1FF;
00131     idx[3] = (v >> 12) & 0x1FF;
00132 }
00133
00134 static inline uint64_t
00135 virt_to_phys_kernel(uint64_t vaddr, uint64_t kphys_base, uint64_t kvirt_base)
00136 {
00137     return kphys_base + (vaddr - kvirt_base);
00138 }
00139
00140 /* ----- */
00141 /* Table allocation */
00142 /* ----- */
00143 static struct page_table *alloc_table(uint64_t *out_phys)
00144 {
00145     uint64_t phys = early_alloc_page();
00146
00147     if (g_hhdm == 0)
00148         panic("paging_bootstrap: HHDM required for table zeroing");

```

```

00149
00150     struct page_table *pt = (struct page_table *) phys_to_virt(phys);
00151     memzero(pt, sizeof(*pt));
00152
00153     if (out_phys)
00154         *out_phys = phys;
00155
00156     return pt;
00157 }
00158
00159 /* This helper is ONLY valid for non-leaf entries. A present PS entry is a
00160 * huge-page leaf, not another page-table pointer. Treating it as a child
00161 * table is memory corruption, so fail loudly if a caller ever tries. */
00162 static struct page_table *
00163 get_or_make_next(struct page_table *cur, uint64_t idx, uint64_t *phys_out)
00164 {
00165     uint64_t e = cur->e[idx];
00166
00167     if (e & PTE_P)
00168     {
00169         if (e & PTE_PS)
00170             panic("paging_bootstrap: attempted descent through huge-page leaf");
00171
00172         uint64_t child_phys = e & ADDR_MASK;
00173         if (phys_out)
00174             *phys_out = child_phys;
00175         return (struct page_table *) phys_to_virt(child_phys);
00176     }
00177
00178     uint64_t child_phys = 0;
00179     struct page_table *child = alloc_table(&child_phys);
00180     cur->e[idx] = (child_phys & ADDR_MASK) | PTE_P | PTE_W;
00181
00182     if (phys_out)
00183         *phys_out = child_phys;
00184
00185     return child;
00186 }
00187
00188 static struct page_table *get_or_make_pd(struct page_table *pml4,
00189                                         uint64_t pml4i,
00190                                         uint64_t pdpti,
00191                                         uint64_t *pd_phys_out)
00192 {
00193     uint64_t junk;
00194     struct page_table *pdpt = get_or_make_next(pml4, pml4i, &junk);
00195     return get_or_make_next(pdpt, pdpti, pd_phys_out);
00196 }
00197
00198 /* ----- */
00199 /* Mapping helpers */
00200 /* ----- */
00201 static void
00202 map_4k(struct page_table *pml4, uint64_t va, uint64_t pa, uint64_t flags)
00203 {
00204     uint64_t idx[4];
00205     uint64_t junk;
00206     vaddr_indices(va, idx);
00207
00208     struct page_table *pdpt = get_or_make_next(pml4, idx[0], &junk);
00209     struct page_table *pd = get_or_make_next(pdpt, idx[1], &junk);
00210     struct page_table *pt = get_or_make_next(pd, idx[2], &junk);
00211
00212     pt->e[idx[3]] = (pa & ADDR_MASK) | (flags & ~ADDR_MASK) | PTE_P;
00213 }
00214
00215 static void
00216 map_2m(struct page_table *pml4, uint64_t va, uint64_t pa, uint64_t flags)
00217 {
00218     uint64_t idx[4];
00219     uint64_t pd_phys;
00220     vaddr_indices(va, idx);
00221
00222     struct page_table *pd = get_or_make_pd(pml4, idx[0], idx[1], &pd_phys);
00223     pd->e[idx[2]] =
00224         (pa & ADDR_MASK_2M) | (flags & ~ADDR_MASK) | PTE_P | PTE_W | PTE_PS;
00225 }
00226
00227 static void
00228 map_1g(struct page_table *pml4, uint64_t va, uint64_t pa, uint64_t flags)
00229 {
00230     uint64_t idx[4];
00231     uint64_t junk;
00232     vaddr_indices(va, idx);
00233
00234     struct page_table *pdpt = get_or_make_next(pml4, idx[0], &junk);
00235     pdpt->e[idx[1]] =

```

```

00236         (pa & ADDR_MASK_1G) | (flags & ~ADDR_MASK) | PTE_P | PTE_W | PTE_PS;
00237     }
00238
00239 static void map_range_4k(struct page_table *pml4,
00240                        uint64_t va,
00241                        uint64_t pa,
00242                        uint64_t len,
00243                        uint64_t flags)
00244 {
00245     uint64_t end = va + len;
00246
00247     for (; va < end; va += PAGE_SIZE, pa += PAGE_SIZE)
00248         map_4k(pml4, va, pa, flags);
00249 }
00250
00251 static void map_range_huge_first(struct page_table *pml4,
00252                                uint64_t va,
00253                                uint64_t pa,
00254                                uint64_t len,
00255                                uint64_t base_flags)
00256 {
00257     uint64_t end = pa + len;
00258
00259     while (pa < end && ((pa | va) & (PAGE_2M_SIZE - 1)) != 0)
00260     {
00261         map_4k(pml4, va, pa, base_flags | PTE_W | PTE_NX);
00262         pa += PAGE_SIZE;
00263         va += PAGE_SIZE;
00264     }
00265
00266     while (pa + PAGE_2M_SIZE <= end)
00267     {
00268         map_2m(pml4, va, pa, base_flags | PTE_NX);
00269         pa += PAGE_2M_SIZE;
00270         va += PAGE_2M_SIZE;
00271     }
00272
00273     while (pa < end)
00274     {
00275         map_4k(pml4, va, pa, base_flags | PTE_W | PTE_NX);
00276         pa += PAGE_SIZE;
00277         va += PAGE_SIZE;
00278     }
00279 }
00280
00281 /* ----- */
00282 /* Kernel + HHDM builders */
00283 /* ----- */
00284 static void map_identity_minimal(struct page_table *pml4)
00285 {
00286     for (uint64_t p = 0; p < PAGE_2M_SIZE; p += PAGE_SIZE)
00287         map_4k(pml4, p, p, PTE_W | PTE_NX);
00288 }
00289
00290 static void map_kernel_higher_half(struct page_table *pml4)
00291 {
00292     if (!Kaddress_req.response)
00293         panic("paging_bootstrap: kernel address request missing");
00294
00295     uint64_t kphys_base = Kaddress_req.response->physical_base;
00296     uint64_t kvirt_base = Kaddress_req.response->virtual_base;
00297
00298     uint64_t t0 = ALIGN_DOWN((uint64_t) __text_start, PAGE_SIZE);
00299     uint64_t t1 = ALIGN_UP((uint64_t) __text_end, PAGE_SIZE);
00300     uint64_t r0 = ALIGN_DOWN((uint64_t) __rodata_start, PAGE_SIZE);
00301     uint64_t r1 = ALIGN_UP((uint64_t) __rodata_end, PAGE_SIZE);
00302     uint64_t d0 = ALIGN_DOWN((uint64_t) __data_start, PAGE_SIZE);
00303     uint64_t d1 = ALIGN_UP((uint64_t) __bss_end, PAGE_SIZE);
00304
00305     if (t1 > t0)
00306         map_range_4k(pml4,
00307                     t0,
00308                     virt_to_phys_kernel(t0, kphys_base, kvirt_base),
00309                     t1 - t0,
00310                     0); /* RX */
00311
00312     if (r1 > r0)
00313         map_range_4k(pml4,
00314                     r0,
00315                     virt_to_phys_kernel(r0, kphys_base, kvirt_base),
00316                     r1 - r0,
00317                     PTE_NX); /* R */
00318
00319     if (d1 > d0)
00320         map_range_4k(pml4,
00321                     d0,
00322                     virt_to_phys_kernel(d0, kphys_base, kvirt_base),

```

```

00323         dl = d0,
00324         PTE_W | PTE_NX); /* RW */
00325     }
00326
00327 static void map_hhdm(struct page_table *pml4)
00328 {
00329     if (!g_hhdm)
00330         panic("paging_bootstrap: HHDM offset missing");
00331
00332     if (!memmap_req.response || memmap_req.response->entry_count == 0)
00333         panic("paging_bootstrap: memmap missing");
00334
00335     /* Build Voyager-owned HHDM mappings. Do not shallow-clone Limine's
00336     * upper-half tables: sharing lower-level paging structures means any
00337     * retrofit edit can mutate the currently active Limine address space. */
00338     for (size_t i = 0; i < memmap_req.response->entry_count; i++)
00339     {
00340         struct limine_memmap_entry *e = memmap_req.response->entries[i];
00341         bool include = false;
00342
00343         switch (e->type)
00344         {
00345             case LIMINE_MEMMAP_USABLE:
00346             case LIMINE_MEMMAP_BOOTLOADER_RECLAIMABLE:
00347             case LIMINE_MEMMAP_ACPI_RECLAIMABLE:
00348             case LIMINE_MEMMAP_ACPI_NVS:
00349             case LIMINE_MEMMAP_KERNEL_AND_MODULES:
00350             case LIMINE_MEMMAP_FRAMEBUFFER:
00351                 include = true;
00352                 break;
00353             default:
00354                 break;
00355         }
00356
00357         if (!include)
00358             continue;
00359
00360         uint64_t base = ALIGN_DOWN(e->base, PAGE_SIZE);
00361         uint64_t end = ALIGN_UP(e->base + e->length, PAGE_SIZE);
00362
00363         if (end <= base)
00364             continue;
00365
00366         map_range_huge_first(pml4,
00367                             g_hhdm + base,
00368                             base,
00369                             end - base,
00370                             0);
00371     }
00372
00373     /* Always keep the first 2 MiB addressable through HHDM. This is written
00374     * last deliberately: it may replace fragmented low-page mappings with
00375     * one equivalent 2 MiB leaf, but nothing later is allowed to descend
00376     * through that leaf. */
00377     map_2m(pml4, g_hhdm, 0, PTE_NX);
00378 }
00379
00380 /* ----- */
00381 /* VA walker + diagnostics */
00382 /* ----- */
00383 static bool walk_va_to_pa(uint64_t va,
00384                          struct page_table *pml4_root,
00385                          uint64_t *pa_out,
00386                          int *level_out)
00387 {
00388     uint64_t idx[4];
00389     vaddr_indices(va, idx);
00390
00391     uint64_t pml4e = pml4_root->e[idx[0]];
00392     if (!(pml4e & PTE_P))
00393         return false;
00394
00395     struct page_table *pdpte =
00396         (struct page_table *) phys_to_virt(pml4e & ADDR_MASK);
00397     uint64_t pdpte = pdpte->e[idx[1]];
00398     if (!(pdpte & PTE_P))
00399         return false;
00400
00401     if (pdpte & PTE_PS)
00402     {
00403         if (level_out)
00404             *level_out = 3;
00405         if (pa_out)
00406             *pa_out = (pdpte & ADDR_MASK_1G) | (va & (PAGE_1G_SIZE - 1));
00407         return true;
00408     }
00409 }

```

```

00410     struct page_table *pd =
00411         (struct page_table *) phys_to_virt(pdpte & ADDR_MASK);
00412     uint64_t pde = pd->e[idx[2]];
00413     if (!(pde & PTE_P))
00414         return false;
00415
00416     if (pde & PTE_PS)
00417     {
00418         if (level_out)
00419             *level_out = 2;
00420         if (pa_out)
00421             *pa_out = (pde & ADDR_MASK_2M) | (va & (PAGE_2M_SIZE - 1));
00422         return true;
00423     }
00424
00425     struct page_table *pt =
00426         (struct page_table *) phys_to_virt(pde & ADDR_MASK);
00427     uint64_t pte = pt->e[idx[3]];
00428     if (!(pte & PTE_P))
00429         return false;
00430
00431     if (level_out)
00432         *level_out = 1;
00433     if (pa_out)
00434         *pa_out = (pte & ADDR_MASK) | (va & (PAGE_SIZE - 1));
00435
00436     return true;
00437 }
00438
00439 static void probe_va(const char *label, uint64_t va, struct page_table *tbl)
00440 {
00441     int lvl = 0;
00442     uint64_t pa = 0;
00443
00444     if (!walk_va_to_pa(va, tbl, &pa, &lvl))
00445     {
00446         printf(" probe %-12s: VA=0x%llx MISSING\n",
00447             label,
00448             (unsigned long long) va);
00449         return;
00450     }
00451
00452     printf(" probe %-12s: VA=0x%llx -> PA=0x%llx (lvl %d)\n",
00453         label,
00454         (unsigned long long) va,
00455         (unsigned long long) pa,
00456         lvl);
00457 }
00458
00459 static void probe_hhdm_phys(const char *label,
00460                             uint64_t phys,
00461                             struct page_table *pml4)
00462 {
00463     uint64_t va = g_hhdm + phys;
00464     uint64_t pa = 0;
00465     int lvl = 0;
00466
00467     if (!walk_va_to_pa(va, pml4, &pa, &lvl))
00468     {
00469         printf("HHDM %s: phys=0x%llx VA=0x%llx MISSING\n",
00470             label,
00471             (unsigned long long) phys,
00472             (unsigned long long) va);
00473         return;
00474     }
00475
00476     printf("HHDM %s: phys=0x%llx VA=0x%llx -> PA=0x%llx (lvl %d)\n",
00477         label,
00478         (unsigned long long) phys,
00479         (unsigned long long) va,
00480         (unsigned long long) pa,
00481         lvl);
00482 }
00483
00484 static void scan_hhdm_range(uint64_t phys_start,
00485                             uint64_t length,
00486                             uint64_t stride,
00487                             struct page_table *pml4)
00488 {
00489     if (stride == 0)
00490         stride = PAGE_2M_SIZE;
00491
00492     uint64_t end = phys_start + length;
00493     int prev_state = -1;
00494     uint64_t run_start = phys_start;
00495
00496     for (uint64_t phys = phys_start; phys < end; phys += stride)

```



```

00497     {
00498         bool present = walk_va_to_pa(g_hhdm + phys, pml4, NULL, NULL);
00499         int state = present ? 1 : 0;
00500
00501         if (prev_state == -1)
00502         {
00503             prev_state = state;
00504             run_start = phys;
00505         }
00506         else if (state != prev_state)
00507         {
00508             printf("HHDM scan: [%#llx .. %#llx] %s\n",
00509                 (unsigned long long) run_start,
00510                 (unsigned long long) phys,
00511                 prev_state ? "PRESENT" : "MISSING");
00512             prev_state = state;
00513             run_start = phys;
00514         }
00515     }
00516
00517     printf("HHDM scan: [%#llx .. %#llx] %s\n",
00518         (unsigned long long) run_start,
00519         (unsigned long long) end,
00520         prev_state > 0 ? "PRESENT" : "MISSING");
00521 }
00522
00523 /* ----- */
00524 /* Safe coverage enforcement */
00525 /* ----- */
00526 static void ensure_hhdm_cover_phys_page(struct page_table *pml4,
00527                                         uint64_t phys_page)
00528 {
00529     uint64_t page = ALIGN_DOWN(phys_page, PAGE_SIZE);
00530     uint64_t va = g_hhdm + page;
00531
00532     /* A huge-page mapping is already sufficient coverage. Never replace or
00533  * descend through it just to obtain a 4 KiB mapping. */
00534     if (walk_va_to_pa(va, pml4, NULL, NULL))
00535         return;
00536
00537     map_4k(pml4, va, page, PTE_W | PTE_NX);
00538 }
00539
00540 static void ensure_hhdm_cover_phys_range(struct page_table *pml4,
00541                                         uint64_t phys,
00542                                         uint64_t len)
00543 {
00544     if (len == 0)
00545         return;
00546
00547     uint64_t page = ALIGN_DOWN(phys, PAGE_SIZE);
00548     uint64_t end = ALIGN_UP(phys + len, PAGE_SIZE);
00549
00550     for (; page < end; page += PAGE_SIZE)
00551         ensure_hhdm_cover_phys_page(pml4, page);
00552 }
00553
00554 /* ----- */
00555 /* Pre-switch validation + install */
00556 /* ----- */
00557 static void pre_switch_audit_and_patch(struct page_table *new_pml4,
00558                                       uint64_t new_pml4_phys)
00559 {
00560     uint64_t old_cr3 = readCR3() & ADDR_MASK;
00561     uint64_t rsp_va = readRSP();
00562     uint64_t rsp_phys = 0;
00563     uint64_t max_phys = get_max_phys();
00564
00565     if (rsp_va >= g_hhdm && rsp_va < g_hhdm + max_phys)
00566     {
00567         rsp_phys = rsp_va - g_hhdm;
00568     }
00569     else if (!walk_va_to_pa(rsp_va, new_pml4, &rsp_phys, NULL))
00570     {
00571         panic("paging_bootstrap: stack VA missing before CR3 switch");
00572     }
00573
00574     /* These calls are deliberately idempotent. Existing 2 MiB/1 GiB HHDM
00575  * leaves count as valid coverage and are left untouched. */
00576     ensure_hhdm_cover_phys_range(new_pml4, 0, PAGE_2M_SIZE);
00577     ensure_hhdm_cover_phys_page(new_pml4, rsp_phys);
00578     ensure_hhdm_cover_phys_page(new_pml4, old_cr3);
00579     ensure_hhdm_cover_phys_page(new_pml4, new_pml4_phys);
00580
00581     printf("Pre-switch validation:\n");
00582     probe_va("stack", rsp_va, new_pml4);
00583     probe_va("rip",

```

```

00584         (uint64_t) (uintptr_t) &pre_switch_audit_and_patch,
00585         new_pml4);
00586 probe_va("HHDM+0", g_hhdm, new_pml4);
00587 probe_va("HHDM+oldCR3", g_hhdm + old_cr3, new_pml4);
00588 probe_va("HHDM+newCR3", g_hhdm + new_pml4_phys, new_pml4);
00589
00590 if (!walk_va_to_pa(rsp_va, new_pml4, NULL, NULL))
00591     panic("paging_bootstrap: stack missing after patching");
00592
00593 if (!walk_va_to_pa((uint64_t) (uintptr_t) &pre_switch_audit_and_patch,
00594                 new_pml4,
00595                 NULL,
00596                 NULL))
00597     panic("paging_bootstrap: kernel RIP missing after patching");
00598
00599 if (!walk_va_to_pa(g_hhdm, new_pml4, NULL, NULL))
00600     panic("paging_bootstrap: HHDM+0 missing after patching");
00601
00602 if (!walk_va_to_pa(g_hhdm + old_cr3, new_pml4, NULL, NULL))
00603     panic("paging_bootstrap: old CR3 page missing from HHDM");
00604
00605 if (!walk_va_to_pa(g_hhdm + new_pml4_phys, new_pml4, NULL, NULL))
00606     panic("paging_bootstrap: new CR3 page missing from HHDM");
00607 }
00608
00609 static void install_new_cr3(uint64_t new_pml4_phys,
00610                             struct page_table *new_pml4)
00611 {
00612     pre_switch_audit_and_patch(new_pml4, new_pml4_phys);
00613
00614     uint64_t old_cr3 = readCR3();
00615     printf("Installing CR3... old=0x%llx new=0x%llx\n",
00616           (unsigned long long) old_cr3,
00617           (unsigned long long) new_pml4_phys);
00618
00619     writeCR3(new_pml4_phys);
00620
00621     uint64_t cr3_after = readCR3();
00622     printf("CR3 after install: 0x%llx\n",
00623           (unsigned long long) cr3_after);
00624
00625     asm volatile("push %%rax; pop %%rax" ::: "rax", "memory");
00626     volatile uint8_t *text_probe =
00627         (volatile uint8_t *) (uintptr_t) &install_new_cr3;
00628     (void) *text_probe;
00629
00630     printf("CR3 install validation complete.\n");
00631 }
00632
00633 /* ----- */
00634 /* Public bootstrap */
00635 /* ----- */
00636 void paging_bootstrap(void)
00637 {
00638     if (!hhdm_request.response)
00639         panic("paging_bootstrap: HHDM response missing");
00640
00641     g_hhdm = hhdm_request.response->offset;
00642     if (g_hhdm == 0)
00643         panic("paging_bootstrap: HHDM offset is zero");
00644
00645     /* New root and every lower-level table are Voyager-owned. We use the
00646     * currently active Limine HHDM only to access/zero these physical pages
00647     * while constructing the replacement address space. */
00648     g_pml4_virt = alloc_table(&g_pml4_phys);
00649
00650     map_identity_minimal(g_pml4_virt);
00651     map_kernel_higher_half(g_pml4_virt);
00652     map_hhdm(g_pml4_virt);
00653
00654     printf("INFO: CR3 to install: 0x%llx\n",
00655           (unsigned long long) g_pml4_phys);
00656     printf("INFO: RSP: 0x%llx\n",
00657           (unsigned long long) readRSP());
00658
00659     printf("Memory Total: %iMiB\n",
00660           bytes_to_mib(get_total_usable_mem_bytes()));
00661
00662     printf(">>> Probing HHDM <<<\n");
00663     printf("-----\n");
00664
00665     probe_hhdm_phys("low+0", 0, g_pml4_virt);
00666     probe_hhdm_phys("old CR3 page", readCR3() & ADDR_MASK, g_pml4_virt);
00667     probe_hhdm_phys("new CR3 page", g_pml4_phys, g_pml4_virt);
00668
00669     scan_hhdm_range(0,
00670                     16ull * 1024 * 1024,

```

```

00671             PAGE_2M_SIZE,
00672             g_pml4_virt);
00673
00674     printf("-----\n");
00675     printf(">> HHDM Probe Done <<\n");
00676
00677     install_new_cr3(g_pml4_phys, g_pml4_virt);
00678 }

```

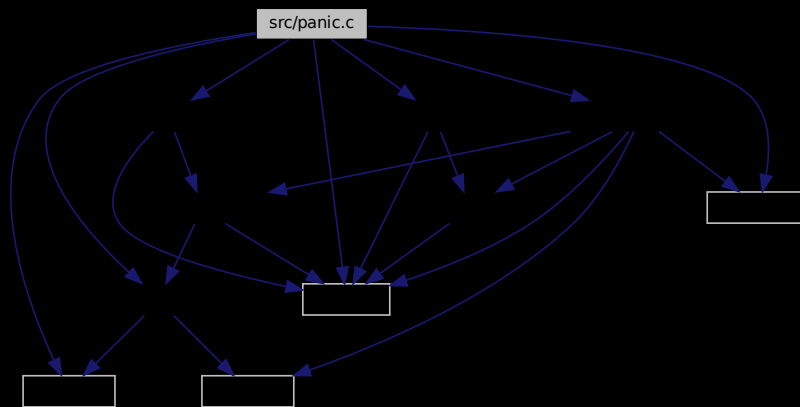
7.149 src/panic.c File Reference

```

#include "include/KernelUtils.h"
#include "include/kernel.h"
#include "include/printf.h"
#include "include/stack_trace.h"
#include <stdarg.h>
#include <stdbool.h>
#include <stdint.h>

```

Include dependency graph for panic.c:



Functions

- void **halt** ()
- void **panic** (const char *fmt,...)

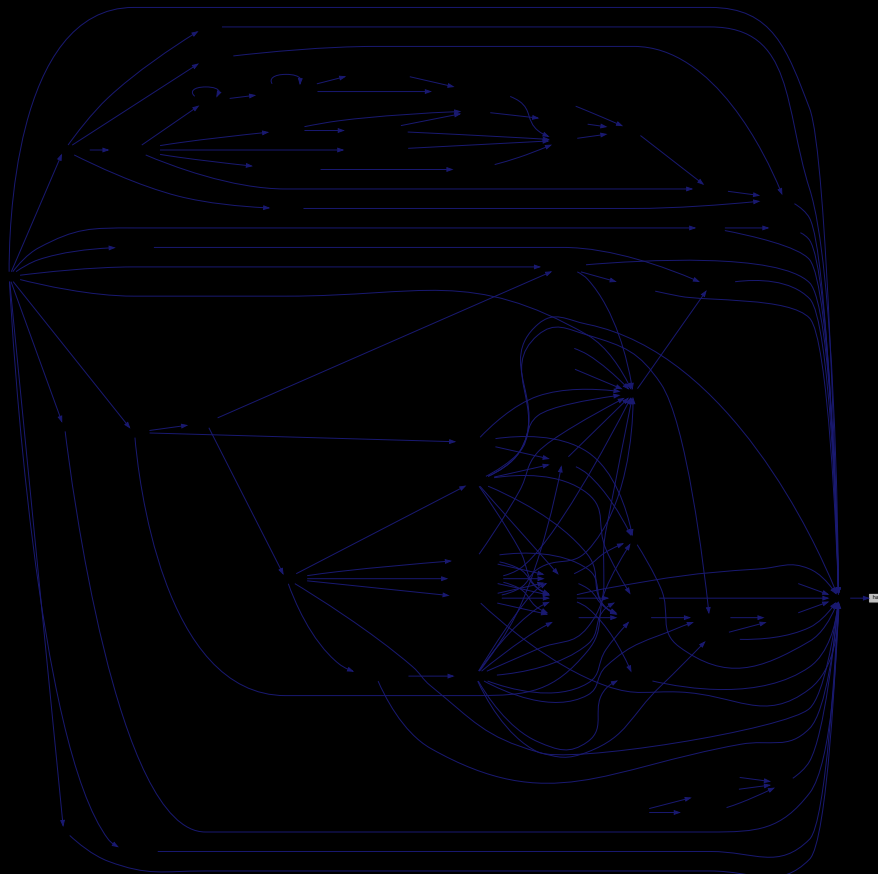
Panic with a nicely formatted message. This is useful for situations where you are certain an unrecoverable error will occur and want to be informative (feel free to abuse sarcasm in them) about the problem.

7.149.1 Function Documentation

7.149.1.1 halt()

```
void halt ( )
```

Here is the caller graph for this function:



7.149.1.2 panic()

```
void panic (
    const char * fmt,
    ... )
```

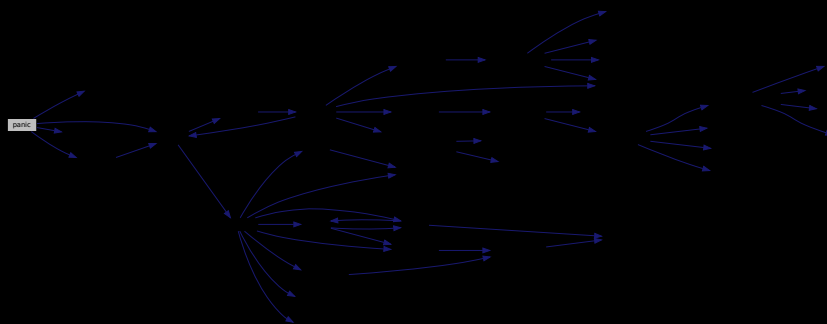
Panic with a nicely formatted message. This is useful for situations where you are certain an unrecoverable error will occur and want to be informative (feel free to abuse sarcasm in them) about the problem.

Parameters

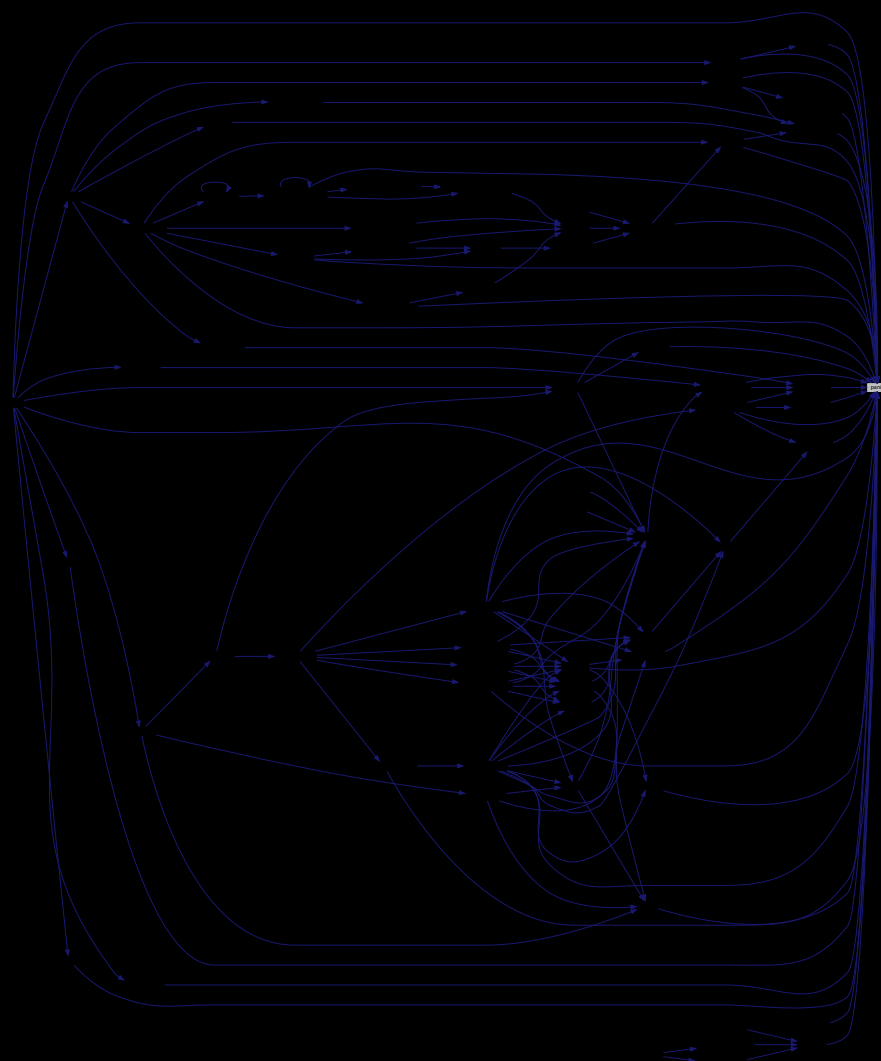
<i>fmt</i>	printf style format string for the panic message
------------	--

Definition at line 17 of file [panic.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.150 panic.c

Go to the documentation of this file.

```

00001 #include "include/KernelUtils.h"
00002 #include "include/kernel.h"
00003 #include "include/printf.h"
00004 #include "include/stack_trace.h"
00005 #include <stdarg.h>
00006 #include <stdbool.h>
00007 #include <stdint.h>
00008
00009 extern void halt();
00010
00017 void panic(const char *fmt, ...)
00018 {
00019     if (init_done == 1)
00020     {
00021         kerror_mode = 1;
00022     }
00023     static char buf[1024];
00024
00025     va_list args;
00026
00027     va_start(args, fmt);
00028
00029     vsnprintf(buf, sizeof(buf), fmt, args);
00030     va_end(args);
00031     printf_("%s\n", "!!!!!!!!!!!!!!!!!!!!!!KERNEL PANIC!!!!!!!!!!!!!!!!!!!!!!");
00032     printf_("%s\n", "-----");
00033     printf("%s", "CAUSE: ");
00034     printf("%s\n", buf);
00035
00036     printf_("%s\n", "-----");
00037
00038     stack_trace(0);
00039     printf("%s\n", "-----");
00040     printf("%s\n", "      INFO: Stack Trace (Dump) as Follows");
00041     printf("%s\n", "-----");
00042     stack_dump_asm(0, 256);
00043     printf("%s\n", "-----");
00044     printf_("%s\n", "End of Trace.");
00045     printf("%s\n", "-----");
00046
00047     printf_("%s\n", "!!!!!!!!!!!!!!!!!!!!!!KERNEL PANIC!!!!!!!!!!!!!!!!!!!!!!");
00048
00049     halt();
00050 }

```

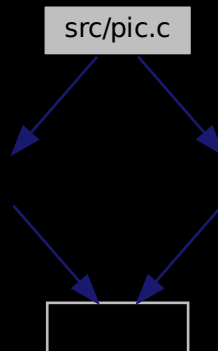
7.151 src/pic.c File Reference

```

#include "include/pic.h"
#include "include/io.h"

```

Include dependency graph for pic.c:



Functions

- void `pic_mask_irq` (uint8_t irq)
Mask one legacy PIC IRQ line.
- void `pic_unmask_irq` (uint8_t irq)
Unmask one legacy PIC IRQ line.
- void `pic_remap_offsets` (uint8_t offset)
Remap PIC IRQ0 to a new IDT vector base.
- void `pic_send_eoi` (uint8_t irq)
Send end-of-interrupt acknowledgement for one legacy PIC IRQ.
- void `pic_enable` ()
Initialize/remap the legacy PICs for Voyager's interrupt layout.

7.151.1 Function Documentation

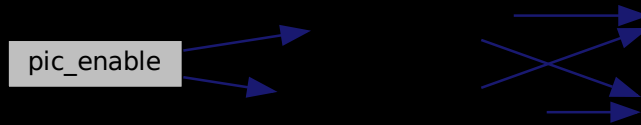
7.151.1.1 `pic_enable()`

```
void pic_enable (  
    void )
```

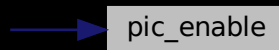
Initialize/remap the legacy PICs for Voyager's interrupt layout.

Definition at line 75 of file `pic.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.151.1.2 `pic_mask_irq()`

```
void pic_mask_irq (
    uint8_t irq )
```

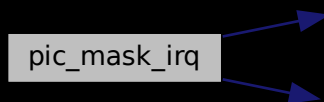
Mask one legacy PIC IRQ line.

Parameters

<i>irq</i>	IRQ number in the range handled by the cascaded PICs.
------------	---

Definition at line 5 of file `pic.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.151.1.3 `pic_remap_offsets()`

```
void pic_remap_offsets (
    uint8_t offset )
```

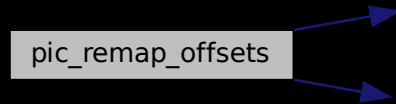
Remap PIC IRQ0 to a new IDT vector base.

Parameters

<i>offset</i>	Vector number corresponding to IRQ0 after remapping.
---------------	--

Definition at line 45 of file `pic.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.151.1.4 `pic_send_eoi()`

```
void pic_send_eoi (
    uint8_t irq )
```

Send end-of-interrupt acknowledgement for one legacy PIC IRQ.

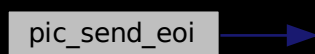
Parameters

<i>irq</i>	Hardware IRQ number, not an arbitrary IDT vector.
------------	---

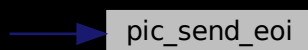
The generic interrupt dispatcher only sends PIC EOI for vectors belonging to the remapped hardware IRQ range. Dynamic/software vectors must not be passed through PIC acknowledgement merely because they used the same dispatcher.

Definition at line 68 of file `pic.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.151.1.5 `pic_unmask_irq()`

```
void pic_unmask_irq (  
    uint8_t irq )
```

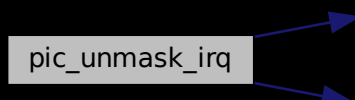
Unmask one legacy PIC IRQ line.

Parameters

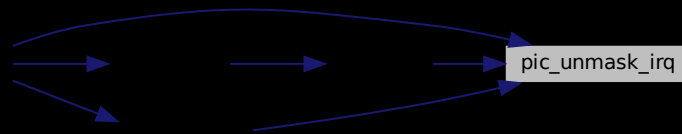
<i>irq</i>	IRQ number in the range handled by the cascaded PICs.
------------	---

Definition at line 25 of file `pic.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.152 pic.c

[Go to the documentation of this file.](#)

```

00001 #include "include/pic.h"
00002 #include "include/io.h"
00003
00004 /* Public API documentation lives in include/pic.h. */
00005 void pic_mask_irq(uint8_t irq)
00006 {
00007     uint16_t port;
00008     uint8_t masks;
00009
00010     if (irq < 8)
00011     {
00012         port = PIC_MASTER_DATA;
00013     }
00014     else
00015     {
00016         port = PIC_SLAVE_DATA;
00017         irq -= 8;
00018     }
00019
00020     masks = inb(port);
00021     masks |= (1 << irq);
00022     outb(port, masks);
00023 }
00024
00025 void pic_unmask_irq(uint8_t irq)
00026 {
00027     uint16_t port;
00028     uint8_t masks;
00029
00030     if (irq < 8)
00031     {
00032         port = PIC_MASTER_DATA;
00033     }
00034     else
00035     {
00036         port = PIC_SLAVE_DATA;
00037         irq -= 8;
00038     }
00039
00040     masks = inb(port);
00041     masks &= ~(1 << irq);
00042     outb(port, masks);
00043 }
00044
00045 void pic_remap_offsets(uint8_t offset)
00046 {
00047     uint8_t master_mask, slave_mask;
00048
00049     master_mask = inb(PIC_MASTER_DATA);
00050     slave_mask = inb(PIC_SLAVE_DATA);
00051
00052     outb(PIC_MASTER_COMMAND, PIC_ICW1_INIT | PIC_ICW1_ICW4);
00053     outb(PIC_SLAVE_COMMAND, PIC_ICW1_INIT | PIC_ICW1_ICW4);
00054
00055     outb(PIC_MASTER_DATA, offset);
00056     outb(PIC_SLAVE_DATA, offset + 0x08);
00057
00058     outb(PIC_MASTER_DATA, 0x04);

```

```

00059     outb(PIC_SLAVE_DATA, 0x02);
00060
00061     outb(PIC_MASTER_DATA, PIC_ICW4_8086);
00062     outb(PIC_SLAVE_DATA, PIC_ICW4_8086);
00063
00064     outb(PIC_MASTER_DATA, master_mask);
00065     outb(PIC_SLAVE_DATA, slave_mask);
00066 }
00067
00068 void pic_send_eoi(uint8_t irq)
00069 {
00070     if (irq >= 8)
00071         outb(PIC_SLAVE_COMMAND, PIC_EOI);
00072     outb(PIC_MASTER_COMMAND, PIC_EOI);
00073 }
00074
00075 void pic_enable()
00076 {
00077     pic_remap_offsets(0x20);
00078     for (uint8_t irq = 0; irq < 16; irq++)
00079         pic_mask_irq(irq);
00080 }

```

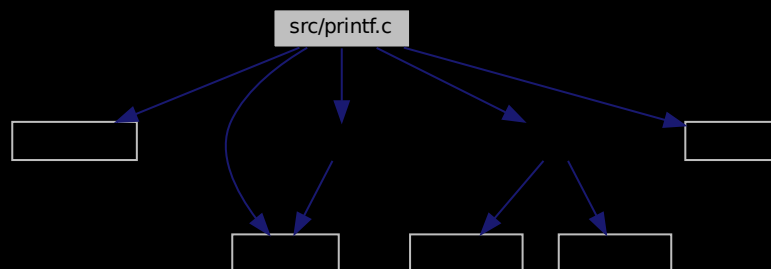
7.153 src/printf.c File Reference

```

#include <stdbool.h>
#include <stdint.h>
#include "include/printf.h"
#include "include/serial.h"
#include <float.h>

```

Include dependency graph for printf.c:



Data Structures

- struct `out_fct_wrap_type`

Macros

- `#define PRINTF_NTOA_BUFFER_SIZE 32U`
- `#define PRINTF_FTOA_BUFFER_SIZE 32U`
- `#define PRINTF_SUPPORT_FLOAT`
- `#define PRINTF_SUPPORT_EXPONENTIAL`
- `#define PRINTF_DEFAULT_FLOAT_PRECISION 6U`
- `#define PRINTF_MAX_FLOAT 1e9`

- `#define PRINTF_SUPPORT_LONG_LONG`
- `#define PRINTF_SUPPORT_PTRDIFF_T`
- `#define FLAGS_ZEROPAD (1U << 0U)`
- `#define FLAGS_LEFT (1U << 1U)`
- `#define FLAGS_PLUS (1U << 2U)`
- `#define FLAGS_SPACE (1U << 3U)`
- `#define FLAGS_HASH (1U << 4U)`
- `#define FLAGS_UPPERCASE (1U << 5U)`
- `#define FLAGS_CHAR (1U << 6U)`
- `#define FLAGS_SHORT (1U << 7U)`
- `#define FLAGS_LONG (1U << 8U)`
- `#define FLAGS_LONG_LONG (1U << 9U)`
- `#define FLAGS_PRECISION (1U << 10U)`
- `#define FLAGS_ADAPT_EXP (1U << 11U)`

Typedefs

- `typedef void(* out_fct_type) (char character, void *buffer, size_t idx, size_t maxlen)`

Functions

- `static void _out_buffer (char character, void *buffer, size_t idx, size_t maxlen)`
- `static void _out_null (char character, void *buffer, size_t idx, size_t maxlen)`
- `static void _out_char (char character, void *buffer, size_t idx, size_t maxlen)`
- `static void _out_fct (char character, void *buffer, size_t idx, size_t maxlen)`
- `static unsigned int _strlen_s (const char *str, size_t maxsize)`
- `static bool _is_digit (char ch)`
- `static unsigned int _atoi (const char **str)`
- `static size_t _out_rev (out_fct_type out, char *buffer, size_t idx, size_t maxlen, const char *buf, size_t len, unsigned int width, unsigned int flags)`
- `static size_t _ntoa_format (out_fct_type out, char *buffer, size_t idx, size_t maxlen, char *buf, size_t len, bool negative, unsigned int base, unsigned int prec, unsigned int width, unsigned int flags)`
- `static size_t _ntoa_long (out_fct_type out, char *buffer, size_t idx, size_t maxlen, unsigned long value, bool negative, unsigned long base, unsigned int prec, unsigned int width, unsigned int flags)`
- `static size_t _ntoa_long_long (out_fct_type out, char *buffer, size_t idx, size_t maxlen, unsigned long long value, bool negative, unsigned long long base, unsigned int prec, unsigned int width, unsigned int flags)`
- `static size_t _etoa (out_fct_type out, char *buffer, size_t idx, size_t maxlen, double value, unsigned int prec, unsigned int width, unsigned int flags)`
- `static size_t _ftoa (out_fct_type out, char *buffer, size_t idx, size_t maxlen, double value, unsigned int prec, unsigned int width, unsigned int flags)`
- `static int _vsnprintf (out_fct_type out, char *buffer, const size_t maxlen, const char *format, va_list va)`
- `int printf_ (const char *format,...)`
- `int sprintf_ (char *buffer, const char *format,...)`
- `int snprintf_ (char *buffer, size_t count, const char *format,...)`
- `int vprintf_ (const char *format, va_list va)`
- `int vsnprintf_ (char *buffer, size_t count, const char *format, va_list va)`
- `int fctprintf (void(*out)(char character, void *arg), void *arg, const char *format,...)`

7.153.1 Macro Definition Documentation

7.153.1.1 FLAGS_ADAPT_EXP

```
#define FLAGS_ADAPT_EXP (1U << 11U)
```

Definition at line 112 of file [printf.c](#).

7.153.1.2 FLAGS_CHAR

```
#define FLAGS_CHAR (1U << 6U)
```

Definition at line 107 of file [printf.c](#).

7.153.1.3 FLAGS_HASH

```
#define FLAGS_HASH (1U << 4U)
```

Definition at line 105 of file [printf.c](#).

7.153.1.4 FLAGS_LEFT

```
#define FLAGS_LEFT (1U << 1U)
```

Definition at line 102 of file [printf.c](#).

7.153.1.5 FLAGS_LONG

```
#define FLAGS_LONG (1U << 8U)
```

Definition at line 109 of file [printf.c](#).

7.153.1.6 FLAGS_LONG_LONG

```
#define FLAGS_LONG_LONG (1U << 9U)
```

Definition at line 110 of file [printf.c](#).

7.153.1.7 **FLAGS_PLUS**

```
#define FLAGS_PLUS (1U << 2U)
```

Definition at line 103 of file [printf.c](#).

7.153.1.8 **FLAGS_PRECISION**

```
#define FLAGS_PRECISION (1U << 10U)
```

Definition at line 111 of file [printf.c](#).

7.153.1.9 **FLAGS_SHORT**

```
#define FLAGS_SHORT (1U << 7U)
```

Definition at line 108 of file [printf.c](#).

7.153.1.10 **FLAGS_SPACE**

```
#define FLAGS_SPACE (1U << 3U)
```

Definition at line 104 of file [printf.c](#).

7.153.1.11 **FLAGS_UPPERCASE**

```
#define FLAGS_UPPERCASE (1U << 5U)
```

Definition at line 106 of file [printf.c](#).

7.153.1.12 **FLAGS_ZEROPAD**

```
#define FLAGS_ZEROPAD (1U << 0U)
```

Definition at line 101 of file [printf.c](#).

7.153.1.13 PRINTF_DEFAULT_FLOAT_PRECISION

```
#define PRINTF_DEFAULT_FLOAT_PRECISION 6U
```

Definition at line 76 of file [printf.c](#).

7.153.1.14 PRINTF_FTOA_BUFFER_SIZE

```
#define PRINTF_FTOA_BUFFER_SIZE 32U
```

Definition at line 58 of file [printf.c](#).

7.153.1.15 PRINTF_MAX_FLOAT

```
#define PRINTF_MAX_FLOAT 1e9
```

Definition at line 82 of file [printf.c](#).

7.153.1.16 PRINTF_NTOA_BUFFER_SIZE

```
#define PRINTF_NTOA_BUFFER_SIZE 32U
```

Definition at line 51 of file [printf.c](#).

7.153.1.17 PRINTF_SUPPORT_EXPONENTIAL

```
#define PRINTF_SUPPORT_EXPONENTIAL
```

Definition at line 70 of file [printf.c](#).

7.153.1.18 PRINTF_SUPPORT_FLOAT

```
#define PRINTF_SUPPORT_FLOAT
```

Definition at line 64 of file [printf.c](#).

7.153.1.19 PRINTF_SUPPORT_LONG_LONG

```
#define PRINTF_SUPPORT_LONG_LONG
```

Definition at line 88 of file [printf.c](#).

7.153.1.20 PRINTF_SUPPORT_PTRDIFF_T

```
#define PRINTF_SUPPORT_PTRDIFF_T
```

Definition at line 95 of file [printf.c](#).

7.153.2 Typedef Documentation

7.153.2.1 out_fct_type

```
typedef void(* out_fct_type) (char character, void *buffer, size_t idx, size_t maxlen)
```

Definition at line 120 of file [printf.c](#).

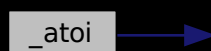
7.153.3 Function Documentation

7.153.3.1 _atoi()

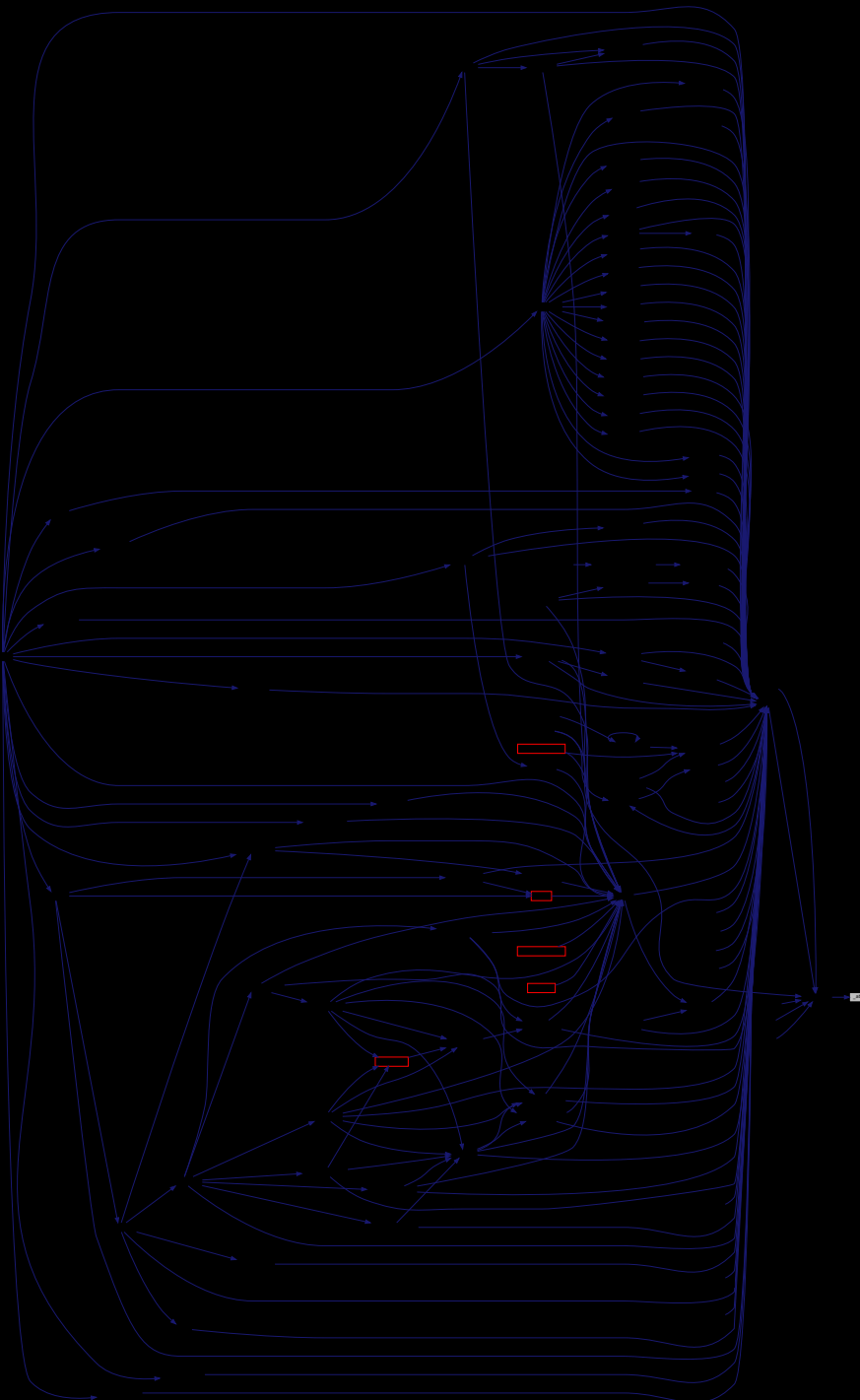
```
static unsigned int _atoi (  
    const char ** str ) [static]
```

Definition at line 198 of file [printf.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.153.3.2 `_etoa()`

```
static size_t _etoa (  
    out_fct_type out,
```

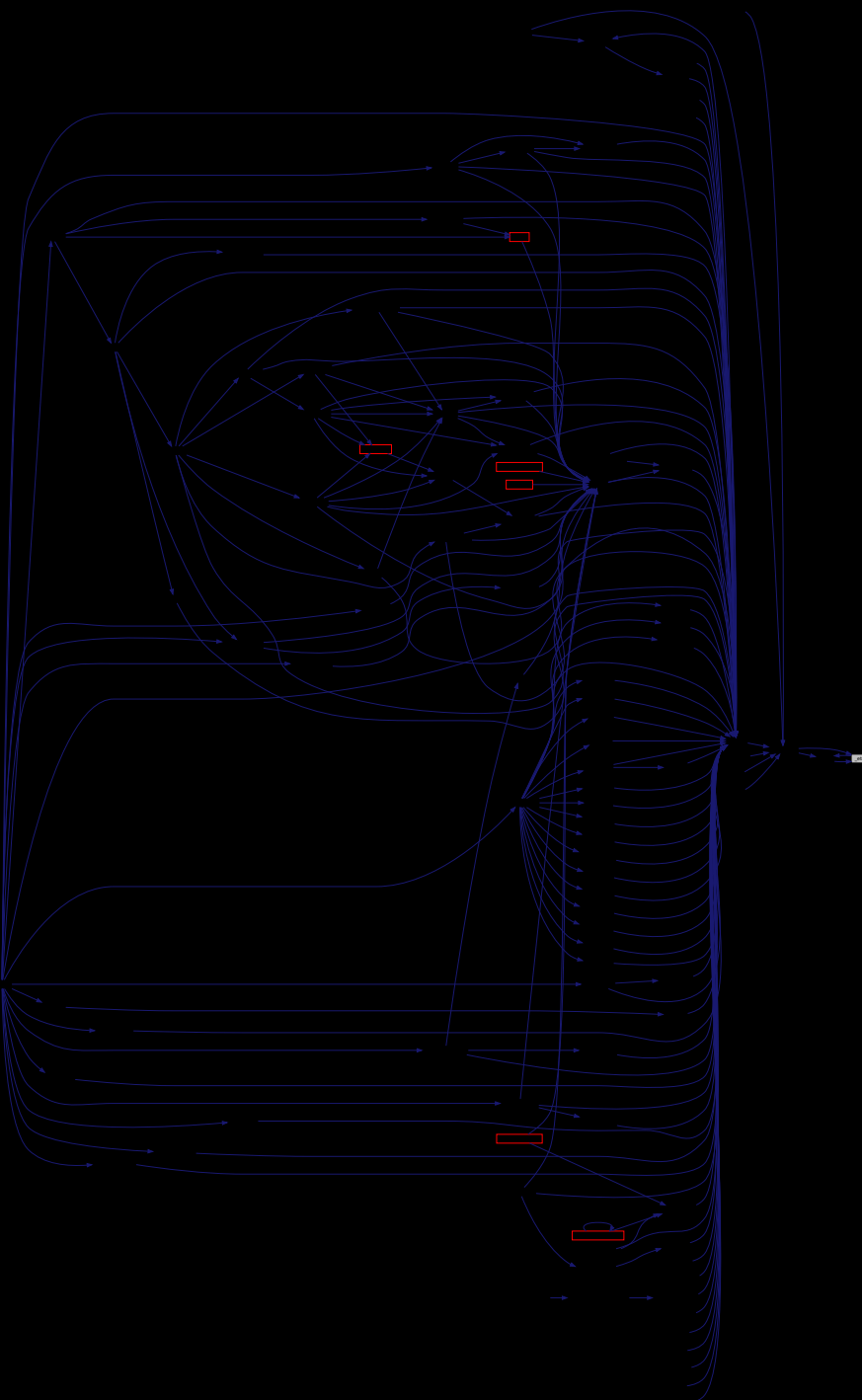
```
char * buffer,  
size_t idx,  
size_t maxlen,  
double value,  
unsigned int prec,  
unsigned int width,  
unsigned int flags ) [static]
```

Definition at line 618 of file [printf.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.153.3.3 _ftoa()

```
static size_t _ftoa (  
    out_fct_type out,
```

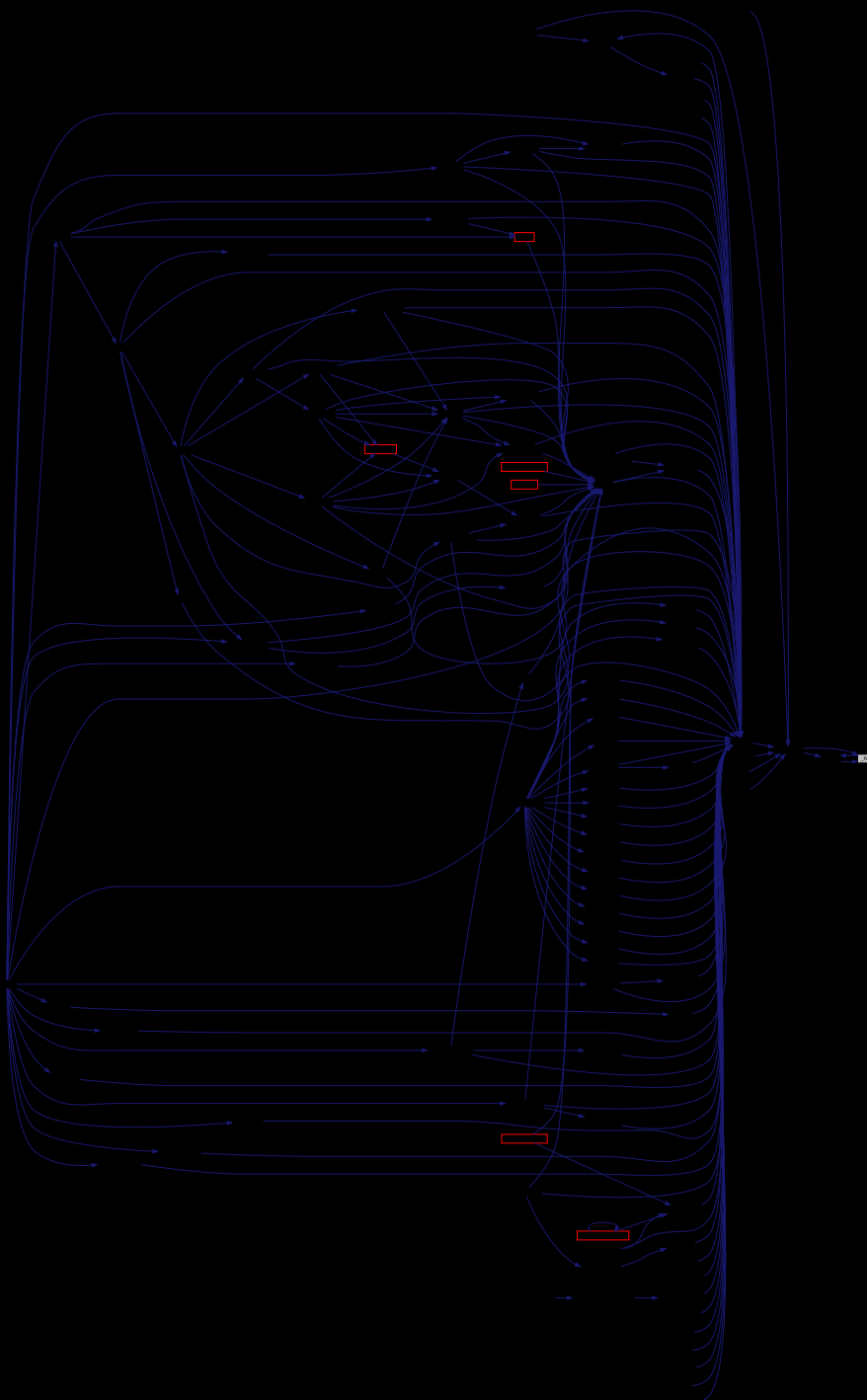
```
char * buffer,  
size_t idx,  
size_t maxlen,  
double value,  
unsigned int prec,  
unsigned int width,  
unsigned int flags ) [static]
```

Definition at line 442 of file [printf.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

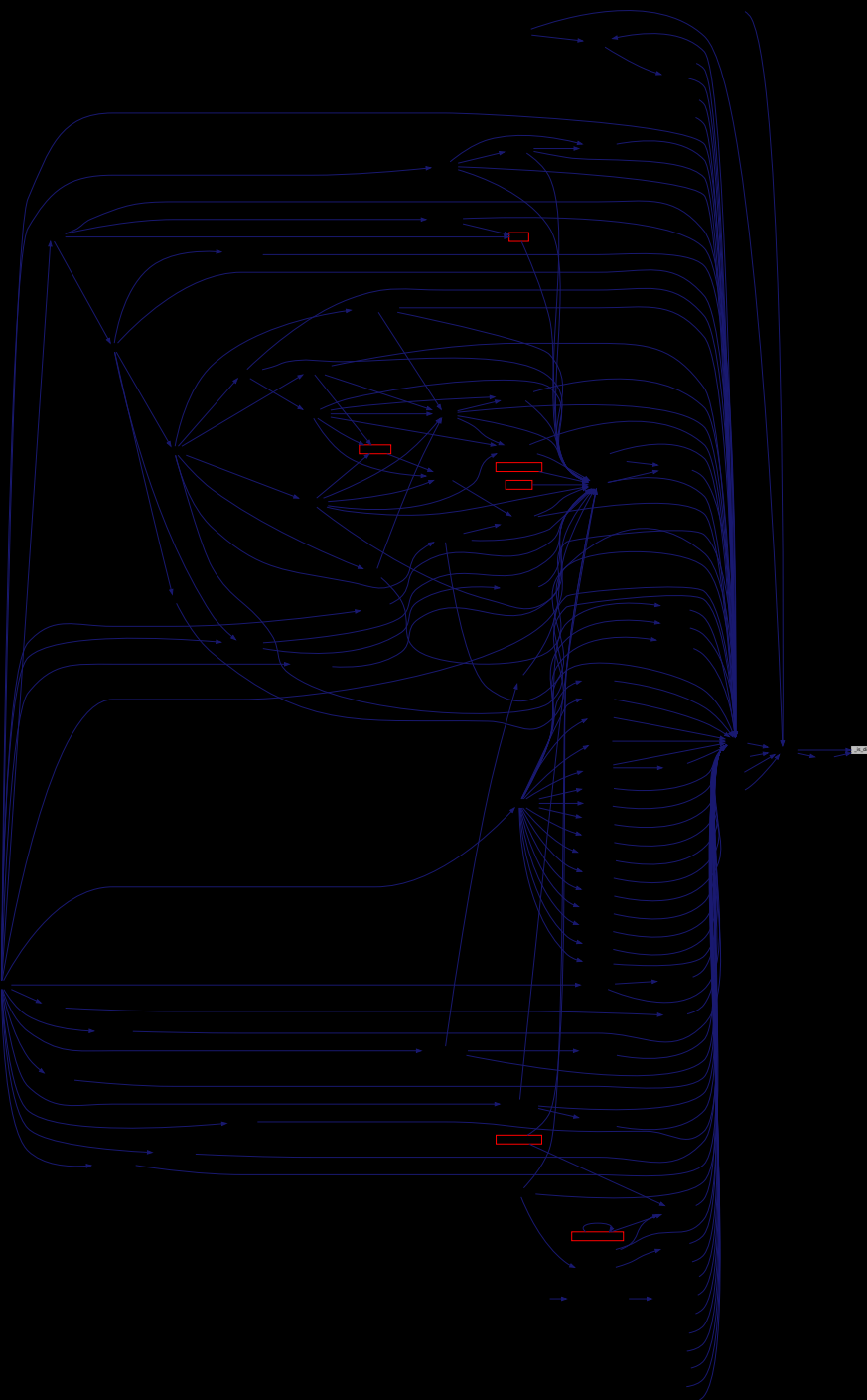


7.153.3.4 `_is_digit()`

```
static bool _is_digit (  
    char ch ) [inline], [static]
```

Definition at line 192 of file `printf.c`.

Here is the caller graph for this function:



7.153.3.5 `_ntoa_format()`

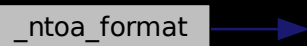
```
static size_t _ntoa_format (
    out_fct_type out,
```



```
char * buffer,  
size_t idx,  
size_t maxlen,  
char * buf,  
size_t len,  
bool negative,  
unsigned int base,  
unsigned int prec,  
unsigned int width,  
unsigned int flags ) [static]
```

Definition at line 248 of file [printf.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.153.3.6 `_ntoa_long()`

```
static size_t _ntoa_long (  
    out_fct_type out,  
    char * buffer,  
    size_t idx,  
    size_t maxlen,  
    ...  
)
```

```
unsigned long value,  
bool negative,  
unsigned long base,  
unsigned int prec,  
unsigned int width,  
unsigned int flags ) [static]
```

Definition at line 331 of file [printf.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.153.3.7 _ntoa_long_long()

```
static size_t _ntoa_long_long (
    out_fct_type out,
```

```
char * buffer,  
size_t idx,  
size_t maxlen,  
unsigned long long value,  
bool negative,  
unsigned long long base,  
unsigned int prec,  
unsigned int width,  
unsigned int flags ) [static]
```

Definition at line 379 of file [printf.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



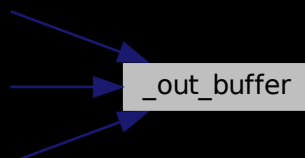
7.153.3.8 _out_buffer()

```
static void _out_buffer (  
    char character,  
    void * buffer,
```

```
size_t idx,  
size_t maxlen ) [inline], [static]
```

Definition at line 134 of file [printf.c](#).

Here is the caller graph for this function:

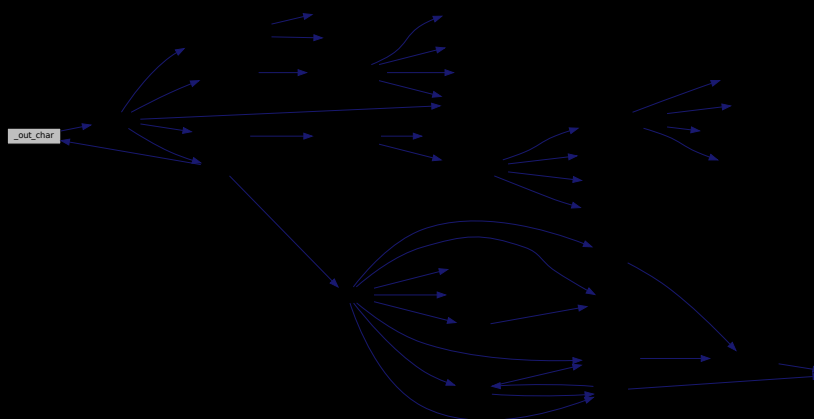


7.153.3.9 `_out_char()`

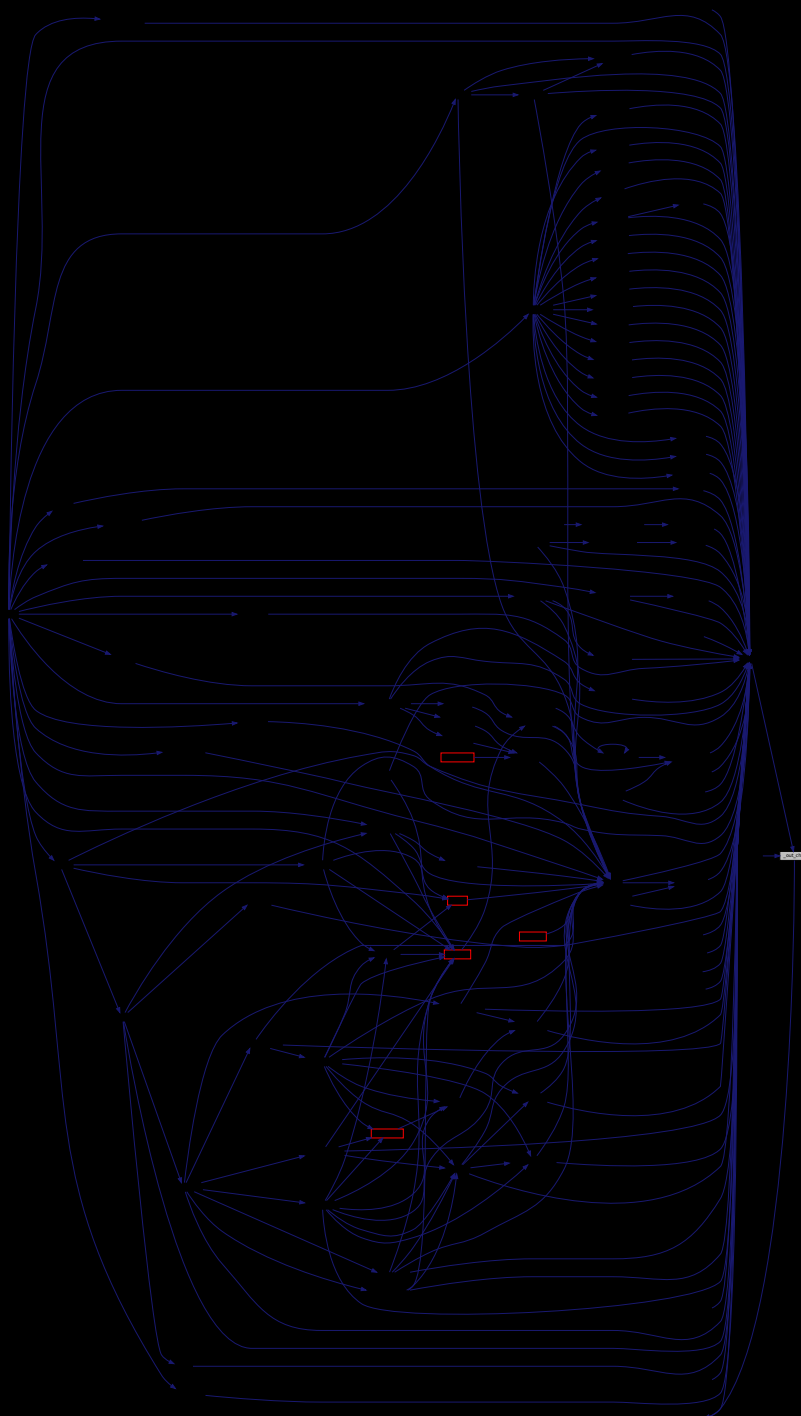
```
static void _out_char (  
    char character,  
    void * buffer,  
    size_t idx,  
    size_t maxlen ) [inline], [static]
```

Definition at line 154 of file [printf.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.153.3.10 `_out_fct()`

```
static void _out_fct (  
    char character,
```



```
void * buffer,  
size_t idx,  
size_t maxlen ) [inline], [static]
```

Definition at line 167 of file [printf.c](#).

Here is the caller graph for this function:

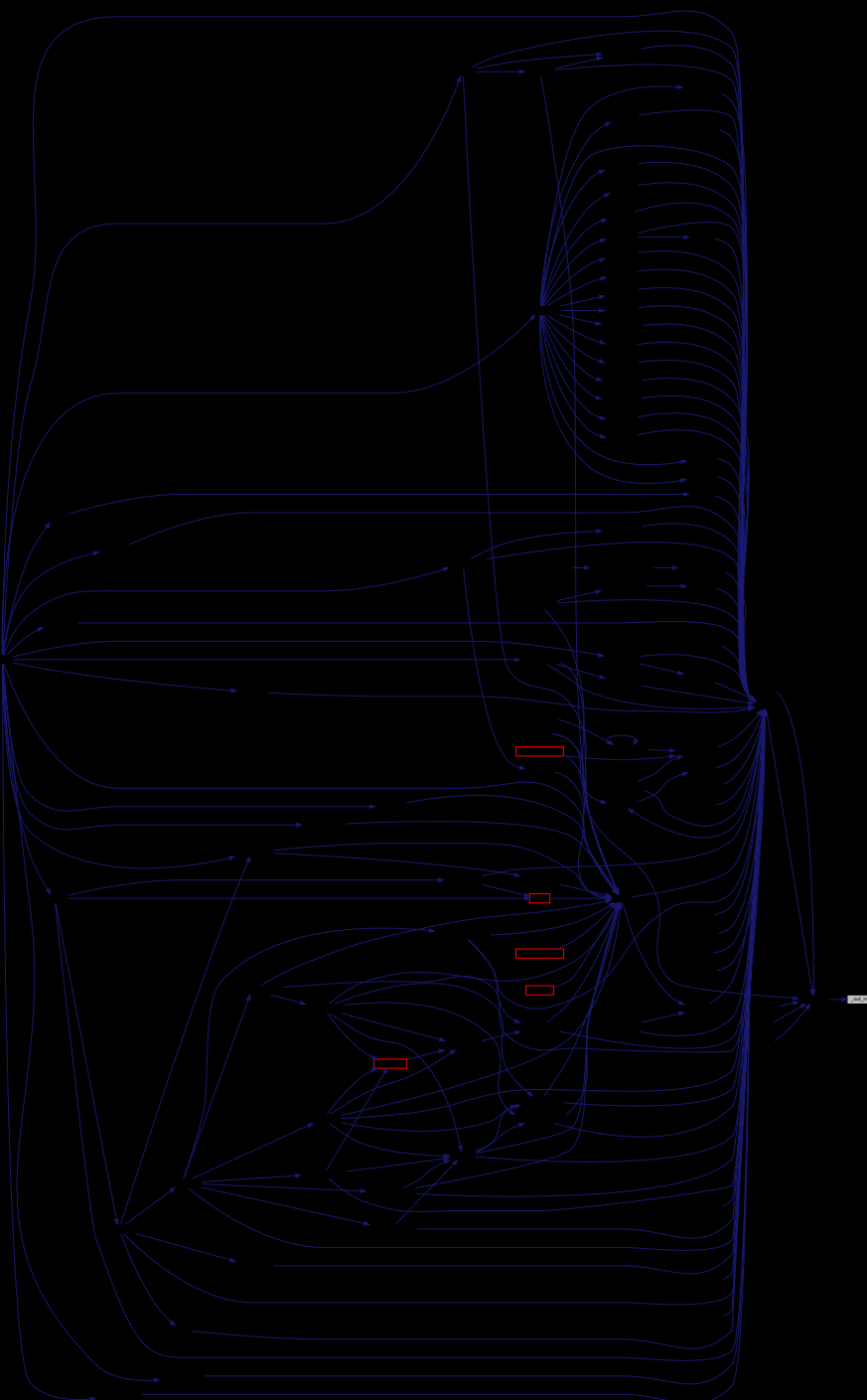


7.153.3.11 `_out_null()`

```
static void _out_null (  
    char character,  
    void * buffer,  
    size_t idx,  
    size_t maxlen ) [inline], [static]
```

Definition at line 144 of file [printf.c](#).

Here is the caller graph for this function:



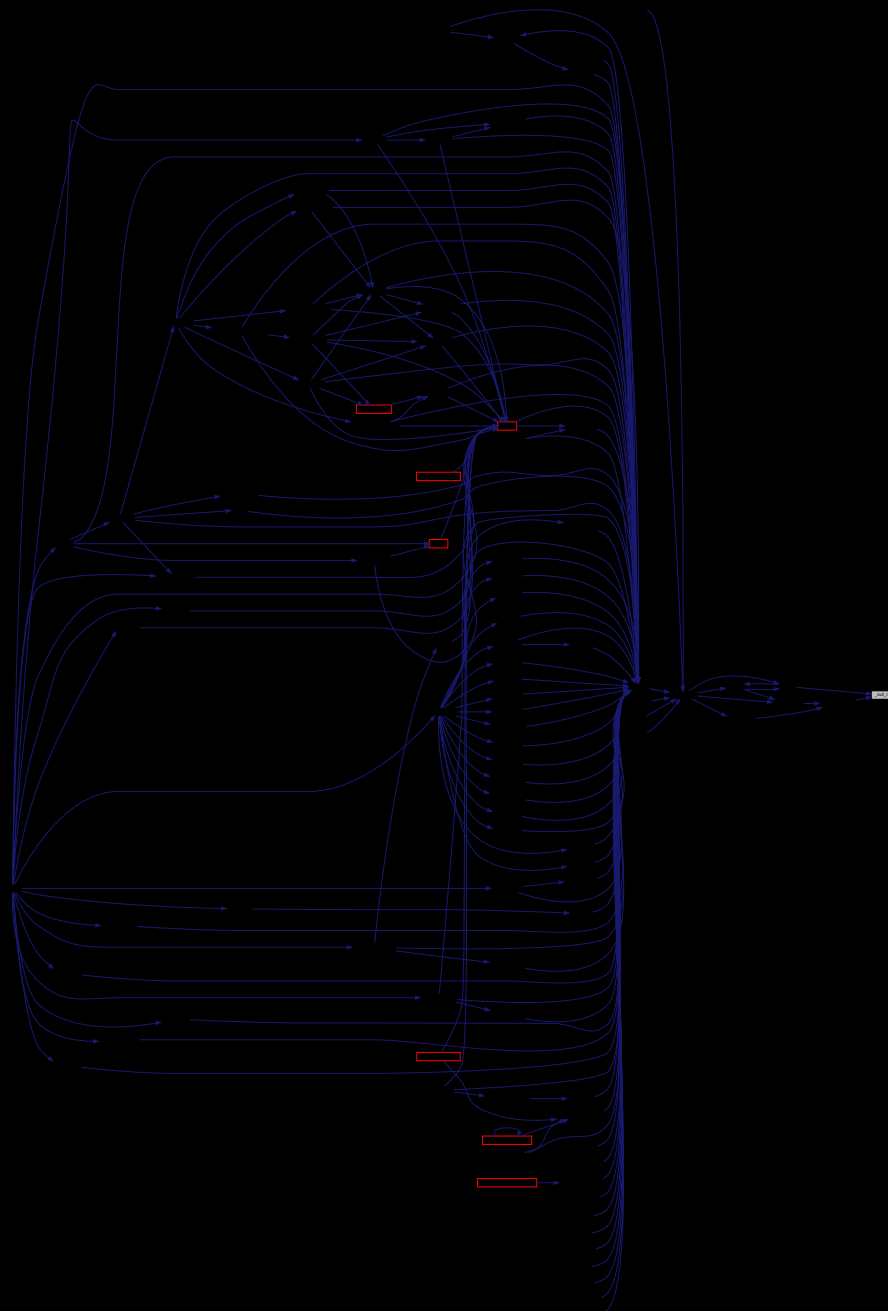
7.153.3.12 _out_rev()

```
static size_t _out_rev (
    out_fct_type out,
```

```
char * buffer,  
size_t idx,  
size_t maxlen,  
const char * buf,  
size_t len,  
unsigned int width,  
unsigned int flags ) [static]
```

Definition at line 209 of file [printf.c](#).

Here is the caller graph for this function:

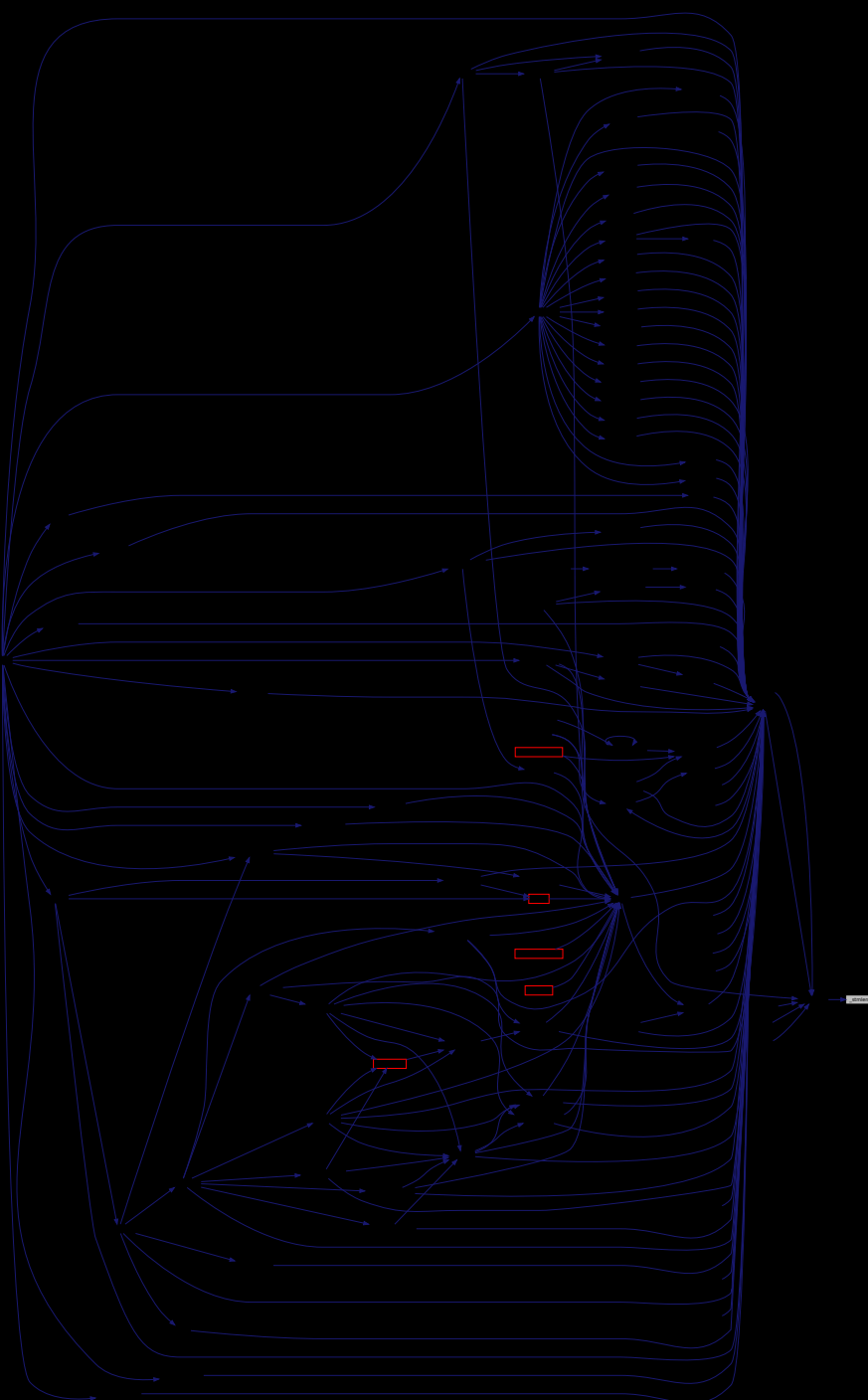


7.153.3.13 _strnlen_s()

```
static unsigned int _strnlen_s (  
    const char * str,  
    size_t maxsize ) [inline], [static]
```

Definition at line 182 of file [printf.c](#).

Here is the caller graph for this function:

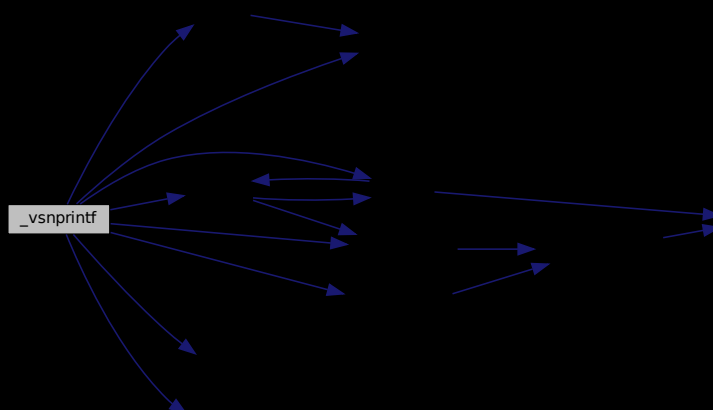


7.153.3.14 `_vsnprintf()`

```
static int _vsnprintf (  
    out_fct_type out,  
    char * buffer,  
    const size_t maxlen,  
    const char * format,  
    va_list va ) [static]
```

Definition at line 776 of file `printf.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.153.3.15 fctprintf()

```
int fctprintf (
    void(*) (char character, void *arg) out,
```

```
void * arg,  
const char * format,  
... )
```

printf with output function You may use this as dynamic alternative to `printf()` with its fixed `_putchar()` output

Parameters

<i>out</i>	An output function which takes one character and an argument pointer
<i>arg</i>	An argument pointer for user data passed to output function
<i>format</i>	A string that specifies the format of the output

Returns

The number of characters that are sent to the output function, not counting the terminating null character

Definition at line 1279 of file printf.c.

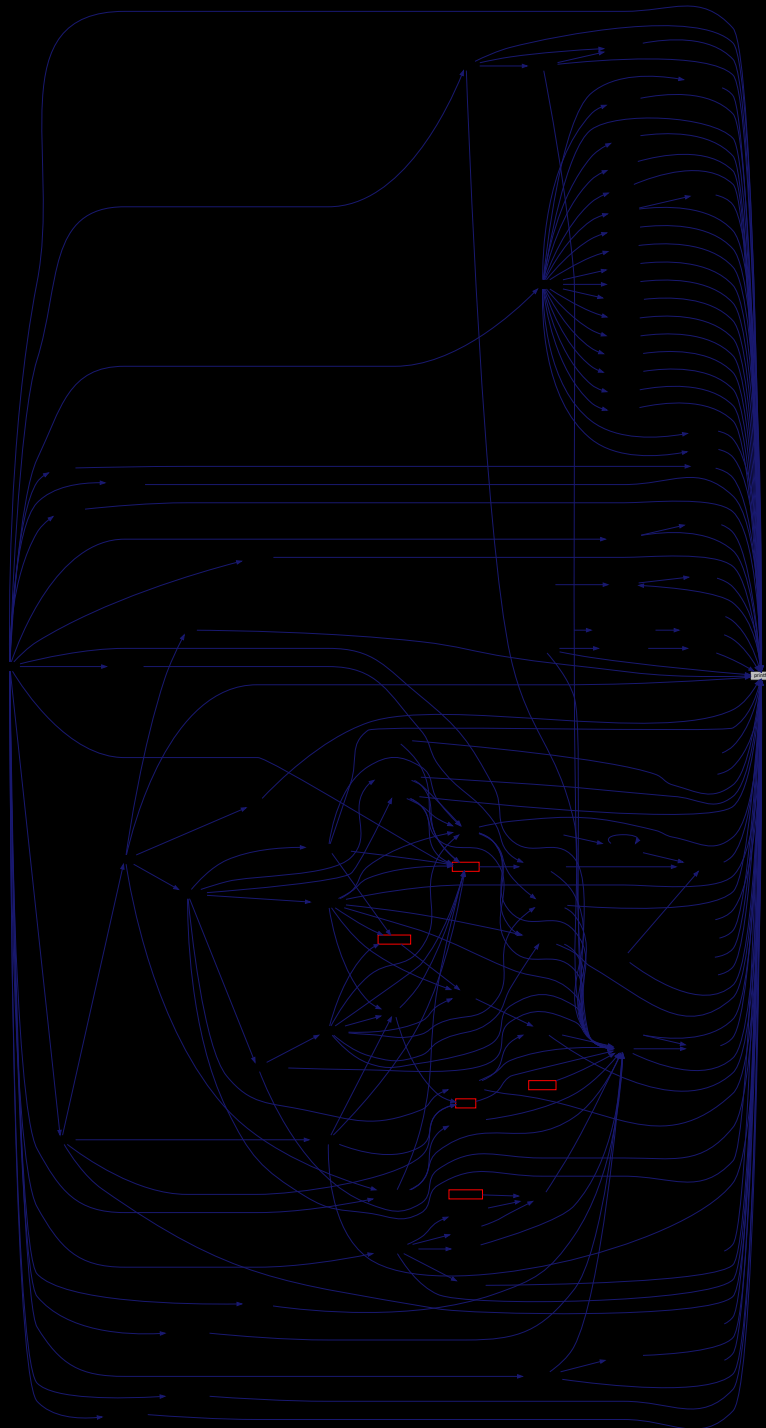
Here is the call graph for this function:

7.153.3.16 printf_()

```
int printf_ (
    const char * format,
    ... )
```

Definition at line 1240 of file printf.c.

Here is the caller graph for this function:



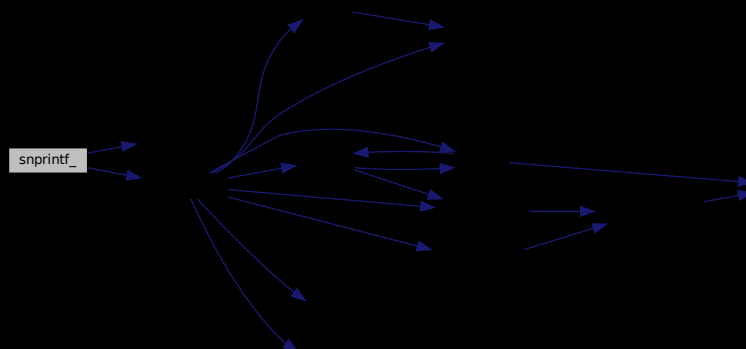
7.153.3.17 snprintf_()

```
int snprintf_ (  
    char * buffer,
```

```
size_t count,  
const char * format,  
... )
```

Definition at line 1259 of file [printf.c](#).

Here is the call graph for this function:

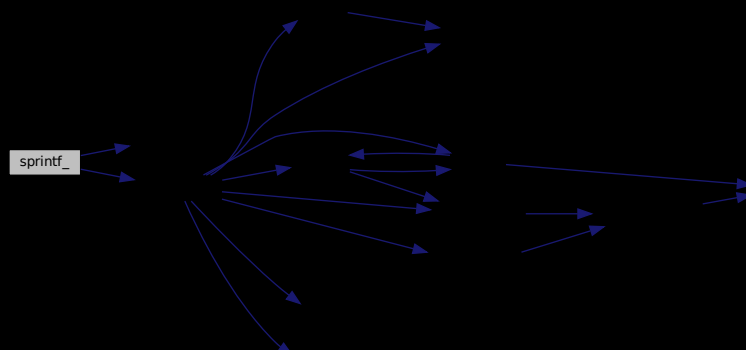


7.153.3.18 `sprintf_()`

```
int sprintf_ (  
    char * buffer,  
    const char * format,  
    ... )
```

Definition at line 1250 of file [printf.c](#).

Here is the call graph for this function:

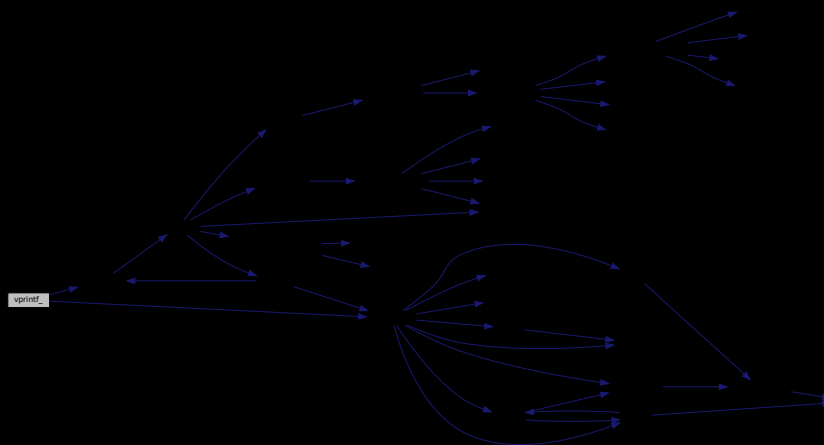


7.153.3.19 vprintf_()

```
int vprintf_ (  
    const char * format,  
    va_list va )
```

Definition at line 1268 of file [printf.c](#).

Here is the call graph for this function:

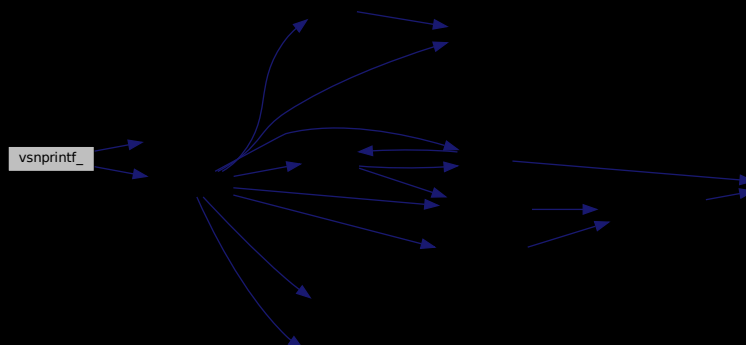


7.153.3.20 vsnprintf_()

```
int vsnprintf_ (  
    char * buffer,  
    size_t count,  
    const char * format,  
    va_list va )
```

Definition at line 1274 of file [printf.c](#).

Here is the call graph for this function:



7.154 printf.c

Go to the documentation of this file.

```

00001
00002 // \author (c) Marco Paland (info@paland.com)
00003 //           2014-2019, PALANDesign Hannover, Germany
00004 //
00005 // \license The MIT License (MIT)
00006 //
00007 // Permission is hereby granted, free of charge, to any person obtaining a copy
00008 // of this software and associated documentation files (the "Software"), to deal
00009 // in the Software without restriction, including without limitation the rights
00010 // to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00011 // copies of the Software, and to permit persons to whom the Software is
00012 // furnished to do so, subject to the following conditions:
00013 //
00014 // The above copyright notice and this permission notice shall be included in
00015 // all copies or substantial portions of the Software.
00016 //
00017 // THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00018 // IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00019 // FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00020 // AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00021 // LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00022 // OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
00023 // THE SOFTWARE.
00024 //
00025 // \brief Tiny printf, sprintf and (v)snprintf implementation, optimized for
00026 // speed on
00027 // embedded systems with a very limited resources. These routines are
00028 // thread safe and reentrant! Use this instead of the bloated
00029 // standard/newlib printf cause these use malloc for printf (and may not
00030 // be thread safe).
00031 //
00032 //
00033
00034 #include <stdbool.h>
00035 #include <stdint.h>
00036
00037 #include "include/printf.h"
00038 #include "include/serial.h"
00039
00040 // define this globally (e.g. gcc -DPRINTF_INCLUDE_CONFIG_H ...) to include the
00041 // printf_config.h header file
00042 // default: undefined
00043 #ifdef PRINTF_INCLUDE_CONFIG_H
00044 #include "printf_config.h"
00045 #endif
00046
00047 // 'ntoa' conversion buffer size, this must be big enough to hold one converted
00048 // numeric number including padded zeros (dynamically created on stack)
00049 // default: 32 byte
00050 #ifndef PRINTF_NTOA_BUFFER_SIZE
00051 #define PRINTF_NTOA_BUFFER_SIZE 32U
00052 #endif
00053
00054 // 'ftoa' conversion buffer size, this must be big enough to hold one converted
00055 // float number including padded zeros (dynamically created on stack)
00056 // default: 32 byte
00057 #ifndef PRINTF_FTOA_BUFFER_SIZE
00058 #define PRINTF_FTOA_BUFFER_SIZE 32U
00059 #endif
00060
00061 // support for the floating point type (%f)
00062 // default: activated
00063 #ifndef PRINTF_DISABLE_SUPPORT_FLOAT
00064 #define PRINTF_SUPPORT_FLOAT
00065 #endif
00066
00067 // support for exponential floating point notation (%e/%g)
00068 // default: activated
00069 #ifndef PRINTF_DISABLE_SUPPORT_EXPONENTIAL
00070 #define PRINTF_SUPPORT_EXPONENTIAL
00071 #endif
00072
00073 // define the default floating point precision
00074 // default: 6 digits
00075 #ifndef PRINTF_DEFAULT_FLOAT_PRECISION
00076 #define PRINTF_DEFAULT_FLOAT_PRECISION 6U
00077 #endif
00078
00079 // define the largest float suitable to print with %f
00080 // default: 1e9
00081 #ifndef PRINTF_MAX_FLOAT
00082 #define PRINTF_MAX_FLOAT 1e9
00083 #endif

```

```

00084
00085 // support for the long long types (%llu or %p)
00086 // default: activated
00087 #ifndef PRINTF_DISABLE_SUPPORT_LONG_LONG
00088 #define PRINTF_SUPPORT_LONG_LONG
00089 #endif
00090
00091 // support for the ptrdiff_t type (%t)
00092 // ptrdiff_t is normally defined in <stddef.h> as long or long long type
00093 // default: activated
00094 #ifndef PRINTF_DISABLE_SUPPORT_PTRDIFF_T
00095 #define PRINTF_SUPPORT_PTRDIFF_T
00096 #endif
00097
00099
00100 // internal flag definitions
00101 #define FLAGS_ZEROPAD (1U << 0U)
00102 #define FLAGS_LEFT (1U << 1U)
00103 #define FLAGS_PLUS (1U << 2U)
00104 #define FLAGS_SPACE (1U << 3U)
00105 #define FLAGS_HASH (1U << 4U)
00106 #define FLAGS_UPPERCASE (1U << 5U)
00107 #define FLAGS_CHAR (1U << 6U)
00108 #define FLAGS_SHORT (1U << 7U)
00109 #define FLAGS_LONG (1U << 8U)
00110 #define FLAGS_LONG_LONG (1U << 9U)
00111 #define FLAGS_PRECISION (1U << 10U)
00112 #define FLAGS_ADAPT_EXP (1U << 11U)
00113
00114 // import float.h for DBL_MAX
00115 #if defined(PRINTF_SUPPORT_FLOAT)
00116 #include <float.h>
00117 #endif
00118
00119 // output function type
00120 typedef void (*out_fct_type)(char character,
00121                             void *buffer,
00122                             size_t idx,
00123                             size_t maxlen);
00124
00125 // wrapper (used as buffer) for output function type
00126 typedef struct
00127 {
00128     void (*fct)(char character, void *arg);
00129     void *arg;
00130 } out_fct_wrap_type;
00131
00132 // internal buffer output
00133 static inline void
00134 _out_buffer(char character, void *buffer, size_t idx, size_t maxlen)
00135 {
00136     if (idx < maxlen)
00137     {
00138         ((char *) buffer)[idx] = character;
00139     }
00140 }
00141
00142 // internal null output
00143 static inline void
00144 _out_null(char character, void *buffer, size_t idx, size_t maxlen)
00145 {
00146     (void) character;
00147     (void) buffer;
00148     (void) idx;
00149     (void) maxlen;
00150 }
00151
00152 // internal _putchar wrapper
00153 static inline void
00154 _out_char(char character, void *buffer, size_t idx, size_t maxlen)
00155 {
00156     (void) buffer;
00157     (void) idx;
00158     (void) maxlen;
00159     if (character)
00160     {
00161         _putchar(character);
00162     }
00163 }
00164
00165 // internal output function wrapper
00166 static inline void
00167 _out_fct(char character, void *buffer, size_t idx, size_t maxlen)
00168 {
00169     (void) idx;
00170     (void) maxlen;
00171     if (character)

```

```

00172     {
00173         // buffer is the output fct pointer
00174         ((out_fct_wrap_type *) buffer)
00175         ->fct(character, ((out_fct_wrap_type *) buffer)->arg);
00176     }
00177 }
00178
00179 // internal secure strlen
00180 // \return The length of the string (excluding the terminating 0) limited by
00181 // 'maxsize'
00182 static inline unsigned int _strnlen_s(const char *str, size_t maxsize)
00183 {
00184     const char *s;
00185     for (s = str; *s && maxsize-- > 0; ++s)
00186         ;
00187     return (unsigned int) (s - str);
00188 }
00189
00190 // internal test if char is a digit (0-9)
00191 // \return true if char is a digit
00192 static inline bool _is_digit(char ch)
00193 {
00194     return (ch >= '0') && (ch <= '9');
00195 }
00196
00197 // internal ASCII string to unsigned int conversion
00198 static unsigned int _atoi(const char **str)
00199 {
00200     unsigned int i = 0;
00201     while (_is_digit(**str))
00202     {
00203         i = i * 10U + (unsigned int) ((*str)++ - '0');
00204     }
00205     return i;
00206 }
00207
00208 // output the specified string in reverse, taking care of any zero-padding
00209 static size_t _out_rev(out_fct_type out,
00210                       char *buffer,
00211                       size_t idx,
00212                       size_t maxlen,
00213                       const char *buf,
00214                       size_t len,
00215                       unsigned int width,
00216                       unsigned int flags)
00217 {
00218     const size_t start_idx = idx;
00219
00220     // pad spaces up to given width
00221     if (!(flags & FLAGS_LEFT) && !(flags & FLAGS_ZEROPAD))
00222     {
00223         for (size_t i = len; i < width; i++)
00224         {
00225             out(' ', buffer, idx++, maxlen);
00226         }
00227     }
00228
00229     // reverse string
00230     while (len)
00231     {
00232         out(buf[--len], buffer, idx++, maxlen);
00233     }
00234
00235     // append pad spaces up to given width
00236     if (flags & FLAGS_LEFT)
00237     {
00238         while (idx - start_idx < width)
00239         {
00240             out(' ', buffer, idx++, maxlen);
00241         }
00242     }
00243
00244     return idx;
00245 }
00246
00247 // internal itoa format
00248 static size_t _ntoa_format(out_fct_type out,
00249                           char *buffer,
00250                           size_t idx,
00251                           size_t maxlen,
00252                           char *buf,
00253                           size_t len,
00254                           bool negative,
00255                           unsigned int base,
00256                           unsigned int prec,
00257                           unsigned int width,
00258                           unsigned int flags)

```

```

00259 {
00260     // pad leading zeros
00261     if (!(flags & FLAGS_LEFT))
00262     {
00263         if (width && (flags & FLAGS_ZEROPAD) &&
00264             (negative || (flags & (FLAGS_PLUS | FLAGS_SPACE))))
00265         {
00266             width--;
00267         }
00268         while ((len < prec) && (len < PRINTF_NTOA_BUFFER_SIZE))
00269         {
00270             buf[len++] = '0';
00271         }
00272         while ((flags & FLAGS_ZEROPAD) && (len < width) &&
00273             (len < PRINTF_NTOA_BUFFER_SIZE))
00274         {
00275             buf[len++] = '0';
00276         }
00277     }
00278     // handle hash
00279     if (flags & FLAGS_HASH)
00280     {
00281         if (!(flags & FLAGS_PRECISION) && len &&
00282             ((len == prec) || (len == width)))
00283         {
00284             len--;
00285             if (len && (base == 16U))
00286             {
00287                 len--;
00288             }
00289         }
00290         if ((base == 16U) && !(flags & FLAGS_UPPERCASE) &&
00291             (len < PRINTF_NTOA_BUFFER_SIZE))
00292         {
00293             buf[len++] = 'x';
00294         }
00295         else if ((base == 16U) && (flags & FLAGS_UPPERCASE) &&
00296             (len < PRINTF_NTOA_BUFFER_SIZE))
00297         {
00298             buf[len++] = 'X';
00299         }
00300         else if ((base == 2U) && (len < PRINTF_NTOA_BUFFER_SIZE))
00301         {
00302             buf[len++] = 'b';
00303         }
00304         if (len < PRINTF_NTOA_BUFFER_SIZE)
00305         {
00306             buf[len++] = '0';
00307         }
00308     }
00309 }
00310
00311 if (len < PRINTF_NTOA_BUFFER_SIZE)
00312 {
00313     if (negative)
00314     {
00315         buf[len++] = '-';
00316     }
00317     else if (flags & FLAGS_PLUS)
00318     {
00319         buf[len++] = '+'; // ignore the space if the '+' exists
00320     }
00321     else if (flags & FLAGS_SPACE)
00322     {
00323         buf[len++] = ' ';
00324     }
00325 }
00326
00327 return _out_rev(out, buffer, idx, maxlen, buf, len, width, flags);
00328 }
00329
00330 // internal itoa for 'long' type
00331 static size_t _ntoa_long(out_fct_type out,
00332     char *buffer,
00333     size_t idx,
00334     size_t maxlen,
00335     unsigned long value,
00336     bool negative,
00337     unsigned long base,
00338     unsigned int prec,
00339     unsigned int width,
00340     unsigned int flags)
00341 {
00342     char buf[PRINTF_NTOA_BUFFER_SIZE];
00343     size_t len = 0;
00344
00345     // no hash for 0 values

```

```

00346     if (!value)
00347     {
00348         flags &= ~FLAGS_HASH;
00349     }
00350
00351     // write if precision != 0 and value is != 0
00352     if (!(flags & FLAGS_PRECISION) || value)
00353     {
00354         do
00355         {
00356             const char digit = (char) (value % base);
00357             buf[len++] =
00358                 digit < 10 ? '0' + digit
00359                 : (flags & FLAGS_UPPERCASE ? 'A' : 'a') + digit - 10;
00360             value /= base;
00361         } while (value && (len < PRINTF_NTOA_BUFFER_SIZE));
00362     }
00363
00364     return _ntoa_format(out,
00365                        buffer,
00366                        idx,
00367                        maxlen,
00368                        buf,
00369                        len,
00370                        negative,
00371                        (unsigned int) base,
00372                        prec,
00373                        width,
00374                        flags);
00375 }
00376
00377 // internal itoa for 'long long' type
00378 #if defined(PRINTF_SUPPORT_LONG_LONG)
00379 static size_t _ntoa_long_long(out_fct_type out,
00380                               char *buffer,
00381                               size_t idx,
00382                               size_t maxlen,
00383                               unsigned long long value,
00384                               bool negative,
00385                               unsigned long long base,
00386                               unsigned int prec,
00387                               unsigned int width,
00388                               unsigned int flags)
00389 {
00390     char buf[PRINTF_NTOA_BUFFER_SIZE];
00391     size_t len = 0U;
00392
00393     // no hash for 0 values
00394     if (!value)
00395     {
00396         flags &= ~FLAGS_HASH;
00397     }
00398
00399     // write if precision != 0 and value is != 0
00400     if (!(flags & FLAGS_PRECISION) || value)
00401     {
00402         do
00403         {
00404             const char digit = (char) (value % base);
00405             buf[len++] =
00406                 digit < 10 ? '0' + digit
00407                 : (flags & FLAGS_UPPERCASE ? 'A' : 'a') + digit - 10;
00408             value /= base;
00409         } while (value && (len < PRINTF_NTOA_BUFFER_SIZE));
00410     }
00411
00412     return _ntoa_format(out,
00413                        buffer,
00414                        idx,
00415                        maxlen,
00416                        buf,
00417                        len,
00418                        negative,
00419                        (unsigned int) base,
00420                        prec,
00421                        width,
00422                        flags);
00423 }
00424 #endif // PRINTF_SUPPORT_LONG_LONG
00425
00426 #if defined(PRINTF_SUPPORT_FLOAT)
00427
00428 #if defined(PRINTF_SUPPORT_EXPONENTIAL)
00429 // forward declaration so that _ftoa can switch to exp notation for values >
00430 // PRINTF_MAX_FLOAT
00431 static size_t _etoa(out_fct_type out,
00432                    char *buffer,

```



```

00433         size_t      idx,
00434         size_t      maxlen,
00435         double      value,
00436         unsigned int prec,
00437         unsigned int width,
00438         unsigned int flags);
00439 #endif
00440
00441 // internal ftoa for fixed decimal floating point
00442 static size_t _ftoa(out_fct_type out,
00443         char      *buffer,
00444         size_t      idx,
00445         size_t      maxlen,
00446         double      value,
00447         unsigned int prec,
00448         unsigned int width,
00449         unsigned int flags)
00450 {
00451     char buf[PRINTF_FTOA_BUFFER_SIZE];
00452     size_t len = 0U;
00453     double diff = 0.0;
00454
00455     // powers of 10
00456     static const double pow10[] = {1,
00457                                     10,
00458                                     100,
00459                                     1000,
00460                                     10000,
00461                                     100000,
00462                                     1000000,
00463                                     10000000,
00464                                     100000000,
00465                                     1000000000};
00466
00467     // test for special values
00468     if (value != value)
00469         return _out_rev(out, buffer, idx, maxlen, "nan", 3, width, flags);
00470     if (value < -DBL_MAX)
00471         return _out_rev(out, buffer, idx, maxlen, "fni-", 4, width, flags);
00472     if (value > DBL_MAX)
00473         return _out_rev(out,
00474                 buffer,
00475                 idx,
00476                 maxlen,
00477                 (flags & FLAGS_PLUS) ? "fni+" : "fni",
00478                 (flags & FLAGS_PLUS) ? 4U : 3U,
00479                 width,
00480                 flags);
00481
00482     // test for very large values
00483     // standard printf behavior is to print EVERY whole number digit -- which
00484     // could be 100s of characters overflowing your buffers == bad
00485     if ((value > PRINTF_MAX_FLOAT) || (value < -PRINTF_MAX_FLOAT))
00486     {
00487 #if defined(PRINTF_SUPPORT_EXPONENTIAL)
00488         return _etoa(out, buffer, idx, maxlen, value, prec, width, flags);
00489 #else
00490         return 0U;
00491 #endif
00492     }
00493
00494     // test for negative
00495     bool negative = false;
00496     if (value < 0)
00497     {
00498         negative = true;
00499         value = 0 - value;
00500     }
00501
00502     // set default precision, if not set explicitly
00503     if (!(flags & FLAGS_PRECISION))
00504     {
00505         prec = PRINTF_DEFAULT_FLOAT_PRECISION;
00506     }
00507     // limit precision to 9, cause a prec >= 10 can lead to overflow errors
00508     while ((len < PRINTF_FTOA_BUFFER_SIZE) && (prec > 9U))
00509     {
00510         buf[len++] = '0';
00511         prec--;
00512     }
00513
00514     int whole = (int) value;
00515     double tmp = (value - whole) * pow10[prec];
00516     unsigned long frac = (unsigned long) tmp;
00517     diff = tmp - frac;
00518
00519     if (diff > 0.5)

```

```

00520     {
00521         ++frac;
00522         // handle rollover, e.g. case 0.99 with prec 1 is 1.0
00523         if (frac >= pow10[prec])
00524         {
00525             frac = 0;
00526             ++whole;
00527         }
00528     }
00529     else if (diff < 0.5)
00530     {
00531     }
00532     else if ((frac == 0U) || (frac & 1U))
00533     {
00534         // if halfway, round up if odd OR if last digit is 0
00535         ++frac;
00536     }
00537
00538     if (prec == 0U)
00539     {
00540         diff = value - (double) whole;
00541         if (!(diff < 0.5) || (diff > 0.5)) && (whole & 1)
00542         {
00543             // exactly 0.5 and ODD, then round up
00544             // 1.5 -> 2, but 2.5 -> 2
00545             ++whole;
00546         }
00547     }
00548     else
00549     {
00550         unsigned int count = prec;
00551         // now do fractional part, as an unsigned number
00552         while (len < PRINTF_FTOA_BUFFER_SIZE)
00553         {
00554             --count;
00555             buf[len++] = (char) (48U + (frac % 10U));
00556             if (!(frac /= 10U))
00557             {
00558                 break;
00559             }
00560         }
00561         // add extra 0s
00562         while ((len < PRINTF_FTOA_BUFFER_SIZE) && (count-- > 0U))
00563         {
00564             buf[len++] = '0';
00565         }
00566         if (len < PRINTF_FTOA_BUFFER_SIZE)
00567         {
00568             // add decimal
00569             buf[len++] = '.';
00570         }
00571     }
00572
00573     // do whole part, number is reversed
00574     while (len < PRINTF_FTOA_BUFFER_SIZE)
00575     {
00576         buf[len++] = (char) (48 + (whole % 10));
00577         if (!(whole /= 10))
00578         {
00579             break;
00580         }
00581     }
00582
00583     // pad leading zeros
00584     if (!(flags & FLAGS_LEFT) && (flags & (FLAGS_ZEROPAD)))
00585     {
00586         if (width && (negative || (flags & (FLAGS_PLUS | FLAGS_SPACE))))
00587         {
00588             width--;
00589         }
00590         while ((len < width) && (len < PRINTF_FTOA_BUFFER_SIZE))
00591         {
00592             buf[len++] = '0';
00593         }
00594     }
00595
00596     if (len < PRINTF_FTOA_BUFFER_SIZE)
00597     {
00598         if (negative)
00599         {
00600             buf[len++] = '-';
00601         }
00602         else if (flags & FLAGS_PLUS)
00603         {
00604             buf[len++] = '+'; // ignore the space if the '+' exists
00605         }
00606         else if (flags & FLAGS_SPACE)

```

```

00607     {
00608         buf[len++] = ' ';
00609     }
00610 }
00611
00612 return _out_rev(out, buffer, idx, maxlen, buf, len, width, flags);
00613 }
00614
00615 #if defined(PRINTF_SUPPORT_EXPONENTIAL)
00616 // internal ftoa variant for exponential floating-point type, contributed by
00617 // Martijn Jasperse <m.jasperse@gmail.com>
00618 static size_t _etoa(out_fct_type out,
00619     char *buffer,
00620     size_t idx,
00621     size_t maxlen,
00622     double value,
00623     unsigned int prec,
00624     unsigned int width,
00625     unsigned int flags)
00626 {
00627     // check for NaN and special values
00628     if ((value != value) || (value > DBL_MAX) || (value < -DBL_MAX))
00629     {
00630         return _ftoa(out, buffer, idx, maxlen, value, prec, width, flags);
00631     }
00632
00633     // determine the sign
00634     const bool negative = value < 0;
00635     if (negative)
00636     {
00637         value = -value;
00638     }
00639
00640     // default precision
00641     if (!(flags & FLAGS_PRECISION))
00642     {
00643         prec = PRINTF_DEFAULT_FLOAT_PRECISION;
00644     }
00645
00646     // determine the decimal exponent
00647     // based on the algorithm by David Gay
00648     // (https://www.ampl.com/netlib/fp/dtoa.c)
00649     union
00650     {
00651         uint64_t U;
00652         double F;
00653     } conv;
00654
00655     conv.F = value;
00656     int exp2 = (int) ((conv.U » 52U) & 0x07FFU) - 1023; // effectively log2
00657     conv.U = (conv.U & ((1ULL « 52U) - 1U)) |
00658         (1023ULL « 52U); // drop the exponent so conv.F is now in [1,2)
00659     // now approximate log10 from the log2 integer part and an expansion of ln
00660     // around 1.5
00661     int expval = (int) (0.1760912590558 + exp2 * 0.301029995663981 +
00662         (conv.F - 1.5) * 0.289529654602168);
00663     // now we want to compute 10^expval but we want to be sure it won't overflow
00664     exp2 = (int) (expval * 3.321928094887362 + 0.5);
00665     const double z = expval * 2.302585092994046 - exp2 * 0.6931471805599453;
00666     const double z2 = z * z;
00667     conv.U = (uint64_t) (exp2 + 1023) « 52U;
00668     // compute exp(z) using continued fractions, see
00669     // https://en.wikipedia.org/wiki/Exponential_function#Continued_fractions_for_ex
00670     conv.F *= 1 + 2 * z / (2 - z + (z2 / (6 + (z2 / (10 + z2 / 14)))));
00671     // correct for rounding errors
00672     if (value < conv.F)
00673     {
00674         expval--;
00675         conv.F /= 10;
00676     }
00677
00678     // the exponent format is "%+03d" and largest value is "307", so set aside
00679     // 4-5 characters
00680     unsigned int minwidth = ((expval < 100) && (expval > -100)) ? 4U : 5U;
00681
00682     // in "%g" mode, "prec" is the number of *significant figures* not decimals
00683     if (flags & FLAGS_ADAPT_EXP)
00684     {
00685         // do we want to fall-back to "%f" mode?
00686         if ((value >= 1e-4) && (value < 1e6))
00687         {
00688             if ((int) prec > expval)
00689             {
00690                 prec = (unsigned) ((int) prec - expval - 1);
00691             }
00692             else
00693             {

```

```

00694         prec = 0;
00695     }
00696     flags |= FLAGS_PRECISION; // make sure _ftoa respects precision
00697     // no characters in exponent
00698     minwidth = 0U;
00699     expval = 0;
00700 }
00701 else
00702 {
00703     // we use one sigfig for the whole part
00704     if ((prec > 0) && (flags & FLAGS_PRECISION))
00705     {
00706         --prec;
00707     }
00708 }
00709 }
00710
00711 // will everything fit?
00712 unsigned int fwidth = width;
00713 if (width > minwidth)
00714 {
00715     // we didn't fall-back so subtract the characters required for the
00716     // exponent
00717     fwidth -= minwidth;
00718 }
00719 else
00720 {
00721     // not enough characters, so go back to default sizing
00722     fwidth = 0U;
00723 }
00724 if ((flags & FLAGS_LEFT) && minwidth)
00725 {
00726     // if we're padding on the right, DON'T pad the floating part
00727     fwidth = 0U;
00728 }
00729
00730 // rescale the float value
00731 if (expval)
00732 {
00733     value /= conv.F;
00734 }
00735
00736 // output the floating part
00737 const size_t start_idx = idx;
00738 idx = _ftoa(out,
00739             buffer,
00740             idx,
00741             maxlen,
00742             negative ? -value : value,
00743             prec,
00744             fwidth,
00745             flags & ~FLAGS_ADAPT_EXP);
00746
00747 // output the exponent part
00748 if (minwidth)
00749 {
00750     // output the exponential symbol
00751     out((flags & FLAGS_UPPERCASE) ? 'E' : 'e', buffer, idx++, maxlen);
00752     // output the exponent value
00753     idx = _ntoa_long(out,
00754                     buffer,
00755                     idx,
00756                     maxlen,
00757                     (expval < 0) ? -expval : expval,
00758                     expval < 0,
00759                     10,
00760                     0,
00761                     minwidth - 1,
00762                     FLAGS_ZEROPAD | FLAGS_PLUS);
00763     // might need to right-pad spaces
00764     if (flags & FLAGS_LEFT)
00765     {
00766         while (idx - start_idx < width)
00767             out(' ', buffer, idx++, maxlen);
00768     }
00769 }
00770 return idx;
00771 }
00772 #endif // PRINTF_SUPPORT_EXPONENTIAL
00773 #endif // PRINTF_SUPPORT_FLOAT
00774
00775 // internal vsnprintf
00776 static int _vsnprintf(out_fct_type out,
00777                      char *buffer,
00778                      const size_t maxlen,
00779                      const char *format,
00780                      va_list va)

```

```

00781 {
00782     unsigned int flags, width, precision, n;
00783     size_t      idx = 0U;
00784
00785     if (!buffer)
00786     {
00787         // use null output function
00788         out = _out_null;
00789     }
00790
00791     while (*format)
00792     {
00793         // format specifier?    %[flags][width][.precision][length]
00794         if (*format != '%')
00795         {
00796             // no
00797             out(*format, buffer, idx++, maxlen);
00798             format++;
00799             continue;
00800         }
00801         else
00802         {
00803             // yes, evaluate it
00804             format++;
00805         }
00806
00807         // evaluate flags
00808         flags = 0U;
00809         do
00810         {
00811             switch (*format)
00812             {
00813                 case '0':
00814                     flags |= FLAGS_ZEROPAD;
00815                     format++;
00816                     n = 1U;
00817                     break;
00818                 case '-':
00819                     flags |= FLAGS_LEFT;
00820                     format++;
00821                     n = 1U;
00822                     break;
00823                 case '+':
00824                     flags |= FLAGS_PLUS;
00825                     format++;
00826                     n = 1U;
00827                     break;
00828                 case ' ':
00829                     flags |= FLAGS_SPACE;
00830                     format++;
00831                     n = 1U;
00832                     break;
00833                 case '#':
00834                     flags |= FLAGS_HASH;
00835                     format++;
00836                     n = 1U;
00837                     break;
00838                 default:
00839                     n = 0U;
00840                     break;
00841             }
00842         } while (n);
00843
00844         // evaluate width field
00845         width = 0U;
00846         if (_is_digit(*format))
00847         {
00848             width = _atoi(&format);
00849         }
00850         else if (*format == '*')
00851         {
00852             const int w = va_arg(va, int);
00853             if (w < 0)
00854             {
00855                 flags |= FLAGS_LEFT; // reverse padding
00856                 width = (unsigned int) -w;
00857             }
00858             else
00859             {
00860                 width = (unsigned int) w;
00861             }
00862             format++;
00863         }
00864
00865         // evaluate precision field
00866         precision = 0U;
00867         if (*format == '.')

```

```

00868     {
00869         flags |= FLAGS_PRECISION;
00870         format++;
00871         if (_is_digit(*format))
00872         {
00873             precision = _atoi(&format);
00874         }
00875         else if (*format == '*')
00876         {
00877             const int prec = (int) va_arg(va, int);
00878             precision = prec > 0 ? (unsigned int) prec : 0U;
00879             format++;
00880         }
00881     }
00882
00883     // evaluate length field
00884     switch (*format)
00885     {
00886     case 'l':
00887         flags |= FLAGS_LONG;
00888         format++;
00889         if (*format == 'l')
00890         {
00891             flags |= FLAGS_LONG_LONG;
00892             format++;
00893         }
00894         break;
00895     case 'h':
00896         flags |= FLAGS_SHORT;
00897         format++;
00898         if (*format == 'h')
00899         {
00900             flags |= FLAGS_CHAR;
00901             format++;
00902         }
00903         break;
00904 #if defined(PRINTF_SUPPORT_PTRDIFF_T)
00905     case 't':
00906         flags |= (sizeof(ptrdiff_t) == sizeof(long) ? FLAGS_LONG
00907                  : FLAGS_LONG_LONG);
00908         format++;
00909         break;
00910 #endif
00911     case 'j':
00912         flags |= (sizeof(intmax_t) == sizeof(long) ? FLAGS_LONG
00913                  : FLAGS_LONG_LONG);
00914         format++;
00915         break;
00916     case 'z':
00917         flags |= (sizeof(size_t) == sizeof(long) ? FLAGS_LONG
00918                  : FLAGS_LONG_LONG);
00919         format++;
00920         break;
00921     default:
00922         break;
00923     }
00924
00925     // evaluate specifier
00926     switch (*format)
00927     {
00928     case 'd':
00929     case 'i':
00930     case 'u':
00931     case 'x':
00932     case 'X':
00933     case 'o':
00934     case 'b':
00935     {
00936         // set the base
00937         unsigned int base;
00938         if (*format == 'x' || *format == 'X')
00939         {
00940             base = 16U;
00941         }
00942         else if (*format == 'o')
00943         {
00944             base = 8U;
00945         }
00946         else if (*format == 'b')
00947         {
00948             base = 2U;
00949         }
00950         else
00951         {
00952             base = 10U;
00953             flags &= ~FLAGS_HASH; // no hash for dec format
00954         }

```

```

00955         // uppercase
00956         if (*format == 'X')
00957         {
00958             flags |= FLAGS_UPPERCASE;
00959         }
00960
00961         // no plus or space flag for u, x, X, o, b
00962         if ((*format != 'i') && (*format != 'd'))
00963         {
00964             flags &= ~(FLAGS_PLUS | FLAGS_SPACE);
00965         }
00966
00967         // ignore '0' flag when precision is given
00968         if (flags & FLAGS_PRECISION)
00969         {
00970             flags &= ~FLAGS_ZEROPAD;
00971         }
00972
00973         // convert the integer
00974         if ((*format == 'i') || (*format == 'd'))
00975         {
00976             // signed
00977             if (flags & FLAGS_LONG_LONG)
00978             {
00979                 #if defined(PRINTF_SUPPORT_LONG_LONG)
00980                 const long long value = va_arg(va, long long);
00981                 idx = _ntoa_long_long(
00982                     out,
00983                     buffer,
00984                     idx,
00985                     maxlen,
00986                     (unsigned long long) (value > 0 ? value
00987                                           : 0 - value),
00988                     value < 0,
00989                     base,
00990                     precision,
00991                     width,
00992                     flags);
00993                 #endif
00994             }
00995             else if (flags & FLAGS_LONG)
00996             {
00997                 const long value = va_arg(va, long);
00998                 idx = _ntoa_long(
00999                     out,
01000                     buffer,
01001                     idx,
01002                     maxlen,
01003                     (unsigned long) (value > 0 ? value : 0 - value),
01004                     value < 0,
01005                     base,
01006                     precision,
01007                     width,
01008                     flags);
01009             }
01010             else
01011             {
01012                 const int value = (flags & FLAGS_CHAR)
01013                                     ? (char) va_arg(va, int)
01014                                     : (flags & FLAGS_SHORT)
01015                                         ? (short int) va_arg(va, int)
01016                                         : va_arg(va, int);
01017                 idx = _ntoa_long(
01018                     out,
01019                     buffer,
01020                     idx,
01021                     maxlen,
01022                     (unsigned int) (value > 0 ? value : 0 - value),
01023                     value < 0,
01024                     base,
01025                     precision,
01026                     width,
01027                     flags);
01028             }
01029         }
01030         else
01031         {
01032             // unsigned
01033             if (flags & FLAGS_LONG_LONG)
01034             {
01035                 #if defined(PRINTF_SUPPORT_LONG_LONG)
01036                 idx = _ntoa_long_long(out,
01037                                     buffer,
01038                                     idx,
01039                                     maxlen,
01040                                     va_arg(va, unsigned long long),
01041                                     false,

```

```

01042                                     base,
01043                                     precision,
01044                                     width,
01045                                     flags);
01046 #endif
01047     }
01048     else if (flags & FLAGS_LONG)
01049     {
01050         idx = _ntoa_long(out,
01051                         buffer,
01052                         idx,
01053                         maxlen,
01054                         va_arg(va, unsigned long),
01055                         false,
01056                         base,
01057                         precision,
01058                         width,
01059                         flags);
01060     }
01061     else
01062     {
01063         const unsigned int value =
01064             (flags & FLAGS_CHAR)
01065             ? (unsigned char) va_arg(va, unsigned int)
01066             : (flags & FLAGS_SHORT)
01067             ? (unsigned short int) va_arg(va, unsigned int)
01068             : va_arg(va, unsigned int);
01069         idx = _ntoa_long(out,
01070                         buffer,
01071                         idx,
01072                         maxlen,
01073                         value,
01074                         false,
01075                         base,
01076                         precision,
01077                         width,
01078                         flags);
01079     }
01080     }
01081     format++;
01082     break;
01083 }
01084 #if defined(PRINTF_SUPPORT_FLOAT)
01085 case 'f':
01086 case 'F':
01087     if (*format == 'F')
01088         flags |= FLAGS_UPPERCASE;
01089     idx = _ftoa(out,
01090                buffer,
01091                idx,
01092                maxlen,
01093                va_arg(va, double),
01094                precision,
01095                width,
01096                flags);
01097     format++;
01098     break;
01099 #if defined(PRINTF_SUPPORT_EXPONENTIAL)
01100 case 'e':
01101 case 'E':
01102 case 'g':
01103 case 'G':
01104     if ((*format == 'g') || (*format == 'G'))
01105         flags |= FLAGS_ADAPT_EXP;
01106     if ((*format == 'E') || (*format == 'G'))
01107         flags |= FLAGS_UPPERCASE;
01108     idx = _etoa(out,
01109                 buffer,
01110                 idx,
01111                 maxlen,
01112                 va_arg(va, double),
01113                 precision,
01114                 width,
01115                 flags);
01116     format++;
01117     break;
01118 #endif // PRINTF_SUPPORT_EXPONENTIAL
01119 #endif // PRINTF_SUPPORT_FLOAT
01120 case 'c':
01121     {
01122         unsigned int l = 1U;
01123         // pre padding
01124         if (!(flags & FLAGS_LEFT))
01125         {
01126             while (l++ < width)
01127             {
01128                 out(' ', buffer, idx++, maxlen);

```



```

01129         }
01130     }
01131     // char output
01132     out((char) va_arg(va, int), buffer, idx++, maxlen);
01133     // post padding
01134     if (flags & FLAGS_LEFT)
01135     {
01136         while (l++ < width)
01137         {
01138             out(' ', buffer, idx++, maxlen);
01139         }
01140     }
01141     format++;
01142     break;
01143 }
01144
01145 case 's':
01146 {
01147     const char *p = va_arg(va, char *);
01148     unsigned int l =
01149         _strlen_s(p, precision ? precision : (size_t) -1);
01150     // pre padding
01151     if (flags & FLAGS_PRECISION)
01152     {
01153         l = (l < precision ? l : precision);
01154     }
01155     if (!(flags & FLAGS_LEFT))
01156     {
01157         while (l++ < width)
01158         {
01159             out(' ', buffer, idx++, maxlen);
01160         }
01161     }
01162     // string output
01163     while ((*p != 0) && (!(flags & FLAGS_PRECISION) || precision--))
01164     {
01165         out(*(p++), buffer, idx++, maxlen);
01166     }
01167     // post padding
01168     if (flags & FLAGS_LEFT)
01169     {
01170         while (l++ < width)
01171         {
01172             out(' ', buffer, idx++, maxlen);
01173         }
01174     }
01175     format++;
01176     break;
01177 }
01178
01179 case 'p':
01180 {
01181     width = sizeof(void *) * 2U;
01182     flags |= FLAGS_ZEROPAD | FLAGS_UPPERCASE;
01183 #if defined(PRINTF_SUPPORT_LONG_LONG)
01184     const bool is_ll = sizeof(uintptr_t) == sizeof(long long);
01185     if (is_ll)
01186     {
01187         idx = _ntoa_long_long(out,
01188                               buffer,
01189                               idx,
01190                               maxlen,
01191                               (uintptr_t) va_arg(va, void *),
01192                               false,
01193                               16U,
01194                               precision,
01195                               width,
01196                               flags);
01197     }
01198     else
01199     {
01200 #endif
01201         idx = _ntoa_long(
01202             out,
01203             buffer,
01204             idx,
01205             maxlen,
01206             (unsigned long) ((uintptr_t) va_arg(va, void *)),
01207             false,
01208             16U,
01209             precision,
01210             width,
01211             flags);
01212 #if defined(PRINTF_SUPPORT_LONG_LONG)
01213     }
01214 #endif
01215     format++;

```

```

01216         break;
01217     }
01218
01219     case '%':
01220         out('%', buffer, idx++, maxlen);
01221         format++;
01222         break;
01223
01224     default:
01225         out(*format, buffer, idx++, maxlen);
01226         format++;
01227         break;
01228     }
01229 }
01230
01231 // termination
01232 out((char) 0, buffer, idx < maxlen ? idx : maxlen - 1U, maxlen);
01233
01234 // return written chars without terminating \0
01235 return (int) idx;
01236 }
01237
01238
01239
01240 int printf(const char *format, ...)
01241 {
01242     va_list va;
01243     va_start(va, format);
01244     char buffer[1];
01245     const int ret = _vsnprintf(_out_char, buffer, (size_t) -1, format, va);
01246     va_end(va);
01247     return ret;
01248 }
01249
01250 int sprintf(char *buffer, const char *format, ...)
01251 {
01252     va_list va;
01253     va_start(va, format);
01254     const int ret = _vsnprintf(_out_buffer, buffer, (size_t) -1, format, va);
01255     va_end(va);
01256     return ret;
01257 }
01258
01259 int snprintf(char *buffer, size_t count, const char *format, ...)
01260 {
01261     va_list va;
01262     va_start(va, format);
01263     const int ret = _vsnprintf(_out_buffer, buffer, count, format, va);
01264     va_end(va);
01265     return ret;
01266 }
01267
01268 int vprintf(const char *format, va_list va)
01269 {
01270     char buffer[1];
01271     return _vsnprintf(_out_char, buffer, (size_t) -1, format, va);
01272 }
01273
01274 int vsnprintf(char *buffer, size_t count, const char *format, va_list va)
01275 {
01276     return _vsnprintf(_out_buffer, buffer, count, format, va);
01277 }
01278
01279 int fctprintf(void (*out)(char character, void *arg),
01280             void *arg,
01281             const char *format,
01282             ...)
01283 {
01284     va_list va;
01285     va_start(va, format);
01286     const out_fct_wrap_type out_fct_wrap = {out, arg};
01287     const int ret = _vsnprintf(
01288         _out_fct, (char *) (uintptr_t) &out_fct_wrap, (size_t) -1, format, va);
01289     va_end(va);
01290     return ret;
01291 }

```

7.155 src/proc.c File Reference

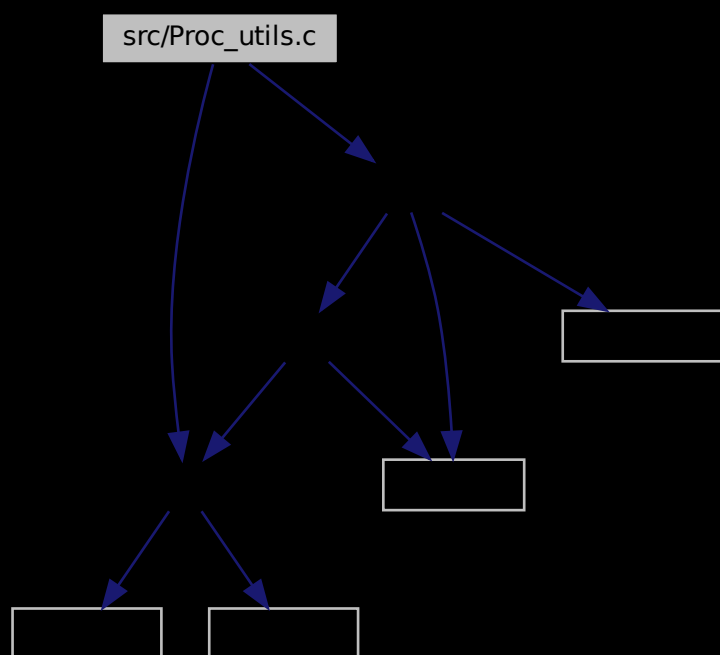
7.156 proc.c

[Go to the documentation of this file.](#)

00001

7.157 src/Proc_utils.c File Reference

```
#include "include/printf.h"
#include "include/sched.h"
Include dependency graph for Proc_utils.c:
```



Functions

- void `soak` (void)
Cooperative scheduler soak worker.

Variables

- uint8_t `soak_counter` = 0

7.157.1 Function Documentation

7.157.1.1 `soak()`

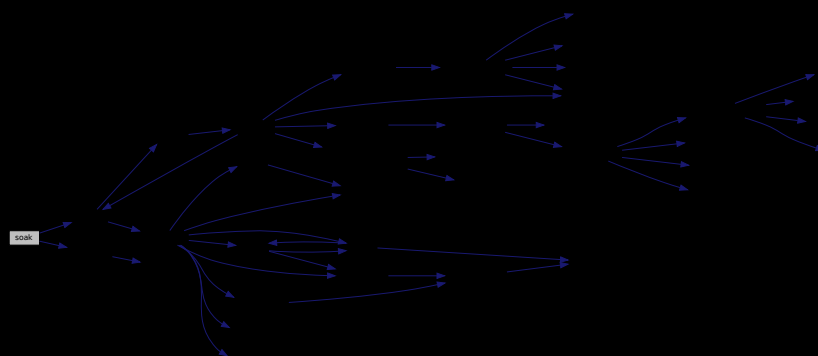
```
void soak (
    void )
```

Cooperative scheduler soak worker.

The VSH `soak` command creates another process running this entry point. Workers identify themselves by PID, repeatedly prove liveness, and yield back into the cooperative round-robin scheduler. Because 0.0.5 has no task-exit path, a soak worker is intentionally long-lived until the system halts.

Definition at line 13 of file `Proc_utils.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.157.2 Variable Documentation

7.157.2.1 `soak_counter`

```
uint8_t soak_counter = 0
```

Copyright (c) 2026 Tristan Adams

This software is released under the MIT License. <https://opensource.org/licenses/MIT>

Definition at line 11 of file `Proc_utils.c`.

7.158 Proc_utils.c

Go to the documentation of this file.

```

00001
00008 #include "include/printf.h"
00009 #include "include/sched.h"
00010
00011 uint8_t soak_counter = 0;
00012
00013 void soak(void)
00014 {
00015     int id = current->pid;
00016
00017     for (;;)
00018     {
00019         printf_("Soaker PID %i Alive\n", id);
00020         schedule();
00021     }
00022 }

```

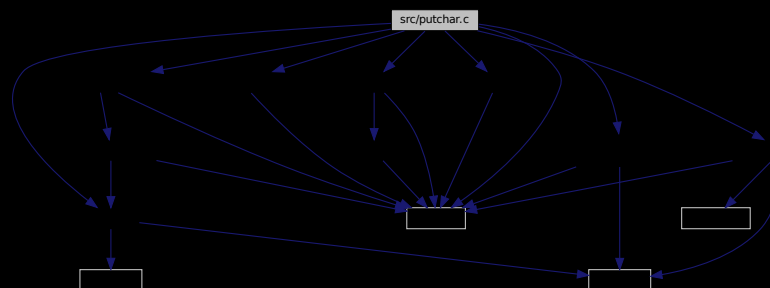
7.159 src/putchar.c File Reference

```

#include "include/KernelUtils.h"
#include "include/kernel.h"
#include "include/paging/paging_bootstrap.h"
#include "include/printf.h"
#include "include/registers.h"
#include "include/serial.h"
#include "include/terminal/term.h"
#include <stdint.h>

```

Include dependency graph for putchar.c:



Macros

- #define LOGBUF_SIZE 256

Functions

- static uint64_t ticks_to_ms (uint64_t ticks)
- static void u64_to_str (uint64_t val, char *buf)
- static void flush_serial_line (void)
- static void buffer_or_flush (char c)
- static bool voyager_page_tables_active (void)
- void _putchar (char character)

Variables

- static char `logbuf` [`LOGBUF_SIZE`]
- static size_t `loglen` = 0
- static uint64_t `tsc_hz` = 2000000000ULL

7.159.1 Macro Definition Documentation

7.159.1.1 LOGBUF_SIZE

```
#define LOGBUF_SIZE 256
```

Definition at line 10 of file `putchar.c`.

7.159.2 Function Documentation

7.159.2.1 _putchar()

```
void _putchar (  
    char character )
```

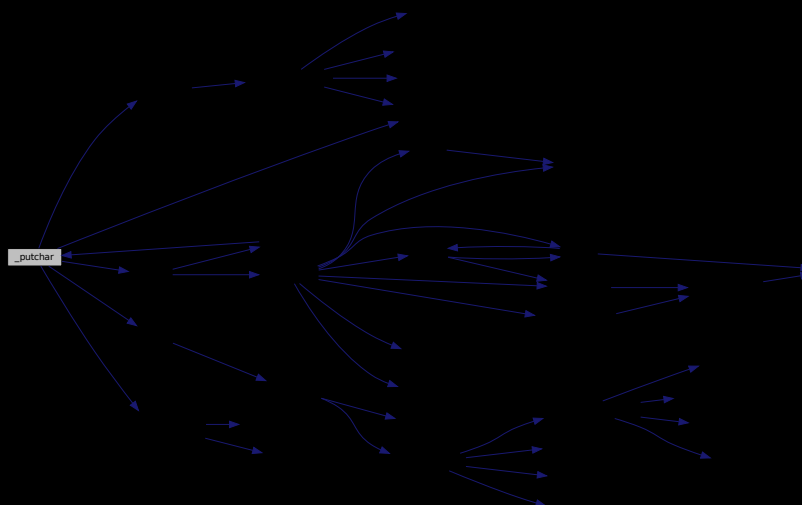
Output a character to a custom device like UART, used by the `printf()` function This function is declared here only. You have to write your custom implementation somewhere

Parameters

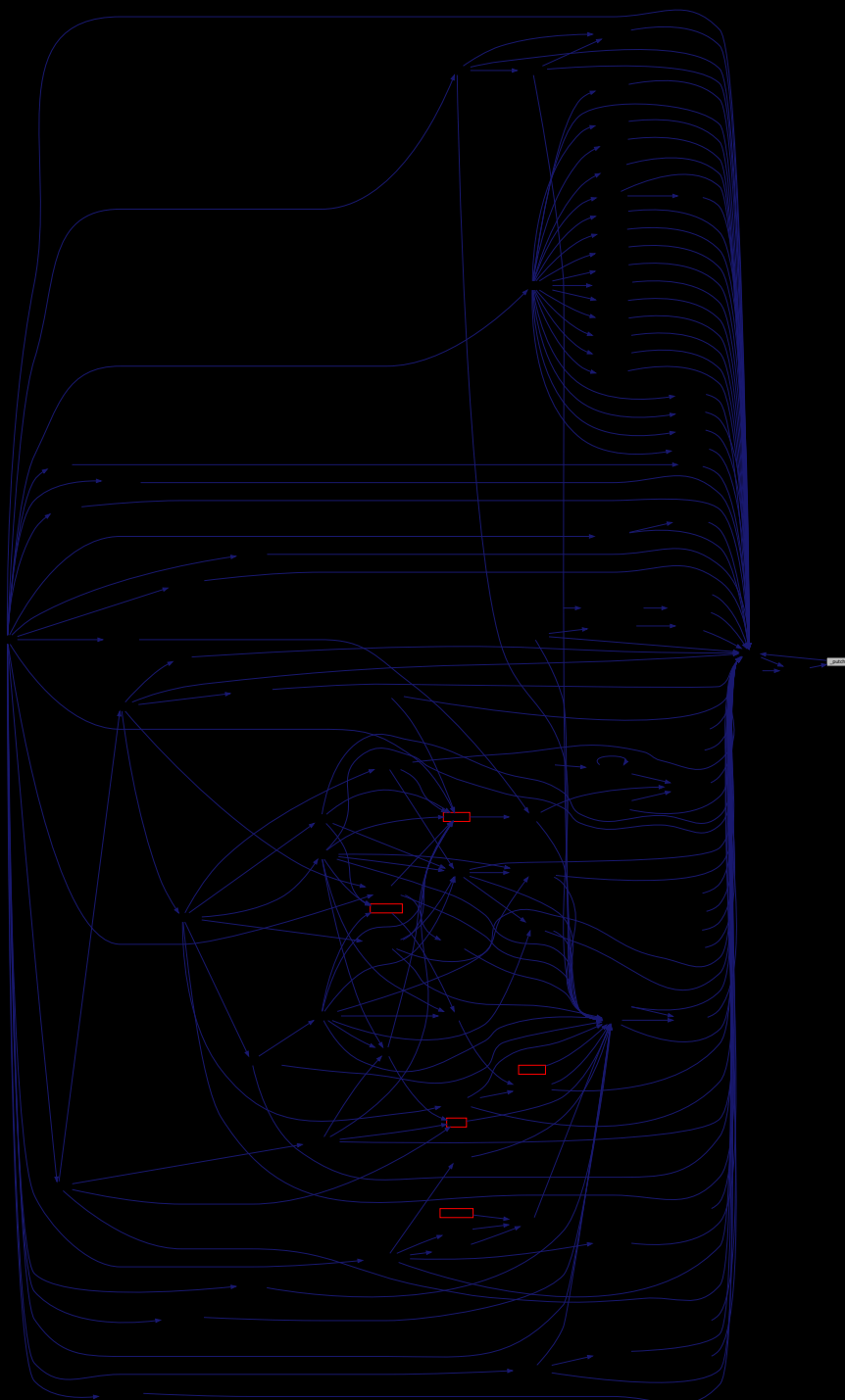
<i>character</i>	Character to output
------------------	---------------------

Definition at line 103 of file `putchar.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

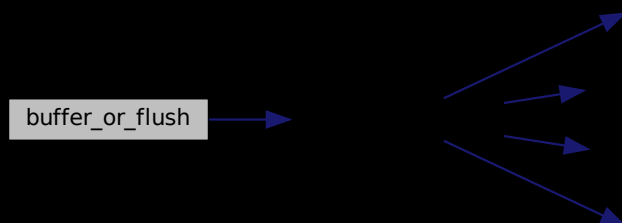


7.159.2.2 `buffer_or_flush()`

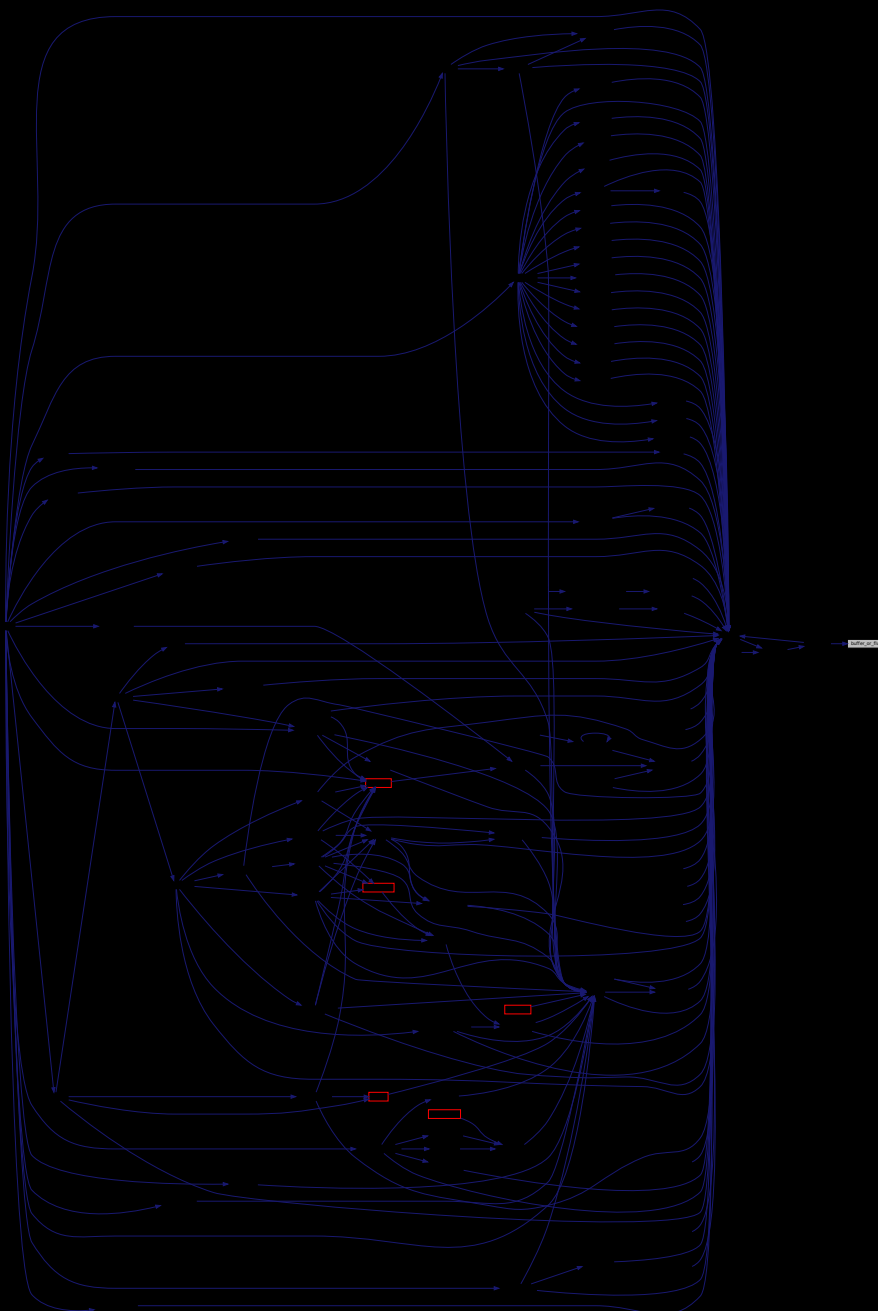
```
static void buffer_or_flush (  
    char c ) [static]
```


Definition at line 79 of file `putchar.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

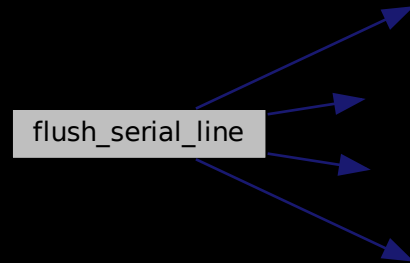


7.159.2.3 flush_serial_line()

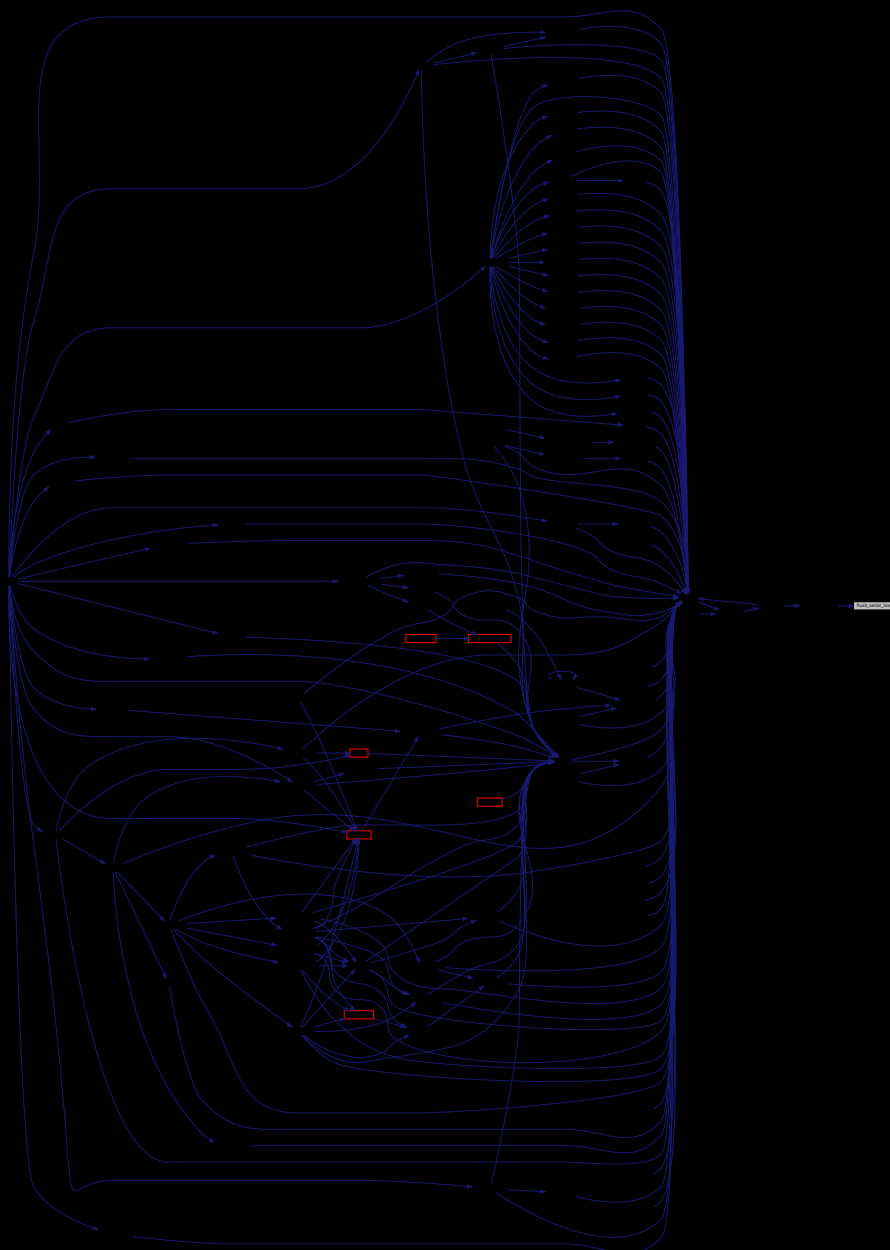
```
static void flush_serial_line (  
    void ) [static]
```

Definition at line 45 of file [putchar.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

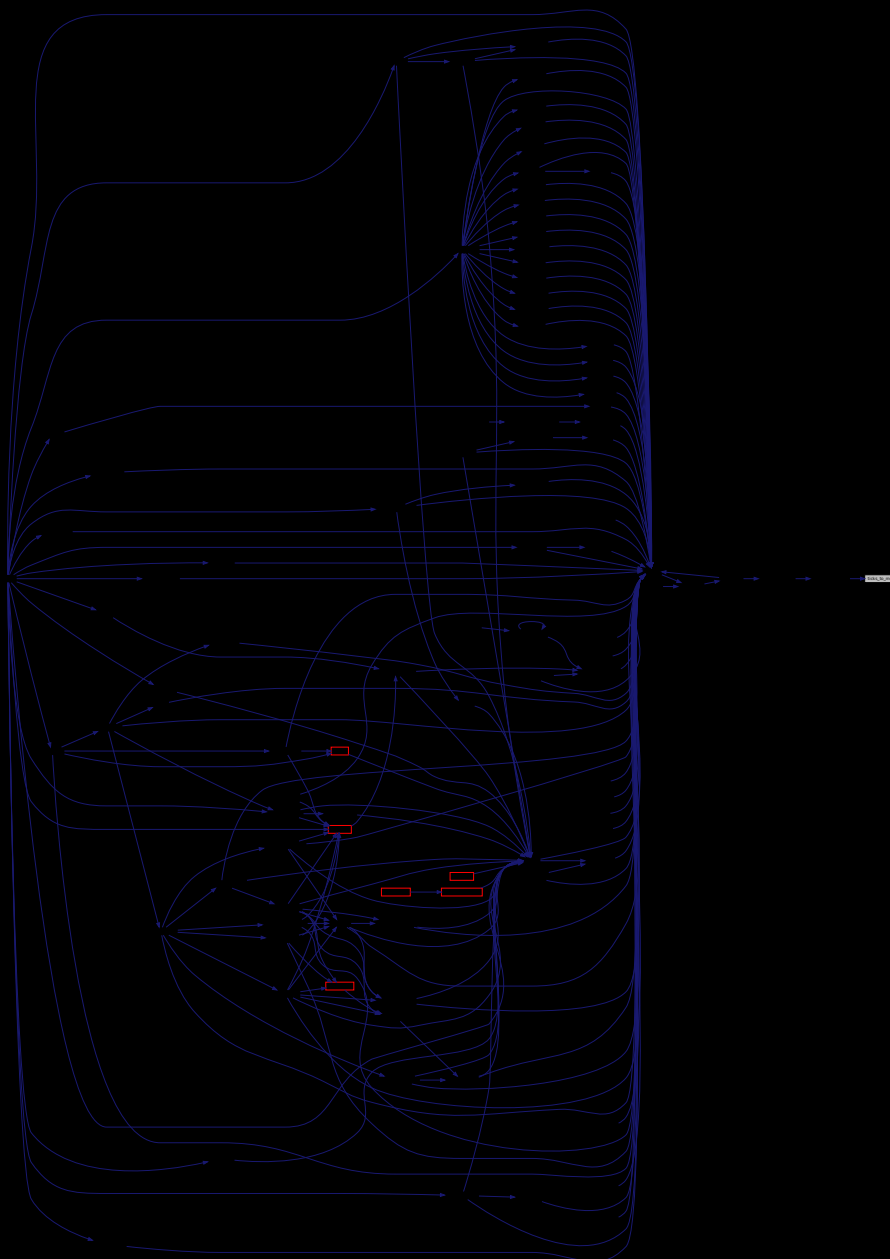


7.159.2.4 ticks_to_ms()

```
static uint64_t ticks_to_ms (  
    uint64_t ticks ) [static]
```

Definition at line 20 of file [putchar.c](#).

Here is the caller graph for this function:

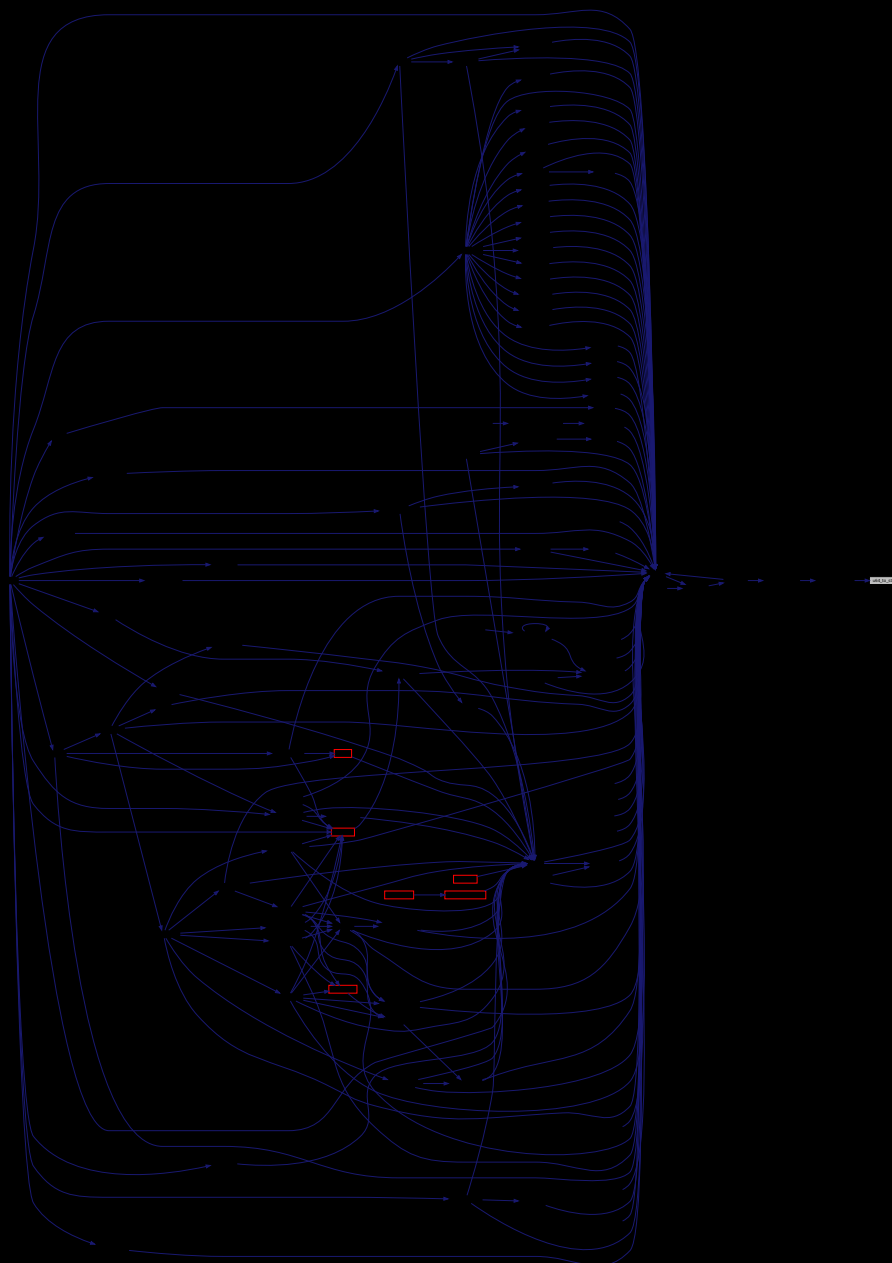


7.159.2.5 u64_to_str()

```
static void u64_to_str (  
    uint64_t val,  
    char * buf ) [static]
```

Definition at line 25 of file [putchar.c](#).

Here is the caller graph for this function:

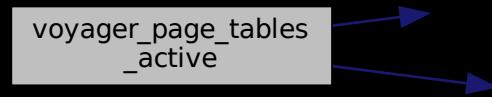


7.159.2.6 voyager_page_tables_active()

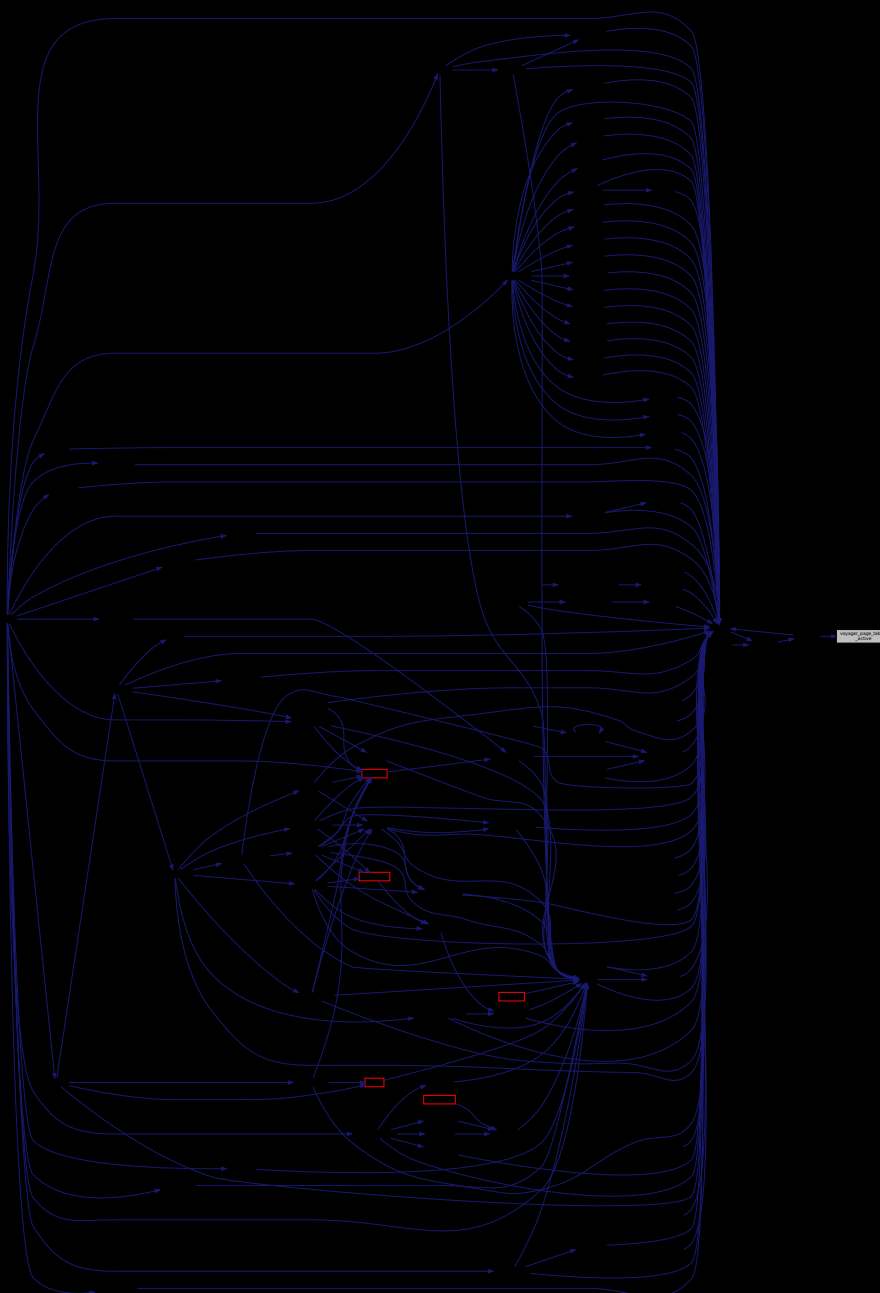
```
static bool voyager_page_tables_active (  
    void ) [inline], [static]
```

Definition at line 91 of file [putchar.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.159.3 Variable Documentation

7.159.3.1 logbuf

```
char logbuf[LOGBUF_SIZE] [static]
```

Definition at line 11 of file `putchar.c`.

7.159.3.2 loglen

```
size_t loglen = 0 [static]
```

Definition at line 12 of file `putchar.c`.

7.159.3.3 tsc_hz

```
uint64_t tsc_hz = 2000000000ULL [static]
```

Definition at line 15 of file `putchar.c`.

7.160 putchar.c

[Go to the documentation of this file.](#)

```
00001 #include "include/KernelUtils.h"
00002 #include "include/kernel.h"
00003 #include "include/paging/paging_bootstrap.h"
00004 #include "include/printf.h"
00005 #include "include/registers.h"
00006 #include "include/serial.h"
00007 #include "include/terminal/term.h"
00008 #include <stdint.h>
00009
00010 #define LOGBUF_SIZE 256
00011 static char logbuf[LOGBUF_SIZE];
00012 static size_t loglen = 0;
00013
00014 /* Calibrated once at boot */
00015 static uint64_t tsc_hz = 2000000000ULL; // TODO: replace with real calibration
00016
00017 /* ----- */
00018 /* Conversion helpers */
00019 /* ----- */
00020 static uint64_t ticks_to_ms(uint64_t ticks)
00021 {
00022     return (ticks * 1000ULL) / tsc_hz;
00023 }
00024
00025 static void u64_to_str(uint64_t val, char *buf)
00026 {
00027     char tmp[21];
00028     int i = 0;
00029     do
00030     {
00031         tmp[i++] = '0' + (val % 10);
00032         val /= 10;
00033     } while (val > 0);
00034     buf[i] = '\0';
00035     for (int j = 0; j < i; j++)
00036     {
00037         buf[j] = tmp[i - j - 1];
00038     }
00039     buf[i] = '\0';
00040 }
00041
00042 /* ----- */
00043 /* Timestamped line flush */
00044 /* ----- */
00045 static void flush_serial_line(void)
00046 {
00047     if (loglen == 0)
00048         return;
00049     logbuf[loglen] = '\0';
00050
00051     // convert ticks → ms
00052     uint64_t ms = ticks_to_ms(get_ts());
00053     char tsbuf[32];
```

```

00055     u64_to_str(ms, tsbuf);
00056
00057     // print [xxx ms] prefix
00058     serial_debug('[');
00059     for (char *p = tsbuf; *p; p++)
00060         serial_debug(*p);
00061     serial_debug('m');
00062     serial_debug('s');
00063     serial_debug(']');
00064     serial_debug(' ');
00065
00066     // print buffered line
00067     for (size_t i = 0; i < loglen; i++)
00068     {
00069         serial_debug(logbuf[i]);
00070     }
00071     serial_debug('\n');
00072
00073     loglen = 0;
00074 }
00075
00076 /* ----- */
00077 /* Buffer helper */
00078 /* ----- */
00079 static void buffer_or_flush(char c)
00080 {
00081     if (c == '\n' || loglen == LOGBUF_SIZE - 1)
00082     {
00083         flush_serial_line();
00084     }
00085     else
00086     {
00087         logbuf[loglen++] = c;
00088     }
00089 }
00090
00091 static inline bool voyager_page_tables_active(void)
00092 {
00093     uint64_t voyager_cr3 = paging_pml4_phys();
00094     if (voyager_cr3 == 0)
00095         return false;
00096
00097     return (readCR3() & ~0xfffffull) == (voyager_cr3 & ~0xfffffull);
00098 }
00099
00100 /* ----- */
00101 /* putchar */
00102 /* ----- */
00103 void _putchar(char character)
00104 {
00105     // Terminal / framebuffer routing (unchanged from your version)
00106     if (kerror_mode == 1)
00107     {
00108         term_write(term_context, &character, sizeof(char));
00109         if (k_mode.timestamp == 0)
00110             serial_debug(character);
00111         else
00112             buffer_or_flush(character);
00113     }
00114     else if (kerror_mode == 2)
00115     {
00116         if (k_mode.timestamp == 0)
00117             serial_debug(character);
00118         else
00119             buffer_or_flush(character);
00120     }
00121     else if (bootspace == 1)
00122     {
00123         if (k_mode.timestamp == 0)
00124             serial_debug(character);
00125         else
00126             buffer_or_flush(character);
00127     }
00128     else if (bootspace == 2)
00129     {
00130         /* Limine's terminal write callback belongs to the bootloader's
00131          * execution environment. Once Voyager installs its own CR3, do not
00132          * execute that callback; the replacement HHDM is deliberately NX.
00133          * Stay on COM1 until Voyager's framebuffer terminal is initialized. */
00134         if (!voyager_page_tables_active())
00135         {
00136             early_term.response->write(
00137                 early_term.response->terminals[0], &character, sizeof(char));
00138         }
00139
00140         if (k_mode.timestamp == 0)
00141             serial_debug(character);

```

```

00142         else
00143             buffer_or_flush(character);
00144     }
00145     else if (bootspace == 3)
00146     {
00147         if (!voyager_page_tables_active())
00148         {
00149             early_term.response->write(
00150                 early_term.response->terminals[0], &character, sizeof(char));
00151         }
00152         else if (k_mode.timestamp == 0)
00153         {
00154             serial_debug(character);
00155         }
00156         else
00157         {
00158             buffer_or_flush(character);
00159         }
00160     }
00161     else
00162     {
00163         if (term_context)
00164         {
00165             term_write(term_context, &character, sizeof(char));
00166             if (k_mode.timestamp == 0)
00167                 serial_debug(character);
00168             else
00169                 buffer_or_flush(character);
00170         }
00171         else
00172         {
00173             if (k_mode.timestamp == 0)
00174                 serial_debug(character);
00175             else
00176                 buffer_or_flush(character);
00177
00178             printf("%s\n", "ERROR: TERMINAL WRITE FAILURE!");
00179             printf("%s\n", "Defaulting to serial");
00180             return;
00181         }
00182     }
00183 }

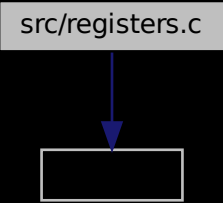
```

7.161 src/registers.c File Reference

#include <stdint.h>

Include dependency graph for registers.c:

src/registers.c



Functions

- void **writeMSR** (uint64_t msr, uint64_t value)
Write a 64-bit model-specific register.
- uint64_t **rdmsr** (uint32_t msr)
Read a 64-bit model-specific register.

7.161.1 Function Documentation

7.161.1.1 rdmsr()

```
uint64_t rdmsr (  
    uint32_t msr )
```

Read a 64-bit model-specific register.

Parameters

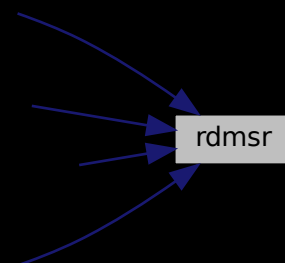
<i>msr</i>	MSR index.
------------	------------

Returns

Current 64-bit MSR value.

Definition at line 14 of file [registers.c](#).

Here is the caller graph for this function:



7.161.1.2 writeMSR()

```
void writeMSR (  
    uint64_t msr,  
    uint64_t value )
```

Write a 64-bit model-specific register.

Parameters

<i>msr</i>	MSR index.
<i>value</i>	Value to write.

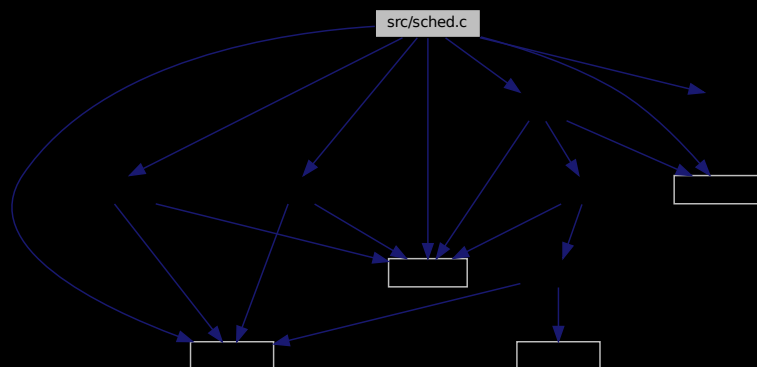
Definition at line 4 of file registers.c.

7.162 registers.c

7.163 src/sched.c File Reference

```
#include "include/sched.h"
#include "include/mm/kmalloc.h"
#include "include/panic.h"
#include "include/string.h"
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
```

Include dependency graph for sched.c:



Macros

- `#define STACK_SIZE (16u * 1024u)`
- `#define STACK_ALIGN 16u`

Functions

- static void `task_returned` (void)
- void `init_scheduler` (void)
Reset scheduler bookkeeping to its pre-dispatch state.
- `process_t * create_process` (void(*entry)(void))
Create a kernel process and append it to the round-robin run queue.
- void `schedule` (void)
Yield the CPU cooperatively to the next process in the circular queue.
- void `scheduler_start` (void)
Begin execution of the initialized run queue.

Variables

- bool `allow_sched` = false
Gate used by the PIT path for scheduler preemption.
- `process_t * current` = NULL
Process whose stack/context is currently executing, or NULL before the first scheduler dispatch.
- static `process_t * run_queue_head` = NULL
- static `process_t * run_queue_tail` = NULL
- static int `next_pid` = 1

7.163.1 Macro Definition Documentation

7.163.1.1 STACK_ALIGN

```
#define STACK_ALIGN 16u
```

Definition at line 10 of file `sched.c`.

7.163.1.2 STACK_SIZE

```
#define STACK_SIZE (16u * 1024u)
```

Definition at line 9 of file `sched.c`.

7.163.2 Function Documentation

7.163.2.1 create_process()

```
process_t * create_process (
    void(*) (void) entry )
```

Create a kernel process and append it to the round-robin run queue.

Parameters

<i>entry</i>	<code>void(void)</code> entry point for the new process.
--------------	--

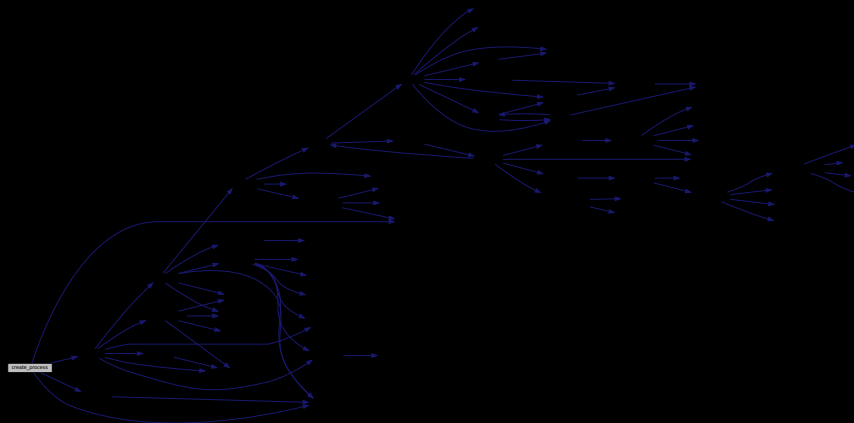
Returns

Pointer to the new process control block.

A 16 KiB stack is allocated from `kmallo`c and initialized with a synthetic context suitable for `switch_to()`. A task that returns from `entry` is treated as a kernel error in 0.0.5 because task exit/reaping is not implemented yet.

Definition at line 40 of file `sched.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.163.2.2 `init_scheduler()`

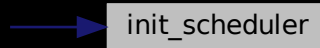
```
void init_scheduler (
    void )
```

Reset scheduler bookkeeping to its pre-dispatch state.

Call once during kernel bring-up before creating the initial tasks.

Definition at line 31 of file `sched.c`.

Here is the caller graph for this function:



7.163.2.3 `schedule()`

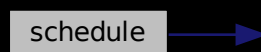
```
void schedule (  
    void )
```

Yield the CPU cooperatively to the next process in the circular queue.

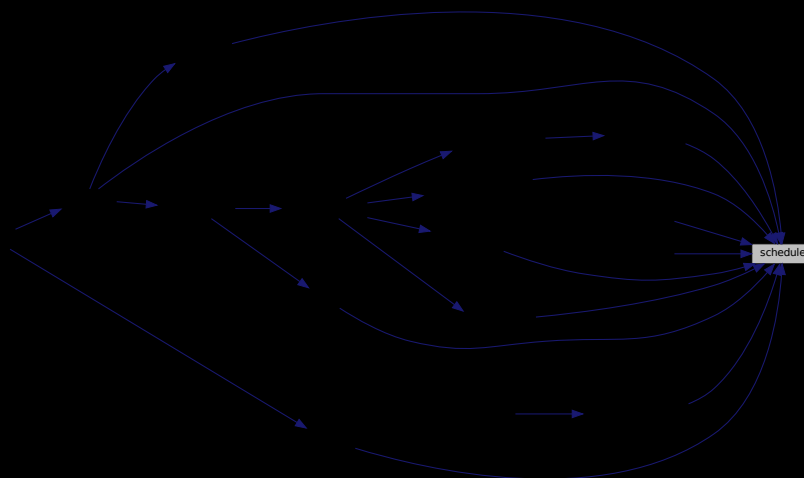
Calling `schedule()` is the 0.0.5 task-yield primitive. With zero runnable tasks it is a no-op; with only the current task runnable it returns without switching.

Definition at line 108 of file `sched.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.163.2.4 scheduler_start()

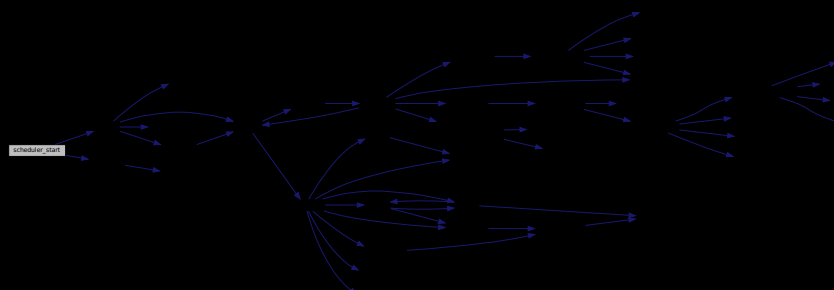
```
void scheduler_start (  
    void )
```

Begin execution of the initialized run queue.

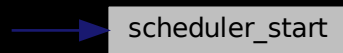
The run queue must contain at least one process and `current` must still be NULL. This function performs the initial dispatch and is not expected to return.

Definition at line 128 of file `sched.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

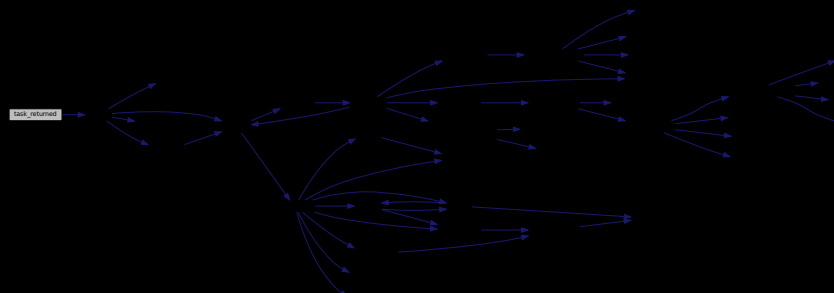


7.163.2.5 task_returned()

```
static void task_returned (
    void ) [static]
```

Definition at line 22 of file [sched.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.163.3 Variable Documentation

7.163.3.1 allow_sched

```
bool allow_sched = false
```

Gate used by the PIT path for scheduler preemption.

This remains false in the qualified 0.0.5 cooperative design. Setting it to true does not by itself make `switch_to()` safe for IRQ-time preemption.

Definition at line 15 of file `sched.c`.

7.163.3.2 current

```
process_t* current = NULL
```

Process whose stack/context is currently executing, or NULL before the first scheduler dispatch.

Definition at line 16 of file `sched.c`.

7.163.3.3 next_pid

```
int next_pid = 1 [static]
```

Definition at line 20 of file `sched.c`.

7.163.3.4 run_queue_head

```
process_t* run_queue_head = NULL [static]
```

Definition at line 18 of file `sched.c`.

7.163.3.5 run_queue_tail

```
process_t* run_queue_tail = NULL [static]
```

Definition at line 19 of file `sched.c`.

7.164 sched.c

Go to the documentation of this file.

```

00001 #include "include/sched.h"
00002 #include "include/mm/kmalloc.h"
00003 #include "include/panic.h"
00004 #include "include/string.h"
00005 #include <stdbool.h>
00006 #include <stddef.h>
00007 #include <stdint.h>
00008
00009 #define STACK_SIZE (16u * 1024u)
00010 #define STACK_ALIGN 16u
00011
00012 /* This remains false for the cooperative 0.0.5 bring-up. The PIT handler uses
00013 * it as the gate for IRQ-time preemption, which is deliberately parked until
00014 * context switching is moved onto a proper interrupt-frame design. */
00015 bool allow_sched = false;
00016 process_t *current = NULL;
00017
00018 static process_t *run_queue_head = NULL;
00019 static process_t *run_queue_tail = NULL;
00020 static int next_pid = 1;
00021
00022 static void task_returned(void)
00023 {
00024     int pid = current ? current->pid : -1;
00025     panic("scheduler: task %i returned unexpectedly", pid);
00026
00027     for (;;)
00028         asm volatile("cli; hlt");
00029 }
00030
00031 void init_scheduler(void)
00032 {
00033     current = NULL;
00034     run_queue_head = NULL;
00035     run_queue_tail = NULL;
00036     next_pid = 1;
00037     allow_sched = false;
00038 }
00039
00040 process_t *create_process(void (*entry)(void))
00041 {
00042     if (!entry)
00043         panic("scheduler: NULL task entry");
00044
00045     process_t *p = kmalloc(sizeof(*p));
00046     if (!p)
00047         panic("scheduler: failed to allocate process structure");
00048
00049     memset(p, 0, sizeof(*p));
00050
00051     uint8_t *stack = kmalloc(STACK_SIZE);
00052     if (!stack)
00053         panic("scheduler: failed to allocate task stack");
00054
00055     memset(stack, 0, STACK_SIZE);
00056
00057     p->pid = next_pid++;
00058     p->state = PROC_READY;
00059
00060     /*
00061     * switch_to() restores this exact layout:
00062     *
00063     *   rsp -> r15
00064     *       r14
00065     *       r13
00066     *       r12
00067     *       rbx
00068     *       rbp
00069     *       initial RIP (consumed by ret)
00070     *       task_returned (return address seen by task entry)
00071     *
00072     * The task-visible RSP is 8 mod 16, matching the SysV AMD64 function
00073     * entry convention.
00074     */
00075     uintptr_t stack_end = (uintptr_t)stack + STACK_SIZE;
00076     stack_end &= ~(uintptr_t)STACK_ALIGN - 1u;
00077     stack_end -= sizeof(uint64_t);
00078
00079     uint64_t *sp = (uint64_t *)stack_end;
00080     *sp = (uint64_t)(uintptr_t)task_returned;
00081
00082     * (--sp) = (uint64_t)(uintptr_t)entry; /* ret target */

```

```

00083     *(&sp) = 0; /* rbp */
00084     *(&rbx) = 0; /* rbx */
00085     *(&r12) = 0; /* r12 */
00086     *(&r13) = 0; /* r13 */
00087     *(&r14) = 0; /* r14 */
00088     *(&r15) = 0; /* r15 */
00089
00090     p->rsp = sp;
00091
00092     if (!run_queue_head)
00093     {
00094         run_queue_head = p;
00095         run_queue_tail = p;
00096         p->next = p;
00097     }
00098     else
00099     {
00100         p->next = run_queue_head;
00101         run_queue_tail->next = p;
00102         run_queue_tail = p;
00103     }
00104
00105     return p;
00106 }
00107
00108 void schedule(void)
00109 {
00110     if (!run_queue_head)
00111         return;
00112
00113     process_t *next = current ? current->next : run_queue_head;
00114     if (!next)
00115         return;
00116
00117     /* A single runnable task has nowhere to yield to after first dispatch. */
00118     if (current && next == current)
00119         return;
00120
00121     if (current)
00122         current->state = PROC_READY;
00123
00124     next->state = PROC_RUNNING;
00125     switch_to(next);
00126 }
00127
00128 void scheduler_start(void)
00129 {
00130     if (!run_queue_head)
00131         panic("scheduler: cannot start with an empty run queue");
00132     if (current)
00133         panic("scheduler: start requested while a task is already current");
00134
00135     /* IRQ-time preemption intentionally remains disabled. Tasks yield by
00136    * calling schedule() from ordinary C context. */
00137     allow_sched = false;
00138     schedule();
00139
00140     panic("scheduler: initial dispatch returned unexpectedly");
00141 }

```

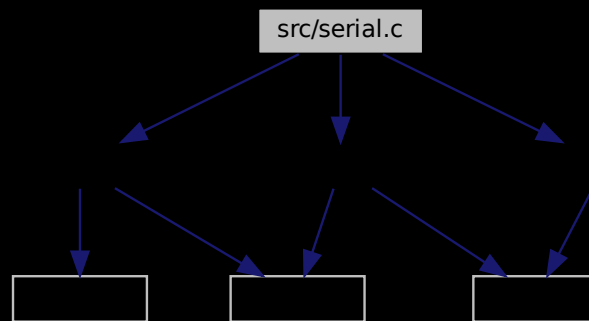
7.165 src/serial.c File Reference

```

#include "include/printf.h"
#include "include/serial.h"
#include "include/string.h"

```

Include dependency graph for serial.c:



Functions

- void `serial_print` (const char *str)
Print a timestamped string to the serial port.
- void `serial_print_with_ts` (const char *msg)
Write a diagnostic message prefixed with the current timestamp.
- void `serial_print_line` (const char *str)
Print a timestamped string followed by a newline.

7.165.1 Function Documentation

7.165.1.1 `serial_print()`

```
void serial_print (  
    const char * str )
```

Print a timestamped string to the serial port.

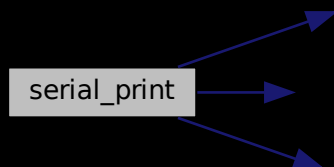
Write a NUL-terminated string to the serial console.

Parameters

<code>str</code>	String to print.
------------------	------------------

Definition at line 9 of file `serial.c`.

Here is the call graph for this function:



7.165.1.2 `serial_print_line()`

```
void serial_print_line (  
    const char * str )
```

Print a timestamped string followed by a newline.

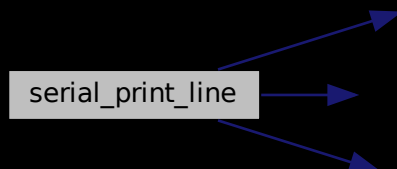
Write a NUL-terminated string followed by a line ending.

Parameters

<i>str</i>	String to print.
------------	------------------

Definition at line 44 of file `serial.c`.

Here is the call graph for this function:



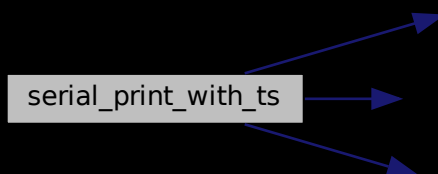
7.165.1.3 serial_print_with_ts()

```
void serial_print_with_ts (
    const char * msg )
```

Write a diagnostic message prefixed with the current timestamp.

Definition at line 24 of file [serial.c](#).

Here is the call graph for this function:



7.166 serial.c

[Go to the documentation of this file.](#)

```
00001 #include "include/printf.h"
00002 #include "include/serial.h"
00003 #include "include/string.h"
00004
00009 void serial_print(const char *str)
00010 {
00011     char timestamp_buf[32];
00012     u64_to_str(get_ts(), timestamp_buf);
00013
00014     serial_debug('[');
00015     for (char *p = timestamp_buf; *p; p++)
00016         serial_debug(*p);
00017     serial_debug(']');
00018     serial_debug(' ');
00019
00020     for (const char *p = str; *p; p++)
00021         serial_debug(*p);
00022 }
00023
00024 void serial_print_with_ts(const char *msg)
00025 {
00026     char ts[32];
00027     u64_to_str(get_ts(), ts);
00028
00029     serial_debug('[');
00030     for (char *p = ts; *p; p++)
00031         serial_debug(*p);
00032     serial_debug(']');
00033     serial_debug(' ');
00034
00035     for (const char *p = msg; *p; p++)
00036         serial_debug(*p);
```



```

00037     serial_debug('\n');
00038 }
00039
00044 void serial_print_line(const char *str)
00045 {
00046     char timestamp_buf[32];
00047     u64_to_str(get_ts(), timestamp_buf);
00048
00049     serial_debug('[');
00050     for (char *p = timestamp_buf; *p; p++)
00051         serial_debug(*p);
00052     serial_debug(']');
00053     serial_debug(' ');
00054
00055     for (const char *p = str; *p; p++)
00056         serial_debug(*p);
00057
00058     serial_debug('\n');
00059 }

```

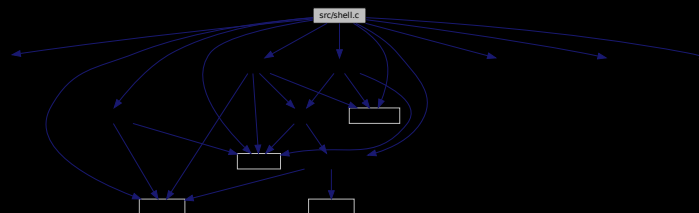
7.167 src/shell.c File Reference

```

#include "include/drivers/keyboard/keyboard.h"
#include "include/mm/kmalloc.h"
#include "include/mm/memtest.h"
#include "include/printf.h"
#include "include/proc_utils.h"
#include "include/sched.h"
#include "include/shell.h"
#include "include/time.h"
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

```

Include dependency graph for shell.c:



Functions

- void **halt** (void)
- static bool **shell_streq** (const char *a, const char *b)
- void **vsh_loop** (void)
Main VSH cooperative task.
- char * **vsh_readline** (void)
Read one line from the keyboard FIFO.
- void **cmd_parser** (const char *str)
Dispatch one NUL-terminated VSH command line.

7.167.1 Function Documentation

7.167.1.1 cmd_parser()

```
void cmd_parser (
    const char * str )
```

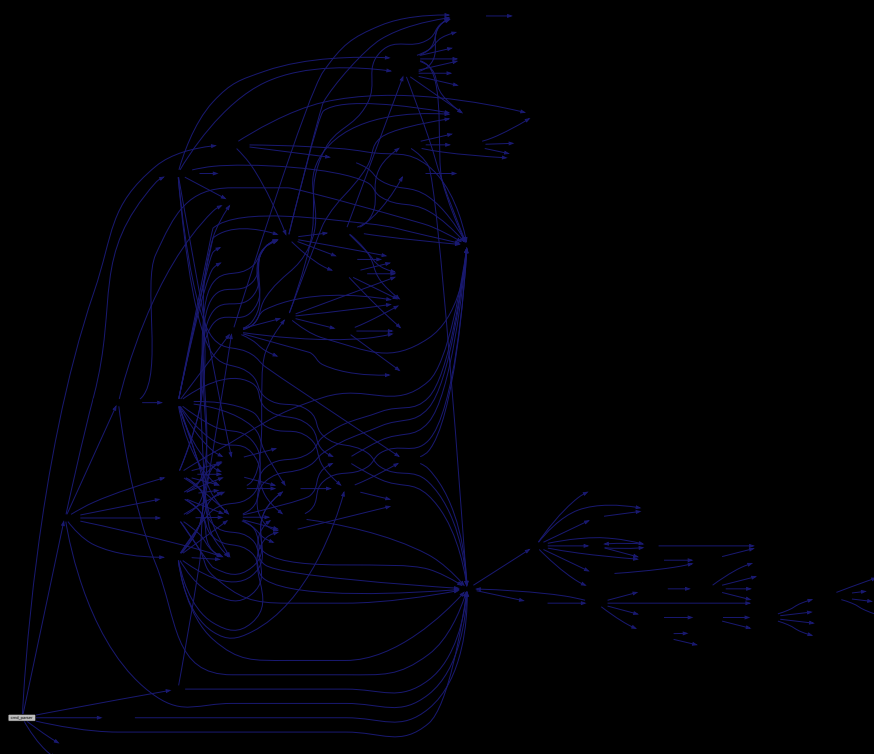
Dispatch one NUL-terminated VSH command line.

Parameters

<i>str</i>	Command string to parse.
------------	--------------------------

Definition at line 110 of file [shell.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.167.1.2 halt()

```
void halt (  
    void )
```

Here is the caller graph for this function:

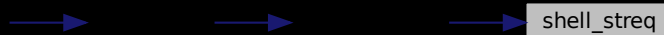


7.167.1.3 shell_streq()

```
static bool shell_streq (  
    const char * a,  
    const char * b ) [static]
```

Definition at line 15 of file [shell.c](#).

Here is the caller graph for this function:



7.167.1.4 vsh_loop()

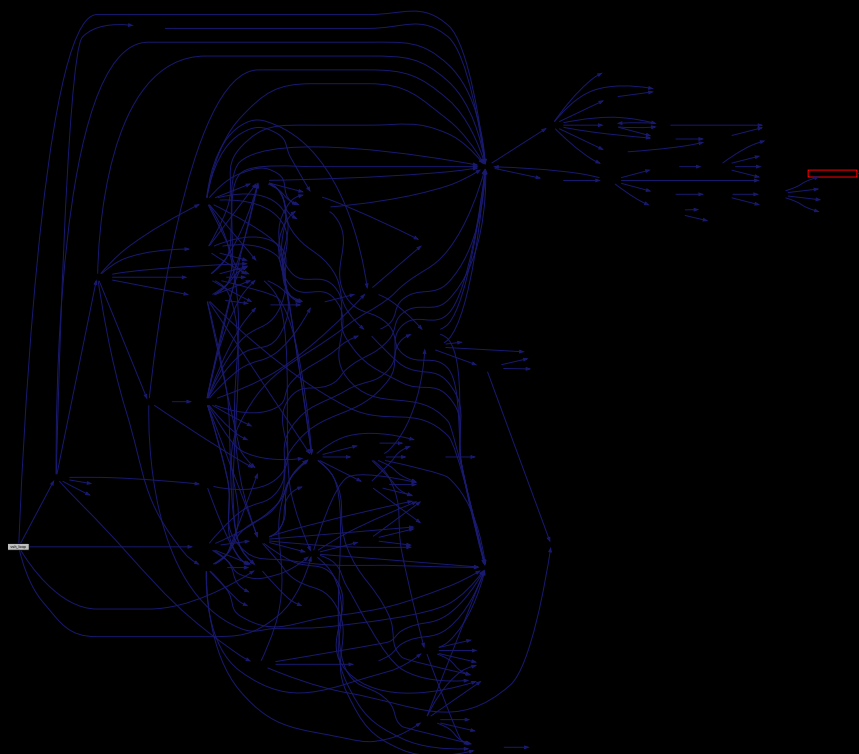
```
void vsh_loop (  
    void )
```

Main VSH cooperative task.

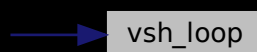
Repeatedly prints a prompt, reads a line from the keyboard FIFO, dispatches the command, frees the line buffer, and yields to the scheduler.

Definition at line 32 of file [shell.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.167.1.5 vsh_readline()

```
char * vsh_readline (  
    void )
```

Read one line from the keyboard FIFO.

Returns

Newly allocated NUL-terminated command string, or NULL if allocation fails.

The buffer is owned by the caller and must be released with `kfree()`. The function yields with `schedule()` whenever no keyboard character is available.

Definition at line 49 of file shell.c.

Here is the call graph for this function:

Here is the caller graph for this function:



7.168 shell.c

```
000001 #include <drivers/keyboard/keyboard.h>
000002 #include "include/mm/kmalloc.h"
000003 #include "include/mm/memtest.h"
000004 #include "include/printf.h"
000005 #include "include/proc_utils.h"
000006 #include "include/sched.h"
000007 #include "include/shell.h"
000008 #include "include/time.h"
000009 #include <stdbool.h>
000010 #include <stddef.h>
000011 #include <stdint.h>
000012
000013 extern void halt(void);
000014
000015 static bool shell_streq(const char *a, const char *b)
000016 {
000017     if (!a || !b)
000018         return false;
```

```

00019
00020     while (*a && *b)
00021     {
00022         if (*a != *b)
00023             return false;
00024         ++a;
00025         ++b;
00026     }
00027
00028     return *a == '\0' && *b == '\0';
00029 }
00030
00031 /* Public VSH API documentation lives in include/shell.h. */
00032 void vsh_loop(void)
00033 {
00034     for (;;)
00035     {
00036         printf_("%s", "> ");
00037
00038         char *line = vsh_readline();
00039         if (line)
00040         {
00041             cmd_parser(line);
00042             kfree(line);
00043         }
00044
00045         schedule();
00046     }
00047 }
00048
00049 char *vsh_readline(void)
00050 {
00051     size_t bufsize = VSH_CMD_BUFFER_SIZE;
00052     size_t pos      = 0;
00053     char *buffer    = kmalloc(bufsize);
00054
00055     if (!buffer)
00056     {
00057         printf_("%s\n", "vsh: Command Buffer Allocation Error!");
00058         return NULL;
00059     }
00060
00061     for (;;)
00062     {
00063         char c = kbd_pop();
00064
00065         if (c == 0)
00066         {
00067             schedule();
00068             continue;
00069         }
00070
00071         if (c == '\n' || c == '\r')
00072         {
00073             buffer[pos] = '\0';
00074             printf_("%s\n", "");
00075             return buffer;
00076         }
00077
00078         if (c == '\b')
00079         {
00080             if (pos > 0)
00081             {
00082                 --pos;
00083                 printf_("%s", "\b \b");
00084             }
00085             continue;
00086         }
00087
00088         if (pos + 1 >= bufsize)
00089         {
00090             size_t newsize = bufsize + VSH_CMD_BUFFER_SIZE;
00091             char *grown    = krealloc(buffer, newsize);
00092             if (!grown)
00093             {
00094                 printf_("%s\n", "vsh: Command Buffer Allocation Error!");
00095                 kfree(buffer);
00096                 return NULL;
00097             }
00098
00099             buffer = grown;
00100             bufsize = newsize;
00101         }
00102
00103         buffer[pos++] = c;
00104
00105         char echo[2] = {c, '\0'};

```

```
00106     printf_("%s", echo);
00107 }
00108 }
00109
00110 void cmd_parser(const char *str)
00111 {
00112     if (!str || str[0] == '\0')
00113         return;
00114
00115     if (shell_streq(str, "time"))
00116     {
00117         print_sys_time();
00118         return;
00119     }
00120
00121     if (shell_streq(str, "memtest"))
00122     {
00123         memtest_run();
00124         return;
00125     }
00126
00127     if (shell_streq(str, "halt") || shell_streq(str, "Halt"))
00128     {
00129         printf_("%s", "Halting System...");
00130         halt();
00131     }
00132
00133     if (shell_streq(str, "soak"))
00134     {
00135         create_process(soak);
00136         return;
00137     }
00138
00139     else
00140     {
00141         printf_("%s\n", "unknown command");
00142     }
00143 }
```

7.169 src/stack_trace.c File Reference

```
#include "include/stack_trace.h"
#include "include/KernelUtils.h"
#include "include/global_defs.h"
#include "include/limine.h"
#include "include/printf.h"
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
```


Parameters

<i>data</i>	Start of the region to print.
<i>size</i>	Number of bytes to dump.

Definition at line 97 of file [stack_trace.c](#).

Here is the caller graph for this function:



7.169.1.2 halt()

```
void halt (  
    void )
```

Here is the caller graph for this function:



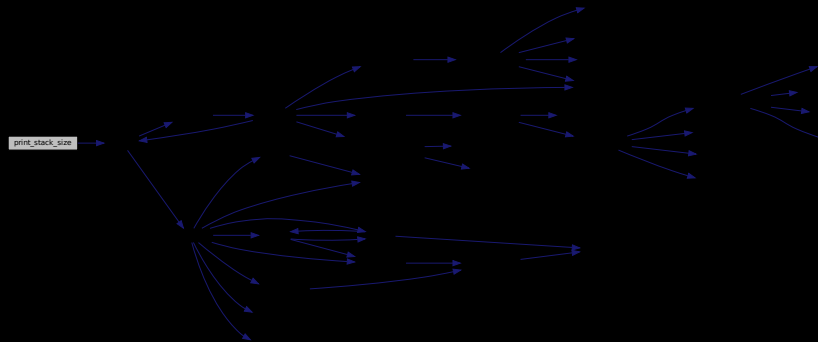
7.169.1.3 print_stack_size()

```
void print_stack_size (
    void )
```

Print the kernel stack size reported by Limine.

Definition at line 84 of file [stack_trace.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.169.1.4 stack_dump()

```
void stack_dump (
    void )
```

Emit the default diagnostic stack dump.

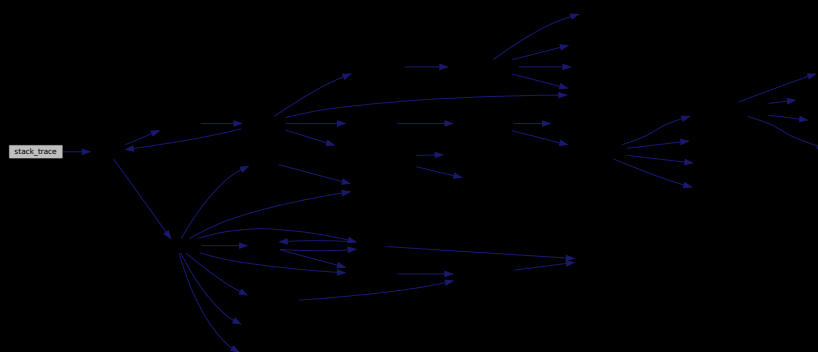
Definition at line 56 of file [stack_trace.c](#).

Parameters

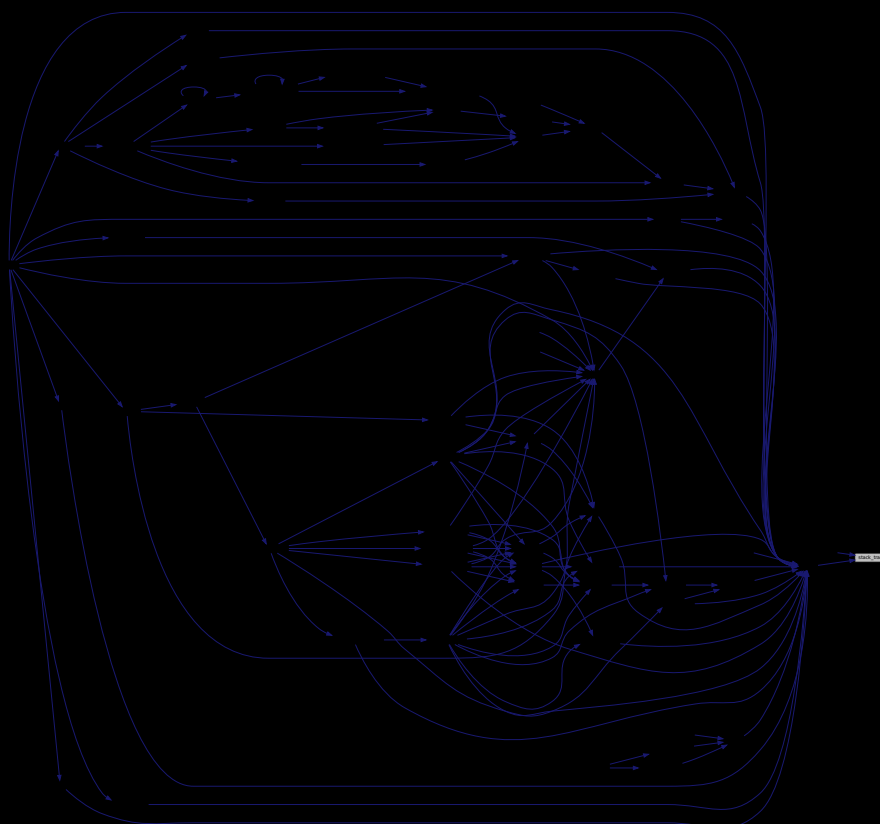
<code>max_frames</code>	Maximum number of frames to report.
-------------------------	-------------------------------------

Definition at line 35 of file `stack_trace.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.169.1.7 stack_trace_asm()

```
void stack_trace_asm (
    uint64_t max_size,
    bool stop )
```

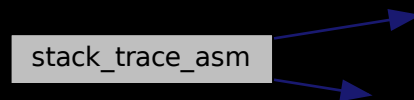
Assembly-assisted stack trace entry point.

Parameters

<i>max_size</i>	Maximum number/size of entries requested by the implementation.
<i>stop</i>	Whether the helper should stop after producing the trace.

Definition at line 14 of file [stack_trace.c](#).

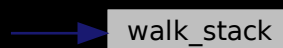
Here is the call graph for this function:



7.169.1.8 walk_stack()

```
uint64_t walk_stack (
    uint64_t * array,
    uint64_t max )
```

Here is the caller graph for this function:



7.169.2 Variable Documentation

7.169.2.1 stack_req

```
volatile struct limine_stack_size_request stack_req [extern]
```

7.170 stack_trace.c

Go to the documentation of this file.

```
00001 #include "include/stack_trace.h"
00002 #include "include/KernelUtils.h"
00003 #include "include/global_defs.h"
00004 #include "include/limine.h"
00005 #include "include/printf.h"
00006 #include <stdbool.h>
00007 #include <stddef.h>
00008 #include <stdint.h>
00009
00010 extern volatile struct limine_stack_size_request stack_req;
00011 extern uint64_t walk_stack(uint64_t *array, uint64_t max);
00012 extern void halt(void);
00013
00014 void stack_trace_asm(uint64_t max_size, bool stop)
00015 {
00016     uint64_t addresses[max_size];
00017     uint64_t array_max = walk_stack(addresses, max_size);
00018
00019     printf("%s\n", "-----");
00020     printf("%s\n", "    INFO: Stack Trace as Follows");
00021     printf("%s\n", "-----");
00022     printf("%s", "Size of address list: ");
00023     printf("%llx\n", array_max);
00024
00025     if (array_max > 0)
00026     {
00027         for (uint64_t i = 0; i + 1 < array_max; i++)
00028             printf("0x%llx\n", addresses[i]);
00029     }
00030
00031     if (stop)
00032         halt();
00033 }
00034
00035 void stack_trace(uint64_t max_frames)
00036 {
00037     if (max_frames == 0)
00038         max_frames = k_mode.stack_trace_size;
00039
00040     struct stack_frame *stk;
00041     asm volatile("mov %%rbp,%0" : "=r"(stk)::);
00042     printf("%s\n", "-----");
00043     printf("%s\n", "    INFO: Stack Trace (Unwind) as Follows");
00044     printf("%s\n", "-----");
00045
00046     for (uint8_t frame = 0; stk && frame < max_frames; ++frame)
00047     {
00048         printf_("%s\n", "0x%llx\n", stk->rip);
00049         stk = stk->rbp;
00050     }
00051     printf("%s\n", "-----");
00052     printf_("%s\n", "End of Trace.");
00053     printf("%s\n", "-----");
00054 }
00055
00056 void stack_dump(void)
00057 {
00058     struct stack_frame *stk;
00059     asm volatile("mov %%rbp,%0" : "=r"(stk)::);
00060     dump_hex(stk, 0x1000);
00061 }
00062
00063 void stack_dump_recursive(uint64_t max_frames)
```

```

00064 {
00065     if (max_frames == 0)
00066         max_frames = k_mode.stack_trace_size;
00067
00068     struct stack_frame *stk;
00069     asm volatile("mov %%rbp,%0" : "=r"(stk)::);
00070     printf("%s\n", "-----");
00071     printf("%s\n", "      INFO: Stack Trace (Unwind) as Follows");
00072     printf("%s\n", "-----");
00073
00074     for (uint8_t frame = 0; stk && frame < max_frames; ++frame)
00075     {
00076         dump_hex(stk->rbp, 0x1000);
00077         stk = stk->rbp;
00078     }
00079     printf("%s\n", "-----");
00080     printf_("%s\n", "End of Trace.");
00081     printf("%s\n", "-----");
00082 }
00083
00084 void print_stack_size(void)
00085 {
00086     if (stack_req.stack_size == 0)
00087     {
00088         printf_("%s\n", "ERROR: Unable to fetch stack size!");
00089     }
00090     else
00091     {
00092         printf_("%s", "INFO: Stack Size: ");
00093         printf_("0x%llx\n", stack_req.stack_size);
00094     }
00095 }
00096
00097 void dump_hex(const void *data, size_t size)
00098 {
00099     char  ascii[17];
00100     size_t i, j;
00101     ascii[16] = '\0';
00102
00103     for (i = 0; i < size; ++i)
00104     {
00105         printf("%02X ", ((unsigned char *) data)[i]);
00106         if (((unsigned char *) data)[i] >= ' ' &&
00107             ((unsigned char *) data)[i] <= '~')
00108         {
00109             ascii[i % 16] = ((unsigned char *) data)[i];
00110         }
00111         else
00112         {
00113             ascii[i % 16] = '.';
00114         }
00115
00116         if ((i + 1) % 8 == 0 || i + 1 == size)
00117         {
00118             printf(" ");
00119             if ((i + 1) % 16 == 0)
00120             {
00121                 printf("|  %s \n", ascii);
00122             }
00123             else if (i + 1 == size)
00124             {
00125                 ascii[(i + 1) % 16] = '\0';
00126                 if ((i + 1) % 16 <= 8)
00127                     printf(" ");
00128                 for (j = (i + 1) % 16; j < 16; ++j)
00129                     printf(" ");
00130                 printf("|  %s \n", ascii);
00131             }
00132         }
00133     }
00134 }

```

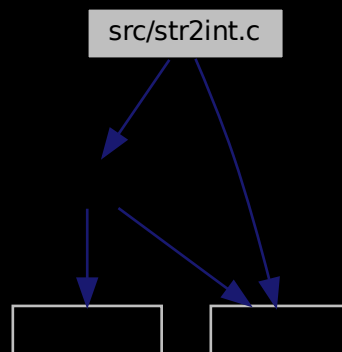
7.171 src/str2int.c File Reference

```

#include "include/string.h"
#include <stdint.h>

```

Include dependency graph for `str2int.c`:



Functions

- `int str2int (char str[])`
Convert a decimal string to an integer. (DEPRECATED)
- `uint64_t str2int2 (char *str)`

7.171.1 Function Documentation

7.171.1.1 `str2int()`

```
int str2int (  
    char str[] )
```

Convert a decimal string to an integer. (DEPRECATED)

Returns

Parsed value, or 0 if the string contains a non-decimal character.

Definition at line 8 of file `str2int.c`.

7.171.1.2 `str2int2()`

```
uint64_t str2int2 (  
    char * str )
```

Definition at line 25 of file `str2int.c`.

7.172 str2int.c

Go to the documentation of this file.

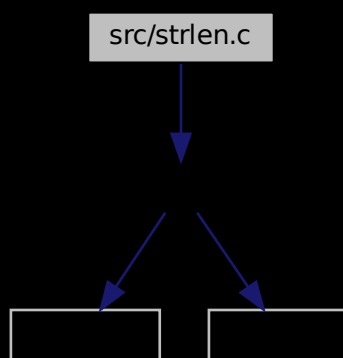
```

00001 #include "include/string.h"
00002 #include <stdint.h>
00003
00008 int str2int(char str[])
00009 {
00010     int i      = 0;
00011     int result = 0;
00012
00013     while (str[i] != '\0')
00014     {
00015         if (str[i] < '0' || str[i] > '9')
00016             return 0;
00017
00018         result = result * 10 + (str[i] - '0');
00019         i++;
00020     }
00021
00022     return result;
00023 }
00024
00025 uint64_t str2int2(char *str)
00026 {
00027     uint64_t res = 0;
00028
00029     for (uint64_t i = 0; str[i] != '\0'; ++i)
00030     {
00031         uint64_t digit = str[i] - '0';
00032         res = res * 10 + digit;
00033     }
00034
00035     return res;
00036 }

```

7.173 src/strlen.c File Reference

#include "include/string.h"
 Include dependency graph for strlen.c:



Functions

- `size_t` **strlen** (const char *str)
Returns the length of a string.

7.173.1 Function Documentation

7.173.1.1 strlen()

```
size_t strlen (  
    const char * str )
```

Returns the length of a string.

Parameters

<i>str</i>	string to parse
------------	-----------------

Returns

The length of the string

Definition at line 8 of file [strlen.c](#).

Here is the caller graph for this function:



7.174 strlen.c

[Go to the documentation of this file.](#)

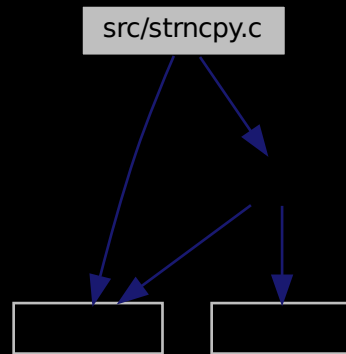
```
00001 #include "include/string.h"  
00002  
00008 size_t strlen(const char *str)  
00009 {  
00010     size_t len = 0;  
00011     while (str[len])  
00012         len++;  
00013     return len;  
00014 }
```

7.175 src/strncpy.c File Reference

```
#include "include/string.h"
```

```
#include <stddef.h>
```

Include dependency graph for strncpy.c:



Functions

- char * [strncpy](#) (char *dest, const char *src, size_t n)

7.175.1 Function Documentation

7.175.1.1 strncpy()

```
char * strncpy (
    char * dest,
    const char * src,
    size_t n )
```

Definition at line 5 of file [strncpy.c](#).

7.176 strncpy.c

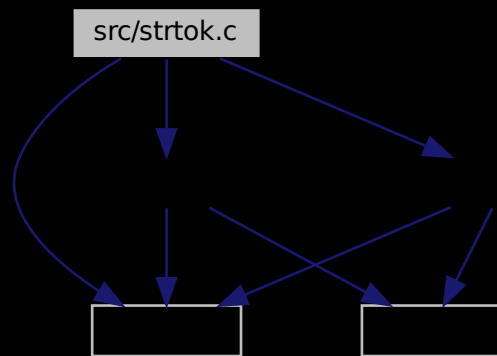
[Go to the documentation of this file.](#)

```

00001 #include "include/string.h"
00002 #include <stddef.h>
00003
00004 // strncpy() function that copies at most n characters of src to dest
00005 char *strncpy(char *dest, const char *src, size_t n)
00006 {
00007     size_t i;
00008
00009     for (i = 0; i < n && src[i] != '\0'; i++)
00010         dest[i] = src[i];
00011
00012     for (; i < n; i++)
00013         dest[i] = '\0';
00014
00015     return dest;
00016 }
```

7.177 src/strtok.c File Reference

```
#include "include/mm/kmalloc.h"
#include "include/string.h"
#include <stddef.h>
Include dependency graph for strtok.c:
```



Functions

- int `strtok` (char *srcstr, char sep, char ***output)

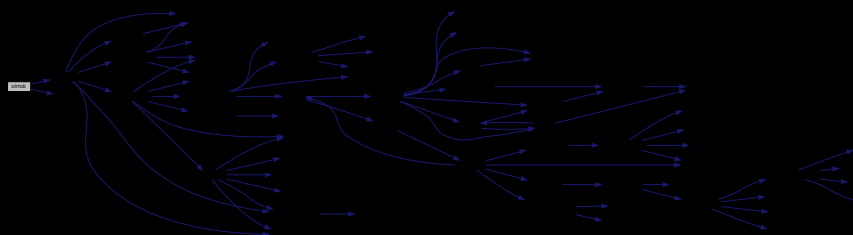
7.177.1 Function Documentation

7.177.1.1 strtok()

```
int strtok (
    char * srcstr,
    char sep,
    char *** output )
```

Definition at line 5 of file `strtok.c`.

Here is the call graph for this function:



7.178 strtok.c

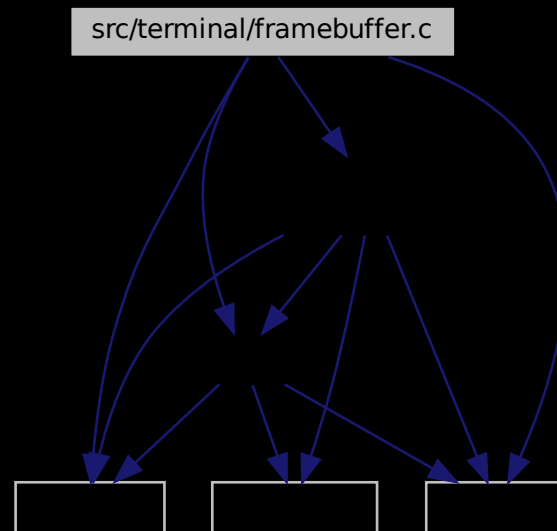
Go to the documentation of this file.

```
00001 #include "include/mm/kmalloc.h"
00002 #include "include/string.h"
00003 #include <stddef.h>
00004
00005 int strtok(char *srcstr, char sep, char ***output)
00006 {
00007     int len = strlen(srcstr);
00008     int numparts = 0;
00009     char **currentpart;
00010
00011     for (int i = 0; i < len; i++)
00012     {
00013         if (srcstr[i] == sep)
00014         {
00015             srcstr[i] = '\\0';
00016             numparts++;
00017         }
00018     }
00019
00020     numparts++;
00021     *output = kmalloc((size_t) numparts * sizeof(char *));
00022     if (*output == NULL)
00023         return 0;
00024
00025     currentpart = *output;
00026     *currentpart = srcstr;
00027
00028     for (int i = 0; i < len; i++)
00029     {
00030         if (srcstr[i] == '\\0')
00031         {
00032             currentpart++;
00033             *currentpart = &srcstr[i + 1];
00034         }
00035     }
00036
00037     return numparts;
00038 }
```

7.179 src/terminal/framebuffer.c File Reference

```
#include <stddef.h>
#include <stdint.h>
#include "../include/terminal/framebuffer.h"
#include "../include/terminal/term.h"
```

Include dependency graph for framebuffer.c:



Macros

- `#define FONT_BYTES ((font_width * font_height * FBTERM_FONT_GLYPHS) / 8)`

Functions

- `void * memset (void *, int, size_t)`
Fill a memory region with one byte value.
- `void * memcpy (void *, const void *, size_t)`
- `static void fbterm_save_state (struct term_context *_ctx)`
- `static void fbterm_restore_state (struct term_context *_ctx)`
- `static void fbterm_swap_palette (struct term_context *_ctx)`
- `static void plot_char (struct term_context *_ctx, struct fbterm_char *c, size_t x, size_t y)`
- `static void plot_char_fast (struct term_context *_ctx, struct fbterm_char *old, struct fbterm_char *c, size_t x, size_t y)`
- `static bool compare_char (struct fbterm_char *a, struct fbterm_char *b)`
- `static void push_to_queue (struct term_context *_ctx, struct fbterm_char *c, size_t x, size_t y)`
- `static void fbterm_revscroll (struct term_context *_ctx)`
- `static void fbterm_scroll (struct term_context *_ctx)`
- `static void fbterm_clear (struct term_context *_ctx, bool move)`
- `static void fbterm_enable_cursor (struct term_context *_ctx)`
- `static bool fbterm_disable_cursor (struct term_context *_ctx)`
- `static void fbterm_set_cursor_pos (struct term_context *_ctx, size_t x, size_t y)`
- `static void fbterm_get_cursor_pos (struct term_context *_ctx, size_t *x, size_t *y)`
- `static void fbterm_move_character (struct term_context *_ctx, size_t new_x, size_t new_y, size_t old_x, size_t old_y)`
- `static void fbterm_set_text_fg (struct term_context *_ctx, size_t fg)`

- static void `fbterm_set_text_bg` (struct `term_context` *_ctx, size_t bg)
- static void `fbterm_set_text_fg_bright` (struct `term_context` *_ctx, size_t fg)
- static void `fbterm_set_text_bg_bright` (struct `term_context` *_ctx, size_t bg)
- static void `fbterm_set_text_fg_rgb` (struct `term_context` *_ctx, uint32_t fg)
- static void `fbterm_set_text_bg_rgb` (struct `term_context` *_ctx, uint32_t bg)
- static void `fbterm_set_text_fg_default` (struct `term_context` *_ctx)
- static void `fbterm_set_text_bg_default` (struct `term_context` *_ctx)
- static void `draw_cursor` (struct `term_context` *_ctx)
- static void `fbterm_double_buffer_flush` (struct `term_context` *_ctx)
- static void `fbterm_raw_putchar` (struct `term_context` *_ctx, uint8_t c)
- static void `fbterm_full_refresh` (struct `term_context` *_ctx)
- static void `fbterm_deinit` (struct `term_context` *_ctx, void(*_free)(void *, size_t))
- struct `term_context` * `fbterm_init` (void *(*_malloc)(size_t), uint32_t *framebuffer, size_t width, size_t height, size_t pitch, uint32_t *canvas, uint32_t *ansi_colours, uint32_t *ansi_bright_colours, uint32_t *default_bg, uint32_t *default_fg, void *font, size_t font_width, size_t font_height, size_t font_spacing, size_t font_scale_x, size_t font_scale_y, size_t margin)

Initialize a framebuffer terminal and return its generic context.

Variables

- static const uint8_t `builtin_font` []

7.179.1 Macro Definition Documentation

7.179.1.1 FONT_BYTES

```
#define FONT_BYTES ((font_width * font_height * FBTERM_FONT_GLYPHS) / 8)
```

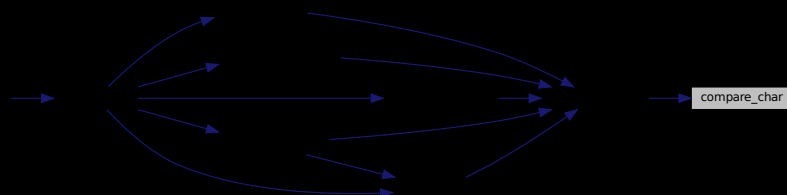
7.179.2 Function Documentation

7.179.2.1 compare_char()

```
static bool compare_char (
    struct fbterm_char * a,
    struct fbterm_char * b ) [inline], [static]
```

Definition at line 460 of file `framebuffer.c`.

Here is the caller graph for this function:

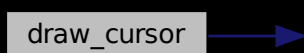


7.179.2.2 draw_cursor()

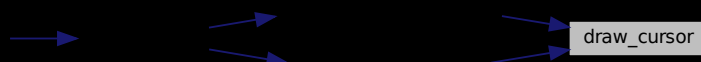
```
static void draw_cursor (  
    struct term_context * _ctx ) [static]
```

Definition at line 733 of file [framebuffer.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.179.2.3 fbterm_clear()

```
static void fbterm_clear (  
    struct term_context * _ctx,  
    bool move ) [static]
```

Definition at line 572 of file [framebuffer.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.179.2.4 fbterm_deinit()

```
static void fbterm_deinit (  
    struct term_context * _ctx,  
    void(*) (void *, size_t) _free ) [static]
```

Definition at line 856 of file `framebuffer.c`.

Here is the caller graph for this function:



7.179.2.5 fbterm_disable_cursor()

```
static bool fbterm_disable_cursor (  
    struct term_context * _ctx ) [static]
```

Definition at line 599 of file `framebuffer.c`.

Here is the caller graph for this function:

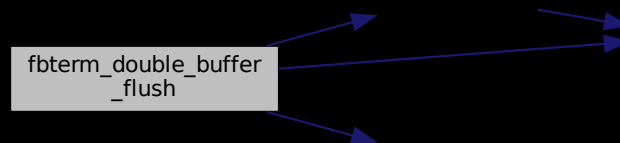


7.179.2.6 fbterm_double_buffer_flush()

```
static void fbterm_double_buffer_flush (  
    struct term_context * _ctx ) [static]
```

Definition at line 759 of file [framebuffer.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.179.2.7 fbterm_enable_cursor()

```
static void fbterm_enable_cursor (  
    struct term_context * _ctx ) [static]
```

Definition at line 592 of file [framebuffer.c](#).

Here is the caller graph for this function:



7.179.2.8 fbterm_full_refresh()

```
static void fbterm_full_refresh (  
    struct term_context * _ctx ) [static]
```

Definition at line 829 of file [framebuffer.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.179.2.9 fbterm_get_cursor_pos()

```
static void fbterm_get_cursor_pos (  
    struct term_context * _ctx,  
    size_t * x,  
    size_t * y ) [static]
```

Definition at line 639 of file [framebuffer.c](#).

Here is the caller graph for this function:



7.179.2.10 fbterm_init()

```
struct term_context * fbterm_init (
    void (*)(size_t) _malloc,
    uint32_t * framebuffer,
    size_t width,
    size_t height,
    size_t pitch,
    uint32_t * canvas,
    uint32_t * ansi_colours,
    uint32_t * ansi_bright_colours,
    uint32_t * default_bg,
    uint32_t * default_fg,
    void * font,
    size_t font_width,
    size_t font_height,
    size_t font_spacing,
    size_t font_scale_x,
    size_t font_scale_y,
    size_t margin )
```

Initialize a framebuffer terminal and return its generic context.

Parameters

<i>_malloc</i>	Allocation function used for terminal-owned dynamic storage.
<i>framebuffer</i>	Kernel-accessible framebuffer base.
<i>width</i>	Framebuffer width in pixels.
<i>height</i>	Framebuffer height in pixels.
<i>pitch</i>	Framebuffer pitch in bytes.
<i>canvas</i>	Optional backing canvas supplied by the caller.
<i>ansi_colours</i>	Optional normal ANSI palette.
<i>ansi_bright_colours</i>	Optional bright ANSI palette.
<i>default_bg</i>	Pointer to the default background colour.
<i>default_fg</i>	Pointer to the default foreground colour.
<i>font</i>	Optional caller-supplied font data.
<i>font_width</i>	Caller-supplied font width when applicable.
<i>font_height</i>	Caller-supplied font height when applicable.
<i>font_spacing</i>	Horizontal font spacing.
<i>font_scale_x</i>	Horizontal scale factor.
<i>font_scale_y</i>	Vertical scale factor.
<i>margin</i>	Terminal margin in pixels/cells as interpreted by the backend.

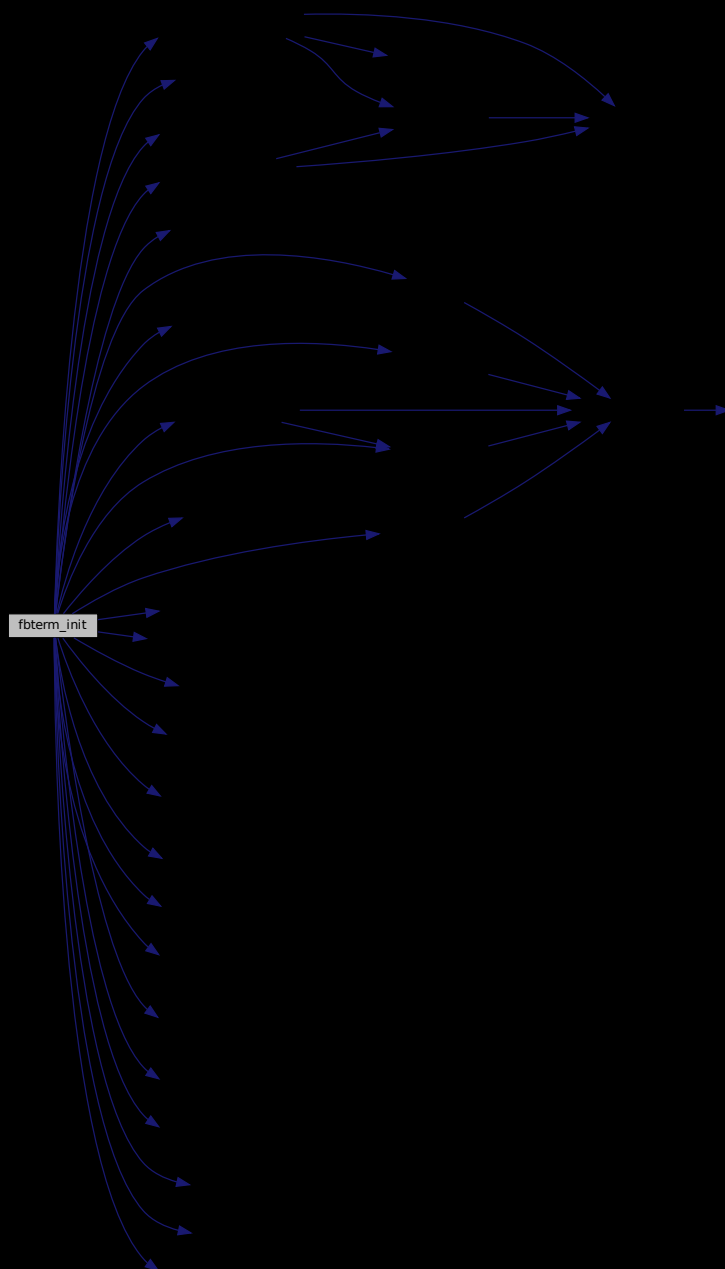
Returns

Initialized backend-independent terminal context, or NULL on failure.

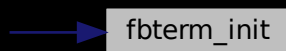
VoyagerOS64 initializes this only after its own page tables and heap are live; the resulting context replaces Limine terminal callbacks for normal kernel console output.

Definition at line 870 of file [framebuffer.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.179.2.11 fbterm_move_character()

```
static void fbterm_move_character (
    struct term_context * _ctx,
    size_t new_x,
    size_t new_y,
    size_t old_x,
    size_t old_y ) [static]
```

Definition at line 647 of file `framebuffer.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.179.2.12 fbterm_raw_putchar()

```
static void fbterm_raw_putchar (  
    struct term_context * _ctx,  
    uint8_t c ) [static]
```

Definition at line 806 of file [framebuffer.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.179.2.13 fbterm_restore_state()

```
static void fbterm_restore_state (  
    struct term_context * _ctx ) [static]
```

Definition at line 365 of file [framebuffer.c](#).

Here is the caller graph for this function:



7.179.2.14 fbterm_revscroll()

```
static void fbterm_revscroll (  
    struct term_context * _ctx ) [static]
```

Definition at line 496 of file [framebuffer.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.179.2.15 fbterm_save_state()

```
static void fbterm_save_state (  
    struct term_context * _ctx ) [static]
```

Definition at line 356 of file [framebuffer.c](#).

Here is the caller graph for this function:



7.179.2.16 fbterm_scroll()

```
static void fbterm_scroll (  
    struct term_context * _ctx ) [static]
```

Definition at line 535 of file [framebuffer.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.179.2.17 fbterm_set_cursor_pos()

```
static void fbterm_set_cursor_pos (  
    struct term_context * _ctx,  
    size_t x,  
    size_t y ) [static]
```

Definition at line 608 of file [framebuffer.c](#).

Here is the caller graph for this function:



7.179.2.18 fbterm_set_text_bg()

```
static void fbterm_set_text_bg (
    struct term_context * _ctx,
    size_t bg ) [static]
```

Definition at line 684 of file [framebuffer.c](#).

Here is the caller graph for this function:

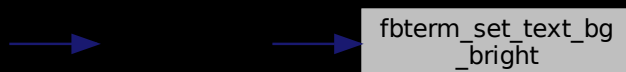


7.179.2.19 fbterm_set_text_bg_bright()

```
static void fbterm_set_text_bg_bright (
    struct term_context * _ctx,
    size_t bg ) [static]
```

Definition at line 698 of file [framebuffer.c](#).

Here is the caller graph for this function:



7.179.2.20 fbterm_set_text_bg_default()

```
static void fbterm_set_text_bg_default (
    struct term_context * _ctx ) [static]
```

Definition at line 726 of file [framebuffer.c](#).

Here is the caller graph for this function:



7.179.2.21 fbterm_set_text_bg_rgb()

```
static void fbterm_set_text_bg_rgb (
    struct term_context * _ctx,
    uint32_t bg ) [static]
```

Definition at line 712 of file [framebuffer.c](#).

Here is the caller graph for this function:



7.179.2.22 fbterm_set_text_fg()

```
static void fbterm_set_text_fg (  
    struct term_context * _ctx,  
    size_t fg ) [static]
```

Definition at line 677 of file [framebuffer.c](#).

Here is the caller graph for this function:



7.179.2.23 fbterm_set_text_fg_bright()

```
static void fbterm_set_text_fg_bright (  
    struct term_context * _ctx,  
    size_t fg ) [static]
```

Definition at line 691 of file [framebuffer.c](#).

Here is the caller graph for this function:



7.179.2.24 fbterm_set_text_fg_default()

```
static void fbterm_set_text_fg_default (
    struct term_context * _ctx ) [static]
```

Definition at line 719 of file [framebuffer.c](#).

Here is the caller graph for this function:



7.179.2.25 fbterm_set_text_fg_rgb()

```
static void fbterm_set_text_fg_rgb (
    struct term_context * _ctx,
    uint32_t fg ) [static]
```

Definition at line 705 of file [framebuffer.c](#).

Here is the caller graph for this function:



7.179.2.26 fbterm_swap_palette()

```
static void fbterm_swap_palette (
    struct term_context * _ctx ) [static]
```

Definition at line 374 of file [framebuffer.c](#).

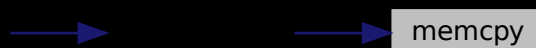
Here is the caller graph for this function:



7.179.2.27 memcpy()

```
void * memcpy (
    void * ,
    const void * ,
    size_t )
```

Here is the caller graph for this function:



7.179.2.28 memset()

```
void * memset (
    void * bufptr,
    int value,
    size_t size )
```

Fill a memory region with one byte value.

Parameters

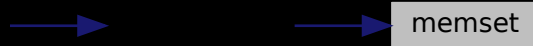
<i>bufptr</i>	Destination memory region.
<i>value</i>	Value whose low eight bits are written to each byte.
<i>size</i>	Number of bytes to set.

Returns

bufptr.

Definition at line 10 of file [memset.c](#).

Here is the caller graph for this function:



7.179.2.29 plot_char()

```
static void plot_char (  
    struct term_context * _ctx,  
    struct fbterm_char * c,  
    size_t x,  
    size_t y ) [static]
```

Definition at line 383 of file [framebuffer.c](#).

Here is the caller graph for this function:



7.179.3 Variable Documentation

7.179.3.1 builtin_font

```
const uint8_t builtin_font[] [static]
```

Definition at line 12 of file [framebuffer.c](#).

7.180 framebuffer.c

Go to the documentation of this file.

```
00001 #include <stddef.h>
00002 #include <stdint.h>
00003
00004 #include "../include/terminal/framebuffer.h"
00005 #include "../include/terminal/term.h"
00006
00007 void *memset(void *, int, size_t);
00008 void *memcpy(void *, const void *, size_t);
00009
00010 // Builtin font originally taken from:
00011 // https://github.com/viler-int10h/vga-text-mode-fonts/raw/master/FONTS/PC-OTHER/TOSH-SAT.F16
00012 static const uint8_t builtin_font[] = {
00013     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00014     0x00, 0x00, 0x00, 0x00, 0x00, 0x3c, 0x42, 0x81, 0x81, 0xa5, 0xa5, 0x81, 0x00, 0x00, 0x00, 0x00, 0x00,
00015     0x81, 0xa5, 0x99, 0x81, 0x42, 0x3c, 0x00, 0x00, 0x00, 0x3c, 0x7e, 0xff, 0x00, 0x00, 0x00, 0x00,
00016     0xff, 0xdb, 0xdb, 0xff, 0xff, 0xdb, 0xe7, 0xff, 0x7e, 0x3c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00017     0x00, 0x00, 0x00, 0x6c, 0xfe, 0xfe, 0xfe, 0x7c, 0x7c, 0x38, 0x38, 0x10, 0x00, 0x00, 0x00, 0x00,
00018     0x10, 0x00, 0x00, 0x00, 0x00, 0x10, 0x10, 0x38, 0x38, 0x7c, 0x7c, 0xfe, 0x00, 0x00, 0x00, 0x00,
00019     0x7c, 0x7c, 0x38, 0x38, 0x10, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x18, 0x18,
00020     0x3c, 0x3c, 0xdb, 0xff, 0xff, 0xdb, 0x18, 0x18, 0x3c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00021     0x00, 0x00, 0x00, 0x18, 0x3c, 0x7e, 0xff, 0xff, 0xff, 0x66, 0x18, 0x18, 0x00, 0x00, 0x00, 0x00, 0x00,
00022     0x3c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x78,
00023     0x78, 0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
00024     0xff, 0xff, 0xe7, 0xc3, 0xc3, 0xe7, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
00025     0x00, 0x00, 0x00, 0x00, 0x00, 0x78, 0xcc, 0x84, 0x84, 0xcc, 0x78, 0x00, 0x00, 0x00, 0x00, 0x00,
00026     0x00, 0x00, 0x00, 0x00, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xc3, 0x99, 0xbd, 0x00, 0x00, 0x00, 0x00,
00027     0xbd, 0x99, 0xc3, 0xff, 0xff, 0xff, 0xff, 0xff, 0x00, 0x00, 0x00, 0x1e, 0x00, 0x00, 0x00, 0x00,
00028     0x0e, 0x1e, 0x32, 0x78, 0xcc, 0xcc, 0xcc, 0x78, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00029     0x00, 0x00, 0x00, 0x78, 0xcc, 0xcc, 0xcc, 0x78, 0x30, 0xfc, 0x30, 0x30, 0x00, 0x00, 0x00, 0x00, 0x00,
00030     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x18, 0x1c, 0x1e, 0x16, 0x12, 0x00, 0x00, 0x00, 0x00,
00031     0x10, 0x10, 0x70, 0xf0, 0xe0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x38, 0x2c, 0x00, 0x00, 0x00, 0x00,
00032     0x26, 0x32, 0x3a, 0x2e, 0x26, 0x22, 0x62, 0xe2, 0xc6, 0x0e, 0x0c, 0x00, 0x00, 0x00, 0x00, 0x00,
00033     0x00, 0x00, 0x00, 0x18, 0x18, 0xdb, 0x3c, 0xe7, 0x3c, 0xdb, 0x18, 0x18, 0x00, 0x00, 0x00, 0x00, 0x00,
00034     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0xc0, 0xe0, 0xf8, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00,
00035     0xf8, 0xe0, 0xc0, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x02,
00036     0x06, 0x0e, 0x3e, 0xfe, 0x3e, 0x0e, 0x06, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00037     0x00, 0x00, 0x30, 0x78, 0xfc, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0xfc, 0x78, 0x00, 0x00, 0x00, 0x00,
00038     0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0xcc, 0xcc, 0xcc, 0xcc, 0xcc, 0xcc, 0xcc, 0xcc, 0xcc, 0xcc,
00039     0xcc, 0xcc, 0x00, 0xcc, 0xcc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7f, 0xdb, 0x00, 0x00, 0x00, 0x00,
00040     0xdb, 0xdb, 0xdb, 0x7b, 0x1b, 0x1b, 0x1b, 0x1b, 0x1b, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00041     0x00, 0x00, 0x7c, 0xc6, 0x60, 0x38, 0x6c, 0xc6, 0xc6, 0xc6, 0x6c, 0x38, 0x0c, 0x00, 0x00, 0x00, 0x00,
00042     0xc6, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00043     0x00, 0x00, 0xfe, 0xfe, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x78, 0x00, 0x00, 0x00, 0x00,
00044     0xfc, 0x30, 0x30, 0x30, 0x30, 0x30, 0xfc, 0x78, 0x30, 0xfc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00045     0x00, 0x00, 0x30, 0x78, 0xfc, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30,
00046     0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30,
00047     0x30, 0x30, 0xfc, 0x78, 0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00048     0x00, 0x18, 0x0c, 0xfe, 0xfe, 0x0c, 0x18, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00049     0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x60, 0xfe, 0xfe, 0x60, 0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00050     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xc0, 0x00,
00051     0xc0, 0xc0, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00052     0x00, 0x24, 0x66, 0xff, 0x66, 0x24, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00053     0x00, 0x00, 0x00, 0x00, 0x10, 0x10, 0x38, 0x38, 0x7c, 0x7c, 0xfe, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00,
00054     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0xfe, 0x7c, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00,
00055     0x38, 0x38, 0x10, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00056     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00057     0x00, 0x00, 0x30, 0x78, 0x78, 0x78, 0x30, 0x30, 0x30, 0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00058     0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0xcc, 0xcc, 0xcc, 0xcc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00059     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x6c, 0x6c, 0x00, 0x00, 0x00, 0x00,
00060     0x6c, 0xfe, 0x6c, 0x6c, 0x6c, 0xfe, 0x6c, 0x6c, 0x6c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
```

```
00061 0x00, 0x18, 0x18, 0x7c, 0xc6, 0xc0, 0xc0, 0x7c, 0x06, 0x06, 0xc6, 0x7c,
00062 0x18, 0x18, 0x00, 0x00, 0x00, 0x00, 0xc6, 0xc6, 0x0c, 0x0c, 0x18, 0x38,
00063 0x30, 0x60, 0x60, 0xc6, 0xc6, 0x00, 0x00, 0x00, 0x00, 0x38, 0xc6,
00064 0x6c, 0x38, 0x30, 0x76, 0xde, 0xcc, 0xcc, 0xde, 0x76, 0x00, 0x00, 0x00,
00065 0x00, 0x00, 0x18, 0x18, 0x18, 0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00066 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x18, 0x30, 0x60, 0x60, 0x60,
00067 0x60, 0x60, 0x60, 0x30, 0x18, 0x00, 0x00, 0x00, 0x00, 0x00, 0x60, 0x30,
00068 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x30, 0x60, 0x00, 0x00, 0x00,
00069 0x00, 0x00, 0x00, 0x00, 0x00, 0x6c, 0x38, 0xfe, 0x38, 0x6c, 0x00, 0x00,
00070 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x18, 0x18, 0x7e,
00071 0x18, 0x18, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00072 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x18, 0x18, 0x30, 0x00, 0x00,
00073 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0x00, 0x00, 0x00, 0x00,
00074 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00075 0x00, 0x00, 0x00, 0x18, 0x18, 0x00, 0x00, 0x00, 0x00, 0x00, 0x06, 0x06,
00076 0x0c, 0x0c, 0x18, 0x38, 0x30, 0x60, 0x60, 0xc0, 0xc0, 0x00, 0x00, 0x00,
00077 0x00, 0x00, 0x7c, 0xc6, 0xc6, 0xc6, 0xd6, 0xd6, 0xd6, 0xc6, 0xc6, 0xc6,
00078 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x18, 0x38, 0x78, 0x18, 0x18, 0x18,
00079 0x18, 0x18, 0x18, 0x18, 0x7e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0xc6,
00080 0x06, 0x06, 0x0c, 0x18, 0x30, 0x60, 0xc0, 0xc0, 0xfe, 0x00, 0x00, 0x00,
00081 0x00, 0x00, 0x7c, 0xc6, 0x06, 0x06, 0x3c, 0x06, 0x06, 0x06, 0xc6,
00082 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x04, 0x0c, 0x1c, 0x3c, 0x6c, 0xcc,
00083 0xfe, 0x0c, 0x0c, 0x0c, 0x0c, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0xc0,
00084 0xc0, 0xc0, 0xfc, 0x06, 0x06, 0x06, 0x06, 0xc6, 0x7c, 0x00, 0x00, 0x00,
00085 0x00, 0x00, 0x3c, 0x60, 0xc0, 0xc0, 0xfc, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6,
00086 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0xc6, 0x06, 0x06, 0x0c, 0x18,
00087 0x30, 0x30, 0x30, 0x30, 0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0xc6,
00088 0xc6, 0xc6, 0x7c, 0xc6, 0xc6, 0xc6, 0xc6, 0x7c, 0x00, 0x00, 0x00,
00089 0x00, 0x00, 0x7c, 0xc6, 0xc6, 0xc6, 0xc6, 0x7e, 0x06, 0x06, 0x06, 0xc,
00090 0x78, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x18,
00091 0x18, 0x00, 0x00, 0x18, 0x18, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00092 0x00, 0x00, 0x00, 0x18, 0x18, 0x00, 0x00, 0x18, 0x18, 0x30, 0x00, 0x00,
00093 0x00, 0x00, 0x06, 0x0c, 0x18, 0x30, 0x60, 0xc0, 0x60, 0x30, 0x18, 0x0c,
00094 0x06, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0x00,
00095 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xc0, 0x60,
00096 0x30, 0x18, 0x0c, 0x06, 0x0c, 0x18, 0x30, 0x60, 0xc0, 0x00, 0x00, 0x00,
00097 0x00, 0x00, 0x7c, 0xc6, 0xc6, 0x06, 0x0c, 0x18, 0x30, 0x30, 0x00, 0x30,
00098 0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0xc6, 0xc6, 0xc6, 0xde, 0xde,
00099 0xde, 0xde, 0xc0, 0xc0, 0x7e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x38, 0xc6,
00100 0xc6, 0xc6, 0xc6, 0xc6, 0xfe, 0xc6, 0xc6, 0xc6, 0xc6, 0x00, 0x00, 0x00,
00101 0x00, 0x00, 0xfc, 0xc6, 0xc6, 0xc6, 0xfc, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6,
00102 0xfc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0xc6, 0xc0, 0xc0, 0xc0, 0xc0,
00103 0xc0, 0xc0, 0xc0, 0xc6, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0xf8, 0xcc,
00104 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xcc, 0xf8, 0x00, 0x00, 0x00,
00105 0x00, 0x00, 0xfe, 0xc0, 0xc0, 0xc0, 0xc0, 0xc0, 0xc0, 0xc0, 0xc0, 0xc0,
00106 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0xc0, 0xc0, 0xc0, 0xfc, 0xc0,
00107 0xc0, 0xc0, 0xc0, 0xc0, 0xc0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0xc6,
00108 0xc0, 0xc0, 0xc0, 0xc6, 0xc6, 0xc6, 0xc6, 0x7e, 0x00, 0x00, 0x00,
00109 0x00, 0x00, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xfe, 0xc6, 0xc6, 0xc6,
00110 0xc6, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfc, 0x30, 0x30, 0x30, 0x30, 0x30,
00111 0x30, 0x30, 0x30, 0x30, 0xfc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x1e, 0x0c,
00112 0x0c, 0x0c, 0x0c, 0x0c, 0x0c, 0x0c, 0xcc, 0xcc, 0x78, 0x00, 0x00, 0x00,
00113 0x00, 0x00, 0xc6, 0xc6, 0xcc, 0xd8, 0xf0, 0xe0, 0xf0, 0xd8, 0xcc, 0xc6,
00114 0xc6, 0x00, 0x00, 0x00, 0x00, 0x00, 0xc0, 0xc0, 0xc0, 0xc0, 0xc0, 0xc0,
00115 0xc0, 0xc0, 0xc0, 0xc0, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0xc6, 0xc6,
00116 0xee, 0xfe, 0xd6, 0xd6, 0xd6, 0xc6, 0xc6, 0xc6, 0xc6, 0x00, 0x00, 0x00,
00117 0x00, 0x00, 0xc6, 0xc6, 0xe6, 0xe6, 0xf6, 0xde, 0xce, 0xc6, 0xc6,
00118 0xc6, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6,
00119 0xc6, 0xc6, 0xc6, 0xc6, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfc, 0xc6,
00120 0xc6, 0xc6, 0xc6, 0xfc, 0xc0, 0xc0, 0xc0, 0xc0, 0x00, 0x00, 0x00,
00121 0x00, 0x00, 0x7c, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xf6, 0xda,
00122 0x6c, 0x06, 0x00, 0x00, 0x00, 0x00, 0xfc, 0xc6, 0xc6, 0xc6, 0xc6, 0xfc,
00123 0xd8, 0xcc, 0xcc, 0xc6, 0xc6, 0x00, 0x00, 0x00, 0x00, 0x7c, 0xc6,
00124 0xc0, 0x60, 0x30, 0x18, 0x0c, 0x06, 0x06, 0xc6, 0x7c, 0x00, 0x00, 0x00,
00125 0x00, 0x00, 0xff, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18,
00126 0x18, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6,
00127 0xc6, 0xc6, 0xc6, 0xc6, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0xc6, 0xc6,
00128 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0x38, 0x10, 0x00, 0x00, 0x00,
00129 0x00, 0x00, 0xc6, 0xc6, 0xc6, 0xc6, 0xd6, 0xd6, 0xd6, 0xd6, 0xfe, 0x6c,
00130 0x6c, 0x00, 0x00, 0x00, 0x00, 0x00, 0xc6, 0xc6, 0xc6, 0x6c, 0x38, 0x38,
00131 0x38, 0x6c, 0xc6, 0xc6, 0xc6, 0xc6, 0x00, 0x00, 0x00, 0x00, 0xcc, 0xcc,
00132 0xcc, 0xcc, 0xcc, 0x78, 0x30, 0x30, 0x30, 0x30, 0x30, 0x00, 0x00, 0x00,
00133 0x00, 0x00, 0xfe, 0x06, 0x06, 0x0c, 0x18, 0x30, 0x60, 0xc0, 0xc0, 0xc0,
00134 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x78, 0x60, 0x60, 0x60, 0x60, 0x60,
00135 0x60, 0x60, 0x60, 0x60, 0x78, 0x00, 0x00, 0x00, 0x00, 0x00, 0xc0, 0xc0,
00136 0x60, 0x60, 0x30, 0x38, 0x18, 0x0c, 0x0c, 0x06, 0x06, 0x00, 0x00, 0x00,
00137 0x00, 0x00, 0x78, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18,
00138 0x78, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x38, 0x6c, 0xc6, 0x00, 0x00,
00139 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00140 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xff, 0x00,
00141 0x00, 0x00, 0x18, 0x18, 0x18, 0x0c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00142 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0x06, 0x06,
00143 0x7e, 0xc6, 0xc6, 0xc6, 0xc6, 0x7e, 0x00, 0x00, 0x00, 0x00, 0x00, 0xc0,
00144 0xc0, 0xdc, 0xe6, 0xc6, 0xc6, 0xc6, 0xc6, 0xe6, 0xdc, 0x00, 0x00, 0x00,
00145 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0xc6, 0xc0, 0xc0, 0xc0, 0xc0, 0xc6,
00146 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x06, 0x06, 0x76, 0xce, 0xc6,
00147 0xc6, 0xc6, 0xc6, 0xce, 0x76, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
```

```
00148 0x00, 0x7c, 0xc6, 0xc6, 0xfe, 0xc0, 0xc0, 0xc0, 0x7e, 0x00, 0x00, 0x00,
00149 0x00, 0x00, 0x1c, 0x36, 0x30, 0x30, 0xfc, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30,
00150 0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x76, 0xce, 0xc6,
00151 0xc6, 0xc6, 0xce, 0x76, 0x06, 0xc6, 0x7c, 0x00, 0x00, 0x00, 0xc0, 0xc0,
00152 0xc0, 0xdc, 0xe6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0x00, 0x00, 0x00,
00153 0x00, 0x00, 0x18, 0x00, 0x00, 0x38, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18,
00154 0x3c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x06, 0x00, 0x00, 0x1e, 0x06, 0x06,
00155 0x06, 0x06, 0x06, 0x06, 0xc6, 0xc6, 0x7c, 0x00, 0x00, 0x00, 0xc0, 0xc0,
00156 0xc0, 0xc6, 0xcc, 0xd8, 0xf0, 0xf0, 0xd8, 0xcc, 0xc6, 0x00, 0x00, 0x00,
00157 0x00, 0x00, 0x38, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18,
00158 0x3c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xec, 0xfe, 0xd6,
00159 0xd6, 0xd6, 0xd6, 0xc6, 0xc6, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00160 0x00, 0xdc, 0xe6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0x00, 0x00, 0x00,
00161 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6,
00162 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xdc, 0xe6, 0xc6,
00163 0xc6, 0xc6, 0xe6, 0xdc, 0xc0, 0xc0, 0xc0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00164 0x00, 0x76, 0xce, 0xc6, 0xc6, 0xc6, 0xce, 0x76, 0x06, 0x06, 0x06, 0x00,
00165 0x00, 0x00, 0x00, 0x00, 0x00, 0xdc, 0xe6, 0xc0, 0xc0, 0xc0, 0xc0, 0xc0,
00166 0xc0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0xc6, 0xc0,
00167 0x70, 0x1c, 0x06, 0xc6, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x30,
00168 0x30, 0xfe, 0xc6, 0x30, 0x30, 0x30, 0x30, 0x36, 0x1c, 0x00, 0x00, 0x00,
00169 0x00, 0x00, 0x00, 0x00, 0x00, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6,
00170 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xc6, 0xc6, 0xc6,
00171 0xc6, 0xc6, 0x6c, 0x38, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00172 0x00, 0xc6, 0xc6, 0xd6, 0xd6, 0xd6, 0xd6, 0xfe, 0x6c, 0x00, 0x00, 0x00,
00173 0x00, 0x00, 0x00, 0x00, 0x00, 0xc6, 0xc6, 0x6c, 0x38, 0x38, 0x6c, 0xc6,
00174 0xc6, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xc6, 0xc6, 0xc6,
00175 0xc6, 0xc6, 0xce, 0x76, 0x06, 0xc6, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00,
00176 0x00, 0xfe, 0x06, 0x0c, 0x18, 0x30, 0x60, 0xc0, 0xfe, 0x00, 0x00, 0x00,
00177 0x00, 0x00, 0x1c, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0xe0, 0x30, 0x30, 0x30,
00178 0x1c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30,
00179 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x00, 0x00, 0x00, 0xe0, 0x30, 0x30,
00180 0x30, 0x30, 0x30, 0x1c, 0x30, 0x30, 0x30, 0x30, 0xe0, 0x00, 0x00, 0x00,
00181 0x00, 0x00, 0x00, 0x76, 0xdc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00182 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x10, 0x38, 0x38, 0x6c,
00183 0x6c, 0xc6, 0xc6, 0xc6, 0xc6, 0x00, 0x00, 0x00, 0x00, 0x00, 0x3c, 0x66,
00184 0xc0, 0xc0, 0xc0, 0xc0, 0xc0, 0xc0, 0x66, 0x3c, 0x18, 0xcc, 0x78, 0x00,
00185 0x00, 0x00, 0x6c, 0x6c, 0x00, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6,
00186 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0c, 0x18, 0x30, 0x00, 0x7c, 0xc6,
00187 0xfe, 0xc0, 0xc0, 0xc0, 0x7e, 0x00, 0x00, 0x00, 0x00, 0x10, 0x38, 0x6c,
00188 0x00, 0x7c, 0x06, 0x06, 0x7e, 0xc6, 0xc6, 0xc6, 0x7e, 0x00, 0x00, 0x00,
00189 0x00, 0x00, 0x6c, 0x6c, 0x00, 0x7c, 0x06, 0x06, 0x7e, 0xc6, 0xc6, 0xc6,
00190 0x7e, 0x00, 0x00, 0x00, 0x00, 0x60, 0x30, 0x18, 0x00, 0x7c, 0x06, 0x06,
00191 0x7e, 0xc6, 0xc6, 0xc6, 0x7e, 0x00, 0x00, 0x00, 0x00, 0x38, 0x6c, 0x38,
00192 0x00, 0x00, 0x7c, 0x06, 0x06, 0x7e, 0xc6, 0xc6, 0xc6, 0x7e, 0x00, 0x00,
00193 0x00, 0x00, 0x00, 0x00, 0x00, 0x7c, 0xc6, 0xc0, 0xc0, 0xc0, 0xc0, 0xc6,
00194 0x7c, 0x18, 0x0c, 0x38, 0x00, 0x10, 0x38, 0x6c, 0x00, 0x7c, 0xc6, 0xc6,
00195 0xfe, 0xc0, 0xc0, 0xc0, 0x7e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x6c, 0x6c,
00196 0x00, 0x7c, 0xc6, 0xc6, 0xfe, 0xc0, 0xc0, 0xc0, 0x7e, 0x00, 0x00, 0x00,
00197 0x00, 0x60, 0x30, 0x18, 0x00, 0x7c, 0xc6, 0xc6, 0xfe, 0xc0, 0xc0, 0xc0,
00198 0x7e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x6c, 0x6c, 0x00, 0x38, 0x18, 0x18,
00199 0x18, 0x18, 0x18, 0x18, 0x3c, 0x00, 0x00, 0x00, 0x00, 0x10, 0x38, 0x6c,
00200 0x00, 0x38, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x3c, 0x00, 0x00, 0x00,
00201 0x00, 0x60, 0x30, 0x18, 0x00, 0x38, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18,
00202 0x3c, 0x00, 0x00, 0x00, 0xc6, 0xc6, 0x10, 0x38, 0x6c, 0xc6, 0xc6, 0xc6,
00203 0xfe, 0xc6, 0xc6, 0xc6, 0xc6, 0x00, 0x00, 0x00, 0x38, 0x6c, 0x38, 0x00,
00204 0x38, 0x6c, 0xc6, 0xc6, 0xc6, 0xc6, 0xfe, 0xc6, 0xc6, 0xc6, 0x00, 0x00,
00205 0x18, 0x30, 0x60, 0x00, 0xfe, 0xc0, 0xc0, 0xfc, 0xc0, 0xc0, 0xc0, 0xc0,
00206 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xec, 0x36, 0x36,
00207 0x76, 0xde, 0xd8, 0xc6, 0x6e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x1e, 0x3c,
00208 0x6c, 0xcc, 0xcc, 0xfe, 0xcc, 0xcc, 0xcc, 0xcc, 0xc6, 0x00, 0x00, 0x00,
00209 0x00, 0x10, 0x38, 0x6c, 0x00, 0x7c, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6,
00210 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x6c, 0x6c, 0x00, 0x7c, 0xc6, 0xc6,
00211 0xc6, 0xc6, 0xc6, 0xc6, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x60, 0x30, 0x18,
00212 0x00, 0x7c, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0x7c, 0x00, 0x00, 0x00,
00213 0x00, 0x10, 0x38, 0x6c, 0x00, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6,
00214 0x7c, 0x00, 0x00, 0x00, 0x00, 0x60, 0x30, 0x18, 0x00, 0xc6, 0xc6, 0xc6,
00215 0xc6, 0xc6, 0xc6, 0xc6, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x6c, 0x6c,
00216 0x00, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0x76, 0x06, 0xc6, 0x7c, 0x00,
00217 0x6c, 0x6c, 0x00, 0x7c, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6,
00218 0x7c, 0x00, 0x00, 0x00, 0x6c, 0x6c, 0x00, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6,
00219 0xc6, 0xc6, 0xc6, 0xc6, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30,
00220 0x30, 0x78, 0xcc, 0xc0, 0xc0, 0xcc, 0x78, 0x30, 0x30, 0x00, 0x00, 0x00,
00221 0x00, 0x00, 0x38, 0x6c, 0x60, 0x60, 0x60, 0xf8, 0x60, 0x60, 0x60, 0xe6,
00222 0xfc, 0x00, 0x00, 0x00, 0x00, 0x00, 0xcc, 0xcc, 0xcc, 0x78, 0x30, 0xfc,
00223 0x30, 0xfc, 0x30, 0x30, 0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0xf8, 0xcc,
00224 0xcc, 0xf8, 0xc4, 0xcc, 0xde, 0xcc, 0xcc, 0xc6, 0x00, 0x00, 0x00,
00225 0x00, 0x00, 0xe, 0x1b, 0x18, 0x18, 0x18, 0x7e, 0x18, 0x18, 0x18, 0x18,
00226 0x18, 0xd8, 0x70, 0x00, 0x00, 0x0c, 0x18, 0x30, 0x00, 0x7c, 0x06, 0x06,
00227 0x7e, 0xc6, 0xc6, 0xc6, 0x7e, 0x00, 0x00, 0x00, 0x00, 0x0c, 0x18, 0x30,
00228 0x00, 0x38, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x3c, 0x00, 0x00, 0x00,
00229 0x00, 0x0c, 0x18, 0x30, 0x00, 0x7c, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6,
00230 0x7c, 0x00, 0x00, 0x00, 0x00, 0x0c, 0x18, 0x30, 0x00, 0xc6, 0xc6, 0xc6,
00231 0xc6, 0xc6, 0xc6, 0xc6, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x76, 0xdc, 0x00,
00232 0x00, 0xdc, 0xe6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0x00, 0x00, 0x00,
00233 0x76, 0xdc, 0x00, 0xc6, 0xc6, 0xe6, 0xf6, 0xfe, 0xde, 0xce, 0xc6, 0xc6,
00234 0xc6, 0x00, 0x00, 0x00, 0x00, 0x78, 0xd8, 0xd8, 0x6c, 0x00, 0xfc, 0x00,
```

```
00235 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x38, 0x6c, 0x6c,
00236 0x38, 0x00, 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00237 0x00, 0x00, 0x18, 0x18, 0x00, 0x18, 0x00, 0x18, 0x30, 0x60, 0xc0, 0xc6, 0xc6,
00238 0x7c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00239 0xfe, 0xc0, 0xc0, 0xc0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00240 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0x06, 0x06, 0x06, 0x00, 0x00, 0x00, 0x00,
00241 0x00, 0x00, 0xc0, 0xc2, 0xc6, 0xcc, 0xd8, 0x30, 0x60, 0xdc, 0x86, 0x0c,
00242 0x18, 0x3e, 0x00, 0x00, 0x00, 0x00, 0xc0, 0xc2, 0xc6, 0xcc, 0xd8, 0x30,
00243 0x66, 0xce, 0x9e, 0x3e, 0x06, 0x06, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x30,
00244 0x00, 0x30, 0x30, 0x30, 0x78, 0x78, 0x78, 0x78, 0x30, 0x00, 0x00, 0x00,
00245 0x00, 0x00, 0x00, 0x00, 0x00, 0x36, 0x6c, 0xd8, 0x6c, 0x36, 0x00, 0x00,
00246 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xd8, 0x6c, 0x36,
00247 0x6c, 0xd8, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x22, 0x88, 0x22, 0x88,
00248 0x22, 0x88, 0x22, 0x88, 0x22, 0x88, 0x22, 0x88, 0x22, 0x88, 0x22, 0x88,
00249 0x55, 0xaa, 0x55, 0xaa, 0x55, 0xaa, 0x55, 0xaa, 0x55, 0xaa, 0x55, 0xaa,
00250 0x55, 0xaa, 0x55, 0xaa, 0xdd, 0x77, 0xdd, 0x77, 0xdd, 0x77, 0xdd, 0x77,
00251 0xdd, 0x77, 0xdd, 0x77, 0xdd, 0x77, 0xdd, 0x77, 0x18, 0x18, 0x18, 0x18,
00252 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18,
00253 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0xf8, 0xf8, 0x18, 0x18, 0x18,
00254 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0xf8, 0xf8, 0x18,
00255 0x18, 0xf8, 0xf8, 0x18, 0x18, 0x18, 0x18, 0x18, 0x36, 0x36, 0x36, 0x36,
00256 0x36, 0x36, 0x36, 0xf6, 0xf6, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36,
00257 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0xfe, 0x36, 0x36, 0x36,
00258 0x36, 0x36, 0x36, 0x36, 0x00, 0x00, 0x00, 0x00, 0xf8, 0xf8, 0x18,
00259 0x18, 0xf8, 0xf8, 0x18, 0x18, 0x18, 0x18, 0x18, 0x36, 0x36, 0x36, 0x36,
00260 0x36, 0xf6, 0xf6, 0x06, 0x06, 0xf6, 0xf6, 0x36, 0x36, 0x36, 0x36,
00261 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36,
00262 0x36, 0x36, 0x36, 0x36, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe, 0xfe, 0x06,
00263 0x06, 0xf6, 0xf6, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36,
00264 0x36, 0xf6, 0xf6, 0x06, 0x06, 0xfe, 0xfe, 0x00, 0x00, 0x00, 0x00,
00265 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0xfe, 0xfe, 0x00, 0x00, 0x00,
00266 0x00, 0x00, 0x00, 0x00, 0x18, 0x18, 0x18, 0x18, 0x18, 0xf8, 0xf8, 0x18,
00267 0x18, 0xf8, 0xf8, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00268 0x00, 0x00, 0x00, 0xf8, 0xf8, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18,
00269 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x1f, 0x1f, 0x00, 0x00, 0x00,
00270 0x00, 0x00, 0x00, 0x00, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18,
00271 0xff, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00272 0x00, 0x00, 0x00, 0xff, 0xff, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18,
00273 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x1f, 0x1f, 0x18, 0x18, 0x18,
00274 0x18, 0x18, 0x18, 0x18, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00275 0xff, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x18, 0x18, 0x18,
00276 0x18, 0x18, 0x18, 0xff, 0xff, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18,
00277 0x18, 0x18, 0x18, 0x18, 0x18, 0x1f, 0x1f, 0x18, 0x18, 0x1f, 0x1f,
00278 0x18, 0x18, 0x18, 0x18, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36,
00279 0x37, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36,
00280 0x36, 0x37, 0x37, 0x30, 0x30, 0x3f, 0x3f, 0x00, 0x00, 0x00, 0x00,
00281 0x00, 0x00, 0x00, 0x00, 0x00, 0x3f, 0x3f, 0x30, 0x30, 0x37, 0x37, 0x36,
00282 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0xf7, 0xf7, 0x00,
00283 0x00, 0xff, 0xff, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00284 0x00, 0xff, 0xff, 0x00, 0x00, 0xf7, 0xf7, 0x36, 0x36, 0x36, 0x36,
00285 0x36, 0x36, 0x36, 0x36, 0x36, 0x37, 0x37, 0x30, 0x30, 0x37, 0x37,
00286 0x36, 0x36, 0x36, 0x36, 0x00, 0x00, 0x00, 0x00, 0x00, 0xff, 0xff, 0x00,
00287 0x00, 0xff, 0xff, 0x00, 0x00, 0x00, 0x00, 0x00, 0x36, 0x36, 0x36,
00288 0x36, 0xf7, 0xf7, 0x00, 0x00, 0xf7, 0xf7, 0x36, 0x36, 0x36, 0x36,
00289 0x18, 0x18, 0x18, 0x18, 0x18, 0xff, 0xff, 0x00, 0x00, 0xff, 0xff, 0x00,
00290 0x00, 0x00, 0x00, 0x00, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36,
00291 0xff, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00292 0x00, 0xff, 0xff, 0x00, 0x00, 0xff, 0xff, 0x18, 0x18, 0x18, 0x18,
00293 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xff, 0xff, 0x36, 0x36, 0x36,
00294 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x3f,
00295 0x3f, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x18, 0x18, 0x18,
00296 0x18, 0x1f, 0x1f, 0x18, 0x18, 0x1f, 0x1f, 0x00, 0x00, 0x00, 0x00,
00297 0x00, 0x00, 0x00, 0x00, 0x00, 0x1f, 0x1f, 0x18, 0x18, 0x1f, 0x1f,
00298 0x18, 0x18, 0x18, 0x18, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x3f,
00299 0x3f, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36,
00300 0x36, 0x36, 0x36, 0xf6, 0xff, 0x36, 0x36, 0x36, 0x36, 0x36, 0x36,
00301 0x18, 0x18, 0x18, 0x18, 0x18, 0xff, 0xff, 0x18, 0x18, 0xff, 0xff,
00302 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0xf8,
00303 0xf8, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00304 0x00, 0x00, 0x00, 0x1f, 0x1f, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18,
00305 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
00306 0xff, 0xff, 0xff, 0xf6, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00307 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0xf0, 0xf0,
00308 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0,
00309 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0, 0xf0,
00310 0xf0, 0xf0, 0xf0, 0xf0, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
00311 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00312 0x00, 0x76, 0xd6, 0xdc, 0xc8, 0xc8, 0xdc, 0xd6, 0x76, 0x00, 0x00, 0x00,
00313 0x00, 0x00, 0x78, 0xcc, 0xcc, 0xcc, 0xd8, 0xcc, 0xcc, 0xcc, 0xcc,
00314 0xd8, 0xc0, 0xc0, 0x00, 0x00, 0x00, 0xfe, 0xc6, 0xc6, 0xc0, 0xc0, 0xc0,
00315 0xc0, 0xc0, 0xc0, 0xc0, 0xc0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00316 0x00, 0x7e, 0xfe, 0x24, 0x24, 0x24, 0x24, 0x66, 0xc6, 0x00, 0x00, 0x00,
00317 0x00, 0x00, 0xfe, 0xfe, 0xc2, 0x60, 0x30, 0x18, 0x30, 0x60, 0xc2, 0xfe,
00318 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x7e, 0xc8, 0xcc,
00319 0xcc, 0xcc, 0xcc, 0xcc, 0x78, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00320 0x00, 0x66, 0x66, 0x66, 0x66, 0x66, 0x66, 0x76, 0x6c, 0x60, 0xc0, 0x00,
00321 0x00, 0x00, 0x00, 0x00, 0x00, 0x66, 0xfc, 0x98, 0x18, 0x18, 0x18, 0x18,
```

```

00322     0x18, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfc, 0x30, 0x30, 0x78, 0xcc, 0xcc,
00323     0xcc, 0x78, 0x30, 0x30, 0xfc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x38, 0x6c,
00324     0xc6, 0xc6, 0xc6, 0xfe, 0xc6, 0xc6, 0xc6, 0x38, 0x00, 0x00, 0x00,
00325     0x00, 0x00, 0x38, 0x6c, 0xc6, 0xc6, 0xc6, 0xc6, 0x6c, 0x6c, 0x6c,
00326     0xee, 0x00, 0x00, 0x00, 0x00, 0x00, 0x78, 0xcc, 0x60, 0x30, 0x78, 0xcc,
00327     0xcc, 0xcc, 0xcc, 0xcc, 0x78, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00328     0x00, 0x76, 0xbb, 0x99, 0x99, 0xdd, 0x6e, 0x00, 0x00, 0x00, 0x00, 0x00,
00329     0x00, 0x00, 0x02, 0x06, 0x3c, 0x6c, 0xce, 0xd6, 0xd6, 0xe6, 0x6c, 0x78,
00330     0xc0, 0x80, 0x00, 0x00, 0x00, 0x00, 0x1e, 0x30, 0x60, 0xc0, 0xc0, 0xfe,
00331     0xc0, 0xc0, 0x60, 0x30, 0x1e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x38, 0x6c,
00332     0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0xc6, 0x00, 0x00, 0x00,
00333     0x00, 0x00, 0x00, 0x00, 0xfe, 0x00, 0x00, 0xfe, 0x00, 0x00, 0xfe, 0x00,
00334     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x30, 0xfc,
00335     0x30, 0x30, 0x00, 0x00, 0xfc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00336     0x60, 0x30, 0x18, 0x0c, 0x18, 0x30, 0x60, 0x00, 0xfc, 0x00, 0x00, 0x00,
00337     0x00, 0x00, 0x00, 0x00, 0x18, 0x30, 0x60, 0xc0, 0x60, 0x30, 0x18, 0x00,
00338     0xfc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x1c, 0x36, 0x36, 0x30, 0x30, 0x30,
00339     0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x30, 0x18, 0x18, 0x18, 0x18,
00340     0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0xd8, 0xd8, 0x70, 0x00, 0x00, 0x00,
00341     0x00, 0x00, 0x00, 0x00, 0x30, 0x30, 0x00, 0xfc, 0x00, 0x30, 0x30, 0x00,
00342     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x76, 0xdc, 0x00,
00343     0x76, 0xdc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x78, 0xcc, 0xcc,
00344     0xcc, 0x78, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00345     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x30, 0x00, 0x00,
00346     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30,
00347     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0f, 0x0c, 0x0c,
00348     0x0c, 0x0c, 0x0c, 0x0c, 0x0c, 0xc, 0xc, 0x6c, 0x3c, 0x1c, 0x0c, 0x00, 0x00,
00349     0x00, 0xd8, 0xee, 0xcc, 0xcc, 0xcc, 0xcc, 0xc, 0x00, 0x00, 0x00, 0x00,
00350     0x00, 0x00, 0x00, 0x00, 0x00, 0x38, 0x6c, 0x0c, 0x18, 0x30, 0x60, 0x7c,
00351     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00352     0x78, 0x78, 0x78, 0x78, 0x78, 0x78, 0x78, 0x00, 0x00, 0x00, 0x00, 0x00,
00353     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00354     0x00, 0x00, 0x00, 0x00};
00355
00356 static void fbterm_save_state(struct term_context *_ctx)
00357 {
00358     struct fbterm_context *ctx = (void *) _ctx;
00359     ctx->saved_state_text_fg = ctx->text_fg;
00360     ctx->saved_state_text_bg = ctx->text_bg;
00361     ctx->saved_state_cursor_x = ctx->cursor_x;
00362     ctx->saved_state_cursor_y = ctx->cursor_y;
00363 }
00364
00365 static void fbterm_restore_state(struct term_context *_ctx)
00366 {
00367     struct fbterm_context *ctx = (void *) _ctx;
00368     ctx->text_fg = ctx->saved_state_text_fg;
00369     ctx->text_bg = ctx->saved_state_text_bg;
00370     ctx->cursor_x = ctx->saved_state_cursor_x;
00371     ctx->cursor_y = ctx->saved_state_cursor_y;
00372 }
00373
00374 static void fbterm_swap_palette(struct term_context *_ctx)
00375 {
00376     struct fbterm_context *ctx = (void *) _ctx;
00377     uint32_t tmp = ctx->text_bg;
00378     ctx->text_bg = ctx->text_fg;
00379     ctx->text_fg = tmp;
00380 }
00381
00382 static void
00383 plot_char(struct term_context *_ctx, struct fbterm_char *c, size_t x, size_t y)
00384 {
00385     struct fbterm_context *ctx = (void *) _ctx;
00386
00387     if (x >= _ctx->cols || y >= _ctx->rows)
00388     {
00389         return;
00390     }
00391
00392     x = ctx->offset_x + x * ctx->glyph_width;
00393     y = ctx->offset_y + y * ctx->glyph_height;
00394
00395     bool *glyph = &ctx->font_bool[c->c * ctx->font_height * ctx->font_width];
00396     // naming: fx,fy for font coordinates, gx,gy for glyph coordinates
00397     for (size_t gy = 0; gy < ctx->glyph_height; gy++)
00398     {
00399         uint8_t fy = gy / ctx->font_scale_y;
00400         volatile uint32_t *fb_line =
00401             ctx->framebuffer + x + (y + gy) * (ctx->pitch / 4);
00402         uint32_t *canvas_line = ctx->canvas + x + (y + gy) * ctx->width;
00403         for (size_t fx = 0; fx < ctx->font_width; fx++)
00404         {
00405             bool draw = glyph[fy * ctx->font_width + fx];
00406             for (size_t i = 0; i < ctx->font_scale_x; i++)
00407             {
00408                 size_t gx = ctx->font_scale_x * fx + i;

```

```

00409         uint32_t bg = c->bg == 0xffffffff ? canvas_line[gx] : c->bg;
00410         uint32_t fg = c->fg == 0xffffffff ? canvas_line[gx] : c->fg;
00411         fb_line[gx] = draw ? fg : bg;
00412     }
00413 }
00414 }
00415 }
00416
00417 static void plot_char_fast(struct term_context *_ctx,
00418     struct fbterm_char *old,
00419     struct fbterm_char *c,
00420     size_t x,
00421     size_t y)
00422 {
00423     struct fbterm_context *ctx = (void *) _ctx;
00424
00425     if (x >= _ctx->cols || y >= _ctx->rows)
00426     {
00427         return;
00428     }
00429
00430     x = ctx->offset_x + x * ctx->glyph_width;
00431     y = ctx->offset_y + y * ctx->glyph_height;
00432
00433     bool *new_glyph =
00434         &ctx->font_bool[c->c * ctx->font_height * ctx->font_width];
00435     bool *old_glyph =
00436         &ctx->font_bool[old->c * ctx->font_height * ctx->font_width];
00437     for (size_t gy = 0; gy < ctx->glyph_height; gy++)
00438     {
00439         uint8_t fy = gy / ctx->font_scale_y;
00440         volatile uint32_t *fb_line =
00441             ctx->framebuffer + x + (y + gy) * (ctx->pitch / 4);
00442         uint32_t *canvas_line = ctx->canvas + x + (y + gy) * ctx->width;
00443         for (size_t fx = 0; fx < ctx->font_width; fx++)
00444         {
00445             bool old_draw = old_glyph[fy * ctx->font_width + fx];
00446             bool new_draw = new_glyph[fy * ctx->font_width + fx];
00447             if (old_draw == new_draw)
00448                 continue;
00449             for (size_t i = 0; i < ctx->font_scale_x; i++)
00450             {
00451                 size_t gx = ctx->font_scale_x * fx + i;
00452                 uint32_t bg = c->bg == 0xffffffff ? canvas_line[gx] : c->bg;
00453                 uint32_t fg = c->fg == 0xffffffff ? canvas_line[gx] : c->fg;
00454                 fb_line[gx] = new_draw ? fg : bg;
00455             }
00456         }
00457     }
00458 }
00459
00460 static inline bool compare_char(struct fbterm_char *a, struct fbterm_char *b)
00461 {
00462     return !(a->c != b->c || a->bg != b->bg || a->fg != b->fg);
00463 }
00464
00465 static void push_to_queue(struct term_context *_ctx,
00466     struct fbterm_char *c,
00467     size_t x,
00468     size_t y)
00469 {
00470     struct fbterm_context *ctx = (void *) _ctx;
00471
00472     if (x >= _ctx->cols || y >= _ctx->rows)
00473     {
00474         return;
00475     }
00476
00477     size_t i = y * _ctx->cols + x;
00478
00479     struct fbterm_queue_item *q = ctx->map[i];
00480
00481     if (q == NULL)
00482     {
00483         if (compare_char(&ctx->grid[i], c))
00484         {
00485             return;
00486         }
00487         q = &ctx->queue[ctx->queue_i++];
00488         q->x = x;
00489         q->y = y;
00490         ctx->map[i] = q;
00491     }
00492
00493     q->c = *c;
00494 }
00495

```

```

00496 static void fbterm_revscroll(struct term_context *_ctx)
00497 {
00498     struct fbterm_context *ctx = (void *) _ctx;
00499
00500     for (size_t i = (_ctx->scroll_bottom_margin - 1) * _ctx->cols - 1; i--)
00501     {
00502         struct fbterm_char *c;
00503         struct fbterm_queue_item *q = ctx->map[i];
00504         if (q != NULL)
00505         {
00506             c = &q->c;
00507         }
00508         else
00509         {
00510             c = &ctx->grid[i];
00511         }
00512         push_to_queue(_ctx,
00513             c,
00514             (i + _ctx->cols) % _ctx->cols,
00515             (i + _ctx->cols) / _ctx->cols);
00516         if (i == _ctx->scroll_top_margin * _ctx->cols)
00517         {
00518             break;
00519         }
00520     }
00521
00522     // Clear the first line of the screen.
00523     struct fbterm_char empty;
00524     empty.c = ' ';
00525     empty.fg = ctx->text_fg;
00526     empty.bg = ctx->text_bg;
00527     for (size_t i = _ctx->scroll_top_margin * _ctx->cols;
00528         i < (_ctx->scroll_top_margin + 1) * _ctx->cols;
00529         i++)
00530     {
00531         push_to_queue(_ctx, &empty, i % _ctx->cols, i / _ctx->cols);
00532     }
00533 }
00534
00535 static void fbterm_scroll(struct term_context *_ctx)
00536 {
00537     struct fbterm_context *ctx = (void *) _ctx;
00538
00539     for (size_t i = (_ctx->scroll_top_margin + 1) * _ctx->cols;
00540         i < _ctx->scroll_bottom_margin * _ctx->cols;
00541         i++)
00542     {
00543         struct fbterm_char *c;
00544         struct fbterm_queue_item *q = ctx->map[i];
00545         if (q != NULL)
00546         {
00547             c = &q->c;
00548         }
00549         else
00550         {
00551             c = &ctx->grid[i];
00552         }
00553         push_to_queue(_ctx,
00554             c,
00555             (i - _ctx->cols) % _ctx->cols,
00556             (i - _ctx->cols) / _ctx->cols);
00557     }
00558
00559     // Clear the last line of the screen.
00560     struct fbterm_char empty;
00561     empty.c = ' ';
00562     empty.fg = ctx->text_fg;
00563     empty.bg = ctx->text_bg;
00564     for (size_t i = (_ctx->scroll_bottom_margin - 1) * _ctx->cols;
00565         i < _ctx->scroll_bottom_margin * _ctx->cols;
00566         i++)
00567     {
00568         push_to_queue(_ctx, &empty, i % _ctx->cols, i / _ctx->cols);
00569     }
00570 }
00571
00572 static void fbterm_clear(struct term_context *_ctx, bool move)
00573 {
00574     struct fbterm_context *ctx = (void *) _ctx;
00575
00576     struct fbterm_char empty;
00577     empty.c = ' ';
00578     empty.fg = ctx->text_fg;
00579     empty.bg = ctx->text_bg;
00580     for (size_t i = 0; i < _ctx->rows * _ctx->cols; i++)
00581     {
00582         push_to_queue(_ctx, &empty, i % _ctx->cols, i / _ctx->cols);

```

```

00583     }
00584
00585     if (move)
00586     {
00587         ctx->cursor_x = 0;
00588         ctx->cursor_y = 0;
00589     }
00590 }
00591
00592 static void fbterm_enable_cursor(struct term_context *_ctx)
00593 {
00594     struct fbterm_context *ctx = (void *) _ctx;
00595
00596     ctx->cursor_status = true;
00597 }
00598
00599 static bool fbterm_disable_cursor(struct term_context *_ctx)
00600 {
00601     struct fbterm_context *ctx = (void *) _ctx;
00602
00603     bool ret          = ctx->cursor_status;
00604     ctx->cursor_status = false;
00605     return ret;
00606 }
00607
00608 static void fbterm_set_cursor_pos(struct term_context *_ctx, size_t x, size_t y)
00609 {
00610     struct fbterm_context *ctx = (void *) _ctx;
00611
00612     if (x >= _ctx->cols)
00613     {
00614         if ((int) x < 0)
00615         {
00616             x = 0;
00617         }
00618         else
00619         {
00620             x = _ctx->cols - 1;
00621         }
00622     }
00623     if (y >= _ctx->rows)
00624     {
00625         if ((int) y < 0)
00626         {
00627             y = 0;
00628         }
00629         else
00630         {
00631             y = _ctx->rows - 1;
00632         }
00633     }
00634     ctx->cursor_x = x;
00635     ctx->cursor_y = y;
00636 }
00637
00638 static void
00639 fbterm_get_cursor_pos(struct term_context *_ctx, size_t *x, size_t *y)
00640 {
00641     struct fbterm_context *ctx = (void *) _ctx;
00642
00643     *x = ctx->cursor_x;
00644     *y = ctx->cursor_y;
00645 }
00646
00647 static void fbterm_move_character(struct term_context *_ctx,
00648                                 size_t          new_x,
00649                                 size_t          new_y,
00650                                 size_t          old_x,
00651                                 size_t          old_y)
00652 {
00653     struct fbterm_context *ctx = (void *) _ctx;
00654
00655     if (old_x >= _ctx->cols || old_y >= _ctx->rows || new_x >= _ctx->cols ||
00656         new_y >= _ctx->rows)
00657     {
00658         return;
00659     }
00660
00661     size_t i = old_x + old_y * _ctx->cols;
00662
00663     struct fbterm_char      *c;
00664     struct fbterm_queue_item *q = ctx->map[i];
00665     if (q != NULL)
00666     {
00667         c = &q->c;
00668     }
00669     else

```



```

00670     {
00671         c = &ctx->grid[i];
00672     }
00673
00674     push_to_queue(_ctx, c, new_x, new_y);
00675 }
00676
00677 static void fbterm_set_text_fg(struct term_context *_ctx, size_t fg)
00678 {
00679     struct fbterm_context *ctx = (void *) _ctx;
00680
00681     ctx->text_fg = ctx->ansi_colours[fg];
00682 }
00683
00684 static void fbterm_set_text_bg(struct term_context *_ctx, size_t bg)
00685 {
00686     struct fbterm_context *ctx = (void *) _ctx;
00687
00688     ctx->text_bg = ctx->ansi_colours[bg];
00689 }
00690
00691 static void fbterm_set_text_fg_bright(struct term_context *_ctx, size_t fg)
00692 {
00693     struct fbterm_context *ctx = (void *) _ctx;
00694
00695     ctx->text_fg = ctx->ansi_bright_colours[fg];
00696 }
00697
00698 static void fbterm_set_text_bg_bright(struct term_context *_ctx, size_t bg)
00699 {
00700     struct fbterm_context *ctx = (void *) _ctx;
00701
00702     ctx->text_bg = ctx->ansi_bright_colours[bg];
00703 }
00704
00705 static void fbterm_set_text_fg_rgb(struct term_context *_ctx, uint32_t fg)
00706 {
00707     struct fbterm_context *ctx = (void *) _ctx;
00708
00709     ctx->text_fg = fg;
00710 }
00711
00712 static void fbterm_set_text_bg_rgb(struct term_context *_ctx, uint32_t bg)
00713 {
00714     struct fbterm_context *ctx = (void *) _ctx;
00715
00716     ctx->text_bg = bg;
00717 }
00718
00719 static void fbterm_set_text_fg_default(struct term_context *_ctx)
00720 {
00721     struct fbterm_context *ctx = (void *) _ctx;
00722
00723     ctx->text_fg = ctx->default_fg;
00724 }
00725
00726 static void fbterm_set_text_bg_default(struct term_context *_ctx)
00727 {
00728     struct fbterm_context *ctx = (void *) _ctx;
00729
00730     ctx->text_bg = 0xffffffff;
00731 }
00732
00733 static void draw_cursor(struct term_context *_ctx)
00734 {
00735     struct fbterm_context *ctx = (void *) _ctx;
00736
00737     size_t i = ctx->cursor_x + ctx->cursor_y * _ctx->cols;
00738     struct fbterm_char c;
00739     struct fbterm_queue_item *q = ctx->map[i];
00740     if (q != NULL)
00741     {
00742         c = q->c;
00743     }
00744     else
00745     {
00746         c = ctx->grid[i];
00747     }
00748     uint32_t tmp = c.fg;
00749     c.fg = c.bg;
00750     c.bg = tmp;
00751     plot_char(_ctx, &c, ctx->cursor_x, ctx->cursor_y);
00752     if (q != NULL)
00753     {
00754         ctx->grid[i] = q->c;
00755         ctx->map[i] = NULL;
00756     }

```

```

00757 }
00758
00759 static void fbterm_double_buffer_flush(struct term_context *_ctx)
00760 {
00761     struct fbterm_context *ctx = (void *) _ctx;
00762
00763     if (ctx->cursor_status)
00764     {
00765         draw_cursor(_ctx);
00766     }
00767
00768     for (size_t i = 0; i < ctx->queue_i; i++)
00769     {
00770         struct fbterm_queue_item *q = &ctx->queue[i];
00771         size_t offset = q->y * _ctx->cols + q->x;
00772         if (ctx->map[offset] == NULL)
00773         {
00774             continue;
00775         }
00776         struct fbterm_char *old = &ctx->grid[offset];
00777         if (q->c.bg == old->bg && q->c.fg == old->fg)
00778         {
00779             plot_char_fast(_ctx, old, &q->c, q->x, q->y);
00780         }
00781         else
00782         {
00783             plot_char(_ctx, &q->c, q->x, q->y);
00784         }
00785         ctx->grid[offset] = q->c;
00786         ctx->map[offset] = NULL;
00787     }
00788
00789     if ((ctx->old_cursor_x != ctx->cursor_x ||
00790         ctx->old_cursor_y != ctx->cursor_y) ||
00791         ctx->cursor_status == false)
00792     {
00793         plot_char(
00794             _ctx,
00795             &ctx->grid[ctx->old_cursor_x + ctx->old_cursor_y * _ctx->cols],
00796             ctx->old_cursor_x,
00797             ctx->old_cursor_y);
00798     }
00799
00800     ctx->old_cursor_x = ctx->cursor_x;
00801     ctx->old_cursor_y = ctx->cursor_y;
00802
00803     ctx->queue_i = 0;
00804 }
00805
00806 static void fbterm_raw_putchar(struct term_context *_ctx, uint8_t c)
00807 {
00808     struct fbterm_context *ctx = (void *) _ctx;
00809
00810     struct fbterm_char ch;
00811     ch.c = c;
00812     ch.fg = ctx->text_fg;
00813     ch.bg = ctx->text_bg;
00814     push_to_queue(_ctx, &ch, ctx->cursor_x++, ctx->cursor_y);
00815     if (ctx->cursor_x == _ctx->cols &&
00816         (ctx->cursor_y < _ctx->scroll_bottom_margin - 1 ||
00817          _ctx->scroll_enabled))
00818     {
00819         ctx->cursor_x = 0;
00820         ctx->cursor_y++;
00821     }
00822     if (ctx->cursor_y == _ctx->scroll_bottom_margin)
00823     {
00824         ctx->cursor_y--;
00825         fbterm_scroll(_ctx);
00826     }
00827 }
00828
00829 static void fbterm_full_refresh(struct term_context *_ctx)
00830 {
00831     struct fbterm_context *ctx = (void *) _ctx;
00832
00833     for (size_t y = 0; y < ctx->height; y++)
00834     {
00835         for (size_t x = 0; x < ctx->width; x++)
00836         {
00837             ctx->framebuffer[y * (ctx->pitch / sizeof(uint32_t)) + x] =
00838                 ctx->canvas[y * ctx->width + x];
00839         }
00840     }
00841
00842     for (size_t i = 0; i < (size_t) _ctx->rows * _ctx->cols; i++)
00843     {

```

```

00844     size_t x = i % _ctx->cols;
00845     size_t y = i / _ctx->cols;
00846
00847     plot_char(_ctx, &ctx->grid[i], x, y);
00848 }
00849
00850 if (ctx->cursor_status)
00851 {
00852     draw_cursor(_ctx);
00853 }
00854 }
00855
00856 static void fbterm_deinit(struct term_context *_ctx,
00857                          void (*_free)(void *, size_t))
00858 {
00859     struct fbterm_context *ctx = (void *) _ctx;
00860
00861     _free(ctx->font_bits, ctx->font_bits_size);
00862     _free(ctx->font_bool, ctx->font_bool_size);
00863     _free(ctx->grid, ctx->grid_size);
00864     _free(ctx->queue, ctx->queue_size);
00865     _free(ctx->map, ctx->map_size);
00866     _free(ctx->canvas, ctx->canvas_size);
00867     _free(ctx, sizeof(struct fbterm_context));
00868 }
00869
00870 struct term_context *fbterm_init(void *(*_malloc)(size_t),
00871                                  uint32_t *framebuffer,
00872                                  size_t width,
00873                                  size_t height,
00874                                  size_t pitch,
00875                                  uint32_t *canvas,
00876                                  uint32_t *ansi_colours,
00877                                  uint32_t *ansi_bright_colours,
00878                                  uint32_t *default_bg,
00879                                  uint32_t *default_fg,
00880                                  void *font,
00881                                  size_t font_width,
00882                                  size_t font_height,
00883                                  size_t font_spacing,
00884                                  size_t font_scale_x,
00885                                  size_t font_scale_y,
00886                                  size_t margin)
00887 {
00888     struct fbterm_context *ctx = _malloc(sizeof(struct fbterm_context));
00889
00890     struct term_context *_ctx = (void *) ctx;
00891
00892     memset(ctx, 0, sizeof(struct fbterm_context));
00893
00894     ctx->cursor_status = true;
00895
00896     if (ansi_colours != NULL)
00897     {
00898         memcpy(ctx->ansi_colours, ansi_colours, sizeof(ctx->ansi_colours));
00899     }
00900     else
00901     {
00902         ctx->ansi_colours[0] = 0x00000000; // black
00903         ctx->ansi_colours[1] = 0x00aa0000; // red
00904         ctx->ansi_colours[2] = 0x0000aa00; // green
00905         ctx->ansi_colours[3] = 0x00aa5500; // brown
00906         ctx->ansi_colours[4] = 0x000000aa; // blue
00907         ctx->ansi_colours[5] = 0x00aa00aa; // magenta
00908         ctx->ansi_colours[6] = 0x0000aaaa; // cyan
00909         ctx->ansi_colours[7] = 0x00aaaaaa; // grey
00910     }
00911
00912     if (ansi_bright_colours != NULL)
00913     {
00914         memcpy(ctx->ansi_bright_colours,
00915              ansi_bright_colours,
00916              sizeof(ctx->ansi_bright_colours));
00917     }
00918     else
00919     {
00920         ctx->ansi_bright_colours[0] = 0x00555555; // black
00921         ctx->ansi_bright_colours[1] = 0x00ff5555; // red
00922         ctx->ansi_bright_colours[2] = 0x0055ff55; // green
00923         ctx->ansi_bright_colours[3] = 0x00ffff55; // brown
00924         ctx->ansi_bright_colours[4] = 0x005555ff; // blue
00925         ctx->ansi_bright_colours[5] = 0x00ff55ff; // magenta
00926         ctx->ansi_bright_colours[6] = 0x0055ffff; // cyan
00927         ctx->ansi_bright_colours[7] = 0x00ffffff; // grey
00928     }
00929
00930     if (default_bg != NULL)

```

```

00931     {
00932         ctx->default_bg = *default_bg;
00933     }
00934     else
00935     {
00936         ctx->default_bg = 0x00000000; // background (black)
00937     }
00938
00939     if (default_fg != NULL)
00940     {
00941         ctx->default_fg = *default_fg;
00942     }
00943     else
00944     {
00945         ctx->default_fg = 0x00aaaaaa; // foreground (grey)
00946     }
00947
00948     ctx->text_fg = ctx->default_fg;
00949     ctx->text_bg = 0xffffffff;
00950
00951     ctx->framebuffer = (void *) framebuffer;
00952     ctx->width       = width;
00953     ctx->height      = height;
00954     ctx->pitch       = pitch;
00955
00956     #define FONT_BYTES ((font_width * font_height * FBTERM_FONT_GLYPHS) / 8)
00957
00958     if (font != NULL)
00959     {
00960         ctx->font_width = font_width;
00961         ctx->font_height = font_height;
00962         ctx->font_bits = _malloc(FONT_BYTES);
00963         memcpy(ctx->font_bits, font, FONT_BYTES);
00964     }
00965     else
00966     {
00967         ctx->font_width = font_width = 8;
00968         ctx->font_height = font_height = 16;
00969         font_spacing    = 1;
00970         ctx->font_bits = _malloc(FONT_BYTES);
00971         memcpy(ctx->font_bits, builtin_font, FONT_BYTES);
00972     }
00973
00974     ctx->font_bits_size = FONT_BYTES;
00975
00976     #undef FONT_BYTES
00977
00978     ctx->font_width += font_spacing;
00979
00980     ctx->font_bool_size =
00981         FBTERM_FONT_GLYPHS * font_height * ctx->font_width * sizeof(bool);
00982     ctx->font_bool = _malloc(ctx->font_bool_size);
00983
00984     for (size_t i = 0; i < FBTERM_FONT_GLYPHS; i++)
00985     {
00986         uint8_t *glyph = &ctx->font_bits[i * font_height];
00987
00988         for (size_t y = 0; y < font_height; y++)
00989         {
00990             // NOTE: the characters in VGA fonts are always one byte wide.
00991             // 9 dot wide fonts have 8 dots and one empty column, except
00992             // characters 0xC0-0xDF replicate column 9.
00993             for (size_t x = 0; x < 8; x++)
00994             {
00995                 size_t offset =
00996                     i * font_height * ctx->font_width + y * ctx->font_width + x;
00997
00998                 if ((glyph[y] & (0x80 » x)))
00999                 {
01000                     ctx->font_bool[offset] = true;
01001                 }
01002                 else
01003                 {
01004                     ctx->font_bool[offset] = false;
01005                 }
01006             }
01007             // fill columns above 8 like VGA Line Graphics Mode does
01008             for (size_t x = 8; x < ctx->font_width; x++)
01009             {
01010                 size_t offset =
01011                     i * font_height * ctx->font_width + y * ctx->font_width + x;
01012
01013                 if (i >= 0xc0 && i <= 0xdf)
01014                 {
01015                     ctx->font_bool[offset] = (glyph[y] & 1);
01016                 }
01017                 else

```

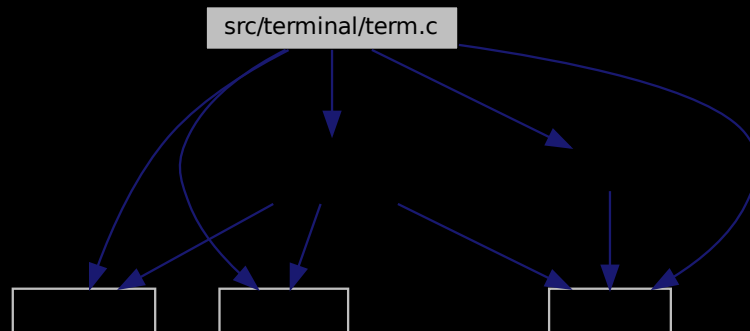
```

01018         {
01019             ctx->font_bool[offset] = false;
01020         }
01021     }
01022 }
01023
01024 ctx->font_scale_x = font_scale_x;
01025 ctx->font_scale_y = font_scale_y;
01026
01027 ctx->glyph_width = ctx->font_width * font_scale_x;
01028 ctx->glyph_height = font_height * font_scale_y;
01029
01030 _ctx->cols = (ctx->width - margin * 2) / ctx->glyph_width;
01031 _ctx->rows = (ctx->height - margin * 2) / ctx->glyph_height;
01032
01033 ctx->offset_x = margin + ((ctx->width - margin * 2) % ctx->glyph_width) / 2;
01034 ctx->offset_y =
01035     margin + ((ctx->height - margin * 2) % ctx->glyph_height) / 2;
01036
01037 ctx->grid_size = _ctx->rows * _ctx->cols * sizeof(struct fbterm_char);
01038 ctx->grid = _malloc(ctx->grid_size);
01039 for (size_t i = 0; i < _ctx->rows * _ctx->cols; i++)
01040 {
01041     ctx->grid[i].c = ' ';
01042     ctx->grid[i].fg = ctx->text_fg;
01043     ctx->grid[i].bg = ctx->text_bg;
01044 }
01045
01046 ctx->queue_size =
01047     _ctx->rows * _ctx->cols * sizeof(struct fbterm_queue_item);
01048 ctx->queue = _malloc(ctx->queue_size);
01049 ctx->queue_i = 0;
01050
01051 ctx->map_size =
01052     _ctx->rows * _ctx->cols * sizeof(struct fbterm_queue_item *);
01053 ctx->map = _malloc(ctx->map_size);
01054
01055 ctx->canvas_size = ctx->width * ctx->height * sizeof(uint32_t);
01056 ctx->canvas = _malloc(ctx->canvas_size);
01057 if (canvas != NULL)
01058 {
01059     memcpy(ctx->canvas, canvas, ctx->canvas_size);
01060 }
01061 else
01062 {
01063     for (size_t i = 0; i < ctx->width * ctx->height; i++)
01064     {
01065         ctx->canvas[i] = ctx->default_bg;
01066     }
01067 }
01068
01069 _ctx->raw_putchar = fbterm_raw_putchar;
01070 _ctx->clear = fbterm_clear;
01071 _ctx->enable_cursor = fbterm_enable_cursor;
01072 _ctx->disable_cursor = fbterm_disable_cursor;
01073 _ctx->set_cursor_pos = fbterm_set_cursor_pos;
01074 _ctx->get_cursor_pos = fbterm_get_cursor_pos;
01075 _ctx->set_text_fg = fbterm_set_text_fg;
01076 _ctx->set_text_bg = fbterm_set_text_bg;
01077 _ctx->set_text_fg_bright = fbterm_set_text_fg_bright;
01078 _ctx->set_text_bg_bright = fbterm_set_text_bg_bright;
01079 _ctx->set_text_fg_rgb = fbterm_set_text_fg_rgb;
01080 _ctx->set_text_bg_rgb = fbterm_set_text_bg_rgb;
01081 _ctx->set_text_fg_default = fbterm_set_text_fg_default;
01082 _ctx->set_text_bg_default = fbterm_set_text_bg_default;
01083 _ctx->move_character = fbterm_move_character;
01084 _ctx->scroll = fbterm_scroll;
01085 _ctx->revscroll = fbterm_revscroll;
01086 _ctx->swap_palette = fbterm_swap_palette;
01087 _ctx->save_state = fbterm_save_state;
01088 _ctx->restore_state = fbterm_restore_state;
01089 _ctx->double_buffer_flush = fbterm_double_buffer_flush;
01090 _ctx->full_refresh = fbterm_full_refresh;
01091 _ctx->deinit = fbterm_deinit;
01092
01093 term_context_reinit(_ctx);
01094
01095 return _ctx;
01096 }
01097

```

7.181 src/terminal/term.c File Reference

```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include "../include/serial.h"
#include "../include/terminal/term.h"
Include dependency graph for term.c:
```



Macros

- `#define CHARSET_DEFAULT 0`
- `#define CHARSET_DEC_SPECIAL 1`

Functions

- `void term_context_reinit (struct term_context *ctx)`
Reset/reinitialize parser-visible state in an existing context.
- `static void term_putchar (struct term_context *ctx, uint8_t c)`
- `void term_write (struct term_context *ctx, const char *buf, size_t count)`
Feed a byte buffer through the terminal parser and backend.
- `static void sgr (struct term_context *ctx)`
- `static void dec_private_parse (struct term_context *ctx, uint8_t c)`
- `static void linux_private_parse (struct term_context *ctx)`
- `static void mode_toggle (struct term_context *ctx, uint8_t c)`
- `static void control_sequence_parse (struct term_context *ctx, uint8_t c)`
- `static void restore_state (struct term_context *ctx)`
- `static void save_state (struct term_context *ctx)`
- `static void escape_parse (struct term_context *ctx, uint8_t c)`
- `static uint8_t dec_special_to_cp437 (uint8_t c)`

Variables

- `static const uint32_t col256 []`

7.181.1 Macro Definition Documentation

7.181.1.1 CHARSET_DEC_SPECIAL

```
#define CHARSET_DEC_SPECIAL 1
```

Definition at line 49 of file `term.c`.

7.181.1.2 CHARSET_DEFAULT

```
#define CHARSET_DEFAULT 0
```

Definition at line 48 of file `term.c`.

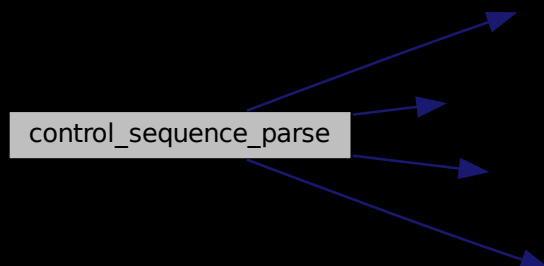
7.181.2 Function Documentation

7.181.2.1 control_sequence_parse()

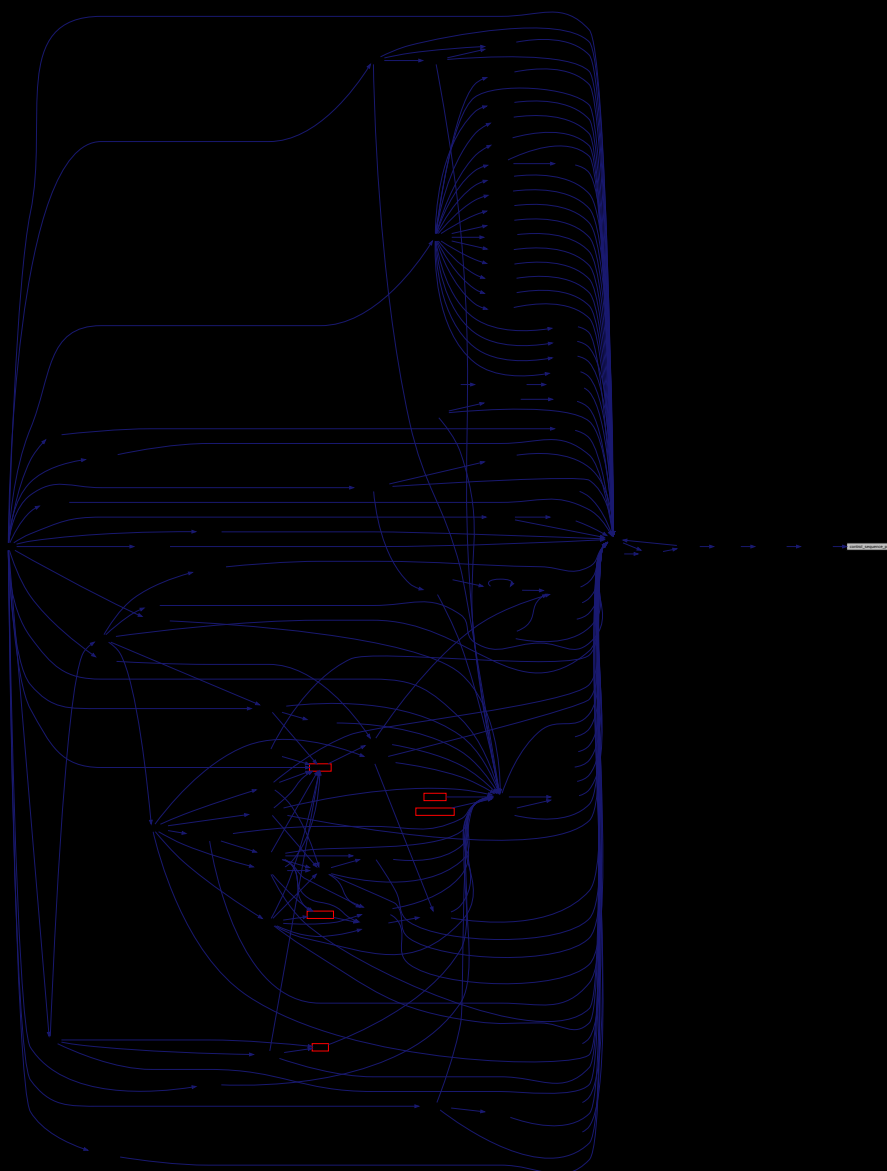
```
static void control_sequence_parse (  
    struct term_context * ctx,  
    uint8_t c ) [static]
```

Definition at line 454 of file `term.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

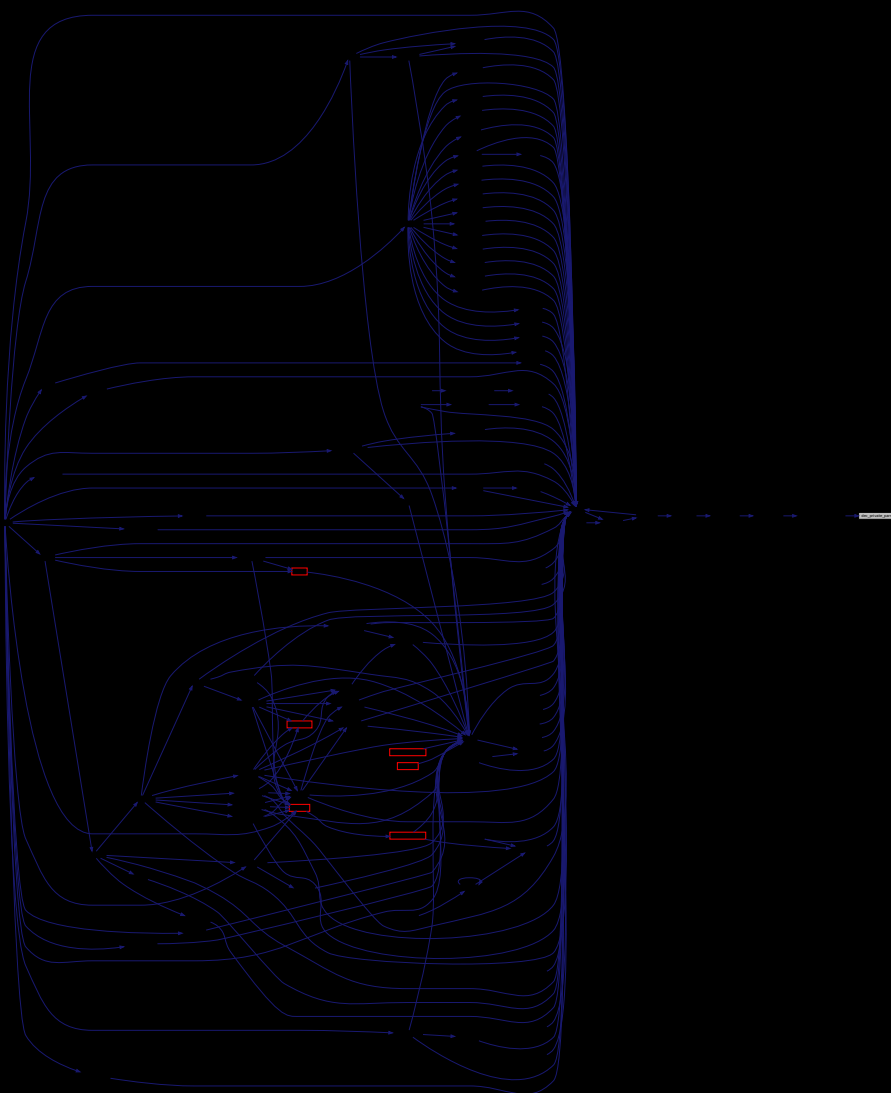


7.181.2.2 dec_private_parse()

```
static void dec_private_parse (  
    struct term_context * ctx,  
    uint8_t c ) [static]
```

Definition at line 350 of file [term.c](#).

Here is the caller graph for this function:

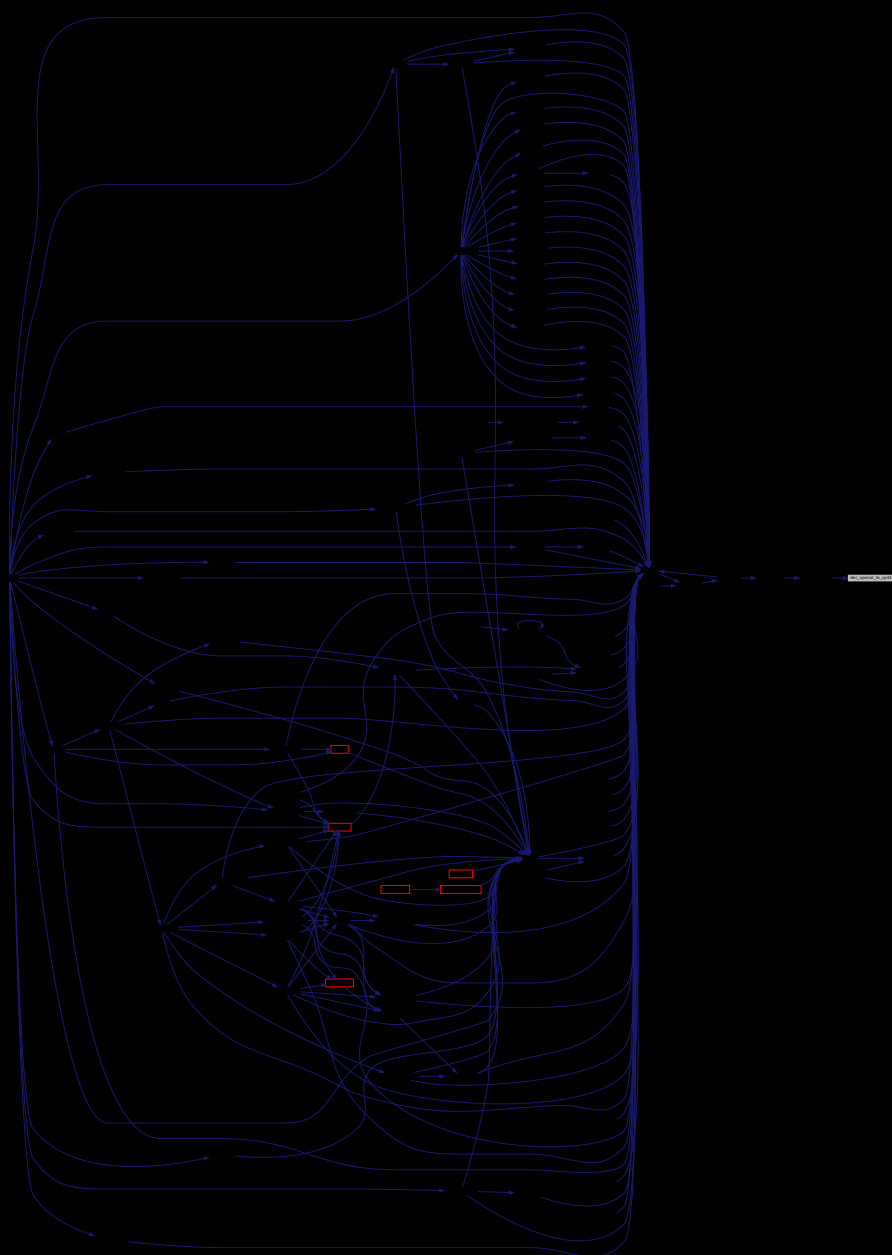


7.181.2.3 `dec_special_to_cp437()`

```
static uint8_t dec_special_to_cp437 (  
    uint8_t c ) [static]
```

Definition at line 894 of file `term.c`.

Here is the caller graph for this function:

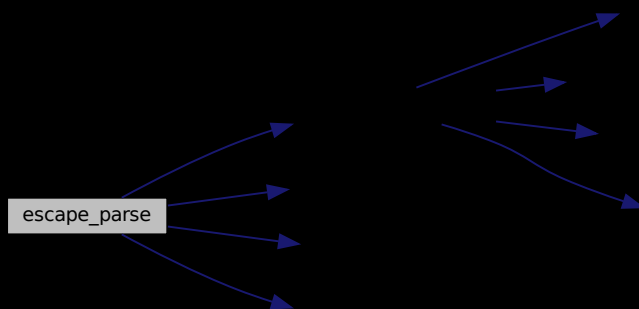


7.181.2.4 `escape_parse()`

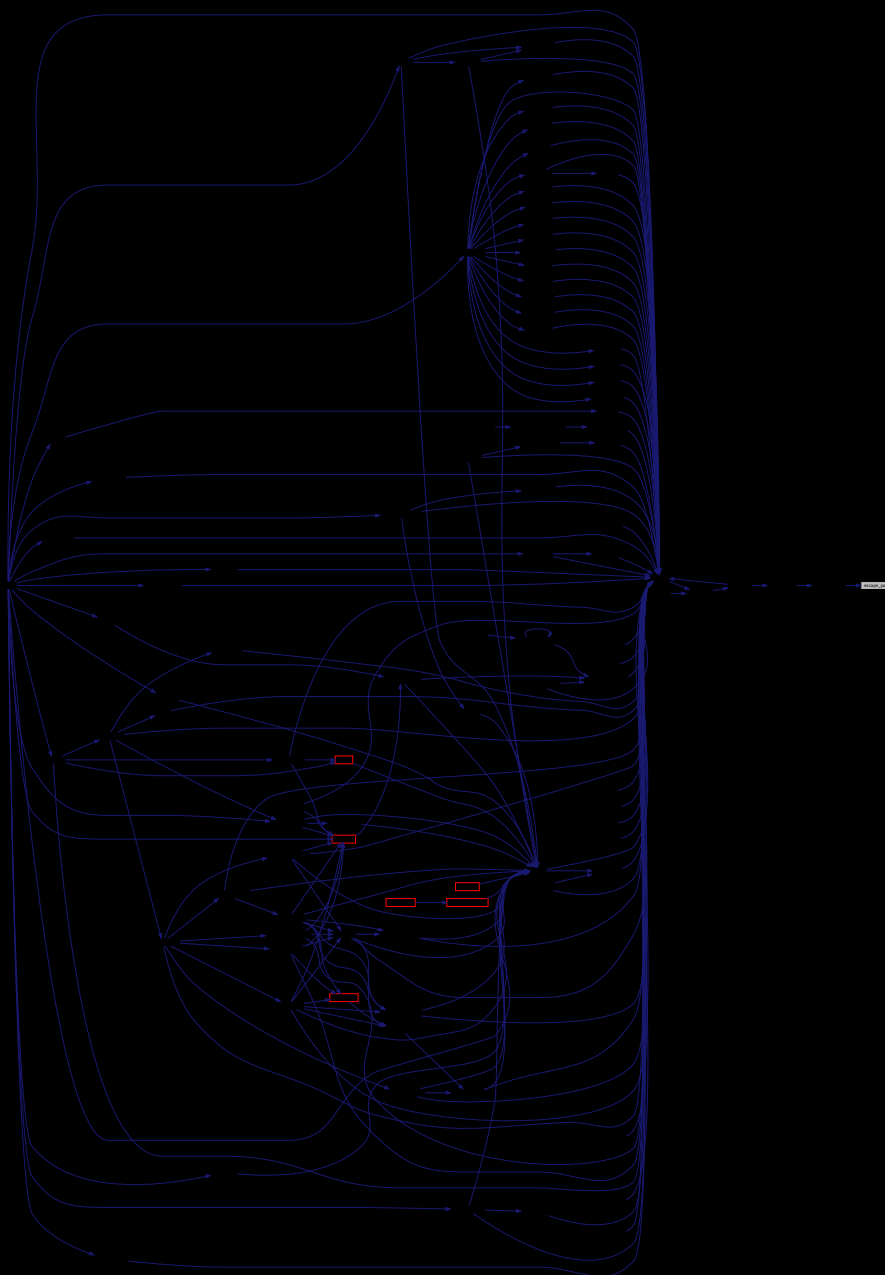
```
static void escape_parse (  
    struct term_context * ctx,  
    uint8_t c ) [static]
```

Definition at line 800 of file `term.c`.

Here is the call graph for this function:



Here is the caller graph for this function:

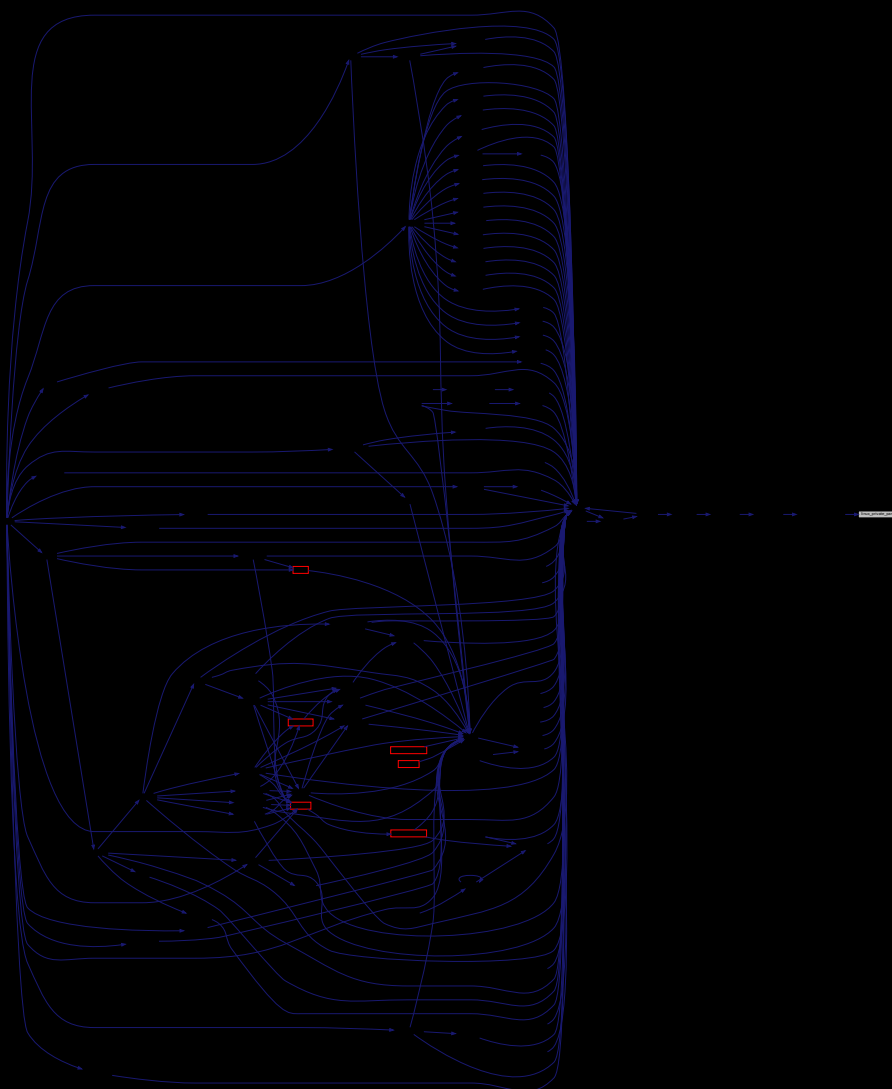


7.181.2.5 linux_private_parse()

```
static void linux_private_parse (  
    struct term_context * ctx ) [static]
```

Definition at line 399 of file `term.c`.

Here is the caller graph for this function:

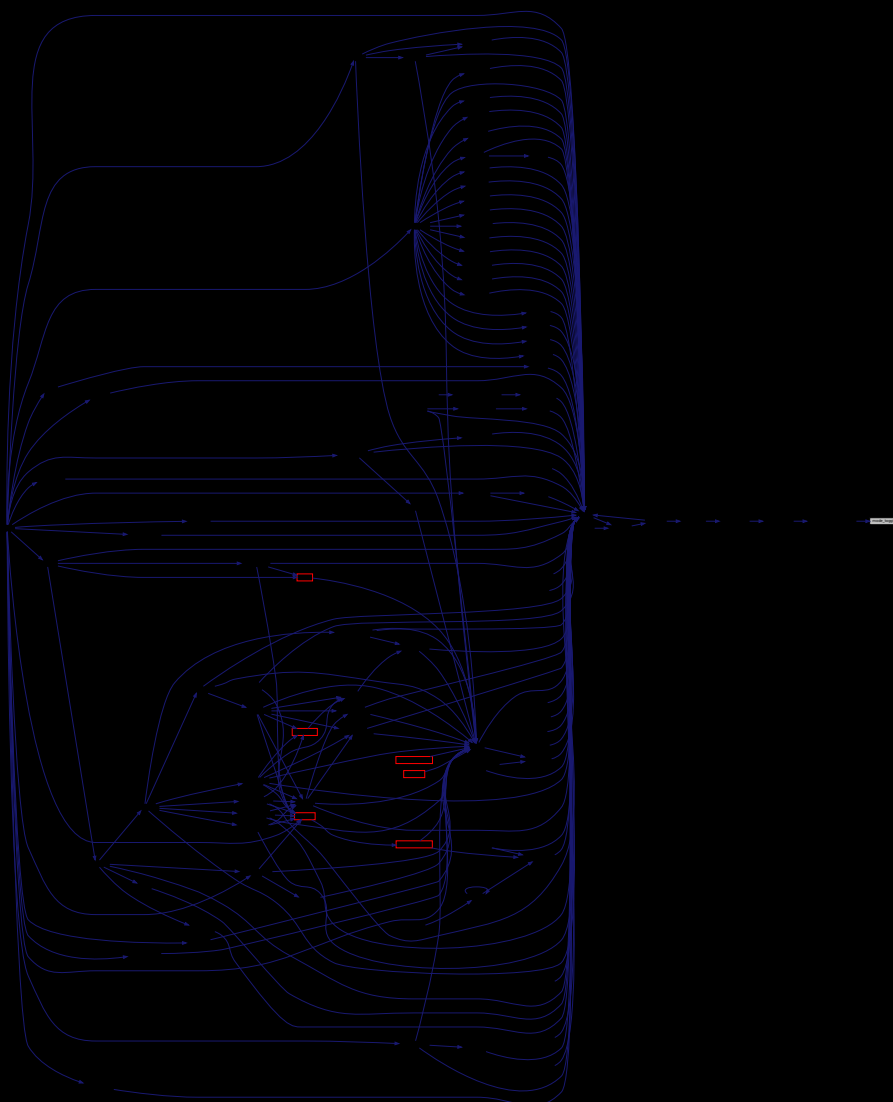


7.181.2.6 mode_toggle()

```
static void mode_toggle (  
    struct term_context * ctx,  
    uint8_t c ) [static]
```

Definition at line 416 of file `term.c`.

Here is the caller graph for this function:

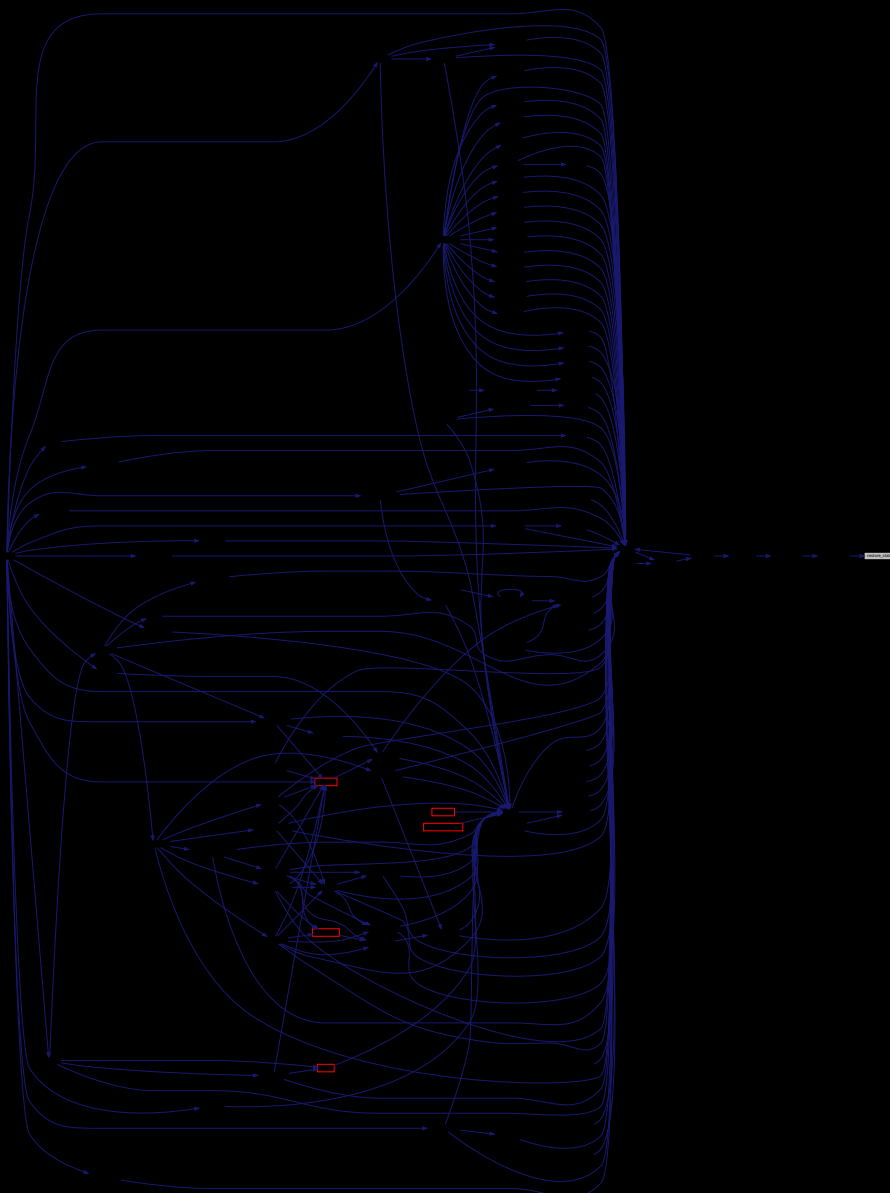


7.181.2.7 restore_state()

```
static void restore_state (  
    struct term_context * ctx ) [static]
```

Definition at line 780 of file `term.c`.

Here is the caller graph for this function:

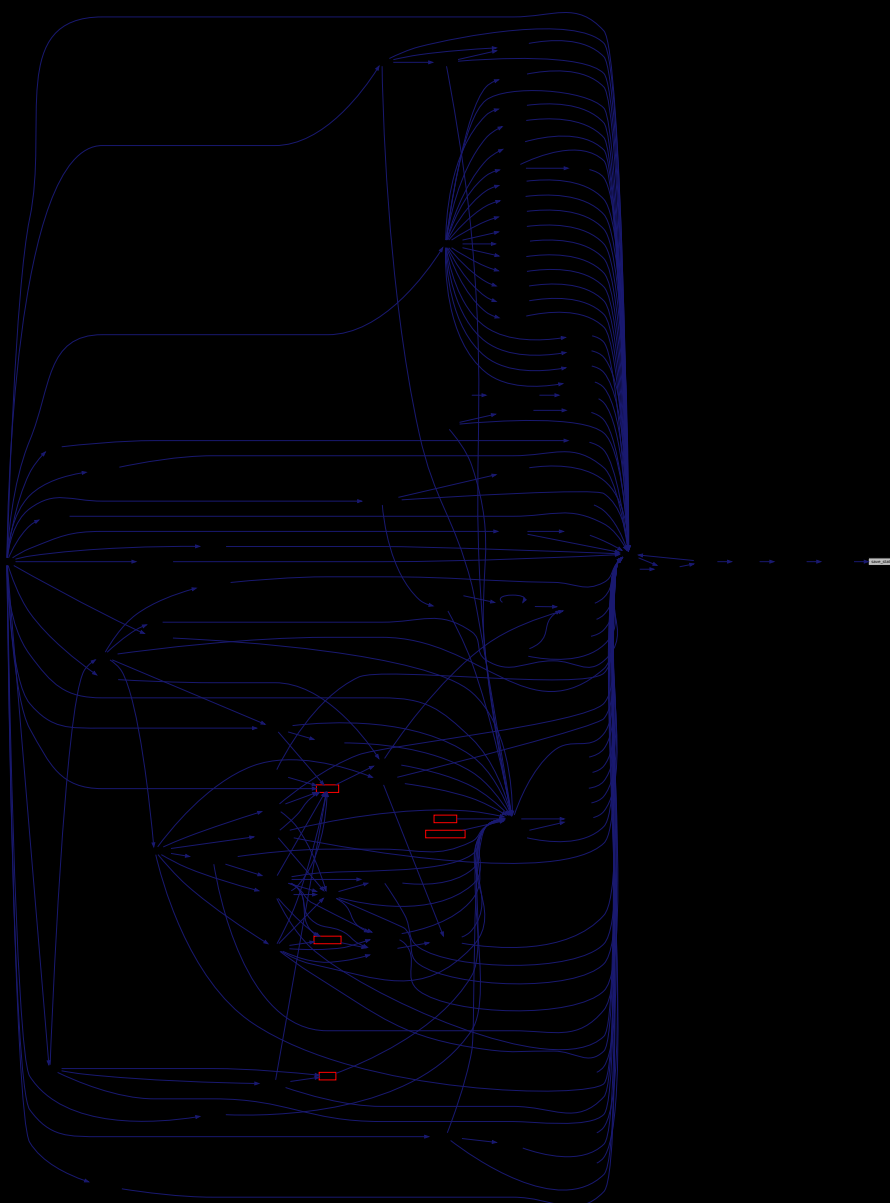


7.181.2.8 `save_state()`

```
static void save_state (  
    struct term_context * ctx ) [static]
```

Definition at line 790 of file `term.c`.

Here is the caller graph for this function:

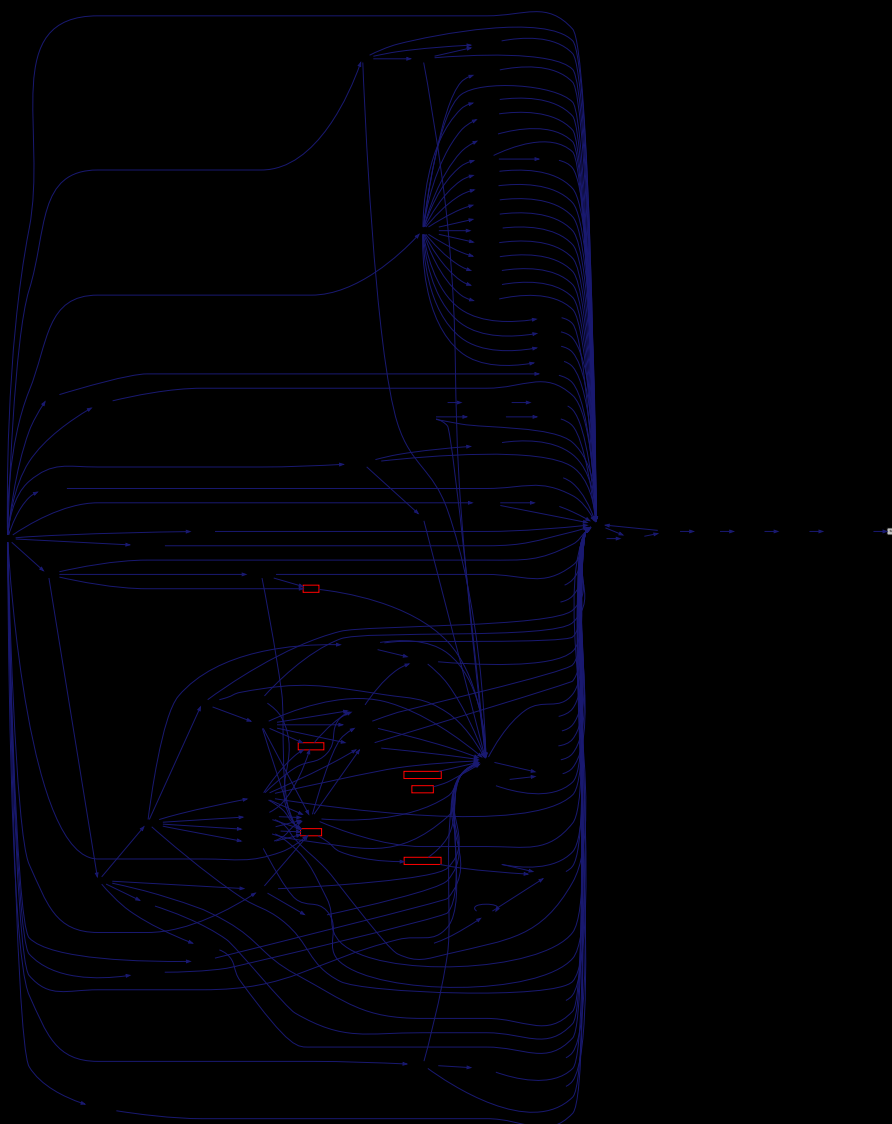


7.181.2.9 sgr()

```
static void sgr (  
    struct term_context * ctx ) [static]
```

Definition at line 93 of file `term.c`.

Here is the caller graph for this function:



7.181.2.10 `term_context_reinit()`

```
void term_context_reinit (  
    struct term_context * ctx )
```

Reset/reinitialize parser-visible state in an existing context.

Parameters

<code>ctx</code>	Terminal context to reinitialize.
------------------	-----------------------------------

Definition at line 51 of file `term.c`.

Here is the caller graph for this function:

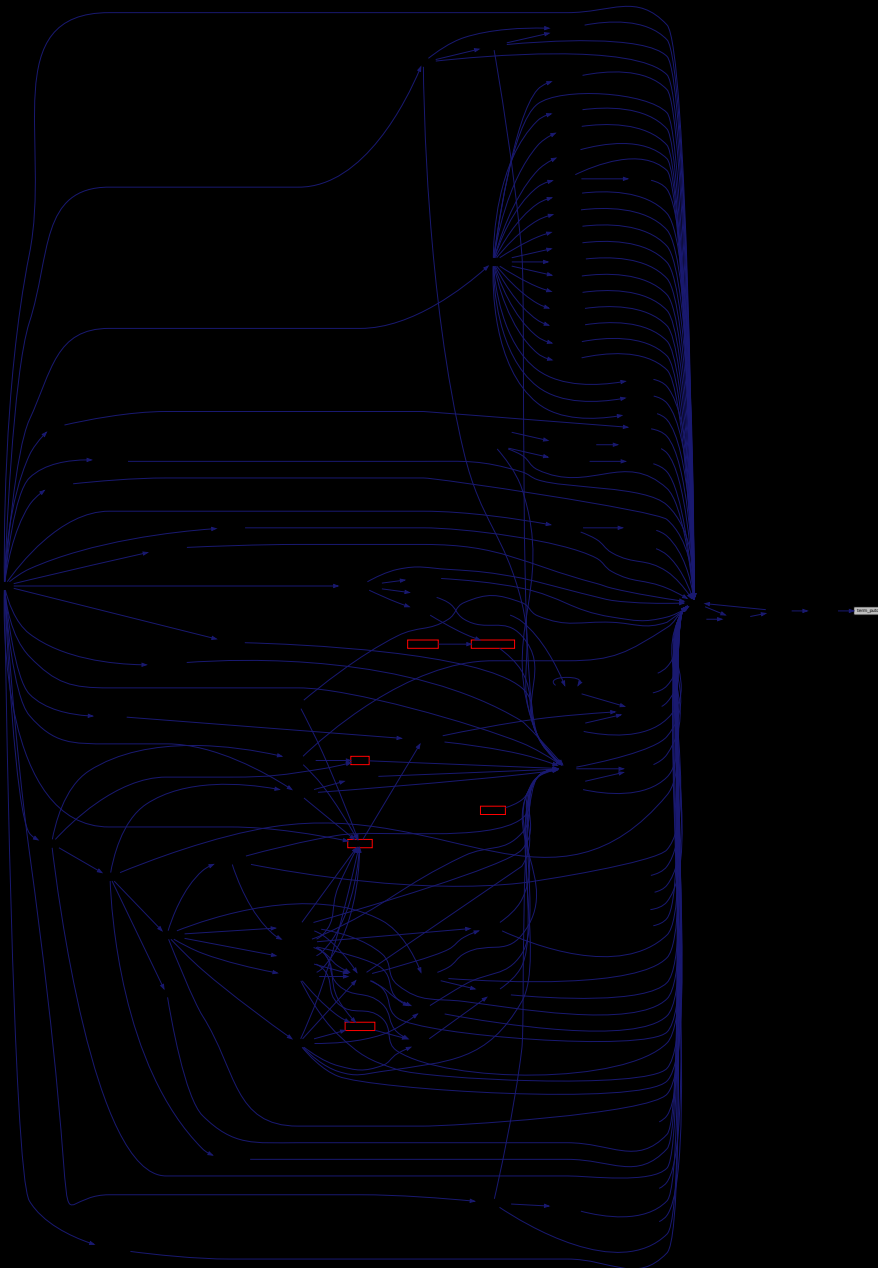


7.181.2.11 `term_putchar()`

```
static void term_putchar (  
    struct term_context * ctx,  
    uint8_t c ) [static]
```

Definition at line 959 of file `term.c`.

Here is the caller graph for this function:

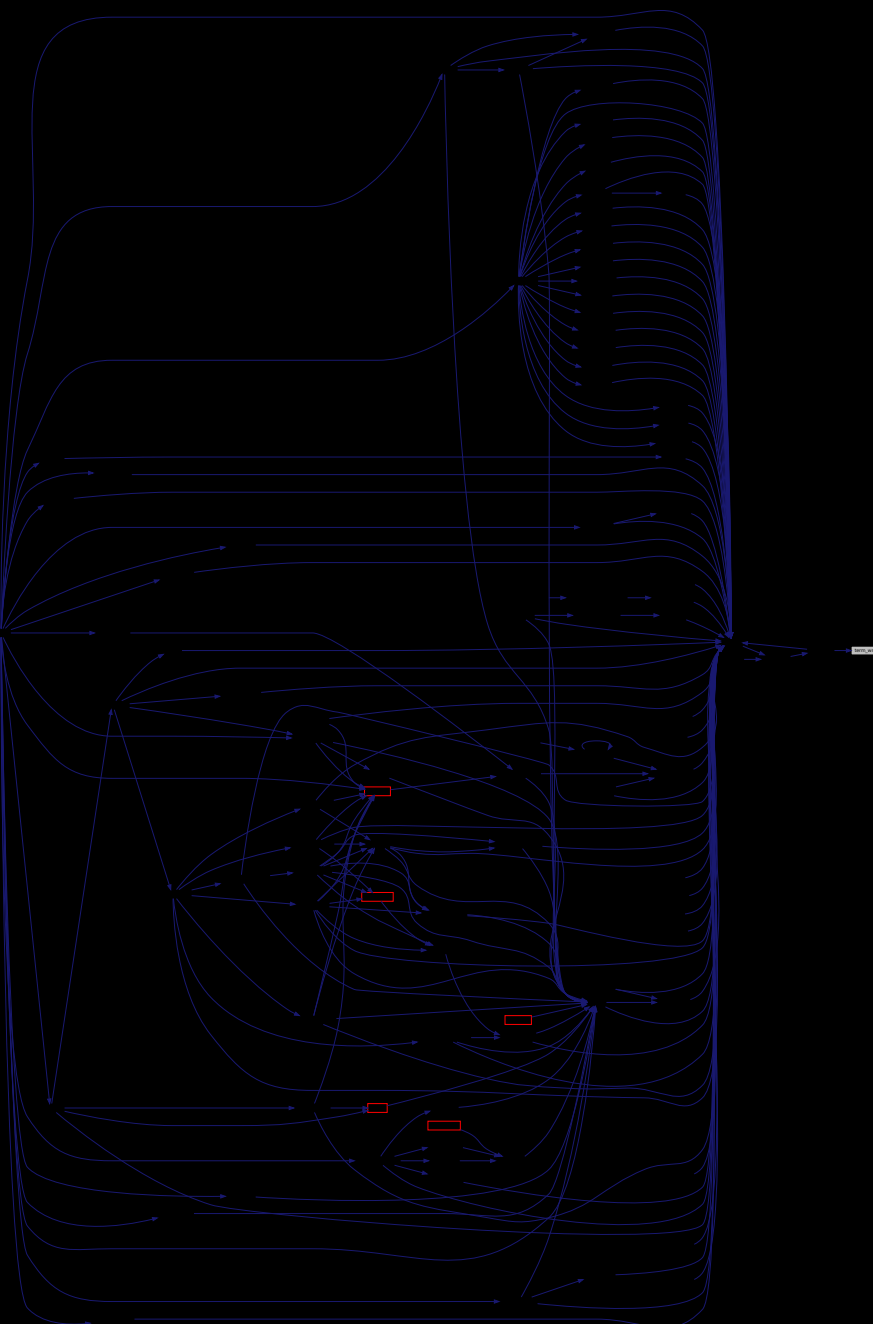


7.181.2.12 term_write()

```
void term_write (
    struct term_context * ctx,
    const char * buf,
    size_t count )
```

Feed a byte buffer through the terminal parser and backend.

Here is the caller graph for this function:



7.181.3 Variable Documentation

7.181.3.1 col256

```
const uint32_t col256[] [static]
```

Definition at line 8 of file [term.c](#).

7.182 term.c

Go to the documentation of this file.

```

00001 #include <stdbool.h>
00002 #include <stddef.h>
00003 #include <stdint.h>
00004
00005 #include "../include/serial.h"
00006 #include "../include/terminal/term.h"
00007
00008 static const uint32_t col256[] = {
00009     0x000000, 0x00005f, 0x000087, 0x0000af, 0x0000d7, 0x0000ff, 0x005f00,
00010     0x005ff5f, 0x005ff87, 0x005faf, 0x005fd7, 0x005fff, 0x008700, 0x00875f,
00011     0x008787, 0x0087af, 0x0087d7, 0x0087ff, 0x00af00, 0x00af5f, 0x00af87,
00012     0x00afaf, 0x00afd7, 0x00afff, 0x00d700, 0x00d75f, 0x00d787, 0x00d7af,
00013     0x00d7d7, 0x00d7ff, 0x00ff00, 0x00ff5f, 0x00ff87, 0x00ffaf, 0x00ffd7,
00014     0x00ffff, 0x5f0000, 0x5f005f, 0x5f0087, 0x5f00af, 0x5f00d7, 0x5f00ff,
00015     0x5f5f00, 0x5f5f5f, 0x5f5f87, 0x5f5faf, 0x5f5fd7, 0x5f5fff, 0x5f8700,
00016     0x5f875f, 0x5f8787, 0x5f87af, 0x5f87d7, 0x5f87ff, 0x5faf00, 0x5faf5f,
00017     0x5faf87, 0x5fafaf, 0x5fafd7, 0x5fafff, 0x5fd700, 0x5fd75f, 0x5fd787,
00018     0x5fd7af, 0x5fd7d7, 0x5fd7ff, 0x5fff00, 0x5fff5f, 0x5fff87, 0x5fffaf,
00019     0x5fffd7, 0x5fffff, 0x870000, 0x87005f, 0x870087, 0x8700af, 0x8700d7,
00020     0x8700ff, 0x875f00, 0x875f5f, 0x875f87, 0x875faf, 0x875fd7, 0x875fff,
00021     0x878700, 0x87875f, 0x878787, 0x8787af, 0x8787d7, 0x8787ff, 0x87af00,
00022     0x87af5f, 0x87af87, 0x87afaf, 0x87afd7, 0x87afff, 0x87d700, 0x87d75f,
00023     0x87d787, 0x87d7af, 0x87d7d7, 0x87d7ff, 0x87ff00, 0x87ff5f, 0x87ff87,
00024     0x87ffaf, 0x87ffd7, 0x87ffff, 0xaf0000, 0xaf005f, 0xaf0087, 0xaf00af,
00025     0xaf00d7, 0xaf00ff, 0xaf5f00, 0xaf5f5f, 0xaf5f87, 0xaf5faf, 0xaf5fd7,
00026     0xaf5fff, 0xaf8700, 0xaf875f, 0xaf8787, 0xaf87af, 0xaf87d7, 0xaf87ff,
00027     0xafaf00, 0xafaf5f, 0xafaf87, 0xafafaf, 0xafafd7, 0xafafff, 0xafd700,
00028     0xafd75f, 0xafd787, 0xafd7af, 0xafd7d7, 0xafd7ff, 0xffff00, 0xffff5f,
00029     0xffff87, 0xffffaf, 0xffffd7, 0xffffff, 0xd70000, 0xd7005f, 0xd70087,
00030     0xd700af, 0xd700d7, 0xd700ff, 0xd75f00, 0xd75f5f, 0xd75f87, 0xd75faf,
00031     0xd75fd7, 0xd75fff, 0xd78700, 0xd7875f, 0xd78787, 0xd787af, 0xd787d7,
00032     0xd787ff, 0xd7af00, 0xd7af5f, 0xd7af87, 0xd7afaf, 0xd7afd7, 0xd7afff,
00033     0xd7d700, 0xd7d75f, 0xd7d787, 0xd7d7af, 0xd7d7d7, 0xd7d7ff, 0xd7ff00,
00034     0xd7ff5f, 0xd7ff87, 0xd7ffaf, 0xd7ffd7, 0xd7ffff, 0xff0000, 0xff005f,
00035     0xff0087, 0xff00af, 0xff00d7, 0xff00ff, 0xff5f00, 0xff5f5f, 0xff5f87,
00036     0xff5faf, 0xff5fd7, 0xff5fff, 0xff8700, 0xff875f, 0xff8787, 0xff87af,
00037     0xff87d7, 0xff87ff, 0xffaf00, 0xffaf5f, 0xffaf87, 0xffafaf, 0xffafd7,
00038     0xffafff, 0xffd700, 0xffd75f, 0xffd787, 0xffd7af, 0xffd7d7, 0xffd7ff,
00039     0xffff00, 0xffff5f, 0xffff87, 0xffffaf, 0xffffd7, 0xffffff, 0x080808,
00040     0x121212, 0x1c1c1c, 0x262626, 0x303030, 0x3a3a3a, 0x444444, 0x4e4e4e,
00041     0x585858, 0x626262, 0x6c6c6c, 0x767676, 0x808080, 0x8a8a8a, 0x949494,
00042     0x9e9e9e, 0xa8a8a8, 0xb2b2b2, 0xbcbcbc, 0xc6c6c6, 0xd0d0d0, 0xdadada,
00043     0xe4e4e4, 0xeeeeee};
00044
00045 // Tries to implement this standard for terminfo
00046 // https://man7.org/linux/man-pages/man4/console_codes.4.html
00047
00048 #define CHARSET_DEFAULT 0
00049 #define CHARSET_DEC_SPECIAL 1
00050
00051 void term_context_reinit(struct term_context *ctx)
00052 {
00053     ctx->tab_size = 8;
00054     ctx->autoflush = true;
00055     ctx->scroll_enabled = true;
00056     ctx->control_sequence = false;
00057     ctx->csi = false;
00058     ctx->escape = false;
00059     ctx->rrr = false;
00060     ctx->discard_next = false;
00061     ctx->bold = false;
00062     ctx->reverse_video = false;
00063     ctx->dec_private = false;
00064     ctx->insert_mode = false;
00065     ctx->g_select = 0;
00066     ctx->charsets[0] = CHARSET_DEFAULT;
00067     ctx->charsets[1] = CHARSET_DEC_SPECIAL;
00068     ctx->current_charset = 0;
00069     ctx->escape_offset = 0;
00070     ctx->esc_values_i = 0;
00071     ctx->saved_cursor_x = 0;
00072     ctx->saved_cursor_y = 0;
00073     ctx->current_primary = (size_t) -1;
00074     ctx->scroll_top_margin = 0;
00075     ctx->scroll_bottom_margin = ctx->rows;
00076 }
00077
00078 static void term_putchar(struct term_context *ctx, uint8_t c);
00079
00080 void term_write(struct term_context *ctx, const char *buf, size_t count)
00081 {
00082     for (size_t i = 0; i < count; i++)

```

```

00083     {
00084         term_putchar(ctx, buf[i]);
00085     }
00086
00087     if (ctx->autoflush)
00088     {
00089         ctx->double_buffer_flush(ctx);
00090     }
00091 }
00092
00093 static void sgr(struct term_context *ctx)
00094 {
00095     size_t i = 0;
00096
00097     if (!ctx->esc_values_i)
00098         goto def;
00099
00100     for (; i < ctx->esc_values_i; i++)
00101     {
00102         size_t offset;
00103
00104         if (ctx->esc_values[i] == 0)
00105         {
00106             def:
00107             if (ctx->reverse_video)
00108             {
00109                 ctx->reverse_video = false;
00110                 ctx->swap_palette(ctx);
00111             }
00112             ctx->bold = false;
00113             ctx->current_primary = (size_t) -1;
00114             ctx->set_text_bg_default(ctx);
00115             ctx->set_text_fg_default(ctx);
00116             continue;
00117         }
00118
00119         else if (ctx->esc_values[i] == 1)
00120         {
00121             ctx->bold = true;
00122             if (ctx->current_primary != (size_t) -1)
00123             {
00124                 if (!ctx->reverse_video)
00125                 {
00126                     ctx->set_text_fg_bright(ctx, ctx->current_primary);
00127                 }
00128                 else
00129                 {
00130                     ctx->set_text_bg_bright(ctx, ctx->current_primary);
00131                 }
00132             }
00133             continue;
00134         }
00135
00136         else if (ctx->esc_values[i] == 22)
00137         {
00138             ctx->bold = false;
00139             if (ctx->current_primary != (size_t) -1)
00140             {
00141                 if (!ctx->reverse_video)
00142                 {
00143                     ctx->set_text_fg(ctx, ctx->current_primary);
00144                 }
00145                 else
00146                 {
00147                     ctx->set_text_bg(ctx, ctx->current_primary);
00148                 }
00149             }
00150             continue;
00151         }
00152
00153         else if (ctx->esc_values[i] >= 30 && ctx->esc_values[i] <= 37)
00154         {
00155             offset = 30;
00156             ctx->current_primary = ctx->esc_values[i] - offset;
00157
00158             if (ctx->reverse_video)
00159             {
00160                 goto set_bg;
00161             }
00162
00163             set_fg:
00164             if (ctx->bold && !ctx->reverse_video)
00165             {
00166                 ctx->set_text_fg_bright(ctx, ctx->esc_values[i] - offset);
00167             }
00168             else
00169             {

```



```
00170         ctx->set_text_fg(ctx, ctx->esc_values[i] - offset);
00171     }
00172     continue;
00173 }
00174
00175 else if (ctx->esc_values[i] >= 40 && ctx->esc_values[i] <= 47)
00176 {
00177     offset = 40;
00178     if (ctx->reverse_video)
00179     {
00180         goto set_fg;
00181     }
00182
00183 set_bg:
00184     if (ctx->bold && ctx->reverse_video)
00185     {
00186         ctx->set_text_bg_bright(ctx, ctx->esc_values[i] - offset);
00187     }
00188     else
00189     {
00190         ctx->set_text_bg(ctx, ctx->esc_values[i] - offset);
00191     }
00192     continue;
00193 }
00194
00195 else if (ctx->esc_values[i] >= 90 && ctx->esc_values[i] <= 97)
00196 {
00197     offset = 90;
00198     ctx->current_primary = ctx->esc_values[i] - offset;
00199
00200     if (ctx->reverse_video)
00201     {
00202         goto set_bg_bright;
00203     }
00204
00205 set_fg_bright:
00206     ctx->set_text_fg_bright(ctx, ctx->esc_values[i] - offset);
00207     continue;
00208 }
00209
00210 else if (ctx->esc_values[i] >= 100 && ctx->esc_values[i] <= 107)
00211 {
00212     offset = 100;
00213     if (ctx->reverse_video)
00214     {
00215         goto set_fg_bright;
00216     }
00217
00218 set_bg_bright:
00219     ctx->set_text_bg_bright(ctx, ctx->esc_values[i] - offset);
00220     continue;
00221 }
00222
00223 else if (ctx->esc_values[i] == 39)
00224 {
00225     ctx->current_primary = (size_t) -1;
00226
00227     if (ctx->reverse_video)
00228     {
00229         ctx->swap_palette(ctx);
00230     }
00231
00232     ctx->set_text_fg_default(ctx);
00233
00234     if (ctx->reverse_video)
00235     {
00236         ctx->swap_palette(ctx);
00237     }
00238     continue;
00239 }
00240
00241 else if (ctx->esc_values[i] == 49)
00242 {
00243     if (ctx->reverse_video)
00244     {
00245         ctx->swap_palette(ctx);
00246     }
00247
00248     ctx->set_text_bg_default(ctx);
00249
00250     if (ctx->reverse_video)
00251     {
00252         ctx->swap_palette(ctx);
00253     }
00254
00255     continue;
00256 }
```

```

00257     }
00258
00259     else if (ctx->esc_values[i] == 7)
00260     {
00261         if (!ctx->reverse_video)
00262         {
00263             ctx->reverse_video = true;
00264             ctx->swap_palette(ctx);
00265         }
00266         continue;
00267     }
00268
00269     else if (ctx->esc_values[i] == 27)
00270     {
00271         if (ctx->reverse_video)
00272         {
00273             ctx->reverse_video = false;
00274             ctx->swap_palette(ctx);
00275         }
00276         continue;
00277     }
00278
00279     // 256/RGB
00280     else if (ctx->esc_values[i] == 38 || ctx->esc_values[i] == 48)
00281     {
00282         bool fg = ctx->esc_values[i] == 38;
00283
00284         i++;
00285         if (i >= ctx->esc_values_i)
00286         {
00287             break;
00288         }
00289
00290         switch (ctx->esc_values[i])
00291         {
00292             case 2:
00293             { // RGB
00294                 if (i + 3 >= ctx->esc_values_i)
00295                 {
00296                     goto out;
00297                 }
00298
00299                 uint32_t rgb_value = 0;
00300
00301                 rgb_value |= ctx->esc_values[i + 1] << 16;
00302                 rgb_value |= ctx->esc_values[i + 2] << 8;
00303                 rgb_value |= ctx->esc_values[i + 3];
00304
00305                 i += 3;
00306
00307                 (fg ? ctx->set_text_fg_rgb
00308                  : ctx->set_text_bg_rgb)(ctx, rgb_value);
00309
00310                 break;
00311             }
00312             case 5:
00313             { // 256 colors
00314                 if (i + 1 >= ctx->esc_values_i)
00315                 {
00316                     goto out;
00317                 }
00318
00319                 uint32_t col = ctx->esc_values[i + 1];
00320
00321                 i++;
00322
00323                 if (col < 8)
00324                 {
00325                     (fg ? ctx->set_text_fg : ctx->set_text_bg)(ctx, col);
00326                 }
00327                 else if (col < 16)
00328                 {
00329                     (fg ? ctx->set_text_fg_bright
00330                      : ctx->set_text_bg_bright)(ctx, col - 8);
00331                 }
00332                 else
00333                 {
00334                     uint32_t rgb_value = col256[col - 16];
00335                     (fg ? ctx->set_text_fg_rgb
00336                      : ctx->set_text_bg_rgb)(ctx, rgb_value);
00337                 }
00338
00339                 break;
00340             }
00341             default:
00342                 continue;
00343         }

```

```
00344     }
00345 }
00346
00347 out;;
00348 }
00349
00350 static void dec_private_parse(struct term_context *ctx, uint8_t c)
00351 {
00352     ctx->dec_private = false;
00353
00354     if (ctx->esc_values_i == 0)
00355     {
00356         return;
00357     }
00358
00359     bool set;
00360
00361     switch (c)
00362     {
00363         case 'h':
00364             set = true;
00365             break;
00366         case 'l':
00367             set = false;
00368             break;
00369         default:
00370             return;
00371     }
00372
00373     switch (ctx->esc_values[0])
00374     {
00375         case 25:
00376         {
00377             if (set)
00378             {
00379                 ctx->enable_cursor(ctx);
00380             }
00381             else
00382             {
00383                 ctx->disable_cursor(ctx);
00384             }
00385             return;
00386         }
00387     }
00388
00389     if (ctx->callback != NULL)
00390     {
00391         ctx->callback(ctx,
00392                     TERM_CB_DEC,
00393                     ctx->esc_values_i,
00394                     (uintptr_t) ctx->esc_values,
00395                     c);
00396     }
00397 }
00398
00399 static void linux_private_parse(struct term_context *ctx)
00400 {
00401     if (ctx->esc_values_i == 0)
00402     {
00403         return;
00404     }
00405
00406     if (ctx->callback != NULL)
00407     {
00408         ctx->callback(ctx,
00409                     TERM_CB_LINUX,
00410                     ctx->esc_values_i,
00411                     (uintptr_t) ctx->esc_values,
00412                     0);
00413     }
00414 }
00415
00416 static void mode_toggle(struct term_context *ctx, uint8_t c)
00417 {
00418     if (ctx->esc_values_i == 0)
00419     {
00420         return;
00421     }
00422
00423     bool set;
00424
00425     switch (c)
00426     {
00427         case 'h':
00428             set = true;
00429             break;
00430         case 'l':
```

```

00431         set = false;
00432         break;
00433     default:
00434         return;
00435     }
00436
00437     switch (ctx->esc_values[0])
00438     {
00439         case 4:
00440             ctx->insert_mode = set;
00441             return;
00442     }
00443
00444     if (ctx->callback != NULL)
00445     {
00446         ctx->callback(ctx,
00447                     TERM_CB_MODE,
00448                     ctx->esc_values_i,
00449                     (uintptr_t) ctx->esc_values,
00450                     c);
00451     }
00452 }
00453
00454 static void control_sequence_parse(struct term_context *ctx, uint8_t c)
00455 {
00456     if (ctx->escape_offset == 2)
00457     {
00458         switch (c)
00459         {
00460             case '[':
00461                 ctx->discard_next = true;
00462                 goto cleanup;
00463             case '?':
00464                 ctx->dec_private = true;
00465                 return;
00466         }
00467     }
00468
00469     if (c >= '0' && c <= '9')
00470     {
00471         if (ctx->esc_values_i == TERM_MAX_ESC_VALUES)
00472         {
00473             return;
00474         }
00475         ctx->rrr = true;
00476         ctx->esc_values[ctx->esc_values_i] *= 10;
00477         ctx->esc_values[ctx->esc_values_i] += c - '0';
00478         return;
00479     }
00480
00481     if (ctx->rrr == true)
00482     {
00483         ctx->esc_values_i++;
00484         ctx->rrr = false;
00485         if (c == ';')
00486             return;
00487     }
00488     else if (c == ';')
00489     {
00490         if (ctx->esc_values_i == TERM_MAX_ESC_VALUES)
00491         {
00492             return;
00493         }
00494         ctx->esc_values[ctx->esc_values_i] = 0;
00495         ctx->esc_values_i++;
00496         return;
00497     }
00498
00499     size_t esc_default;
00500     switch (c)
00501     {
00502         case 'J':
00503         case 'K':
00504         case 'q':
00505             esc_default = 0;
00506             break;
00507         default:
00508             esc_default = 1;
00509             break;
00510     }
00511
00512     for (size_t i = ctx->esc_values_i; i < TERM_MAX_ESC_VALUES; i++)
00513     {
00514         ctx->esc_values[i] = esc_default;
00515     }
00516
00517     if (ctx->dec_private == true)

```

```

00518     {
00519         dec_private_parse(ctx, c);
00520         goto cleanup;
00521     }
00522
00523     bool r          = ctx->scroll_enabled;
00524     ctx->scroll_enabled = false;
00525     size_t x, y;
00526     ctx->get_cursor_pos(ctx, &x, &y);
00527
00528     switch (c)
00529     {
00530     case 'F':
00531         x = 0;
00532         // FALLTHRU
00533     case 'A':
00534     {
00535         if (ctx->esc_values[0] > y)
00536             ctx->esc_values[0] = y;
00537         size_t orig_y          = y;
00538         size_t dest_y          = y - ctx->esc_values[0];
00539         bool will_be_in_scroll_region = false;
00540         if ((ctx->scroll_top_margin >= dest_y &&
00541              ctx->scroll_top_margin <= orig_y) ||
00542             (ctx->scroll_bottom_margin >= dest_y &&
00543              ctx->scroll_bottom_margin <= orig_y))
00544         {
00545             will_be_in_scroll_region = true;
00546         }
00547         if (will_be_in_scroll_region && dest_y < ctx->scroll_top_margin)
00548         {
00549             dest_y = ctx->scroll_top_margin;
00550         }
00551         ctx->set_cursor_pos(ctx, x, dest_y);
00552         break;
00553     }
00554     case 'E':
00555         x = 0;
00556         // FALLTHRU
00557     case 'e':
00558     case 'B':
00559     {
00560         if (y + ctx->esc_values[0] > ctx->rows - 1)
00561             ctx->esc_values[0] = (ctx->rows - 1) - y;
00562         size_t orig_y          = y;
00563         size_t dest_y          = y + ctx->esc_values[0];
00564         bool will_be_in_scroll_region = false;
00565         if ((ctx->scroll_top_margin >= orig_y &&
00566              ctx->scroll_top_margin <= dest_y) ||
00567             (ctx->scroll_bottom_margin >= orig_y &&
00568              ctx->scroll_bottom_margin <= dest_y))
00569         {
00570             will_be_in_scroll_region = true;
00571         }
00572         if (will_be_in_scroll_region && dest_y >= ctx->scroll_bottom_margin)
00573         {
00574             dest_y = ctx->scroll_bottom_margin - 1;
00575         }
00576         ctx->set_cursor_pos(ctx, x, dest_y);
00577         break;
00578     }
00579     case 'a':
00580     case 'C':
00581         if (x + ctx->esc_values[0] > ctx->cols - 1)
00582             ctx->esc_values[0] = (ctx->cols - 1) - x;
00583         ctx->set_cursor_pos(ctx, x + ctx->esc_values[0], y);
00584         break;
00585     case 'D':
00586         if (ctx->esc_values[0] > x)
00587             ctx->esc_values[0] = x;
00588         ctx->set_cursor_pos(ctx, x - ctx->esc_values[0], y);
00589         break;
00590     case 'c':
00591         if (ctx->callback != NULL)
00592         {
00593             ctx->callback(ctx, TERM_CB_PRIVATE_ID, 0, 0, 0);
00594         }
00595         break;
00596     case 'd':
00597         ctx->esc_values[0] -= 1;
00598         if (ctx->esc_values[0] >= ctx->rows)
00599             ctx->esc_values[0] = ctx->rows - 1;
00600         ctx->set_cursor_pos(ctx, x, ctx->esc_values[0]);
00601         break;
00602     case 'G':
00603     case '':
00604         ctx->esc_values[0] -= 1;

```

```

00605         if (ctx->esc_values[0] >= ctx->cols)
00606             ctx->esc_values[0] = ctx->cols - 1;
00607         ctx->set_cursor_pos(ctx, ctx->esc_values[0], y);
00608         break;
00609     case 'H':
00610     case 'f':
00611         ctx->esc_values[0] -= 1;
00612         ctx->esc_values[1] -= 1;
00613         if (ctx->esc_values[1] >= ctx->cols)
00614             ctx->esc_values[1] = ctx->cols - 1;
00615         if (ctx->esc_values[0] >= ctx->rows)
00616             ctx->esc_values[0] = ctx->rows - 1;
00617         ctx->set_cursor_pos(ctx, ctx->esc_values[1], ctx->esc_values[0]);
00618         break;
00619     case 'n':
00620         switch (ctx->esc_values[0])
00621         {
00622             case 5:
00623                 if (ctx->callback != NULL)
00624                 {
00625                     ctx->callback(ctx, TERM_CB_STATUS_REPORT, 0, 0, 0);
00626                 }
00627                 break;
00628             case 6:
00629                 if (ctx->callback != NULL)
00630                 {
00631                     ctx->callback(ctx, TERM_CB_POS_REPORT, x + 1, y + 1, 0);
00632                 }
00633                 break;
00634         }
00635         break;
00636     case 'q':
00637         if (ctx->callback != NULL)
00638         {
00639             ctx->callback(ctx, TERM_CB_KBD_LEDS, ctx->esc_values[0], 0, 0);
00640         }
00641         break;
00642     case 'J':
00643         switch (ctx->esc_values[0])
00644         {
00645             case 0:
00646             {
00647                 size_t rows_remaining = ctx->rows - (y + 1);
00648                 size_t cols_diff = ctx->cols - (x + 1);
00649                 size_t to_clear = rows_remaining * ctx->cols + cols_diff;
00650                 for (size_t i = 0; i < to_clear; i++)
00651                 {
00652                     ctx->raw_putchar(ctx, ' ');
00653                 }
00654                 ctx->set_cursor_pos(ctx, x, y);
00655                 break;
00656             }
00657             case 1:
00658             {
00659                 ctx->set_cursor_pos(ctx, 0, 0);
00660                 bool b = false;
00661                 for (size_t yc = 0; yc < ctx->rows; yc++)
00662                 {
00663                     for (size_t xc = 0; xc < ctx->cols; xc++)
00664                     {
00665                         ctx->raw_putchar(ctx, ' ');
00666                         if (xc == x && yc == y)
00667                         {
00668                             ctx->set_cursor_pos(ctx, x, y);
00669                             b = true;
00670                             break;
00671                         }
00672                     }
00673                     if (b == true)
00674                         break;
00675                 }
00676                 break;
00677             }
00678             case 2:
00679             case 3:
00680                 ctx->clear(ctx, false);
00681                 break;
00682         }
00683         break;
00684     case '@':
00685         for (size_t i = ctx->cols - 1; i--;)
00686         {
00687             ctx->move_character(ctx, i + ctx->esc_values[0], y, i, y);
00688             ctx->set_cursor_pos(ctx, i, y);
00689             ctx->raw_putchar(ctx, ' ');
00690             if (i == x)
00691             {

```

```

00692         break;
00693     }
00694 }
00695 ctx->set_cursor_pos(ctx, x, y);
00696 break;
00697 case 'P':
00698     for (size_t i = x + ctx->esc_values[0]; i < ctx->cols; i++)
00699         ctx->move_character(ctx, i - ctx->esc_values[0], y, i, y);
00700 ctx->set_cursor_pos(ctx, ctx->cols - ctx->esc_values[0], y);
00701 // FALLTHRU
00702 case 'X':
00703     for (size_t i = 0; i < ctx->esc_values[0]; i++)
00704         ctx->raw_putchar(ctx, ' ');
00705 ctx->set_cursor_pos(ctx, x, y);
00706 break;
00707 case 'm':
00708     sgr(ctx);
00709     break;
00710 case 's':
00711     ctx->get_cursor_pos(
00712         ctx, &ctx->saved_cursor_x, &ctx->saved_cursor_y);
00713     break;
00714 case 'u':
00715     ctx->set_cursor_pos(ctx, ctx->saved_cursor_x, ctx->saved_cursor_y);
00716     break;
00717 case 'K':
00718     switch (ctx->esc_values[0])
00719     {
00720         case 0:
00721         {
00722             for (size_t i = x; i < ctx->cols; i++)
00723                 ctx->raw_putchar(ctx, ' ');
00724             ctx->set_cursor_pos(ctx, x, y);
00725             break;
00726         }
00727         case 1:
00728         {
00729             ctx->set_cursor_pos(ctx, 0, y);
00730             for (size_t i = 0; i < x; i++)
00731                 ctx->raw_putchar(ctx, ' ');
00732             break;
00733         }
00734         case 2:
00735         {
00736             ctx->set_cursor_pos(ctx, 0, y);
00737             for (size_t i = 0; i < ctx->cols; i++)
00738                 ctx->raw_putchar(ctx, ' ');
00739             ctx->set_cursor_pos(ctx, x, y);
00740             break;
00741         }
00742     }
00743     break;
00744 case 'r':
00745     ctx->scroll_top_margin = 0;
00746     ctx->scroll_bottom_margin = ctx->rows;
00747     if (ctx->esc_values_i > 0)
00748     {
00749         ctx->scroll_top_margin = ctx->esc_values[0] - 1;
00750     }
00751     if (ctx->esc_values_i > 1)
00752     {
00753         ctx->scroll_bottom_margin = ctx->esc_values[1];
00754     }
00755     if (ctx->scroll_top_margin >= ctx->rows ||
00756         ctx->scroll_bottom_margin > ctx->rows ||
00757         ctx->scroll_top_margin >= (ctx->scroll_bottom_margin - 1))
00758     {
00759         ctx->scroll_top_margin = 0;
00760         ctx->scroll_bottom_margin = ctx->rows;
00761     }
00762     ctx->set_cursor_pos(ctx, 0, 0);
00763     break;
00764 case 'l':
00765 case 'h':
00766     mode_toggle(ctx, c);
00767     break;
00768 case ']':
00769     linux_private_parse(ctx);
00770     break;
00771 }
00772 ctx->scroll_enabled = r;
00773
00774 cleanup:
00775 ctx->control_sequence = false;
00776 ctx->escape = false;
00777 }

```

```

00779
00780 static void restore_state(struct term_context *ctx)
00781 {
00782     ctx->bold = ctx->saved_state_bold;
00783     ctx->reverse_video = ctx->saved_state_reverse_video;
00784     ctx->current_charset = ctx->saved_state_current_charset;
00785     ctx->current_primary = ctx->saved_state_current_primary;
00786
00787     ctx->restore_state(ctx);
00788 }
00789
00790 static void save_state(struct term_context *ctx)
00791 {
00792     ctx->save_state(ctx);
00793
00794     ctx->saved_state_bold = ctx->bold;
00795     ctx->saved_state_reverse_video = ctx->reverse_video;
00796     ctx->saved_state_current_charset = ctx->current_charset;
00797     ctx->saved_state_current_primary = ctx->current_primary;
00798 }
00799
00800 static void escape_parse(struct term_context *ctx, uint8_t c)
00801 {
00802     ctx->escape_offset++;
00803
00804     if (ctx->control_sequence == true)
00805     {
00806         control_sequence_parse(ctx, c);
00807         return;
00808     }
00809
00810     if (ctx->csi == true)
00811     {
00812         ctx->csi = false;
00813         goto is_csi;
00814     }
00815
00816     size_t x, y;
00817     ctx->get_cursor_pos(ctx, &x, &y);
00818
00819     switch (c)
00820     {
00821     case '[':
00822         is_csi:
00823         for (size_t i = 0; i < TERM_MAX_ESC_VALUES; i++)
00824             ctx->esc_values[i] = 0;
00825         ctx->esc_values_i = 0;
00826         ctx->rrr = false;
00827         ctx->control_sequence = true;
00828         return;
00829     case '7':
00830         save_state(ctx);
00831         break;
00832     case '8':
00833         restore_state(ctx);
00834         break;
00835     case 'c':
00836         term_context_reinit(ctx);
00837         ctx->clear(ctx, true);
00838         break;
00839     case 'D':
00840         if (y == ctx->scroll_bottom_margin - 1)
00841         {
00842             ctx->scroll(ctx);
00843             ctx->set_cursor_pos(ctx, x, y);
00844         }
00845         else
00846         {
00847             ctx->set_cursor_pos(ctx, x, y + 1);
00848         }
00849         break;
00850     case 'E':
00851         if (y == ctx->scroll_bottom_margin - 1)
00852         {
00853             ctx->scroll(ctx);
00854             ctx->set_cursor_pos(ctx, 0, y);
00855         }
00856         else
00857         {
00858             ctx->set_cursor_pos(ctx, 0, y + 1);
00859         }
00860         break;
00861     case 'M':
00862         // "Reverse linefeed"
00863         if (y == ctx->scroll_top_margin)
00864         {
00865             ctx->revscroll(ctx);

```



```

00866         ctx->set_cursor_pos(ctx, 0, y);
00867     }
00868     else
00869     {
00870         ctx->set_cursor_pos(ctx, 0, y - 1);
00871     }
00872     break;
00873 case 'Z':
00874     if (ctx->callback != NULL)
00875     {
00876         ctx->callback(ctx, TERM_CB_PRIVATE_ID, 0, 0, 0);
00877     }
00878     break;
00879 case '(':
00880 case ')':
00881     ctx->g_select = c - '\0';
00882     break;
00883 case '\e':
00884     if (ctx->in_bootloader == true)
00885     {
00886         ctx->raw_putchar(ctx, c);
00887     }
00888     break;
00889 }
00890
00891 ctx->escape = false;
00892 }
00893
00894 static uint8_t dec_special_to_cp437(uint8_t c)
00895 {
00896     switch (c)
00897     {
00898     case '\0':
00899         return 0x04;
00900     case '0':
00901         return 0xdb;
00902     case '~':
00903         return 0x18;
00904     case ',':
00905         return 0x1b;
00906     case '.':
00907         return 0x19;
00908     case 'a':
00909         return 0xb1;
00910     case 'f':
00911         return 0xf8;
00912     case 'g':
00913         return 0xf1;
00914     case 'h':
00915         return 0xb0;
00916     case 'j':
00917         return 0xd9;
00918     case 'k':
00919         return 0xbf;
00920     case 'l':
00921         return 0xda;
00922     case 'm':
00923         return 0xc0;
00924     case 'n':
00925         return 0xc5;
00926     case 'q':
00927         return 0xc4;
00928     case 's':
00929         return 0x5f;
00930     case 't':
00931         return 0xc3;
00932     case 'u':
00933         return 0xb4;
00934     case 'v':
00935         return 0xc1;
00936     case 'w':
00937         return 0xc2;
00938     case 'x':
00939         return 0xb3;
00940     case 'y':
00941         return 0xf3;
00942     case 'z':
00943         return 0xf2;
00944     case '~':
00945         return 0xfa;
00946     case '_':
00947         return 0xff;
00948     case '+':
00949         return 0x1a;
00950     case '{':
00951         return 0xe3;
00952     case '}':

```

```

00953         return 0x9c;
00954     }
00955
00956     return c;
00957 }
00958
00959 static void term_putchar(struct term_context *ctx, uint8_t c)
00960 {
00961     if (ctx->discard_next ||
00962         (ctx->in_bootloader == false && (c == 0x18 || c == 0x1a)))
00963     {
00964         ctx->discard_next = false;
00965         ctx->escape = false;
00966         ctx->csi = false;
00967         ctx->control_sequence = false;
00968         ctx->g_select = 0;
00969         return;
00970     }
00971
00972     if (ctx->escape == true)
00973     {
00974         escape_parse(ctx, c);
00975         return;
00976     }
00977
00978     if (ctx->g_select)
00979     {
00980         ctx->g_select--;
00981         switch (c)
00982         {
00983             case 'B':
00984                 ctx->charsets[ctx->g_select] = CHARSET_DEFAULT;
00985                 break;
00986             case 'O':
00987                 ctx->charsets[ctx->g_select] = CHARSET_DEC_SPECIAL;
00988                 break;
00989         }
00990         ctx->g_select = 0;
00991         return;
00992     }
00993
00994     size_t x, y;
00995     ctx->get_cursor_pos(ctx, &x, &y);
00996
00997     switch (c)
00998     {
00999         case 0x00:
01000         case 0x7f:
01001             return;
01002         case 0x9b:
01003             ctx->csi = true;
01004             // FALLTHRU
01005         case '\e':
01006             ctx->escape_offset = 0;
01007             ctx->escape = true;
01008             return;
01009         case '\t':
01010             if ((x / ctx->tab_size + 1) >= ctx->cols)
01011             {
01012                 ctx->set_cursor_pos(ctx, ctx->cols - 1, y);
01013                 return;
01014             }
01015             ctx->set_cursor_pos(
01016                 ctx, (x / ctx->tab_size + 1) * ctx->tab_size, y);
01017             return;
01018         case 0x0b:
01019         case 0x0c:
01020         case '\n':
01021             if (y == ctx->scroll_bottom_margin - 1)
01022             {
01023                 ctx->scroll(ctx);
01024                 ctx->set_cursor_pos(ctx, 0, y);
01025             }
01026             else
01027             {
01028                 ctx->set_cursor_pos(ctx, 0, y + 1);
01029             }
01030             return;
01031         case '\b':
01032             ctx->set_cursor_pos(ctx, x - 1, y);
01033             return;
01034         case '\r':
01035             ctx->set_cursor_pos(ctx, 0, y);
01036             return;
01037         case '\a':
01038             // The bell is handled by the kernel
01039             if (ctx->callback != NULL)

```

```

01040         {
01041             ctx->callback(ctx, TERM_CB_BELL, 0, 0, 0);
01042         }
01043         return;
01044     case 14:
01045         // Move to G1 set
01046         ctx->current_charset = 1;
01047         return;
01048     case 15:
01049         // Move to G0 set
01050         ctx->current_charset = 0;
01051         return;
01052 }
01053
01054 if (ctx->insert_mode == true)
01055 {
01056     for (size_t i = ctx->cols - 1;; i--)
01057     {
01058         ctx->move_character(ctx, i + 1, y, i, y);
01059         if (i == x)
01060         {
01061             break;
01062         }
01063     }
01064 }
01065
01066 // Translate character set
01067 switch (ctx->charsets[ctx->current_charset])
01068 {
01069     case CHARSET_DEFAULT:
01070         break;
01071     case CHARSET_DEC_SPECIAL:
01072         c = dec_special_to_cp437(c);
01073 }
01074
01075 ctx->raw_putchar(ctx, c);
01076 }

```

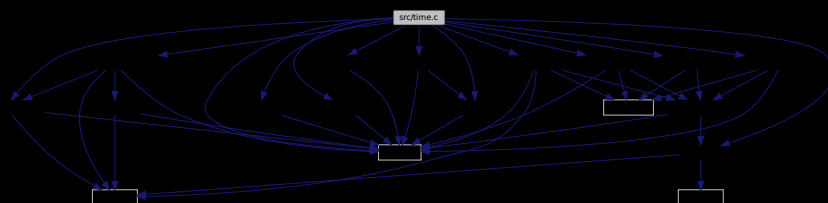
7.183 src/time.c File Reference

```

#include "include/apic/lapic.h"
#include "include/idt.h"
#include "include/io.h"
#include "include/kernel.h"
#include "include/lib/vector.h"
#include "include/limine.h"
#include "include/lock.h"
#include "include/mm/kmalloc.h"
#include "include/pic.h"
#include "include/printf.h"
#include "include/sched.h"
#include "include/time.h"
#include <stdint.h>

```

Include dependency graph for time.c:



Macros

- #define CHANNEL_ZERO 0x40

- `#define MODE_CMD 0x43`
- `#define CURRENT_YEAR 2026`
- `#define CMOS_ADDR 0x70`
- `#define CMOS_DATA 0x71`

Functions

- `void config_PIT (uint8_t freq)`
- `static VECTOR_TYPE (struct timer *)`
- `void timer_arm (struct timer *timer)`
Arm a software timer.
- `void timer_disarm (struct timer *timer)`
Disarm a software timer.
- `void time_init (void)`
Initialize RTC/PIT-facing timekeeping state during kernel bring-up.
- `uint16_t pit_get_current_count (void)`
Read the PIT's current counter value.
- `void pit_set_reload_value (uint16_t new_count)`
Program the PIT reload/divisor value directly.
- `void pit_set_frequency (uint64_t frequency)`
Program the PIT for an approximate requested frequency.
- `void pit_sleep (uint64_t ms)`
Busy/blocking PIT-based sleep helper used by current kernel code.
- `void sys_clock_handler (void)`
Primary PIT/system-clock interrupt handler.
- `void sys_clock_handler_alt (void)`
Alternate system-clock handler retained by the current timer code.
- `void task_switch_handler (void)`
- `void delta_int (uint64_t delta, uint8_t vector)`
- `void init_PIT (void)`
Program the PIT using the current default timer configuration.
- `int get_update_flag (void)`
- `uint8_t get_RTC_register (int reg)`
- `void read_rtc (void)`
Read a stable calendar/time snapshot from the RTC/CMOS path.
- `void print_date (void)`
Print the current calendar date.
- `int dayofweek (int y, int m, int d)`
Compute weekday index for a Gregorian calendar date.
- `void print_sys_time (void)`
Print current system time information for VSH diagnostics.
- `void print_load_time (void)`
Print boot/load-time diagnostics.

Variables

- int8_t `century_register` = 0x00
- uint8_t `second`
- uint8_t `minute`
- uint8_t `hour`
- uint8_t `day`
- uint8_t `month`
- uint16_t `year`
- uint64_t `system_timer_ms` = 0
- uint64_t `system_timer_fractions` = 0
- static volatile uint64_t `pit_ticks` = 0
- bool `timer_fired`
- struct timespec `time_mono` = {0, 0}
- struct timespec `time_real` = {0, 0}
- static spinlock_t `timers_lock` = SPINLOCK_INIT
- static const char * `weekday` []
- static const char * `month_str` []
- uint8_t `task_timer_count` = 0

7.183.1 Macro Definition Documentation

7.183.1.1 CHANNEL_ZERO

```
#define CHANNEL_ZERO 0x40
```

Definition at line 15 of file `time.c`.

7.183.1.2 CMOS_ADDR

```
#define CMOS_ADDR 0x70
```

Definition at line 18 of file `time.c`.

7.183.1.3 CMOS_DATA

```
#define CMOS_DATA 0x71
```

Definition at line 19 of file `time.c`.

7.183.1.4 CURRENT_YEAR

```
#define CURRENT_YEAR 2026
```

Definition at line 17 of file [time.c](#).

7.183.1.5 MODE_CMD

```
#define MODE_CMD 0x43
```

Definition at line 16 of file [time.c](#).

7.183.2 Function Documentation

7.183.2.1 config_PIT()

```
void config_PIT (
    uint8_t freq )
```

7.183.2.2 dayofweek()

```
int dayofweek (
    int y,
    int m,
    int d )
```

Compute weekday index for a Gregorian calendar date.

Parameters

<i>y</i>	Full year.
<i>m</i>	Month number.
<i>d</i>	Day of month.

Returns

Weekday index used by the date-printing code.

Definition at line 305 of file [time.c](#).

Here is the caller graph for this function:



7.183.2.3 `delta_int()`

```
void delta_int (
    uint64_t delta,
    uint8_t vector )
```

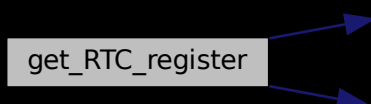
Definition at line 187 of file `time.c`.

7.183.2.4 `get_RTC_register()`

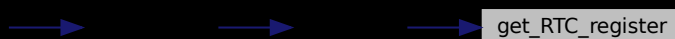
```
uint8_t get_RTC_register (
    int reg )
```

Definition at line 206 of file `time.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



Here is the caller graph for this function:



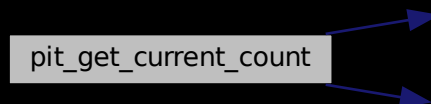
7.183.2.7 `pit_get_current_count()`

```
uint16_t pit_get_current_count (  
    void )
```

Read the PIT's current counter value.

Definition at line 107 of file `time.c`.

Here is the call graph for this function:



Here is the caller graph for this function:



7.183.2.8 pit_set_frequency()

```
void pit_set_frequency (
    uint64_t frequency )
```

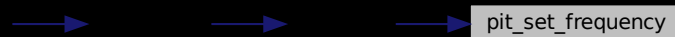
Program the PIT for an approximate requested frequency.

Definition at line 122 of file [time.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



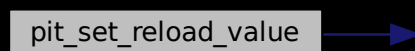
7.183.2.9 pit_set_reload_value()

```
void pit_set_reload_value (
    uint16_t new_count )
```

Program the PIT reload/divisor value directly.

Definition at line 115 of file [time.c](#).

Here is the call graph for this function:



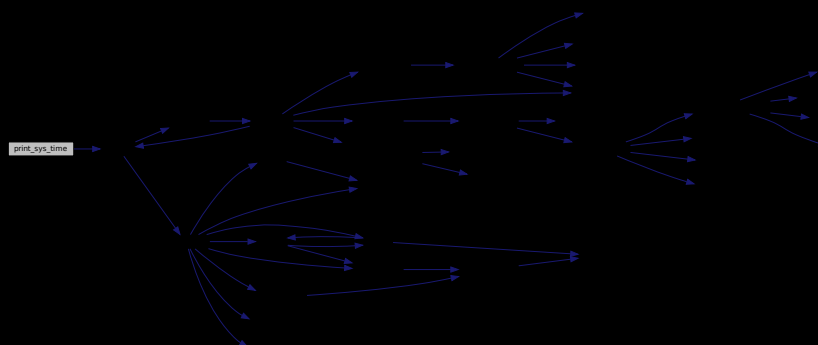
7.183.2.13 print_sys_time()

```
void print_sys_time (  
    void )
```

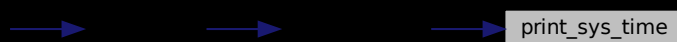
Print current system time information for VSH diagnostics.

Definition at line 313 of file [time.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



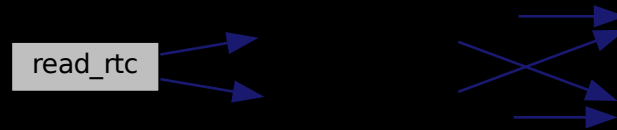
7.183.2.14 read_rtc()

```
void read_rtc (  
    void )
```

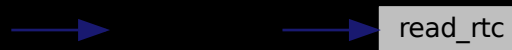
Read a stable calendar/time snapshot from the RTC/CMOS path.

Definition at line 212 of file [time.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



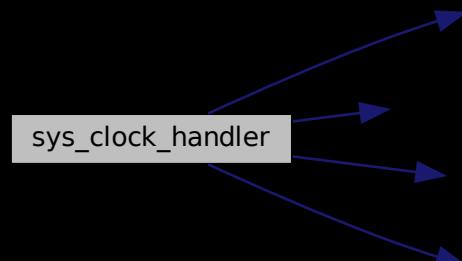
7.183.2.15 `sys_clock_handler()`

```
void sys_clock_handler (  
    void )
```

Primary PIT/system-clock interrupt handler.

Definition at line 137 of file `time.c`.

Here is the call graph for this function:



7.183.2.16 sys_clock_handler_alt()

```
void sys_clock_handler_alt (  
    void )
```

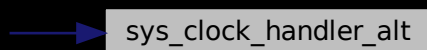
Alternate system-clock handler retained by the current timer code.

Definition at line 167 of file [time.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:

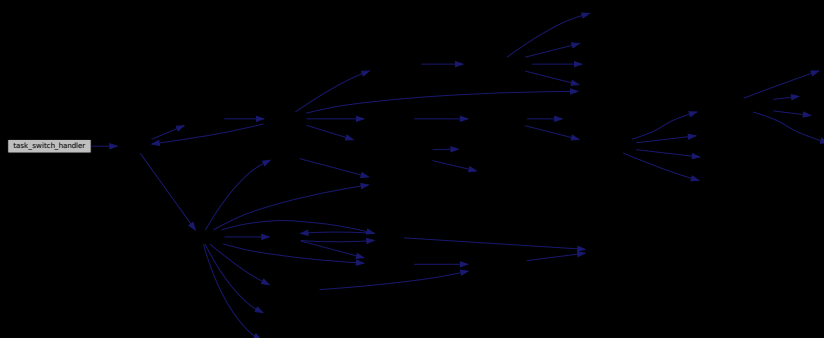


7.183.2.17 task_switch_handler()

```
void task_switch_handler (  
    void )
```

Definition at line 181 of file [time.c](#).

Here is the call graph for this function:



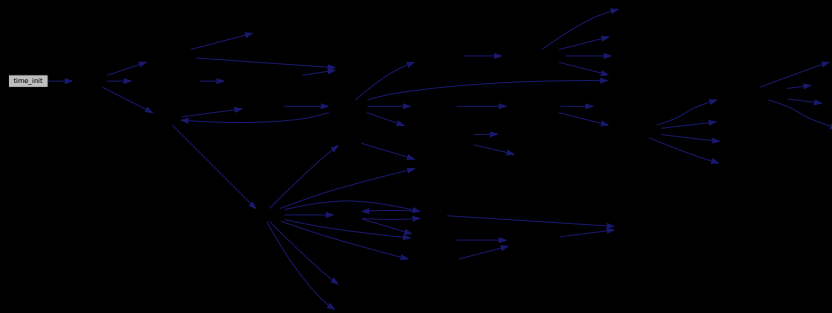
7.183.2.18 time_init()

```
void time_init (
    void )
```

Initialize RTC/PIT-facing timekeeping state during kernel bring-up.

Definition at line 84 of file [time.c](#).

Here is the call graph for this function:



Here is the caller graph for this function:



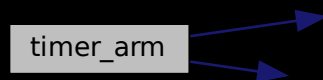
7.183.2.19 timer_arm()

```
void timer_arm (
    struct timer * timer )
```

Arm a software timer.

Definition at line 53 of file [time.c](#).

Here is the call graph for this function:



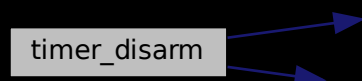
7.183.2.20 `timer_disarm()`

```
void timer_disarm (  
    struct timer * timer )
```

Disarm a software timer.

Definition at line 64 of file `time.c`.

Here is the call graph for this function:



7.183.2.21 `VECTOR_TYPE()`

```
static VECTOR_TYPE (  
    struct timer * ) [static]
```

Definition at line 38 of file `time.c`.

7.183.3 Variable Documentation

7.183.3.1 century_register

```
int8_t century_register = 0x00
```

Definition at line 23 of file [time.c](#).

7.183.3.2 day

```
uint8_t day
```

Definition at line 25 of file [time.c](#).

7.183.3.3 hour

```
uint8_t hour
```

Definition at line 25 of file [time.c](#).

7.183.3.4 minute

```
uint8_t minute
```

Definition at line 25 of file [time.c](#).

7.183.3.5 month

```
uint8_t month
```

Definition at line 25 of file [time.c](#).

7.183.3.6 month_str

```
const char* month_str[] [static]
```

Initial value:

```
= { "Jan",  
    "Feb",  
    "Mar",  
    "Apr",  
    "May",  
    "Jun",  
    "Jul",  
    "Aug",  
    "Sep",  
    "Oct",  
    "Nov",  
    "Dec" }
```

Definition at line 94 of file [time.c](#).

7.183.3.7 pit_ticks

```
volatile uint64_t pit_ticks = 0 [static]
```

Definition at line 30 of file [time.c](#).

7.183.3.8 second

```
uint8_t second
```

RTC calendar fields populated by [read_rtc\(\)](#).

Definition at line 25 of file [time.c](#).

7.183.3.9 system_timer_fractions

```
uint64_t system_timer_fractions = 0
```

Fractional/auxiliary timer accumulator used by the current implementation.

Definition at line 29 of file [time.c](#).

7.183.3.10 system_timer_ms

```
uint64_t system_timer_ms = 0
```

Historical millisecond-named timer counter.

Definition at line 28 of file [time.c](#).

7.183.3.11 task_timer_count

```
uint8_t task_timer_count = 0
```

Definition at line 179 of file [time.c](#).

7.183.3.12 time_mono

```
struct timespec time_mono = {0, 0}
```

Monotonic time accumulator maintained by the current timer subsystem.

Definition at line 34 of file [time.c](#).

7.183.3.13 time_real

```
struct timespec time_real = {0, 0}
```

Real-time accumulator/state maintained by the current timer subsystem.

Definition at line 35 of file [time.c](#).

7.183.3.14 timer_fired

```
bool timer_fired
```

Definition at line 32 of file [time.c](#).

7.183.3.15 timers_lock

```
spinlock_t timers_lock = SPINLOCK_INIT [static]
```

Definition at line 37 of file [time.c](#).

7.183.3.16 weekday

```
const char* weekday[] [static]
```

Initial value:

```
= {  
    "Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"}  
}
```

Definition at line 91 of file [time.c](#).

7.183.3.17 year

```
uint16_t year
```

Full RTC year value after century handling.

Definition at line 26 of file [time.c](#).

7.184 time.c

[Go to the documentation of this file.](#)

```
00001 #include "include/apic/lapic.h"  
00002 #include "include/idt.h"  
00003 #include "include/io.h"  
00004 #include "include/kernel.h"  
00005 #include "include/lib/vector.h"  
00006 #include "include/limine.h"  
00007 #include "include/lock.h"  
00008 #include "include/mm/kmalloc.h"  
00009 #include "include/pic.h"  
00010 #include "include/printf.h"  
00011 #include "include/sched.h"  
00012 #include "include/time.h"  
00013 #include <stdint.h>  
00014  
00015 #define CHANNEL_ZERO 0x40  
00016 #define MODE_CMD 0x43  
00017 #define CURRENT_YEAR 2026  
00018 #define CMOS_ADDR 0x70  
00019 #define CMOS_DATA 0x71  
00020  
00021 extern void config_PIT(uint8_t freq);  
00022  
00023 int8_t century_register = 0x00;  
00024  
00025 uint8_t second, minute, hour, day, month;  
00026 uint16_t year;  
00027  
00028 uint64_t system_timer_ms = 0;  
00029 uint64_t system_timer_fractions = 0;  
00030 static volatile uint64_t pit_ticks = 0;  
00031  
00032 bool timer_fired;
```

```

00033
00034 struct timespec time_mono = {0, 0};
00035 struct timespec time_real = {0, 0};
00036
00037 static spinlock_t timers_lock = SPINLOCK_INIT;
00038 static VECTOR_TYPE(struct timer *) armed_timers = VECTOR_INIT;
00039
00040 extern volatile struct limine_boot_time_request boot_time_req;
00041
00042 struct timer *timer_new(struct timespec when)
00043 {
00044     struct timer *timer = kmalloc(sizeof(*timer));
00045     if (timer == NULL)
00046         return NULL;
00047     timer->when = when;
00048     timer_arm(timer);
00049     return timer;
00050 }
00051
00052 void timer_arm(struct timer *timer)
00053 {
00054     spinlock_test_and_acq(&timers_lock);
00055     timer->index = armed_timers.length;
00056     timer->fired = false;
00057     VECTOR_PUSH_BACK(&armed_timers, timer);
00058     spinlock_release(&timers_lock);
00059 }
00060
00061 void timer_disarm(struct timer *timer)
00062 {
00063     spinlock_test_and_acq(&timers_lock);
00064     if (armed_timers.length == 0 || timer->index == -1 ||
00065         (size_t) timer->index >= armed_timers.length)
00066     {
00067         goto cleanup;
00068     }
00069     armed_timers.data[timer->index] =
00070         VECTOR_ITEM(&armed_timers, armed_timers.length - 1);
00071     VECTOR_REMOVE(&armed_timers, armed_timers.length - 1);
00072     timer->index = -1;
00073 }
00074
00075 cleanup:
00076     spinlock_release(&timers_lock);
00077 }
00078
00079 void time_init(void)
00080 {
00081     struct limine_boot_time_response *boot_time_resp = boot_time_req.response;
00082     time_real.tv_sec = boot_time_resp->boot_time;
00083     init_PIT();
00084 }
00085
00086 static const char *weekday[] = {
00087     "Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"};
00088
00089 static const char *month_str[] = {"Jan",
00090     "Feb",
00091     "Mar",
00092     "Apr",
00093     "May",
00094     "Jun",
00095     "Jul",
00096     "Aug",
00097     "Sep",
00098     "Oct",
00099     "Nov",
00100     "Dec"};
00101
00102 uint16_t pit_get_current_count(void)
00103 {
00104     outb(0x43, 0x00);
00105     uint8_t lo = inb(0x40);
00106     uint8_t hi = inb(0x40);
00107     return ((uint16_t) hi << 8) | lo;
00108 }
00109
00110 void pit_set_reload_value(uint16_t new_count)
00111 {
00112     outb(0x43, 0x34);
00113     outb(0x40, (uint8_t) new_count);
00114     outb(0x40, (uint8_t) (new_count >> 8));
00115 }

```

```

00120 }
00121
00122 void pit_set_frequency(uint64_t frequency)
00123 {
00124     uint64_t new_divisor = PIT_DIVIDEND / frequency;
00125     if (PIT_DIVIDEND % frequency > frequency / 2)
00126         new_divisor++;
00127     pit_set_reload_value((uint16_t) new_divisor);
00128 }
00129
00130 void pit_sleep(uint64_t ms)
00131 {
00132     uint64_t target = pit_ticks + ms;
00133     while (pit_ticks < target)
00134         asm volatile("hlt");
00135 }
00136
00137 void sys_clock_handler(void)
00138 {
00139     struct timespec interval = {.tv_sec = 0, .tv_nsec = 1000000000 / PIT_FREQ};
00140
00141     time_mono = timespec_add(time_mono, interval);
00142     time_real = timespec_add(time_real, interval);
00143
00144     if (spinlock_test_and_acq(&timers_lock))
00145     {
00146         for (size_t i = 0; i < armed_timers.length; i++)
00147         {
00148             struct timer *timer = VECTOR_ITEM(&armed_timers, i);
00149             if (timer->fired)
00150                 continue;
00151
00152             timer->when = timespec_sub(timer->when, interval);
00153             if (timer->when.tv_sec == 0 && timer->when.tv_nsec == 0)
00154                 timer->fired = true;
00155         }
00156         spinlock_release(&timers_lock);
00157     }
00158
00159     if (timer_fired == false)
00160         timer_fired = true;
00161
00162     system_timer_fractions++;
00163     system_timer_ms++;
00164     pit_ticks++;
00165 }
00166
00167 void sys_clock_handler_alt(void)
00168 {
00169     if (timer_fired == false)
00170         timer_fired = true;
00171
00172     system_timer_fractions++;
00173     system_timer_ms++;
00174
00175     if (allow_sched == true)
00176         schedule();
00177 }
00178
00179 uint8_t task_timer_count = 0;
00180
00181 void task_switch_handler(void)
00182 {
00183     printf("%s\n", "Switched!");
00184     task_timer_count = 0;
00185 }
00186
00187 void delta_int(uint64_t delta, uint8_t vector)
00188 {
00189     (void) delta;
00190     (void) vector;
00191 }
00192
00193 void init_PIT(void)
00194 {
00195     pit_set_frequency(PIT_FREQ);
00196     pic_unmask_irq(0);
00197     printf("%s\n", "PIT Online!");
00198 }
00199
00200 int get_update_flag(void)
00201 {
00202     outb(CMOS_ADDR, 0x0A);
00203     return (inb(CMOS_DATA) & 0x80);
00204 }
00205
00206 uint8_t get_RTC_register(int reg)

```

```

00207 {
00208     outb(CMOS_ADDR, reg);
00209     return inb(CMOS_DATA);
00210 }
00211
00212 void read_rtc(void)
00213 {
00214     uint8_t century = 0;
00215     uint8_t last_second;
00216     uint8_t last_minute;
00217     uint8_t last_hour;
00218     uint8_t last_day;
00219     uint8_t last_month;
00220     uint16_t last_year;
00221     uint8_t last_century = 0;
00222     uint8_t registerB;
00223
00224     while (get_update_flag())
00225         ;
00226
00227     second = get_RTC_register(0x00);
00228     minute = get_RTC_register(0x02);
00229     hour = get_RTC_register(0x04);
00230     day = get_RTC_register(0x07);
00231     month = get_RTC_register(0x08);
00232     year = get_RTC_register(0x09);
00233     if (century_register != 0)
00234         century = get_RTC_register(century_register);
00235
00236     do
00237     {
00238         last_second = second;
00239         last_minute = minute;
00240         last_hour = hour;
00241         last_day = day;
00242         last_month = month;
00243         last_year = year;
00244         last_century = century;
00245
00246         while (get_update_flag())
00247             ;
00248
00249         second = get_RTC_register(0x00);
00250         minute = get_RTC_register(0x02);
00251         hour = get_RTC_register(0x04);
00252         day = get_RTC_register(0x07);
00253         month = get_RTC_register(0x08);
00254         year = get_RTC_register(0x09);
00255         if (century_register != 0)
00256             century = get_RTC_register(century_register);
00257     } while ((last_second != second) || (last_minute != minute) ||
00258             (last_hour != hour) || (last_day != day) ||
00259             (last_month != month) || (last_year != year) ||
00260             (last_century != century));
00261
00262     registerB = get_RTC_register(0x0B);
00263
00264     if (!(registerB & 0x04))
00265     {
00266         second = (second & 0x0F) + ((second / 16) * 10);
00267         minute = (minute & 0x0F) + ((minute / 16) * 10);
00268         hour = ((hour & 0x0F) + ((hour & 0x70) / 16) * 10) | (hour & 0x80);
00269         day = (day & 0x0F) + ((day / 16) * 10);
00270         month = (month & 0x0F) + ((month / 16) * 10);
00271         year = (year & 0x0F) + ((year / 16) * 10);
00272         if (century_register != 0)
00273             century = (century & 0x0F) + ((century / 16) * 10);
00274     }
00275
00276     if (!(registerB & 0x02) && (hour & 0x80))
00277         hour = ((hour & 0x7F) + 12) % 24;
00278
00279     if (century_register != 0)
00280     {
00281         year += (uint16_t) century * 100u;
00282     }
00283     else
00284     {
00285         year += (CURRENT_YEAR / 100) * 100;
00286         if (year < CURRENT_YEAR)
00287             year += 100;
00288     }
00289 }
00290
00291 void print_date(void)
00292 {
00293     read_rtc();

```



```
00294
00295     printf_("%s", weekday[dayofweek(year, month, day)]);
00296     printf_("%s", " ");
00297     printf_("%i", day);
00298     printf_("%s", " ");
00299     printf_("%s", month_str[month - 1]);
00300     printf_("%s", " ");
00301     printf_("%i\n", year);
00302 }
00303
00304 // Courtesy of Tomohiko Sakamoto
00305 int dayofweek(int y, int m, int d)
00306 {
00307     static int t[] = {0, 3, 2, 5, 0, 3, 5, 1, 4, 6, 2, 4};
00308     if (m < 3)
00309         y -= 1;
00310     return (y + y / 4 - y / 100 + y / 400 + t[m - 1] + d) % 7;
00311 }
00312
00313 void print_sys_time(void)
00314 {
00315     printf_("%s", "Elapsed system time is: ");
00316     printf_("%i", system_timer_ms);
00317     printf_("%s", ".");
00318     printf_("%i", system_timer_fractions);
00319     printf_("%s\n", " ms.");
00320 }
00321
00322 void print_load_time(void)
00323 {
00324     printf_("%s\n", "Kernel Loaded");
00325     printf_("%s", "Loadtime roughly: ");
00326     printf_("%i", system_timer_ms);
00327     printf_("%s", ".");
00328     printf_("%i", system_timer_fractions);
00329     printf_("%s\n", " ms.");
00330 }
```


Index

- `_CPU_H`
 - `cpu.h`, 255
- `_CPU_UTILS_H`
 - `cpuUtils.h`, 272
- `_HASHMAP_H`
 - `hashmap.h`, 373
- `_IO_H`
 - `io.h`, 361
- `_KERNEL_H`
 - `kernel.h`, 365
- `_KERNEL_UTILS_H`
 - `KernelUtils.h`, 368
- `_LOCK_C_H`
 - `lock.h`, 406
- `_PAGING_H`
 - `paging.h`, 433
- `_PANIC_H`
 - `panic.h`, 448
- `_PRINTF_H`
 - `printf.h`, 458
- `_SCHED_H`
 - `sched.h`, 480
- `_SHELL_H`
 - `shell.h`, 494
- `_STACK_TRACE_H`
 - `stack_trace.h`, 499
- `_TIME_H`
 - `time.h`, 527
- `_TSS_H`
 - `tss.h`, 544
- `__attribute__`, 21
 - `attributes`, 21
 - `base`, 22
 - `base_high`, 22
 - `base_low`, 22
 - `base_mid`, 22
 - `cs`, 22
 - `ist`, 22
 - `limine_requests.c`, 583
 - `limit`, 23
 - `paging.h`, 435
 - `paging_bootstrap.c`, 680
 - `panic.h`, 448
 - `rsv0`, 23
- `__bss_end`
 - `paging_bootstrap.c`, 698
- `__data_start`
 - `paging_bootstrap.c`, 698
- `__kernel_end`
 - `neo_framealloc.c`, 657
 - `paging_bootstrap.c`, 698
- `__kernel_start`
 - `neo_framealloc.c`, 657
 - `paging_bootstrap.c`, 698
- `__rodata_end`
 - `paging_bootstrap.c`, 698
- `__rodata_start`
 - `paging_bootstrap.c`, 698
- `__text_end`
 - `paging_bootstrap.c`, 698
- `__text_start`
 - `paging_bootstrap.c`, 699
- `_atoi`
 - `printf.c`, 724
- `_etoa`
 - `printf.c`, 725
- `_ftoa`
 - `printf.c`, 727
- `_is_digit`
 - `printf.c`, 729
- `_ntoa_format`
 - `printf.c`, 730
- `_ntoa_long`
 - `printf.c`, 732
- `_ntoa_long_long`
 - `printf.c`, 734
- `_out_buffer`
 - `printf.c`, 736
- `_out_char`
 - `printf.c`, 737
- `_out_fct`
 - `printf.c`, 738
- `_out_null`
 - `printf.c`, 739
- `_out_rev`
 - `printf.c`, 740
- `_putchar`
 - `printf.h`, 460
 - `putchar.c`, 768
- `_start`
 - `kernel.c`, 563
- `_strlen_s`
 - `printf.c`, 741
- `_vsprintf`
 - `printf.c`, 742
- `a`
 - `limine_uuid`, 102
- `access_flag`

- GDT_Entry, 47
- GDT_Entry_16, 49
- GDT_Entry_32, 51
- acpi.c
 - acpi_find_sdt, 142
 - acpi_init, 143
 - halt, 143
 - rsdp, 144
 - rsdp_req, 144
 - rsdt, 144
 - sdt, 145
 - use_xsdt, 144
- acpi.h
 - acpi_find_sdt, 238
 - acpi_init, 238
- acpi_find_sdt
 - acpi.c, 142
 - acpi.h, 238
- acpi_init
 - acpi.c, 143
 - acpi.h, 238
- acpi_support
 - kswitches, 57
- active
 - cpu_local, 30
- addr_debug
 - kswitches, 57
- ADDR_MASK
 - paging_bootstrap.c, 678
- ADDR_MASK_1G
 - paging_bootstrap.c, 678
- ADDR_MASK_2M
 - paging_bootstrap.c, 678
- address
 - madt_io_apic, 104
- addresses
 - stack_return, 121
- ALIGN_16BIT
 - cpuUtils.c, 198
 - global_defs.h, 345
- ALIGN_4K
 - global_defs.h, 345
- ALIGN_DOWN
 - early_alloc.c, 210
 - kmalloc.c, 593
 - neo_framealloc.c, 643
 - paging_bootstrap.c, 678
- ALIGN_UP
 - early_alloc.c, 210
 - kmalloc.c, 593
 - neo_framealloc.c, 643
 - paging_bootstrap.c, 679
- ALIGNMENT
 - kmalloc.c, 593
- ALLOC_FLAG
 - kmalloc.c, 593
- alloc_table
 - paging_bootstrap.c, 680
- alloc_xsave
 - cpuUtils.c, 170
 - cpuUtils.h, 277
- allocate_from_block
 - kmalloc.c, 597
- allocations
 - memtest_stats, 109
- allow_sched
 - sched.c, 788
 - sched.h, 486
- ansi_bright_colours
 - fbterm_context, 35
- ansi_colours
 - fbterm_context, 35
- apic_id
 - madt_io_apic, 104
 - madt_lapic, 106
- arg
 - out_fct_wrap_type, 111
- array_size
 - stack_return, 121
- ASSERT
 - global_defs.h, 345
- attributes
 - __attribute__, 21
- autoflush
 - term_context, 123
- b
 - limine_uuid, 102
- base
 - __attribute__, 22
 - limine_memmap_entry, 86
- base_frame
 - isr_xframe_t, 53
- base_high
 - __attribute__, 22
 - GDT_Entry, 47
 - GDT_Entry_16, 49
 - GDT_Entry_32, 51
 - TSS_Entry, 138
- base_low
 - __attribute__, 22
 - GDT_Entry, 48
 - GDT_Entry_16, 49
 - GDT_Entry_32, 51
 - TSS_Entry, 138
- base_mid
 - __attribute__, 22
 - TSS_Entry, 139
- base_middle
 - GDT_Entry, 48
 - GDT_Entry_16, 49
 - GDT_Entry_32, 51
- base_up
 - TSS_Entry, 139
- basic_heap_test
 - memtest.c, 619
- bg

- fbterm_char, 32
- Black
 - kernel.c, 561
- BLK_SIZE
 - kmalloc.c, 594
- block_footer, 23
 - size_and_flags, 23
- block_header, 24
 - next_free, 24
 - prev_free, 24
 - size_and_flags, 25
- block_t
 - kmalloc.c, 596
- Blue
 - kernel.c, 561
- blue_mask_shift
 - limine_framebuffer, 75
- blue_mask_size
 - limine_framebuffer, 75
- bm_clear
 - neo_framealloc.c, 644
- bm_set
 - neo_framealloc.c, 644
- bm_test
 - neo_framealloc.c, 644
- bold
 - term_context, 123
- Boot and Kernel Bring-up, 13
- boot_time
 - limine_boot_time_response, 62
- bootspace
 - kernel.c, 571
 - kernel.h, 365
- bpp
 - fbterm_context, 35
 - limine_framebuffer, 75
- breakpoint
 - kernel.c, 564
- bsp
 - cpu_local, 30
- buffer_or_flush
 - putchar.c, 770
- builtin_font
 - framebuffer.c, 835
- bus_source
 - madt_iso, 105
- bytes_to_gib
 - KernelUtils.c, 577
 - KernelUtils.h, 368
- bytes_to_mib
 - KernelUtils.c, 578
 - KernelUtils.h, 368
- c
 - fbterm_char, 32
 - fbterm_queue_item, 42
 - limine_uuid, 102
- callback
 - term_context, 123
- canvas
 - fbterm_context, 35
- canvas_size
 - fbterm_context, 35
- CAS
 - global_defs.h, 345
- century_register
 - time.c, 892
- cfg_XCR0
 - cpuUtils.c, 171
- CHANNEL_ZERO
 - time.c, 879
- CHARSET_DEC_SPECIAL
 - term.c, 849
- CHARSET_DEFAULT
 - term.c, 849
- charsets
 - term_context, 123
- check_acpi
 - cpuUtils.c, 171
 - cpuUtils.h, 277
- check_apic
 - cpuUtils.c, 172
 - cpuUtils.h, 278
- check_avx
 - cpuUtils.c, 173
 - cpuUtils.h, 279
- check_ds
 - cpuUtils.c, 174
 - cpuUtils.h, 280
- check_fpu
 - cpuUtils.c, 174
 - cpuUtils.h, 280
- check_fxr
 - cpuUtils.c, 175
 - cpuUtils.h, 281
- check_htt
 - cpuUtils.c, 176
 - cpuUtils.h, 282
- check_mca
 - cpuUtils.c, 176
 - cpuUtils.h, 282
- check_mce
 - cpuUtils.c, 177
 - cpuUtils.h, 283
- check_oxsave
 - cpuUtils.c, 178
 - cpuUtils.h, 284
- check_pae
 - cpuUtils.c, 179
 - cpuUtils.h, 285
- check_pcid
 - cpuUtils.c, 179
 - cpuUtils.h, 285
- check_rdrand
 - cpuUtils.c, 180
 - cpuUtils.h, 286
- check_rdseed

- cpuUtils.c, 181
- cpuUtils.h, 287
- check_sep
 - cpuUtils.c, 181
 - cpuUtils.h, 287
- check_sse
 - cpuUtils.c, 182
 - cpuUtils.h, 288
- check_tm
 - cpuUtils.c, 183
 - cpuUtils.h, 289
- check_tsc
 - cpuUtils.c, 184
 - cpuUtils.h, 290
- check_xsave
 - cpuUtils.c, 184
 - cpuUtils.h, 290
- checksum
 - rsdp, 114
 - sdt, 117
- clear
 - term_context, 124
- clear_slots
 - memtest.c, 619
- CLR_ALLOC
 - kmalloc.c, 594
- cmd_parser
 - shell.c, 795
 - shell.h, 494
- CMOS_ADDR
 - time.c, 879
- CMOS_DATA
 - time.c, 879
- coalesce
 - kmalloc.c, 598
- col256
 - term.c, 864
- cols
 - term_context, 124
- columns
 - limine_terminal, 98
- compare_char
 - framebuffer.c, 817
- compute_max_phys_and_count
 - neo_framealloc.c, 645
- config_PIT
 - time.c, 880
- consts.h
 - KERNEL_VERSION, 252
- control_registers
 - isr_xframe_t, 53
- control_sequence
 - term_context, 124
- control_sequence_parse
 - term.c, 849
- cpu.c
 - fpu_bank_size, 167
 - fpu_rest, 167
- fpu_save, 168
- sysenter, 168
- this_cpu, 166
- cpu.h
 - _CPU_H, 255
 - cpu_init, 256
 - cpuid, 256
 - CPUID_AVX, 255
 - CPUID_AVX512, 255
 - CPUID_SEP, 255
 - CPUID_XSAVE, 255
 - disable_interrupts, 256
 - enable_interrupts, 257
 - fpu_bank_size, 266
 - fpu_rest, 266
 - fpu_save, 267
 - fxrstor, 257
 - fxsave, 257
 - get_fs_base, 258
 - get_gs_base, 258
 - get_kernel_gs_base, 259
 - interrupt_state, 259
 - interrupt_toggle, 260
 - rdrand, 261
 - rdseed, 261
 - rdtsc, 262
 - set_fs_base, 262
 - set_gs_base, 263
 - set_kernel_gs_base, 263
 - sysenter, 267
 - this_cpu, 264
 - wrmsr, 265
 - xrstor, 266
 - xsave, 266
- cpu_ctx, 25
 - cs, 26
 - ds, 26
 - err, 26
 - es, 26
 - r10, 26
 - r11, 26
 - r12, 27
 - r13, 27
 - r14, 27
 - r15, 27
 - r8, 27
 - r9, 27
 - rax, 28
 - rbp, 28
 - rbx, 28
 - rcx, 28
 - rdi, 28
 - rdx, 28
 - rflags, 29
 - rip, 29
 - rsi, 29
 - rsp, 29
 - ss, 29

- cpu_init
 - cpu.h, 256
- cpu_local, 30
 - active, 30
 - bsp, 30
 - cpu_number, 31
 - idle_thread, 31
 - lapic_freq, 31
 - lapic_id, 31
 - last_run_queue_index, 31
 - timer_function, 31
 - tss, 32
- cpu_number
 - cpu_local, 31
- cpuid
 - cpu.h, 256
- CPUID_AVX
 - cpu.h, 255
- CPUID_AVX512
 - cpu.h, 255
- cpuid_check_acpi
 - cpuUtils.c, 185
- cpuid_check_apic
 - cpuUtils.c, 185
- cpuid_check_avx
 - cpuUtils.c, 186
- cpuid_check_ds
 - cpuUtils.c, 186
- cpuid_check_fpu
 - cpuUtils.c, 186
- cpuid_check_fxsr
 - cpuUtils.c, 187
- cpuid_check_htt
 - cpuUtils.c, 187
- cpuid_check_mca
 - cpuUtils.c, 187
- cpuid_check_mce
 - cpuUtils.c, 188
- cpuid_check_oxsave
 - cpuUtils.c, 188
- cpuid_check_pae
 - cpuUtils.c, 188
- cpuid_check_pcid
 - cpuUtils.c, 189
- cpuid_check_rdrand
 - cpuUtils.c, 189
- cpuid_check_rdseed
 - cpuUtils.c, 189
- cpuid_check_sep
 - cpuUtils.c, 190
- cpuid_check_sse
 - cpuUtils.c, 190
- cpuid_check_tm
 - cpuUtils.c, 190
- cpuid_check_tsc
 - cpuUtils.c, 191
- cpuid_check_xsave
 - cpuUtils.c, 191
- cpuid_readout
 - cpuUtils.c, 191
 - cpuUtils.h, 291
- CPUID_SEP
 - cpu.h, 255
- CPUID_VENDOR_AMD
 - cpuUtils.h, 272
- CPUID_VENDOR_AO486
 - cpuUtils.h, 272
- CPUID_VENDOR_BHYVE
 - cpuUtils.h, 272
- CPUID_VENDOR_CENTAUR
 - cpuUtils.h, 273
- CPUID_VENDOR_CYRIX
 - cpuUtils.h, 273
- CPUID_VENDOR_ELBRUS
 - cpuUtils.h, 273
- CPUID_VENDOR_HYGON
 - cpuUtils.h, 273
- CPUID_VENDOR_HYPERV
 - cpuUtils.h, 273
- CPUID_VENDOR_INTEL
 - cpuUtils.h, 273
- CPUID_VENDOR_KVM
 - cpuUtils.h, 274
- CPUID_VENDOR_NEXGEN
 - cpuUtils.h, 274
- CPUID_VENDOR_NSC
 - cpuUtils.h, 274
- CPUID_VENDOR_OLDAMD
 - cpuUtils.h, 274
- CPUID_VENDOR_OLDAO486
 - cpuUtils.h, 274
- CPUID_VENDOR_OLDTRANSMETA
 - cpuUtils.h, 274
- CPUID_VENDOR_PARALLELS
 - cpuUtils.h, 275
- CPUID_VENDOR_PARALLELS_ALT
 - cpuUtils.h, 275
- CPUID_VENDOR_QEMU
 - cpuUtils.h, 275
- CPUID_VENDOR_QNX
 - cpuUtils.h, 275
- CPUID_VENDOR_RISE
 - cpuUtils.h, 275
- CPUID_VENDOR_SIS
 - cpuUtils.h, 275
- CPUID_VENDOR_TRANSMETA
 - cpuUtils.h, 276
- CPUID_VENDOR_UMC
 - cpuUtils.h, 276
- CPUID_VENDOR_VIA
 - cpuUtils.h, 276
- CPUID_VENDOR_VIRTUALBOX
 - cpuUtils.h, 276
- CPUID_VENDOR_VMWARE
 - cpuUtils.h, 276
- CPUID_VENDOR_VORTEX

- cpuUtils.h, 276
- CPUID_VENDOR_XEN
 - cpuUtils.h, 277
- CPUID_VENDOR_ZHAOXIN
 - cpuUtils.h, 277
- CPUID_XSAVE
 - cpu.h, 255
- cpuUtils.c
 - ALIGN_16BIT, 198
 - alloc_xsave, 170
 - cfg_XCR0, 171
 - check_acpi, 171
 - check_apic, 172
 - check_avx, 173
 - check_ds, 174
 - check_fpu, 174
 - check_fxsr, 175
 - check_htt, 176
 - check_mca, 176
 - check_mce, 177
 - check_oxsave, 178
 - check_pae, 179
 - check_pcid, 179
 - check_rdrand, 180
 - check_rdseed, 181
 - check_sep, 181
 - check_sse, 182
 - check_tm, 183
 - check_tsc, 184
 - check_xsave, 184
 - cpuid_check_acpi, 185
 - cpuid_check_apic, 185
 - cpuid_check_avx, 186
 - cpuid_check_ds, 186
 - cpuid_check_fpu, 186
 - cpuid_check_fxsr, 187
 - cpuid_check_htt, 187
 - cpuid_check_mca, 187
 - cpuid_check_mce, 188
 - cpuid_check_oxsave, 188
 - cpuid_check_pae, 188
 - cpuid_check_pcid, 189
 - cpuid_check_rdrand, 189
 - cpuid_check_rdseed, 189
 - cpuid_check_sep, 190
 - cpuid_check_sse, 190
 - cpuid_check_tm, 190
 - cpuid_check_tsc, 191
 - cpuid_check_xsave, 191
 - cpuid_readout, 191
 - fpu_init, 192
 - get_apic_base_address, 193
 - get_model, 193
 - get_xsave_size, 194
 - halt, 194
 - has_ACPI, 198
 - no_avx, 194
 - no_oxsave, 195
- no_sse, 195
- no_xsave, 196
- rand_asm, 196
- rdrand_asm, 197
- rdseed_asm, 197
- read_XCR0, 198
- test_em, 198
- cpuUtils.h
 - _CPU_UTILS_H, 272
 - alloc_xsave, 277
 - check_acpi, 277
 - check_apic, 278
 - check_avx, 279
 - check_ds, 280
 - check_fpu, 280
 - check_fxsr, 281
 - check_htt, 282
 - check_mca, 282
 - check_mce, 283
 - check_oxsave, 284
 - check_pae, 285
 - check_pcid, 285
 - check_rdrand, 286
 - check_rdseed, 287
 - check_sep, 287
 - check_sse, 288
 - check_tm, 289
 - check_tsc, 290
 - check_xsave, 290
 - cpuid_readout, 291
 - CPUID_VENDOR_AMD, 272
 - CPUID_VENDOR_AO486, 272
 - CPUID_VENDOR_BHYVE, 272
 - CPUID_VENDOR_CENTAUR, 273
 - CPUID_VENDOR_CYRIX, 273
 - CPUID_VENDOR_ELBRUS, 273
 - CPUID_VENDOR_HYGON, 273
 - CPUID_VENDOR_HYPERV, 273
 - CPUID_VENDOR_INTEL, 273
 - CPUID_VENDOR_KVM, 274
 - CPUID_VENDOR_NEXGEN, 274
 - CPUID_VENDOR_NSC, 274
 - CPUID_VENDOR_OLDAMD, 274
 - CPUID_VENDOR_OLDAO486, 274
 - CPUID_VENDOR_OLDTRANSMETA, 274
 - CPUID_VENDOR_PARALLELS, 275
 - CPUID_VENDOR_PARALLELS_ALT, 275
 - CPUID_VENDOR_QEMU, 275
 - CPUID_VENDOR_QNX, 275
 - CPUID_VENDOR_RISE, 275
 - CPUID_VENDOR_SIS, 275
 - CPUID_VENDOR_TRANSMETA, 276
 - CPUID_VENDOR_UMC, 276
 - CPUID_VENDOR_VIA, 276
 - CPUID_VENDOR_VIRTUALBOX, 276
 - CPUID_VENDOR_VMWARE, 276
 - CPUID_VENDOR_VORTEX, 276
 - CPUID_VENDOR_XEN, 277

- CPUID_VENDOR_ZHAOXIN, 277
- fpu_init, 292
- get_model, 293
- has_ACPI, 294
- rand_asm, 293
- xsave_bank, 294
- cr0
 - isr_xframe_t, 53
- cr2
 - isr_xframe_t, 53
- cr3
 - isr_xframe_t, 53
- cr4
 - isr_xframe_t, 54
- create_process
 - sched.c, 784
 - sched.h, 482
- creator_id
 - sdt, 117
- creator_revision
 - sdt, 118
- cs
 - __attribute__, 22
 - cpu_ctx, 26
 - isr_xframe_t, 54
- csi
 - term_context, 124
- current
 - sched.c, 789
 - sched.h, 486
- current_charset
 - term_context, 124
- current_pml4_phys
 - kmalloc.c, 598
- current_primary
 - term_context, 124
- CURRENT_YEAR
 - time.c, 879
- cursor_status
 - fbterm_context, 35
- cursor_x
 - fbterm_context, 36
- cursor_y
 - fbterm_context, 36
- Cyan
 - kernel.c, 562
- d
 - limine_uuid, 102
- data
 - rsdt, 116
- day
 - time.c, 892
 - time.h, 539
- dayofweek
 - time.c, 880
 - time.h, 528
- DC
 - gdt.h, 340
- dec_private
 - term_context, 125
- dec_private_parse
 - term.c, 850
- dec_special_to_cp437
 - term.c, 851
- default_bg
 - fbterm_context, 36
- default_fg
 - fbterm_context, 36
- deinit
 - term_context, 125
- delta_int
 - time.c, 881
- desc
 - gdt.c, 222
- Device Drivers, 17
- Diagnostics and Qualification, 19
- disable_cursor
 - term_context, 125
- disable_interrupts
 - cpu.h, 256
- discard_next
 - term_context, 125
- double_buffer_flush
 - term_context, 125
- draw_cursor
 - framebuffer.c, 817
- ds
 - cpu_ctx, 26
- dss
 - isr_xframe_t, 54
- dump_hex
 - stack_trace.c, 802
 - stack_trace.h, 499
- DuplicateRecursive
 - paging.c, 663
- dyn_isr_handler
 - interrupts.c, 552
- E2BIG
 - errno.h, 311
- EACCESS
 - errno.h, 312
- EADDRINUSE
 - errno.h, 312
- EADDRNOTAVAIL
 - errno.h, 312
- EADV
 - errno.h, 312
- EAFNOSUPPORT
 - errno.h, 312
- EAGAIN
 - errno.h, 312
- EALREADY
 - errno.h, 313
- Early Physical Allocator, 14
- early_alloc.c
 - ALIGN_DOWN, 210

- ALIGN_UP, 210
- early_alloc_page, 210
- early_init, 211
- early_pages_used, 212
- early_reserved_region, 212
- early_set_region, 213
- g_base, 214
- g_cursor, 215
- g_end, 215
- g_region_forced, 215
- memmap_ready, 213
- memmap_req, 215
- memzero, 214
- PAGE_SIZE, 210
- pick_region_from_memmap, 214
- early_alloc.h
 - early_alloc_page, 307
 - early_init, 307
 - early_pages_used, 307
 - early_reserved_region, 308
 - early_set_region, 308
 - paddr_t, 306
- early_alloc_page
 - early_alloc.c, 210
 - early_alloc.h, 307
 - neo_framealloc.c, 645
- early_init
 - early_alloc.c, 211
 - early_alloc.h, 307
 - neo_framealloc.c, 646
- early_pages_used
 - early_alloc.c, 212
 - early_alloc.h, 307
- early_reserved_region
 - early_alloc.c, 212
 - early_alloc.h, 308
 - neo_framealloc.c, 647
- early_set_region
 - early_alloc.c, 213
 - early_alloc.h, 308
- early_term
 - kernel.c, 571
 - kernel.h, 365
- EBADE
 - errno.h, 313
- EBADF
 - errno.h, 313
- EBADFD
 - errno.h, 313
- EBADMSG
 - errno.h, 313
- EBADR
 - errno.h, 313
- EBADRQC
 - errno.h, 314
- EBADSLT
 - errno.h, 314
- EBFONT
 - errno.h, 314
- EBUSY
 - errno.h, 314
- ECANCELED
 - errno.h, 314
- ECHILD
 - errno.h, 314
- ECHRNG
 - errno.h, 315
- ECOMM
 - errno.h, 315
- ECONNABORTED
 - errno.h, 315
- ECONNREFUSED
 - errno.h, 315
- ECONNRESET
 - errno.h, 315
- EDEADLK
 - errno.h, 315
- EDEADLOCK
 - errno.h, 316
- EDESTADDRREQ
 - errno.h, 316
- edid_size
 - limine_framebuffer, 75
- EDOM
 - errno.h, 316
- EDOTDOT
 - errno.h, 316
- EDQUOT
 - errno.h, 316
- EEXIST
 - errno.h, 316
- EFAULT
 - errno.h, 317
- EFBIG
 - errno.h, 317
- EHOSTDOWN
 - errno.h, 317
- EHOSTUNREACH
 - errno.h, 317
- EHWPOISON
 - errno.h, 317
- EIDRM
 - errno.h, 317
- EILSEQ
 - errno.h, 318
- EINPROGRESS
 - errno.h, 318
- EINTR
 - errno.h, 318
- EINVAL
 - errno.h, 318
- EIO
 - errno.h, 318
- EISCONN
 - errno.h, 318
- EISDIR

- errno.h, 319
- EISNAM
 - errno.h, 319
- EKEYEXPIRED
 - errno.h, 319
- EKEYREJECTED
 - errno.h, 319
- EKEYREVOKED
 - errno.h, 319
- EL2HLT
 - errno.h, 319
- EL2NSYNC
 - errno.h, 320
- EL3HLT
 - errno.h, 320
- EL3RST
 - errno.h, 320
- ELIBACC
 - errno.h, 320
- ELIBBAD
 - errno.h, 320
- ELIBEXEC
 - errno.h, 320
- ELIBMAX
 - errno.h, 321
- ELIBSCN
 - errno.h, 321
- ELNRNG
 - errno.h, 321
- ELOOP
 - errno.h, 321
- EMEDIUMTYPE
 - errno.h, 321
- EMFILE
 - errno.h, 321
- EMLINK
 - errno.h, 322
- EMSGSIZE
 - errno.h, 322
- EMULTIHOP
 - errno.h, 322
- enable_cursor
 - term_context, 125
- enable_interrupts
 - cpu.h, 257
- ENAMETOOLONG
 - errno.h, 322
- ENAVAIL
 - errno.h, 322
- ENETDOWN
 - errno.h, 322
- ENETRESET
 - errno.h, 323
- ENETUNREACH
 - errno.h, 323
- ENFILE
 - errno.h, 323
- ENOANO
 - errno.h, 323
- ENOBUFS
 - errno.h, 323
- ENOCSS
 - errno.h, 323
- ENODATA
 - errno.h, 324
- ENODEV
 - errno.h, 324
- ENOENT
 - errno.h, 324
- ENOEXEC
 - errno.h, 324
- ENOKEY
 - errno.h, 324
- ENOLCK
 - errno.h, 324
- ENOLINK
 - errno.h, 325
- ENOMEDIUM
 - errno.h, 325
- ENOMEM
 - errno.h, 325
- ENOMSG
 - errno.h, 325
- ENONET
 - errno.h, 325
- ENOPKG
 - errno.h, 325
- ENOPROTOOPT
 - errno.h, 326
- ENOSPC
 - errno.h, 326
- ENOSR
 - errno.h, 326
- ENOSTR
 - errno.h, 326
- ENOSYS
 - errno.h, 326
- ENOTBLK
 - errno.h, 326
- ENOTCONN
 - errno.h, 327
- ENOTDIR
 - errno.h, 327
- ENOTEMPTY
 - errno.h, 327
- ENOTNAM
 - errno.h, 327
- ENOTRECOVERABLE
 - errno.h, 327
- ENOTSOCK
 - errno.h, 327
- ENOTSUP
 - errno.h, 328
- ENOTTY
 - errno.h, 328
- ENOTUNIQ

- errno.h, 328
- ensure_hhdm_cover_phys_page
 - paging_bootstrap.c, 681
- ensure_hhdm_cover_phys_range
 - paging_bootstrap.c, 682
- entry_count
 - limine_memmap_response, 88
- ENXIO
 - errno.h, 328
- EOPNOTSUPP
 - errno.h, 328
- EOVERFLOW
 - errno.h, 328
- EOWNERDEAD
 - errno.h, 329
- EPERM
 - errno.h, 329
- EPFNOSUPPORT
 - errno.h, 329
- EPIPE
 - errno.h, 329
- EPROTO
 - errno.h, 329
- EPROTONOSUPPORT
 - errno.h, 329
- EPROTOTYPE
 - errno.h, 330
- ERANGE
 - errno.h, 330
- EREMCHG
 - errno.h, 330
- EREMOTE
 - errno.h, 330
- EREMOTEIO
 - errno.h, 330
- ERESTART
 - errno.h, 330
- ERFKILL
 - errno.h, 331
- EROFS
 - errno.h, 331
- err
 - cpu_ctx, 26
- errno.h
 - E2BIG, 311
 - EACCES, 312
 - EADDRINUSE, 312
 - EADDRNOTAVAIL, 312
 - EADV, 312
 - EAFNOSUPPORT, 312
 - EAGAIN, 312
 - EALREADY, 313
 - EBADE, 313
 - EBADF, 313
 - EBADFD, 313
 - EBADMSG, 313
 - EBADR, 313
 - EBADRQC, 314
 - EBADSLT, 314
 - EBFONT, 314
 - EBUSY, 314
 - ECANCELED, 314
 - ECHILD, 314
 - ECHRNG, 315
 - ECOMM, 315
 - ECONNABORTED, 315
 - ECONNREFUSED, 315
 - ECONNRESET, 315
 - EDEADLK, 315
 - EDEADLOCK, 316
 - EDESTADDRREQ, 316
 - EDOM, 316
 - EDOTDOT, 316
 - EDQUOT, 316
 - EEXIST, 316
 - EFAULT, 317
 - EFBIG, 317
 - EHOSTDOWN, 317
 - EHOSTUNREACH, 317
 - EHWPISON, 317
 - EIDRM, 317
 - EILSEQ, 318
 - EINPROGRESS, 318
 - EINTR, 318
 - EINVAL, 318
 - EIO, 318
 - EISCONN, 318
 - EISDIR, 319
 - EISNAM, 319
 - EKEYEXPIRED, 319
 - EKEYREJECTED, 319
 - EKEYREVOKED, 319
 - EL2HLT, 319
 - EL2NSYNC, 320
 - EL3HLT, 320
 - EL3RST, 320
 - ELIBACC, 320
 - ELIBBAD, 320
 - ELIBEXEC, 320
 - ELIBMAX, 321
 - ELIBSCN, 321
 - ELNRNG, 321
 - ELOOP, 321
 - EMEDIUMTYPE, 321
 - EMFILE, 321
 - EMLINK, 322
 - EMSGSIZE, 322
 - EMULTIHOP, 322
 - ENAMETOOLONG, 322
 - ENAVAIL, 322
 - ENETDOWN, 322
 - ENETRESET, 323
 - ENETUNREACH, 323
 - ENFILE, 323
 - ENOANO, 323
 - ENOBUFFS, 323

ENOCSS, 323
ENODATA, 324
ENODEV, 324
ENOENT, 324
ENOEXEC, 324
ENOKEY, 324
ENOLCK, 324
ENOLINK, 325
ENOMEDIUM, 325
ENOMEM, 325
ENOMSG, 325
ENONET, 325
ENOPKG, 325
ENOPROTOOPT, 326
ENOSPC, 326
ENOSR, 326
ENOSTR, 326
ENOSYS, 326
ENOTBLK, 326
ENOTCONN, 327
ENOTDIR, 327
ENOTEMPTY, 327
ENOTNAM, 327
ENOTRECOVERABLE, 327
ENOTSOCK, 327
ENOTSUP, 328
ENOTTY, 328
ENOTUNIQ, 328
ENXIO, 328
EOPNOTSUPP, 328
EOVERFLOW, 328
EOWNERDEAD, 329
EPERM, 329
EPFNOSUPPORT, 329
EPIPE, 329
EPROTO, 329
EPROTONOSUPPORT, 329
EPROTOTYPE, 330
ERANGE, 330
EREMCHG, 330
EREMOTE, 330
EREMOTEIO, 330
ERESTART, 330
ERFKILL, 331
EROFS, 331
ESHUTDOWN, 331
ESOCKTNOSUPPORT, 331
ESPIPE, 331
ESRCH, 331
ESRMNT, 332
ESTALE, 332
ESTRPIPE, 332
ETIME, 332
ETIMEDOUT, 332
ETOOMANYREFS, 332
ETXTBSY, 333
EUCLEAN, 333
EUNATCH, 333
EUSERS, 333
EWOULDBLOCK, 333
EXDEV, 333
EXFULL, 334
error_code
 isr_xframe_t, 54
es
 cpu_ctx, 26
esc_values
 term_context, 126
esc_values_i
 term_context, 126
escape
 term_context, 126
escape_offset
 term_context, 126
escape_parse
 term.c, 852
ESHUTDOWN
 errno.h, 331
ESOCKTNOSUPPORT
 errno.h, 331
ESPIPE
 errno.h, 331
ESRCH
 errno.h, 331
ESRMNT
 errno.h, 332
ESTALE
 errno.h, 332
ESTRPIPE
 errno.h, 332
ETIME
 errno.h, 332
ETIMEDOUT
 errno.h, 332
ETOOMANYREFS
 errno.h, 332
ETXTBSY
 errno.h, 333
EUCLEAN
 errno.h, 333
EUNATCH
 errno.h, 333
EUSERS
 errno.h, 333
EWOULDBLOCK
 errno.h, 333
exception_messages
 interrupts.c, 554
EXDEV
 errno.h, 333
Execute
 gdt.h, 340
EXFULL
 errno.h, 334
ext_checksum
 rsdp, 115

- fbr_req
 - kernel.c, 571
- fbterm_char, 32
 - bg, 32
 - c, 32
 - fg, 33
- fbterm_clear
 - framebuffer.c, 818
- fbterm_context, 33
 - ansi_bright_colours, 35
 - ansi_colours, 35
 - bpp, 35
 - canvas, 35
 - canvas_size, 35
 - cursor_status, 35
 - cursor_x, 36
 - cursor_y, 36
 - default_bg, 36
 - default_fg, 36
 - font_bits, 36
 - font_bits_size, 36
 - font_bool, 37
 - font_bool_size, 37
 - font_height, 37
 - font_scale_x, 37
 - font_scale_y, 37
 - font_width, 37
 - framebuffer, 38
 - glyph_height, 38
 - glyph_width, 38
 - grid, 38
 - grid_size, 38
 - height, 38
 - map, 39
 - map_size, 39
 - offset_x, 39
 - offset_y, 39
 - old_cursor_x, 39
 - old_cursor_y, 39
 - pitch, 40
 - queue, 40
 - queue_i, 40
 - queue_size, 40
 - saved_state_cursor_x, 40
 - saved_state_cursor_y, 40
 - saved_state_text_bg, 41
 - saved_state_text_fg, 41
 - term, 41
 - text_bg, 41
 - text_fg, 41
 - width, 41
- fbterm_deinit
 - framebuffer.c, 819
- fbterm_disable_cursor
 - framebuffer.c, 819
- fbterm_double_buffer_flush
 - framebuffer.c, 819
- fbterm_enable_cursor
 - framebuffer.c, 820
- FBTERM_FONT_GLYPHS
 - framebuffer.h, 513
- fbterm_full_refresh
 - framebuffer.c, 820
- fbterm_get_cursor_pos
 - framebuffer.c, 821
- fbterm_init
 - framebuffer.c, 821
 - framebuffer.h, 513
- fbterm_move_character
 - framebuffer.c, 824
- fbterm_queue_item, 42
 - c, 42
 - x, 43
 - y, 43
- fbterm_raw_putchar
 - framebuffer.c, 824
- fbterm_restore_state
 - framebuffer.c, 825
- fbterm_revscroll
 - framebuffer.c, 825
- fbterm_save_state
 - framebuffer.c, 826
- fbterm_scroll
 - framebuffer.c, 826
- fbterm_set_cursor_pos
 - framebuffer.c, 827
- fbterm_set_text_bg
 - framebuffer.c, 827
- fbterm_set_text_bg_bright
 - framebuffer.c, 828
- fbterm_set_text_bg_default
 - framebuffer.c, 828
- fbterm_set_text_bg_rgb
 - framebuffer.c, 829
- fbterm_set_text_fg
 - framebuffer.c, 829
- fbterm_set_text_fg_bright
 - framebuffer.c, 830
- fbterm_set_text_fg_default
 - framebuffer.c, 830
- fbterm_set_text_fg_rgb
 - framebuffer.c, 831
- fbterm_swap_palette
 - framebuffer.c, 831
- fct
 - out_fct_wrap_type, 111
- fctprintf
 - printf.c, 744
 - printf.h, 462
- fg
 - fbterm_char, 33
- fill_slot
 - memtest.c, 620
- fired
 - timer, 133
- fl_find_fit

- kmalloc.c, 599
- fl_push
 - kmalloc.c, 600
- fl_remove
 - kmalloc.c, 600
- flags
 - limine_smp_request, 95
 - madt_iso, 105
 - madt_lapic, 106
 - madt_nmi, 107
 - TSS_Entry, 139
- flags2
 - TSS_Entry, 139
- FLAGS_ADAPT_EXP
 - printf.c, 720
- FLAGS_CHAR
 - printf.c, 721
- FLAGS_HASH
 - printf.c, 721
- FLAGS_LEFT
 - printf.c, 721
- FLAGS_LONG
 - printf.c, 721
- FLAGS_LONG_LONG
 - printf.c, 721
- FLAGS_PLUS
 - printf.c, 721
- FLAGS_PRECISION
 - printf.c, 722
- FLAGS_SHORT
 - printf.c, 722
- FLAGS_SPACE
 - printf.c, 722
- FLAGS_UPPERCASE
 - printf.c, 722
- FLAGS_ZEROPAD
 - printf.c, 722
- flush_serial_line
 - putchar.c, 772
- font_bits
 - fbterm_context, 36
- font_bits_size
 - fbterm_context, 36
- font_bool
 - fbterm_context, 37
- font_bool_size
 - fbterm_context, 37
- FONT_BYTES
 - framebuffer.c, 817
- font_height
 - fbterm_context, 37
- font_scale_x
 - fbterm_context, 37
- font_scale_y
 - fbterm_context, 37
- font_width
 - fbterm_context, 37
- FOOTER_OF
 - kmalloc.c, 601
- footer_t
 - kmalloc.c, 597
- fpu_allowed
 - kswitches, 57
- fpu_bank_size
 - cpu.c, 167
 - cpu.h, 266
- fpu_init
 - cpuUtils.c, 192
 - cpuUtils.h, 292
- fpu_rest
 - cpu.c, 167
 - cpu.h, 266
- fpu_save
 - cpu.c, 168
 - cpu.h, 267
- fragmentation_test
 - memtest.c, 620
- frame_addr_of
 - neo_framealloc.c, 648
- frame_alloc
 - neo_framealloc.c, 648
 - neo_framealloc.h, 426
- frame_free_count
 - neo_framealloc.c, 649
 - neo_framealloc.h, 427
- frame_index_of
 - neo_framealloc.c, 650
- frame_init
 - neo_framealloc.c, 650
 - neo_framealloc.h, 427
- frame_is_allocated
 - neo_framealloc.c, 651
 - neo_framealloc.h, 428
- frame_supplier.c
 - g_boot_phys_page, 218
- frame_supplier.h
 - FRAME_SUPPLIER_H, 423
 - g_boot_phys_page, 424
 - PagingGetFreeFrame, 423
 - phys_page_supplier_t, 423
- FRAME_SUPPLIER_H
 - frame_supplier.h, 423
- frame_total_count
 - neo_framealloc.c, 652
 - neo_framealloc.h, 429
- framebuffer
 - fbterm_context, 38
- framebuffer.c
 - builtin_font, 835
 - compare_char, 817
 - draw_cursor, 817
 - fbterm_clear, 818
 - fbterm_deinit, 819
 - fbterm_disable_cursor, 819
 - fbterm_double_buffer_flush, 819
 - fbterm_enable_cursor, 820

- fbterm_full_refresh, 820
- fbterm_get_cursor_pos, 821
- fbterm_init, 821
- fbterm_move_character, 824
- fbterm_raw_putchar, 824
- fbterm_restore_state, 825
- fbterm_revscroll, 825
- fbterm_save_state, 826
- fbterm_scroll, 826
- fbterm_set_cursor_pos, 827
- fbterm_set_text_bg, 827
- fbterm_set_text_bg_bright, 828
- fbterm_set_text_bg_default, 828
- fbterm_set_text_bg_rgb, 829
- fbterm_set_text_fg, 829
- fbterm_set_text_fg_bright, 830
- fbterm_set_text_fg_default, 830
- fbterm_set_text_fg_rgb, 831
- fbterm_swap_palette, 831
- FONT_BYTES, 817
- memcpy, 832
- memset, 832
- plot_char, 833
- plot_char_fast, 833
- push_to_queue, 834
- framebuffer.h
 - FBTERM_FONT_GLYPHS, 513
 - fbterm_init, 513
- framebuffer_bytes
 - paging_bootstrap.c, 699
- framebuffer_count
 - limine_framebuffer_response, 79
- framebuffer_phys
 - paging_bootstrap.c, 699
- frees
 - memtest_stats, 109
- FTR_SIZE
 - kmalloc.c, 594
- full_refresh
 - term_context, 126
- fxrstor
 - cpu.h, 257
- fxsave
 - cpu.h, 257
- g_alloc_bm
 - neo_framealloc.c, 657
- g_alloc_bm_bytes
 - paging_bootstrap.c, 699
- g_alloc_bm_phys_base
 - paging_bootstrap.c, 699
- g_base
 - early_alloc.c, 214
- g_basic_sizes
 - memtest.c, 633
- g_boot_phys_page
 - frame_supplier.c, 218
 - frame_supplier.h, 424
- g_cursor
 - early_alloc.c, 215
- g_end
 - early_alloc.c, 215
- g_frame_count
 - neo_framealloc.c, 657
- g_free_count
 - neo_framealloc.c, 657
- g_free_list
 - kmalloc.c, 610
- g_heap_curr
 - kmalloc.c, 611
- g_heap_lock
 - kmalloc.c, 611
- g_hhdm
 - paging_bootstrap.c, 699
- g_hint
 - neo_framealloc.c, 658
- g_max_phys
 - neo_framealloc.c, 658
- g_memlock
 - neo_framealloc.c, 658
- g_pml4_phys
 - paging_bootstrap.c, 700
- g_pml4_virt
 - paging_bootstrap.c, 700
- g_pmm_frames
 - memtest.c, 633
- g_region_forced
 - early_alloc.c, 215
- g_select
 - term_context, 126
- g_slots
 - memtest.c, 633
- g_usable_bm
 - neo_framealloc.c, 658
- g_usable_bm_bytes
 - paging_bootstrap.c, 700
- g_usable_bm_phys_base
 - paging_bootstrap.c, 700
- GDT, 43
 - kernelCS, 44
 - kernelData, 44
 - null, 44
 - seg_16_code, 44
 - seg_16_data, 45
 - seg_32_code, 45
 - seg_32_data, 45
 - tss, 45
 - userCode, 45
 - userData, 45
 - userNull, 46
- gdt
 - gdt.c, 222
- gdt.c
 - desc, 222
 - gdt, 222
 - gdt_load_tss, 220
 - gdt_reload, 220

- ist1Stack, 223
- ist2Stack, 223
- LoadGDT_Stage1, 221
- rsp0, 223
- tss, 223
- TssStack, 224
- gdt.h
 - DC, 340
 - Execute, 340
 - gdt_load_tss, 340
 - gdt_reload, 341
 - GDTAccess16Code, 338
 - GDTAccess16Data, 338
 - GDTAccess32Code, 338
 - GDTAccess32Data, 338
 - GDTAccessDPL, 338
 - GDTAccessFlag, 340
 - GDTAccessKernelCode, 339
 - GDTAccessKernelData, 339
 - GDTAccessUserCode, 339
 - GDTAccessUserData, 339
 - GDTKernelBaseSelector, 339
 - GDTTSSSegment, 339
 - GDTUserBaseSelector, 340
 - LoadGDT_Stage1, 341
 - Present, 340
 - ReadWrite, 340
 - rsp0, 342
 - Segments, 340
- GDT_Desc, 46
 - offset, 46
 - size, 46
- GDT_Entry, 47
 - access_flag, 47
 - base_high, 47
 - base_low, 48
 - base_middle, 48
 - limit_flags, 48
 - limit_low, 48
- GDT_Entry_16, 48
 - access_flag, 49
 - base_high, 49
 - base_low, 49
 - base_middle, 49
 - limit_flags, 50
 - limit_low, 50
- GDT_Entry_32, 50
 - access_flag, 51
 - base_high, 51
 - base_low, 51
 - base_middle, 51
 - limit_flags, 51
 - limit_low, 51
- gdt_load_tss
 - gdt.c, 220
 - gdt.h, 340
- gdt_reload
 - gdt.c, 220
 - gdt.h, 341
- GDTAccess16Code
 - gdt.h, 338
- GDTAccess16Data
 - gdt.h, 338
- GDTAccess32Code
 - gdt.h, 338
- GDTAccess32Data
 - gdt.h, 338
- GDTAccessDPL
 - gdt.h, 338
- GDTAccessFlag
 - gdt.h, 340
- GDTAccessKernelCode
 - gdt.h, 339
- GDTAccessKernelData
 - gdt.h, 339
- GDTAccessUserCode
 - gdt.h, 339
- GDTAccessUserData
 - gdt.h, 339
- GDTKernelBaseSelector
 - gdt.h, 339
- GDTTSSSegment
 - gdt.h, 339
- GDTUserBaseSelector
 - gdt.h, 340
- general_registers
 - isr_xframe_t, 54
- get_apic_base_address
 - cpuUtils.c, 193
- get_cursor_pos
 - term_context, 127
- get_fs_base
 - cpu.h, 258
- get_gs_base
 - cpu.h, 258
- get_kernel_gs_base
 - cpu.h, 259
- get_max_phys
 - paging_bootstrap.c, 682
- get_memory_size
 - KernelUtils.c, 578
 - KernelUtils.h, 369
- get_memory_size_gib
 - KernelUtils.c, 579
 - KernelUtils.h, 369
- get_model
 - cpuUtils.c, 193
 - cpuUtils.h, 293
- get_or_make_next
 - paging_bootstrap.c, 682
- get_or_make_pd
 - paging_bootstrap.c, 683
- get_RTC_register
 - time.c, 881
- get_total_usable_mem_bytes
 - paging_bootstrap.c, 684

- get_ts
 - serial.h, 489
- get_update_flag
 - time.c, 881
- get_xsave_size
 - cpuUtils.c, 194
- GetOrAllocEntry
 - paging.c, 664
- GetOrNullifyEntry
 - paging.c, 665
- global_defs.h
 - ALIGN_16BIT, 345
 - ALIGN_4K, 345
 - ASSERT, 345
 - CAS, 345
 - HANDLE, 346
 - lsize_t, 346
 - MAX, 345
 - MIN, 346
 - mode_t, 347
 - PACKED, 346
 - ssize_t, 347
- glyph_height
 - fbterm_context, 38
- glyph_width
 - fbterm_context, 38
- gpt_disk_uuid
 - limine_file, 72
- gpt_part_uuid
 - limine_file, 72
- Green
 - kernel.c, 562
- green_mask_shift
 - limine_framebuffer, 76
- green_mask_size
 - limine_framebuffer, 76
- grid
 - fbterm_context, 38
- grid_size
 - fbterm_context, 38
- gsi
 - madt_iso, 105
- gsib
 - madt_io_apic, 104
- halt
 - acpi.c, 143
 - cpuUtils.c, 194
 - idt.c, 227
 - ioapic.c, 150
 - kernel.c, 564
 - panic.c, 709
 - shell.c, 796
 - stack_trace.c, 803
- HANDLE
 - global_defs.h, 346
- has_ACPI
 - cpuUtils.c, 198
 - cpuUtils.h, 294
- hash
 - hashmap.h, 377
- hashmap.h
 - _HASHMAP_H, 373
 - hash, 377
 - HASHMAP_DELETE, 373
 - HASHMAP_GET, 374
 - HASHMAP_INIT, 374
 - HASHMAP_INSERT, 374
 - HASHMAP_KEY_DATA_MAX, 375
 - HASHMAP_REMOVE, 375
 - HASHMAP_SGET, 375
 - HASHMAP_SINSERT, 375
 - HASHMAP_SREMOVE, 376
 - HASHMAP_TYPE, 376
- HASHMAP_DELETE
 - hashmap.h, 373
- HASHMAP_GET
 - hashmap.h, 374
- HASHMAP_INIT
 - hashmap.h, 374
- HASHMAP_INSERT
 - hashmap.h, 374
- HASHMAP_KEY_DATA_MAX
 - hashmap.h, 375
- HASHMAP_REMOVE
 - hashmap.h, 375
- HASHMAP_SGET
 - hashmap.h, 375
- HASHMAP_SINSERT
 - hashmap.h, 375
- HASHMAP_SREMOVE
 - hashmap.h, 376
- HASHMAP_TYPE
 - hashmap.h, 376
- HDR_SIZE
 - kmalloc.c, 594
- HEAP_BASE
 - kmalloc.c, 594
- heap_growth_test
 - memtest.c, 621
- HEAP_LIMIT
 - kmalloc.c, 594
- heap_map_more
 - kmalloc.c, 601
- height
 - fbterm_context, 38
 - limine_framebuffer, 76
- hello_general_floatius
 - kernel.c, 565
- hhdn_request
 - kernel.c, 571
 - memtest.c, 634
 - neo_framealloc.c, 658
 - paging_bootstrap.c, 700
- HIGHER_HALF_KERNEL_MEMORY_OFFSET
 - paging.h, 434
- HIGHER_HALF_MEMORY_OFFSET

- paging.h, 434
- hour
 - time.c, 892
 - time.h, 539
- hw_rng_support
 - kswitches, 57
- id
 - limine_5_level_paging_request, 59
 - limine_boot_time_request, 61
 - limine_bootloader_info_request, 63
 - limine_dtb_request, 65
 - limine_efi_system_table_request, 67
 - limine_entry_point_request, 69
 - limine_framebuffer_request, 78
 - limine_hhdm_request, 80
 - limine_kernel_address_request, 82
 - limine_kernel_file_request, 84
 - limine_mmap_request, 87
 - limine_module_request, 89
 - limine_rsdp_request, 91
 - limine_smbios_request, 93
 - limine_smp_request, 95
 - limine_stack_size_request, 96
 - limine_terminal_request, 100
 - madt_header, 103
- idle_thread
 - cpu_local, 31
- idt
 - idt.c, 233
- idt.c
 - halt, 227
 - idt, 233
 - idt_allocate_vector, 227
 - idt_free_vector, 228
 - idt_init, 229
 - idt_reg_test, 229
 - idt_reload, 230
 - idt_set_descriptor, 230
 - idtr, 234
 - isr_delta, 234
 - isr_stub_table, 234
 - test_handler, 231
 - vectors, 234
 - yield_isr_test, 232
 - yield_register, 233
 - YIELD_VECTOR, 227
- idt.h
 - idt_allocate_vector, 352
 - IDT_CPU_EXCEPTION_COUNT, 349
 - IDT_DESCRIPTOR_CALL, 350
 - IDT_DESCRIPTOR_EXCEPTION, 350
 - IDT_DESCRIPTOR_EXTERNAL, 350
 - IDT_DESCRIPTOR_PRESENT, 350
 - IDT_DESCRIPTOR_RING1, 350
 - IDT_DESCRIPTOR_RING2, 351
 - IDT_DESCRIPTOR_RING3, 351
 - IDT_DESCRIPTOR_X16_INTERRUPT, 351
 - IDT_DESCRIPTOR_X16_TRAP, 351
 - IDT_DESCRIPTOR_X32_INTERRUPT, 351
 - IDT_DESCRIPTOR_X32_TASK, 351
 - IDT_DESCRIPTOR_X32_TRAP, 352
 - idt_free_vector
 - idt.c, 228
 - idt.h, 353
 - IDT_HDW_INTERRUPT_COUNT
 - idt.h, 352
 - idt_init
 - idt.c, 229
 - idt.h, 354
 - IDT_MAX_DESCRIPTOR
 - idt.h, 352
 - idt_reg_test
 - idt.c, 229
 - idt.h, 354
 - idt_reload
 - idt.c, 230
 - idt.h, 355
 - IDT_DESCRIPTOR_X32_INTERRUPT, 351
 - IDT_DESCRIPTOR_X32_TASK, 351
 - IDT_DESCRIPTOR_X32_TRAP, 352
 - idt_free_vector, 353
 - IDT_HDW_INTERRUPT_COUNT, 352
 - idt_init, 354
 - IDT_MAX_DESCRIPTOR, 352
 - idt_reg_test, 354
 - idt_reload, 355
 - idt_set_descriptor, 355
 - isr_delta, 357
 - isr_stub_table, 357
 - yield_register, 356
- idt_allocate_vector
 - idt.c, 227
 - idt.h, 352
- IDT_CPU_EXCEPTION_COUNT
 - idt.h, 349
- IDT_DESCRIPTOR_CALL
 - idt.h, 350
- IDT_DESCRIPTOR_EXCEPTION
 - idt.h, 350
- IDT_DESCRIPTOR_EXTERNAL
 - idt.h, 350
- IDT_DESCRIPTOR_PRESENT
 - idt.h, 350
- IDT_DESCRIPTOR_RING1
 - idt.h, 350
- IDT_DESCRIPTOR_RING2
 - idt.h, 351
- IDT_DESCRIPTOR_RING3
 - idt.h, 351
- IDT_DESCRIPTOR_X16_INTERRUPT
 - idt.h, 351
- IDT_DESCRIPTOR_X16_TRAP
 - idt.h, 351
- IDT_DESCRIPTOR_X32_INTERRUPT
 - idt.h, 351
- IDT_DESCRIPTOR_X32_TASK
 - idt.h, 351
- IDT_DESCRIPTOR_X32_TRAP
 - idt.h, 352
- idt_free_vector
 - idt.c, 228
 - idt.h, 353
- IDT_HDW_INTERRUPT_COUNT
 - idt.h, 352
- idt_init
 - idt.c, 229
 - idt.h, 354
- IDT_MAX_DESCRIPTOR
 - idt.h, 352
- idt_reg_test
 - idt.c, 229
 - idt.h, 354
- idt_reload
 - idt.c, 230
 - idt.h, 355

- idt_set_descriptor
 - idt.c, 230
 - idt.h, 355
- idtr
 - idt.c, 234
- in_bootloader
 - term_context, 127
- inb
 - io.c, 556
 - io.h, 361
- index
 - timer, 133
- init_done
 - kernel.c, 571
 - kernel.h, 365
- init_PIT
 - time.c, 882
 - time.h, 528
- init_scheduler
 - sched.c, 785
 - sched.h, 483
- insert_mode
 - term_context, 127
- install_new_cr3
 - paging_bootstrap.c, 684
- int2srt.c
 - int_to_str, 550
 - u64_to_str, 550
- int_to_str
 - int2srt.c, 550
 - string.h, 506
- interrupt_state
 - cpu.h, 259
- interrupt_toggle
 - cpu.h, 260
- Interrupts and Descriptor Tables, 16
- interrupts.c
 - dyn_isr_handler, 552
 - exception_messages, 554
 - irq_handler, 553
 - isr_exception_handler, 553
- io.c
 - inb, 556
 - io_wait, 558
 - outb, 558
- io.h
 - _IO_H, 361
 - inb, 361
 - io_wait, 362
 - outb, 362
- io_apic_from_gsi
 - ioapic.c, 150
- io_apic_gsi_count
 - ioapic.c, 151
- io_apic_read
 - ioapic.c, 151
- io_apic_set_gsi_redirect
 - ioapic.c, 151
- ioapic.h, 245
- io_apic_set_irq_redirect
 - ioapic.c, 152
 - ioapic.h, 245
- io_apic_write
 - ioapic.c, 153
- io_wait
 - io.c, 558
 - io.h, 362
- ioapic.c
 - halt, 150
 - io_apic_from_gsi, 150
 - io_apic_gsi_count, 151
 - io_apic_read, 151
 - io_apic_set_gsi_redirect, 151
 - io_apic_set_irq_redirect, 152
 - io_apic_write, 153
- ioapic.h
 - io_apic_set_gsi_redirect, 245
 - io_apic_set_irq_redirect, 245
- iopbOffset
 - TSS, 135
- irq_handler
 - interrupts.c, 553
- irq_source
 - madt_iso, 105
- IS_ALLOC
 - kmalloc.c, 595
- IsHigherHalf
 - paging.h, 435
- IsKernelHigherHalf
 - paging.h, 435
- isr_delta
 - idt.c, 234
 - idt.h, 357
- isr_exception_handler
 - interrupts.c, 553
- isr_stub_table
 - idt.c, 234
 - idt.h, 357
- isr_xframe_t, 52
 - base_frame, 53
 - control_registers, 53
 - cr0, 53
 - cr2, 53
 - cr3, 53
 - cr4, 54
 - cs, 54
 - dss, 54
 - error_code, 54
 - general_registers, 54
 - rax, 54
 - rbp, 55
 - rbx, 55
 - rcx, 55
 - rdi, 55
 - rdx, 55
 - rflags, 55

- rip, 56
- rsi, 56
- rsp, 56
- vector, 56
- ist
 - __attribute__, 22
- ist1
 - TSS, 135
- ist1Stack
 - gdt.c, 223
- ist2
 - TSS, 135
- ist2Stack
 - gdt.c, 223
- ist3
 - TSS, 136
- ist4
 - TSS, 136
- ist5
 - TSS, 136
- ist6
 - TSS, 136
- ist7
 - TSS, 136
- k_char
 - keyboard.c, 207
- k_getchar
 - keyboard.c, 204
 - keyboard.h, 297
- k_mode
 - KernelUtils.c, 580
 - KernelUtils.h, 371
- Kaddress_req
 - kernel.c, 571
 - neo_framealloc.c, 658
 - paging_bootstrap.c, 700
- kbd_head
 - keyboard.c, 207
- kbd_pop
 - keyboard.c, 204
 - keyboard.h, 297
- kbd_push
 - keyboard.c, 204
- kbd_stack
 - keyboard.c, 207
- KBD_STACK_SIZE
 - keyboard.c, 203
- kbd_tail
 - keyboard.c, 207
- Kernel Heap, 15
- kernel.c
 - _start, 563
 - Black, 561
 - Blue, 561
 - bootspace, 571
 - breakpoint, 564
 - Cyan, 562
 - early_term, 571
 - fbr_req, 571
 - Green, 562
 - halt, 564
 - hello_general_floatius, 565
 - hhdm_request, 571
 - init_done, 571
 - Kaddress_req, 571
 - KERNEL_VERSION, 562
 - kerror_mode, 572
 - limine_debug, 565
 - memmap_req, 572
 - memory_bringup, 566
 - proc_a, 566
 - proc_b, 567
 - Purple, 562
 - Red, 562
 - scheduler_heartbeat, 567
 - start_interrupts, 568
 - stop_interrupts, 568
 - supply_from_early, 569
 - supply_from_frame, 569
 - task_switch_int, 570
 - term_bg, 572
 - term_context, 572
 - term_fg, 572
 - walk_stack, 570
 - White, 562
 - Yellow, 563
- kernel.h
 - _KERNEL_H, 365
 - bootspace, 365
 - early_term, 365
 - init_done, 365
 - kerror_mode, 366
- KERNEL_VERSION
 - consts.h, 252
 - kernel.c, 562
- kernelCS
 - GDT, 44
- kernelData
 - GDT, 44
- KernelUtils.c
 - bytes_to_gib, 577
 - bytes_to_mib, 578
 - get_memory_size, 578
 - get_memory_size_gib, 579
 - k_mode, 580
 - memmap_req, 581
 - print_memmap, 580
- KernelUtils.h
 - _KERNEL_UTILS_H, 368
 - bytes_to_gib, 368
 - bytes_to_mib, 368
 - get_memory_size, 369
 - get_memory_size_gib, 369
 - k_mode, 371
 - print_memmap, 370
 - restore_floats, 371

- save_floats, 371
 - term_context, 371
- kerror_mode
 - kernel.c, 572
 - kernel.h, 366
- keyboard.c
 - k_char, 207
 - k_getchar, 204
 - kbd_head, 207
 - kbd_pop, 204
 - kbd_push, 204
 - kbd_stack, 207
 - KBD_STACK_SIZE, 203
 - kbd_tail, 207
 - keyboard_handler, 205
 - keyboard_init, 206
- keyboard.h
 - k_getchar, 297
 - kbd_pop, 297
 - KEYBOARD_DATA_PORT, 296
 - keyboard_handler, 298
 - keyboard_init, 299
 - KEYBOARD_STATUS_PORT, 297
- keyboard_charmap
 - keyboard_map.h, 301
- KEYBOARD_DATA_PORT
 - keyboard.h, 296
- keyboard_handler
 - keyboard.c, 205
 - keyboard.h, 298
- keyboard_init
 - keyboard.c, 206
 - keyboard.h, 299
- keyboard_map
 - keyboard_map.h, 301
- keyboard_map.h
 - keyboard_charmap, 301
 - keyboard_map, 301
- KEYBOARD_STATUS_PORT
 - keyboard.h, 297
- kfree
 - kmalloc.c, 602
 - kmalloc.h, 413
- kheap.h
 - KHEAP_H, 410
 - kheap_init, 411
 - kheap_reserve_pages, 411
- KHEAP_H
 - kheap.h, 410
- kheap_init
 - kheap.h, 411
- kheap_reserve_pages
 - kheap.h, 411
- kmalloc
 - kmalloc.c, 603
 - kmalloc.h, 414
- kmalloc.c
 - ALIGN_DOWN, 593
- ALIGN_UP, 593
- ALIGNMENT, 593
- ALLOC_FLAG, 593
- allocate_from_block, 597
- BLK_SIZE, 594
- block_t, 596
- CLR_ALLOC, 594
- coalesce, 598
- current_pml4_phys, 598
- fl_find_fit, 599
- fl_push, 600
- fl_remove, 600
- FOOTER_OF, 601
- footer_t, 597
- FTR_SIZE, 594
- g_free_list, 610
- g_heap_curr, 611
- g_heap_lock, 611
- HDR_SIZE, 594
- HEAP_BASE, 594
- HEAP_LIMIT, 594
- heap_map_more, 601
- IS_ALLOC, 595
- kfree, 602
- kmalloc, 603
- kmalloc_dump, 604
- kmalloc_init, 605
- kmalloc_usable_size, 606
- krealloc, 606
- MIN_SPLIT_SIZE, 595
- NEXT_BLOCK, 607
- OVERHEAD, 595
- PAGE_SIZE, 595
- PAGING_FLAG_NO_EXECUTE, 595
- PAGING_FLAG_PRESENT, 595
- PAGING_FLAG_WRITABLE, 596
- panic, 607
- PREV_BLOCK, 609
- ReadCR3, 609
- SET_ALLOC, 596
- SET_SIZE, 596
- total_size_for_payload, 610
- USABLE, 596
- kmalloc.h
 - kfree, 413
 - kmalloc, 414
 - kmalloc_dump, 415
 - KMALLOC_H, 413
 - kmalloc_init, 416
 - kmalloc_usable_size, 416
 - krealloc, 417
- kmalloc_dump
 - kmalloc.c, 604
 - kmalloc.h, 415
- KMALLOC_H
 - kmalloc.h, 413
- kmalloc_init
 - kmalloc.c, 605

- kmalloc.h, 416
- kmalloc_usable_size
 - kmalloc.c, 606
 - kmalloc.h, 416
- krealloc
 - kmalloc.c, 606
 - kmalloc.h, 417
- kswitches, 56
 - acpi_support, 57
 - addr_debug, 57
 - fpu_allowed, 57
 - hw_rng_support, 57
 - liballoc_debug, 58
 - mem_readout_unit, 58
 - sched_debug, 58
 - stack_trace_on_fault, 58
 - stack_trace_size, 58
 - timestamp, 58
- lapic.c
 - lapic_eoi, 158
 - LAPIC_EOI_ACK, 156
 - lapic_get_id, 158
 - lapic_init, 159
 - lapic_read, 159
 - LAPIC_REG_CMCI, 156
 - LAPIC_REG_EOI, 156
 - LAPIC_REG_ICR0, 157
 - LAPIC_REG_ICR1, 157
 - LAPIC_REG_ID, 157
 - LAPIC_REG_LVT_TIMER, 157
 - LAPIC_REG_SPURIOUS, 157
 - LAPIC_REG_TIMER_CURCNT, 157
 - LAPIC_REG_TIMER_DIV, 158
 - LAPIC_REG_TIMER_INITCNT, 158
 - lapic_timer_calibrate, 160
 - lapic_timer_handler, 161
 - lapic_timer_oneshot, 162
 - lapic_timer_stop, 162
 - lapic_write, 163
 - timer_vec, 163
- lapic.h
 - lapic_eoi, 248
 - lapic_get_id, 248
 - lapic_init, 249
 - lapic_send_ipi, 249
 - lapic_timer_calibrate, 250
 - lapic_timer_oneshot, 250
- lapic_eoi
 - lapic.c, 158
 - lapic.h, 248
- LAPIC_EOI_ACK
 - lapic.c, 156
- lapic_freq
 - cpu_local, 31
- lapic_get_id
 - lapic.c, 158
 - lapic.h, 248
- lapic_id
 - cpu_local, 31
- lapic_init
 - lapic.c, 159
 - lapic.h, 249
- lapic_read
 - lapic.c, 159
- LAPIC_REG_CMCI
 - lapic.c, 156
- LAPIC_REG_EOI
 - lapic.c, 156
- LAPIC_REG_ICR0
 - lapic.c, 157
- LAPIC_REG_ICR1
 - lapic.c, 157
- LAPIC_REG_ID
 - lapic.c, 157
- LAPIC_REG_LVT_TIMER
 - lapic.c, 157
- LAPIC_REG_SPURIOUS
 - lapic.c, 157
- LAPIC_REG_TIMER_CURCNT
 - lapic.c, 157
- LAPIC_REG_TIMER_DIV
 - lapic.c, 158
- LAPIC_REG_TIMER_INITCNT
 - lapic.c, 158
- lapic_send_ipi
 - lapic.h, 249
- lapic_timer_calibrate
 - lapic.c, 160
 - lapic.h, 250
- lapic_timer_handler
 - lapic.c, 161
- lapic_timer_oneshot
 - lapic.c, 162
 - lapic.h, 250
- lapic_timer_stop
 - lapic.c, 162
- lapic_write
 - lapic.c, 163
- last_run_queue_index
 - cpu_local, 31
- length
 - limine_mmap_entry, 86
 - madt_header, 103
 - rsdp, 115
 - sdt, 118
 - TSS_Entry, 139
- liballoc_debug
 - kswitches, 58
- limine.h
 - LIMINE_5_LEVEL_PAGING_REQUEST, 390
 - LIMINE_BOOT_TIME_REQUEST, 390
 - LIMINE_BOOTLOADER_INFO_REQUEST, 390
 - LIMINE_COMMON_MAGIC, 391
 - LIMINE_DTB_REQUEST, 391
 - LIMINE_EFI_SYSTEM_TABLE_REQUEST, 391
 - limine_entry_point, 398

- LIMINE_ENTRY_POINT_REQUEST, 391
- LIMINE_FRAMEBUFFER_REQUEST, 391
- LIMINE_FRAMEBUFFER_RGB, 392
- limine_goto_address, 398
- LIMINE_HHDM_REQUEST, 392
- LIMINE_KERNEL_ADDRESS_REQUEST, 392
- LIMINE_KERNEL_FILE_REQUEST, 392
- LIMINE_MEDIA_TYPE_GENERIC, 392
- LIMINE_MEDIA_TYPE_OPTICAL, 393
- LIMINE_MEDIA_TYPE_TFTP, 393
- LIMINE_MEMMAP_ACPI_NVFS, 393
- LIMINE_MEMMAP_ACPI_RECLAIMABLE, 393
- LIMINE_MEMMAP_BAD_MEMORY, 393
- LIMINE_MEMMAP_BOOTLOADER_RECLAIMABLE, 393
- LIMINE_MEMMAP_FRAMEBUFFER, 394
- LIMINE_MEMMAP_KERNEL_AND_MODULES, 394
- LIMINE_MEMMAP_REQUEST, 394
- LIMINE_MEMMAP_RESERVED, 394
- LIMINE_MEMMAP_USABLE, 394
- LIMINE_MODULE_REQUEST, 394
- LIMINE_PTR, 395
- LIMINE_RSDP_REQUEST, 395
- LIMINE_SMBIOS_REQUEST, 395
- LIMINE_SMP_REQUEST, 395
- LIMINE_STACK_SIZE_REQUEST, 395
- limine_terminal_callback, 398
- LIMINE_TERMINAL_CB_BELL, 396
- LIMINE_TERMINAL_CB_DEC, 396
- LIMINE_TERMINAL_CB_KBD_LEDS, 396
- LIMINE_TERMINAL_CB_LINUX, 396
- LIMINE_TERMINAL_CB_MODE, 396
- LIMINE_TERMINAL_CB_POS_REPORT, 396
- LIMINE_TERMINAL_CB_PRIVATE_ID, 397
- LIMINE_TERMINAL_CB_STATUS_REPORT, 397
- LIMINE_TERMINAL_CTX_RESTORE, 397
- LIMINE_TERMINAL_CTX_SAVE, 397
- LIMINE_TERMINAL_CTX_SIZE, 397
- LIMINE_TERMINAL_FULL_REFRESH, 397
- LIMINE_TERMINAL_REQUEST, 398
- limine_terminal_write, 398
- LIMINE_5_LEVEL_PAGING_REQUEST
 - limine.h, 390
- limine_5_level_paging_request, 59
 - id, 59
 - LIMINE_PTR, 59
 - revision, 60
- limine_5_level_paging_response, 60
 - revision, 60
- LIMINE_BOOT_TIME_REQUEST
 - limine.h, 390
- limine_boot_time_request, 61
 - id, 61
 - LIMINE_PTR, 61
 - revision, 61
- limine_boot_time_response, 62
 - boot_time, 62
 - revision, 62
- LIMINE_BOOTLOADER_INFO_REQUEST
 - limine.h, 390
- limine_bootloader_info_request, 62
 - id, 63
 - LIMINE_PTR, 63
 - revision, 63
- limine_bootloader_info_response, 63
 - LIMINE_PTR, 64
 - revision, 64
- LIMINE_COMMON_MAGIC
 - limine.h, 391
- limine_debug
 - kernel.c, 565
- LIMINE_DTB_REQUEST
 - limine.h, 391
- limine_dtb_request, 65
 - id, 65
 - LIMINE_PTR, 65
 - revision, 65
- limine_dtb_response, 66
 - LIMINE_PTR, 66
 - revision, 66
- LIMINE_EFI_SYSTEM_TABLE_REQUEST
 - limine.h, 391
- limine_efi_system_table_request, 66
 - id, 67
 - LIMINE_PTR, 67
 - revision, 67
- limine_efi_system_table_response, 68
 - LIMINE_PTR, 68
 - revision, 68
- limine_entry_point
 - limine.h, 398
- LIMINE_ENTRY_POINT_REQUEST
 - limine.h, 391
- limine_entry_point_request, 68
 - id, 69
 - LIMINE_PTR, 69
 - revision, 69
- limine_entry_point_response, 70
 - revision, 70
- limine_file, 71
 - gpt_disk_uuid, 72
 - gpt_part_uuid, 72
 - LIMINE_PTR, 71, 72
 - mbr_disk_id, 72
 - media_type, 72
 - part_uuid, 73
 - partition_index, 73
 - revision, 73
 - size, 73
 - tftp_ip, 73
 - tftp_port, 73
 - unused, 74
- limine_framebuffer, 74
 - blue_mask_shift, 75
 - blue_mask_size, 75

- bpp, 75
- edid_size, 75
- green_mask_shift, 76
- green_mask_size, 76
- height, 76
- LIMINE_PTR, 75
- memory_model, 76
- pitch, 76
- red_mask_shift, 76
- red_mask_size, 77
- unused, 77
- width, 77
- LIMINE_FRAMEBUFFER_REQUEST
 - limine.h, 391
- limine_framebuffer_request, 77
 - id, 78
 - LIMINE_PTR, 78
 - revision, 78
- limine_framebuffer_response, 78
 - framebuffer_count, 79
 - LIMINE_PTR, 79
 - revision, 79
- LIMINE_FRAMEBUFFER_RGB
 - limine.h, 392
- limine_goto_address
 - limine.h, 398
- LIMINE_HHDM_REQUEST
 - limine.h, 392
- limine_hhdm_request, 79
 - id, 80
 - LIMINE_PTR, 80
 - revision, 80
- limine_hhdm_response, 81
 - offset, 81
 - revision, 81
- LIMINE_KERNEL_ADDRESS_REQUEST
 - limine.h, 392
- limine_kernel_address_request, 81
 - id, 82
 - LIMINE_PTR, 82
 - revision, 82
- limine_kernel_address_response, 82
 - physical_base, 83
 - revision, 83
 - virtual_base, 83
- LIMINE_KERNEL_FILE_REQUEST
 - limine.h, 392
- limine_kernel_file_request, 83
 - id, 84
 - LIMINE_PTR, 84
 - revision, 84
- limine_kernel_file_response, 85
 - LIMINE_PTR, 85
 - revision, 85
- LIMINE_MEDIA_TYPE_GENERIC
 - limine.h, 392
- LIMINE_MEDIA_TYPE_OPTICAL
 - limine.h, 393
- LIMINE_MEDIA_TYPE_TFTP
 - limine.h, 393
- LIMINE_MEMMAP_ACPI_NVS
 - limine.h, 393
- LIMINE_MEMMAP_ACPI_RECLAIMABLE
 - limine.h, 393
- LIMINE_MEMMAP_BAD_MEMORY
 - limine.h, 393
- LIMINE_MEMMAP_BOOTLOADER_RECLAIMABLE
 - limine.h, 393
- limine_memmap_entry, 85
 - base, 86
 - length, 86
 - type, 86
- LIMINE_MEMMAP_FRAMEBUFFER
 - limine.h, 394
- LIMINE_MEMMAP_KERNEL_AND_MODULES
 - limine.h, 394
- LIMINE_MEMMAP_REQUEST
 - limine.h, 394
- limine_memmap_request, 86
 - id, 87
 - LIMINE_PTR, 87
 - revision, 87
- LIMINE_MEMMAP_RESERVED
 - limine.h, 394
- limine_memmap_response, 88
 - entry_count, 88
 - LIMINE_PTR, 88
 - revision, 88
- LIMINE_MEMMAP_USABLE
 - limine.h, 394
- LIMINE_MODULE_REQUEST
 - limine.h, 394
- limine_module_request, 89
 - id, 89
 - LIMINE_PTR, 89
 - revision, 89
- limine_module_response, 90
 - LIMINE_PTR, 90
 - module_count, 90
 - revision, 90
- LIMINE_PTR
 - limine.h, 395
 - limine_5_level_paging_request, 59
 - limine_boot_time_request, 61
 - limine_bootloader_info_request, 63
 - limine_bootloader_info_response, 64
 - limine_dtb_request, 65
 - limine_dtb_response, 66
 - limine_efi_system_table_request, 67
 - limine_efi_system_table_response, 68
 - limine_entry_point_request, 69
 - limine_file, 71, 72
 - limine_framebuffer, 75
 - limine_framebuffer_request, 78
 - limine_framebuffer_response, 79
 - limine_hhdm_request, 80

- limine_kernel_address_request, 82
- limine_kernel_file_request, 84
- limine_kernel_file_response, 85
- limine_memmap_request, 87
- limine_memmap_response, 88
- limine_module_request, 89
- limine_module_response, 90
- limine_rsdp_request, 91
- limine_rsdp_response, 92
- limine_smbios_request, 93
- limine_smbios_response, 94
- limine_smp_request, 95
- limine_stack_size_request, 96
- limine_terminal, 98
- limine_terminal_request, 99
- limine_terminal_response, 101
- limine_requests.c
 - __attribute__, 583
- LIMINE_RSDP_REQUEST
 - limine.h, 395
- limine_rsdp_request, 91
 - id, 91
 - LIMINE_PTR, 91
 - revision, 91
- limine_rsdp_response, 92
 - LIMINE_PTR, 92
 - revision, 92
- LIMINE_SMBIOS_REQUEST
 - limine.h, 395
- limine_smbios_request, 92
 - id, 93
 - LIMINE_PTR, 93
 - revision, 93
- limine_smbios_response, 94
 - LIMINE_PTR, 94
 - revision, 94
- LIMINE_SMP_REQUEST
 - limine.h, 395
- limine_smp_request, 95
 - flags, 95
 - id, 95
 - LIMINE_PTR, 95
 - revision, 95
- LIMINE_STACK_SIZE_REQUEST
 - limine.h, 395
- limine_stack_size_request, 96
 - id, 96
 - LIMINE_PTR, 96
 - revision, 97
 - stack_size, 97
- limine_stack_size_response, 97
 - revision, 97
- limine_terminal, 98
 - columns, 98
 - LIMINE_PTR, 98
 - rows, 99
- limine_terminal_callback
 - limine.h, 398
- LIMINE_TERMINAL_CB_BELL
 - limine.h, 396
- LIMINE_TERMINAL_CB_DEC
 - limine.h, 396
- LIMINE_TERMINAL_CB_KBD_LEDS
 - limine.h, 396
- LIMINE_TERMINAL_CB_LINUX
 - limine.h, 396
- LIMINE_TERMINAL_CB_MODE
 - limine.h, 396
- LIMINE_TERMINAL_CB_POS_REPORT
 - limine.h, 396
- LIMINE_TERMINAL_CB_PRIVATE_ID
 - limine.h, 397
- LIMINE_TERMINAL_CB_STATUS_REPORT
 - limine.h, 397
- LIMINE_TERMINAL_CTX_RESTORE
 - limine.h, 397
- LIMINE_TERMINAL_CTX_SAVE
 - limine.h, 397
- LIMINE_TERMINAL_CTX_SIZE
 - limine.h, 397
- LIMINE_TERMINAL_FULL_REFRESH
 - limine.h, 397
- LIMINE_TERMINAL_REQUEST
 - limine.h, 398
- limine_terminal_request, 99
 - id, 100
 - LIMINE_PTR, 99
 - revision, 100
- limine_terminal_response, 100
 - LIMINE_PTR, 101
 - revision, 101
 - terminal_count, 101
- limine_terminal_write
 - limine.h, 398
- limine_uuid, 101
 - a, 102
 - b, 102
 - c, 102
 - d, 102
- limit
 - __attribute__, 23
- limit_flags
 - GDT_Entry, 48
 - GDT_Entry_16, 50
 - GDT_Entry_32, 51
- limit_low
 - GDT_Entry, 48
 - GDT_Entry_16, 50
 - GDT_Entry_32, 51
- lint
 - madt_nmi, 107
- linux_private_parse
 - term.c, 854
- live_bytes
 - memtest_stats, 109
- LoadGDT_Stage1

- gdt.c, 221
- gdt.h, 341
- lock.h
 - _LOCK_C_H, 406
 - smartlock_acquire, 407
 - SMARTLOCK_INIT, 406
 - smartlock_release, 407
 - spinlock_acquire, 407
 - SPINLOCK_INIT, 406
 - spinlock_release, 408
 - spinlock_t, 407
 - spinlock_test_and_acq, 408
- logbuf
 - putchar.c, 778
- LOGBUF_SIZE
 - putchar.c, 768
- loglen
 - putchar.c, 778
- lsize_t
 - global_defs.h, 346
- madt.c
 - madt_init, 147
 - typeof, 148
- madt.h
 - madt_init, 241
 - PACKED, 242
 - VECTOR_TYPE, 242
- madt_header, 103
 - id, 103
 - length, 103
- madt_init
 - madt.c, 147
 - madt.h, 241
- madt_io_apic, 103
 - address, 104
 - apic_id, 104
 - gsib, 104
 - reserved, 104
- madt_iso, 105
 - bus_source, 105
 - flags, 105
 - gsi, 105
 - irq_source, 105
- madt_lapic, 106
 - apic_id, 106
 - flags, 106
 - processor_id, 106
- madt_nmi, 107
 - flags, 107
 - lint, 107
 - processor, 107
- map
 - fbterm_context, 39
- map_1g
 - paging_bootstrap.c, 684
- map_2m
 - paging_bootstrap.c, 685
- map_4k
 - paging_bootstrap.c, 685
- map_hhdm
 - paging_bootstrap.c, 686
- map_identity_minimal
 - paging_bootstrap.c, 687
- map_kernel_higher_half
 - paging_bootstrap.c, 687
- map_range_4k
 - paging_bootstrap.c, 688
- map_range_huge_first
 - paging_bootstrap.c, 688
- map_size
 - fbterm_context, 39
- mark_range_allocated
 - neo_framealloc.c, 652
- mark_range_usable_and_free
 - neo_framealloc.c, 653
- MAX
 - global_defs.h, 345
- mbr_disk_id
 - limine_file, 72
- media_type
 - limine_file, 72
- mem_readout_unit
 - kswitches, 58
- memcmp
 - memcmp.c, 586
 - string.h, 507
- memcmp.c
 - memcmp, 586
- memcpy
 - framebuffer.c, 832
 - memcpy.c, 587
 - string.h, 507
- memcpy.c
 - memcpy, 587
- memmap_ready
 - early_alloc.c, 213
- memmap_req
 - early_alloc.c, 215
 - kernel.c, 572
 - KernelUtils.c, 581
 - neo_framealloc.c, 659
 - paging_bootstrap.c, 701
- memmove
 - memmove.c, 589
 - string.h, 508
- memmove.c
 - memmove, 589
- Memory Management, 14
- memory_bringup
 - kernel.c, 566
- memory_model
 - limine_framebuffer, 76
- memset
 - framebuffer.c, 832
 - memset.c, 590
 - string.h, 508

- memset.c
 - memset, 590
- memset8
 - neo_framealloc.c, 654
- memtest.c
 - basic_heap_test, 619
 - clear_slots, 619
 - fill_slot, 620
 - fragmentation_test, 620
 - g_basic_sizes, 633
 - g_pmm_frames, 633
 - g_slots, 633
 - heap_growth_test, 621
 - hhdm_request, 634
 - memtest_fail, 622
 - MEMTEST_GROWTH_SIZE, 617
 - MEMTEST_GROWTH_SLOTS, 617
 - MEMTEST_PAGE_SIZE, 617
 - MEMTEST_PMM_MAX_FRAMES, 617
 - MEMTEST_RANDOM_MAX_ALLOC, 618
 - MEMTEST_RANDOM_OPS, 618
 - memtest_run, 623
 - MEMTEST_SLOT_COUNT, 618
 - memtest_slot_t, 618
 - memtest_stats_t, 618
 - page_pattern, 624
 - payload_pattern, 624
 - pmm_roundtrip_test, 625
 - random_cycle, 625
 - randomized_heap_test, 626
 - realloc_test, 627
 - release_slot, 628
 - set_slot, 628
 - stats_add_live, 629
 - verify_alignment, 629
 - verify_all, 630
 - verify_no_overlap, 631
 - verify_prefix, 631
 - verify_slot, 632
 - xorshift64, 633
- memtest.h
 - memtest_run, 420
 - VOYAGER_MEMTEST_H, 419
- memtest_fail
 - memtest.c, 622
- MEMTEST_GROWTH_SIZE
 - memtest.c, 617
- MEMTEST_GROWTH_SLOTS
 - memtest.c, 617
- MEMTEST_PAGE_SIZE
 - memtest.c, 617
- MEMTEST_PMM_MAX_FRAMES
 - memtest.c, 617
- MEMTEST_RANDOM_MAX_ALLOC
 - memtest.c, 618
- MEMTEST_RANDOM_OPS
 - memtest.c, 618
- memtest_run
 - memtest.c, 623
 - memtest.h, 420
- memtest_slot, 108
 - ptr, 108
 - size, 108
 - tag, 108
- MEMTEST_SLOT_COUNT
 - memtest.c, 618
- memtest_slot_t
 - memtest.c, 618
- memtest_stats, 109
 - allocations, 109
 - frees, 109
 - live_bytes, 109
 - operations, 110
 - peak_live_bytes, 110
 - reallocations, 110
- memtest_stats_t
 - memtest.c, 618
- memzero
 - early_alloc.c, 214
 - neo_framealloc.c, 654
 - paging_bootstrap.c, 689
- MIN
 - global_defs.h, 346
- MIN_SPLIT_SIZE
 - kmalloc.c, 595
- minute
 - time.c, 892
 - time.h, 539
- MODE_CMD
 - time.c, 880
- mode_t
 - global_defs.h, 347
- mode_toggle
 - term.c, 855
- module_count
 - limine_module_response, 90
- module_request
 - neo_framealloc.c, 659
- month
 - time.c, 892
 - time.h, 540
- month_str
 - time.c, 892
- move_character
 - term_context, 127
- n_frame_free
 - neo_framealloc.c, 655
 - neo_framealloc.h, 429
- neo_framealloc.c
 - __kernel_end, 657
 - __kernel_start, 657
 - ALIGN_DOWN, 643
 - ALIGN_UP, 643
 - bm_clear, 644
 - bm_set, 644
 - bm_test, 644

- compute_max_phys_and_count, 645
- early_alloc_page, 645
- early_init, 646
- early_reserved_region, 647
- frame_addr_of, 648
- frame_alloc, 648
- frame_free_count, 649
- frame_index_of, 650
- frame_init, 650
- frame_is_allocated, 651
- frame_total_count, 652
- g_alloc_bm, 657
- g_frame_count, 657
- g_free_count, 657
- g_hint, 658
- g_max_phys, 658
- g_memlock, 658
- g_usable_bm, 658
- hhdm_request, 658
- Kaddress_req, 658
- mark_range_allocated, 652
- mark_range_usable_and_free, 653
- memmap_req, 659
- memset8, 654
- memzero, 654
- module_request, 659
- n_frame_free, 655
- PAGE_SIZE, 643
- panic, 656
- neo_framealloc.h
 - frame_alloc, 426
 - frame_free_count, 427
 - frame_init, 427
 - frame_is_allocated, 428
 - frame_total_count, 429
 - n_frame_free, 429
- next
 - process, 113
- NEXT_BLOCK
 - kmalloc.c, 607
- next_free
 - block_header, 24
- next_pid
 - sched.c, 789
- no_avx
 - cpuUtils.c, 194
- no_oxsave
 - cpuUtils.c, 195
- no_sse
 - cpuUtils.c, 195
- no_xsave
 - cpuUtils.c, 196
- null
 - GDT, 44
- oem_id
 - rsdp, 115
 - sdt, 118
- oem_revision
 - sdt, 118
- oem_table_id
 - sdt, 118
- offset
 - GDT_Desc, 46
 - limine_hhdm_response, 81
- offset_x
 - fbterm_context, 39
- offset_y
 - fbterm_context, 39
- OffsetToVirtualAddress
 - paging.c, 666
- old_cursor_x
 - fbterm_context, 39
- old_cursor_y
 - fbterm_context, 39
- operations
 - memtest_stats, 110
- out_fct_type
 - printf.c, 724
- out_fct_wrap_type, 110
 - arg, 111
 - fct, 111
- outb
 - io.c, 558
 - io.h, 362
- OVERHEAD
 - kmalloc.c, 595
- p4Offset
 - PageTableOffset, 111
- PACKED
 - global_defs.h, 346
 - madt.h, 242
- paddr_t
 - early_alloc.h, 306
- PAGE_1G_SIZE
 - paging_bootstrap.c, 679
- PAGE_2M_SIZE
 - paging_bootstrap.c, 679
- PAGE_ADDRESS_MASK
 - paging.h, 434
- PAGE_FLAG_MASK
 - paging.h, 434
- page_pattern
 - memtest.c, 624
- PAGE_SIZE
 - early_alloc.c, 210
 - kmalloc.c, 595
 - neo_framealloc.c, 643
 - paging_bootstrap.c, 679
- PageTableOffset, 111
 - p4Offset, 111
 - pdOffset, 112
 - pdpOffset, 112
 - ptOffset, 112
- Paging and Address Translation, 16
- paging.c
 - DuplicateRecursive, 663

- GetOrAllocEntry, 664
- GetOrNullifyEntry, 665
- OffsetToVirtualAddress, 666
- PagingDuplicate, 667
- PagingIdentityMap, 667
- PagingMapMemory, 669
- PagingPhysicalMemory, 670
- PagingUnmapMemory, 671
- ReadCR3, 671
- VirtualAddressToOffsets, 672
- WriteCR3, 673
- paging.h
 - _PAGING_H_, 433
 - __attribute__, 435
 - HIGHER_HALF_KERNEL_MEMORY_OFFSET, 434
 - HIGHER_HALF_MEMORY_OFFSET, 434
 - IsHigherHalf, 435
 - IsKernelHigherHalf, 435
 - PAGE_ADDRESS_MASK, 434
 - PAGE_FLAG_MASK, 434
 - PAGING_EXPORT, 434
 - PAGING_FLAG_LARGER_PAGES, 435
 - PAGING_FLAG_NO_EXECUTE, 435
 - PAGING_FLAG_PAT0, 435
 - PAGING_FLAG_PAT1, 435
 - PAGING_FLAG_PAT2, 435
 - PAGING_FLAG_PRESENT, 435
 - PAGING_FLAG_USER_ACCESSIBLE, 435
 - PAGING_FLAG_WRITABLE, 435
 - PAGING_FLAG_WRITE_COMBINE, 435
 - PagingDuplicate, 436
 - PagingFlag, 434
 - PagingIdentityMap, 436
 - PagingMapMemory, 437
 - PagingPhysicalMemory, 438
 - PagingSetActivePageTable, 439
 - PagingUnmapMemory, 439
 - TranslateToHighHalfMemoryAddress, 440
 - TranslateToKernelMemoryAddress, 440
 - TranslateToKernelPhysicalMemoryAddress, 441
 - TranslateToPhysicalMemoryAddress, 441
- paging_bootstrap
 - paging_bootstrap.c, 689
 - paging_bootstrap.h, 445
- paging_bootstrap.c
 - __attribute__, 680
 - __bss_end, 698
 - __data_start, 698
 - __kernel_end, 698
 - __kernel_start, 698
 - __rodata_end, 698
 - __rodata_start, 698
 - __text_end, 698
 - __text_start, 699
 - ADDR_MASK, 678
 - ADDR_MASK_1G, 678
 - ADDR_MASK_2M, 678
 - ALIGN_DOWN, 678
 - ALIGN_UP, 679
 - alloc_table, 680
 - ensure_hhdm_cover_phys_page, 681
 - ensure_hhdm_cover_phys_range, 682
 - framebuffer_bytes, 699
 - framebuffer_phys, 699
 - g_alloc_bm_bytes, 699
 - g_alloc_bm_phys_base, 699
 - g_hhdm, 699
 - g_pml4_phys, 700
 - g_pml4_virt, 700
 - g_usable_bm_bytes, 700
 - g_usable_bm_phys_base, 700
 - get_max_phys, 682
 - get_or_make_next, 682
 - get_or_make_pd, 683
 - get_total_usable_mem_bytes, 684
 - hhdm_request, 700
 - install_new_cr3, 684
 - Kaddress_req, 700
 - map_1g, 684
 - map_2m, 685
 - map_4k, 685
 - map_hhdm, 686
 - map_identity_minimal, 687
 - map_kernel_higher_half, 687
 - map_range_4k, 688
 - map_range_huge_first, 688
 - memmap_req, 701
 - memzero, 689
 - PAGE_1G_SIZE, 679
 - PAGE_2M_SIZE, 679
 - PAGE_SIZE, 679
 - paging_bootstrap, 689
 - paging_pml4_phys, 690
 - paging_pml4_virt, 691
 - panic, 692
 - phys_to_virt, 692
 - pre_switch_audit_and_patch, 693
 - probe_hhdm_phys, 693
 - probe_va, 694
 - PTE_NX, 679
 - PTE_P, 679
 - PTE_PS, 680
 - PTE_U, 680
 - PTE_W, 680
 - readCR3, 694
 - readRSP, 695
 - scan_hhdm_range, 695
 - vaddr_indices, 696
 - virt_to_phys_kernel, 696
 - walk_va_to_pa, 696
 - writeCR3, 697
- paging_bootstrap.h
 - paging_bootstrap, 445
 - paging_pml4_phys, 445
 - paging_pml4_virt, 446

- PAGING_EXPORT
 - paging.h, 434
- PAGING_FLAG_LARGER_PAGES
 - paging.h, 435
- PAGING_FLAG_NO_EXECUTE
 - kmalloc.c, 595
 - paging.h, 435
- PAGING_FLAG_PAT0
 - paging.h, 435
- PAGING_FLAG_PAT1
 - paging.h, 435
- PAGING_FLAG_PAT2
 - paging.h, 435
- PAGING_FLAG_PRESENT
 - kmalloc.c, 595
 - paging.h, 435
- PAGING_FLAG_USER_ACCESSIBLE
 - paging.h, 435
- PAGING_FLAG_WRITABLE
 - kmalloc.c, 596
 - paging.h, 435
- PAGING_FLAG_WRITE_COMBINE
 - paging.h, 435
- paging_pml4_phys
 - paging_bootstrap.c, 690
 - paging_bootstrap.h, 445
- paging_pml4_virt
 - paging_bootstrap.c, 691
 - paging_bootstrap.h, 446
- PagingDuplicate
 - paging.c, 667
 - paging.h, 436
- PagingFlag
 - paging.h, 434
- PagingGetFreeFrame
 - frame_supplier.h, 423
- PagingIdentityMap
 - paging.c, 667
 - paging.h, 436
- PagingMapMemory
 - paging.c, 669
 - paging.h, 437
- PagingPhysicalMemory
 - paging.c, 670
 - paging.h, 438
- PagingSetActivePageTable
 - paging.h, 439
- PagingUnmapMemory
 - paging.c, 671
 - paging.h, 439
- panic
 - kmalloc.c, 607
 - neo_framealloc.c, 656
 - paging_bootstrap.c, 692
 - panic.c, 710
- panic.c
 - halt, 709
 - panic, 710
- panic.h
 - _PANIC_H, 448
 - __attribute__, 448
- part_uuid
 - limine_file, 73
- partition_index
 - limine_file, 73
- payload_pattern
 - memtest.c, 624
- pdOffset
 - PageTableOffset, 112
- pdpOffset
 - PageTableOffset, 112
- peak_live_bytes
 - memtest_stats, 110
- phys_page_supplier_t
 - frame_supplier.h, 423
- phys_to_virt
 - paging_bootstrap.c, 692
- Physical Memory Manager, 15
- physical_base
 - limine_kernel_address_response, 83
- pic.c
 - pic_enable, 713
 - pic_mask_irq, 714
 - pic_remap_offsets, 715
 - pic_send_eoi, 716
 - pic_unmask_irq, 717
- pic.h
 - pic_enable, 453
 - PIC_EOI, 450
 - PIC_ICW1_ICW4, 450
 - PIC_ICW1_INIT, 450
 - PIC_ICW1_INTERVAL4, 450
 - PIC_ICW1_LEVEL, 451
 - PIC_ICW1_SINGLE, 451
 - PIC_ICW4_8086, 451
 - PIC_ICW4_AUTO, 451
 - PIC_ICW4_BUF_MASTER, 451
 - PIC_ICW4_BUF_SLAVE, 451
 - pic_mask_irq, 453
 - PIC_MASTER, 452
 - PIC_MASTER_COMMAND, 452
 - PIC_MASTER_DATA, 452
 - pic_remap_offsets, 454
 - pic_send_eoi, 455
 - PIC_SLAVE, 452
 - PIC_SLAVE_COMMAND, 452
 - PIC_SLAVE_DATA, 452
 - pic_unmask_irq, 456
- pic_enable
 - pic.c, 713
 - pic.h, 453
- PIC_EOI
 - pic.h, 450
- PIC_ICW1_ICW4
 - pic.h, 450
- PIC_ICW1_INIT

- pic.h, 450
- PIC_ICW1_INTERVAL4
 - pic.h, 450
- PIC_ICW1_LEVEL
 - pic.h, 451
- PIC_ICW1_SINGLE
 - pic.h, 451
- PIC_ICW4_8086
 - pic.h, 451
- PIC_ICW4_AUTO
 - pic.h, 451
- PIC_ICW4_BUF_MASTER
 - pic.h, 451
- PIC_ICW4_BUF_SLAVE
 - pic.h, 451
- pic_mask_irq
 - pic.c, 714
 - pic.h, 453
- PIC_MASTER
 - pic.h, 452
- PIC_MASTER_COMMAND
 - pic.h, 452
- PIC_MASTER_DATA
 - pic.h, 452
- pic_remap_offsets
 - pic.c, 715
 - pic.h, 454
- pic_send_eoi
 - pic.c, 716
 - pic.h, 455
- PIC_SLAVE
 - pic.h, 452
- PIC_SLAVE_COMMAND
 - pic.h, 452
- PIC_SLAVE_DATA
 - pic.h, 452
- pic_unmask_irq
 - pic.c, 717
 - pic.h, 456
- pick_region_from_memmap
 - early_alloc.c, 214
- pid
 - process, 113
- PIT_DIVIDEND
 - time.h, 527
- PIT_FREQ
 - time.h, 527
- pit_get_current_count
 - time.c, 883
 - time.h, 529
- pit_set_frequency
 - time.c, 883
 - time.h, 530
- pit_set_reload_value
 - time.c, 884
 - time.h, 530
- pit_sleep
 - time.c, 885
- time.h, 531
- pit_ticks
 - time.c, 893
- pitch
 - fbterm_context, 40
 - limine_framebuffer, 76
- plot_char
 - framebuffer.c, 833
- plot_char_fast
 - framebuffer.c, 833
- pmm_roundtrip_test
 - memtest.c, 625
- pre_switch_audit_and_patch
 - paging_bootstrap.c, 693
- Present
 - gdt.h, 340
- PREV_BLOCK
 - kmalloc.c, 609
- prev_free
 - block_header, 24
- print_date
 - time.c, 885
 - time.h, 531
- print_load_time
 - time.c, 886
 - time.h, 532
- print_memmap
 - KernelUtils.c, 580
 - KernelUtils.h, 370
- print_stack_size
 - stack_trace.c, 803
 - stack_trace.h, 500
- print_sys_time
 - time.c, 886
 - time.h, 533
- printf
 - printf.h, 458
- printf.c
 - _atoi, 724
 - _etoa, 725
 - _ftoa, 727
 - _is_digit, 729
 - _ntoa_format, 730
 - _ntoa_long, 732
 - _ntoa_long_long, 734
 - _out_buffer, 736
 - _out_char, 737
 - _out_fct, 738
 - _out_null, 739
 - _out_rev, 740
 - _strlen_s, 741
 - _vsprintf, 742
 - fctprintf, 744
 - FLAGS_ADAPT_EXP, 720
 - FLAGS_CHAR, 721
 - FLAGS_HASH, 721
 - FLAGS_LEFT, 721
 - FLAGS_LONG, 721

- FLAGS_LONG_LONG, 721
- FLAGS_PLUS, 721
- FLAGS_PRECISION, 722
- FLAGS_SHORT, 722
- FLAGS_SPACE, 722
- FLAGS_UPPERCASE, 722
- FLAGS_ZEROPAD, 722
- out_fct_type, 724
- printf_, 745
- PRINTF_DEFAULT_FLOAT_PRECISION, 722
- PRINTF_FTOA_BUFFER_SIZE, 723
- PRINTF_MAX_FLOAT, 723
- PRINTF_NTOA_BUFFER_SIZE, 723
- PRINTF_SUPPORT_EXPONENTIAL, 723
- PRINTF_SUPPORT_FLOAT, 723
- PRINTF_SUPPORT_LONG_LONG, 723
- PRINTF_SUPPORT_PTRDIFF_T, 724
- snprintf_, 747
- sprintf_, 748
- vprintf_, 748
- vsnprintf_, 749
- printf.h
 - _PRINTF_H_, 458
 - _putchar, 460
 - fctprintf, 462
 - printf, 458
 - printf_, 463
 - snprintf, 459
 - snprintf_, 465
 - sprintf, 459
 - sprintf_, 466
 - vprintf, 460
 - vprintf_, 466
 - vsnprintf, 460
 - vsnprintf_, 467
- printf_
 - printf.c, 745
 - printf.h, 463
- PRINTF_DEFAULT_FLOAT_PRECISION
 - printf.c, 722
- PRINTF_FTOA_BUFFER_SIZE
 - printf.c, 723
- PRINTF_MAX_FLOAT
 - printf.c, 723
- PRINTF_NTOA_BUFFER_SIZE
 - printf.c, 723
- PRINTF_SUPPORT_EXPONENTIAL
 - printf.c, 723
- PRINTF_SUPPORT_FLOAT
 - printf.c, 723
- PRINTF_SUPPORT_LONG_LONG
 - printf.c, 723
- PRINTF_SUPPORT_PTRDIFF_T
 - printf.c, 724
- probe_hhdm_phys
 - paging_bootstrap.c, 693
- probe_va
 - paging_bootstrap.c, 694
- proc.h
 - proc_yield, 469
- proc_a
 - kernel.c, 566
- proc_b
 - kernel.c, 567
- PROC_BLOCKED
 - sched.h, 482
- PROC_READY
 - sched.h, 482
- PROC_RUNNING
 - sched.h, 482
- proc_state_t
 - sched.h, 481
- Proc_utils.c
 - soak, 765
 - soak_counter, 766
- proc_utils.h
 - soak, 471
- proc_yield
 - proc.h, 469
- process, 112
 - next, 113
 - pid, 113
 - rsp, 113
 - state, 114
- process_t
 - sched.h, 481
- processor
 - madt_nmi, 107
- processor_id
 - madt_lapic, 106
- PS/2 Keyboard, 18
- PTE_NX
 - paging_bootstrap.c, 679
- PTE_P
 - paging_bootstrap.c, 679
- PTE_PS
 - paging_bootstrap.c, 680
- PTE_U
 - paging_bootstrap.c, 680
- PTE_W
 - paging_bootstrap.c, 680
- ptOffset
 - PageTableOffset, 112
- ptr
 - memtest_slot, 108
- Purple
 - kernel.c, 562
- push_to_queue
 - framebuffer.c, 834
- putchar.c
 - _putchar, 768
 - buffer_or_flush, 770
 - flush_serial_line, 772
 - logbuf, 778
 - LOGBUF_SIZE, 768
 - loglen, 778

- ticks_to_ms, 774
- tsc_hz, 779
- u64_to_str, 775
- voyager_page_tables_active, 776
- queue
 - fbterm_context, 40
- queue_i
 - fbterm_context, 40
- queue_size
 - fbterm_context, 40
- r10
 - cpu_ctx, 26
- r11
 - cpu_ctx, 26
- r12
 - cpu_ctx, 27
- r13
 - cpu_ctx, 27
- r14
 - cpu_ctx, 27
- r15
 - cpu_ctx, 27
- r8
 - cpu_ctx, 27
- r9
 - cpu_ctx, 27
- rand_asm
 - cpuUtils.c, 196
 - cpuUtils.h, 293
- random_cycle
 - memtest.c, 625
- randomized_heap_test
 - memtest.c, 626
- raw_putchar
 - term_context, 127
- rax
 - cpu_ctx, 28
 - isr_xframe_t, 54
- rbp
 - cpu_ctx, 28
 - isr_xframe_t, 55
 - stack_frame, 120
- rbx
 - cpu_ctx, 28
 - isr_xframe_t, 55
- rcx
 - cpu_ctx, 28
 - isr_xframe_t, 55
- rdi
 - cpu_ctx, 28
 - isr_xframe_t, 55
- rdmsr
 - registers.c, 782
 - registers.h, 473
- rdrand
 - cpu.h, 261
- rdrand_asm
 - cpuUtils.c, 197
- rdseed
 - cpu.h, 261
- rdseed_asm
 - cpuUtils.c, 197
- rdtsc
 - cpu.h, 262
- rdx
 - cpu_ctx, 28
 - isr_xframe_t, 55
- read_rtc
 - time.c, 887
 - time.h, 534
- read_XCR0
 - cpuUtils.c, 198
- readCR2
 - registers.h, 473
- ReadCR3
 - kmalloc.c, 609
 - paging.c, 671
- readCR3
 - paging_bootstrap.c, 694
 - registers.h, 474
- readCR4
 - registers.h, 475
- readCRO
 - registers.h, 476
- readRSP
 - paging_bootstrap.c, 695
 - registers.h, 476
- ReadWrite
 - gdt.h, 340
- realloc_test
 - memtest.c, 627
- reallocations
 - memtest_stats, 110
- Red
 - kernel.c, 562
- red_mask_shift
 - limine_framebuffer, 76
- red_mask_size
 - limine_framebuffer, 77
- refcount
 - smartlock, 119
- registers.c
 - rdmsr, 782
 - writeMSR, 782
- registers.h
 - rdmsr, 473
 - readCR2, 473
 - readCR3, 474
 - readCR4, 475
 - readCRO, 476
 - readRSP, 476
 - writeCR0, 476
 - writeCR3, 477
 - writeCR4, 477
 - writeMSR, 477

- release_slot
 - memtest.c, 628
- reserved
 - madt_io_apic, 104
 - rsdp, 115
- reserved0
 - TSS, 136
 - TSS_Entry, 139
- reserved1
 - TSS, 137
- reserved2
 - TSS, 137
- reserved3
 - TSS, 137
- restore_floats
 - KernelUtils.h, 371
- restore_state
 - term.c, 856
 - term_context, 127
- Retired / Historical Code, 20
- reverse_video
 - term_context, 128
- revision
 - limine_5_level_paging_request, 60
 - limine_5_level_paging_response, 60
 - limine_boot_time_request, 61
 - limine_boot_time_response, 62
 - limine_bootloader_info_request, 63
 - limine_bootloader_info_response, 64
 - limine_dtb_request, 65
 - limine_dtb_response, 66
 - limine_efi_system_table_request, 67
 - limine_efi_system_table_response, 68
 - limine_entry_point_request, 69
 - limine_entry_point_response, 70
 - limine_file, 73
 - limine_framebuffer_request, 78
 - limine_framebuffer_response, 79
 - limine_hhdm_request, 80
 - limine_hhdm_response, 81
 - limine_kernel_address_request, 82
 - limine_kernel_address_response, 83
 - limine_kernel_file_request, 84
 - limine_kernel_file_response, 85
 - limine_memmap_request, 87
 - limine_memmap_response, 88
 - limine_module_request, 89
 - limine_module_response, 90
 - limine_rsdp_request, 91
 - limine_rsdp_response, 92
 - limine_smbios_request, 93
 - limine_smbios_response, 94
 - limine_smp_request, 95
 - limine_stack_size_request, 97
 - limine_stack_size_response, 97
 - limine_terminal_request, 100
 - limine_terminal_response, 101
 - rsdp, 115
 - sdt, 118
- revscroll
 - term_context, 128
- rflags
 - cpu_ctx, 29
 - isr_xframe_t, 55
- rip
 - cpu_ctx, 29
 - isr_xframe_t, 56
 - stack_frame, 120
- rows
 - limine_terminal, 99
 - term_context, 128
- rrr
 - term_context, 128
- rsdp, 114
 - acpi.c, 144
 - checksum, 114
 - ext_checksum, 115
 - length, 115
 - oem_id, 115
 - reserved, 115
 - revision, 115
 - rsdt_addr, 115
 - signature, 116
 - xsdt_addr, 116
- rsdp_req
 - acpi.c, 144
- rsdt, 116
 - acpi.c, 144
 - data, 116
- rsdt_addr
 - rsdp, 115
- rsi
 - cpu_ctx, 29
 - isr_xframe_t, 56
- rsp
 - cpu_ctx, 29
 - isr_xframe_t, 56
 - process, 113
- rsp0
 - gdt.c, 223
 - gdt.h, 342
 - TSS, 137
- rsp1
 - TSS, 137
- rsp2
 - TSS, 137
- rsv0
 - __attribute__, 23
- run_queue_head
 - sched.c, 789
- run_queue_tail
 - sched.c, 789
- save_floats
 - KernelUtils.h, 371
- save_state
 - term.c, 857

- term_context, 128
- saved_cursor_x
 - term_context, 128
- saved_cursor_y
 - term_context, 129
- saved_state_bold
 - term_context, 129
- saved_state_current_charset
 - term_context, 129
- saved_state_current_primary
 - term_context, 129
- saved_state_cursor_x
 - fbterm_context, 40
- saved_state_cursor_y
 - fbterm_context, 40
- saved_state_reverse_video
 - term_context, 129
- saved_state_text_bg
 - fbterm_context, 41
- saved_state_text_fg
 - fbterm_context, 41
- scan_hhdm_range
 - paging_bootstrap.c, 695
- sched.c
 - allow_sched, 788
 - create_process, 784
 - current, 789
 - init_scheduler, 785
 - next_pid, 789
 - run_queue_head, 789
 - run_queue_tail, 789
 - schedule, 786
 - scheduler_start, 787
 - STACK_ALIGN, 784
 - STACK_SIZE, 784
 - task_returned, 788
- sched.h
 - _SCHED_H, 480
 - allow_sched, 486
 - create_process, 482
 - current, 486
 - init_scheduler, 483
 - PROC_BLOCKED, 482
 - PROC_READY, 482
 - PROC_RUNNING, 482
 - proc_state_t, 481
 - process_t, 481
 - schedule, 483
 - scheduler_start, 484
 - switch_to, 485
 - SYSCALL_EXIT, 480
 - SYSCALL_FORK, 481
 - SYSCALL_YIELD, 481
- sched_debug
 - kswitches, 58
- schedule
 - sched.c, 786
 - sched.h, 483
- Scheduler and Processes, 17
- scheduler_heartbeat
 - kernel.c, 567
- scheduler_start
 - sched.c, 787
 - sched.h, 484
- scroll
 - term_context, 129
- scroll_bottom_margin
 - term_context, 130
- scroll_enabled
 - term_context, 130
- scroll_top_margin
 - term_context, 130
- sdt, 117
 - acpi.c, 145
 - checksum, 117
 - creator_id, 117
 - creator_revision, 118
 - length, 118
 - oem_id, 118
 - oem_revision, 118
 - oem_table_id, 118
 - revision, 118
 - signature, 119
- second
 - time.c, 893
 - time.h, 540
- seg_16_code
 - GDT, 44
- seg_16_data
 - GDT, 45
- seg_32_code
 - GDT, 45
- seg_32_data
 - GDT, 45
- Segments
 - gdt.h, 340
- serial.c
 - serial_print, 792
 - serial_print_line, 793
 - serial_print_with_ts, 794
- serial.h
 - get_ts, 489
 - serial_debug, 489
 - SERIAL_H, 488
 - serial_print, 490
 - serial_print_line, 491
 - serial_print_with_ts, 492
- serial_debug
 - serial.h, 489
- SERIAL_H
 - serial.h, 488
- serial_print
 - serial.c, 792
 - serial.h, 490
- serial_print_line
 - serial.c, 793

- serial.h, 491
- serial_print_with_ts
 - serial.c, 794
 - serial.h, 492
- SET_ALLOC
 - kmalloc.c, 596
- set_cursor_pos
 - term_context, 130
- set_fs_base
 - cpu.h, 262
- set_gs_base
 - cpu.h, 263
- set_kernel_gs_base
 - cpu.h, 263
- SET_SIZE
 - kmalloc.c, 596
- set_slot
 - memtest.c, 628
- set_text_bg
 - term_context, 130
- set_text_bg_bright
 - term_context, 130
- set_text_bg_default
 - term_context, 131
- set_text_bg_rgb
 - term_context, 131
- set_text_fg
 - term_context, 131
- set_text_fg_bright
 - term_context, 131
- set_text_fg_default
 - term_context, 131
- set_text_fg_rgb
 - term_context, 131
- sgr
 - term.c, 858
- shell.c
 - cmd_parser, 795
 - halt, 796
 - shell_streq, 797
 - vsh_loop, 797
 - vsh_readline, 798
- shell.h
 - _SHELL_H, 494
 - cmd_parser, 494
 - VSH_CMD_BUFFER_SIZE, 494
 - vsh_loop, 495
 - vsh_readline, 496
 - VSH_VERSION, 494
- shell_streq
 - shell.c, 797
- signature
 - rsdp, 116
 - sdt, 119
- size
 - GDT_Desc, 46
 - limine_file, 73
 - memtest_slot, 108
- size_and_flags
 - block_footer, 23
 - block_header, 25
- smartlock, 119
 - refcount, 119
 - thread, 119
- smartlock_acquire
 - lock.h, 407
- SMARTLOCK_INIT
 - lock.h, 406
- smartlock_release
 - lock.h, 407
- snprintf
 - printf.h, 459
- snprintf_
 - printf.c, 747
 - printf.h, 465
- soak
 - Proc_utils.c, 765
 - proc_utils.h, 471
- soak_counter
 - Proc_utils.c, 766
- spinlock_acquire
 - lock.h, 407
- SPINLOCK_INIT
 - lock.h, 406
- spinlock_release
 - lock.h, 408
- spinlock_t
 - lock.h, 407
- spinlock_test_and_acq
 - lock.h, 408
- sprintf
 - printf.h, 459
- sprintf_
 - printf.c, 748
 - printf.h, 466
- src/acpi/acpi.c, 141, 145
- src/acpi/madt.c, 147, 148
- src/apic/ioapic.c, 149, 154
- src/apic/lapic.c, 155, 164
- src/cpu.c, 165, 168
- src/cpuUtils.c, 169, 199
- src/drivers/keyboard/keyboard.c, 202, 208
- src/early_alloc.c, 209, 216
- src/frame_supplier.c, 218
- src/gdt.c, 219, 224
- src/idt.c, 226, 235
- src/include/acpi/acpi.h, 236, 239
- src/include/acpi/madt.h, 240, 243
- src/include/apic/ioapic.h, 243, 246
- src/include/apic/lapic.h, 247, 251
- src/include/consts.h, 251, 252
- src/include/cpu.h, 252, 267
- src/include/cpuUtils.h, 270, 295
- src/include/drivers/keyboard/keyboard.h, 296, 300
- src/include/drivers/keyboard/keyboard_map.h, 300, 302
- src/include/early_alloc.h, 305, 309

- src/include/errno.h, 309, 334
- src/include/gdt.h, 336, 343
- src/include/global_defs.h, 344, 347
- src/include/idt.h, 348, 358
- src/include/interrupts.h, 358, 360
- src/include/io.h, 360, 363
- src/include/kernel.h, 364, 366
- src/include/KernelUtils.h, 366, 372
- src/include/lib/hashmap.h, 372, 377
- src/include/lib/vector.h, 380, 386
- src/include/limine.h, 388, 399
- src/include/lock.h, 405, 409
- src/include/mm/kheap.h, 410, 411
- src/include/mm/kmalloc.h, 411, 418
- src/include/mm/memtest.h, 418, 421
- src/include/paging/frame_supplier.h, 421, 424
- src/include/paging/neo_framealloc.h, 424, 431
- src/include/paging/paging.h, 432, 442
- src/include/paging/paging_bootstrap.h, 443, 447
- src/include/panic.h, 447, 448
- src/include/pic.h, 449, 457
- src/include/printf.h, 457, 468
- src/include/proc.h, 468, 470
- src/include/proc_utils.h, 470, 471
- src/include/registers.h, 472, 478
- src/include/sched.h, 478, 487
- src/include/serial.h, 487, 493
- src/include/shell.h, 493, 497
- src/include/stack_trace.h, 498, 505
- src/include/string.h, 506, 511
- src/include/syscall.h, 511
- src/include/terminal/framebuffer.h, 512, 516
- src/include/terminal/term.h, 517, 523
- src/include/time.h, 525, 541
- src/include/tss.h, 542, 544
- src/include/vgafont.h, 544, 545
- src/include/voyager_docs.h, 549
- src/int2srt.c, 550, 551
- src/interrupts.c, 552, 554
- src/io.c, 556, 559
- src/kernel.c, 560, 573
- src/KernelUtils.c, 576, 581
- src/limine_requests.c, 583, 584
- src/lock_c.c, 585
- src/memcmp.c, 585, 586
- src/memcpy.c, 587, 588
- src/memmove.c, 588, 589
- src/memset.c, 590, 591
- src/mm/kmalloc.c, 591, 611
- src/mm/memtest.c, 615, 634
- src/neo_framealloc.c, 642, 659
- src/paging.c, 662, 673
- src/paging/paging_bootstrap.c, 676, 701
- src/panic.c, 709, 712
- src/pic.c, 712, 718
- src/printf.c, 719, 750
- src/proc.c, 764
- src/Proc_utils.c, 765, 767
- src/putchar.c, 767, 779
- src/registers.c, 781, 783
- src/sched.c, 783, 790
- src/serial.c, 791, 794
- src/shell.c, 795, 799
- src/stack_trace.c, 801, 808
- src/str2int.c, 809, 811
- src/strlen.c, 811, 812
- src/strncpy.c, 813
- src/strtok.c, 814, 815
- src/terminal/framebuffer.c, 815, 835
- src/terminal/term.c, 848, 865
- src/time.c, 877, 895
- ss
 - cpu_ctx, 29
- ssize_t
 - global_defs.h, 347
- STACK_ALIGN
 - sched.c, 784
- stack_dump
 - stack_trace.c, 804
 - stack_trace.h, 501
- stack_dump_asm
 - stack_trace.h, 501
- stack_dump_recursive
 - stack_trace.c, 805
 - stack_trace.h, 502
- stack_frame, 120
 - rbp, 120
 - rip, 120
- stack_req
 - stack_trace.c, 808
 - stack_trace.h, 505
- stack_return, 121
 - addresses, 121
 - array_size, 121
- STACK_SIZE
 - sched.c, 784
- stack_size
 - limine_stack_size_request, 97
- stack_trace
 - stack_trace.c, 805
 - stack_trace.h, 503
- stack_trace.c
 - dump_hex, 802
 - halt, 803
 - print_stack_size, 803
 - stack_dump, 804
 - stack_dump_recursive, 805
 - stack_req, 808
 - stack_trace, 805
 - stack_trace_asm, 806
 - walk_stack, 807
- stack_trace.h
 - _STACK_TRACE_H, 499
 - dump_hex, 499
 - print_stack_size, 500
 - stack_dump, 501

- stack_dump_asm, 501
 - stack_dump_recursive, 502
 - stack_req, 505
 - stack_trace, 503
 - stack_trace_asm, 504
- stack_trace_asm
 - stack_trace.c, 806
 - stack_trace.h, 504
- stack_trace_on_fault
 - kswitches, 58
- stack_trace_size
 - kswitches, 58
- start_interrupts
 - kernel.c, 568
- state
 - process, 114
- stats_add_live
 - memtest.c, 629
- stop_interrupts
 - kernel.c, 568
- str2int
 - str2int.c, 810
 - string.h, 509
- str2int.c
 - str2int, 810
 - str2int2, 810
- str2int2
 - str2int.c, 810
 - string.h, 509
- string.h
 - int_to_str, 506
 - memcmp, 507
 - memcpy, 507
 - memmove, 508
 - memset, 508
 - str2int, 509
 - str2int2, 509
 - strlen, 509
 - strncpy, 510
 - strtok, 510
 - u64_to_str, 510
- strlen
 - string.h, 509
 - strlen.c, 812
- strlen.c
 - strlen, 812
- strncpy
 - string.h, 510
 - strncpy.c, 813
- strncpy.c
 - strncpy, 813
- strtok
 - string.h, 510
 - strtok.c, 814
- strtok.c
 - strtok, 814
- supply_from_early
 - kernel.c, 569
- supply_from_frame
 - kernel.c, 569
- swap_palette
 - term_context, 132
- switch_to
 - sched.h, 485
- sys_clock_handler
 - time.c, 888
 - time.h, 534
- sys_clock_handler_alt
 - time.c, 889
 - time.h, 535
- SYSCALL_EXIT
 - sched.h, 480
- SYSCALL_FORK
 - sched.h, 481
- SYSCALL_YIELD
 - sched.h, 481
- sysenter
 - cpu.c, 168
 - cpu.h, 267
- system_timer_fractions
 - time.c, 893
 - time.h, 540
- system_timer_ms
 - time.c, 893
 - time.h, 540
- tab_size
 - term_context, 132
- tag
 - memtest_slot, 108
- task_returned
 - sched.c, 788
- task_switch_handler
 - time.c, 889
- task_switch_int
 - kernel.c, 570
- task_timer_count
 - time.c, 894
- term
 - fbterm_context, 41
- term.c
 - CHARSET_DEC_SPECIAL, 849
 - CHARSET_DEFAULT, 849
 - col256, 864
 - control_sequence_parse, 849
 - dec_private_parse, 850
 - dec_special_to_cp437, 851
 - escape_parse, 852
 - linux_private_parse, 854
 - mode_toggle, 855
 - restore_state, 856
 - save_state, 857
 - sgr, 858
 - term_context_reinit, 859
 - term_putchar, 860
 - term_write, 862
- term.h

- TERM_CB_BELL, 518
- TERM_CB_DEC, 519
- TERM_CB_KBD_LEDS, 519
- TERM_CB_LINUX, 519
- TERM_CB_MODE, 519
- TERM_CB_POS_REPORT, 519
- TERM_CB_PRIVATE_ID, 519
- TERM_CB_STATUS_REPORT, 520
- term_context_reinit, 520
- TERM_MAX_ESC_VALUES, 520
- term_write, 521
- term_bg
 - kernel.c, 572
- TERM_CB_BELL
 - term.h, 518
- TERM_CB_DEC
 - term.h, 519
- TERM_CB_KBD_LEDS
 - term.h, 519
- TERM_CB_LINUX
 - term.h, 519
- TERM_CB_MODE
 - term.h, 519
- TERM_CB_POS_REPORT
 - term.h, 519
- TERM_CB_PRIVATE_ID
 - term.h, 519
- TERM_CB_STATUS_REPORT
 - term.h, 520
- term_context, 122
 - autoflush, 123
 - bold, 123
 - callback, 123
 - charsets, 123
 - clear, 124
 - cols, 124
 - control_sequence, 124
 - csi, 124
 - current_charset, 124
 - current_primary, 124
 - dec_private, 125
 - deinit, 125
 - disable_cursor, 125
 - discard_next, 125
 - double_buffer_flush, 125
 - enable_cursor, 125
 - esc_values, 126
 - esc_values_i, 126
 - escape, 126
 - escape_offset, 126
 - full_refresh, 126
 - g_select, 126
 - get_cursor_pos, 127
 - in_bootloader, 127
 - insert_mode, 127
 - kernel.c, 572
 - KernelUtils.h, 371
 - move_character, 127
 - raw_putchar, 127
 - restore_state, 127
 - reverse_video, 128
 - revscroll, 128
 - rows, 128
 - rrr, 128
 - save_state, 128
 - saved_cursor_x, 128
 - saved_cursor_y, 129
 - saved_state_bold, 129
 - saved_state_current_charset, 129
 - saved_state_current_primary, 129
 - saved_state_reverse_video, 129
 - scroll, 129
 - scroll_bottom_margin, 130
 - scroll_enabled, 130
 - scroll_top_margin, 130
 - set_cursor_pos, 130
 - set_text_bg, 130
 - set_text_bg_bright, 130
 - set_text_bg_default, 131
 - set_text_bg_rgb, 131
 - set_text_fg, 131
 - set_text_fg_bright, 131
 - set_text_fg_default, 131
 - set_text_fg_rgb, 131
 - swap_palette, 132
 - tab_size, 132
- term_context_reinit
 - term.c, 859
 - term.h, 520
- term_fg
 - kernel.c, 572
- TERM_MAX_ESC_VALUES
 - term.h, 520
- term_putchar
 - term.c, 860
- term_write
 - term.c, 862
 - term.h, 521
- Terminal and Console, 18
- terminal_count
 - limine_terminal_response, 101
- test_em
 - cpuUtils.c, 198
- test_handler
 - idt.c, 231
- text_bg
 - fbterm_context, 41
- text_fg
 - fbterm_context, 41
- tftp_ip
 - limine_file, 73
- tftp_port
 - limine_file, 73
- this_cpu
 - cpu.c, 166
 - cpu.h, 264

- thread
 - smartlock, 119
- ticks_to_ms
 - putchar.c, 774
- time.c
 - century_register, 892
 - CHANNEL_ZERO, 879
 - CMOS_ADDR, 879
 - CMOS_DATA, 879
 - config_PIT, 880
 - CURRENT_YEAR, 879
 - day, 892
 - dayofweek, 880
 - delta_int, 881
 - get_RTC_register, 881
 - get_update_flag, 881
 - hour, 892
 - init_PIT, 882
 - minute, 892
 - MODE_CMD, 880
 - month, 892
 - month_str, 892
 - pit_get_current_count, 883
 - pit_set_frequency, 883
 - pit_set_reload_value, 884
 - pit_sleep, 885
 - pit_ticks, 893
 - print_date, 885
 - print_load_time, 886
 - print_sys_time, 886
 - read_rtc, 887
 - second, 893
 - sys_clock_handler, 888
 - sys_clock_handler_alt, 889
 - system_timer_fractions, 893
 - system_timer_ms, 893
 - task_switch_handler, 889
 - task_timer_count, 894
 - time_init, 890
 - time_mono, 894
 - time_real, 894
 - timer_arm, 890
 - timer_disarm, 891
 - timer_fired, 894
 - timers_lock, 894
 - VECTOR_TYPE, 891
 - weekday, 895
 - year, 895
- time.h
 - _TIME_H, 527
 - day, 539
 - dayofweek, 528
 - hour, 539
 - init_PIT, 528
 - minute, 539
 - month, 540
 - PIT_DIVIDEND, 527
 - PIT_FREQ, 527
 - pit_get_current_count, 529
 - pit_set_frequency, 530
 - pit_set_reload_value, 530
 - pit_sleep, 531
 - print_date, 531
 - print_load_time, 532
 - print_sys_time, 533
 - read_rtc, 534
 - second, 540
 - sys_clock_handler, 534
 - sys_clock_handler_alt, 535
 - system_timer_fractions, 540
 - system_timer_ms, 540
 - time_init, 536
 - time_mono, 540
 - time_real, 541
 - time_t, 528
 - timer_arm, 536
 - timer_disarm, 537
 - timer_new, 537
 - timespec_add, 538
 - timespec_sub, 538
 - year, 541
- time_init
 - time.c, 890
 - time.h, 536
- time_mono
 - time.c, 894
 - time.h, 540
- time_real
 - time.c, 894
 - time.h, 541
- time_t
 - time.h, 528
- Timekeeping, 19
- timer, 132
 - fired, 133
 - index, 133
 - when, 133
- timer_arm
 - time.c, 890
 - time.h, 536
- timer_disarm
 - time.c, 891
 - time.h, 537
- timer_fired
 - time.c, 894
- timer_function
 - cpu_local, 31
- timer_new
 - time.h, 537
- timer_vec
 - lapic.c, 163
- timers_lock
 - time.c, 894
- timespec, 133
 - tv_nsec, 134
 - tv_sec, 134

- timespec_add
 - time.h, 538
- timespec_sub
 - time.h, 538
- timestamp
 - kswitches, 58
- total_size_for_payload
 - kmalloc.c, 610
- TranslateToHighHalfMemoryAddress
 - paging.h, 440
- TranslateToKernelMemoryAddress
 - paging.h, 440
- TranslateToKernelPhysicalMemoryAddress
 - paging.h, 441
- TranslateToPhysicalMemoryAddress
 - paging.h, 441
- tsc_hz
 - putchar.c, 779
- TSS, 134
 - iopbOffset, 135
 - ist1, 135
 - ist2, 135
 - ist3, 136
 - ist4, 136
 - ist5, 136
 - ist6, 136
 - ist7, 136
 - reserved0, 136
 - reserved1, 137
 - reserved2, 137
 - reserved3, 137
 - rsp0, 137
 - rsp1, 137
 - rsp2, 137
- tss
 - cpu_local, 32
 - GDT, 45
 - gdt.c, 223
- tss.h
 - _TSS_H, 544
- TSS_Entry, 138
 - base_high, 138
 - base_low, 138
 - base_mid, 139
 - base_up, 139
 - flags, 139
 - flags2, 139
 - length, 139
 - reserved0, 139
- TssStack
 - gdt.c, 224
- tv_nsec
 - timespec, 134
- tv_sec
 - timespec, 134
- type
 - limine_memmap_entry, 86
- typeof
- madt.c, 148
- u64_to_str
 - int2srt.c, 550
 - putchar.c, 775
 - string.h, 510
- unused
 - limine_file, 74
 - limine_framebuffer, 77
- USABLE
 - kmalloc.c, 596
- use_xsdt
 - acpi.c, 144
- userCode
 - GDT, 45
- userData
 - GDT, 45
- userNull
 - GDT, 46
- vaddr_indices
 - paging_bootstrap.c, 696
- vector
 - isr_xframe_t, 56
- vector.h
 - VECTOR_ENSURE_LENGTH, 382
 - VECTOR_FIND, 382
 - VECTOR_FOR_EACH, 382
 - VECTOR_INIT, 383
 - VECTOR_INSERT, 383
 - VECTOR_INVALID_INDEX, 383
 - VECTOR_ITEM, 384
 - VECTOR_PUSH_BACK, 384
 - VECTOR_PUSH_FRONT, 384
 - VECTOR_REMOVE, 384
 - VECTOR_REMOVE_BY_VALUE, 385
 - VECTOR_TYPE, 385
- VECTOR_ENSURE_LENGTH
 - vector.h, 382
- VECTOR_FIND
 - vector.h, 382
- VECTOR_FOR_EACH
 - vector.h, 382
- VECTOR_INIT
 - vector.h, 383
- VECTOR_INSERT
 - vector.h, 383
- VECTOR_INVALID_INDEX
 - vector.h, 383
- VECTOR_ITEM
 - vector.h, 384
- VECTOR_PUSH_BACK
 - vector.h, 384
- VECTOR_PUSH_FRONT
 - vector.h, 384
- VECTOR_REMOVE
 - vector.h, 384
- VECTOR_REMOVE_BY_VALUE
 - vector.h, 385

VECTOR_TYPE
 madt.h, 242
 time.c, 891
 vector.h, 385
vectors
 idt.c, 234
verify_alignment
 memtest.c, 629
verify_all
 memtest.c, 630
verify_no_overlap
 memtest.c, 631
verify_prefix
 memtest.c, 631
verify_slot
 memtest.c, 632
vgafont
 vgafont.h, 545
vgafont.h
 vgafont, 545
virt_to_phys_kernel
 paging_bootstrap.c, 696
virtual_base
 limine_kernel_address_response, 83
VirtualAddressToOffsets
 paging.c, 672
Voyager Shell (VSH), 19
VOYAGER_MEMTEST_H
 memtest.h, 419
voyager_page_tables_active
 putchar.c, 776
vprintf
 printf.h, 460
vprintf_
 printf.c, 748
 printf.h, 466
VSH_CMD_BUFFER_SIZE
 shell.h, 494
vsh_loop
 shell.c, 797
 shell.h, 495
vsh_readline
 shell.c, 798
 shell.h, 496
VSH_VERSION
 shell.h, 494
vsprintf
 printf.h, 460
vsprintf_
 printf.c, 749
 printf.h, 467

walk_stack
 kernel.c, 570
 stack_trace.c, 807
walk_va_to_pa
 paging_bootstrap.c, 696
weekday
 time.c, 895

when
 timer, 133
White
 kernel.c, 562
width
 fbterm_context, 41
 limine_framebuffer, 77
writeCR0
 registers.h, 476
WriteCR3
 paging.c, 673
writeCR3
 paging_bootstrap.c, 697
 registers.h, 477
writeCR4
 registers.h, 477
writeMSR
 registers.c, 782
 registers.h, 477
wrmsr
 cpu.h, 265

x
 fbterm_queue_item, 43
xorshift64
 memtest.c, 633
xrstor
 cpu.h, 266
xsave
 cpu.h, 266
xsave_bank
 cpuUtils.h, 294
xsdt_addr
 rsdp, 116

y
 fbterm_queue_item, 43
year
 time.c, 895
 time.h, 541
Yellow
 kernel.c, 563
yield_isr_test
 idt.c, 232
yield_register
 idt.c, 233
 idt.h, 356
YIELD_VECTOR
 idt.c, 227